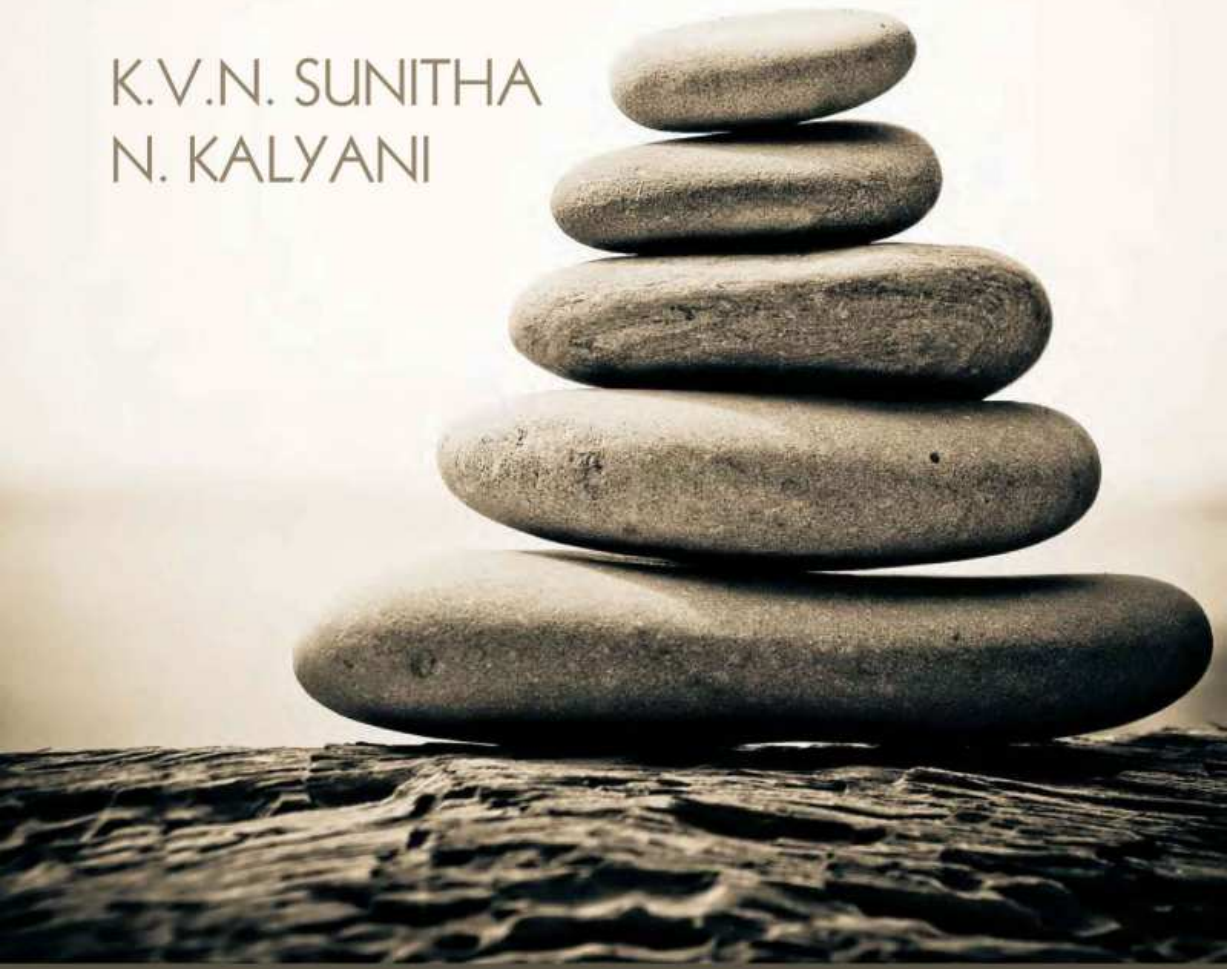
FORMAL LANGUAGES AND AUTOMATA THEORY

K.V.N. SUNITHA
N. KALYANI



ALWAYS LEARNING PEARSON

Formal Languages and Automata Theory

K. V. N Sunitha
Principal
BVRIT Hyderabad College of Engineering for Women
Nizampet, Hyderabad

N. Kalyani
Professor
Department of Computer Science Engineering
G. Narayanamma Institute of Technology and science
Hyderabad

PEARSON
Chennai • Delhi

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Copyright © 2016 Pearson India Education Services Pvt. Ltd

Published by Pearson India Education Services Pvt. Ltd, CIN: U72200TN2005PTC057128, formerly known as TutorVista Global Pvt. Ltd, licensee of Pearson Education in South Asia.

No part of this eBook may be used or reproduced in any manner whatsoever without the publisher's prior written consent.

This eBook may or may not include all assets that were part of the print version. The publisher reserves the right to remove any material in this eBook at any time.

ISBN 978-93-325-3728-6
eISBN 978-93-325-4164-1

Head Office: A-8 (A), 7th Floor, Knowledge Boulevard, Sector 62, Noida 201 309, Uttar Pradesh, India.

Registered Office: Module G4, Ground Floor, Elnet Software City, TS-140, Block 2 & 9, Rajiv Gandhi Salai, Taramani, Chennai 600 113, Tamil Nadu, India.

Fax: 080-30461003, Phone: 080-30461060

www.pearson.co.in, Email: companysecretary.india@pearson.com

Dedicated

To our parents

Late Sri K. Subbaiah and Smt K. Subba Lakshamma

and

Late Sri Nara China Rangaiah and Mrs N. Vengamma

To our in-laws

Late Mr M. Chidambaram and Mrs Rama Subbamma

and

Mr and Mrs J. Musaliah, Mr and Mrs T. Venkataiah

for instilling in us the thirst for knowledge and the quest for excellence.

To our families

Mr M. Chidambhara Murthy, Charan Teja and Uday Teja

and

Mr J. Ch. Subbarao, Pranavi and Akhilesh

for their love, encouragement and support.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

This page is intentionally left blank

Brief Contents

Preface	xv
Acknowledgements	xviii
List of Important Symbols	xix
List of Important Abbreviations	xxiii
About the Authors	xxv
1. Mathematical Preliminaries and Formal Languages	1
2. Finite Automata	31
3. Regular Languages and Regular Grammars	101
4. Context Free Grammars and Context Free Languages	149
5. Push Down Automata	201
6. Turing Machines	237
7. Undecidability and Computability	281
8. Non-deterministic Polynomial Completeness	309
9. LR(k) and LL(1) Grammars	327
Appendix A: Proposition and Predicate Logic	353
Appendix B: Frequently Asked University Questions with Solutions	365
References	449
Index	451

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

This page is intentionally left blank

Contents

Preface *xv*
Acknowledgements *xviii*
List of Important Symbols *xix*
List of Important Abbreviations *xxiii*
About the Authors *xxv*

1. Mathematical Preliminaries and Formal Languages 1

1.1 Set Theory—1
1.1.1 Describing a Set 2
1.1.2 Empty Set 2
1.1.3 Identity and Cardinality 3
1.1.4 Subset 3
1.1.5 Power Sets 3
1.1.6 Operations on Sets: Union, Intersection 3
1.1.7 Set Theoretic Equalities 5
1.1.8 Sequence versus Set 6
1.1.9 Ordered Pairs 6
1.1.10 Cartesian Product 6
1.2 Relations—6
1.2.1 Binary Relation 7
1.2.2 Domain and Range of Relation 7
1.2.3 Operations on Relations 7
1.2.4 Properties of Relations 8
1.3 Functions—9
1.3.1 Definitions 9
1.3.2 Types of Functions 10
1.4 Alphabet, String and Language—11
1.4.1 Operations on Language 12
1.4.2 Grammars 13
1.4.3 Types of Grammars–Chomsky Hierarchy 13

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

1.5 Graphs and Trees—16
1.5.1 Directed Graph 16
1.5.2 Undirected Graph 17
1.5.3 Trees 17
1.6 Theorem Proving—18
1.6.1 Proof by Induction 19
1.6.2 Proof by Contradiction 20
1.6.3 Proof by Example 21
Summary 21
Short Answers 22
Fill in the Blanks 23
Objective Question Bank 24
Exercises 27

2. Finite Automata 31

2.1 Finite-state Machine—31
2.1.1 Finite-Automaton Model 32
2.1.2 Properties of Transition Function 'χ' 33
2.1.3 Transition Diagram 34
2.1.4 Transition Table 34
2.2 Language Acceptance—35
2.3 Two Types of Finite Automata—37
2.3.1 Deterministic Finite Automata (DFA) 37
2.3.2 Non-deterministic Finite Automaton (NFA) 40
2.3.3 Acceptance of NFA 40
2.4 Equivalence of DFAs and NFAs—42
2.5 Converting NFA (M_N) to DFA (M_D)—Subset Construction—43
2.6 NFA with Epsilon-(ε) Transitions—46
2.6.1 Epsilon Closure (ε-closure) 47
2.6.2 Eliminating ε-Transitions 48
2.6.3 Converting NFA with ε-Transition to NFA without ε-Transition 48
2.6.4 Converting NFA with ε-Transition to DFA 50
2.7 Comparison Method for Testing Equivalence of Two FAs—52
2.8 Reduction of Number of States in FA—53
2.8.1 Indistinguishable States 53
2.8.2 Equivalent Classes 53
2.8.3 Minimization of DFA 54
2.8.4 Minimization of DFA Using Myhill Nerode Theorem 58

2.9 Finite Automata with Output—60
2.9.1 Moore Machine 61
2.9.2 Mealy Machine 61
2.9.3 Equivalence Between Moore and Mealy Machines 63
2.9.4 Interconversions Between Machines 63
2.10 Applications of Finite Automata with Output—65
2.10.1 The Full-adder 65
2.10.2 The String Sequence Detector 67
Solved Problems 68
Summary 91
Short Answers 91
Fill in the Blanks 92
Objective Question Bank 93
Exercises 98

3. Regular Languages and Regular Grammars 101

3.1 Regular Expressions—101
3.2 Regular Sets—102
3.3 Identity Rules for Regular Expressions—104
3.4 Algebraic Laws for Regular Expressions—104
3.5 Equivalence of Finite Automata with Regular Expressions—105
3.6 Constructing Regular Expression for Given DFA—109
3.6.1 Arden's Theorem 111
3.6.2 Arden's Theorem in Construction of RE 112
3.6.3 Construction of RE Using Generalized NFA 113
3.7 Pumping Lemma of Regular Expressions—114
3.7.1 Formal Definition of the Pumping Lemma 114
3.8 Regular Grammar—115
3.8.1 Equivalence of Regular Grammar and Finite Automata 115
3.8.2 Converting Finite Automaton to Regular Grammar 118
3.9 Closure Properties of Regular Sets—120
3.10 Applications of Regular Expressions—121
3.10.1 Lexical Analysis 121
3.10.2 Finding Patterns 122
3.11 Decision Properties of Regular Languages—123
3.11.1 Conversion from NFA to DFA 123

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

3.11.2 Emptiness Membership and Equivalence 124
Solved Problems 124
Summary 136
Short Answers 136
Fill in the Blanks 140
Objective Question Bank 140
Exercises 145

4. Context Free Grammars and Context Free Languages 149

4.1 Context Free Grammars—149
4.2 Derivation of CFGs—151
4.3 Understanding the Language Defined by Grammars—153
4.3.1 Leftmost and Rightmost Derivations 154
4.3.2 Derivation Tree 155
4.3.3 Equivalence of Parse Trees and Derivations 156
4.4 Ambiguous Grammar—156
4.4.1 Removing Ambiguity 159
4.4.2 Inherent Ambiguity 160
4.5 Simplification of Grammars—160
4.5.1 Elimination of Useless Symbols 161
4.5.2 Elimination of ε-Productions 163
4.5.3 Eliminating Unit Productions 164
4.6 Normal Forms—166
4.6.1 The Chomsky Normal Form 166
4.6.2 The Greibach Normal Form 170
4.7 Pumping Lemma for CFL—173
4.7.1 Lemma 173
4.8 Decision Algorithms for CFLs—174
4.8.1 Finiteness and Infiniteness 175
4.9 Membership—176
4.10 Closure Properties of CFLs—179
4.11 Applications of CFG—181
Solved Problems 182
Summary 189
Short Answers 189
Fill in the Blanks 193
Objective Question Bank 194
Exercises 197

5. Push Down Automata 201

5.1 Pushdown Automata—201
- 5.1.1 Graphical Representation of PDA 202
- 5.1.2 Instantaneous Description of PDA 203
- 5.1.3 Language Acceptance by PDA 204

5.2 Equivalence of Acceptance of Final State and Empty Stack—210

5.3 Types of PDAs—211
- 5.3.1 Deterministic PDA 211
- 5.3.2 Closure Properties of DCFL 212
- 5.3.3 Decision Properties of DCFLs 214
- 5.3.4 DPDA and Regular Languages 214
- 5.3.5 DPDA and Ambiguous Grammar 214

5.4 Equivalence of PDA's and CFG's—216
- 5.4.1 Constructing PDA for Given CFG 216
- 5.4.2 Constructing CFG for the Given PDA 217

5.5 Two-stack PDA—222

5.6 Applications of PDA—223
- 5.6.1 PDA as a Parser 223
- 5.6.2 Top-down Parser Using the PDA 224

Solved Problems 226

Summary 229

Short Answers 229

Fill in the Blanks 231

Objective Question Bank 232

Exercises 235

6. Turing Machines 237

6.1 Turing Assumptions—237
- 6.1.1 Instantaneous Description 238
- 6.1.2 Turing Machine as Language Accepter 239

6.2 Turing Machine as a Computational Machine—239

6.3 Techniques for Turing Machine Construction—241
- 6.3.1 Storage in Finite Control 241
- 6.3.2 Multi-track Tape 243
- 6.3.3 Checking off Symbols 244
- 6.3.4 Subroutines 246
- 6.3.5 Shifting Over 248

6.4 Types of Turing Machines—249
- 6.4.1 Non-deterministic Turing Machines 249

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

6.4.2 Turing Machines with Two-dimensional Tapes 250
6.4.3 Turing Machines with Multiple Tapes 251
6.4.4 Turing Machines with Multiple Heads 252
6.4.5 Turing Machines with Infinite Tape 252
6.5 Church's Thesis—253
6.6 Turing Machines as Enumerators—254
6.7 Universal Turing Machine—254
6.8 Counter Machine—256
6.9 Recursive and Recursively Enumerable Languages—257
6.10 Linear Bound Automata and Context Sensitive Language—258
6.10.1 Equivalence of LBA's and CSG's 258
Solved Problems 259
Summary 274
Short Answers 274
Fill in the Blanks 276
Objective Question Bank 276
Exercises 280

7. Undecidability and Computability 281

7.1 Decision Problems—281
7.2 Decidability and Decidable Languages—282
7.2.1 Decidable Problems Concerning Regular Languages 282
7.2.2 Decidable Problems Concerning Context Free Languages 284
7.3 Halting Problem—285
7.3.1 The Halting Problem for Turing Machines 286
7.4 Diagonalization Method—286
7.4.1 Undecidable Problems 288
7.5 Post's Correspondence Problem—288
7.5.1 The Undecidability of Post's Correspondence Problem 290
7.5.2 Modified Version of PCP 290
7.6 Reducibility—294
7.6.1 Properties 295
7.6.2 Mapping Reducibility 296
7.6.3 Formal Definition of Mapping Reducibility 296
7.7 Recursion Theorem—296
7.7.1 Applications and Uses of Recursion 297

7.8 Rice's Theorem—298
7.9 Ackermann's Function—300
Solved Problems 301
Summary 302
Short Answers 302
Fill in the Blanks 305
Objective Question Bank 305
Exercises 307

8. Non-deterministic Polynomial Completeness 309

8.1 NP-hard and NP-complete—309
8.1.1 Classification of Problems 309
8.2 P Problems—309
8.3 NP Problems—310
8.4 Tractable Problems—310
8.5 NP-complete—310
8.6 NP-hard—312
8.7 Examples of Problems in Different Classes—313
8.8 NP-completeness—315
8.9 Reduction—316
8.9.1 Computational Complexity 319
8.9.2 0–1 Knapsack Problem 319
8.9.3 Computational Complexity 321
Solved Problems 322
Summary 323
Short Answers 324
Fill in the Blanks 324
Objective Question Bank 325
Exercises 326

9. LR(k) and LL(1) Grammars 327

9.1 LL(1) Grammar—327
9.2 Rules for Verifying Whether the Given Grammar Is LL(1) or Not—329
9.3 LR(K) Grammars—330
9.4 Properties of LR(k) Grammars—331
9.5 Construction of LR(0) Items for Context Free Grammars—332

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

9.6 Definition of LR(0) Grammar—332
9.7 LR(1) Grammar—336
Solved Problems 337
Summary 348
Short Answers 349
Fill in the Blanks 349
Objective Question Bank 350
Exercises 351

Appendix A: Proposition and Predicate Logic 353

A.1 Propositions—353
A.2 Connectives—353
A.3 Well-Formed Formula—355
A.3.1 Truth Table for a Well-formed Formula 356
A.4 Logical Identities—357
A.5 Normal Forms of Well-formed Formals—358
A.5.1 Construction to Obtain a Disjunctive Normal Form of a Given Formula 358
A.6 Principal Disjunctive Normal Form—359
A.6.1 Construction to Obtain the Principal Disjunctive Normal Form of a Given Formula 359
A.7 Predicate Calculus—361
A.8 Universal and Existential Quantifier—362
A.9 Well-formed Formulas of Predicate Calculus—363
A.10 Rules of Inference for Predicate Calculus—363
Summary 364

Appendix B: Frequently Asked University Questions with Solutions 365

Part A - Brief Questions —365
Part B - Detailed Questions —379

References 449
Index 451

Preface

Introduction

Formal Languages and Automata Theory is a branch of computer science and mathematics that deals with the mathematical abstraction model of computation and its relation to Formal Languages. It becomes increasingly important to revisit theoretical foundations, learn the appropriate techniques for answering theory-based questions, and build one's confidence in implementing this knowledge when building computer applications. Students study this subject because it helps any real-time problem to be explained with abstract models, formulated in a simple way that can be analyzed and used to prove results. Knowledge of set theory and proof by induction are helpful prerequisites of the subject.

Purpose

There are a number of excellent books on the sub-fields of computational theory, which include logic, automata theory and computational complexity. However, there is no single book on the subject that covers theory and solved problems, especially in automata theory, with equal emphasis on both these vital elements. Our primary motivation to write this book has been to share our professional experience with students to bridge the gap between their theoretical knowledge and ability to find solutions to a variety of problems using theoretical concepts. In our treatment of the theory and its practical application, we further strive to increase their confidence levels in applying the ideas and proof techniques in diverse environments.

Target Audience

Undergraduate and postgraduate students of computer science, engineering, and mathematics pursuing courses such as B.Tech., M.Sc., M.Tech. and MCA will find this textbook handy in their study of the subjects on theory of computation, automata theory, formal languages, and computational models.

This book covers the topics prescribed in the syllabi of most Indian universities.

About the Book

This book is intended to expose students to the theoretical development of computer science. It also provides conceptual tools that practitioners use in computer engineering. It also equips the student to design new programming languages for specialized applications through its thorough treatment of grammars and different automatas.

Problem-solving skills are essential for learning any mathematical subject. This book provides an assortment of problems illustrative of each method. Each of these problems

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

is solved in all possible ways for the student's benefit. The book also presents exercises that are as much thought-provoking as they are challenging and are designed to hone the student's analytical skills.

Road Map for Various Target Courses

All the units are mandatory to be covered in the subject.

Salient Features

We cover the concepts systematically, introducing the fundamental concepts, models, techniques and results that form the basic paradigms of computing. The highlight of the book is the variety of approaches it advances to solve a problem. The book also takes a close look at the application of theoretical concepts to solve real-world problems.

Formal languages and automata theory is a core topic in the field of computer science and any computer science and engineering student must study this subject. With an aim to make the subject coherent, this book.

- Builds on theory from the elementary ideas and introduces all necessary mathematical models gradually, in the context in which they are used, providing pointed examples that make the language intelligible.
- Probes the concepts methodically with an extensive use of definitions, proofs, solved examples, exercises and applications of the models.
- Supplies a summary, additional (progressively challenging) problems, objective question bank and fill in the blanks for each chapter.
- Reviews mathematical preliminaries such as set theory, relations, graphs, trees, functions, and mathematical induction.
- Explores regular languages, covering the mechanisms for representing languages, the closure properties of such languages, the existence of other languages, and other structural properties
- Investigates the class of context-free languages, including context-free grammars, pushdown automata, their equivalence, closure properties, and existence of non-context-free languages
- Examines the importance of Turing machines as language recognizer, language generator and as computing model.
- Explains the complexity theory, LL(1), LR(k) grammars, propositional and predicate logic with simple examples.

Organization

Chapter 1 is about the basic concepts required to understand the rest of the chapters. Chapter 2 describes the formal languages, their representation using grammars and classification of languages in Chomsky hierarchy. Chapter 3 details the finite state machine, construction of deterministic and non-deterministic automata, minimization of automata, automata with output and equivalence of different machines. Chapter 4 focuses on the regular expressions and construction of NFA, the procedure for minimization, the closure properties of regular languages and pumping lemma for regular languages.

Importance of CFG and CFL is discussed in Chapter 5. This chapter also includes the procedure of constructing parse trees and representing the grammar in normal forms

besides expounding on the closure properties of CFL's and pumping lemma for CFL's. Chapter 6 spells out the details of Push down automata and the inter conversions from PDA and CFGs. Chapter 7 explains Turing machine (TM) representation and the design of TM for language recognition and language generation. The chapter also traces the evolution of TM as a computational model. Chapters 8, 9 and 10 address the undecidability and computability issues, while elucidating the nature of NP-hard and NP-complete problems, LL(1) and LR(k) grammars and the propositional and predicate logics.

Feedback

Any suggestions for improving the contents of the text are always welcome. Please send your suggestions and feedback to any of the following mails:

k.v.n.sunitha@gmail.com

or

kalyani_nara@rediffmail.com

K. V. N. Sunitha

N. Kalyani

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] konkurcomputer.ir

Acknowledgements

Many people have contributed either directly or indirectly to this book. Our special thanks to Sojan Jose, S. Shankari and C. Purushothaman of Pearson India Education Services Pvt. Ltd, Chennai for their constructive criticism and suggestions for improvement in the book and for bringing out this book in time.

All Authors

I sincerely thank my Chairman Sri K. V. Vishnu Raju, Vice Chairman Sri Ravi Chandran Raja Gopal, Director Sri Ram Kumar of Sri Vishnu Educational Society, family and friends for their encouragement and support in completing this book.

K. V. N. Sunitha

I sincerely thank my Chairman Sri P. Subba Reddy, Director Sri I. Rama Chandra Reddy and my colleagues of G. Narayanamma Institute of Technology & Science for their constant support. My special thanks to my mentor Dr K. V. N. Sunitha, my family and friends for their motivation and support in completing this book.

N. Kalyani

List of Important Symbols

Symbol	Meaning	Section in which the symbol and its explanation appear first
$a \in A$	a belongs to the set A	1.1
$a \notin A$	a does not belong to the set A	1.1
$\varnothing$	Empty set (Phi)	1.1.2
$\Leftrightarrow$	If and only if	1.1.3
$\|A\|$	Cardinality of A	1.1.3
$A \subseteq B$	A is a subset of the set B	1.1.4
$A \subset B$	A is proper subset of the set B	1.1.4
$\wp(A)$	Power set of A	1.1.5
$A \cup B$	Union of sets A and B	1.1.6
$A \cap B$	Intersection of sets A and B	1.1.6
$A - B$	Complement of B in A	1.1.6
A'	Complement of A	1.1.6
<a, b>	Ordered pair	1.1.10
$A \times B$	Cartesian product	1.1.11
aRb or Rab	a is related to b under the relation R	1.2
R'	Complement of relation	1.2.3
R^{-1}	Inverse of relation R	1.2.3
F(x)	Image of x under f	1.3
$F: A \rightarrow B$	Function from A to B	1.3.2
G O F	Composition of F and G	1.3.2
idA or IA	identity function on A	1.3.2
Σ	Alphabet set	1.4
Σ^*	Set of all strings over alphabet Σ	1.4
Σ^+	Set of all strings over non-empty alphabet Σ	1.4
ε	Empty string or epsilon	1.4

(continued)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

(Continued)

Symbol	Meaning	Section in which the symbol and its explanation appear first
(V, T, P, S)	A grammar	1.4.2
α	String of terminals and non-terminals	1.4.3
B	String of terminals and non-terminals	1.4.3
Δ	Transition function	2.1
Q	Set of states	2.1
F	Set of final states	2.1
$\in$	Belongs to	2.4
$\hat{\delta}$	Extended transition function	2.6.1
$\cup\varepsilon$-closure	Union of null closure of	2.6.1
Π_k	k-Equivalence class	2.8.3
$\Rightarrow$	Implies	2.8.2
Δ	Set of output symbols	2.9.1
Λ	Out function	2.9.1
R	Regular expression	3.3
h	Homomorphism function	3.9
$A \overset{*}{\Rightarrow} \alpha$	A derives α on one or more substitutions	4.10
$A \Rightarrow \alpha$	A derives α on single substitutions	4.10
Γ	Stack symbols	5.1.1
Z_0	Initial stack symbol	5.1.1
γ	The content of the stack read from top to bottom	5.1.3
$(q, x, S) \Big\vert\overline{\overset{*}{M}} (q, \varepsilon, \alpha)$	Left configuration derives the right configuration on machine M on one or more derivations	5.4.1
B	Blank symbol on tape	6.1.1
<l, q, r>	Instantaneous description of TM	6.1.2
$\surd$	Check of symbol	6.3.3
#	Special tape symbol	6.5
$\varnothing$, $	Special tape-end markers in LBA	6.10
$\forall$	For all values of	

(continued)

(Continued)

Symbol	Meaning	Section in which the symbol and its explanation appear first
$\sum_{j=1}^{n}$	Summation series for values of j varying from 1 to n	
$\neg$	Negation or NOT	11.2
$\wedge$	Conjunction or AND	11.2
$\vee$	Disjunction or OR	11.2
$\rightarrow$	Implication or IF..THEN..	11.2
$\leftrightarrow$	Biconditional	11.2
$\exists$	There exists	11.8

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

This page is intentionally left blank

List of Important Abbreviations

URG	Unrestricted Grammar
CSG	Context Sensitive Grammar
CFG	Context Free Grammar
RG	Regular Grammar
RL	Regular Language
CFL	Context Free Language
CSL	Context Sensitive Language
REL	Recursively Enumerable Language
CPU	Central Processing Unit
FA	Finite Automata
NFA	Non-deterministic Finite Automata
DFA	Deterministic Finite Automata
FSM	Finite State Machine
LMD	Left-most Derivation
RMD	Right-most Derivation
ID	Instantaneous Description
CNF	Chomsky Normal Form
GNF	Greibach Normal Form
RHS	Right-hand Side
LHS	Left-hand Side
CYK	Cocke–Younger–Kasami
PDA	Pushdown Automata
LBA	Linear-bound Automata
TM	Turing Machine
LIFO	Last-in-first-out
NPDA	Non-deterministic Pushdown Automata
DPDA	Deterministic Pushdown Automata
YACC	Yet another Compiler Compiler

(continued)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

(Continued)

DCFL	Deterministic Context-Free Language
DCFG	Deterministic Context-Free Grammar
NTM	Non-deterministic Turing Machine
PCP	Post-Correspondence Problem
MPCP	Modified Post-Correspondence Problem
NP	Non-deterministic Polynomial time
NPC	NP-Complete
TSP	Travelling Salesman Problem
LR(K)	Left-to-right scan of input, producing a right-most derivation using K symbols look ahead
LL(1)	Left-to-right scan of input, using Left-most derivation with 1 symbol look ahead
w.f.f.	well formed formulae
RE	Regular Expression
GNFA	Generalized NFA

About the Authors



Dr K. V. N. Sunitha, Principal, BVRIT Hyderabad College of Engineering for Women, Nizampet, Hyderabad, obtained her B.Tech in ECE from Nagarjuna University and M.Tech in Computer Science from REC Warangal. She received her Ph.D. from JNTU, Hyderabad in 2006. She is a JNTUH ratified professor with 23 years of teaching experience. She received 'Academic Excellence Award' from the management of G. Narayanamma Institute of Technology & Science in 2005. She also received 'Best Computer Science Engineering Teacher Award for the Year 2007' from the Indian Society for Technical Education ISTE. She has been recognized and invited by AICTE as NBA expert evaluator. Her autobiography is included in 'Marquis Who is Who in the World', 28th edition, 2011, since August 2012. She is the author of two books, 'Programming in UNIX and Compiler Design' by BS Publications and 'Compiler Construction', by Pearson India Ltd. She has published more than 75 papers in national and international journals and conferences. She is a reviewer for many national and international journals. She is fellow of Institute of Engineers, Sr member for IEEE & International Association CSIT and life member of many technical associations such as CSI and ACM.



Dr N. Kalyani obtained B.Tech in Civil from Osmania University in 1994, M.Tech in Computer Science from JNTUH in 2001 and Ph.D. from JNTUH in 2012. She has working experience of 5 years as Design Engineer in R. K. Engineers, Hyderabad and 14 years of teaching for both UG and PG students. She is an expert in automata theory, compiler design, advanced UNIX programming, design and analysis of algorithms, advanced data structures and human computer interaction. She worked as an assistant professor for 3 years in Gokaraju Rangaraju Institute of Engineering and Technology. She is currently working as a Professor in the Department of Computer Science Engineering, G. Narayanamma Institute of Technology and Science, Hyderabad. She is recipient of 'Academic Excellence Award' by GNITS in 2008. She is the author of 'Advanced Unix Programming and Compiler Design Lab Manual', a book by BSP publications. She has published 15 papers in national and international journals and conferences. She is a life member of CSI & ISTE technical associations. She is a BOS member, MCA Department, Gokaraju Rangaraju Institute of Engineering and Technology, Bachupally. She was a resource person for various workshops and a guest lecturer at various engineering colleges. Her areas of interest include speech and natural language processing.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

This page is intentionally left blank

Mathematical Preliminaries and Formal Languages

1

Formal languages and automata theory are based on mathematical computations. These computations are used to represent various mathematical models.

In this chapter, we discuss the mathematical preliminaries that form the foundation of computation. We discuss mathematical basics such as set theory, relations and functions. The basics of graph theory, language and grammar are discussed. A description of different techniques to prove theorems are also explained here. We study many interesting models such as finite automata, pushdown automata, linear bound automata and Turing machine. The theory about these models is automata theory. We will also discuss formal languages. A formal language is a set of words, that is, finite strings of letters or symbols. Formal languages are studied in the fields of logic, computer science and linguistics. Their most important practical application is to define the syntax of programming language constructs. The branch of mathematics and computer science that is concerned only with the purely syntactical aspects of such languages, that is, their internal structural patterns, is known as formal language theory.

This subject is a precursor to the work carried out in the field of computer science. The topics that we will be studying here were developed before the actual development of the field of computer science with significant contributions from scientists and mathematicians who worked in the fields of logic and linguistics.

The development of the field has assumed great importance in computer science, since a thorough knowledge of the basics is a must for budding computer engineers. Formal language is an interesting subject that goes hand-in-hand with designing and analysing in real sense. It is not just a theoretical subject but one that tells us about practical model design.

1.1 Set Theory

The concept of sets is extremely basic and finds application in all fields of mathematics. Sets are used to describe a group of similar objects.

Definition 1: Set is defined as a collection of objects.

These objects are called elements of the set. All the elements are enclosed within curly brackets '{}', and every element is separated by commas. If a is an element of set A, then we say that $a \in A$ (read as a 'belongs to' A) and if a is not an element of A, then we say that $a \notin A$ (read as a 'does not belong to' A). Sets can be *finite* or *infinite*.

There is exactly one set, the *empty set*, or the *null set,* which has no members at all.

A set with only one member is called a *singleton* or a *singleton set* (*singleton of* a).

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

1.1.1 Describing a Set

Typically, a set is denoted by a capital letter. There are three main ways to specify a set:

1. by listing all its members (*list notation*);
2. by stating a property of its elements (*predicate notation*);
3. by defining a set of rules that generates (defines) its members (*recursive rules*)

List notation: The first method is suitable only for finite sets. In this case, we list the elements of a set separated by commas and enclosed in braces.

Example 1.1 {1, 12, 45}, {George Washington, Bill Clinton}, {a, b, d, m}.

{1, 2, 3, 4,...} – this is not a real list notation, it is not a finite list, but it is common practice as long as the continuation is clear.

Note that we do not consider the order of elements in the list, and elements can be listed several times. {1, 12, 45}, {12, 1, 45, 1} and {45, 12, 45, 1} are different representations of the same set.

Predicate notation

Example 1.1 {x | x is a natural number and x < 8}.

Reading: 'the set of all x such that x is a natural number and is less than 8'

So, the second part of this notation is a property the members of the set share (a condition or a predicate that holds for members of this set).

Other examples:

{x | x is a letter of the English alphabet}

{y | y is a student of CSE and y is older than 25}

General form:

{x | P(x)}, where P is some predicate (condition, property).

The language to describe these predicates is not usually fixed in a strict way.

Recursive rules (always safe)

Example – the set E of even numbers greater than 3:

a) $4 \in E$

b) if $x \in E$, then $x + 2 \in E$

c) Nothing else belongs to E.

The first rule is the basis of recursion, the second one generates new elements from the elements defined before and the third rule restricts the defined set to the elements generated by rules (a) and (b).

1.1.2 Empty Set

Definition: A set having no element in it is called empty set.

It is denoted by A = { } and can be written as ∅ (Phi)

$|\varepsilon| = 0$ $|\varnothing| = 1$ as empty string is present

1.1.3 Identity and Cardinality

Two sets are identical *if and only if* they have exactly the same members, that is, A = B iff for every x, $x \in A \Leftrightarrow x \in B$.

For example, A = {0, 2, 4}
B = {x | x is an even number less than 5}
then A = B.

From the definition of identity, it follows that there exists only one empty set; its identity is fully determined by the absence of members. Note that empty list notation {} is not usually used for the empty set, we have a special symbol ϕ for it.

The number of elements in a set A is called the *cardinality* of A, written |A|. The cardinality of a finite set is a natural number. Infinite sets also have cardinalities, but they are not natural numbers.

1.1.4 Subset

A set A is a *subset of a set* B iff every element of A is also an element of B. Such a relation between sets is denoted by $A \subseteq B$. If $A \subseteq B$ and $A \neq B$, we call A a proper subset of B and write $A \subset B$. (Caution: sometimes $\subset$ is used the way we are using $\subseteq$.)

Both signs can be negated using a slash (/) through the sign.

Example 1.1 $\{a, b\} \subseteq \{d, a, b, e\}$ and $\{a, b\} \subset \{d, a, b, e\}$, $\{a, b\} \subseteq \{a, b\}$, note that $\{a, b\} \not\subset \{a, b\}$.

Note that the empty set is a subset of every set. $\phi \subseteq A$ for every set A.

Be careful about the difference between 'member of' and 'subset of'!

1.1.5 Power Sets

The set of all subsets of a set A is called the *power set* of A and is denoted as $\wp(A)$ or sometimes as 2^A.

For example, if A = {a,b}, $\wp(A) = \{\{\phi\}, \{a\}, \{b\}, \{a, b\}\}$.

From the example above: $a \in A$; $\{a\} \subseteq A$; $\{a\} \in \wp(A)$

$$\phi \in A;\ \phi \subseteq A;\ \phi \notin \wp(A);\ \phi \subseteq \wp(A)$$

1.1.6 Operations on Sets: Union, Intersection

We define several operations on sets. Let A and B be arbitrary sets.

The **union** of A and B, written $A \cup B$, is the set whose elements are just the elements of A or B or of both. In the predicate notation, the definition is

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

Example 1.1 Let K = {a, b}, L = {c, d} and M = {b, d}, then

$K \cup L = \{a, b, c, d\}$
$K \cup M = \{a, b, d\}$
$L \cup M = \{b, c, d\}$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$$(K \cup L) \cup M = K \cup (L \cup M) = \{a, b, c, d\}$$
$$K \cup K = K$$
$$K \cup \phi = \phi \cup K = K = \{a, b\}$$

The **intersection** of A and B, written $A \cap B$, is the set whose elements are just the elements of both A and B. In the predicate notation, the definition is

$$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$$

Example 1.5 (using the previous K, L, M)

$$K \cap L = \phi$$
$$K \cap M = \{b\}$$
$$L \cap M = \{d\}$$
$$(K \cap L) \cap M = K \cap (L \cap M) = \phi$$
$$K \cap K = K$$
$$K \cap \phi = \phi \cap K = \phi.$$

Example 1.5

i) Find $A \cup B$ (union operation) if $A = \{1, 2, 3\}$ $B = \{1, 2, 4\}$.

$A \cup B = \{1, 2, 3, 4\}$, i.e. combination of both the sets.

ii) Find $A \cap B$ (intersection operation) if $A = \{1, 2, 3\}$ $B = (2, 3, 4\}$.

$A \cap B = \{2, 3\}$, i.e. collection of common elements from both the sets.

More operations on sets: *difference, complement*

Another binary operation on arbitrary sets is the **difference** A minus B, written $A - B$, that 'subtracts' from A all elements that are in B (also called *relative complement*, the *complement of* B *relative to* A). The predicate notation defines this operation as follows:

$$A - B = \{x \mid x \in A \text{ and } x \notin B\}$$

Example 1.5 (using the previous K, L, M)

$$K - L = \{a, b\}$$
$$K - M = \{a\}$$
$$L - M = \{c\}$$
$$K - K = \phi$$
$$K - \phi = K$$
$$\phi - K = \phi.$$

$A - B$ is the *relative complement* of B relative to A. This operation is to be distinguished from the **complement** of a set A, written A', which is the set consisting of everything not in A. In predicate notation,

$$A' = \{x \mid x \notin A\}$$

It is natural to ask where do these objects that do not belong to A come from. In this case, it is presupposed that there exists a *universe of discourse*, and all other sets are subsets of this set. The universe of discourse is conventionally denoted by the symbol U.

Then, we have

$$A' = U - A$$

Example 1.8

i) A – B is the difference operation
Example: A = {1, 2, 3} and B = {1, 2, 4}

A – B = {3}, i.e. elements present in A but not in B
B – A = {4}, i.e. elements present in B but not in A

ii) Complement of A = U – A where U is universal set.
Example: U = {10, 20, 30, 40, 50} A = {10, 20}

A′ = {30, 40, 50}, i.e. elements in universal set not in A.

1.1.7 Set Theoretic Equalities

There are a number of general laws about sets that follow from the definitions of set theoretic operations, subsets and so on. Useful operations are shown below. They are grouped under their traditional names. The equations below hold for any sets X, Y, Z and universal set U.

1. **Idempotent Laws**
   a) $X \cup X = X$
   b) $X \cap X = X$
2. **Commutative Laws**
   a) $X \cup Y = Y \cup X$
   b) $X \cap Y = Y \cap X$
3. **Associative Laws**
   a) $(X \cup Y) \cup Z = X \cup (Y \cup Z)$
   b) $(X \cap Y) \cap Z = X \cap (Y \cap Z)$
4. **Distributive Laws**
   a) $X \cup (Y \cap Z) = (X \cup Y) \cap (X \cup Z)$
   b) $X \cap (Y \cup Z) = (X \cap Y) \cup (X \cap Z)$
5. **Identity Laws**
   a) $X \cup \phi = X$
   b) $X \cup U = U$
   c) $X \cap \phi = \phi$
   d) $X \cap U = X$
6. **Complement Laws**
   a) $X \cup X' = U$
   b) $(X')' = X$
   c) $X \cap X' = \phi$
   d) $X - Y = X \cap Y'$
7. **De Morgan's Laws**
   a) $(X \cup Y)' = X' \cap Y'$
   b) $(X \cap Y)' = X' \cup Y'$
8. **Consistency Principle**
   a) $X \subseteq Y$ iff $X \cup Y = Y$
   b) $X \subseteq Y$ iff $X \cap Y = X$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

1.1.8 Sequence versus Set

In sequence, order of elements is important and repetition is not allowed.

Example 1.5 Seq (1, 2, 3) ≠ Seq (2, 3, 1)

Seq (1, 2, 2) ≠ Seq (1, 2)

K elements in sequence is called K-tuple.

1.1.9 Ordered Pairs

As we see, there is no order imposed on the elements of a set. To describe functions and relations, we will need the notion of an *ordered pair*, written <a, b>, for example, in which a is considered the *first member (element)* and b is the *second member (element)* of the pair. So, in general, <a, b> ≠ <b, a> (whereas for a set, {a, b} = {b, a})

The ordered pair can be defined as a set as follows:

Definition: <a, b> =def {{a}, {a, b}}

1.1.10 Cartesian Product

The Cartesian product of two sets A and B is a set of all possible ordered pairs whose first component is a member of A and whose second component is a member of B. It is denoted by 'X'.

$$A \times B = \{(a, b) \mid a \in A \text{ and } b \in B\}$$

Example 1.10 A = {a, b} B = {0, 1, 2}

$$A \times B = \{(a, 0), (a, 1), (a, 2), (b, 0), (b, 1), (b, 2)\}$$

1.2 Relations

In natural language, *relations* are a kind of links existing between objects.

Examples:
'Mother of', 'neighbour of', 'part of', 'is older than', 'is an ancestor of', 'is a subset of' and so on. These are binary relations. Formally, we will define relations between elements of sets.

We may write Rab or aRb for a bears a relation R to b. And when we formalize relations as sets of ordered pairs of elements, we will officially write <a, b> ∈ R.

Relations can be represented in two ways.

a) As matrix
b) As graph

Representing as Matrix: Let R be a relation on sets A and B. Set A has m elements and set B has n elements. Then, the relation matrix M has m×n elements defined as follows:

$$M_{i,j} = 0 \text{ if R does not contain } (x_i, y_j)$$
$$1 \text{ if R contains } (x_i, y_j)$$

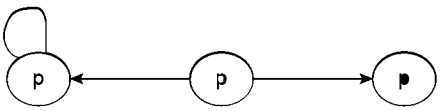
For example, consider two sets X = {1, 2, 3} and y ={a, b}. If there is a relation R on X, Y as (1, a) (2, a) (3, b), then the relation matrix M is given by:

$$M = \begin{array}{c|cc} & a & b \\ 1 & 1 & 0 \\ 2 & 1 & 0 \\ 3 & 0 & 1 \end{array}$$

Representing as a Graph: Consider a relation on a set X. If a set has n elements, then the graph will have n nodes. The edges in the graph indicate the relation R.

For Example: *Set* x = {p, q, r} R = {(p, p), (q, p), (q, r)}

The relation graph is given as



1.2.1 Binary Relation

If A and B are any sets and R ⊆ A × B, we call R a *binary relation from* A *to* B or a *binary relation between* A *and* B. A relation R ⊆ A × A is called a relation in or on A.

1.2.2 Domain and Range of Relation

The set **dom** R = {a | <a, b> ∈ R for some b} is called the *domain* of the relation R and the set **range** R = {b | <a, b> ∈ R for some a} is called the *range* of the relation R.

We may visually represent a relation R between two sets A and B by arrows in a diagram displaying the members of both sets.

A = {a, b}, B = {c, d, e} and the arrows represent a set theoretic relation R = {<a, d>, <a, e>, <b, c>}.

1.2.3 Operations on Relations

The *complement* of a relation R ⊆ A × B is defined as R′ = (A × B) − R.

Note that the complement of a relation depends on the universe we are considering. A given relation may certainly be a subset of more than one Cartesian product, and its complement will differ according to the Cartesian product we are taking to be the relevant universe.

Example 1.11

i) What is the complement of the relation R = {<a, d>, <a, e>, <b, c>} on the universe {a, b} × {c, d, e}?

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Answer: $R' = \{<a, c>, <b, d>, <b, e>\}$.

The *inverse* of a relation $R \subseteq A \times B$ is defined as the relation $R^{-1} \subseteq B \times A$,

$$R^{-1} = \{<b, a> \mid <a, b> \in R\}.$$ Note that $(R^{-1})^{-1} = R$.

ii) For the relation R given above,

$$R' = \{<a, c>,<b, d>, <b, e>\}$$
$$R^{-1} = \{<d, a>, <e, a>, <c, b>\}$$

More examples: Let X be the set of natural numbers {0, 1, 2, 3, 4, ...}. Let R be 'is less than' on (i.e. on X) Then what is R'? What is R^{-1}?

The pairs in R' would be of the form <a, b> where a is greater than b as $R' = (A \times B) - R$. The pairs in R^{-1} would be of the form <a, b> where a is greater than b as $R^{-1} = \{<a, b> \mid <b, a> \in R\}$. Both would be same.

We have focused so far on *binary* relations, that is, sets of ordered pairs. In a similar way, we could define *ternary*, *quaternary* or just *n-ary* relations consisting, respectively, of ordered triples, quadruples or n-tuples. A unary relation R on a set A is just a subset of the set A. A relation R is a collection for the sets that represent the pair of elements.

Example: If (a, b) is in R, we can represent their relation as aRb.

1.2.4 Properties of Relations

Here are some properties a relations on a set A can have.

1. A relation on A is said to be *reflexive* if for each $a \in A$, a is related to a. If we let R denote the relation, then we have aRa for each $a \in A$.

Example:

$$A = \{a, b\}\ R = \{(a, a), (b, b)\}$$

Example: Of a non-reflexive relation is the relation 'is the father of' on a set of people. As no person is the father of themself the relation is not reflexive.

2. A relation on A is said to be *irreflexive* if for each $a \in A$, a is not related to a. This is not the negation of the definition of reflexive. The relation 'is the father of' is irreflexive.
3. A relation R on A is *symmetric* if iRj implies jRi

Example:

$$R = \{(a, b), (b, a)\}$$

The relation 'is the sister of' is symmetric on a set of females and not symmetric on a set that contains a brother and sister. The empty relation on a set is an example of a symmetric relation since the statement 'if aRb' is always false.

4. A relation R on A is *antisymmetric* if iRj implies jRi is false.

Example:

$$R = \{(a, b)\}$$

5. A relation R on A is *transitive* if given aRb and bRc then aRc.

Example:

$$R = \{(a, b), (b, a),(a, a)\}$$

6. A relation that is reflexive, symmetric and transitive on set S is *equivalent relation*.

Example: Let the set {a, b, c} have the relation as {(a, a), (b, b), (c, c), (b, c), (c, b)}. It is reflexive with {(a, a), (b, b)}. It is symmetric as it has ordered pairs {(b, c) (c, b)}. It is also transitive because {(c, b), (b, c) (c, c)}. Hence the relation is equivalence relation.

1.3 Functions

Functions play a fundamental role in all areas of mathematics, as well as in other sciences and engineering. The characteristic property of a function, in the most abstract sense, is that it relates exactly one output to each of its admissible inputs. For example, associate each of a set of words with its own first letter. Functions in algebra are usually expressed in terms of algebraic operations.

Examples: $f(x) = x^2 + 1$
$f(x) =$ the mother of x

Intuitively, a function may be thought of as a 'process' or as a correspondence.
A function is generally represented in set theoretic terms as a special kind of relation.

1.3.1

Definition: A relation F from A to B is a function from A to B if and only if it meets both of the following conditions:

1. Each element in the domain of F is paired with just one element in the range, i.e., from <a, b> ∈ F and <a, c> ∈ F follows that b = c.
2. The domain of F is equal to A, **dom** F = A.

Equivalent Definition: A function is a subset R of A × B such that each element of A occurs as the first member of exactly *one* ordered pair in R.

For example, consider the sets A = {a, b} and B = {1, 2, 3}. The following relations from A to B are functions from A to B:

$$P = \{<a, 1>, <b, 1>\}$$
$$Q = \{<a, 2>, <b, 3>\}$$

The following relations from A to B are not functions from A to B:

$$S = \{<a, 1>\}$$
$$T = \{<a, 2>, <b, 1>, <b, 3>\}$$

S does not satisfy the condition 2 and T fails to meet condition 1. S is a function on the smaller domain {a}; T is not a function at all.

Much of the terminology used in talking about functions is the same as that for relations. We say that a function with domain A and range a subset of B is a function from A to B, while one in A × A is said to be a function *in* or *on* A. The notation F: A → B is used for 'F is a function from A to B'. Elements of the domain of a function are called *arguments* and their correspondents in the range are called *values*. If <a, b> ∈ F, the familiar notation F(a) = b is used. 'Map' and 'mapping' are commonly used synonyms for 'function'. A function *maps* each *argument* onto a corresponding *value*. A function F: $A'' \rightarrow A$ is also called an n-ary *operation* in A.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Examples:

i) Unary function f: N → N — Example: f(n) = n + 1 successor function
ii) Binary function f: N X N → N — Example: f(x, 4) = x + Y
iii) N-ary function f: N_1 X N_2 ... N_k → N — K-ary function

Functions as processes. Sometimes functions are considered in a different way, as processes, something such as devices or boxes with inputs and outputs. We put the argument in the input and get the value of the function in output.

1.3.2 Types of Functions

Functions from A to B in the general case are said to be *into* B. If the range of the function equals B, then the function is *onto* B (or *surjection*).

1. **One-to-one function**

   A function F: A → B is called *one-to-one* function (or *injection*) just in case no member of B is assigned to more than one member of A (so if a ≠ b, then F(a) ≠ F(b)).

   There is a one-to-one correspondence between the elements of the set A and the set B (Figure 1.1).

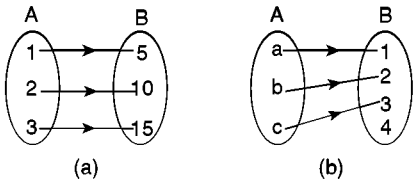


Fig. 1.1 *One-to-One Function*

2. **Many-to-one function**

   There is many-to-one correspondence between the elements of the set A and the set B (Figure 1.2).

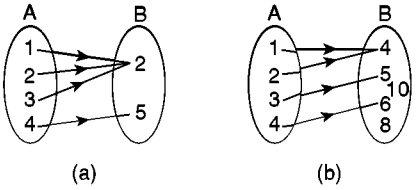


Fig. 1.2 *Many-to-One Function*

3. **Onto function**

   Every element of the set B has at least one pre-image.

   In Figure 1.3(a), the function is one-to-one and onto (or *bijection*), while in Figure 1.3(b), the function is many-to-one and onto (or *surjection*). A function that is both one-to-one and onto is called a *one-to-one correspondence*. It is easy to see that if

a function F is one-to-one correspondence, then the relation F – 1 is a function and one-to-one correspondence.

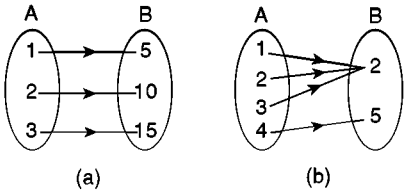


Fig. 1.3 *Onto Function*

4. **Into function**

There is at least one element of B that has no pre-image.

In Figure 1.4(a), the function is one-to-one and into, while in Figure 1.4(b), the function is many-to-one and into.

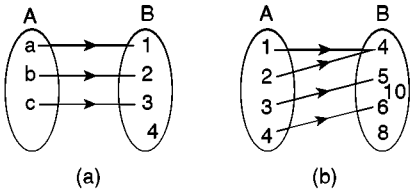


Fig. 1.4 *Into Function*

Function Composition: Given two functions F: A → B and G: B → C, we may form a new function from A to C, called the *composition* of F and G, written G O F. Function composition is defined as

G O F = {<x, z> |for some y, <x, y> ∈ F and <y, z> ∈ G}

The function F: A → A such that F = {<x, x>|x ∈ A} is called the *identity function on* A, written idA (or 1A). Given a function F: A → B that is a one-to-one correspondence, we have the following equations:

$$F^{-1} \text{ O } F = idA,$$
$$F \text{ O } F^{-1} = idA$$

The definition of composition need not be restricted to functions but can be applied to relations in general. Given relations R ⊆ A × B and S ⊆ B × C the composite of R and S written S O R = {<x, z> | for some y, <x, y> ∈ R and <y, z> ∈ S}

1.4 Alphabet, String and Language

A *language* is a dynamic set of visual, auditory or tactile symbols of communication and the elements used to manipulate them. *Language* can also refer to the use of such systems as a general phenomenon.

Symbol and alphabet: Symbol is an abstract entity. It cannot be formerly defined as points in geometry.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example 1.10 Letters, digits or special symbols such as $, @, # and so on.

Alphabet: Finite collection of symbols denoted by Σ.

Example 1.10 English alphabet $\Sigma = \{a, b, \ldots\ldots z\}$

Binary alphabet $\Sigma = \{0, 1\}$

String/word: A set of symbols from alphabet

Example 1.10 001, 110, 1111 strings from binary alphabet.

a01 is not a string from binary alphabet.

A *word* over an alphabet can be any finite sequence, or a string or a group of letters. The set of all words over an alphabet Σ is usually denoted by Σ^*. For any alphabet, there is only one word of length 0, the *empty word*, which is often denoted by e, ε or Λ.

Note: An empty string can be denoted by ε.

ε indicates no input symbols, i.e. zero input symbols.

$$|\varepsilon| = 0.$$

- Prefix of any string is any number of leading symbols of string.
- Suffix of any string is any number of trailing symbols of string.
- Proper substring is any substring except string itself.

Example 1.10

String : 'abc'
Prefix : ε, a, ab, abc
Suffix : ε, c, bc, abc
Substring : ε, a, b, c, bc, ab, abc (but not 'ac')

Language is a set of words or sentences.

1.4.1 Operations on Language

If L_1 and L_2 are two languages then

i) *Union* of two languages is denoted as $L_1 + L_2$ or $L_1 \cup L_2$
By union, we get the words from both languages.

ii) Concatenation of two languages is denoted as L_1L_2
By concatenation, one can combine two words to form a new word, whose length is the sum of the lengths of the original words. The result of concatenating a word with the empty word is the original word.

iii) *Kleene's closure* Σ^* – the language consisting of all words that are concatenations of 0 or more words in the original language (including null string $\varnothing$).

Example 1.10

i) $\Sigma = \{x\}$ $\Sigma^* = \{\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots\ldots\ldots\ldots\ldots\ldots\}$
$\Sigma^* = \{\varepsilon, x, xx, xxx \ldots\ldots\ldots\ldots\}$

ii) $\Sigma = \{a, b\}$ $\Sigma^* = \{\varepsilon, a, b, aa, ab, bb, ba, aaa, aab, \ldots\ldots\}$

iii) Positive closure $\Sigma^+ = \Sigma^* - \{\varepsilon\}$

$\Sigma^* = \Sigma^+ + \varepsilon$

$L_1 = \{good, bad\}$ $L_2 = \{boy, girl\}$

$L_1 \cup L_2 = \{good, bad, boy, girl\}$

$L_1L_2 = \{goodboy, goodgirl, badboy, badgirl\}$

1.4.2 Grammars

The grammar is basically defined as a set of 4-tuple (V, T, P, S)
where

V is a set of non-terminals (variables),

T is a set of terminals (primitive symbols),

P is a set of productions (rules) that relate the non-terminals and terminals

and S is start symbol with which strings in grammar are derived.

These productions define the strings belonging to the corresponding language. The motivation for these grammars was from the description of natural language using rules. Let us examine the rules used to define a sentence in English language.

<sentence> → <noun> <verb>

<noun> → <com –noun> | <prop –noun>

<verb> → **ate** | **sat** | **ran**

<com –noun> → **Rama** | **Sita**

<prop –noun> → **She** | **He**

Using these set of rules, many sentences can be derived by substituting for variables.

1. Rama ate.
2. Sita sat.
3. She ran.
4. He sat.

Language Acceptance: Start with start symbol, at every step, replace the non-terminal by right-hand side (RHS) of the rule. Continue this until a string of terminals is derived. The string of terminals gives the language accepted by grammar.

Consider the language represented by a^+, represents a set {a, aa, aaa, ….}. To generate strings of this language, we define grammar as S ➔ a and S ➔ aS. Now, we get strings as follows starting with S.

S ➔ a {a}

S ➔ aS ➔ aa {aa}

S ➔ aS ➔ aaS ➔ aaa {aaa}

1.4.3 Types of Grammars–Chomsky Hierarchy

Linguist Noam Chomsky defined a hierarchy of languages, in terms of complexity. This four-level hierarchy, called the *Chomsky hierarchy*, corresponds to four classes of machines. Each higher level in the hierarchy incorporates the lower levels, that is, anything that can be computed by a machine at the lowest level can also be computed by a machine at the next highest level.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

The Chomsky hierarchy classifies grammars according to the form of their productions into the following levels:

a) **Type 0 grammars–unrestricted grammars:** These grammars include all formal grammars. In unrestricted grammars (URGs), all the productions are of the form $\alpha \rightarrow \beta$ where α and β may have any number of terminals and non-terminals, that is, no restrictions on either side of productions. Every grammar is included in it if it has at least one non-terminal on the left-hand side (LHS).

Example 1.11

$$aA \rightarrow abCB$$
$$aA \rightarrow bAA$$
$$bA \rightarrow a$$
$$S \rightarrow aAb \mid \varepsilon$$

They generate exactly all languages that can be recognized by a Turing machine. The language that is recognized by a Turing machine is defined as set of all the strings on which it halts. These languages are also known as the *recursively enumerable languages*.

b) **Type 1 grammars–context-sensitive grammars:** These grammars define the context-sensitive languages. In context-sensitive grammar (CSG), all the productions of the form $\alpha \rightarrow \beta$ where length of α is less than or equal to length of β i.e. $|\alpha| \leq |\beta|$, α and β may have any number of terminals and non-terminals.

These grammars can have rules of the form $\alpha A \beta \rightarrow \alpha \gamma \beta$ with A as non-terminal and α, β and γ are strings of terminals and non-terminals. We can replace A by γ where A lies between α and β. Hence the name CSG. The strings α and β may be empty, but γ must be nonempty. It cannot include the rule $S \rightarrow \varepsilon$. These languages are exactly all languages that can be recognized by a linear bound automata.

Example 1.10

$$aAbcD \rightarrow abcDbcD$$

c) **Type 2 grammars – context-free grammars:** These grammars define the context-free languages. These are defined by rules of the form $\alpha \rightarrow \beta$ with $|\alpha| \leq | \beta$ where $|\alpha| = 1$ and is a non-terminal and β is a string of terminals and non-terminals. We can replace α by β regardless of where it appears. Hence the name context-free grammar (CFG). These languages are exactly those languages that can be recognized by a pushdown automaton. Context-free languages define the syntax of all programming languages.

Example 1.10

1. $S \rightarrow aS \mid S\ a \mid a$
2. $S \rightarrow aAA \mid bBB \mid \varepsilon$

d) **Type 3 grammars – regular grammars:** These grammars generate the regular languages. Such a grammar restricts its rules to a single non-terminal on the LHS. The RHS consists of either a single terminal or string of terminals with single non-terminal on left or right end. Here rules can be of the form $A \rightarrow a\ B \mid a$ or $A \rightarrow Ba \mid a$.

The rule $S \rightarrow \varepsilon$ is also allowed here. These languages are exactly those languages that can be recognized by a finite state automaton. This family of formal languages can be obtained by regular expressions also. Regular languages are used to define search patterns and the lexical structure of programming languages.

Example 1.10

Right linear grammar: $A \rightarrow a\,A \mid a$
Left linear grammar: $A \rightarrow A\,a \mid a$

An example of regular grammar (RG) G with $V = \{S, A\}$, $\Sigma = \{a, b, c\}$, P consists of the following rules:

$$S \rightarrow aS,\ S \rightarrow bA,\ A \rightarrow \varepsilon,\ A \rightarrow cA$$

and S is the start symbol. This grammar describes the same language as the regular expression a*bc*.

Every regular language is context-free, every context-free language is context-sensitive and every context-sensitive language is recursively enumerable. These are all proper inclusions, meaning that there exist recursively enumerable languages that are not context-sensitive, context-sensitive languages that are not context-free and context-free languages that are not regular.

Table 1.1 summarizes each of Chomsky's four types of grammars, the class of languages it generates, the type of automaton that recognizes it and the form of rules it must have.

Table 1.1 *Chomsky's hierarchy*

Grammar	Languages	Automaton	Production rules
Type 0	Recursively enumerable	Turing machine	$\alpha \rightarrow \beta$ No restrictions on β, α should have atleast one non terminal
Type 1	Context-sensitive	Linear bounded automata	$\alpha \rightarrow \beta$, $\lvert\alpha\rvert \leq \lvert\beta\rvert$
Type 2	Context-free	Pushdown automaton	$\alpha \rightarrow \beta$, $\lvert\alpha\rvert = 1$
Type 3	Regular	Finite state automaton	$\alpha \rightarrow \beta$, $\alpha = \{V\}$ and $\beta = V\{T\}^*$ or $\{T\}^*V$ or T^*

Example 1.11 Give a CSG but not CFG for $(a^n \mid n \geq 1)$

$$S \rightarrow aS \mid B$$
$$aS \rightarrow aa$$
$$B \rightarrow a$$

Example 1.10 Give a CFG but not regular for $(a^n \mid n \geq 1)$

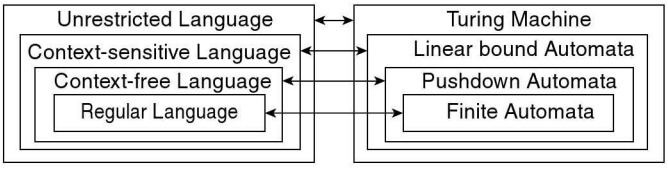
$$S \rightarrow A\,S \mid a$$
$$A \rightarrow a$$

Example 1.10 Give an RG for $(a^n \mid n \geq 1)$

$$S \rightarrow aS \mid a$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

The hierarchy of languages and the machine that can recognize the same is shown below.



Every RG is context-free, every CFG is context-sensitive and every CSG is unrestricted. So the family of regular languages can be recognized by any machine. CFLs are recognized by pushdown automata, linear bound automata and Turing machines. CSLs are recognized by linear bound automata and Turing machines. Unrestricted languages are recognized by only Turing machines.

1.5 Graphs and Trees

A *graph* G is a pair (V, E), where V is a finite set and E is a relation on V. The elements of V are called *nodes* or *vertices*. The elements of E are called *edges* or *arcs*. In G if <u, v> is an edge, then u is a *predecessor* of v and v is a *successor* of u. The graph is said to be *ordered* if some ordering is assumed on the predecessors of each node *and* on the successors of each node.

A *path* in G is a sequence $v_1, ..., v_n$ of nodes such that v_i is a successor of v_{i-1} for i = 2, ..., n. The *length* of the path is n − 1. The path is a *cycle* if n > 1 and $v_n = v_1$.

A graph $G_1 = (V_1, E_1)$ is said to be a *subgraph* of a graph $G_2 = (V_2, E_2)$, if $V_1 \subseteq V_2$ and $E_1 \subseteq E_2$.

A graph is constructed by a nonempty set of vertices (V) and a set of edges (E). Each edge is a pair of vertices from V for instance.

$$e_i = (V_j, V_K) \text{ is edge from } V_j \text{ to } V_K$$
$$V = \{V_1, V_2, \ldots\ldots\ldots\ldots\}$$
$$E = \{e_1, e_2, \ldots\ldots\ldots\ldots\}$$

Each graph G = (V, E) can be represented by a diagram of the following form:

1.5.1 Directed Graph

A graph with directed edges is called directed graph. Edges are given by ordered pairs. Figure 1.5 is a directed graph. The ordered pair $<V_1, V_2>$ is associated with edge e_1 and V_1 is a predecessor of V_2 or V_2 is a successor of V_1.

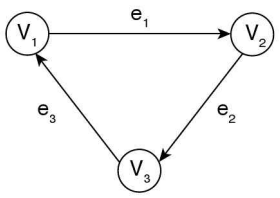


Fig. 1.5 *Directed Graph*

1.5.2 Undirected Graph

Figures 1.6 and 1.7 depict undirected graphs; the unordered pair $<V_1, V_2>$ is associated with edge e_1 and the pair $<V_2, V_2>$ in Figure 1.7 is associated with e_4, e_4 is a self-loop.

Degree: The degree of a vertex in an undirected graph is the number of edges for vertex. A self-loop is counted twice. For directed graphs, the degree is given in terms of indegree and outdegree. Indegree is the number of edges entering to the vertex and outdegree is the number of vertices outgoing from the vertex. In Figure 1.7, the degree of vertex V_1, V_2 and V_3 is 2, 4 and 2. In Figure 1.8, the indegree of V_1, V_2, V_3 and V_4 is 1, 2, 3 and 1. The outdegree of V_1, V_2, V_3 and V_4 is 2, 2, 1 and 2, respectively.

A path in a graph is a sequence of vertices $V_1, V_2, \ldots V_k$ $k \geq 1$ such that $<V_i, V_{i+1}>$ is an edge for each i, $1< i< k$. We say the path is from V_i to V_k. If $v \rightarrow k$ is an edge, then we say v is predecessor of k and k is a successor of v.

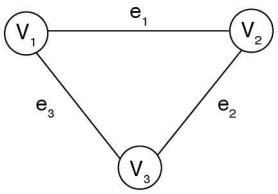


Fig. 1.6 *Undirected Graph*

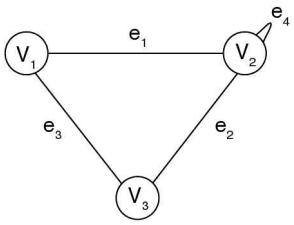


Fig. 1.7 *Undirected Graph with Self Loop*

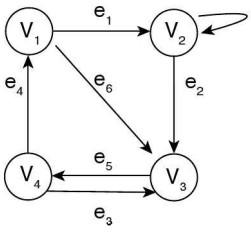


Fig. 1.8 *Directed Graph with Self Loop*

1.5.3 Trees

Tree is a collection of vertices and edges with the following properties.

1. There is one vertex called root which has no predecessors and from which there is a path to every vertex.
2. From this root node, all the successors are ordered from the left.

Example: In Figure 1.9, the computer is a root node. CPU, monitor, keyboard and mouse are the interior nodes and also siblings (children of some parent). Motherboard and memory hard disk are the leaf nodes for the CPU. Monitor is a parent node or father of menu button and cathode ray tube, whereas menu button is a left child and CRT is right child of monitor.

The only difference between graphs and trees is that graphs do not have a special node called root node. There is a special terminology for trees that differs from the general

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] konkurcomputer.ir

terminology for arbitrary graphs. A successor of a vertex is called a son, and a predecessor is called a father. If there is a path from vertex V_1 to V_2, then V_1 is said to be an ancestor to V_2 and V_2 is said to be descendent of V_1. A vertex with no sons is called leaf node and other nodes are called *interior nodes*. In Figure 1.9, a key is a son of node keyboard. CPU, monitor, keyboard and mouse are interior nodes. Motherboard, memory and hard disk are leaf nodes.

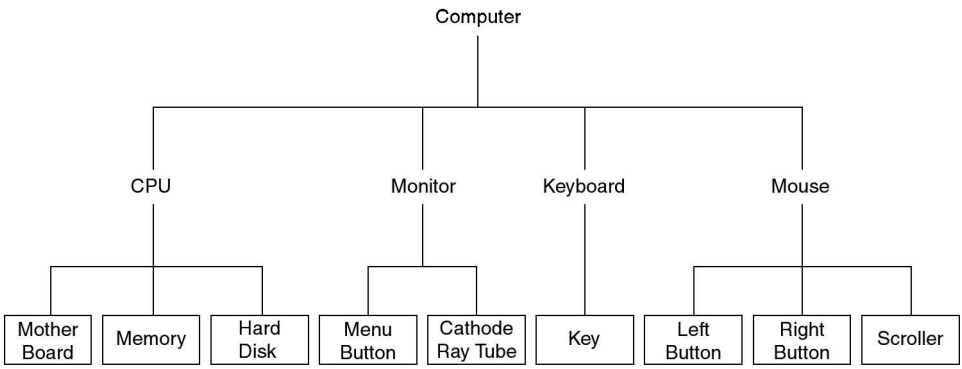


Fig. 1.9 *Tree Representation of Accessories of Computer*

Binary tree: A binary tree is a data structure on which each node has at most two children, which are called left child and right child.

Leaf is a node having no children. In Figure 1.10, nodes 8, 7 and 13 are leaf nodes.

The depth of node n is the length of path from the root to the node. The set of all nodes at a given depth is called level. The node 8 has depth 2. At level 2 the nodes present are 9 and 11.

The height of the tree is the maximum depth. It is maximum level number −1. In the Figure 1.10 the node 13 is at level 4. Hence the height is 3.

A complete binary tree is a binary tree in which there are 2^K nodes at every depth K where K < n (the height of the tree).

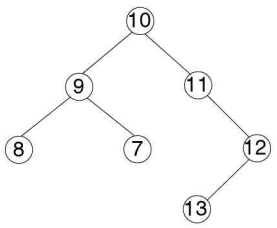


Fig. 1.10 *Sample Binary Tree*

1.6 Theorem Proving

In many mathematical textbooks, we will see *definitions* and theorems. The definitions are defining technical words and give a name to a special subclass of objects. Definitions have the form 'Y is a *blob* if and only if *condition*'. This *biconditional* form is used in two ways. First, suppose

we have an object Y and want to know if it is a blob then we see if Y satisfies the condition. If it does, then Y is a blob, otherwise it is not. Second, suppose we have an object Y that is a blob. Then, we know that Y *does* satisfy the condition and may make use of this fact in proof.

Theorem statements have two forms. The first is a conditional statement 'If *condition* then *conclusion*'. This form is an implication, and it means that the conclusion is true whenever the condition is true.

There are many ways to prove the definitions or theorems.

i) Proof by induction
ii) Proof by contradiction
iii) Proof by example

(i) and (ii) are used widely in this book.

1.6.1 Proof by Induction

Many theorems are proved by mathematical induction. Suppose we have a statement P(n) about a non-negative integer n. A commonly chosen example is to take P(n) to be

$$1 + 2 + 3 + \ldots\ldots\ldots\ldots\ldots\ldots n = \frac{n(n+1)}{2}$$

The principle of mathematical induction is that P(n) follows from

a) $P(0)$ and
b) P(n − 1) implies P(n) for N ≥ 1

Condition (a) is an inductive proof, called the 'basis'.
Condition (b) is called the inductive step.
The LHS of (b), that is P(n − 1), is called the inductive hypothesis.

Example 1.24

Basis of induction:

n = 1 then LHS = 1
RHS = n(n + 1)/2 = 2/2 = 1

Induction hypothesis:
We assume n = k. Then equation becomes

$$1 + 2 + 3 + \ldots\ldots\ldots\ldots\ldots + K = K(K + 1)/2$$

Inductive step:
We assume that equation is true for n = k. And then check if it is also true for n = K + 1 or not.

$$LHS = 1 + 2 + 3\ldots\ldots\ldots\ldots K + (K + 1)$$

$$RHS = \frac{K(K+1)}{2} + (K+1) = \frac{K^2 + 3K + 1}{2} = \frac{(K+1)(K+2)}{2}$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example 1.24 Prove by mathematical induction $n^4 - 4n^2$ is divisible by 3 for $n \geq 0$.

Solution:

Basic step: For $n = 0$

$n^4 - 4n^2 = 0$ that is divisible by 3

Inductive hypothesis

let $n^4 - 4n^2$ is divisible by 3

Induction step: $(n + 1)^4 - 4(n + 1)^2$

$$= ((n + 1)^2)^2 - 4(n + 1)^2$$
$$= (n^2 + 2n + 1)^2 - (2n + 2)^2$$
$$= (n^2 + 2n + 1 + 2n + 2)\,(n^2 + 2n + 1 - 2n - 2)$$

Since the above one is of the form $a^2 - b^2 = (a + b)(a - b)$

$$= (n^4 + 4n + 3)\,(n^2 - 1)$$
$$= n^4 + 4n^3 + 3n^2 - 3 - 4n - n^2$$
$$= (n^4 - 4n^2) + 6n^2 - 3 + 4(n^3 - n) \dashrightarrow \quad (1)$$

We now prove that $(n^3 - n)$ is divisible by 3.
Basis for $n = 0$, $n^3 - n = 0$ is divisible by 3.

Inductive Step: True for $n + 1$

$$= (n + 1)^3 - (n + 1)$$
$$= (n^3 + 3n^2 + 3n + 1) - (n + 1)$$
$$= n^3 + 3n^2 + 2n = (n^3 - n) + 3n^2 + 3n$$

divisible by 3
Hence (1) is divisible by 3. Hence proved.

1.6.2 Proof by Contradiction

In a proof by contradiction, we assume, along with the hypotheses, the *logical negation* of the result we wish to prove and then reach some kind of contradiction.

Principle of contradiction is as follows.

If we want to prove 'If P, then Q'

1. We assume P and not Q.
2. We arrive at some conclusion contradicting one of our assumptions, or something obviously untrue for not Q.
3. This contradicts our assumption for P and not Q.

Example 1.24 There are no rational number solutions to the equation $x^3 + x + 1 = 0$ using proof by contradiction.

Solution:

Step 1: Assume to the contrary there is a rational number p/q, in reduced form, with p not equal to 0, which satisfies the equation.

Then, we have $p^3/q^3 + p/q + 1 = 0$.

After multiplying each side of the equation by q^3, we get the equation

$$p^3 + p\,q^2 + q^3 = 0$$

Step 2: There are four cases to consider for p and q.

Case 1: If p and q are both odd, then the LHS of the above equation is odd. But zero is not odd, which leaves us with a contradiction.

Case 2: If p is even and q is odd, then the LHS is odd, again a contradiction.

Case 3: If p is odd and q is even, we get the same contradiction.

Case 4: If p and q are even, this is not possible because we assumed that p/q is in reduced form.

This completes the proof.

1.6.3 Proof by Example

Proof by example is the simplest method which involves in deriving general conclusions based on one or more examples. For instance if there is an item X in group A having a property P, then any item Y in the same group G would have the same property.

Example 1.24 Sum of an odd and an even number is always odd.

Proof: To prove that the sum of an odd and an even number is always odd,

Let us consider an even number $X = 2K$ where K is some natural number.

Let Y be an odd number $Y = 2Q + 1$ where Q is some natural number.

Now to prove that sum of X and Y is always odd

$$X + Y = 2K + 2Q + 1 = 2(K + Q) + 1$$

K and Q are natural numbers. Hence K + Q is also a natural number. Hence 2(K + Q) + 1 is always odd.

Summary

1. A set is a collection of objects.
2. The number of elements in a set is called the *cardinality* of set.
3. Empty set is a subset of every set.
4. A relation is a set of ordered pairs.
5. Relation is an equivalence relation if it is reflexive, symmetric and transitive.
6. Every function is a relation but every relation is not a function.
7. Number of edges incident on any node is called degree of the node.
8. If a set X has n elements, then there are $2^{n \times n}$ relation from X to X.
9. A grammar is a set of finite rules that may define infinite sentences.
10. RG is a subset of CSG.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

11. CSG is a subset of CFG.
12. CFG is a subset of URG.
13. Turing machine recognizes type 0 grammars.
14. PDA recognizes type1 grammars.
15. LBA recognizes type 2 grammars.
16. Finite automata recognizes type 3 grammars.

Short Answers

1. Define sets and what are the different types of representations.

   Answer: A set is defined as collection of objects. These objects are called elements of the set. There are three main ways to specify a set

   a) By listing all its members(list notation)
   b) By stating property of its elements(predicate notation)
   c) By defining the set of rules which generates to defines its members(recursive rules)

2. What is power set? Find power set for A={2, 4, 6, 8}

   Answer: The set of all subsets of a set A is called the power set of A and denoted as $\wp(A)$ or sometimes as 2A.

   $\wp(A) = \{\{\Phi\}, \{2\}, \{4\}, \{6\}, \{8\}, \{2, 4\}, \{2, 6\}, \{2, 6\}, \{2, 8\}, \{4, 6\}, \{4, 8\}, \{6, 8\}, \{2, 4, 6\}, \{2, 4, 8\}, \{2, 6, 8\}, \{4, 6, 8\}, \{2, 4, 6, 8\}\}$

3. Name the operations that can be applied on a set.

   Answer: The operations applied on sets include Union, Intersection, Difference and Complement.

4. List the properties of relations and when the relation is said to be equivalence.

   Answer: The properties of relations are

   a) Reflexive
   b) Irreflexive
   c) Symmetric
   d) Antisymmetric
   e) Transitive

   If a relation satisfies symmetric, reflexive, symmetric and transitive properties then it is said to be equivalence relation.

5. Name different types of functions

   Answer: The type of functions are

   a) One to one
   b) Onto
   c) Into
   d) Many to one

6. What are different types of languages and list the machines that recognize the languages

   Answer: There are four types of languages as listed below

   1. type 0 – Unrestricted language recognised by Turing Machine
   2. type 1 – Context sensitive language recognised by Linear Bound Automaton
   3. type 2 – Context free language recognised by Pushdown Automaton
   4. type 3 – Regular language recognised by Finite Automaton

Fill in the Blanks

1. If U = {a, b, c, d, e, f}, X = {a, b, c}, Y = {d, e, f), then X – Y is _____.
2. If A = {0, 1}, 2^A is _____.
3. An equivalence relation is _____.
4. If A = aab then A^2 is _____.
5. If A = 111 then εA is _____.
6. $A^* - A^+ =$
7. If xRy and yRx, then relation is _____.
8. Every relation is a function (True/False).
9. If X = {a, b}, then number of relations from X to X is _____.
10. If a set A has n elements, then power set of A is _____.
11. All grammars are expressed in _____ tuple form.
12. _____ in the grammar are used to define the strings belonging to the language.
13. _____ defines the hierarchy of different levels of the language.
14. A production is said to be valid belonging to any type of grammar if it has at least _____ on the LHS of the production.
15. _____ grammars have both terminals and non-terminals on the LHS and also the length of LHS is less than or equal to the RHS of the production.
16. _____ grammars are the only grammars that allow strings of terminals and non-terminals on both side of the production with only constraint that the left side should have at least one non-terminal.
17. All context-sensitive languages are also _____ languages.
18. _____ is the machine to recognize the context-sensitive languages.
19. The grammars that allow only one non-terminal on the left side and at most one non-terminal on the right side of the production are _____ grammars.
20. _____ languages are used to define the lexical structure of the programming languages.
21. _____ are the languages that are accepted by pushdown automata.
22. The machine that can accept all classes of languages is _____.
23. Right linear grammars are _____ of CFG.
24. CFGs are _____ CSG.
25. URGs are _____ of CSGs.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

26. The following grammar is RG (True/False):

$$\Sigma \rightarrow A\alpha \mid \alpha$$

27. The following grammar is RG (True/False):

$$\Sigma \rightarrow A\alpha \mid \alpha \quad A \rightarrow \alpha A \mid \alpha$$

Answers

1. {a, b, c}
2. {∅, {0}, {1}, {0,1}}
3. reflexive, symmetric and transitive
4. aabaab
5. 111
6. ε
7. symmetric
8. False
9. 4
10. 2^n
11. Four
12. Production
13. Chomsky hierarchy
14. One non-terminal
15. Context-sensitive or type 1
16. Unrestricted or type 0
17. Unrestricted languages / recurssively enumerable
18. Linear bound automata
19. Regular or type 3
20. Regular languages
21. Regular and context-free languages
22. Turing machine
23. Subset
24. Subset
25. Superset
26. True
27. False

Objective Question Bank

1. Read the following:
   1. ε
   2. a
   3. ax where x is palindrome
   4. xa where x is a palindrome
   5. axa where x is a palindrome

   Which of the following is correct?
   a) 1 and 2 are the only palindromes
   b) 3 and 4 are the only palindromes
   c) 1, 2 and 5 are the only palindromes
   d) 5 is palindrome but not 1 and 2
2. Read the following:
   1. The relation R_1 on integers defined by iR_1j if and only if $i = j$.
   2. The relation R_2 on people defined by pR_2q if and only if p and q were born at the same hour of the same day of some year.
   3. The Relation R on set of integers is defined by pRq if and only if p is equal to q.

   Which of the following is correct?
   a) 1 is the only equivalence relation
   b) 2 and 3 are the only equivalence relations
   c) 1 and 3 are equivalence relation but not 2
   d) All are equivalence relations.
3. What is the reflexive closure of the relation?
   {(1,2), (2,3), (3,4), (5,4)} on the set {1, 2, 3, 4, 5}

a) {(1,2), (2,3), (3,4), (5,4)}
b) {(1,1), (1,2), (2,3), (3,4), (5,4)}
c) {(1,1), (1,2), (2,2), (2,3), (3,3), (3,4), (5,5), (5,4)}
d) {(1,1), (1,2),(2,2), (2,3), (3,3), (3,4), (4,4), (5,5), (5,4)}

4. What is the symmetric closure of the relation {(1,2), (2,3), (3,4), (5,4,)} on the set {1, 2, 3, 4, 5}?
   a) {(1,2), (2,1), (2,3), (3,4), (4,3), (5,4), (4,5)}
   b) {(1,2), (2,1), (2,3), (3,2), (3,4), (4,3), (5,4), (4,5)}
   c) {(1,2), (2,3), (3,4), (5,4)}
   d) {2,1), (3,2), (4,3), (4,5)}

5. What is the reflexive symmetric closure of the relation {(1,2), (2,3), (5,4)} on the set {1, 2, 3, 4, 5}?
   a) {(1,2), (2,3), (2,1), (3,2), (5,4), (4,5)}
   b) {(1,2), (1,1), (2,1), ((2,3), (2,2), (3,2), (5,5), (5,4), (4,5)}
   c) {(1,2), (2,3), (5,4), (1,1), (2,2), (3,3), (4,4), (5,5), (2,1), (3,2), (4,5)}
   d) {(1,2), (1,1), (2,3), (2,2), (5,4), (4,5)}

6. Read the following the statements:
   1. Any subset of countably infinite set is either finite or countably infinite
   2. The set of all ordered pairs of integers is countably infinite
   3. The union of a countably infinite collection of countably infinite sets is countably infinite
   4. The Cartesian product of a countably infinite collection of a countably infinite set is countably infinite

   Identify the correct statements of the above
   a) 1 and 2 only b) 1, 2 and 3 only
   c) 3 and 4 only d) All are correct.

7. Consider the relation {(1,1), (1,2), (2,3), (3,4)} on set {1, 2, 3, 4}. Which of the following is correct?
   a) {(1,1), (1,2), (2,3), (2,2), (3,4), (3,3)} is reflexive symmetric closure of given relation
   b) {(1,1), (1,2), (2,1), (2,3), (3,2), (3,4)(4,3)} is transitive closure of given relation
   c) {(1,1), (1,2), (2,3), (1,3), (2,3), (3,4),(2,4)} is symmetric closure of given relation
   d) {(1,1), (1,2), (2,1), (2,3), (3,2), (3,4),(4,3)} is symmetric closure of given relation

8. Which of the following relation on set {1, 2, 3, 4} satisfies antisymmetric relation?
   a) {(1,1), (2,3), (3,4), (2,2), (3,2)}
   b) {(1,1), (2,2), (3,3), (4,3), (3,4)}
   c) {(1,1), (2,3), (3,4), (1,3)}
   d) {(1,2), (2,1), (1,3), (3,3)}

9. Read the following statements:
   1. Points on a line
   2. The power set of natural numbers
   3. The set of formal languages
   4. The number of people living on the earth

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Which of the following is correct?
a) 1 and 4 are countable but 2 and 3 are not
b) 1, 2 and 3 are uncountable sets and 4 is the only countable set
c) 3 and 4 are countable but 1 and 2 are not
d) 2 and 4 are countable but 1 and 3 are not

10. The sum of degree of n vertices of a tree is equal to
a) $n-1$ b) $2n-1$ c) $2n-2$ d) $2n$

11. If a set X has m elements and set Y has n elements, then how many relations are there from X to Y?
a) nm b) 2^{mxn} c) 2^{n+m} d) None

12. Which of the following is true?
a) $\varnothing \cup \{\varnothing\} = \{\varnothing\}$ b) $\varnothing \cap \{\varnothing\} = \{\varnothing\}$
c) $\varnothing \cap \{a, \varnothing\} = \{\varnothing\}$ d) All

13. The total number of edges in any tree with n vertices is
a) $n(n-1)/2$ b) $n/2$ c) n d) $n-1$

14. A graph having multiple edges between vertices is called
a) Simple graph b) Multigraph c) Complete graph d) None

15. The power set of $\varnothing$ is
a) $\{\{\varnothing\}\}$ b) $\{\varnothing\}$ c) $\{\varnothing, \varnothing\}$ d) None

16. Let R be a nonempty relation on a collection of sets defined by A R B (A relates to B) if and only if $A \cap B = \varnothing$. Pick the true statement:
a) R is reflexive and transitive
b) R is symmetric and not transitive
c) R is an equivalence relation
d) R is not reflexive and not symmetric

17. The word 'format' in formal languages means
a) The symbols used have well-defined meaning
b) They are unnecessary, in reality
c) Only the form of the string of symbols is significant
d) None of the above

18. A grammar G has productions of the form $X \rightarrow 0Y \mid Y0 \mid a$. Then, G is
a) regular b) CFG c) non-regular d) Both (a) and (b)

19. A grammar G is called a CFG if the production has
a) Only one terminal on its LHS
b) Any combination of terminal and non-terminal on its RHS
c) Both (a) and (b)
d) RHS always starting with a non-terminal

20. A grammar G is said to be RG if the production has
a) Only one terminal on its LHS
b) At most one non-terminal on its RHS
c) Either (a) or (b)
d) Both (a) and (b)

21. Which of the following statements is true?
    a) All RGs are CFGs
    b) All CFGs are RGs
    c) RGs are not CFGs
    d) None of these

22. Which of the following is RG?
    a) $S \rightarrow 0S|S0|0$
    b) $S \rightarrow aS\ |bA, A \rightarrow bA|\ b$
    c) $S \rightarrow aSa\ |\ b$
    d) $S \rightarrow aAb, A \rightarrow aA|b$

23. Which of the following is CFG?
    a) $S \rightarrow 0A|B1|0, 0A \rightarrow 0A0|0, B1 \rightarrow 1B1|1$
    b) $S \rightarrow aS\ |\ bA, bA \rightarrow bA\ |\ b$
    c) $S \rightarrow aSa\ |\ bA, bA \rightarrow aAbA\ |\ c$
    d) $S \rightarrow aAb, A \rightarrow AaAbA\ |\ ab$

24. Which of the following is CSG?
    a) $S \rightarrow 0A|B1|0, 0A \rightarrow 0A0|0, B1 \rightarrow 1B1|1$
    b) $S \rightarrow aS\ |\ bA, bA \rightarrow bA\ |\ bb$
    c) $S \rightarrow aSa\ |bA, bA \rightarrow aAbA|\ c$
    d) $S \rightarrow aAb, A \rightarrow AaAbA|\ \varepsilon$

25. What is the language generated by the grammar $S \rightarrow 0S\ |1A, 1A \rightarrow 1A|\ 11$?
    a) Regular language
    b) Context-free language
    c) Context-sensitive language
    d) None of the above

26. What is the language generated by the grammar $S \rightarrow 0A|B1|0, 0A \rightarrow 0A0|0, B1 \rightarrow 1B1|\ 1$
    a) Context-free language
    b) Context-sensitive language
    c) Unrestricted language
    d) All the above

Answers

1.	c	2.	d	3.	d	4.	b	5.	c	6.	d	7.	d	8.	c	9.	a
10.	c	11.	b	12	d	13.	d	14.	b	15.	a	16.	d	17.	c	18.	b
19.	c	20.	d	21.	a	22.	b	23.	d	24.	b	25.	c	26.	c		

Exercises

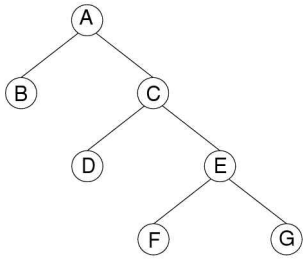
1. Write the difference between the + closure and * closure?
2. Define language, grammar.
3. Describe the following sets:
    a) the set containing empty set
    b) the set containing string India
    c) the set containing all natural numbers less than 7.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

4. Set A = {a, b, c, d} and B = {a, b}
   a) Is A a subset of B?
   b) Is B a subset of A?
   c) What is A ∪ B, A ∪ B, AXB and A – B?
   d) What is 2^B?
5. If A is a set with three elements, how many elements are there in power set of C? Justify your answer with proof.
6. Set A = {1, 2, 3, 4, 5} B = {6, 7, 8, 9, 10}, the unary function f:X ➔ Y and binary function g: XXY ➔ X are defined as follows.

N	f(n)	G	6	7	8	9	10
1	3	1	4	4	2	3	5
2	5	2	1	1	1	1	1
3	3	3	2	3	1	2	4
4	2	4	4	2	2	4	1
5	4	5	2	2	2	2	2

   i) What is the value of f(3), g(3, 8)?
   ii) What the domain and ranges of f and g?
   iii) What is value of g(4, f(4))?
7. Consider the undirected graph G = (V, E) where V is a set of nodes = {1, 2, 3, 4, 5}. The set of edges is {{1, 4}{2, 3}{2, 4}{3, 5}{1, 5}}. What is the degree of node 4?
8. In the following tree



   i) Which nodes are the sons of E?
   ii) Which node is the father of C?
   iii) What is the length of path from A to E?
   iv) Which node is a root?
9. Define language, grammar.
10. Explain the types of grammars along with corresponding recognizers.
11. What is Chomsky hierarchy of grammars?
12. Prove the language equivalence of CFG and PDA.
13. Prove the language equivalence of FA and RG.
14. Prove the language equivalence of LBA and CSL.
15. Prove the language equivalence of TM and recursively enumerable languages.
    i) Which nodes are the sons of E?
    ii) Which node is the father of C?

iii) What is the length of path from A to F?
iv) Which node is a root?

16. Prove by mathematical induction
    i) $1 + 4 + 7 + \ldots.(3n - 2) = n(3n - 1)/2$
    ii) $\sum_{K=1}^{n} K^2 = n(n + 1)(2n + 1)/6$
17. Give some examples of strings which are not in the language
    a) $L = \{WCW^R/W \in \{a, b\}\}$
    b) $L = \{O^m O^n O^{m+n}/m \geq 1 \text{ and } n \geq 1\}$
    c) Set of all strings with equal number of O's and 1's $\{ww^R /w \text{ is in } (0 + 1)^+\}$
    d) Write the set $\varnothing$ X (1, 2}
    e) Write the set {a} X {a, b} X {a, b, c}

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

This page is intentionally left blank

2 Finite Automata

In this chapter, we introduce the definition of Finite Automaton (FA) and also some real-time problems that can be solved using FA. Representation of FA is done using 5-tuple form, transition table and transition diagram. Definitions of non-deterministic and deterministic FAs are explained with examples and, further, how these automata act as language acceptor is elaborated. Equivalence of NFAs-with-ε to NFAs-without-ε, and equivalence of NFAs to DFAs are explained. Minimizing the given automaton using Myhill Nerode and π construction method, and testing the equivalence of two FAs are discussed. Moore and Mealy machines, which represent Finite Automaton with output, are explained with examples.

2.1 Finite-state Machine

The finite-state system represents a mathematical model of a system with some input. The model finally gives some output. The input supplied to the machine is processed by various states. These states are called intermediate states.

Good example of finite-state systems is a control mechanism of elevator. This mechanism remembers only the current floor number pressed; it does not remember any of the previously pressed numbers.

Another good example is a bulb operated with a single switch. Switch has two possibilities: it closes or opens the circuit. When the circuit is closed the current passes and the bulb glows.

From the above statement it is clear that the bulb will be either in ON state or OFF state. If it is in ON state and if the current passes through it, then the bulb will glow. If no current is passed then it changes to OFF state. The flow of current is indicated by value 1, and blocked state of current flow is indicated by 0. The functioning of the bulb can be shown in the form of a table:

State	Current = 1	Current = 0
ON	ON	OFF
OFF	ON	OFF

This is a finite system that represents the functioning of a bulb. Many real-time problems can be represented using such mathematical models.

The finite-state systems are useful in designing text editors, lexical analysers and natural language processing.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Definition 1: A finite automaton is formerly defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states which is non-empty
- Σ is the input alphabet
- q_0 is the initial state
- F is a set of final states and $F \subseteq Q$
- δ is a transition function or mapping function $Qx\ \Sigma \rightarrow Q$ using this the next state can be determined depending on the current input.

2.1.1 Finite-Automaton Model

The finite automaton can be represented as input tape and finite control as shown in the Figure 2.1:

i) **Input tape:** is a linear tape having some cells that can hold an input symbol from Σ.
ii) **Finite control:** The finite control indicates the current state and it decides on the next state on receiving particular input from input tape. The tape reader reads the cells one by one from left to right, and at any instance only one input symbol is read.

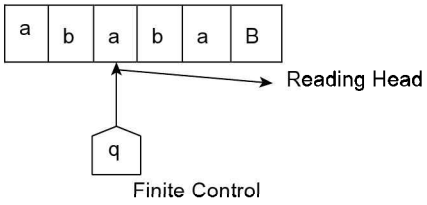


Fig. 2.1 *Finite Automaton*

The reading head examines the read symbol, and the head moves to the right with or without changing the state. When the entire string is read, if finite control is in final state, the string is accepted; else it is rejected. Finite automaton can be represented by the transition diagram, in which the vertices represent the states and edges represent transitions. We can model the real-word problems as a finite automaton and this helps in understanding the behaviour and in analysing the behaviour.

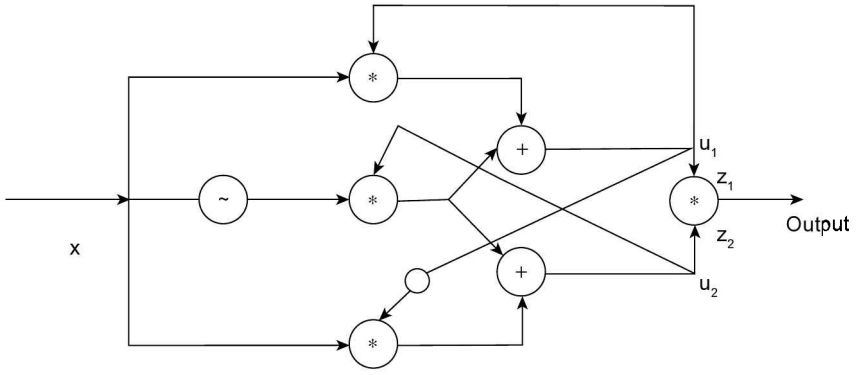


Fig. 2.2 *Combinational Circuit*

Example 2.1 Consider the following combinational circuit shown in Figure 2.2 represent the circuit using finite state system.

A circle with * (or ⊛) represents AND gate, ⊕ represents OR gate and ⊖ represents inverter or NOT gate. By changing the signals at U_1, U_2 and i/p signal **x** we get output $Z_1 * Z_2$. This behaviour can be shown as an FA:

If output expected is 1, then states [11] become an accepting state of the finite automaton. Figure 2.3 gives a simple model of a combinational circuit, which helps us in analysing the behaviour, by changing signal at Z_1Z_2.

	0	1
$U_1\ U_2$	$z_1\ z_2$	$z_1\ z_2$
00	00	01
01	11	01
11	11	10
10	00	10

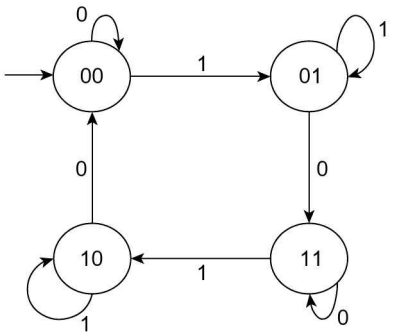


Fig. 2.3 *Finite Automaton for Combinational Circuit*

Example 2.1 Lexical analyser behaviour can be shown as an FA. Consider the lexical analyser that matches words such as 'the', 'this', 'that', 'to' shown as FA in Figure 2.4.

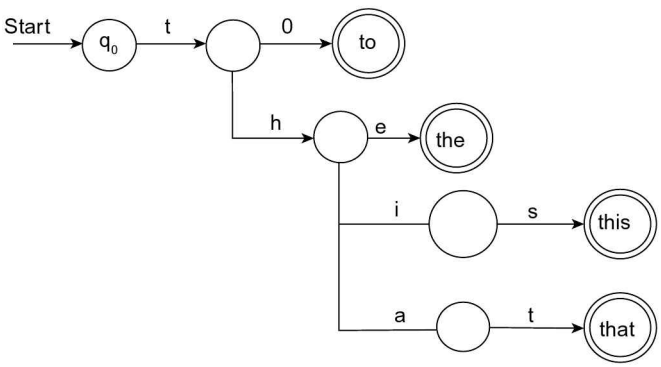


Fig. 2.4 *Simple Lexical Analyzer*

These systems are called finite automaton as the number of possible states and the number of letters in the alphabet are, both, finite. It is automaton because the change of state is totally governed by the input.

2.1.2 Properties of Transition Function 'χ'

The transition function δ defined as $Q \times \Sigma \rightarrow Q$ holds the following properties for all states $q \in Q$ and $a \in \Sigma^*$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

1. $\delta(q, \varepsilon) = q$. The states of the FA are changed only by an input symbol.
2. For all strings w and i/p symbol a,

$$\delta(q, aw) = \delta(\delta(q, a), w)$$

An FA can be represented by a

a) Transition diagram

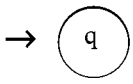
b) Transition table

2.1.3 Transition Diagram

A transition graph contains

a) Set of states as circles

Start state q_0 with arrow



Final state by double circle (Figure 2.5).

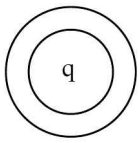


Fig. 2.5 *Representation of Final States*

b) A finite set of transitions (edges|labels) that shows how to go from one state to another state as shown in Figure 2.6.

2.1.4 Transition Table

Following is a tabular representation where rows correspond to states and columns correspond to input. Start state is given by $\rightarrow$ and the final state by *

Example 2.3 $M = \{\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\}\}$

$\delta(q_0, a) = q_1$ $\quad \delta(q_0, b) = q_2$

$\delta(q_1, a) = q_2$ $\quad \delta(q_1, b) = q_0$

$\delta(q_2, a) = q_2$ $\quad \delta(q_2, b) = q_2$

Δ/Σ	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_2	q_0
$*q_2$	q_2	q_2

The same table can be shown as a transition diagram (Figure 2.6)

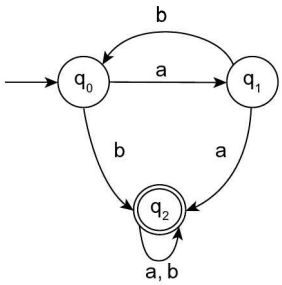


Fig. 2.6 *Transition Diagram*

2.2 Language Acceptance

A string w is accepted by finite automaton U = {Q, Σ, δ, q_0, F} if δ (q_0, w) = P for some P in F. This concludes that string is accepted when it enters into the final state on inputting the last element.

Example 2.3 Let us check if the input string 1010 is accepted or not: by FA shown in Figure 2.7.

$$\delta(q_0, 1010) = \delta(q_2, 010) = \delta(q_3, 10) = \delta(q_1, 0) = q_0$$

$$q_0 \xrightarrow{1} q_2 \xrightarrow{1} q_3 \xrightarrow{1} q_1 \xrightarrow{1} q_0$$

Here q_0 is the final state. Hence, the string is accepted.

b) Check 11111

$$q_0 \xrightarrow{1} q_2 \xrightarrow{1} q_0 \xrightarrow{1} q_2 \xrightarrow{1} q_0 \xrightarrow{1} q_2$$

$q_2 \notin$ F. Hence, this string is rejected.

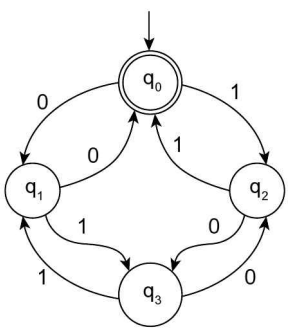


Fig. 2.7 *Finite Automaton*

Example 2.3 Give the language defined by the following FA: shown in Figure 2.8.

If we list the different strings accepted by this automata, we get

{1, 01, 0001, 10101, 011111.....}

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

We observe that all the strings accepted are ending with 1.

$$L(M) = \{w/w \text{ ends with } 1 \text{ on } \Sigma = \{0, 1\}\}.$$

Language accepted by machine M (denoted by L(M)) is given by the set of strings that are ending with 1.

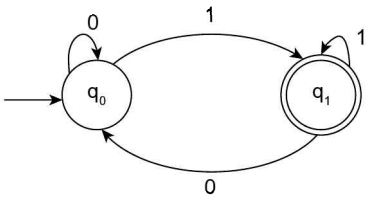


Fig. 2.8 *FA for Example 2.5*

Example 2.3 Identify the language defined by following machine: in Figure 2.9.

In this automaton, since the initial state and the final state are same, it accepts ε and all strings that end with 0. Hence the language accepted by this L(M) = {w/w is ε or all strings that end with 0}

or

L(M) = {w/w is ε or all strings that do not end with 1}

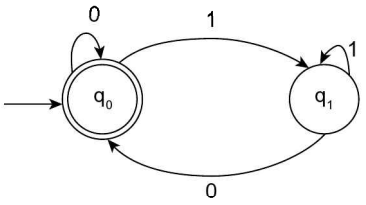


Fig. 2.9 *FA for Example 2.6*

Example 2.3 Identify the language defined by following machine:in Figure 2.10.

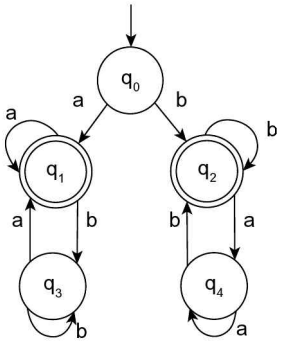


Fig. 2.10 *FA for Example 2.7*

Solution:
The strings valid in the language are

$$L = \{a, aa, aba, abba, aaa, \ldots\ldots$$
$$b, bb, bab, bbb, baab, \ldots\ldots\}$$

L(m) = {w/w contains all strings that start and end with same symbol}

2.3 Two Types of Finite Automata

(a) Deterministic finite automata (DFA)
(b) Non-deterministic finite automata (NFA)

In DFA, there will be a unique transition in any state on an input symbol. In NFA, there can be more than one transition on an input symbol. Hence, DFA is faster than NFA. The Figure 2.10 is an example of a DFA. The Figure 2.11 is an example of NFA.

In Figure 2.11 in state q_0 on 0, it is either in state q_0 or in state q_1. Hence it is a NFA.

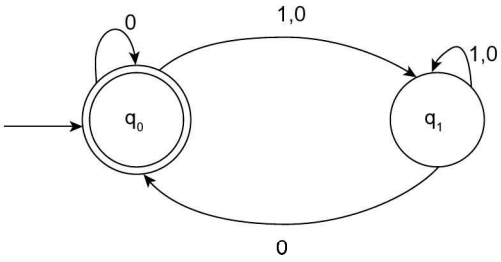


Fig. 2.11 *Non Deterministic Finite Automaton*

2.3.1 Deterministic Finite Automata (DFA)

Definition 2: Deterministic finite automata can be defined as quintuple

$M = (Q, \Sigma, \delta, q_0, F)$

where
Q = a non-empty finite set, of states
Σ = input alphabet
q_0 = initial start state
F = set of final states
δ = transition function that takes two arguments, a state and an input symbol, and returns output as state, that is, δ: $Q \times \Sigma \rightarrow Q$

Example: $\delta(q_1, a) = q_1$

DFA can be used as finite acceptor because its sole job is to accept certain input strings and reject others.

It is also called language recognizer because it merely recognizes whether the input strings are in the language or not.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example 2.8 Design a DFA which accepts only input 101 over the set {0, 1}

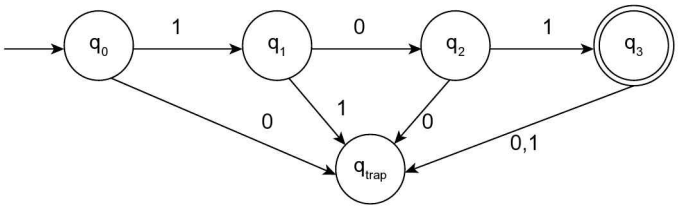


Fig. 2.12 *FA with Trap State*

Solution: Here q_{trap} is called trap state/dummy state where unnecessary transitions are thrown away (Figure 2.12).

Example 2.3 Design a DFA that accepts even number of 0's and even number of 1's.

Solution: This FA will have four different possibilities:

Even number of 0's and even number of 1's – q_0
Odd number of 0's and even number of 1's – q_1
Even number of 0's and odd number of 1's – q_2
Odd number of 0's and odd number of 1's – q_3

where states are q_0, q_1, q_2 and q_3. Since the state q_0 indicates the condition of even number of 0's and even number of 1's, this state is made the final state. The DFA is given by Figure 2.13.

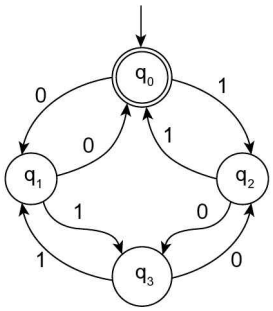


Fig. 2.13 *DFA for Example 2.9*

Example 2.10 Design a DFA that accepts all the strings with at most 3 a's.

Solution: Strings that have more than three a's should not be accepted. There are five possibilities. Hence, the DFA requires five states:

No a's – q_0 (Accepting state)
One a – q_1 (Accepting state)
Two a's – q_2 (Accepting state)
Three a's – q_3 (Accepting state)
Four a's – q_4 (for dummy state indicating a non-accepting state)

Deterministic finite automata can be constructed by expanding the listed conditions as shown in Figure 2.14.

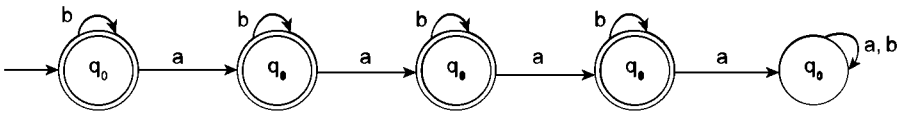


Fig. 2.14 *DFA for Example 2.10*

Example 2.11 Draw a DFA that recognizes the set of all strings starting with prefix ab with alphabets from $\Sigma = \{a. b\}$.

Solution: This automaton should accept strings that start with ab (and then followed by arbitrary elements). It should reject strings that start with b. Hence, if it sees b at the initial state, it should enter the dead state, from which there is no path to the final state. After a, it should find b. Hence, in the second state if it sees a, it should enter the dead state. Once it sees ab, it is the final state and remains in the same state on any element. Figure 2.15 shows the resultant DFA.

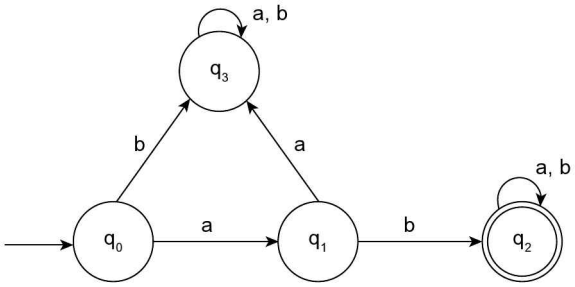


Fig. 2.15 *DFA for Example 2.11*

Example 2.10 Design a DFA that accepts even number of a's over $\Sigma = \{a\}$.

Solution: This automaton should accept strings that have even number of a's. On seeing a's which form an even count, the DFA should be in final state and on seeing odd count of a's, it should be in non-final state as shown in Figure 2.16.

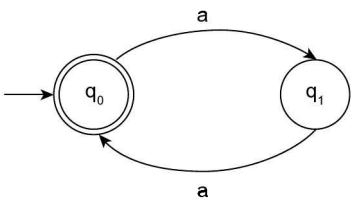


Fig. 2.16 *DFA for Example 2.12*

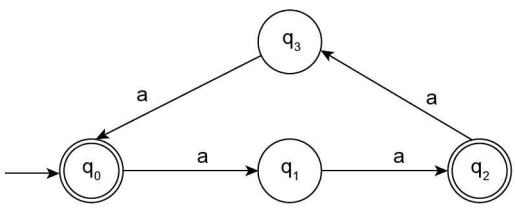


Fig. 2.17 *DFA for Example 2.12 With More States*

This DFA is not unique. The same machine can also be designed as in Figure 2.17.

Both DFAs represent the same language but with different number of states. It is always better to have less number of states as it simplifies the implementation.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example 2.11 DFA that accepts odd number of 1's on {0, 1}.

Solution: This automaton should accept strings that have odd number of 1's and any number of 0's. On seeing 1's that form an odd count, the DFA should be in the final state and, on seeing even count of 1's, it should be in non-final state as shown in Figure 2.18.

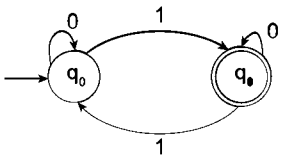


Fig. 2.18 *DFA for Example 2.13*

Example 2.10 Design a DFA that contains 001 as substring in all strings over $\Sigma = \{0, 1\}$.

This automaton should accept strings that have substring '001'. On seeing '0' go to next q_1 state, otherwise be in same state. In q_1 state, on seeing '0', go to next q_2 state; otherwise go to the initial state, as the substring would not be '00'. In q_2 state, on seeing '1', go to next q_3 state, otherwise be in same state, as it would satisfy the required condition that 1 should be preceded by '00'. Declare q_3 as final state, as the required total condition is satisfied and is shown in Figure 2.19.

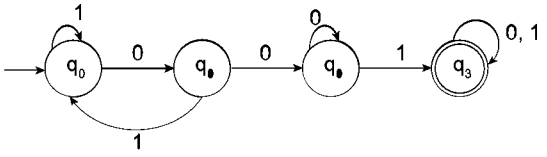


Fig. 2.19 *DFA for Example 2.14*

2.3.2 Non-deterministic Finite Automaton (NFA)

In this automaton, for a given input symbol, there can be more than one transition from a state. Such automaton is called Non-deterministic Finite Automaton. NFA is mathematically described as a quintuple.

Definition 3: Non-deterministic finite automata can be defined as quintuple

$M = (Q, \Sigma, \delta, q_0, F)$

where Q = Non-empty finite set of states

Σ = input alphabet which includes

q_0 = initial start state

F = set of final states

δ = transition function that takes two arguments (a state and an input symbol) and returns an output as state, that is, δ: $Q \times \Sigma \rightarrow 2^Q$

2.3.3 Acceptance of NFA

Acceptance of a string is defined as 'reaching final states on processing the input string' i.e. if there exits atleast one path from start state to final state, then the string is accepted by NFA.

Check if 0100 is accepted or not. by automata in Figure 2.20.

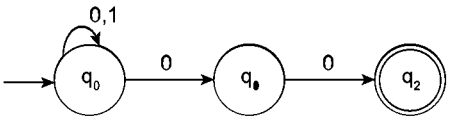
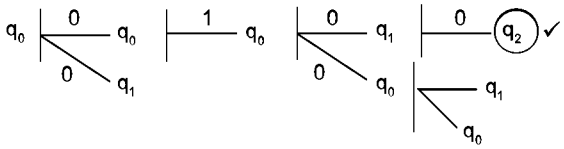


Fig. 2.20 *Simple NFA*

As there is a path from q_0 to final state q_n the string is accepted. Thus we check all possible paths from start state.



Note: It is easy to construct a NFA than a DFA. But for a given string, the processing time of NFA is greater than that of a DFA.

Example 2.11 Design NFA for $\{abab^n/n >= 0\}$

***Solution*:** Read the sequence a, b, a in state q_0 and q_1, q_2 and q_3 in sequence. Since b^n, $n \geq 0$ indicates zero or more no of b's in state q_3 add self loop in this state and declare it as final state and is shown in Figure 2.21.

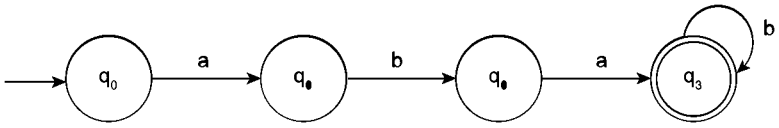


Fig. 2.21 *NFA for Example 2.15*

Example 2.10 Design NFA for the set of strings such that 5th symbol from right end is 1.

Solution: Since fifth symbol from right end is 1, we can have minimum 6 states.
The first transition should be on 1, remaining can be either 0 or 1 and is shown in Figure 2.22.

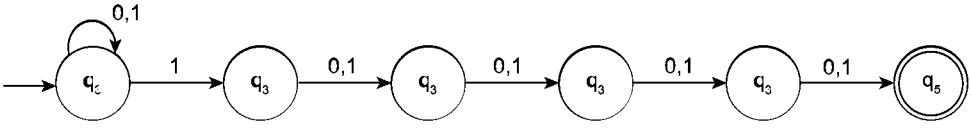


Fig. 2.22 *NFA for Example 2.16*

Example 2.11 Design a NFA and DFA accepting all the strings ending with 01 over $\Sigma = \{0, 1\}$

***Solution*:** We can have any string on 0 or 1 but it should end with 0 followed by 1. The corresponding NFA is shown in Figure 2.23.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

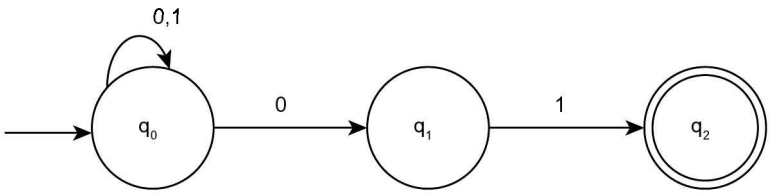


Fig. 2.23 *NFA for Example 2.17*

Corresponding DFA is shown in Figure 2.24.

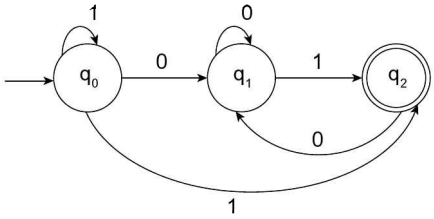


Fig. 2.24 *DFA for Example 2.17*

So drawing NFA is simpler than drawing DFA.

2.4 Equivalence of DFAs and NFAs

Theorem 1: Let L be a set accepted by a NFA. Then there exists a DFA that accepts L.

Proof: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a NFA accepting L.
Define DFA $M^1 = (Q^1, \Sigma^1, \delta^1, q_0^1, F^1)$ as follows.

The states of M^1 are all the subsets of the set of states of M, that is, $Q^1 = 2^Q$.

If $Q = \{A, B\}$ then $Q^1 = \{\phi, [A], [B], [AB]\}$. F^1 is the set of all states in Q^1 containing a final state of M. If $F = \{B\}$ then $F^1 = \{[B], [AB]\}$.

An element of Q^1 will be denoted by $[q_1, q_2, \ldots\ldots q_i]$ where $q_1, q_2, \ldots\ldots q_i$ are in Q.

Note: $[q_1, q_2, \ldots q_i]$ is a single state of DFA corresponding to the set of states of NFA.

$$q_0^1 = [q_0]$$

We define $\delta^1([q_1, q_2, \ldots q_i], a) = [P_1, P_2, \ldots\ldots P_i]$

$$\text{iff } \delta(\{q_1, q_2, \ldots q_i\}a) = \{P_1, P_2, \ldots P_i\},$$

that is, δ^1 applied to an element $[q_1, q_2, \ldots q_i]$ of Q^1 is computed by applying δ to each state of Q represented by $[q_1, q_2, \ldots q_i]$. It is easy to show by induction on length of the input string x that

$$\delta^1(q_0^1, x) = [q_1, q_2, \ldots q_i]$$

$$\text{iff } \delta(q_0, x) = \{q_1, q_2, \ldots q_i\}$$

Basis: The result is trivial for $|x| = 0$. As $\delta(q_0^1, \varepsilon) = [q_0] \Rightarrow q_0^1 = [q_0]$

Induction: Suppose that the hypothesis is true for inputs of length m or less. Let 'xa' be a string of length m + 1 with a in Σ. Then

$$\delta^1(q_0, xa) = \delta^1(\delta^1(q_0, x), a)$$

By inductive hypothesis

$$\delta^1(q_0, x) = [P_1, P_2, \ldots P_j]$$

$$\text{iff } \delta(q_0, x) = \{P_1, P_2, \ldots P_j\}$$

But by the definition of δ^1

$$\delta^1([P_1, P_2, \ldots P_j]a) = [r_1, r_2, \ldots r_k]$$

Thus

$$\delta^1(q_0^1, xa) = [r_1, r_2, \ldots r_k]$$

$$\text{iff } \delta(q_0, xa) = \{r_1, r_2, \ldots r_k\}$$

This establishes the inductive hypothesis.

Now we have to prove that the $L(M) = L(M^1)$.

The string x is accepted by NFA or DFA only if it is in one of the final states.

For a string x in NFA, let $\delta(q_0, x) = P$ where $P \in F$. Then $\delta^1(q_0, x) = [P]$ where $[P] \in F^1$, Hence, the string x is accepted if it is accepted by NFA.

Note

- Every language that can be described by NFA can be described by some DFA
- DFA in practice has more states than NFA
- DFA equivalent to NFA can have at most 2^n states whereas NFA has only n states.

2.5 Converting NFA (M_N) to DFA (M_D)— Subset Construction

Let $M_N = (Q_N, \Sigma_N, \delta_n, q_{ON}, F_N)$ be the given NFA to construct equivalent DFA M_D. Define M_D as follows.

(i) $Q_D = 2^{Q_N}$. If NFA has n states, DFA can have at most 2^n states.

(ii) $\Sigma_n = \Sigma_D$

(iii) $[q_0] = \{q_n\}$

(iv) F_D = Set of all states of Q_D that contains at least one of the final states of F_N.

(v) $\delta_D((q_1, q_2, q_3), a) = \delta_n(q_1, a) \cup \delta_n(q_2, a) \cup \delta_n(q_3, a)$

$= \{P_1, P_2, P_3\}$ say

Add the state $[P_1, P_2, P_3]$ to Q_D if it is not there.

Example 2.18 Convert the NFA shown in the Figure 2.25 to DFA.

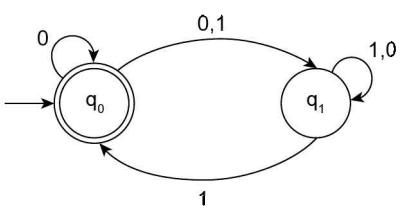


Fig. 2.25 *NFA*

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Solution:

Step 1: Find the possible set of states Q.

Then Q has 2^2 (= 4) states, and it is the set of all subsets of q_0, q_1:
$\{\emptyset, [q_0], [q_1], [q_0q_1]\}$

Step 2: Find the initial state.

$[q_0] = q_0$

Step 3: Define the transitions on 0, 1, on each state.

$\delta(q_0, 0) = [q_0, q_1]$
$\delta(q_0, 1) = [q_1]$
$\delta(q_1, 0) = [q_1]$
$\delta(q_1, 1) = [q_0, q_1]$
$\delta([q_0, q_1], 0) = \delta(q_0, 0) \cup \delta(q_1, 0) = [q_0, q_1]$
$\delta([q_0, q_1], 1) = \delta(q_0, 1) \cup \delta(q_1, 1) = [q_0, q_1]$
F = The set of states that contain q_0, called the final states in DFA.
$[q_0], [q_0, q_1]$

Step 4: Transition diagram of the DFA is shown in Figure 2.26. In the set of states, we have not used the state $\emptyset$ as it is not reached.

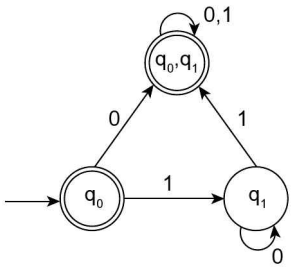


Fig. 2.26 *Equivalent DFA*

Example 2.10 Convert the NFA in Figure 2.27 to DFA.

Solution:

Q has 2^3 (= 8) states, and it is the set of all subsets of q_0, q_1, q_2:
$\{\emptyset, [q_0], [q_1], [q_2], [q_0, q_1], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$
$\Sigma = 0, 1$
$q_0 = [q_0]$
$F = \{[q_2], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$
δ is given by $\delta_D([q_1\ q_2], a) = \delta_n(q_1, a) \cup \delta_n(q_2, a)$

where δ_n is transition function of NFA

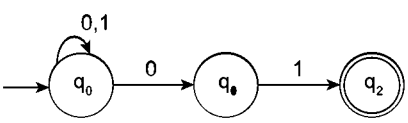


Fig. 2.27 *NFA*

δ	0	1
Ø	Ø	Ø
→[q_0]	[q_0, q_1]	[q_0]
[q_1]	Ø	[q_2]
[q_2]	Ø	Ø
[q_0, q_1]	[q_0, q_1]	[q_0, q_2]
*[q_0, q_2]	[q_0, q_1]	[q_0]
[q_1, q_2]	Ø	[q_2]
[q_0, q_1, q_2]	[q_0, q_1]	[q_0, q_2]

The states ϕ, [q_1], [q_2], [q_1, q_2] and [q_0, q_1, q_2] are not reachable from start state. Hence, they cannot define any string. So they can be thrown away. Hence the DFA can be simplified as in Figure 2.28.

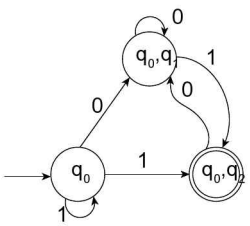


Fig. 2.28 *Equivalent DFA for Example 2.19*

To get this **simplified DFA,** construct the states of DFA as follows:

(i) Start with the initial state. Do not add all subsets of states as there may be unnecessary states.

(ii) After finding the transition on this initial state, include only the resultant states into the list until no new state is added to the list. For example, if $\delta(q_0, a) = \{q_0, q_1\}$ (say), then add this as a new state in DFA and find the transition from this state on the input symbol.

(iii) Declare the states as final if they have at least one final state of the NFA.

Example 2.20 Convert the following NFA to a DFA.

δ	0	1
→ q_0	{q_1 q_2}	{q_0}
q_1	{q_0 q_1}	Ø
*q_2	{q_1}	{q_0 q_1}

***Solution*:** DFA is

Q has 2^3 (= 8) states, and it is the set of all subsets of q_0, q_1, q_2:
{Ø, [q_0], [q_1], [q_2], [q_0, q_1], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]}

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$\Sigma = 0, 1$
$q_0 = [q_0]$
$F = \{[q_2], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$

δ	0	1
$\rightarrow [q_0]$	$[q_1\ q_2]$	$[q_0]$
$*[q_1\ q_2]$	$[q_0\ q_1]$	$[q_0\ q_1]$
$[q_0\ q_1]$	$[q_0\ q_1\ q_2]$	$[q_0]$
$*[q_0\ q_1\ q_2]$	$[q_0\ q_1\ q_2]$	$[q_0\ q_1]$

The transition diagram of DFA is as shown in Figure 2.29.

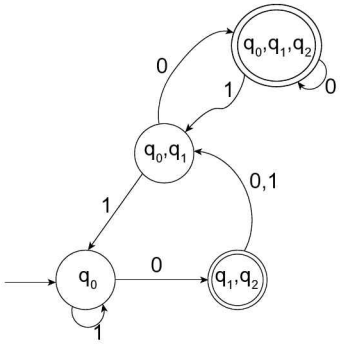


Fig. 2.29 *DFA for Example 2.20*

2.6 NFA with Epsilon- (ε) Transitions

We can extend a NFA by introducing 'ε-moves' that allow us to make a transition on the empty string. There would be an edge labelled ε between two states and this edge allows transition from one state to another even without receiving an input symbol. This is another mechanism that allows NFA to be in multiple states at once. Constructing such NFA is easy, but the NFA thus constructed is not that powerful. The NFA with ε-moves is given by M = (Q, Σ, δ, q_0, F) where δ is defined as Q X Σ ∪ {ε} → 2^Q.

Example 2.11 Figure 2.30 gives NFA with ε transitions and it accepts strings of the form $\{0^n1^m2^o/n, m, o \geq 0\}$, that is, any number of 0's followed by any number of 1's followed by any number of 2's.

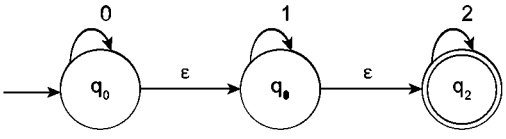


Fig. 2.30 *NFA with ε–Transitions*

Example 2.10 Design NFA for language L = $\{0^K \mid k \text{ is multiple of 2 or 3}\}$.

NFA for set of strings that have the characteristic that the number of 0's in them is a multiple of 2 is given in Figure 2.31.

NFA for set of strings that have the characteristic that the number of 0's in them is a multiple of 3 is given in Figure 2.32.

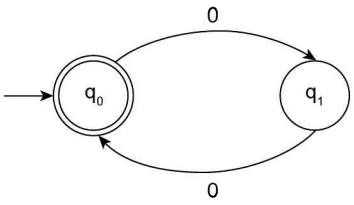


Fig. 2.31 *NFA for 0^{2k}*

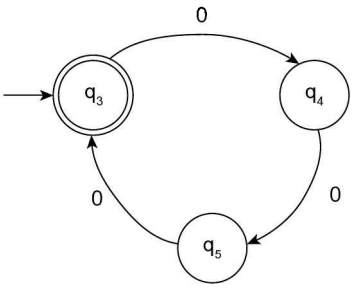


Fig. 2.32 *NFA for 0^{3k}*

Combining these two NFAs, we get Figure 2.33.

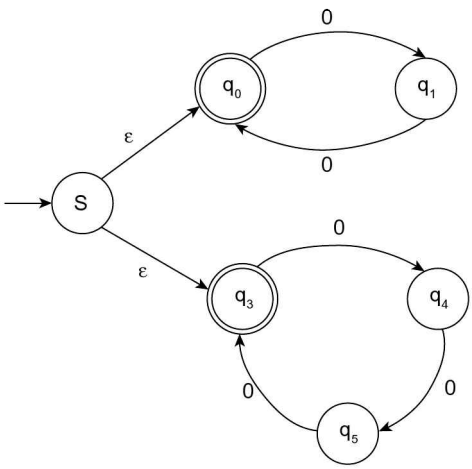


Fig. 2.33 *NFA for $0^{2k}/0^{3k}$*

2.6.1 Epsilon Closure (ε-closure)

Epsilon closure of a state is simply the set of all states that we can reach by ε input. This is denoted by either $\hat{\varepsilon}$ (q) or ε-closure (q). In Figure 2.30: the epsilon closure of q_0, q_1, q_2 states are

$$\varepsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$
$$\varepsilon\text{-closure}(q_1) = \{q_1, q_2\}$$
$$\varepsilon\text{-closure}(q_2) = \{q_2\}$$

Let us define the extended transition function for a NFA with ε transitions. For a regular NFA, for the induction step we defined

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$$\hat{\delta}(q, w) = \{p_1, p_2, \ldots p_k\}$$

$$\hat{\delta}(p_i, a) = S_i \text{ for } i = 1, 2, \ldots k$$

Then, we defined $\hat{\delta}(q, wa) = S_1 \cup S_2 \cup \ldots \cup S_k$.

For a NFA with ε, we change for the definition of $\hat{\delta}(q, wa)$ as follows:

$$\hat{\delta}(q, wa) = \cup\, \varepsilon\text{-closure}(S_1 \cup S_2 \cup \ldots \cup S_k)$$

This new definition includes the original sets $S_1, S_2 \ldots, S_k$ as well as the states we can reach via ε transitions from these states.

2.6.2 Eliminating ε-Transitions

ε-Transitions are used for convenience in some cases, but they do not increase the power of the NFA. To eliminate them, we can convert a NFA with ε into an equivalent NFA without ε, by eliminating the ε edges and replacing them with the edges labelled with symbols present in Σ. We can also convert NFA with ε into an equivalent DFA, by steps quite similar to those we took for converting a normal NFA to a DFA, except that we must now follow all ε-transitions and add those to our set of states.

2.6.3 Converting NFA with ε-Transition to NFA without ε-Transition

For each state compute ε-closure(q) on each input symbol a ∈ Σ. If the ε-closure of a state contains a final state then make the state final.

Let us consider the NFA with ε-transitions depicted in Figure 2.34.

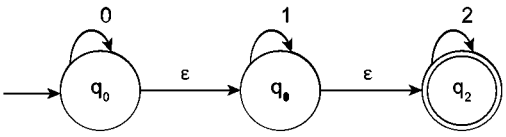


Fig. 2.34 *NFA with t*

The transition table is

	a = 0	a = 1	a = 2	a = ε
$\rightarrow q_0$	q_0	Ø	Ø	q_1
q_1	Ø	q_1	Ø	q_2
$*q_2$	Ø	Ø	q_2	Ø

Step 1: Find ε-closure of each state.

ε-closure $(q_0) = \{q_0, q_1, q_2\}$
ε-closure $(q_1) = \{q_1, q_2\}$
ε-closure $(q_2) = \{q_2\}$

Step 2: Find the transition on each state for each element.

$\hat{\delta}\,(q_0, 0) = \varepsilon\text{-closure}\,(\delta(\varepsilon\text{-closure}\,(q_0), 0))$

$= \varepsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 0))$
$= \varepsilon\text{-closure}(\{q_0\}, \cup\{\emptyset\}, \cup\{\emptyset\})$
$= \{q_0, q_1, q_2\}$

$\hat{\delta}(q_0, 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 1))$
$= \varepsilon\text{-closure}(\{\emptyset\}, \cup\{q_1\}, \cup\{\emptyset\})$
$= \{q_1, q_2\}$

$\hat{\delta}(q_0, 2) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), 2))$
$= \varepsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 2))$
$= \varepsilon\text{-closure}(\{\emptyset\}, \cup\{\emptyset\}, \cup\{q_2\})$
$= \{q_2\}$

$\hat{\delta}(q_1, 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1), 0))$
$= \varepsilon\text{-closure}(\delta(\{q_1, q_2\}, 0))$
$= \varepsilon\text{-closure}(\{\emptyset\})$
$= \{\emptyset\}$

$\hat{\delta}(q_1, 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_1, q_2\}, 1))$
$= \varepsilon\text{-closure}(\{q_1\}, \cup\{\emptyset\})$
$= \{q_1, q_2\}$

$\hat{\delta}(q_1, 2) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1), 2))$
$= \varepsilon\text{-closure}(\delta(\{q_1, q_2\}, 2))$
$= \varepsilon\text{-closure}(\{\emptyset\}, \cup\{q_2\})$
$= \{q_2\}$

$\hat{\delta}(q_2, 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_2), 0))$
$= \varepsilon\text{-closure}(\delta(\{q_2\}, 0))$
$= \varepsilon\text{-closure}(\{\emptyset\})$
$= \{\emptyset\}$

$\hat{\delta}(q_2, 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_2), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_2\}, 1))$
$= \varepsilon\text{-closure}(\{\emptyset\})$
$= \{\emptyset\}$

$\hat{\delta}(q_2, 2) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_2), 2))$
$= \varepsilon\text{-closure}(\delta(\{q_2\}, 2))$
$= \varepsilon\text{-closure}(\{q_2\})$
$= \{q_2\}$

NFA without ε-transitions is

	a = 0	a = 1	a = 2
$\rightarrow *q_0$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
$*q_1$	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$	$\{q_2\}$

Transition diagram of this NFA without ε-transitions is given in Figure 2.35.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

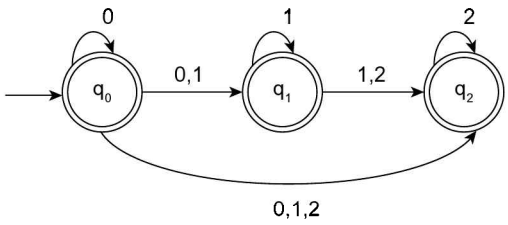


Fig. 2.35 *NFA without* ε

2.6.4 Converting NFA with ε-transition to DFA

1. Compute ε^* for the current state, and this results in a set of states S.
2. $\delta(S, a)$ is computed for all $a \in \Sigma$ by the following steps:
   a. Let $S = \{p_1, p_2, \ldots p_k\}$
   b. Compute $R = \delta(S, a)$ by

$$R = \bigcup_{p_i \in S} \delta(p_i, a) = \{r_1, r_2, r_3 \ldots r_m\}$$

   This set is achieved by following input a, not by following any ε-transition.
   c. Add the ε-transitions by computing ε-closure(R).
3. Make a state an accepting state if it includes any final state in the NFA.

Note: The ε (epsilon)-transition refers to a transition from one state to another without reading any input symbol (i.e. without the tape containing the input string moving). Epsilon transitions can be inserted between any two states.

Consider the NFA- ε move machine $M = \{Q, \Sigma, \delta, q_0, F\}$.

$Q = \{q_0, q_1, q_2\}$
$\Sigma = \{a, b, c\}$ and ε moves

	a = 0	a = 1	a = 2	a = ε
$\rightarrow q_0$	q_0	Ø	Ø	q_1
q_1	Ø	q_1	Ø	q_2
$*q_2$	Ø	Ø	q_2	Ø

DFA construction

Step 1: Q has 2^3 (= 8) states, and it is the set of all subsets of all subsets of q_0, q_1, q_2:
$\{Ø, [q_0], [q_1], [q_2], [q_0, q_1], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$
$\Sigma = 0, 1, 2$
$q_0 = \hat{\varepsilon}\ (q_0)$
$F = \{[q_2], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$

Step 2: Compute ε-closure of each state.

$\hat{\varepsilon}\ (q_0) = \{q_0, q_1, q_2\}$
$\hat{\varepsilon}\ (q_1) = \{q_1, q_2\}$
$\hat{\varepsilon}\ (q_2) = \{q_2\}$

Step 3: Find the initial state by finding the ε-closure of the initial state.

$$q_0 = \hat{\varepsilon}\ (q_0) = [q_0, q_1, q_2]$$

Step 4: Explore the states that are valid states in DFA starting at the new initial state. Explore each new state by finding the transitions on every input element:

$$\begin{aligned}\hat{\delta}([q_0, q_1, q_2], 0) &= \varepsilon\text{-closure}(\delta([q_0, q_1, q_2], 0))\\ &= \varepsilon\text{-closure}(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0))\\ &= \varepsilon\text{-closure}(\{q_0\} \cup \{\emptyset\} \cup \{\emptyset\})\\ &= [q_0, q_1, q_2]\end{aligned}$$

$$\begin{aligned}\hat{\delta}([q_0, q_1, q_2], 1) &= \varepsilon\text{-closure}(\delta([q_0, q_1, q_2], 1))\\ &= \varepsilon\text{-closure}(\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1))\\ &= \varepsilon\text{-closure}(\{\emptyset\} \cup \{q_1\} \cup \{\emptyset\})\\ &= [q_1, q_2]\end{aligned}$$

$$\begin{aligned}\hat{\delta}([q_0, q_1, q_2], 2) &= \varepsilon\text{-closure}(\delta([q_0, q_1, q_2], 2))\\ &= \varepsilon\text{-closure}(\delta(q_0, 2) \cup \delta(q_1, 2) \cup \delta(q_2, 2))\\ &= \varepsilon\text{-closure}(\{\emptyset\} \cup \{\emptyset\} \cup \{q_2\})\\ &= [q_2]\end{aligned}$$

The next new state is $[q_1, q_2]$ and the transitions for the new states are

$$\begin{aligned}\hat{\delta}([q_1, q_2], 0) &= \varepsilon\text{-closure}(\delta([q_1, q_2], 0))\\ &= \varepsilon\text{-closure}(\delta(q_1, 0) \cup \delta(q_2, 0))\\ &= \varepsilon\text{-closure}(\{\emptyset\} \cup \{\emptyset\})\\ &= [\emptyset]\end{aligned}$$

$$\begin{aligned}\hat{\delta}([q_1, q_2], 1) &= \varepsilon\text{-closure}(\delta([q_1, q_2], 1))\\ &= \varepsilon\text{-closure}(\delta(q_1, 1) \cup \delta(q_2, 1))\\ &= \varepsilon\text{-closure}(\{q_1\} \cup \{\emptyset\})\\ &= [q_1, q_2]\end{aligned}$$

$$\begin{aligned}\hat{\delta}([q_1, q_2], 2) &= \varepsilon\text{-closure}(\delta([q_1, q_2], 2))\\ &= \varepsilon\text{-closure}(\delta(q_1, 2) \cup \delta(q_2, 2))\\ &= \varepsilon\text{-closure}(\{\emptyset\} \cup \{q_2\})\\ &= [q_2]\end{aligned}$$

The third new state is $[q_2]$.

$$\begin{aligned}\hat{\delta}([q_2], 0) &= \varepsilon\text{-closure}(\delta([q_2], 0))\\ &= \varepsilon\text{-closure}(\delta(q_2\ 0))\\ &= \varepsilon\text{-closure}(\{\emptyset\})\\ &= [\emptyset]\end{aligned}$$

$$\begin{aligned}\hat{\delta}([q_2], 1) &= \varepsilon\text{-closure}(\delta([q_2], 1))\\ &= \varepsilon\text{-closure}(\delta(q_2\ 1))\\ &= \varepsilon\text{-closure}(\{\emptyset\})\\ &= [\emptyset]\end{aligned}$$

$$\begin{aligned}\hat{\delta}([q_2], 2) &= \varepsilon\text{-closure}(\delta([q_2], 2))\\ &= \varepsilon\text{-closure}(\delta(q_2\ 2))\\ &= \varepsilon\text{-closure}(\{q_2\}) = [q_2]\end{aligned}$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

The last new state is [Ø] which is a dummy state, and the transitions are

Note: $\hat{\varepsilon}(\emptyset) = \{\emptyset\}$

$$\hat{\delta}([\emptyset], 0) = \varepsilon\text{-closure}(\delta([\emptyset], 0)) = \varepsilon\text{-closure}(\{\emptyset\}) = [\emptyset]$$

$$\hat{\delta}([\emptyset], 1) = \varepsilon\text{-closure}(\delta([\emptyset], 1)) = \varepsilon\text{-closure}(\{\emptyset\}) = [\emptyset]$$

$$\hat{\delta}([\emptyset], 2) = \varepsilon\text{-closure}(\delta([\emptyset], 2)) = \varepsilon\text{-closure}(\{\emptyset\}) = [\emptyset]$$

DFA transition table

	a = 0	a = 1	a = 2
$\rightarrow\varepsilon^*(q_0)$ $=^* [q_0 q_1 q_2]$	$[q_0 q_1 q_2]$	$[q_1 q_2]$	$[q_2]$
$^*[q_1 q_2]$	[Ø]	$[q_1 q_2]$	$[q_1 q_2]$
$^*[q_2]$	[Ø]	[Ø]	$[q_2]$
[Ø]	[Ø]	[Ø]	[Ø]

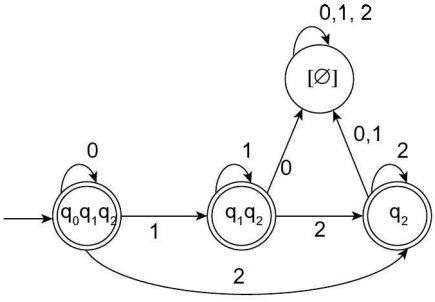


Fig. 2.36 *DFA*

2.7 Comparison Method for Testing Equivalence of Two FAs

Let M and M^1 be two FAs over Σ. We construct a comparison table consisting of n + 1 columns where n is the number of input symbols.

1. 1st column consists of pair of nodes of the form (q, q^1) where q ∈ M and q^1 ∈ M^1.
2. If (q, q^1) appears in some row of 1st column, then the corresponding entry in the column of a (a ∈ Σ) is (q_a, q_a^1) where (q_a, q_a^1) are reachable from q and q^1 on a.
3. Table is constructed by starting with a pair of initial vertices q_{in} and q_{in}^1 of M and M^1. We complete the construction by considering the pairs that are not in 1st column but are in the 2nd and subsequent columns.

i) If we reach a pair (q, q[1]) such that q is in the set of final states of M and q[1] is non-final state of M[1] then terminate the construction and conclude that M and M[1] are not equivalent.

ii) If the construction is terminated when no new element that are not the 1st column appears in the 2nd and subsequent columns, conclude that M and M[1] are equivalent. To show the equivalence, let us consider the two DFA's as shown in the Figures 2.37 and 2.38. Start at the initial state $[q_0, q_4]$.

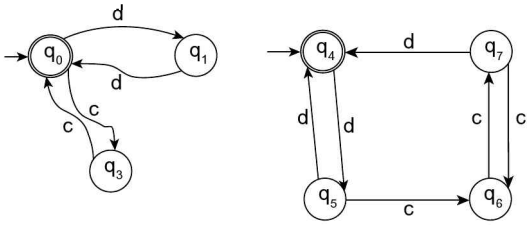


Fig. 2.37 *Finite Automata for Comparision*

By looking at the number of states, we cannot conclude whether they are equivalent or not.

$[q_0, q_4]$	$[q_3, \emptyset]$	$[q_1, q_5]$

Since we do not have a pair $(q_{M1}q_{M2})$ for input c, M_1 and M_2 are not equivalent.

2.8 Reduction of Number of States in FA

- Any DFA defines a unique language, but the converse of this is not true, that is, given a language, we cannot claim that there is a unique DFA associated.
- For one language, there can be many DFAs. So the number of states can consierably differ. By using comparison method we test this.

2.8.1 Indistinguishable States

Two states p and q of a DFA are indistinguishable if

$$\delta^*(p, w) \in F \text{ implies } \delta^*(q, w) \in F$$

and

$$\delta^*(p, w) \notin F \text{ implies } \delta^*(q, w) \notin F$$

Or for some string $w \in \Sigma^*$ if $\delta^*(p, w) \in F$ and
$\delta^*(q, w) \notin F$ (or vice versa), states p and q are said to be distinguishable by a string w.

2.8.2 Equivalent Classes

The concept of equivalent class is used in minimizing the number of states. States that are equivalent can be combined together into one class called equivalent class. Let us see the definition of equivalent states.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Definition 4: Two states q_1 and q_2 are equivalent (denoted by $q_1 \equiv q_2$) if both $\delta(q_1, a)$ and $\delta(q_2, a)$ are final states or both of them are non-final states for all $a \in \Sigma$. These states are said to be 0-equivalent.

Definition 5: Two states q_1 and q_2 are K-equivalent (denoted $K \geq O$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states, or both are non-final states for all string x's of length K or less.

Therefore, any two final states are K-equivalent if they belong to the same set in K − 1 step, otherwise they are not K-equivalent.

Properties:

P_1: The relation between q_1 and q_2 is an equivalent relation (i.e. K-equivalence relation) i.e it is reflexive, symmetric and transitive.

P_2: Every equivalence relation partitions set is also K-equivalence relation partition set Q.

P_3: If q_1 and q_2 are K-equivalent for all $K \geq 0$, then they are equivalent.

P_4: If q_1 and q_2 are (K + 1)-equivalent then they are K-equivalent.

2.8.3 Minimization of DFA

For any deterministic automaton, that has more number of states, we can construct its equivalent DFA with a less number of states. we can minimize the states of DFA as follows.

Construction of Minimum Automaton

1. Initially construct 0-equivalence class by $\prod_0 = \{Q_1^0, Q_2^0\}$ where Q_1^0 is the set of final states and $Q_2^0 = Q - Q_1^0$ is the set of non-final states.
2. Construct $\prod_{K+1}$ from $\prod_K$ by partitioning further:
   a. Let Q_1^K be any subset in $\prod_K$. If q_1 and q_2 are in Q_1^K they are (K + 1)-equivalent provided $\delta(q_1, a)$ and $\delta(q_2, a)$ are K-equivalent.
   b. Find out whether $\delta(q_1, a)$ and $\delta(q_2, a)$ are in the same equivalence class in $\prod_K$ for every $a \in \Sigma$. If so, q_1 and q_2 are (k + 1)-equivalent. This way Q_i^k is further divided into (K + 1)-equivalence classes. Repeat this for every Q_i^k in $\prod_K$ to get all the elements of $\prod_{K+1}$.
3. Construct $\prod_n$ for n = 1, 2, 3, ... until $\prod_n = \prod_{n+1}$.
4. Construct the minimum state DFA with the states obtained by equivalent classes π_n.

First approach:

Example 2.23 Find minimum finite-state automata for the DFA shown in Figure 2.38.

	0	1
→a	b	f
b	g	c
*c	a	c
e	h	f
f	c	g
g	g	e
h	g	c

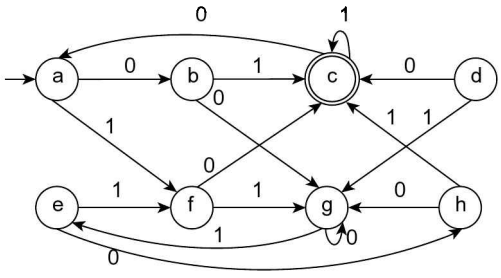


Fig. 2.38 *DFA*

Any two final states are 0-equivalent, and any two non-final states are also 0-equivalent.

$$\Pi_0 (1, 2) = \{\{c\}, \{a, b, d, e, f, g, h\}\}$$

	a	b	d	e	f	g	h
0	2	2	1	2	1	2	2
1	2	1	2	2	2	2	1

From the above table, we find a, e, g are 1-equivalent, b, h are 1 equivalent and d, f are 1-equivalent. Hence,

$\Pi_1 (1, 3, 4, 5) = \{\{c\}, \{a, e, g\}, \{b, h\}, \{d, f\}\}$

Using the new classes, we find whether they are 2-equivalent.

	a	b	d	e	f	g	h
0	4	3	1	4	1	3	3
1	5	1	3	5	3	3	1

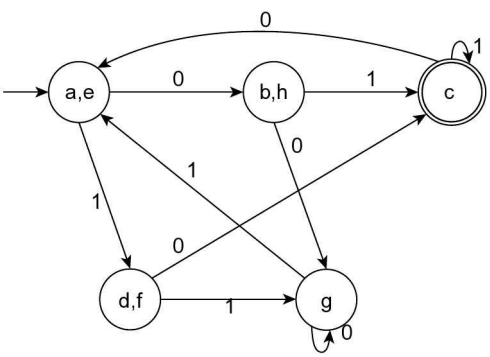
$\Pi_2 (1, 6, 7, 8, 9) = \{\{c\},\{a, e\}, \{b, h\}, \{d, f\}, \{g\}\}$

						g	

$\Pi_3 (1, 6, 7, 8, 9) = \{\{c\},\{a, e\}, \{b, h\}, \{d, f\}, \{g\}\}$

	a	b	d	e	f	g	h
0	7	9	1	7	1	9	9
1	8	1	9	8	9	6	1

From the above two relations, Π_2 and Π_3 are same. Hence, the final set of states are the sets 1, 6, 7, 8, 9 where {a, e}, {b, h}, {d, f}, {g}, {c} are all 3-equivalent. The minimized DFA is depicted in Figure 2.39.



	0	1
→{a, e}	{b, h}	{d, f}
{b, h}	{g}	{c}
*{c}	{a, e}	{c}
{d,f}	{c}	{g}
{g}	{g}	{a, e}

Fig. 2.39 *Minimum State DFA*

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Second approach:

Example 2.24 Construct the minimum DFA for the given DFA shown in Figure 2.40.

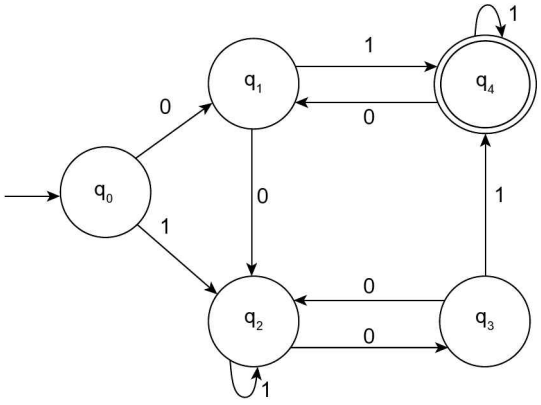


Fig. 2.40 *DFA*

Solution: Let us represent the DFA as a transition table.

	a = 0	a = 1
$\rightarrow q_0$	q_1	q_2
q_1	q_2	q_4
q_2	q_3	q_2
q_3	q_2	q_4
$*q_4$	q_1	q_4

a) Initially we identify 0-equivalence: $\prod_0 = \{Q_1^0, Q_2^0\}$ where Q_1^0 is the set of final states and $Q_2^0 = Q - Q_1^0$ is set of non-final states.

$Q_1^0 = \{q_4\}$
$Q_2^0 = \{q_0, q_1, q_2, q_3\}$

b) Construct $\prod_1$ from $\prod_0$ identifying the equivalent states in $\{Q_1^0, Q_2^0\}$.
Q_1^0 cannot be divided as it has only one state.
Q_2^0 has four states. We need to identify whether they are 1-equivalent.
Compare q_0, q_1 on input 0 and 1

$$\delta(q_0, 0) = q_1$$

$$\delta(q_1, 0) = q_2$$

Both of these resultant states belong to Q_2^0.

$$\delta(q_0, 1) = q_2$$

$$\delta(q_1, 1) = q_4$$

These resultant states belong to different sets in $\prod_0$

$\Rightarrow q_0$ is not 1-equivalent to q_1

Compare q_0, q_2 on input 0 and 1

$$\left.\begin{aligned}\delta(q_0, 0) &= q_1\\ \delta(q_2, 0) &= q_3\end{aligned}\right\}$$ both resultant states belong to Q_2^0.

$$\left.\begin{aligned}\delta(q_0, 1) &= q_2\\ \delta(q_2, 1) &= q_2\end{aligned}\right\}$$ both resultant states belong to Q_2^0.

$\Rightarrow q_0$ is 1-equivalent to q_2

Compare q_0, q_3 on input 0 and 1

$$\left.\begin{aligned}\delta(q_0, 0) &= q_1\\ \delta(q_3, 0) &= q_2\end{aligned}\right\}$$ both resultant states belong to Q_2^0.

$$\left.\begin{aligned}\delta(q_0, 1) &= q_2\\ \delta(q_3, 1) &= q_4\end{aligned}\right\}$$ both resultant states belong to different sets in $\prod_0$

q_0 is not 1-equivalent to q_3

Therefore,

$$\prod_1 = \{Q_1^1, Q_2^1, Q_3^1\}$$

where

$$Q_1^1 = \{q_4\}$$
$$Q_2^1 = \{q_0, q_2\}$$
$$Q_3^1 = \{q_1, q_3\}$$

c) Construct $\prod_2$ from $\prod_1$ identifying the equivalent states in $\{Q_1^1, Q_2^1, Q_3^1\}$.

Q_1^1 cannot be divided as it has only one state.

Q_2^1 and Q_3^1 has two states each, we need to identify whether they are equivalent.

Compare q_0, q_2 on input 0 and 1

$$\left.\begin{aligned}\delta(q_0, 0) &= q_1\\ \delta(q_2, 0) &= q_3\end{aligned}\right\}$$ both resultant states are same and belong to Q_3^1.

$$\left.\begin{aligned}\delta(q_0, 1) &= q_2\\ \delta(q_2, 1) &= q_2\end{aligned}\right\}$$ both resultant states are same and belong to Q_2^1.

$\Rightarrow q_0$ is 2- equivalent to q_2

Compare q_1, q_3 on input 0 and 1

$$\left.\begin{aligned}\delta(q_1, 0) &= q_2\\ \delta(q_3, 0) &= q_2\end{aligned}\right\}$$ both resultant states are same and belong to Q_2^1.

$$\left.\begin{aligned}\delta(q_1, 1) &= q_4\\ \delta(q_3, 1) &= q_4\end{aligned}\right\}$$ both resultant states are same and belong to Q_1^1.

$\Rightarrow q_1$ is 2- equivalent to q_3

Therefore,

$$\prod_2 = \{Q_1^2, Q_2^2, Q_3^2\}$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

where

$Q_1^2 = \{q_4\}$
$Q_2^2 = \{q_0, q_2\}$
$Q_3^2 = \{q_1, q_3\}$

d) We see that $\prod_2$ is equal to $\prod_1$. The states q_0 and q_2 are considered a single state, denoted q_{02}, and q_1 and q_3 are considered a single state, denoted q_{13}. Minimized DFA is

	a = 0	a = 1
$\rightarrow q_{02}$	q_{13}	q_{02}
q_{13}	q_{02}	q_4
$*q_4$	q_{13}	q_4

2.8.4 Minimization of DFA Using Myhill Nerode Theorem

Myhill Nerode theorem is used if we have to prove the given language is not regular and also to eliminate useless states in the given DFA.

Myhill Nerode Theorem:

- The language L accepted by a DFA is regular if and only if the number of equivalence classes of R_L is finite.
- The number of states in the smallest automaton accepting L is equal to the number equivalence classes in R_L. Therefore, R_L is of finite index.

Let ≡ (equivalence) on the states in M be such that

$p \equiv q$ if and only if, for each input string x, $\delta(p, x) = \delta(q, x) = q_a$

where q_a is accepting state, then we say

$\Rightarrow$ p is equivalent to q or P is not distinguishable from q.

If $\delta(p, x) = q_a$ and $\delta(q, x) = q_n$ for some $q_a \in F$ and $q_n \notin F$ then we say

$\Rightarrow$ p is distinguishable from q.

Algorithm for Finding Distinguishable States

- For each pair [p, q] where $p \in F$ and $q \in \{Q - F\}$, mark (p, q) = X
- For each pair of distinct states [p, q] in F X F or (Q – F) X (Q – F) do
  - If for some input symbol a $\delta([p, q], a) = [r, s]$, if [r, s] = X then
  - Mark [p, q] = X
  - Recursively mark all unmarked pairs which lead to [p, q] on input for all $a \in \Sigma$.
  - Else
  - for all input symbols a do

    put [p, q] on the list for $\delta([p, q], a)$ unless $\delta([p, q], a) = [r, r]$

Each unmarked pair [p, q] indicates the equivalent states.

Example 2.24 Find minimum-state automaton equivalent to the transition diagram given in Figure 2.41.

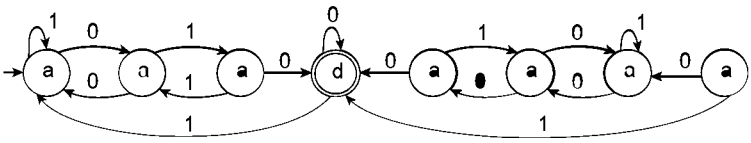


Fig. 2.41 *DFA*

Solution: The distinguishable states are marked with symbol X. The relation among the states is represented as a matrix of size n × n. If p is distinguishable from q then q is also distinguishable to p and therefore, it is sufficient to have a lower matrix to represent the relation of one state to all other states.

Step 1: First mark for all states [p, q] where state p is final and q is non-final.

[d, a] = X [d, b] = X [d, c] = X [d, e] = X

[d, f] = X [d, g] = X [d, h] = X

Step 2: Find the states that are distinguishable from state a.

δ([a, b], 0) = [b, a] δ([a, b], 1) = [a, c]

δ([a, c], 0) = [b, d] δ([a, c], 1) = [a, b] ⇒mark[a, c] = X as [b, d] = X

⇒mark[a, b] = X as [a, c] = X

δ([a, e], 0) = [b, d] δ([a, e], 1) = [a, f] ⇒mark[a, e] = X as [b, d] = X

δ([a, f], 0) = [b, g] δ([a, f], 1) = [a, e] ⇒mark[a, f] = X as [a, e] = X

δ([a, g], 0) = [b, f] δ([a, g], 1) = [a, g]

δ([a, h], 0) = [b, g] δ([a, h], 1) = [a, d] ⇒mark[a, h] = X as [a, d] = X

Find the states that are distinguishable from state b

δ([b, c], 0) = [a, d] δ([b, c], 1) = [c, b] ⇒mark[b, c] = X as [a, d] = X

δ([b, e], 0) = [a, d] δ([b, e], 1) = [c, f] ⇒mark[b, e] = X as [a, d] = X

δ([b, f], 0) = [a, g] δ([b, f], 1) = [c, e]

δ([b, g], 0) = [a, f] δ([b, g], 1) = [c, g] ⇒mark[b, g] = X as [a, f] = X

δ([b, h], 0) = [a, g] δ([b, h], 1) = [c, d] ⇒mark[b, h] = X as [c, d] = X

Find the states that are distinguishable from state c

δ([c, e], 0) = [d, d] δ([c, e], 1) = [b, f]

δ([c, f], 0) = [d, g] δ([c, f], 1) = [b, e] ⇒mark[c, f] = X as [d, g] = X

δ([c, g], 0) = [d, f] δ([c, g], 1) = [b, g] ⇒mark[c, g] = X as [d, f] = X

δ([c, h], 0) = [d, g] δ([c, h], 1) = [b, d] ⇒mark[c, h] = X as [d, g] = X

Find the states that are distinguishable from state e

δ([e, f], 0) = [d, g] δ([e, f], 1) = [f, e] ⇒mark[e, f] = X as [d, g] = X

δ([e, g], 0) = [d, f] δ([e, g], 1) = [f, g] ⇒mark[e, g] = X as [d, f] = X

δ([e, h], 0) = [d, g] δ([e, h], 1) = [f, d] ⇒mark[e, h] = X as [d, g] = X

Find the states that are distinguishable from state f

δ([f, g], 0) = [g, f] δ([f, g], 1) = [e, g] ⇒mark[f, g] = X as [e, g] = X

δ([f, h], 0) = [g, g] δ([f, h], 1) = [e, d] ⇒mark[f, h] = X as [e, d] = X

Find the states that are distinguishable from state g

δ([g, h], 0) = [f, g] δ([g, h], 1) = [g, d] ⇒mark[g, h] = X as [g, d] = X

Figure 2.42 shows the lower triangle with the entries computed.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

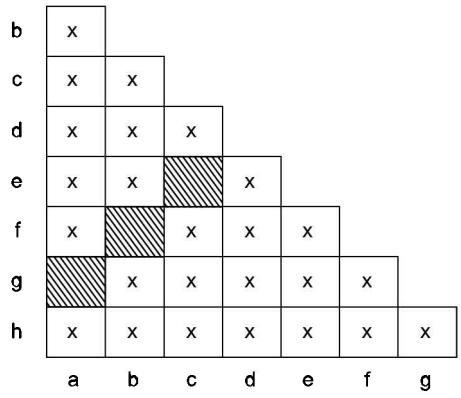


Fig. 2.42

From the lower triangle of the matrix it is clear that states [a, g], [b, f] and [c, e] belong to same class. These states can be merged, and the minimized DFA is

	a = 0	a = 1
→[a,g]	[b,f]	[a,g]
[b,f]	[a,g]	[c,e]
[c,e]	[d]	[b,f]
*[d]	[d]	[a,g]
[h]	[a,g]	[d]

2.9 Finite Automata with Output

The finite automaton designed so far gives outputs depicting whether the given strings are accepted or not. We can say that the output has only two possibilities, that is 'yes or no', But if it is required to have more than two outputs, we can associate the output with either state or transition. If the output is associated with state, the machine is called Moore machine (Figure 2.43(a)), and if the output is associated with transition, the machine is called as Mealy machine (Figure 2.43(b)).

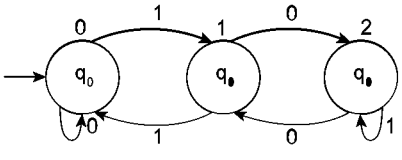


Fig. 2.43(a) *Moore Machine*

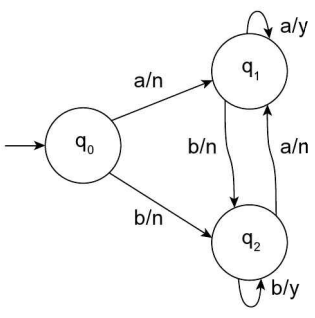


Fig. 2.43(b) *Mealy Machine*

The Moore machine machine shown in Figure 2.43(a) gives the output as **integer mod 3** for the input represented in binary form. For example, 5 in binary is 101, and the sequence taken on this input is q_0, q_1, q_2, q_2. Since it finally is in q_2 state, the output generated at the end is 2, which is the output corresponding to q_2 state.

The Mealy Machine in Figure 2.43(b) is designed to generate the output as '***y***' if current input matches with the previous input, and to output '***n***' if the input does not match for the input symbols on {a|b}*. For input abb, the final output would be '***y***', and for input aba, the output would be '***n***'.

Note: Moore machine generates output for ε, and Mealy machine generates output only on some input symbol.

2.9.1 Moore Machine

Definition 6: A Moore machine is represented as a six-tuple $M = \{Q, \Sigma, \Delta, \delta, \lambda, q_0\}$ where

Q – Set of finite states.

Σ – Set of finite input symbols.

Δ – Set of finite output symbols.

δ – is a mapping function from $Q \times \Sigma$ to Q (i.e. $Q \times \Sigma \rightarrow Q$).

λ – is a mapping function which maps Q to Δ (i.e. $Q \rightarrow \Delta$).

q_0 – is the initial state.

The state is associated with output. Whenever the Moore machine enters any state, it generates an output associated with that state. If the outputs are only two, then it can be viewed as a special case of finite automata.

2.9.2 Mealy Machine

Definition 7: A Mealy machine is represented as a six-tuple $M = \{Q, \Sigma, \Delta, \delta, \lambda, q_0\}$ where

Q – Set of finite states.

Σ – Set of finite input symbols.

Δ – Set of finite output symbols.

δ – is a mapping function from $Q \times \Sigma$ to Q (i.e. $Q \times \Sigma \rightarrow Q$).

λ – is a mapping function which maps $Q \times \Sigma$ to Δ (i.e. $Q \times \Sigma \rightarrow \Delta$).

q_0 – is the initial state.

The transition is associated with output. Whenever this machine enters any state on a particular input, it generates output.

Example 2.23 Design a Moore machine for inputs that are integer strings represented as binary, and give the output 0 for multiples of 2 and 1 for others.

Solution: Integer numbers, represented as binary, and their corresponding outputs are given below.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Integer number	Binary representation	Output
1	1	1
2	10	0
3	11	1
4	100	0
5	101	1
6	110	0
7	111	1

Let three states be defined by {I, q_0, q_1} where I is the initial state, q_0 is a state associated with the output 0 and q_1 is a state associated with 1. Add path in the automaton such that it enters the corresponding output state on the input sequence and the machine is shown Figure 2.44.

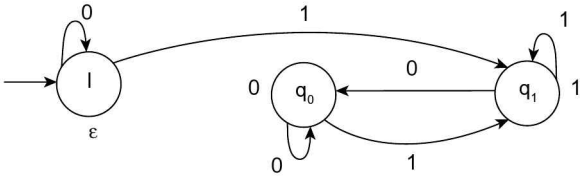


Fig. 2.44

Example 2.27 Design a Melay machine to generate unary representatio for the given binary representation such that, the number of 0's in unary is given as $N_u(0) = N_b(0) + 2 * N_b(1)$ if it starts with 1, otherwise $N_u(0) = 2 * N_b(0) + N_b(1)$.

Solution: The Mealy machine requires three states. In the first state (I) on input 0, it changes to state q_0 indicating that input starts with 0, and on 1 input moves to state q_1 indicating that input starts with 1. In q_0 for input 0, it generates output 00, and for input 1, it generates output 0. Similarly in q_1 state, for input 0, it generates output 0, and for input 1, it generates output 00. A Mealy machine is shown in Figure 2.45.

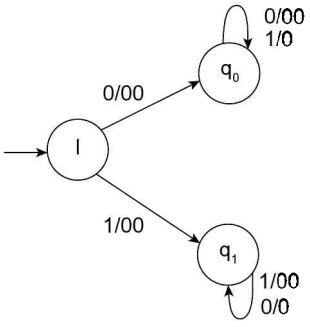


Fig. 2.45

2.9.3 Equivalence Between Moore and Mealy Machines

Theorem 2: If $M_1 = \{Q, \Sigma, \Delta, \delta, \lambda, q_0\}$ is a Moore machine, then there is a Mealy machine M_2 equivalent to M_1.

Proof: Let $M_2 = \{Q, \Sigma, \Delta, \delta, \lambda', q_0\}$ be a Mealy machine where λ' is defined by

$$\lambda'(q, a) = \lambda(\delta(q, a)) \text{ for all } q \text{ in } Q \text{ and } a \text{ in } \Sigma.$$

If there is any input **x** processed in M_1 that generates a sequence of states $q_1, q_2, \ldots q_n$ then the same sequence would be generated in M_2.

With each transition, M_2 emits the output that M_1 associates with the state entered.

Theorem 3: If $M_1 = \{Q, \Sigma, \Delta, \delta, \lambda, q_0\}$ is a Mealy machine, then there is a Moore machine M_2 equivalent to M_1.

Proof: Let $M_2 = \{\{Q \times \Delta\}, \Sigma, \Delta, \delta', \lambda', [q_0, b_0]\}$ where b_0 is an arbitrarily selected member of Δ. Let the states in M_2 be pairs $[q, b]$ where $q \in Q$ and $b \in \Delta$.

Let δ' be defined by $\delta'([q, b], a) = [\delta(q, a), \lambda(q, a)]$.
Let λ' be defined by $\lambda'([q, b]) = b$.

Using induction it can be shown that M_2 can simulate all the moves of M_1 i.e if M_1 enter states sequence as $q_0, q_1, q_2, \ldots q_n$ on input $a_1, a_2, a_3 \ldots a_n$ and emits output $b_1, b_2, \ldots b_n$, then M_2 enters states $[q_0, b_0], [q_1, b_1], [q_2, b_2] \ldots [q_n, b_n]$ and emits outputs $b_0, b_1, b_2, \ldots b_n$.

2.9.4 Interconversions Between Machines

Example 2.24 Construct a Moore machine for the following Mealy machine.

Current State	A = 0		A = 1	
	Next state	Output	Next state	Output
$\rightarrow q_1$	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0

Solution:

1. Frame the possible states of the form [q a] where $q \in Q$ and $a \in \Delta$. The number of states generated would be $n \times m$ where n is the number of states in Q and m is the number of output symbols, and out of these states, only few would be necessary. To include only the required ones, first identify the state q_i which is associated with different outputs.
2. Represent of q_i as different states, such that the number of states is equal to the number of different outputs associated with q_i.
3. Reconstruct the table with new states.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

4. Construct its equivalent Mealy machine.
5. If the initial state is associated with output then create a new state and add the transitions similar to initial state q_1 and add the output as ε.

Step 1: In the given table, q_1 and q_3 are associated with only one output. But q_2 and q_4 are associated with two different outputs.

Step 2: Represent q_2 as two states as q_{20} and q_{21}, represent q_4 as two states q_{40} and q_{41}.

Step 3: Reconstruct the table with new states.

Current State	A = 0		A = 1	
	Next state	Output	Next state	Output
$\rightarrow q_1$	q_3	0	q_{20}	0
q_{20}	q_1	1	q_{40}	0
q_{21}	q_1	1	q_{40}	0
q_3	q_{21}	1	q_1	1
q_{40}	q_{41}	1	q_3	0
q_{41}	q_{41}	1	q_3	0

Step 4: Construction of equivalent Moore machine.

Current State	Next State		OutPut
	a = 0	a = 1	
$\rightarrow q_1$	q_3	q_{20}	1
q_{20}	q_1	q_{40}	0
q_{21}	q_1	q_{40}	1
q_3	q_{21}	q_1	0
q_{40}	q_{41}	q_3	0
q_{41}	q_{41}	q_3	1

Step 5: Since q_1 state is associated with output 1 add a new initial state q_0 and associate with it the transitions of q_1, but with output ε.

Current State	Next State		OutPut
	a = 0	a = 1	
$\rightarrow q_0$	q_3	q_{20}	ε
q_1	q_3	q_{20}	1
q_{20}	q_1	q_{40}	0
q_{21}	q_1	q_{40}	1
q_3	q_{21}	q_1	0
q_{40}	q_{41}	q_3	0
q_{41}	q_{41}	q_3	1

Example 2.29 Construct the Mealy machine for the given Moore machine.

Current State	Next State		OutPut
	a = 0	a = 1	
→ q_1	q_1	q_2	0
q_2	q_1	q_3	0
q_3	q_1	q_3	1

Solution:
We construct the Mealy machine by associating the output with transition. The resultant table is

Current State	A = 0		A = 1	
	Next state	Output	Next state	Output
→ q_1	q_1	0	q_2	0
q_2	q_1	0	q_3	1
q_3	q_1	0	q_3	1

We observe that the transitions corresponding to q_2 state and q_3 state are same. Hence, we can eliminate q_3 state and modify the Mealy machine to

Current State	A = 0		A = 1	
	Next state	Output	Next state	Output
→ q_1	q_1	0	q_2	0
q_2	q_1	0	q_2	1

2.10 Applications of Finite Automata with Output

Moore machines are useful to describe language recognizers, to describe counters, such as a modulo-4 counter. Mealy machines are useful to describe DFA in which an output is associated with the transition. A full-adder changes the states to reflect the carry-in, and it outputs the sum as the transition takes place.

2.10.1 The Full-adder

Moore machine: Here we use the carry-in as the state. The DFA will output the sum, but this example will ignore that and focus on the state transitions.
Note that a two-state Moore machine does not easily display the sum.

A complete Moore machine model of a full-adder would require four states.

q_0 Carry-In = 0 Sum = 0
q_1 Carry-In = 0 Sum = 1

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

q_2 Carry-In = 1 Sum = 0
q_3 Carry-In = 1 Sum = 1

Mealy machine:

To simplify the definition, let $Q = \{q_0, q_1\}$, where

q_0 is the state previously labelled as 'Carry-In = 0' and

q_1 is the state previously labelled as 'Carry-In = 1'.

The input alphabet is $\Sigma = \{<0, 0>, <0, 1>, <1, 0>, <1, 1>\}$.
Please note that this alphabet has four symbols.

The output alphabet is $\Delta = \{0, 1\}$.

The transition function:

$\delta(q_0, <0, 0>) = q_0$ $\delta(q_1, <0, 0>) = q_0$
$\delta(q_0, <0, 1>) = q_0$ $\delta(q_1, <0, 1>) = q_1$
$\delta(q_0, <1, 0>) = q_0$ $\delta(q_1, <1, 0>) = q_1$
$\delta(q_0, <1, 1>) = q_1$ $\delta(q_1, <1, 1>) = q_1$

The output function:

$\lambda(q_0, <0, 0>) = 0$ $\lambda(q_1, <0, 0>) = 1$
$\lambda(q_0, <0, 1>) = 1$ $\lambda(q_1, <0, 1>) = 0$
$\lambda(q_0, <1, 0>) = 1$ $\lambda(q_1, <1, 0>) = 0$
$\lambda(q_0, <1, 1>) = 0$ $\lambda(q_1, <1, 1>) = 1$

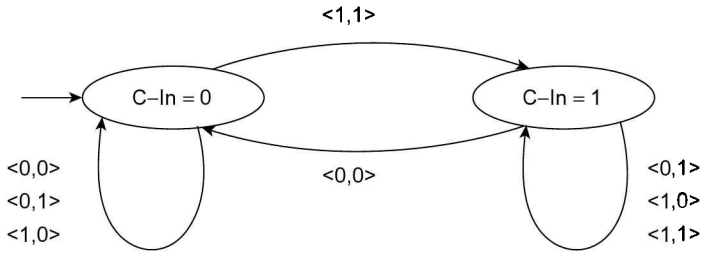


Fig. 2.46

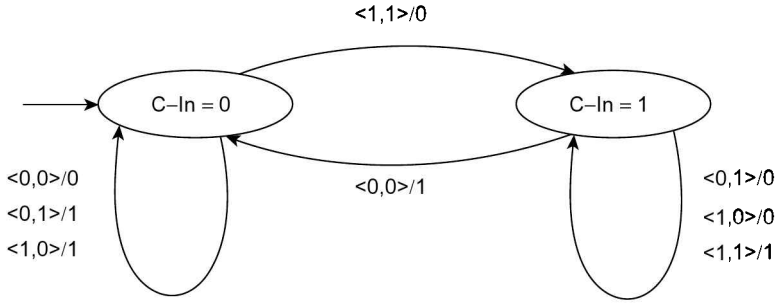


Fig. 2.47

2.10.2 The String Sequence Detector

We now investigate two implementations of a 11011 sequence detector.
Each of these machines accepts strings, one symbol at a time.
The alphabet for each machine is $\Sigma = \{0, 1\}$.
Each machine will accept any string with a five-symbol suffix 11011.

Moore machine: Let $Q = \{q_1, q_2, q_3, q_4, q_5, q_6\}$ where q_1 is the start state, $\Sigma = \{0, 1\}$ $F = \{q_6\}$
The transition function is given by the following equations.

$$\delta(q_1, 0) = q_1, \delta(q_2, 0) = q_1, \delta(q_3, 0) = q_4, \delta(q_4, 0) = q_1, \delta(q_5, 0) = q_1, \delta(q_6, 0) = q_1$$

$$\delta(q_1, 1) = q_2, \delta(q_2, 1) = q_3, \delta(q_3, 1) = q_3, \delta(q_4, 1) = q_5, \delta(q_5, 1) = q_6, \delta(q_6, 1) = q_2$$

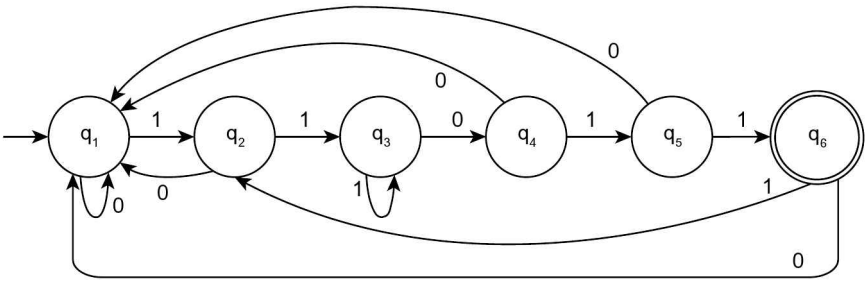


Fig. 2.48

Mealy machine: Here the input and output alphabets are considered same.
$Q = \{q_1, q_2, q_3, q_4, q_5\}$ where q_1 is the start state
$\Sigma = \{0, 1\}$
$\Delta = \{0, 1\}$

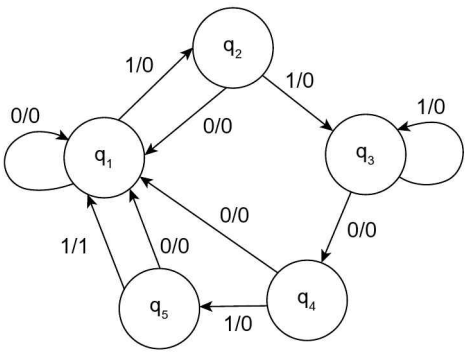


Fig. 2.49

The transition function is given by the following:

1. $\delta(q_1, 0) = q_1, \delta(q_2, 0) = q_1, \delta(q_3, 0) = q_4, \delta(q_4, 0) = q_1, \delta(q_5, 0) = q_1,$
   $\delta(q_1, 1) = q_2, \delta(q_2, 1) = q_3, \delta(q_3, 1) = q_3, \delta(q_4, 1) = q_5, \delta(q_5, 1) = q_1.$

The output function is given by the following:

2. $1(q_1, 0) = 0, \lambda(q_2, 0) = 0, \lambda(q_3, 0) = 0, \lambda(q_4, 0) = 0, \lambda(q_5, 0) = 0,$
   $\lambda(q_1, 1) = 0, \lambda(q_2, 1) = 0, \lambda(q_3, 1) = 0, \lambda(q_4, 1) = 0, \lambda(q_5, 1) = 1.$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Solved Problems

Problem 1: Consider a toy game shown in Figure 2.50. A marble is dropped in at A or B. Levers X_1, X_2, X_3 cause the marble to fall either to the left or right. Whenever a marble encounters a lever, marble causes the lever to change state, so that the next marble to encounter the lever will take the opposite branch.

Model this toy by an FA. Denote marble input at A by input 0 and a marble input at B by input **1**. A sequence of inputs is accepted if the last marble comes out at D. Describe the set accepted by FA.

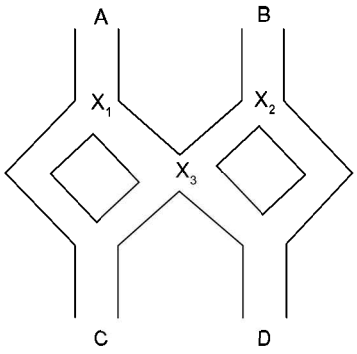


Fig. 2.50

Solution: This problem can be modelled using finite automata in which each lever has two possible moves, that is, **L** for left and **R** for right. We assume that 0 indicates the marble is dropped from A and 1 indicates that the marble is dropped from B. Initially in the start state, it is assumed that all levers are towards left, indicated by state LLL (for positions of X_1, X_2 and X_3, respectively). If the marble is dropped from A only X_1 would change from L to R and if the marble is dropped from B then the lever X_2, X_3 would change to R. The change of lever states is shown in the following table.

Note: Marble coming out of D wins the game.

Set of accepting states are * marked.

$X_1 X_2 X_3$	0	1
→ LLL	RLL_C	LRR_C
RLL_C	LLR_C	RRR_C
LRR_C	RRR_C	RLR_D*
LRL_C	RRL_C	LLL_D*
RRR_C	LRL_D*	RLR_D*
LLR_D*	RLR_C	LRL_D*
RLR_D*	LRR_C	RLL_D*
LLL_D*	RLL_C	LRR_C

(*continued*)

LRL_D*	RRL_C	LLL_D*
RRL_D*	LRR_D*	RLL_D*
RLR_D	LLL_D*	RRL_D*
RLR_C	LLL_D*	RRL_D*
RLL_D*	LLR_C	RRL_D*
LRR_D*	RRR_C	LLR_D*
LLR_C	RLR_C	LRL_D*

Problem 2: Design an FA that accepts strings in L such that integer number 'a' when represented in binary form is divisible by 3.

Solution: It is required to construct a DFA such that any integer in binary form that is divisible by 3 should enter into final state on seeing the last digit.

When we divide any number by 3, there are three possible remainders 0, 1 or 2. So, it is required to have three states to indicate each possibility. So, let

q_0 - represent remainder 0
q_1 - represent remainder 1
q_2 - represent remainder 2. The states are shown in the Figure 2.51

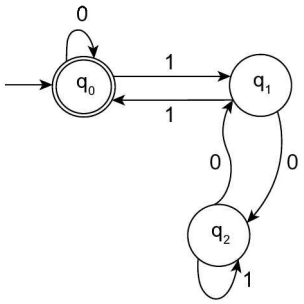


Fig. 2.51

Since the number should be divisible by 3, which gives the remainder as zero, the state q_0 is made as final state.

Integer number	Binary number	Result	State
0	0	0	q_0
1	1	1	q_1
2	10	2	q_2
3	11	0	q_0
4	100	1	q_1
5	101	2	q_2
6	110	0	q_0

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Problem 3: Design DFA that accepts all strings with 0's and 1's such that any five consecutive symbols have at least two 0's.

Solution: This problem can be solved by keeping track of the previous four symbols and the current symbol. We make an assumption that the string would have at least a length of five symbols to be accepted. This requires many states shown below. We use a special state D to indicate a dummy state. The minimum length of string is assumed to be of length 5.

Note: The suffix of every state keeps track of the sequence of symbols seen so far.

State ↓Input →	a = 0	a = 1
→ S	q_0	q_1
q_0	q_{00}	q_{01}
q_1	q_{10}	q_{11}
q_{00}	q_{000}	q_{001}
q_{01}	q_{010}	q_{011}
q_{10}	q_{100}	q_{101}
q_{11}	q_{110}	q_{111}
q_{000}	q_{0000}	q_{0001}
q_{001}	q_{0010}	q_{0011}
q_{010}	q_{0100}	q_{0101}
q_{011}	q_{0110}	q_{0111}
q_{100}	q_{1000}	q_{1001}
q_{101}	q_{1010}	q_{1011}
q_{110}	q_{1100}	q_{1101}
q_{111}	q_{1110}	D
q_{0000}	q_{00000}	q_{00001}
q_{0001}	q_{00010}	q_{00011}
q_{0010}	q_{00100}	q_{00101}
q_{0011}	q_{00110}	q_{00111}
q_{0100}	q_{01000}	q_{01001}
q_{0101}	q_{01010}	q_{01011}
q_{0110}	q_{01100}	q_{01101}
q_{0111}	q_{01110}	D
q_{1000}	q_{10000}	q_{10001}
q_{1001}	q_{10010}	q_{10011}
q_{1010}	q_{10100}	q_{10101}
q_{1011}	q_{10110}	D
q_{1100}	q_{11000}	q_{11001}

(*continued*)

State ↓Input →	a = 0	a = 1
q_{1101}	q_{11010}	D
q_{1110}	q_{11100}	D
D	D	D
$*q_{00000}$	q_{00000}	q_{00001}
$*q_{00001}$	q_{00010}	q_{00011}
$*q_{00010}$	q_{00100}	q_{00101}
$*q_{00011}$	q_{00110}	q_{00111}
$*q_{00100}$	q_{01000}	q_{01001}
$*q_{00101}$	q_{01010}	q_{01011}
$*q_{00110}$	q_{01100}	q_{01101}
$*q_{00111}$	q_{01110}	D
$*q_{01000}$	q_{10000}	q_{10001}
$*q_{01001}$	q_{10010}	q_{10011}
$*q_{01010}$	q_{10100}	q_{10101}
$*q_{01011}$	q_{10110}	D
$*q_{01100}$	q_{11000}	q_{11001}
$*q_{01101}$	q_{11010}	D
$*q_{01110}$	q_{11100}	D
$*q_{10000}$	q_{00000}	q_{00001}
$*q_{10001}$	q_{00010}	q_{00011}
$*q_{10010}$	q_{00100}	q_{00101}
$*q_{10011}$	q_{00110}	q_{00111}
$*q_{10100}$	q_{01000}	q_{01001}
$*q_{10101}$	q_{01010}	q_{01011}
$*q_{10110}$	q_{01100}	q_{01101}
$*q_{11000}$	q_{10000}	q_{10001}
$*q_{11001}$	q_{10010}	q_{10011}
$*q_{11010}$	q_{10100}	q_{10101}
$*q_{11100}$	q_{11000}	q_{11001}

Problem 4: Design a DFA that accepts the set of all strings beginning with 1 (which, interpreted as binary representation of an integer, is congruent to zero modulo 5).

Solution: Possible outputs with modulo 5 operation are 0, 1, 2, 3, 4. Let these states be denoted by q_0 q_1 q_2 q_3 q_4 (representing remainders 0, 1, 2, 3, 4, respectively). It is also said that the string should begin with 1. If the string starts with 0 it should not be accepted. Hence, 0 is not a valid integer in the set. So on seeing 0, it should enter into a dummy state. Add a new initial state S and a dummy state q_d. Figure 2.52 shows the designed DFA.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Integer number	Binary number	Result	State
0	0	0	q_d
1	1	1	q_1
2	10	2	q_2
3	11	3	q_3
4	100	4	q_4
5	101	0	q_0
6	110	1	q_1
7	111	2	q_2
8	1000	3	q_3
9	1001	4	q_4
10	1010	0	q_0

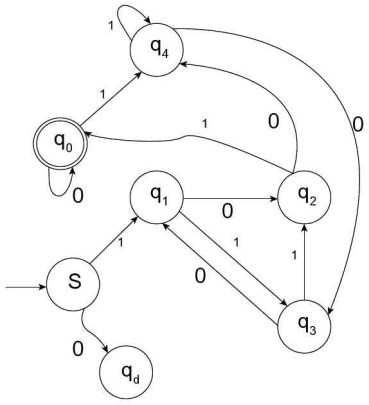


Fig. 2.52 *DFA for Integer Mod 5*

Problem 5: Design a DFA to accept strings having 111 as substring.

Solution: The language has strings which are of the form {111, 0111, 1111, 1110, 00111, 01111, 10111, 11111,.....}. The DFA should have four states. In the 0th state, if it finds 1, it changes to 1st state. In 1st state if it finds 1, it changes to 2nd state. In 2nd state if it finds 1, it changes to 3rd state. Declare 3rd state as final and be in the same state on 0 or 1. In states other than the 3rd, if it finds 0, it should start from initial state as there would not be 111's. The DFA is shown in Figure 2.53.

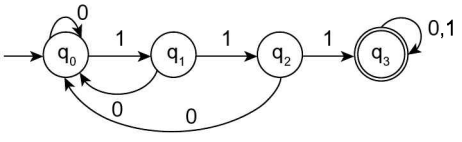


Fig. 2.53 *DFA for Problem 5*

Problem 6: Design DFA for L(M) = {$a^n b \mid n \geq 0$}

Solution: Start with initial state q_0. Define a^n by taking loop. To have b next, have another state q_1.The transitions on q_1 can be defined to move on to dummy state q_2.

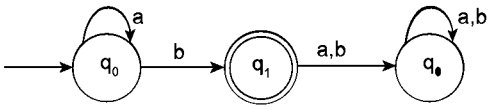


Fig. 2.54 *DFA for Strings with a's Followed by a b*

Problem 7: Design DFA to accept set of all strings ending in 00.

Solution: The language has strings which are of the form {00, 100, 000, 0000, 1000, 0100, 1100, ……}. The DFA should have three states. In the 0th state, if it finds 0, it changes to 1st state. In 1st state if it finds 0, it changes to 2nd state. Declare 2nd state as final and be in the same state on 0. In any state, if it finds 1, it should change to initial state as it should end with 00. The DFA is shown Figure 2.55.

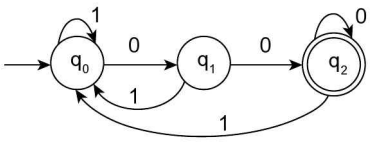


Fig. 2.55 *DFA for Strings Ending with 00*

Problem 8: Design DFA to accept the set of all strings not containing 101 as substring.

Solution: The language has strings of the form {ε, 0, 1, 00, 10, 01, 11, 001,.}. It should not contain strings that have 101 as substring. First define DFA with four states with 101 string. Declare the last state q_3 as dead state. The remaining states can be declared as final states as it has to accept strings of length 0, 1, 2,…. The moves on each state are defined as shown in Figure 2.56.

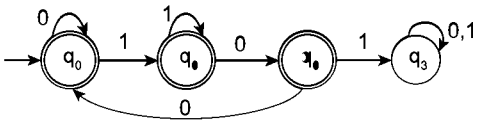


Fig. 2.56 *DFA for String with Substring 101*

Problem 9: If L(M) = {$0^m 1^n \mid m, n \geq 0$}, design DFA for M.

Solution: Strings of the form {ε, 0, 1, 00, 01, 11, 000, 001, 011, 111, 0000, 0001, 0011, 0111, 1111,….} are valid in the language. Since ε is valid, initial state is declared as final state. It stays in the same state q_0 on 0's and changes to q_1 on 1. Declare q_1 as final state.

Be in q_1 state on 1 and move to dead state q_2 on 0. Be in this same state whether on 0 or on 1, as they are invalid strings.

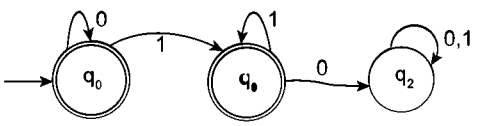


Fig. 2.57 *DFA for Strings with 0's Followed by 1's*

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Problem 10: Design DFA to accept the set of all strings containing three consecutive 0's.

Solution: The language has strings of the form {000, 1000, 0000, 0001, 11000, 10000, 01000, 00000,.....}. The DFA should have four states. In the 0th state, if it finds 0, it changes to 1st state. In 1st state, if it finds 0, it changes to 2nd state. In 2nd state, if it finds 0, change to 3rd state. Declare 3rd state as final and be in the same state on 1 or 0. In states other than the3^{rd}, if it finds 1, it should start from initial state as there would not be 000's. The DFA is shown in Figure 2.58.

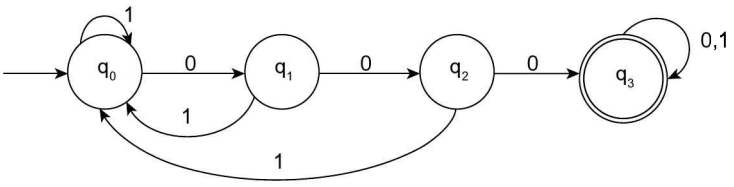


Fig. 2.58 *DFA for Strings with Three Consecutive 0's*

Problem 11: Design DFA that accepts even number of a's or even number of b's.

Solution: The language has strings which are of the form {aa, bb, abb, bab, bba, aab, aba, aab,....}. The DFA should have four states. q_0 indicates the condition where number of a's and b's are even. q_2 indicates the condition where number of a's is even and number of b's is odd. q_1 indicates that number of b's is even and number of a's is odd, and q_4 indicates that number of a's is odd and, further, number of b's is also odd. The DFA is shown in Figure 2.59.

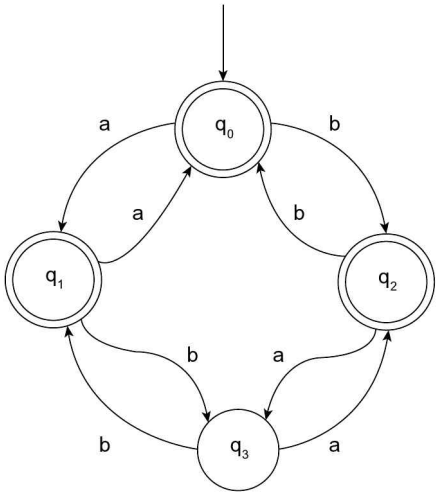


Fig. 2.59 *DFA for Even Number of a's or b's*

Problem 12: Design the NFA that accepts 1100 only.

Solution: The NFA for the given condition can be drawn in many ways. Samples are shown in Figure 2.60.

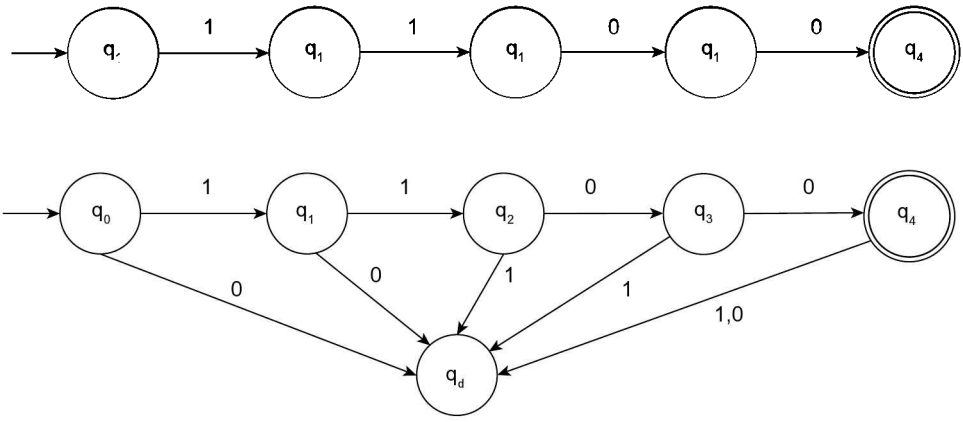


Fig. 2.60 *DFA for String 1100*

Problem 13: Design the NFA that accepts strings that contains either two consecutive 0's or two consecutive 1's.

Solution: The language has strings of the form {00, 11, 000, 100, 001, 110, 011, 111…}

Therefore, whether on two consecutive 0's or 1's, go to state q_3 using q_1 or q_2. In q_0 and q_3 states, define moves to be in the same state on 0/1 as shown in Figure 2.61.

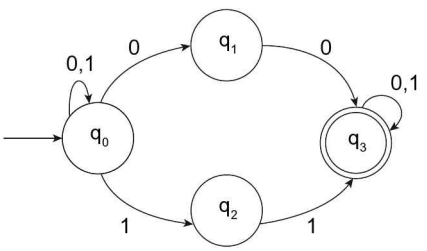


Fig. 2.61 *DFA for 00 or 11*

Problem 14: Design the NFA to accept (a) all strings containing 1100 as substring or (b) the set of all strings in which a pair of 1's is followed by a pair of 0's.

Solution:

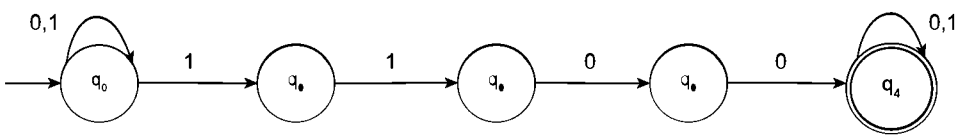


Fig. 2.62 *NFA for Substring 1100*

DFA is difficult to construct, but it is fast in processing strings. NFA is easy to construct, but it is slow in processing. Hence, it is always desirable to construct the NFA and convert it to a DFA, as for every NFA that accepts L three exists an equivalent DFA that accepts the same L and is shown in Figure 2.62.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Problem 15: Convert the following NFA to DFA.

δ	0	1
→p	{pq}	p
q	r	r
r	s	-
*s	s	s

Solution: Start with initial state: find transitions. Whenever you get new set of states, add it to set of states and find transitions on input symbols.

δ	0	1
→ [p]	[pq]	[p]
[pq]	[pqr]	[pr]
[pr]	[pqs]	[p]
[pqr]	[pqrs]	[pr]
*[pqs]	[pqrs]	[prs]
*[pqrs]	[pqrs]	[prs]
*[prs]	[pqs]	[ps]
*[ps]	[pqs]	[ps]

Problem 16: Design NFA for language L(M) = 01*0*1.

Solution: The strings are of the form: 0 followed by zero or more 1's, followed by zero or more 0's but ending with 1.

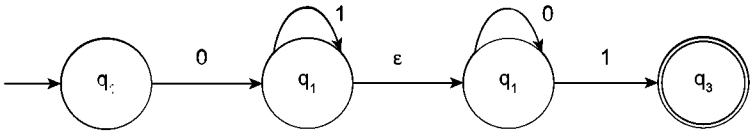


Fig. 2.63 *NFA for Problem 16*

Problem 17: Design NFA for language L(M) = 01* | 0*1.

Solution: The language has strings {0, 01, 011, 0111 .....} ∪ {1, 01, 001, 0001,....}.
This can be drawn in two ways as shown in Figure 2.64:

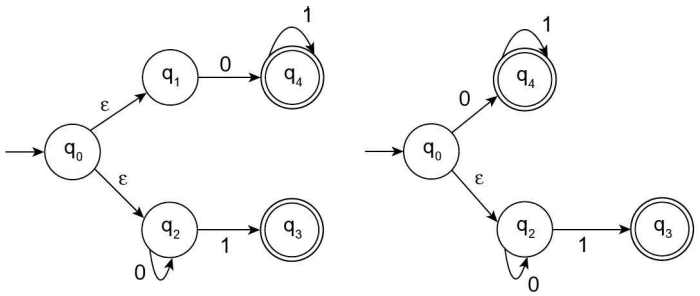


Fig. 2.64 *Two Possible Ways of Drawing NFA for 10* / 0*1*

Problem 18: Find whether the following two are equivalent or not.

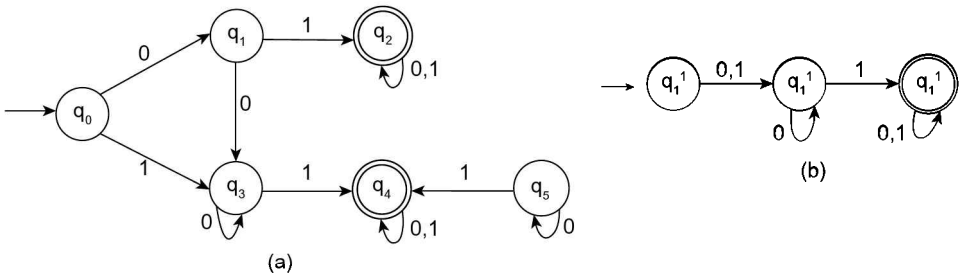


Fig. 2.65

Solution: Equivalence of two automata (i.e. if they accept same language):

1. Two FAs over Σ are equivalent if they accept the same set of strings over Σ.
2. When two FAs are not equivalent, there exists some string w over Σ for which, one automaton reaches a final state on w whereas the other reaches a non-final state.

we can simplfy DFA in Figure 2.65(a) by removing state q_5 as it is not reachable.

Current State (M_1, M_2)	Next state (M_1, M_2)	
	0	1
$\rightarrow (q_0, q_1{}^1)$	$(q_1, q_2{}^1)$	$(q_3, q_2{}^1)$
$(q_1, q_2{}^1)$	$(q_3, q_2{}^1)$	$(q_2, q_3{}^1)$
$(q_3, q_2{}^1)$	$(q_3, q_2{}^1)$	$(q_4, q_3{}^1)$
$(q_2, q_3{}^1)$	$(q_2, q_3{}^1)$	$(q_2, q_3{}^1)$
$(q_4, q_3{}^1)$	$(q_4, q_3{}^1)$	$(q_4, q_3{}^1)$

It is observed that for all pairs (p, q), where p $\in$ states in Figure 2.65(a) and q $\in$ states in Figure 2.65(b), they both either go to non-final state or final state. Hence, these two are said to be equivalent.

We can simplify state q_5 not reachable.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Problem 19: Reduce the DFA shown in Figure 2.66.

	0	1
→1	2	3
2	4	5
3	6	7
4	4	5
5	6	7
*6	4	5
7	6	7

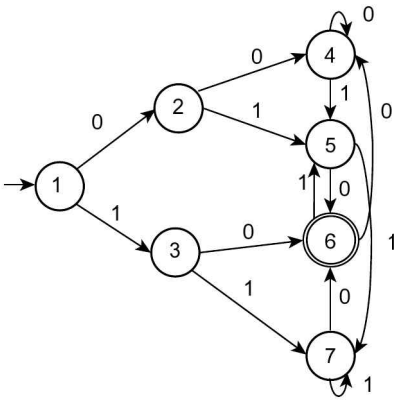


Fig. 2.66

Here in this problem equivalent classes are named using alphabets A, B, C ...

Solution:

Π_0 {A, B} = {{6}, {1, 2, 3, 4, 5, 7}}

	1	2	3	4	5	7
0	B	B	A	B	A	A
1	B	B	B	B	B	B

Π_1 {A, C, D} = {{6}, {1, 2, 4}, {3, 5, 7}}

	1	2	3	4	5	7
0	C	C	A	C	A	A
1	D	D	D	D	D	D

Π_1 {A, C, D} = {{6}, {1, 2, 4}, {3, 5, 7}}

	0	1
→{1, 2, 4}	{1, 2, 4}	{3, 5, 7}
{3, 5, 7}	{6}	{3, 57}
*{6}	{1, 2, 4}	{3, 5, 7}

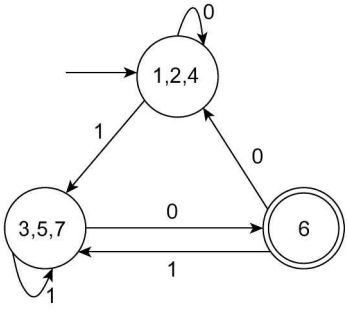


Fig. 2.67

The language accepted by this DFA is set of all strings of 0's and 1' is that end with 10.

Problem 20: Minimize the DFA shown in Figure 2.68.

	a	b
$\rightarrow Q_0$	Q_1	Q_0
Q_1	Q_0	Q_2
Q_2	Q_3	Q_1
$*Q_3$	Q_3	Q_0
Q_4	Q_3	Q_5
Q_5	Q_6	Q_4
Q_6	Q_5	Q_6
Q_7	Q_6	Q_3

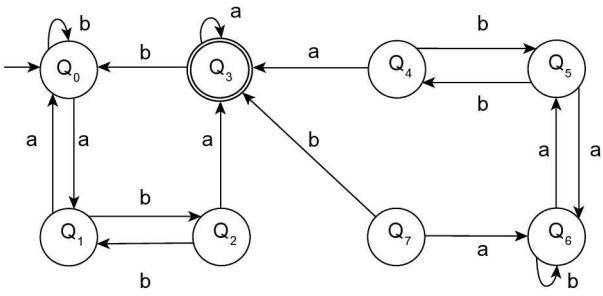


Fig. 2.68

Solution:

Π_0 {1, 2} = {q_3} {q_0 q_1 q_2 q_4 q_5 q_6 q_7}

a	a	a	a	a	a	a	a
a	a	a	a	a	a	a	a

Π_1 {1, 3, 4, 5} = {q_3} {q_0 q_1 q_5 q_6} {q_2 q_4} {q_7}

a	a	a	a	a	a	a	a
a	a	a	a	a	a	a	a

Π_2 {1, 4, 6, 7} = {q_3} {q_0 q_6}{q_2 q_4} {q_1 q_5} {q_7}

a	a	a	a	a	a	a	a
a	a	a	a	a	a	a	a

Π_3 {1, 4, 6,7} = {q_3} {q_0 q_6} {q_2 q_4} {q_1 q_5} {q_7}

a	a	a	a	a	a	a	a
a	a	a	a	a	a	a	a

	a	b
$\rightarrow Q_{0,6}$	$Q_{1,5}$	$Q_{0,6}$
$Q_{1,5}$	$Q_{0,6}$	$Q_{2,4}$
$Q_{2,4}$	Q_3	$Q_{1,5}$
$*Q_3$	Q_3	$Q_{0,6}$

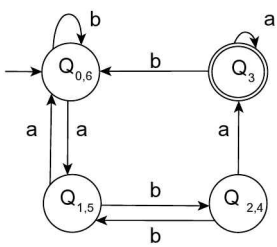


Fig. 2.69

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Note: If the DFA in Figure 2.68 is observed carefully, it is clear that the states Q_4, Q_5, Q_6 and Q_7 are not reachable from Q_0 state they can be eliminated directly.

Problem 21: Minimize the finite automata given in Figure 2.70.

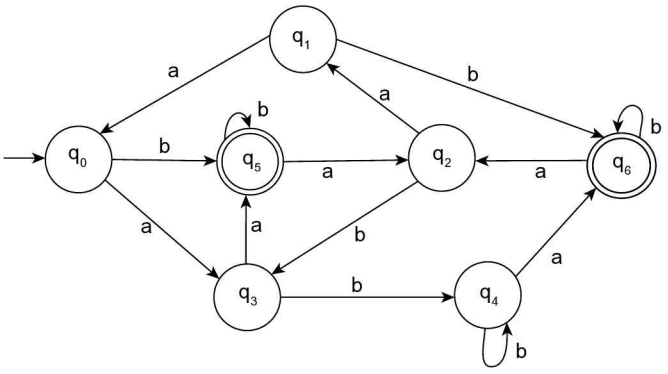


Fig. 2.70

Solution: Let us represent the DFA as a transition table.

	a = a	a = b
$\rightarrow q_0$	q_3	q_5
q_1	q_0	q_6
q_2	q_1	q_3
q_3	q_5	q_4
q_4	q_6	q_4
$*q_5$	q_2	q_5
$*q_6$	q_2	q_6

1. Initially we identify 0-equivalence as $\prod_0 = \{Q_1^0, Q_2^0\}$ where Q_1^0 is set of final states and $Q_2^0 = Q - Q_1^0$ is set of non-final states.

   $Q_1^0 = \{q_5, q_6\}$

   $Q_2^0 = \{q_0, q_1, q_2, q_3, q_4\}$

2. Construct $\prod_1$ from $\prod_0$ identifying the equivalent states in $\{Q_1^0, Q_2^0\}$.

   Q_1^0 has two states and Q_2^0 has five states, and we need to identify whether they are 1-equivalent.

   Compare q_5, q_6 on inputs a and b.

   $\left.\begin{array}{l}\delta(q_5, a) = q_2 \\ \delta(q_6, a) = q_2\end{array}\right\}$ both resultant states belong to Q_2^0.

   $\left.\begin{array}{l}\delta(q_5, b) = q_5 \\ \delta(q_6, b) = q_6\end{array}\right\}$ both both resultant states belong to Q_1^0.

   $\Rightarrow q_5$ is 1-equivalent to q_6 hence Q_1^0 cannot be divided.

Compare q_0, q_1 on inputs a and b.

$\left.\begin{array}{l}\delta(q_0, a) = q_3 \\ \delta(q_1, a) = q_0\end{array}\right\}$ both resultant states belong to Q_2^0.

$\left.\begin{array}{l}\delta(q_0, b) = q_5 \\ \delta(q_1, b) = q_6\end{array}\right\}$ both resultant states belong to Q_1^0.

$\Rightarrow q_0$ is 1-equivalent to q_1.

Compare q_0, q_2 on inputs a and b.

$\left.\begin{array}{l}\delta(q_0, a) = q_3 \\ \delta(q_2, a) = q_1\end{array}\right\}$ both resultant states belong to Q_2^0.

$\left.\begin{array}{l}\delta(q_0, b) = q_5 \\ \delta(q_2, b) = q_3\end{array}\right\}$ these two states belong to different sets in $\prod_0$.

$\Rightarrow q_0$ is not 1-equivalent to q_2. Hence Q_2^0 is split into two sets.

Compare q_0, q_3 on inputs a and b.

$\left.\begin{array}{l}\delta(q_0, a) = q_3 \\ \delta(q_3, a) = q_5\end{array}\right\}$ these two states belong to different sets in $\prod_0$

$\left.\begin{array}{l}\delta(q_0, b) = q_5 \\ \delta(q_3, b) = q_4\end{array}\right\}$ these two states belong to different sets in $\prod_0$

$\Rightarrow q_0$ is not 1-equivalent to q_3

Compare q_0, q_4 on input a and b.

$\left.\begin{array}{l}\delta(q_0, a) = q_3 \\ \delta(q_4, a) = q_6\end{array}\right\}$ these two states belong to different sets in $\prod_0$

$\left.\begin{array}{l}\delta(q_0, b) = q_5 \\ \delta(q_4, b) = q_4\end{array}\right\}$ these two states belong to different sets in $\prod_0$

$\Rightarrow q_0$ is not 1-equivalent to q_4

Therefore, $\prod_1 = \{Q_1^1, Q_2^1, Q_3^1\}$

where

$Q_1^1 = \{q_5, q_6\}$

$Q_2^1 = \{q_0, q_1\}$

$Q_3^1 = \{q_2, q_3, q_4\}$

3. Construct $\prod_2$ from $\prod_1$, identifying the equivalent states in $\{Q_1^1, Q_2^1, Q_3^1\}$.

Q_1^1 and Q_2^1 have two states, and Q_3^1 has three states. We need to identify whether they are 2-equivalent.

Compare q_5, q_6 on inputs a and b.

$\left.\begin{array}{l}\delta(q_5, a) = q_2 \\ \delta(q_6, a) = q_2\end{array}\right\}$ both resultant states belong to Q_3^1.

$\left.\begin{array}{l}\delta(q_5, b) = q_5 \\ \delta(q_6, b) = q_6\end{array}\right\}$ both resultant states belong to Q_1^1.

$\Rightarrow q_5$ is 2-equivalent to q_6,Hence Q_1^1 cannot be divided.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Compare q_0, q_1 on inputs a and b.

$\delta(q_0, a) = q_3$
$\delta(q_1, a) = q_0$ } these two states belong to different sets in $\prod_1$

$\delta(q_0, b) = q_5$
$\delta(q_1, b) = q_6$ } both resultant states belong to Q_1^1.

$\Rightarrow q_0$ is not 2-equivalent to q_1

Compare q_2, q_3 on inputs a and b.

$\delta(q_2, a) = q_1$
$\delta(q_3, a) = q_5$ } these two states belong to different sets in $\prod_1$

$\delta(q_2, b) = q_3$
$\delta(q_3, b) = q_4$ } both resultant states belong to Q_3^1.

$\Rightarrow q_2$ is not 2-equivalent to q_3.

Compare q_2, q_4 on inputs a and b.

$\delta(q_2, a) = q_1$
$\delta(q_4, a) = q_6$ } these two states belong to different sets in $\prod_1$

$\delta(q_2, b) = q_3$
$\delta(q_4, b) = q_4$ } both resultant states belong to Q_3^1.

$\Rightarrow q_2$ is not 2-equivalent to q_4.

Therefore, $\prod_2 = \{Q_1^2, Q_2^2, Q_3^2, Q_4^2, Q_5^2\}$,

where

$Q_1^2 = \{q_5, q_6\}$
$Q_2^2 = \{q_0\}$
$Q_3^2 = \{q_1\}$
$Q_4^2 = \{q_2\}$
$Q_5^2 = \{q_3, q_4\}$

4. Construct $\prod_3$ from $\prod_2$, identifying the equivalent states in $\{Q_1^2, Q_2^2, Q_3^2, Q_4^2, Q_5^2\}$.

Q_1^2, Q_5^2 have only a single state. Hence, these sets cannot be divided. Q_1^2 and Q_5^2 have two states; we need to identify whether they are 3-equivalent.

Compare q_5, q_6 on inputs a and b.

$\delta(q_5, a) = q_2$
$\delta(q_6, a) = q_2$ } both resultant states belong to Q_4^2.

$\delta(q_5, b) = q_5$
$\delta(q_6, b) = q_6$ } both resultant states belong to Q_1^2.

$\Rightarrow q_5$ is 3-equivalent to q_6 Q_1^1 cannot be divided.

Compare q_3, q_4 on inputs a and b.

$\delta(q_3, a) = q_5$
$\delta(q_4, a) = q_6$ } both resultant states belong to Q_1^2

$\delta(q_3, b) = q_4$
$\delta(q_4, b) = q_4$.

$\Rightarrow q_3$ is 3-equivalent to q_4

Therefore, $\prod_3 = \{Q_1^3, Q_2^3, Q_3^3, Q_4^3, Q_5^3\}$,
where
$Q_1^1 = \{q_5, q_6\}$
$Q_2^1 = \{q_0\}$
$Q_3^1 = \{q_1\}$
$Q_4^1 = \{q_2\}$
$Q_5^1 = \{q_3, q_4\}$

5. We see that $\prod_3$ is equal to $\prod_2$. The states q_5 and q_6 are considered as the single state q_{56}, and q_3 and q_4 are considered as the single state q_{34}. Minimized DFA is

	a = a	a = b
→ q_0	q_{34}	q_{56}
q_1	q_0	q_{56}
q_2	q_1	q_{34}
q_{34}	q_{56}	q_{34}
*q_{56}	q_2	q_{56}

Problem 22: Construct Moore machine for the following Mealy machine (Figure 2.71).

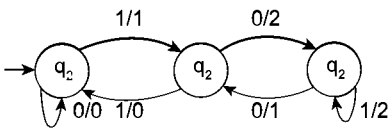


Fig. 2.71

Solution:

Current State	A = 0		A = 1	
	Next State	Output	Next state	Output
→ q_0	q_0	0	q_1	1
q_1	q_2	2	q_0	0
q_2	q_1	1	q_2	2

In the given table, all states are associated with only one output. No state needs to be split as all states are with one output.

Equivalent Moore Machine

Current State	Next State		OutPut
	a = 0	a = 1	
→q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_1	q_2	2

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Problem 23: Give Mealy and Moore machine for input from (0+1)* satisfying the given condition. If input ends with 101, output is A if input ends with 110, output is B; otherwise output is C.

Solution: For this problem, it is easy to construct Moore machine and then convert it into its equivalent Mealy machine.

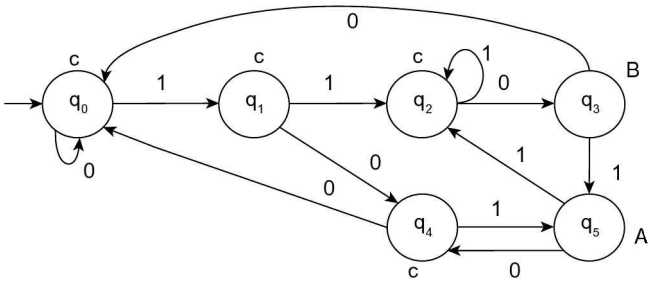


Fig. 2.72

Current State	Next State		OutPut
	a = 0	a = 1	
$\rightarrow q_0$	q_0	q_1	C
q_1	q_4	q_3	C
q_2	q_3	q_2	C
q_3	q_0	q_5	B
q_4	q_0	q_5	C
q_5	q_4	q_2	A

Its corresponding Mealy machine is as follows.

Current State	A = 0		A = 1	
	Next state	Output	Next state	Output
$\rightarrow q_0$	q_0	C	q_1	C
q_1	q_4	C	q_3	B
q_2	q_3	B	q_2	C
q_3	q_0	C	q_5	A
q_4	q_0	C	q_5	A
q_5	q_4	C	q_2	C

Problem 24: Give Mealy and Moore machines for input from (0 + 1 + 2)* to print the residue modulo 5 of the input treated as a ternary (base 3, with digits 0, 1, 2) numbers.

Solution: To construct the Moore machine we have to understand the possible outputs. Since it is residue modulo 5, the possible outputs are 0, 1, 2, 3 and 4. The possible inputs would be the ternary representations of the integers.

Example:

Integer number	Ternary representation	Output
1	1	1
2	2	2
3	10	3
4	11	4
5	12	0
6	20	1
7	21	2
8	22	3
9	100	4
10	101	0
11	102	1
12	110	2
13	111	3
14	112	4

The Moore machine is given in Figure 2.73.

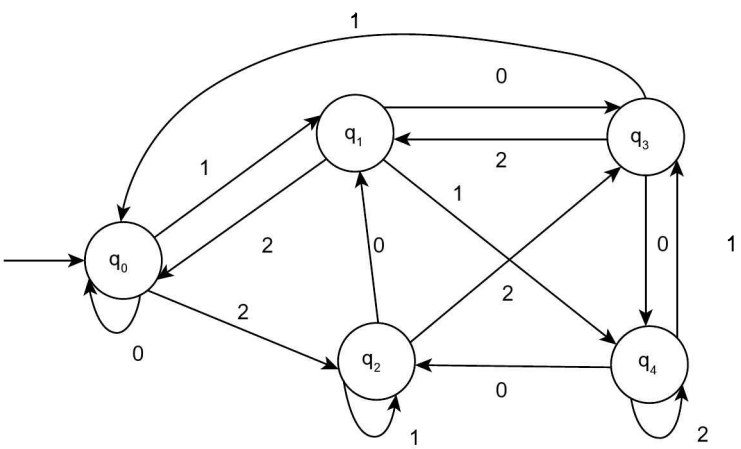


Fig. 2.73

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Current State	Next State			Output
	a = 0	a = 1	a = 2	
$\rightarrow q_0$	q_0	q_1	q_2	0
q_1	q_3	q_4	q_0	1
q_2	q_1	q_2	q_3	2
q_3	q_4	q_0	q_1	3
q_4	q_2	q_3	q_4	4

The corresponding Mealy machine is

Current State	a = 0		a = 1		a = 2	
	Next state	Output	Next state	Output	Next state	Output
$\rightarrow q_0$	q_0	0	q_1	1	q_2	2
q_1	q_3	3	q_4	4	q_0	0
q_2	q_1	1	q_2	2	q_3	3
q_3	q_4	4	q_0	0	q_1	1
q_4	q_2	2	q_3	3	q_4	4

Problem 25: Draw a DFA to accept strings with a's and b's such that the number of a's is a multiple of 3 and the number of b's is a multiple of 2.

Solution: To construct a DFA satisfying both conditions, first let us construct DFAs independent of each condition and then merge them to get the final DFA.

1. a's in multiples of 3 (Figure 2.74):

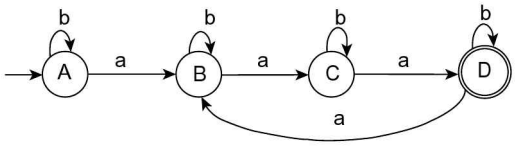


Fig. 2.74

2. b's in multiples of 2 (Figure 2.75):

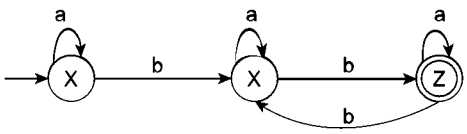


Fig. 2.75

To construct a DFA satisfying both the conditions, merge the two DFAs.

The states in the final DFA are {[AX], [AY], [AZ], [BX], [BY], [BZ], [CX], [CY], [CZ], [DX], [DY], [DZ]}

The state which satisfies both the conditions is [DZ]. To find the transitions of the DFA, start at the initial state [AX]. The transitions are defined as follows:

$\delta([PQ], a) = [\delta(P, a)\ \delta(Q, a)]$ where P is a state of the 1st DFA and Q is a state of the 2nd DFA.

	i = a	i = b
→[AX]	[BX]	[AY]
[AY]	[BY]	[AZ]
[AZ]	[BZ]	[AY]
[BX]	[CX]	[BY]
[BY]	[CY]	[BZ]
[BZ]	[CZ]	[BY]
[CX]	[DX]	[CY]
[CY]	[DY]	[CZ]
[CZ]	[DZ]	[CY]
[DX]	[BX]	[DY]
[DY]	[BY]	[DZ]
*[DZ]	[BZ]	[DY]

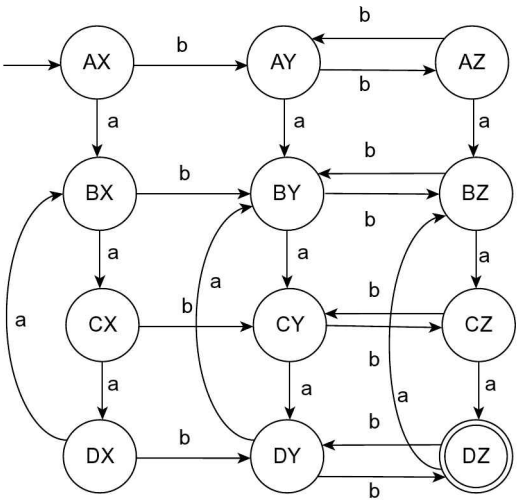
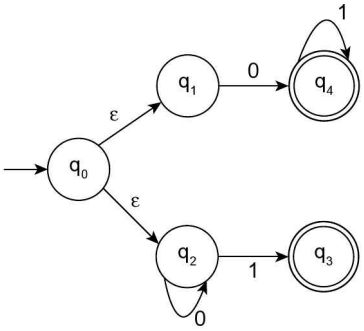


Fig. 2.76

Problem 26: Construct the DFA for the following NFA with ε moves:



Solution:

Step 1: Compute ε-closure of each state.

$\hat{\varepsilon}(q_0) = \{q_0, q_1, q_2\}$

$\hat{\varepsilon}(q_1) = \{q_1\}$

$\hat{\varepsilon}(q_2) = \{q_2\}$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$\hat{\varepsilon}(q_3) = \{q_3\}$
$\hat{\varepsilon}(q_4) = \{q_4\}$

Step 2: Explore the valid states in DFA that are computed in step 2.

$\hat{\delta}(q_0, 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), 0))$
$= \varepsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 0))$
$= \varepsilon\text{-closure}(\{\varnothing\}, \cup \{q_4\}, \cup\{q_2\})$
$= [q_2 q_4]$

$\hat{\delta}(q_0, 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 1))$
$= \varepsilon\text{-closure}(\{\varnothing\}, \cup\{\varnothing\}, \cup\{q_3\})$
$= [q_3]$

$\hat{\delta}([q_2 q_4], 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_2 q_4]), 0))$
$= \varepsilon\text{-closure}(\delta(\{q_2, q_4\}, 0))$
$= \varepsilon\text{-closure}(\{q_2\}, \cup\{\varnothing\})$
$= [q_2]$

$\hat{\delta}([q_2 q_4], 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_2 q_4]), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_2, q_4\}, 1))$
$= \varepsilon\text{-closure}(\{q_3\}, \cup\{q_4\})$
$= [q_3 q_4]$

$\hat{\delta}([q_3], 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_3]), 0))$
$= \varepsilon\text{-closure}(\delta(\{q_3\}, 0))$
$= \varepsilon\text{-closure}(\{\varnothing\})$
$= [\varnothing]$

$\hat{\delta}([q_3], 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_3]), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_3\}, 1))$
$= \varepsilon\text{-closure}(\{\varnothing\})$
$= [\varnothing]$

$\hat{\delta}([q_2], 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_2]), 0))$
$= \varepsilon\text{-closure}(\delta(\{q_2\}, 0))$
$= \varepsilon\text{-closure}(\{q_2\})$
$= [q_2]$

$\hat{\delta}([q_2], 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_1]), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_2\}, 1))$
$= \varepsilon\text{-closure}(\{q_3\})$
$= [q_3]$

$\hat{\delta}([q_3, q_4], 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_3, q_4]), 0))$
$= \varepsilon\text{-closure}(\delta(\{q_3\}, 0) \cup \delta(\{q_4\}, 0))$
$= \varepsilon\text{-closure}(\{\varnothing \cup \varnothing\})$
$= [\varnothing]$

$\hat{\delta}([q_3, q_4], 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}([q_3, q_4]), 1))$
$= \varepsilon\text{-closure}(\delta(\{q_3\}, 1) \cup \delta(\{q_4\}, 1))$

$= \varepsilon\text{-closure}(\{\varnothing \cup q_4\})$

$= [q_4]$

$\hat{\delta}([q_4], 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(\delta(q_4, 0))))$

$= \varnothing$

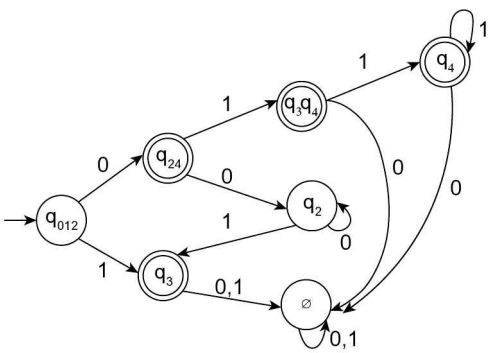
$\hat{\delta}([q_4], 1) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(\delta(q_4, 1))))$

$= [q_4]$

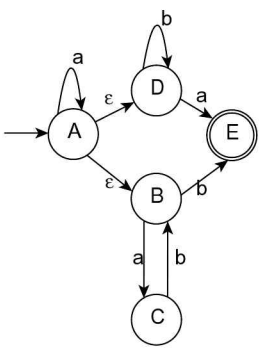
Similarly, we compute for every new state that appears, and the computed values are used to get the following table:

	a = 0	a = 1
$\rightarrow\varepsilon^*(q_0)$		
$=^* [q_0\, q_1\, q_2]$	$[q_2\, q_4]$	$[q_3]$
$^*[q_2\, q_4]$	$[q_2]$	$[q_3\, q_4]$
$^* [q_3]$	$[\varnothing]$	$[\varnothing]$
$[q_2]$	$[q_2]$	$[q_3]$
$^*[q_3\, q_4]$	$[\varnothing]$	$[q_4]$
$[q_4]$	$[\varnothing]$	$[q_4]$
$[\varnothing]$	$[\varnothing]$	$[\varnothing]$

Transition diagram for DFA is shown in the following figure:



Problem 27: Convert the following NFA with ε to DFA.



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Solution:

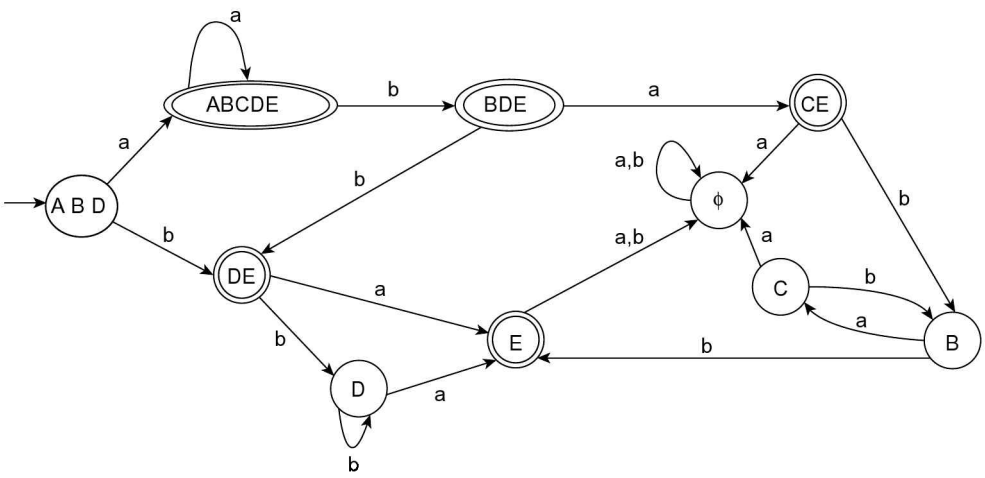
Step 1: Compute ε-closure of each state.

$\hat{\varepsilon}$ (A) = {A, B, D}
$\hat{\varepsilon}$ (B) = {B}
$\hat{\varepsilon}$ (C) = {C}
$\hat{\varepsilon}$ (D) = {D}
$\hat{\varepsilon}$ (E) = {E}

Step 2: Explore the valid states in DFA that are computed in step 2.

	i/p = a	i/p = b
→ε*(A) =[ABD]	[ABCDE]	[DE]
[ABCDE]	[ABCDE]	[BDE]
[DE]	[E]	[D]
[BDE]	[CE]	[DE]
[E]	[Ø]	[Ø]
[D]	[E]	[D]
[CE]	[Ø]	[B]
[Ø]	[Ø]	[Ø]
[B]	[C]	[E]
[C]	[Ø]	[B]

Transition diagram for DFA is shown in the following figure:



Summary

1. Finite Automata can be represented as a 5-tuple, transition table or transition diagram.
2. FA can be used as language acceptor.
3. For every NFA with ε, we can construct a NFA without ε.
4. We can construct equivalent DFA for any NFA.
5. Two finite automata are said to equivalent if the language accepted by the two machines is same.
6. Number of states in FA can be minimized by Myhill Nerode theorem or by π constructions method.
7. Moore and Mealy machines are the special finite-state machines with output.
8. In Moore machine, output is associated with state.
9. In Mealy machine, output is associated with state and input.

Short Answers

1. What is a finite automaton? Give two examples.

   Answer: Finite automaton is a mathematical model of a system consisting of finite set of state, set of transition from state to state that occurs on input symbols from alphabet.

   Examples: Text editorand lexical analyser are examples.

2. Enumerate the differences between NFA and DFA.

   Answer: DFA has moves well-defined on the current state and current input symbol. In NFA, the moves are not well defined. In NFA, multiple moves are defined either on ε or single input symbol.

3. List any four ways of theorem proving. (Nov 2007)

   Answer: The four ways of theorem proving are

   (i) Proof by induction
   (ii) Proof by contradiction
   (iii) Proof by transposition
   (iv) Proof by construction

4. What is meant by equivalent states in DFA? (Nov 2007)

   Answer: Definitions 4 and 5 of Section 1.8.2.

5. Define automata.

   Answer: Definition 1 of Section 1.2.

6. What is the principle of mathematical induction?

   Answer: See Section 1.1

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

7. Construct a DFA, over $\Sigma = \{a, b\}$, that produces not more than 3 a's.
   Answer: See Example 14.
8. Define the languages accepted by NFA and DFA
   Answer: Language of DFA $A = \{Q, \Sigma, \delta, q_0, F\}$ is defined by

$$\{w \mid \delta`(q_0, w) = P \text{ for some } P \text{ in } F\}$$

   Language of NFA $A = \{Q, \Sigma, \delta, q_0, F\}$ is defined by
   $\{w \mid \delta`(q_0, w) \cap F \neq \varnothing\}$; that is, there is at least one final state in the resultant set.

Fill in the Blanks

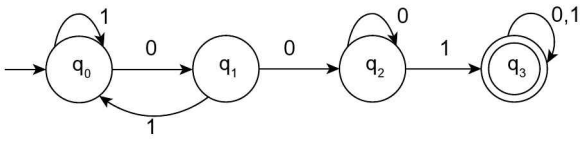
1. The number of states in DFA is _____ than the number of states in NFA for the same language.
2. The transition function for NFA is a mapping function given as _____.
3. The transition function for DFA is a mapping function given as _____.
4. The finite automata to recognize n words each of maximum length m require _____ states.
5. The time required to process a string of length x by NFA with N states is _____ compared to the time required by a DFA for the same language.
6. The minimum number of states required to accept the language $L = \{w/w \in (a + b)^*\}$ is _____ .
7. The minimum number of states required to accept the language $L = \{xay/x, y \in (a + b)^*\}$ is _____ .
8. Finite automata for accepting words 'this' and 'that' require _____ states.
9. _____ of a state is the set of states that can be reached by e-transitions.
10. NFA with ε can increase the processing time of NFA (True/False).
11. Elimination of ε-edges from NFA increases_____.
12. The two states q_1 and q_2 are said to be _____ if both $\delta(q_1, a)$ and (q_2, a) reach final states or both of them reach non-final states for all $a \in \Sigma$.
13. The output in _____ machine is associated with transition.
14. All Moore machines have an equivalent finite automata (True/False).
15. For every finite automaton, there is always an equivalent Moore and Mealy machines (True/False).

Answers

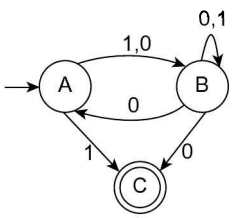
1. greater.
2. $Q \times \Sigma$ to 2^Q
3. $Q \times \Sigma$ to Q
4. 2^{mn}
5. less
6. one
7. two
8. seven
9. epsilon closure
10. true
11. number of edges
12. equivalent
13. Melay
14. False
15. True

Objective Question Bank

1. FA has
   a) Unlimited memory c) Limited Memory
   b) No memory at all d) none of the above
2. Consider the following deterministic finite-state automaton M:



   Let S denote the set of four-bit binary strings in which the first is 0 and fourth is 0. Find the number of strings in S that are accepted by M. It is
   a) 2 b) 5 c) 7 d) 8
3. How many two-state FAs can be drawn over the alphabet {0, 1} to accept an empty language ?
   a) 12 b) 14 c) 20 d) 15
4. How many two-state FAs can be drawn over the alphabet {0, 1} to acept the language $(0+1)^*$
   a) 12 b) 14 c) 20 d) 15
5. The smallest FA that accepts the language {x/length of x is divisible by 3} has
   a) 2 states b) 3states c) 4 states d) 5 states
6. How many DFAs exist with two states over the input alphabet (a, b)?
   a) 16 b) 26 c) 32 d) 64
7. How many DFAs exist with three states over the input alphabet {0, 1}?
   a) 144 b) 6561 c) 5832 d) 729
8. The recognizing capabilities of NDFSM and DFSM
   a) may be different b) must be different
   c) must be same d) none of the above
9. What is the minimum number of states of the NFA that accepts the language $\{abab^n \mid n \geq 0\} \cup \{aba^n \mid n \geq 0\}$?
   a) 4 b) 3 c) 5 d) 9
10. What is the minimum number of states in the NFA accepting the language{ab, abc}?
    a) 4 b) 5 c) 3 d) 2
11. Consider the NFA M shown below:



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

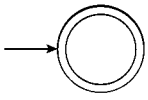
Let the language accepted by M be L. Let L_1 be the language accepted by the NFA M_1 obtained by changing non-accepting states of M to accepting states. Which of the following statements is true?

a) $L_1 = \{0, 1)^* - L$ b) $L_1 = \{0, 1\}^*$
c) $L_1 \neq L$ d) $L_1 = L$

12. The basic limitation of FSM is that
    a) It cannot remember arbitrarily large amount of information.
    b) It sometimes recognizes grammars that are not regular.
    c) It sometimes fails to recognize grammars that are regular.
    d) All of the above comments are true.

13. Given an arbitrary NFA with N states, the maximum number of states in an equivalent minimized DFA is at least
    a) N^2 b) N c) 2^N d) N!

14. The FSM pictured below recognized



    a) all strings b) no string
    c) ε alone d) None of the above

15. The number of states of the FSM required to simulate the behaviour of a computer, with a memory capable of storing 'm' words, each of length 'n' bits, is
    a) $m \times 2^n$ b) 2^{mn}
    c) 2^{m+n} d) None of the above.

16. Consider the following 2-DFA $(\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \delta\, q_0, \{q_2\})$, where δ is

	0	1
$\rightarrow q_0$	q_0, R	q_1, R
q_1	q_1, R	q_2, R
$*\, q_2$	q_2, R	q_3, L
q_3	q_4, L	q_3, L
q_4	q_0, R	q_4, L

Which of the following strings is accepted by the above FA?

a) 1100011001000 b) 1100000000111
c) 1000001110000 d) 1100001100110

17. Choose the incorrect statement
    a) Moore and Melay machines are FSMs with output capability.
    b) Any given Moore machine has an equivalent Melay machine.
    c) Any given Melay machine has an equivalent Moore machine.
    d) Moore machine is not a FSM.

18. The major difference between Moore and Melay machines is that
    a) The output of the former depends on the present state and present input.
    b) The output of the former depends only on the present state.
    c) The output of the former depends only on the present input.
    d) None of the above.

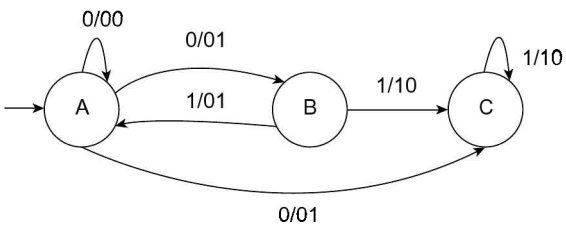
19. An FSM with output capability can be used to add two given integers in binary representation. This is
    a) True b) False
    c) May be true d) None of the above

20. A finite-state machine has a single input x, single output Z and the following state-table:

Present state	Next state, Z	
	x = 0	x = 1
A	D, 0	B, 0
B	B, 1	C, 1
C	B, 0	D, 1
D	B, 1	C, 0

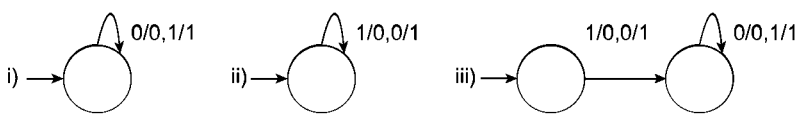
    If the initial state is unknown, then the shortest input sequence to reach the final state C is
    a) 01 b) 10 c) 101 d) 110

21. The finite-state machine described by the following state diagram with A as starting state, where an arc label is denoted by x/y (x stands for 1-bit input and y stands for 2-bit output).



    a) Outputs the sum of the present previous bits of the input.
    b) Outputs 01 whenever input sequence contains 11.
    c) Outputs 00 whenever input sequence contains 10.
    d) None of the above.

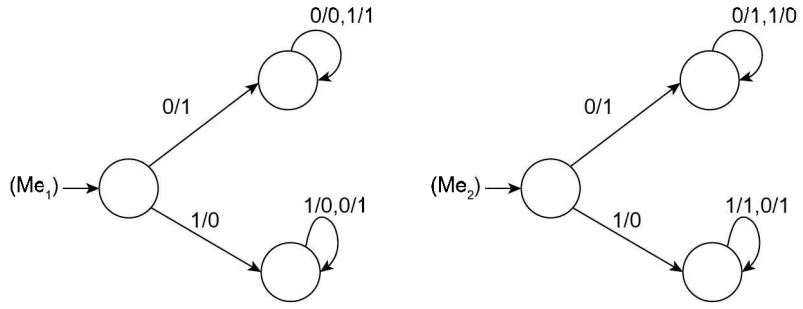
22. Let $(Me)^2$ mean that given a Melay machine, an input string is processed and then the output string immediately is fed into the machine (as input) and reprocessed. Only this second resultant output is considered the final output of $(Me)^2$. If the final output string is the same as the original input string, we say that $(Me)^2$ has an identity property. Symbolically, we write $(Me)^2$ = identity. Consider the following machines.



    Which of the above machines have the identity property?
    a) i) only
    b) i) and ii) but not iii)
    c) i) and iii) but not ii)
    d) All have identity property.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

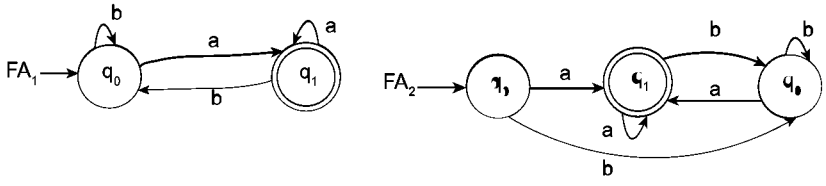
23. Let (Me_1) (Me_2) mean that an input string is processed on Me_1 and then the output string is immediately fed into Me_2 (as input) and reprocessed. Only this second resultant output is considered the final output of $(Me_1)(Me_2)$. If the output string is the same as the original input string, we say that $(Me_1)(Me_2)$ has the identity property, symbolically written $(Me_1)(Me_2)$ = identity. Consider following machines.



Which of the following is most appropriate?

a) $(Me_1)(Me_2) = (Me_2)(Me_1)$
b) (Me_2) is the inverse machine of Me_1
c) (Me_1) is the inverse machine of Me_2.
d) All of the above are false.

24. Consider the following FAs:



Which of the following is true?

a) $FA_1 \subset FA_2$ b) $FA_2 \subset FA_1$
c) $FA_1 = FA_2$ d) none of the above

25. Consider a DFA over $\Sigma = (a, b)$ accepting all strings which have number of a's divisible by 6 and number of b's divisible by 8. What is the minimum number of states that the DFA will have?

a) 8 b) 14 c) 15 d) 48

26. What is the number of states in the minimized DFA that accepts all strings whose 7th symbol from Right end is 1

a) 254 b) 256 c) 8 d) 237

27. Which of the following statements is true?

a) The union of two equivalence relations is also an equivalence relation.
b) Regularity is preserved under the operation of string reversal.
c) All subsets of regular sets are regular.
d) A minimal DFA that is equivalent to a NFA with n nodes has always 2^n states.

28. Identify correct statements from the following.

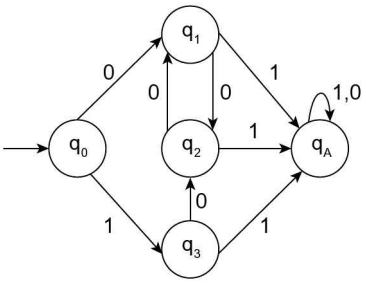
I. A FSM can be designed to add two integers of any arbitrary length (arbitrary number of digits).

II Every subset of a countable set is countable.

a) I only b) Neither I nor II

c) II only d) I and II

29. How many states are there in minimized DFA of the following DFA:



a) 2 b) 4 c) 3 d) 5

30. Consider the sequential machine M given below:

	0	1
A	C, 0	B, 1
B	F, 1	D, 1
C	D, 0	E, 1
D	C, 0	F, 1
E	D, 1	C, 1
F	C, 1	C, 1
G	C, 1	D, 1

Which of the following sequential machines is the standard (i.e. reduced equivalent machine of M)?

a)

	0	1
A	B, 0	C, 1
B	B, 0	D, 1
C	D, 1	B, 1
D	B, 1	B, 1

b)

	0	1
A	B, 0	C, 1
B	D, 1	B, 1
D	A, 1	A, 1
E	D, 1	A, 0

c)

	0	1
A	B, 0	C, 1
B	B, 0	C, 1
C	C, 1	B, 1

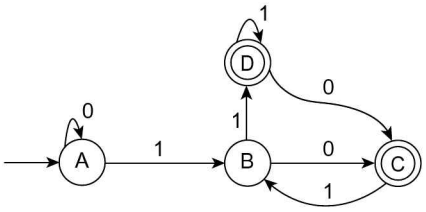
d)

	0	1
A	B, 0	C, 1
B	B, 0	C, 1
C	C, 1	B, 1

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

31. One fine day I draw a DFA for a language which I know very well and place in the house. On that night my house was robbed. The robber did the following:

He just removed one of non-starting states from my DFA, removed all edges associated with it, redrew it on another paper and robbed my original paper. Next day morning I found that my paper was robbed and saw another FA there. From that day I am thinking what language could be accepted by that FA. My original DFA is shown below.



Can you help me in finding out the state to be removed to satisfy the following.

i. The finite automata accepts all strings whose last but one symbol is 1.
ii. The finite automata accepts all strings which end with 11.

a) A b) B c) C d) D

Answers

1. c	2. a	3. c	4. c	5. b	6. d	7. c	8. c	9. c	10. a
11. b	12. a	13. c	14. c	15. b	16. d	17. d	18. b	19. a	20. a
21. a	22. d	23. d	24. c	25. d	26. b	27. b	28. d	29. c	30. a
31.c									

Exercises

1. Define a DFA with an example?
2. Design a DFA which accepts any number of a's followed by a string ba followed by a string a's and b's?
3. What is a transition graph and transition table?
4. Obtain a DFA to accept strings of a's and b's, except those containing a substring aab?
5. Design a DFA to accept strings having
   a. exactly one a
   b. at least one 'a'
   c. not more than 3a's
6. Obtain a DFA to accept strings of 0's, 1's and 2's, beginning with a 1, followed by odd number of 0's and ending with a 2?
7. Obtain a DFA to accept strings starting with two 0's and ending with at least two 1's?
8. Obtain a DFA to accept a strings of a's and b's with at most two consecutive b's [abababb]?
9. What is a NFA, write the procedure to convert a NFA to a DFA?

10. Obtain a DFA to accept the integer numbers represented in binary and is a multiple of 5.
11. Obtain a DFA to accept a language L that has strings of a's and b's ending with ab.
12. Construct a DFA to test whether a binary number is
    a. Divisible by 6
    b. Divisible by 7
13. Obtain a DFA whose number of a's is divisible by 7 and number of b's is divisible by 5?
14. Obtain a DFA to strings that begin or end with 0, 1?
15. Construct a DFA to accept the strings over z = {a, b} with the property that every block of length of 5 contains at least two a's?
16. Write the differences between DFA and NFA?
17. Obtain a DFA to accept a language L that has strings of a's and b's ending with ab or ba.
18. Convert the following NFA into its equivalent DFA.
19. Define epsilon-closure?
20. Obtain a DFA to accept a language L that has strings with a's and b's such that difference in number of a's and b's is multiple of 3.
21. Obtain a DFA to accept the language L = {awa |w is string with a's and b's}
22. Convert the given ε -NFA to its equivalent DFA.
23. Define the following terms:
    a. DFA
    b. NFA
    c. ε-NFA
24. Construct DFAs that accept the following languages on the alphabet $\Sigma = \{a, b\}$
    a. all strings with exactly one a
    b. all strings with at least one a
    c. all strings except those which end with abb
25. Convert to DFA the following NFA:

	0	1
q_0	q_1	ϕ
$*q_1$	q_0	q_1, q_2
q_2	ϕ	q_1

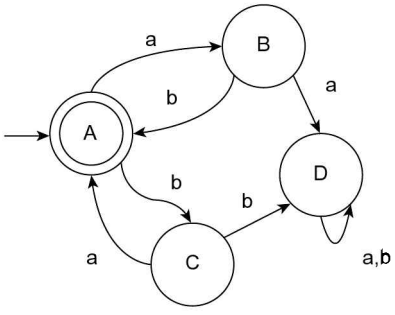
26. Define finite automata. Discuss why we study Automata.
27. Design the DFAs for the following languages.
    a. L = {w/|w| mod 3 = 0, w is string with a's and b's}
    b. set of all strings (on the alphabet $\Sigma = \{0, 1\}$) that either begin or end, or begin as well as end with the substring 01
    c. decimal strings divisible by 3
28. Explain the procedure to construct DFA for the given NFA using subset construction scheme.
29. Show that every positive integer can be expressed as a product of prime numbers.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

30. Design a NFA to recognize the following set of the strings:
    a. abc, abd, aacd
    b. 0101, 101, 011 $\Sigma = \{0, 1\}$
31. Obtain a DFA to accept binary numbers (over the alphabets {0, 1}) that are divisible by 5 and start with 1.
    Hint: 101, 1011, 101011 strings should be valid, 0101, 01011 are not valid.
32. Obtain a DFA to accept a language L = {w/|w|mod 3 > = |w|mod 2} where w $\in \Sigma^*$ and $\Sigma = \{a, b\}$
    Hint: Strings of length 3, 9, 15, 21 are not valid or in general 3 + 6 n/n ≥ 0 are not valid.
33. Consider the DFA

	0	1
→A	B	A
B	A	C
C	D	B
*D	D	A
E	D	F
F	G	E
G	F	G
H	G	D

    a. Draw the table of distinguishabilities for the DFA
    b. Construct the minimum state-equivalent DFA.
34. Find the language accepted by the following and prove that the FA generates strings where each prefix has at most one more a than b's and atmost one b more than a's.



    Hint: This can be proved by deducing the properties of the strings, taking the automaton to each of four states and using mathematical induction on the length of the string.
35. In converting NFA to DFA, the number of states may increase substantially. Give upper and lower bounds on the maximum increase in number of states for an n-state NFA.
    Hint: Consider the Theorem 1 in 2.4 and the Example 2.17.

Regular Languages and Regular Grammars

3

- Regular Expressions are shorthand notations to describe a language. They are used in many programming languages and Language tools such as lex, vi editor, PHP and PERL. They are used as powerful tool in search engines.

In this chapter we introduce Regular Expression (RE), Regular Sets and representation of regular sets using regular expressions. Operations on regular expressions along with identity rules are described. Equivalence of Finite Automata (FA) and regular expressions & equivalence of Regular expressions and finite automata are explained. Interconversion between regular expression to NFA and DFA to Regular expression using recursive procedure, Arden's theorem and generalized non-deterministic finite automata is explained. Pumping lemma and its use in identifying regular languages are discussed. Closure properties of regular languages and the applications of regular expression are described.

3.1 Regular Expressions

Regular Expressions are useful for representing certain sets of strings in an algebraic fashion. RE describes the language accepted by finite automata.

Definition of Regular Expression over Σ

- Any terminal symbol/element of Σ is RE
  Example: Φ, ϵ, a in Σ
  Φ is a regular expression and denotes the empty set.
  ϵ is a regular expression and denotes the set $\{\epsilon\}$.
  a is a regular expression and denotes the set {a}.
- **Union** of two regular expressions R_1 and R_2 is a regular expression R ($R = R_1 + R_2$).
  Example: Let a be a regular expression, R_1.
  Let b be a regular expression, R_2.
  (a + b) is also a regular expression, R, having the elements {a, b}.
- **Concatenation** of two regular expressions R_1 and R_2 (written as $R_1 \cdot R_2$) is also a regular expression (i.e. $R = R_1 \cdot R_2$)
  Example: Let a be a regular expression R_1.
  Let b be a regular expression R_2.
  (a.b) is also a regular expression R having the element {ab}.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

- **Iteration** (Closure) of a regular expression R (written as R*) is also a regular expression.

  Let a be regular expression.
  Then ϵ, a, aa ... are also regular expressions.
  If L is a language represented by the regular expression R then the Kleene closure of L is denoted as L* and is given by

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

  The positive closure of L, denoted L^+, is the set

$$L^+ = \bigcup_{i=1}^{\infty} L^i$$

- If R is a regular expression, then (R)* is also a regular expression.
- Regular expressions over Σ are precisely those expressions obtained recursively by the application of the above rules, once or several times.

The basic operations performed on regular expressions are Union, Concatenation and Kleene closure. Among these three operations, closure has highest precedence, next highest is for concatenation and least precedence is for union.

Example: An identifier in any high level language is given by RE.

An identifier starts with a letter and is followed by any number of letters or digits. The regular expression can be given as l(l|d)* where l indicates letters a to z and d indicates digits from 0 to 9.

3.2 Regular Sets

Definition 2: Any set represented by a regular expression is called a regular set.

If a, b are the elements of Σ, then regular expressions

- a denotes the set {a}.
- a + b denotes the set {a, b}.
- ab denotes the set {ab}.
- a* denotes the set {ε, a, aa,.....}.
- (a + b)* denotes the set {ε, a, b, aa, ab, ba, bb, aaa, ...}.

Examples for representing regular sets by regular expressions are given in the table below.

Regular Set	Regular Expression
{101}	101
{ε, a}	ε + a
{ε, a, aa, ab, ba, bb, ...}	(a + b)*
{ab, ba}	ab + ba

We depict some more sets by regular expressions in the following list.

1. All strings of 0's and 1's

Solution: The language has the elements of {ε, 0, 1, 00, 01, 10, 11, 000,.....}. Hence the regular expression is **(0 + 1)***.

2. Set of all strings of 0's and 1's ending in 00

Solution: The language has the elements of {00, 000, 100, 0000, 0100, 1000, 1100, 00000,.....}. This can be written as {ε, 0, 1, 00, 01, 10, 11, 000,.....}00.
Hence the regular expression is **(0 + 1)*00**.

3. Set of all strings of 0's and 1's beginning with 0 and ending with 1

Solution: The language has the elements of {01, 001, 011, 0001, 0011, 0101, 0110, 00001,.....}. This can be written as 0{ε, 0, 1, 00, 01, 10, 11, 000,.....}1.
Hence the regular expression is **0(0 + 1)*1**.

4. Set of all strings having even number of 1's

Solution: The language has the elements of {ε, 11, 1111, 111111, 11111111, ...}. Hence the regular expression is **(11)***.

5. Set of all strings having odd number of 1's

Solution: The language has the elements of {1, 111, 11111, 1111111, 111111111, ...}. This can be written as {ε, 11, 1111, 111111, 11111111, ...}1
Hence the regular expression is **1(11)* or (11)*1**.
Note: 1* is wrong as (11)* + 1(11)* = 1*.

6. Strings of 0's and 1's with at least two consecutive 0's

Solution: The language has the elements of {00, 000, 001, 100, 0000, 0010, 0011, 0100, 1000, 1100, 00001,.....}. These strings can be generated by the regular expression **(0 + 1)*00(0 + 1)***.

7. All strings of 0's and 1's beginning with 1 or 0 and not having two consecutive 0's

Solution: 11111……..1* is allowed. 101010……..(10)* is allowed.
RE for strings with 0's and 1's that do not have two consecutive 0's is (1 + 10)*.
Since the string may either start with 0 or 1 the expression can be written as (0 + ε) (1 + 10)*.

8. Set of all strings ending with 011

Solution: The language has the elements of {011, 0011, 1011, 00011, 01011, 10011, 11011, ...}. This can be written as {ε, 0, 1, 00, 01, 10, 11, 000,.....} 011.
Hence the regular expression is **(0 + 1)*011**.

9. Set of all strings with 0's followed by 1's, followed by 2's such that it has at least one 0 followed by at least one 1 followed by at least one 2

Solution: The language has the elements of {012, 0012, 0112, 0122, 00112, 00122, 01122, 001122,.....}.
Hence the regular expression is $\mathbf{0^+ 1^+ 2^+}$.

10. Set of all strings of 0's and 1's whose last two symbols are the same

Solution: The language has the elements of {00, 11, 011, 000, 0011, 0111, 0100, 0000,.....}.
Hence the regular expression is **(0 + 1)*(00 + 11)**.

11. Set of strings in which every 0 is immediately followed by at least two 1's

Note: To find the RE consider two Possibilities:
i) Strings with only 1's
ii) Every 0 preceded by 11 (i.e. 011)
Hence the regular expression is (1 + 011)*.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

12. Strings of 0's and 1's with a substring 1100

Solution: The language has the elements {1100, 11000, 11001, 11100, 110000, 110010, 110011, 011100, 111000, 111100, 1100001,.....}. These strings can be generated by a regular expression **(0 + 1)*1100 (0 + 1)***.

3.3 Identity Rules for Regular Expressions

P and Q are two equivalent regular expressions (i.e. P and Q represent the same set of strings), then to simplify the regular expressions, the following identity rules can be used:

1. $\Phi + R = R, \varepsilon + R = R + \varepsilon$
2. $\varepsilon R = R\varepsilon = R$
3. $R + R = R$
4. $RR^* = R^*R = R^+$
5. $\varepsilon + RR^* = R^* = \varepsilon + R^*R$
6. $\Phi R = R\Phi = \Phi$
7. $\varepsilon^* = \varepsilon, \quad \Phi^* = \varepsilon$
8. $(R^*)^* = R^*$
9. $(PQ)^*P = P(QP)^*$
10. $(P + Q)R = PR + QR$
11. $R(P + Q) = RP + RQ$
12. $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$

3.4 Algebraic Laws for Regular Expressions

The above identity rules are useful to express the regular expression in a simplified form. But to simplify the expression by applying the identity rules, the expression should be in the required form. This requires the expressions to be rewritten without changing the language. To identify the ways in which the expression can be modified, we can apply the following associative rules. The expression obtained is equivalent to the original expression.

1. Union operation on regular expressions are commutative i.e. $r + s = s + r$.
2. Union operation on regular expressions are associative i.e. $(r + s) + t = r + (s + t)$.
3. Concatenation operation on regular expressions are associative i.e. $r(st) = (rs)t$
4. Concatenation is right distributive over union i.e. $(r + s).t = rt + st$
5. Concatenation is left distributive over union i.e. $t.(r + s) = tr + ts$
6. $\Phi^* = \epsilon$

Example 3.1 Prove or disprove following:

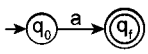
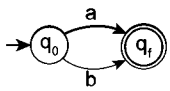
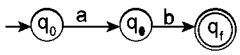
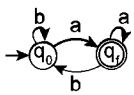
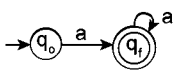
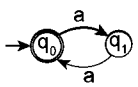
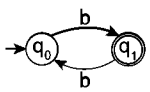
$$(r + s)^* = r^* + s^*$$

Regular expression on LHS accepts strings formed by r and s in any form. RE on RHS accepts strings formed ony with r's or only with s's. For example, **rs** $\in$ LHS but **rs** $\notin$ RHS. Hence $(r + s)^* = r^* + s^*$ is not true.

3.5 Equivalence of Finite Automata with Regular Expressions

Regular expressions and finite automata are equivalent in their descriptive power. The equivalence can be proved by mathematical induction on the size of a regular expression (induction enables us to prove that there is a NFA with ε-transitions denoting the same language). That is, for any given regular expression, we can construct its equivalent finite automaton that recognizes the language it describes and vice versa. The following Figure 3.1 shows the relation between NFA, RE and DFA. Table 3.1 relates the FA with corresponding Regular set/regular expression.

Table 3.1 *Relation between FA, Rs & RE*

S. No.	Finite automata	Regular set	Regular expression
1.		ϕ	Φ
2.		{ε}	ε
3.		{a}	a
4.		{a, b}	a + b a | b
5.		{ab}	a.b
6.		{a, aa,…., ba, bba, … baba…}	b*a(a*bb*a)* or (b*aa*b)*b*a
7.		{ϵ, a, aa, aaa,…….}	a*
8.		{a, aa, aaa, ….}	a^+
9.		{ϵ, aa,$(aa)^2$, …}	(aa)*
10.		{b, bbb, $b(bb)^2$, …..}	b(bb)*

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

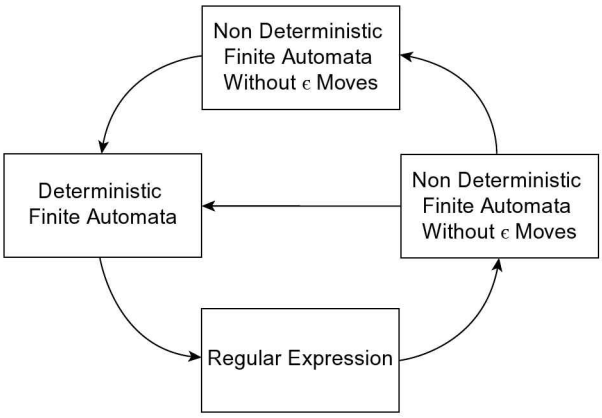


Fig. 3.1 *Equivalance of FA & RE*

Theorem 1: Let r be a regular expression Then there exists a NFA with ε-transitions that accepts L(r).

Proof: This statement can be proved by **mathematical induction.**

To show that it is true, let k = Ø. The corresponding FA is shown in Table 3.1-1.

Let k = 0. The string with length zero is ε and its corresponding FA is shown in Table 3.1-2.

Let k = 1. The string of length 1 in Σ {a, b} is either a or b, and the corresponding FA is shown in Table 3.1-4.

Let us assume that Theorem 1 is true for k = 2, 3, 4,.....i–1. We have to show that Theorem 1is true for k = i. To prove this we have to consider three possible operations applied on regular expression and show that for each there exists a FA.

Case 1: Let r_1 and r_2 be two regular expressions with less than i operations that have FAs $M_1 = \{Q_1, \Sigma_1, \delta_1, q_1, \{f_1\}\}$ and $M_2 = \{Q_2, \Sigma_2, \delta_2, q_2, \{f_2\}\}$. We show that r formed using union operation ($r = r_1 + r_2$) has automaton M that accepts the language $L(M_1) \cup L(M_2)$ is shown in Figure 3.2.

$M = \{\{Q_1 \cup Q_2 \cup \{q_0, f_0\}\}, \{\Sigma_1 \cup \Sigma_2\}, \delta, q_0, \{f_0\}\}$ where δ is defined by

1. $\delta(q_0, \varepsilon) = \{q_1, q_2\}$
2. $\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$.
3. $\delta(q, a) = \delta_2(q, a)$ for q in $Q_2 - \{f_2\}$ and a in $\Sigma_2 \cup \{\varepsilon\}$.
4. $\delta(f_1, \varepsilon) = \delta_2(f_2, \varepsilon) = \{f_0\}$

By inductive hypothesis, there are no transitions out of the final states. Thus, all moves of M_1 and M_2 are present in M. Any string x valid in M_1 or M_2 must be valid in M. For such a string, there exists a path from q_0 to f_0. To prove this, observe rule 1 which shows path from q_0 to either q_1 or q_2. Since x is valid in either M_1 or in M_2, there exists a path from q_1 to f_1, or from q_2 to f_2. By rules 2 and 3 all these transitions are also present in M. By rule 4 there is a path from f_1 and f_2 to $f_0$on ε. Hence there is a path from initial state to final state, which indicates x is also valid in M.

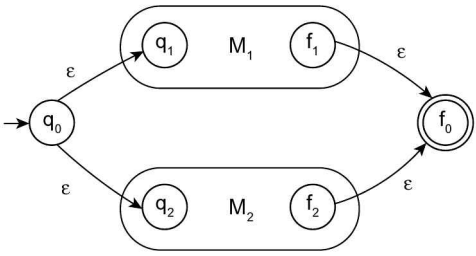


Fig. 3.2 *FA for Union*

Case 2: Let r_1 and r_2 be two regular expressions (with less than i operations) that have FAs M_1 and M_2 given by $M_1 = \{Q_1, \Sigma_1, \delta_1, q_1, \{f_1\}\}$ and $M_2 = \{Q_2, \Sigma_2, \delta_2, q_2, \{f_2\}\}$. We show in Figure 3.3 that r formed using concatenation operation ($r = r_1.r_2$) has automaton M that accepts the language $L(M_1).L(M_2)$:

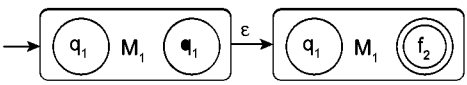


Fig. 3.3 *FA for Concatenation*

We note that $M = \{\{Q_1 \cup Q_2\}, \{\Sigma_1 \cup \Sigma_2\}, \delta, q_1, \{f_2\}\}$ where δ is defined by

1. $\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$.
2. $\delta(f_1, \varepsilon) = \{q_2\}$
3. $\delta(q, a) = \delta_2(q, a)$ for q in Q_2 and a in $\Sigma_2 \cup \{\varepsilon\}$.

Any string w = xy is valid if x is in M_1 and y is in M_2. For such a string, there exists a path from q_1 to f_2 where q_1 is initial state and f_2 is final state in M. To prove this, observe rule 1 which shows path from q_1 to f_1 on x as it is valid in M_1. By rule 2 there is path from f_1 to q_2 on ε. Rule 3 includes path from q_2 to f_2 on string y. Hence there is path from initial state q_1 to final state f_2 which is the concatenated string x and y. Since w = x.y, w is a valid string in M.

Case 3: Let r_1 be a regular expression with less than i operations that has FA given by $M_1 = \{Q_1, \Sigma_1, \delta_1, q_1, \{f_1\}\}$. We show in Figure 3.4 that r formed using repeated concatenation operation ($r = r_1^*$) has automaton M that accepts the language $L(M_1)^*$:

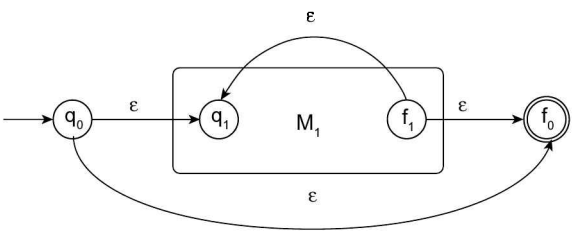


Fig. 3.4 *FA for Closure*

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

We note that $M = \{\{Q_1 \cup \{q_0, f_0\}\}, \Sigma_1, \delta, q_0, \{f_0\}\}$ where δ is defined by

1. $\delta(q_0, \varepsilon) = \delta(f_1, \varepsilon) = \{q_1, f_0\}$
2. $\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$.

Any string w = {ε, x, xx,..} is valid if x is in M_1. For such a string, there exists a path from q_0 to f_0. If string is ε then there is path from q_0 to f_0 using rule 1. For string x there is a path from q_0 to q_1 on ε, q_1 to f_1 on x and f_1 to f_0 on ε. To accept string xx, xxx then on reaching to state f_1, path is taken from f_1 to q_1 on ε to accept next x. Hence $L(M) = L(M_1)^*$.

Example 3.2 Construct NFA for the regular expression (1 + 0)0*.

Solution: The given expression can be expressed as a regular expression formed by applying a sequence of operations on individual expression as shown below:

$R = (R_1 + R_2)R_3^*$ where $R_1 = 1$, $R_2 = 0$ and $R_3 = 0$

The FA for the expressions R_1, R_2, R_3 can be constructed as shown in Figure 3.5.

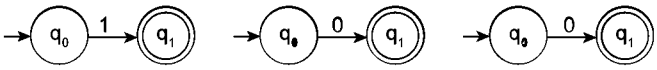


Fig. 3.5 *FA for R_1, R_2 & R_3*

Applying union rules for $(R_1 + R_2)$, we get FA as shown in Figure 3.6.

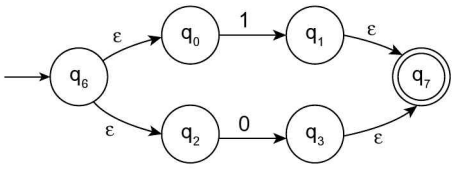


Fig. 3.6 *FA for $(R_1 + R_2)$*

Applying closure rule for R_3 to get R_3^* as shown in Figure 3.7.

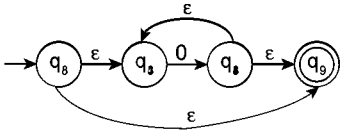


Fig. 3.7 *FA For R_3^**

Applying concatenation rules, we get the final NFA as shown in Figure 3.8.

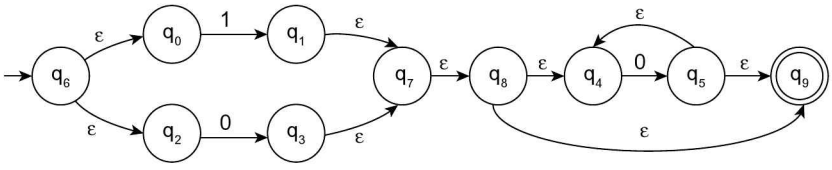


Fig. 3.8 *FA for (0 + 1)0**

3.6 Constructing Regular Expression for Given DFA

Theorem 2: Let M be a deterministic finite automaton. Then there exists regular expression R that generates strings that are accepted by M. that is, L(R) = L(M).

Proof: Let the language L accepted by the DFA M (defined by M = ($\{q_1, q_2, \ldots q_n\}$, Σ, δ, q_1, F)). We have to construct RE equivalent to it. Find R^k_{ij} where i is initial state and j is final state. While moving from state i to state j, it would go through k states, and k is not greater than the number of states in the automaton.

Start with the transition table for the DFA with the following state naming conventions: the first state is 1 or q_1 or s_1, which is the starting state. States are numbered consecutively, 1, 2, 3, ..., n. The transition table is a typical DFA where the table entries are sets of states, and Φ, the empty set, is allowed.

The set of final states F must be known. We call the variable r a regular expression. We may denote by r_{ij} (with i, j subscripts) as a regular expression. Note r_{12} is just a (possibly) different regular expression from r_{53}. Since we need to build multiple columns of REs, we will use a superscript in naming of regular expressions.

Hence, for us, $r^1_{12}.r^3_{64}, r^k_{1k}, r^{k-1}_{ij}$ are the names of diffrent regular expressions. We have to build a table with n^2 rows and n + 1 columns.

	K = 0	K = 1	K = 2	.....	K = n
r_{11}	r^0_{11}	r^1_{11}	r^2_{11}	.....	r^n_{11}
.....	.....	.....	.....	.....	.....
r_{1n}	r^0_{1n}	r^1_{1n}	r^2_{1n}	.....	r^n_{1n}
r_{21}	r^0_{21}	r^1_{21}	r^2_{21}	.....	r^n_{21}
.....	.....	.....	.....	.....	.....
r_{2n}	r^0_{2n}	r^1_{2n}	r^2_{2n}	.....	r^n_{2n}
.....	.....	.....	.....	.....	.....
r_{n1}	r^0_{n1}	r^1_{n1}	r^2_{n1}	.....	r^n_{n1}
.....	.....	.....	.....	.....	.....
r_{nn}	r^0_{nn}	r^1_{nn}	r^2_{nn}	.....	r^n_{nn}

Note: There will be n^2 rows, all pairs of numbers from 1 to n as r_{11}, r_{12}...r_{1n} and so on. Now to build the entries for the column K = 0 we use.

$$r^0_{ij} = \{a / \delta(q_i, a) = q_j\} \quad \text{if } i \neq j$$

$$\{a / \delta(q_i, a) = q_j\} \cup \{\varepsilon\} \quad \text{if } i = j$$

where δ is the transition table function a is some symbol from Σ and q is some state.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Let us note r_{ij}^0 would be either $\varnothing$ / ε / a or $(a + \varepsilon)$ notice there is no Kleene Star or concatenation in this column.

Next, build the k = 1 column:

$$r_{ij}^1 = r_{i1}^0 (r_{11}^0)^* r_{1j}^0 + r_{ij}^0$$

Next, build the k = 2 column:

$$r_{ij}^2 = r_{i2}^1 (r_{22}^1)^* r_{2j}^1 + r_{ij}^1$$

repeat the process of finding expressions for k columns.

$$r_{ij}^k = r_{ik}^{k-1} (r_{kk}^{k-1})^* r_{kj}^{k-1} + r_{ij}^{k-1}$$

Note: All items are from the previous column.

Finally, for final states p, q, r, the regular expression is

$$r_{1p}^n + r_{1q}^n + r_{1r}^n$$

Where 1 is initial state and n is number of states in automata.

Note that this is from a constructive proof that every DFA has a language for which there is a corresponding regular expression.

Some minimization rules for regular expressions can be applied at every step using identity rules of regular expressions.

$$x^* (a + b) + (a + b) = x^* (a + b)$$

$$x^* y + y = x^* y$$

$$(x + \varepsilon)x^* = x^* (x + \varepsilon) = x^*$$

$$(x + \varepsilon)(x + \varepsilon)^* (x + \varepsilon) = x^*$$

Example 3.3 Construct RE for the DFA shown in Figure 3.9.

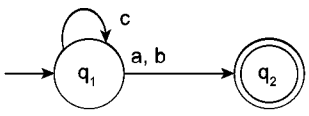


Fig. 3.9 *FA for c*(a+b)*

$r_{11}^0 = c + \varepsilon$, $r_{12}^0 = a + b$, $r_{21}^0 = \Phi$, $r_{22}^0 = \varepsilon$

$r_{11}^1 = r_{11}^0 (r_{11}^0)^* r_{11}^0 + r_{11}^0 = (c + \varepsilon)(c + \varepsilon)^*(c + \varepsilon) + (c + \varepsilon) = (c + \varepsilon)^* = c^*$

$r_{12}^1 = r_{11}^0 (r_{11}^0)^* r_{12}^0 + r_{12}^0 = (c + \varepsilon)(c + \varepsilon)^*(a + b) + (a + b)$

$= ((c + \varepsilon)(c + \varepsilon)^* + \varepsilon)(a + b) = c^*(a + b)$

$r_{21}^1 = r_{21}^0 (r_{11}^0)^* r_{11}^0 + r_{21}^0 = \Phi(c + \varepsilon)^*(c + \varepsilon) + \Phi = \Phi$

$r_{22}^1 = r_{21}^0 (r_{11}^0)^* r_{12}^0 + r_{22}^0 = \Phi(c + \varepsilon)^*(a + b) + \varepsilon = \varepsilon$

Since there are only two states and the final state is 2, it is sufficient if regular expression for r_{12}^2 is computed.

$$r_{12}^2 = r_{11}^1 (r_{11}^1)^* r_{12}^1 + r_{12}^1 = c^*(c^*)^*c^*(a + b) + c^* (a + b) = c^*(a + b)$$

	K = 0	K = 1	K = 2
r_{11}	$c + \varepsilon$	c^*	
r_{12}	$a + b$	$c^*(a + b)$	$c^*(a + b)$
r_{21}	Φ	Φ	
r_{22}	ε	ε	

The final regular expression minimizes to c* (a + b)

3.6.1 Arden's Theorem

This theorem is useful to simplify regular expressions. Let C and B be two REs over Σ and let A be unknown. If C does not contain ε, then the equation for A given as A = B + AC has a unique solution given by

$$A = BC^*$$

Proof: Let us verify whether A = BC* is a unique solution,

Substitute A = BC* in A = B + AC

B + (BC*) C + B (ε + C*C) = BC*

Hence, A + BC is satisfied when A = BC*. So it is a solution of the given equation.

To prove uniqueness, replace A by B + AC

$A = B + (B + AC)C = B + BC + AC^2$

$= B + BC + (B + AC)C^2 = B + BC + BC^2 + AC^3$

$= B + BC + BC^2 + (B + AC)\,C^3 = B + BC + BC^2 + BC^3 + AC^4$

$= B\,(\varepsilon + C + C^2 + C^3$------$) = BC^*$

Uniqueness is now proved.

Example 3.4 Given a RE which represents set L of strings in which every 0 is immediately followed by at least two 1's as (1 + 011)*, prove that the RE ε + 1*(011)* (1* (011)*)* also represents the same expression.

Let P = 1*(011)*

The given expression is represented as ε + PP* = P*

Hence P* = (1*(011)*)*

Let P = 1 and Q = 011. Then the expression can be represented as

(P*Q*)* = (P + Q)* Using the identity rule (P + Q)* = (P*Q*)* = (P* + Q*)*

= (1 + 011)* Hence proved.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

3.6.2 Arden's Theorem in Construction of RE

This is simple procedure that involves framing equations and solving them. Equations are framed using the following conditions:

i. If q_i and q_j are two nodes and there is an edge labelled a from q_i to q_j, then add the equation $q_j = q_i\, a$.
ii. If q_j has edges from $q_1, q_2, \ldots q_i$ with labels $a_1, a_2, \ldots a_i$, then add the equation $q_j = q_1a_1 + q_2a_2 + \ldots\ldots q_ia_i$.
iii. Add ε if the state is initial state.

Using these set of rules, we get n equations for n states that can be solved by elimination and applying Arden's theorem.

Using identity rules of regular expressions reduce the equation to $q_i = \alpha_i + q_i\, \beta_i$ where β_i is not ε. Then this can be replaced as $q_i = \alpha_i\, \beta_i{}^*$

Solve for all q_i where $i = 0, 1, \ldots\ldots n$. Take union of the REs for all $q_i \in F$.

Example 3.4 Construct regular expression for the DFA shown in Figure 3.10.

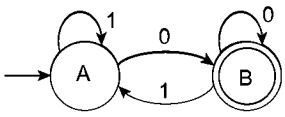


Fig. 3.10 *FA for Strings Ending with 0*

The equations that can be framed are

$$A = B1 + A1 + \varepsilon \quad (1)$$
$$B = A0 + B0 \quad (2)$$

Equation (2) is in the required form to apply Arden's theorem.

$$\begin{aligned} B &= A0 + B0 \\ &= A0(0)^* \end{aligned} \quad (3)$$

Substituting the expression of B in Equation (1) we get

$$\begin{aligned} A &= (A0(0)^*)1 + A1 + \varepsilon \\ &= \varepsilon + (A0(0)^*)1 + A1 \\ &= \varepsilon + A(00^*1 + 1) \\ &= \varepsilon\,(00^*1 + 1)^* \\ &= ((00^* + \varepsilon)\,1)^* \\ &= (0^*1)^* \end{aligned}$$

Substitute the expression of A in (3), we get

$$B = (0^*1)^*00^*$$

The regular expression for the given DFA is **$(0^*1)^*00^*$**.

3.6.3 Construction of RE Using Generalized NFA

Procedure

1. For the given DFA, construct its equivalent GNFA by adding two new states (one initial and one final) making the number of states n + 2 where n is the number of states in the given DFA.
2. From the initial state, add a ε edge to the initial state of DFA.
3. From every final state of DFA, add ε edges to the new final state.
4. Let the total number of states in the GNFA be n. Convert this to GNFA with n – 1 states by removing one state excluding new initial and new final state. On removing a state V_i, add the new edges from state V_j to V_k if there is a path via V_i
5. Repeat step 4 until the GNFA contains only the added initial and final states.

Example 3.6 Construct regular expression for the DFA shown in Figure 3.11.

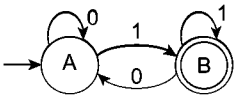


Fig. 3.11 *FA for Strings Ending with 1*

Step 1: GNFA constructed is shown in Figure 3.12

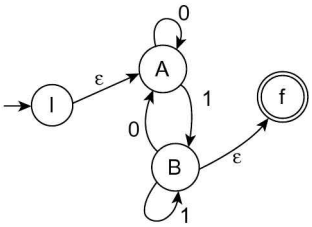


Fig. 3.12 *GNFA*

Step 2: On elimination of A the GNFA is as shown in Figure 3.13

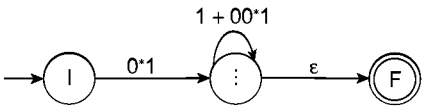


Fig. 3.13 *GNFA After Eliminating A*

Step 3: On elimination of B the GNFA is shown in Figure 3.14

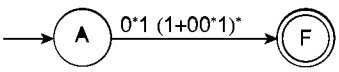


Fig. 3.14 *GNFA After Eliminating B*

The regular expression is $0*1(1 + 00*1)* = 0*1((\varepsilon + 00*)1)* =$ **0*1(0*1)*.**

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

3.7 Pumping Lemma of Regular Expressions

The pumping lemma is generally used to prove a language is not regular. Language is said to be regular

- If a DFA, NFA or NFA – ε machine can be constructed to exactly accept a language.
- If a regular expression can be constructed to exactly generate the strings in a language.

To prove a language is not regular one requires a specific definition of the language and properties of the pumping lemma for regular languages.

Let formal statements A and B be given with relation 'A implies B'. If B is proved to be false then it implies that A is also false.. This is proof by contradiction.

For the pumping lemma, the statement A is 'L is a regular language' and the statement B is a statement from the predicate calculus.

3.7.1 Formal Definition of the Pumping Lemma

Lemma: If L is a regular language represented with an automaton with a maximum of n states, then there is a word in L such that (the length) $|Z| \geq n$. Further, we may write $Z = UVW$ in such a way that $|UV| \leq n$, $|V| \geq 1$ and, for all $i \geq 0$, UV^iW is in L.

Proof: Consider a DFA which has n states. For a string which is valid, which goes through all the states will have a length at least $n - 1$. If we consider a string of length n or more than n then, there would be two states in the path, q_i and q_j such that $q_i = q_j$ as shown in Figure 3.15.

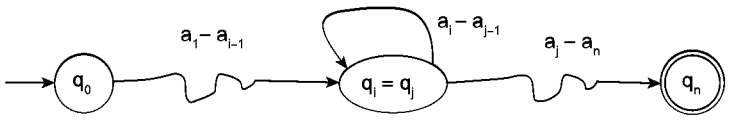


Fig. 3.15 *General NFA with n States*

The strings with length n would go through $n + 1$ states, and since there are only n states there at least one state that is repeated. If q_i and q_j are two such states, then the string $Z = a_1a_2\ldots a_{i-1}a_i\ldots a_{j-1}a_j\ldots a_n$ formed can be divided into three substrings UVW such that the string $U = a_1a_2\ldots a_{i-1}$ has a path from q_1 to q_i, the string $V = a_i\ldots a_{j-1}$ has a path from q_j to q_i and the string $W = a_j\ldots a_n$ has a path from q_j to q_n. Since the string $a_i\ldots a_{j-1}$ takes from q_j to q_i, if this string is pumped any number of times there would be path from q_1 to q_n as $a_1a_2\ldots a_{i-1}(a_i\ldots a_{j-1})^*a_j\ldots a_n$, which makes the string (UV^iW) valid. To make this possible, the substrings to be considered should satisfy the following conditions:

1. $|UV| \leq n$. This is the maximum possible string that can be selected to have path using n unique states.
2. $|V| > 0$. To show that there is path from q_j to q_i (even if both are same states) there should be at least one move which makes to leave the state and the enter the state.

The two common ways to use the pumping lemma to prove the language is NOT regular are:

(a) Show that there is no possible n, if there exists n, this is usually accomplished by showing a contradiction such as $(n + 1)\ (n+1) < n^2\ n + n$.

(b) Show there is no way to partition Z into U, V and W such that UV^iW is in L, typically for a value $i = 0$ or $i = 2$. Be sure to cover all cases by argument or enumerating cases.

Note: The pumping lemma only applies to languages (sets of strings) with infinite cardinality. A DFA can be constructed for any finite set of strings.

Example 3.4 State whether $L = \{a^n b^n \mid n > 0\}$ is regular.

Proof: Let us assume that the language is regular. According to pumping lemma if the string $Z = a^n b^n$, let n be the number of states such that $|Z| \geq n$, the string Z can be represented using the substring UVW. If the language is regular, then for all i, UV^iW is in L.

To check whether it is regular, it is required to consider three possible cases: The string V is formed with (1) only a's, (2) only b's or (3) or combination of a's and b's.

Case 1: V is a string such that it contains only a's.
$V = a^x$ such that $x \geq 1$.
Let $i = 0$. Then the string formed is UW. The string would be of the form
$a^{n-x}b^n \notin L$.

Case 2: V is a string such that it contains only b's.
$V = b^x$ such that $x \geq 1$.
Let $i = 0$. Then the string formed is UW; the string would be of the form
$a^n b^{n-x} \notin L$.

Case 3: V is a string such that it contains combination of a's and b's.
$V = a^x b^x$ such that $x \geq 1$.
If $i = 1$, then the string formed is UVW. The string would be of the form
$a^{n-x} a^x b^x b^{n-x} \in L$.
If $i = 2$, then the string formed is UV^2W. The string would be of the form
$a^{n-x} a^x b^x a^x b^x b^{n-x} \notin L$.

Since in all the three possible cases there exists value of i such that the string is not in L, hence the language is not regular.

3.8 Regular Grammar

In the previous chapter, we have learnt about the effective use of regular expressions for defining regular languages. In fact, we have learnt two ways of classifying regular languages: i) regular expressions and ii) finite automata. There is yet another way of expressing the regular languages, that is by regular grammar.

3.8.1 Equivalence of Regular Grammar and Finite Automata

The regular grammars generate strings of regular languages if the grammar is right linear or left linear. This is proved by the following theorems.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Theorem 3: If the language L has a regular grammar, then L is a regular set.

Proof: Let L = L(G) where G = {V, T, P, S} and G is right linear grammar.

We have to construct a NFA with ε such that it accepts the strings generated by the grammar G.

Let M = (Q, T, δ, [S], {[ε]}) where the terms are defined as.

- Q consists of the symbols of [α] such that α is S or some suffix of the right-hand side of the production in P.

Example: If S→aA is a production, then the possible states in Q would be {[S], [aA], [A], [ε]}.

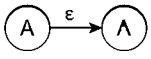
- The start symbol of the grammar is considered as the starting state in NFA.
- [ε] is the final state in the NFA.
- The transitions δ are defined by the following rules:
  - **Rule 1:** If A is a non-terminal in the grammar and A→α is the production, then add the transition δ([A], ε) = [α] for each non-terminal.
  - **Rule 2:** If a is a terminal and α is some possible suffix of the form T* or T*V, then add a transition δ([aα], a) = [α].

The NFA constructed by using these rules would accept all the strings generated by the grammar *G*. This can be proved using mathematical induction.

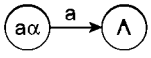
Procedure for Converting Regular Grammar to Finite Automaton

The conversion procedure depends on the type of given grammar. If the grammer is LLG then follow all the steps 1 through 8. If the grammer is RLG then follow the steps 2 through 6.

1. First convert the given left linear grammar into right linear grammar.
   Example: If the grammar is S→Aa, A→Aa|a, then its right linear grammar would be S→aA, A→aA|a
2. Take all possible suffixes of r.h.s of rule as states of FA.
   Example: In S→01S, possible suffixes are [01S], [1S], [S], [ε]
3. Draw FA with start state as initial state and [ε] as final state.
4. If A is a non-terminal in the grammar and A→α is a production, then add the transition



5. If a is a terminal and α is some possible suffix of the form T* or T*V, then add a transition by δ([aα], a) = [α].



6. This NFA corresponds to right linear grammar. If the given NFA is left linear then follow steps 7 & 8 other wise stop.
7. Reverse the NFA M` to get M such that
   - Initial state of M` is made final state of M.
   - Final state of M` is made initial state of M.
   - Reverse all the edges of M` and add to M.
8. Obtained NFA M corresponds to the given left linear grammar.

Example 3.8 Construct NFA for the following grammar S→aA | b, A→aA|a.

Solution: The above grammar is right linear grammar, which accepts the strings of the form (b + aa*)

Let M = (Q, Σ, δ, [S], [ε]) where Q = {[S], [aA], [A], [a], [b], [ε]),
T = {a, b} and δ is obtained by

Applying Rule 1

δ([S], ε) = [aA], [b]
δ([A], ε) = [aA], [a]

Applying Rule 2

δ([aA], a) = [A]
δ([a], a) = [ε]
δ([b], b) = [ε]. The resultant NFA is shown in the Figure 3.8

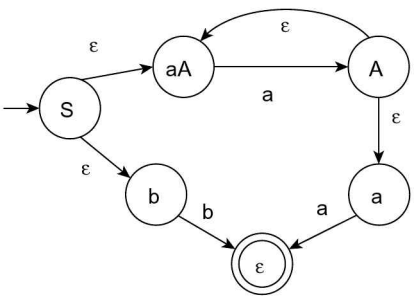


Fig. 3.16 *NFA for Example 3.8*

Example 3.8 Construct NFA for the following grammar: S→Ab |ab, A→Ab | Bb, B→ Ba | a

Solution: The above grammar is left linear grammar which accepts the language of the form $\{a^n b^m \mid n,m \geq 1\}$. It should be converted to right linear grammar.
When the above grammar is converted to right linear grammar, it is

S→bA |ba, A→bA | bB, B→ aB | a.

Let M = (Q, Σ, δ, [S], [ε]), where Q = {[S], [bA], [A], [ba], [a], [bB], [B], [aB], [ε]),

T = {a, b} and δ is obtained by Applying Rule 1

δ([S], ε) = [bA], [ba]
δ([A], ε) = [bA], [bB]
δ([B], ε) = [aB], [a]

Applying Rule 2

δ([bA], b) = [A]
δ([ba], b) = [a]
δ([a], a) = [ε]
δ([bB], b) = [B]
δ([aB], a) = [B]

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

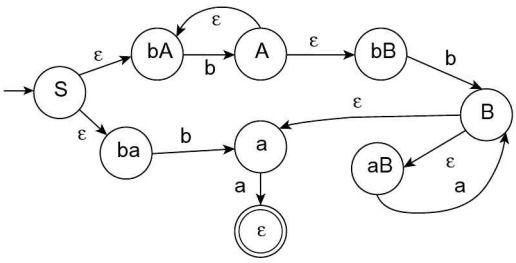


Fig. 3.17 *NFA for RLG*

This is NFA for right linear grammar. To get the final NFA for the given left linear grammar, we reverse the NFA and the result obtained is shown in Figure 3.18.

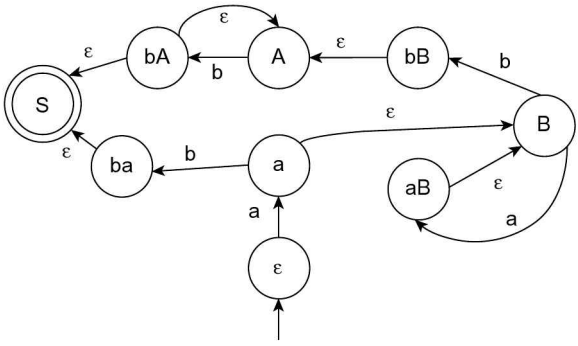


Fig. 3.18 *NFA for LLG*

3.8.2 Converting Finite Automaton to Regular Grammar

Theorem 4: If L is a regular set, then L is generated by some left linear grammar and by some right linear grammar.

Let L be the language for some DFA M = (Q, Σ, δ, q_0, F). Let us assume that q_0 is not the final state. For this we have to find grammar G such that L(G) = L(M).

Let G = (Q, Σ, P, q_0) where

- Q is the set of states, which are considered as variables in the grammar.
- The input symbols are considered as the terminal symbols.
- The start state of NFA is considered as the starting symbol in the grammar.
- The productions are defined from transitions δ: as
  - If there is a transition of the form δ(q, a) = p, then add a production **q→ap** if p is some non-final state.
  - If there is a transition of the form δ(q, a) = p, then add a production **q→ap | a** if p is some final state.

To show that language generated by the right linear grammar and the NFA is one and the same, we can use mathematical induction.

To construct a left linear grammar, first we reverse the given DFA. Then we construct right linear grammar for the reversed DFA and then reverse the productions to get left linear grammar.

Example 3.10 Find right linear grammar for the DFA in Figure 3.19:

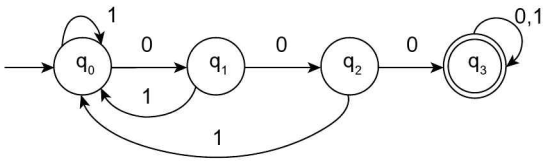


Fig. 3.19 *FA for Problem 3.10*

For the given DFA, the equivalent right linear grammar is

$V = \{q_0, q_1, q_2, q_3\}$

$T = \{0, 1\}$

$S = q_0$

$P = \{q_0 \rightarrow 1q_0 \mid 0q_1,$

$q_1 \rightarrow 0q_2 \mid 1q_0,$

$q_2 \rightarrow 0q_3 \mid 1q_0, \mid 0$

$q_3 \rightarrow 1q_3 \mid 0q_3, \mid 1 \mid 0\}$

Example 3.6 Find left linear grammar for the DFA in Figure 3.20:

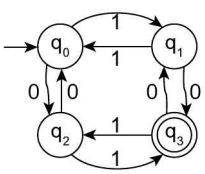


Fig. 3.20 *FA for Example 3.11*

To find the left linear grammar, first we reverse the DFA by reversing all the edges; and making the initial state as final, and the final state as initial.

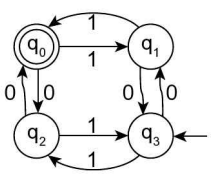


Fig. 3.21 *Reversed DFA*

For reversed DFA, the equivalent right linear grammar is

$V = \{q_0, q_1, q_2, q_3\}$

$T = \{0, 1\}$

$S = q_3$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$P = \{q_3 \rightarrow 1q_2 \mid 0q_1,$
$q_1 \rightarrow 0q_3 \mid 1q_0 \mid 1$
$q_2 \rightarrow 0q_0 \mid 1q_3 \mid 0,$
$q_0 \rightarrow 1q_1 \mid 0q_2\}$

To get the left linear grammar reverse all the productions of the above grammar.

$P = \{q_3 \rightarrow q_2\ 1 \mid q_1 0,$
$q_1 \rightarrow q_3 0 \mid q_0 1 \mid 1$
$q_2 \rightarrow q_0 0 \mid q_3 1 \mid 0,$
$q_0 \rightarrow q_1 1 \mid q_2 0\}$

3.9 Closure Properties of Regular Sets

Closure properties are theorems which show that the class of regular languages is closed under the operations mentioned. The theorems are of the form 'if certain languages are regular, and if a language L is formed from them by certain operation such as union, intersection, etc. then L is also regular'. In general, closure properties convey the fact that when one (or several) languages are regular, then certain related languages are also regular.

The principal closure properties of regular languages are:

1. **Union** of two regular languages is regular.

   If L and M are regular languages, then so is $L \cup M$.
2. **Intersection** of two regular languages is regular.

   If L and M are regular languages, then so is $L \cap M$.
3. **Complement** of a language is regular.

   If L is a regular language over alphabet Σ, then $\Sigma^* - L$ is also a regular language.
4. **Difference** of two regular languages is regular.

   If L and M are regular languages, then so is $L - M$.
5. **Reversal** of a regular language is regular.

   Reversing a string means writing the string backwards. For example, reverse of the string abcde is the string edcba.

   The reversal of a language L is the language consisting of reversal of all its strings reversed. For example, if $L = \{001,110\}$, then
   $L(R) = \{100,011\}$.
6. **Closure** of a regular language is regular.

   If L is a regular language, then so is L^*.
7. **Concatenation** of regular languages is regular.

   If L and M are regular languages, then so is LM.
8. **Homomorphism** of a regular language is regular.

   A homomorphism is the substitution of strings for symbols.

   For example, let the function h be defined by $h(0) = a$ and $h(1) = b$. Then h applied to 0011 is simply aabb.

   If h is a homomorphism on alphabet Σ and if w is a string of symbols abcd...z, then h (w) = h (a) h (b) h(c) h (d)...h (z)

   The mathematical definition for homomorphism is
   $h: \Sigma^* \rightarrow \Gamma^*$ such that $\forall\ x, y \in \Sigma^*$ and $h(x), h(y) \in \tau^*$.

A homomorphism can also be applied to a language by applying it to each string in the language. Let L be a language over alphabet Σ, and h be a homomorphism on Σ, then

h (L) = {h(w) | w is in L}

The theorem can be stated as follows: 'If L is a regular language over alphabet Σ, and if h is a homomorphism on Σ, then h(L) is also regular'.

9. **Inverse homomorphism** of two regular languages is regular.

Suppose h be a homomorphism from some alphabet Σ to strings in another alphabet T and suppose L be a language over T, then h inverse of L, h′(L), is the set of strings w in Σ* such that h(w) is in L.

The theorem states that: 'If h is a homomorphism from alphabet Σ to alphabet T, and L is a regular language on T, then h′(L) is also a regular language.'

3.10 Applications of Regular Expressions

A regular expression gives a picture of the pattern we want to search in the text. When the pattern is specified using a regular expression, it is compiled into a deterministic or non-deterministic automata behind and then simulates the program to search for the given pattern. Regular expressions gained a wide usage in the UNIX operating system. The other applications are defining patterns for tokens which are used in lexical Analysis phase and also in text processing.

3.10.1 Lexical Analysis

The compiler has a lexical analysis phase which forms the most important initial processing where the source program is scanned for recognizing the tokens. The tokens are defined by a regular expression for each pattern.

Example 3.10 An identifier is a token which has a pattern: 'an alphabet followed by any number of alphanumeric characters'. The corresponding regular expression is

$$[A - Za - z]\,[A - Za - z0 - 9]^*.$$

When the expression is specified using regular expression, the corresponding finite automata generated is shown in Figure 3.22.

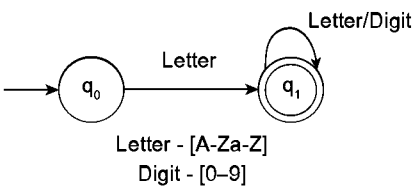


Fig. 3.22 *DFA for Identifier*

This finite automaton is used to recognize any string that starts with an alphabet and has alphanumeric characters. To recognize some words as keywords, then we can add few more states as shown in Figure 3.23. If a final state is reached then return the keywords relating to that state. This finite automaton is a non-deterministic finite automaton where we can recognize both identifiers and keywords.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

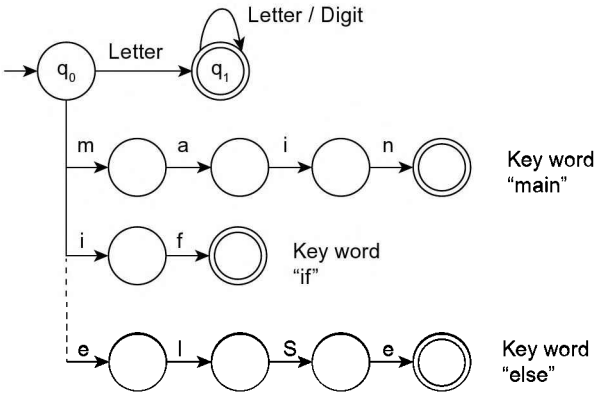


Fig. 3.23 *DFA for Keywords*

3.10.2 Finding Patterns

The regular expression technology is found to be useful in finding words corresponding to some patterns to be searched which are defined vaguely. There are many applications, that are listed below,

- To scan a very large number of Web pages and detect addresses.
- To create mailing list.
- To classify the business by their location and to answer the queries.
- To search files in the system based on a pattern.

Example 3.10 Suppose we want to search for a file which starts with **sy** and which is a **dll** file. Then, we can write a regular expression as **sy*.dll.** The expression identifies and lists all files that start with sy and is a dll file. The results are shown in Figure 3.25.



Fig. 3.24 *Example for Application of FA*

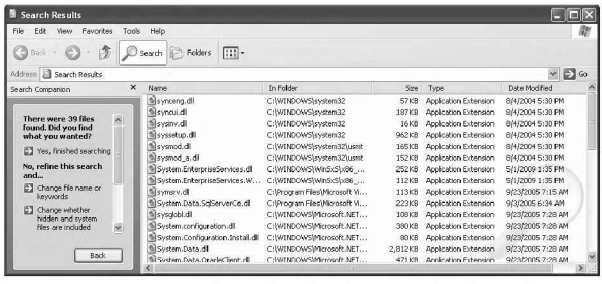


Fig. 3.25 *Example for Application of FA*

3.11 Decision Properties of Regular Languages

So far we have seen that there are different ways of representing a regular language and procedures to convert from one into the another. In general, irrespective of the representation, there are some fundamental questions that need to be answered about the language.

- Is the given language empty?
- Is the language finite?
- Does the string belong to the language?
- Are the two languages equivalent?

To answer such fundamental questions, we need to understand the time complexity of the algorithm that performs the conversion.

3.11.1 Conversion from NFA to DFA

Conversion from NFA to DFA would involve computing ε-closure of n states and then computing its equivalent DFA by subset construction.

- Computing ε-closure for each state requires $O(n^2)$ as the graph would be stored in matrix form. Hence computing ε-closure of n states would require $O(n^3)$ time.
- After computing ε-closure, to compute the equivalent DFA by subset construction would take 2^n as the number of states is 2^n and, for each state, the computation of transition would require $O(n^3)$. This is because we should refer to ε-closure information and transition table of the NFA. Therefore, the total time is $O(n^3 2^n)$.
- The number of inputs can be assumed constant as it does not depend on the number of states.

Hence the total time of conversion from NFA to DFA would be $O(n^3 2^n)$.

Other Conversion

- The conversion from DFA to NFA requires us to modify the transition table of DFA by putting a set of brackets around states. If the output is a NFA – ε, it requires us to add a column for ε. Hence the time for this conversion would be O(n).
- The conversion is done by using a recursive procedure. We observe that at each of n rounds, we construct a regular expression that is formed from four expressions of the previous round. To compute the expression, the conversion requires $O(n^3 2^n)$.
- To convert from NFA to RE, conversion from NFA to DFA is required first. This conversion requires the time $O(n^3 2^n)$. It implies that there are those many states. To convert from NFA to RE would require $O(n^3 4^{n^3 2^n})$.
- Conversion from RE to NFA takes linear time which is O(n) on a regular expression of length n.
- Conversion from NFA – ε to NFA without ε takes $O(n^3)$ as computation of ε-closure of every state is required.

The following table gives the time for conversion from one form to the other.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

	NFA – ε	NFA	DFA	RE
NFA – ε	–	$O(n^3)$	$O(n^3 2^n)$	$O(n^3 4^{n^3 2^n})$
NFA	O(n)	–	$O(n^3 2^n)$	$O(n^3 4^{n^3 2^n})$
DFA	O(n)	O(n)	–	$O(n^3 4^n)$
RE	O(n)	$O(n^3)$	$O(n^3 2^n)$	–

3.11.2 Emptiness Membership and Equivalence

To find whether the given language is **empty** or not depends on the representation. If the representation is in FA, then it is required to find whether there exists a path from the initial state to the final state; i.e, if the final state is reachable from initial state on any string, or if there exists a path, then the answer NO: otherwise the answer is YES. Answering the question requires $O(n^2)$ time. If the representation is in RE, then it is converted from RE to NFA, and that requires O(n) time and then check for reachability which requires $O(n^2)$ time. This can also be done by inspecting the occurrence of ε and applying the identity rules of regular expression.

To find whether the string w is a member of L or not, check whether there exists a path from initial state to final state while processing the given string. In case of DFA, the time required would depend on the length of the string. In case of NFA, the time complexity would be $O(mn^2)$ where m is the length of string and n stands for the number of states in NFA. If it is represented as RE of size s, then it should be converted to NFA – ε with at most 2s states in O(s) time. To perform simulation it requires $O(ns^2)$ time for input of length n.

To find whether the given two languages L and M are equivalent, it is first required to convert both of them into ts equivalent DFAs. Now the DFAs can be compared by considering the states from both, with all possible combinations. If both states are identified distinguishable, then the answer is NO, otherwise the answer is YES. For comparison, the time required is $O(n^2)$. The final time required depends even on the conversion process (to DFA).

Solved Problems

Problem 1: Prove or disprove the relation s(rs + s)*r = rr*s(rr*s)*.

Solution:

rs ∉ LHS. rs ∈ RHS. Hence they are not same

Problem 2: Prove or disprove the relation (rs + r)*r = r(sr + r)*.

Solution:

Let a = s + ϵ

LHS = (r(s + ϵ))*r = (ra)*r

RHS = r((s + ϵ)r)* = r(ar)* = (ra)*r using the identity rule (PQ)*P = P(QP)*

Hence, both are same.

Problem 3: Prove that $\epsilon + a^*(b)^*(a^*(b)^*)^* = (a + b)^*$.

Solution:

Let $P = a^*(b)^*$

The given expression is represented as $\epsilon + PP^* = P^*$.

Hence, $p^* = (a^*(b)^*)^*$

Let $P = a$ and $Q = b$. Then the expression can be represented as

$(P^*Q^*)^*$

$= (P + Q)^*$ using the identity rule $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$

$= (a + b)^*$

Hence it is proved that both are same.

Problem 4: Prove that $(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) = 0^*1(0 + 10^*1)^*$.

Solution:

Let $P = 1 + 00^*1$ $Q = 0 + 10^*1$

LHS $= P + PQ^*$ $Q = P(\epsilon + Q^*Q) = PQ^*$

$= (1 + 00^*1)(0 + 10^*1)^*$

$= (\epsilon + 00^*)\, 1(0 + 10^*1)^* = 0^*\, 1(0 + 10^*1)^*$

Hence it is proved that both are same.

Problem 5: Construct NFA for the regular expression $(11 + 0)^*(00 + 1)^*$.

Solution: The given expression can be expressed as a regular expression formed by applying a sequence of operations on individual expression as shown below:

$R = R_1^*R_2^*$ where $R_1 = (11 + 0)$, $R_2 = (00 + 1)$

The FA for the expressions R_1, R_2 can be constructed as in Figure 3.26.

R_1

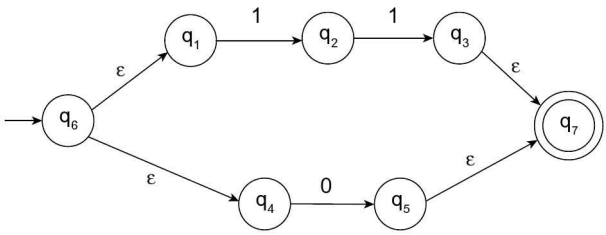


Fig. 3.26 *NFA for Expression R_1*

R_1^*

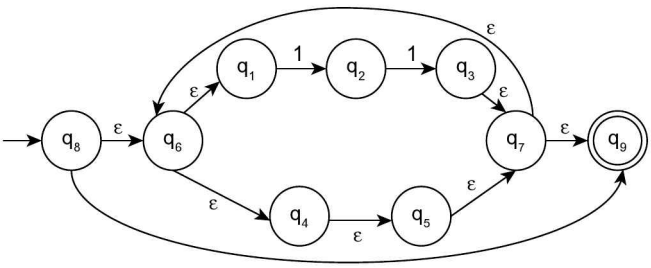


Fig. 3.27 *NFA for Expression R_1^**

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Similarly R_2*

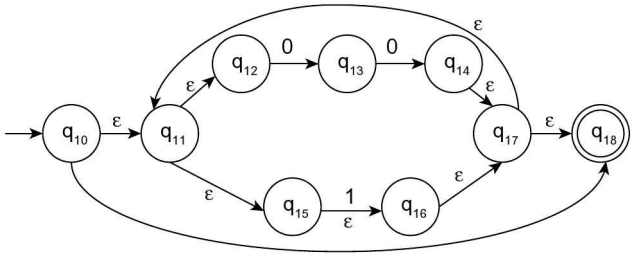


Fig. 3.28 *NFA for Expression R_2**

Final NFA for $R = R_1{}^*R_2{}^*$ is

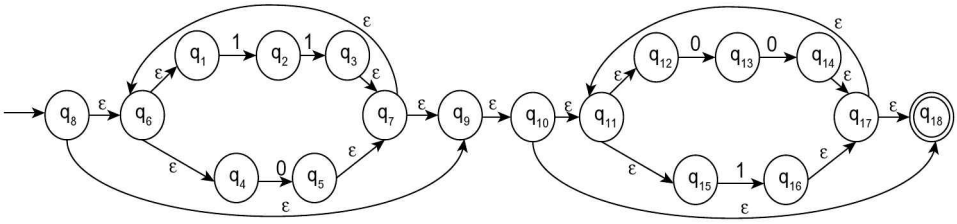


Fig. 3.29 *NFA for Expression R*

Problem 6: Find the regular expression for Figure 3.30.

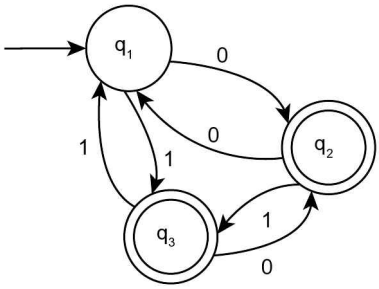


Fig. 3.30 *DFA for Problem 6*

Solution:

$r_{11}^0 = \varepsilon$ $r_{12}^0 = 0$ $r_{13}^0 = 1$

$r_{21}^0 = 0$ $r_{22}^0 = \varepsilon$ $r_{23}^0 = 1$

$r_{31}^0 = 1$ $r_{32}^0 = 0$ $r_{33}^0 = \varepsilon$

$r_{11}^1 = r_{11}^0(r_{11}^0)^* r_{11}^0 + r_{11}^0 = (\varepsilon)^* \varepsilon + = \varepsilon$

$r_{12}^1 = r_{11}^0(r_{11}^0)^* r_{12}^0 + r_{12}^0 = (\varepsilon)^*0 + 0 = 0$

$r_{13}^1 = r_{11}^0(r_{11}^0)^* r_{13}^0 + r_{13}^0 = (\varepsilon)^*1 + 1 = 1$

$r_{21}^1 = r_{21}^0 (r_{11}^0)^* r_{11}^0 + r_{21}^0 = 0(\varepsilon)^*\varepsilon + 0 = 0$

$r_{22}^1 = r_{21}^0 (r_{11}^0)^* r_{12}^0 + r_{22}^0 = 0(\varepsilon)^*0 + \varepsilon = 00 + \varepsilon$

$r_{23}^1 = r_{21}^0 (r_{11}^0)^* r_{13}^0 + r_{23}^0 = 0(\varepsilon)^*1 + 1 = (0 + \varepsilon)1$

$r_{31}^1 = r_{31}^0 (r_{11}^0)^* r_{11}^0 + r_{31}^0 = 1(\varepsilon)^* \varepsilon + 1 = 1$

$r_{32}^1 = r_{31}^0 (r_{11}^0)^* r_{12}^0 + r_{32}^0 = 1(\varepsilon)^*0 + 0 = (1 + \varepsilon)0$

$r_{33}^1 = r_{31}^0 (r_{11}^0)^* r_{13}^0 + r_{33}^0 = 1(\varepsilon)^*1 + \varepsilon = 11 + \varepsilon$

$K = 2$

$r_{11}^2 = r_{12}^1 (r_{22}^1)^* r_{21}^1 + r_{11}^1 = 0(00 + \varepsilon)^* 0 + \varepsilon = 0(00)^*0 + \varepsilon = 00(00)^* + \varepsilon = (00)^*$

$r_{12}^2 = r_{12}^1 (r_{22}^1)^* r_{22}^1 + r_{12}^1 = 0(00 + \varepsilon)^*(00 + \varepsilon) + 0 = 0\ ((00 + \varepsilon)^*(00 + \varepsilon) + \varepsilon) = 0(00)^*$

$r_{13}^2 = r_{12}^1 (r_{22}^1)^* r_{23}^1 + r_{13}^1 = 0(00 + \varepsilon)^*(0 + \varepsilon)1 + 1 = (0(00)^* (0 + \varepsilon) + \varepsilon)1 = 0^*1$

$r_{21}^2 = r_{22}^1 (r_{22}^1)^* r_{21}^1 + r_{21}^1 = (00 + \varepsilon)\ (00 + \varepsilon)^*0 + 0 = (00)^*0 + 0 = ((00)^* + \varepsilon)0 = (00)^*0$

$r_{22}^2 = r_{22}^1 (r_{22}^1)^* r_{22}^1 + r_{22}^1 = (00 + \varepsilon)\ (00 + \varepsilon)^*\ (00 + \varepsilon) + (00 + \varepsilon) = (00)^*$

$r_{23}^2 = r_{22}^1 (r_{22}^1)^* r_{23}^1 + r_{23}^1 = (00 + \varepsilon)(00 + \varepsilon)^*\ (0 + \varepsilon)1 + (0 + \varepsilon)1 = (00)^*\ (0 + \varepsilon)1 = 0^*1$

$r_{31}^2 = r_{32}^1 (r_{22}^1)^* r_{21}^1 + r_{31}^1$

$= (1 + \varepsilon)0(00 + \varepsilon)^*0 + 1 = (1 + \varepsilon)(00)^*00 + 1 = 1(00)^*00 + (00)^*00 + 1$

$= 1((00)^*00 + \varepsilon) + (00)^*00 = 1(00)^* + (00)^*00 = (1 + 00)(00)^*$

$r_{32}^2 = r_{32}^1 (r_{22}^1)^* r_{22}^1 + r_{32}^1 = (1 + \varepsilon)0(00 + \varepsilon)^*(00 + \varepsilon) + (1 + \varepsilon)\ 0 = (1 + \varepsilon)0((00 + \varepsilon)^*(00 + \varepsilon) + \varepsilon)$

$= (1 + \varepsilon)0(00)^*$

$r_{33}^2 = r_{32}^1 (r_{22}^1)^* r_{23}^1 + r_{33}^1 = (1 + \varepsilon)0(00 + \varepsilon)^*(0 + \varepsilon)1 + (11 + \varepsilon)$

$= (10 + 0)(00)^*(01 + 1) + 11 + \varepsilon$

$= 10(00)^*01 + 10(00)^*1 + 0(00)^*01 + 0(00)^*1 + 11 + \varepsilon$

$= 1(00)^*1 + 10(00)^*1 + 11 + (00)^+1 + 0(00)^*1 + \varepsilon$

$= 1((00)^* + 0(00)^* + \varepsilon)1 + ((00)^+ + 0(00)^*)1 + \varepsilon$

$= 10^*1 + 0^+1 + \varepsilon$

$= (1 + 0)0^*1 + \varepsilon$

	K = 0	K = 1	K = 2
r_{11}	ε	ε	(00)*
r_{12}	0	0	0(00)*
r_{13}	1	1	0*1

(continued)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

	K = 0	K = 1	K = 2
r_{21}	0	0	(00)*0
r_{22}	ε	(00 + ε)	(00)*
r_{23}	1	(0 + ε)1	0*1
r_{31}	1	1	(1 + ε)(00)$^+$ + 1
r_{32}	0	(1 + ε)0	(1 + ε)0(00)*
r_{33}	ε	(11 + ε)	(1 + 0)0*1 + ε

Since there are only three states and the final states are q_2 and q_3, it is sufficient to find the regular expressions for r_{12}^3, r_{13}^3 with K = 3 and compute the union of these expressions:

$$r_{12}^3 = r_{13}^2(r_{33}^2)^* r_{32}^2 + r_{12}^2 \text{ and } r_{13}^3 = r_{13}^2(r_{33}^2)^* r_{33}^2 + r_{13}^2$$

The final regular expression is $r_{12}^3 + r_{13}^3$.

Let $p = r_{13}^2 = 0*1$,

$q = r_{33}^2 = (1 + 0)0*1 + \varepsilon$,

$r = r_{32}^2 = (1 + \varepsilon)0(00)*$,

$s = r_{12}^2 = 0(00)*$

The final regular expression is

$p(q)*r + s + p(q)*q + p$

$= p((q)*r + (q)*q + \varepsilon) + s$

$= p((q)*(r + q) + \varepsilon) + s$

$= 0*1(((1 + 0)0*1)*((1 + \varepsilon)0(00)* + (1 + 0)0*1 + \varepsilon) + \varepsilon) + 0(00)*$

$((00)*(\varepsilon + 0)1)((00 + 1)(00)*(1 + 01) + 01)*((00 + 1)(00)*0 + 0 + \varepsilon) + (00)*0$

Problem 7: Construct the regular expression for the DFA in Figure 3.31.

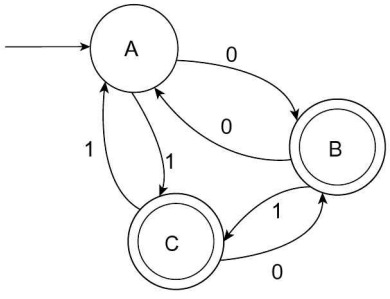


Fig. 3.31 *DFA for Problem 7*

Solution:

A = B0 + C1 + ε (1)

B = A0 + C0 (2)

$C = A1 + B1$ (3)

Substituting the expression for B in A we get

$A = A00 + C00 + C1 + \varepsilon$

$= C(00 + 1) + \varepsilon\, A00$

Using Arden's theorem we get

$A = (C(00 + 1) + \varepsilon)(00)^* = C(00 + 1)(00)^* + (00)^*$ (4)

Substitute Expression 4 of A in Equation 2

$B = (C(00 + 1)(00)^* + (00)^*)0 + C0$

$= C(00 + 1)(00)^*0 + C0 + (00)^*0$ (5)

Substituting Expressions 4 and 5 (for A and B, respectively) in Equation 3, we get

$C = (C(00 + 1)(00)^* + (00)^*)1 + (C(00 + 1)(00)^*0 + C0 + (00)^*0)1$

$= C(00 + 1)(00)^*1 + C(00 + 1)(00)^*01 + C01 + (00)^*1 + (00)^*01$

$= C((00 + 1)(00)^*(1 + 01) + 01) + (00)^*(\varepsilon + 0)1$

Applying Arden's theorem, we get

$C = ((00)^*(\varepsilon + 0)1)((00 + 1)(00)^*(1 + 01) + 01)^*$

Now B can be found by substituting in Equation 5 the expression for C we obtained:

$B = C((00 + 1)(00)^*0 + 0) + (00)^*0$

$= ((00)^*(\varepsilon + 0)1)((00 + 1)(00)^*(1 + 01) + 01)^*((00 + 1)(00)^*0 + 0) + (00)^*0$

The final regular expression is B + C and it is given by

$((00)^*(\varepsilon + 0)1)((00 + 1)(00)^*(1 + 01) + 01)^*((00 + 1)(00)^*0 + 0 + \varepsilon) + (00)^*0$

Simplifying the above regular expression we get

$(0^*1)\,(((00)^* + 0) + 10^*1)^*\,((00 + 1)(00)^*0 + 0 + \varepsilon) + (00)^*0$

Problem 8: Construct the regular expression for the DFA shown in Figure 3.32.

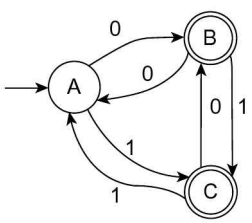


Fig. 3.32 *DFA for Problem 8*

Solution:

Step 1: GNFA is shown in Figure 3.33

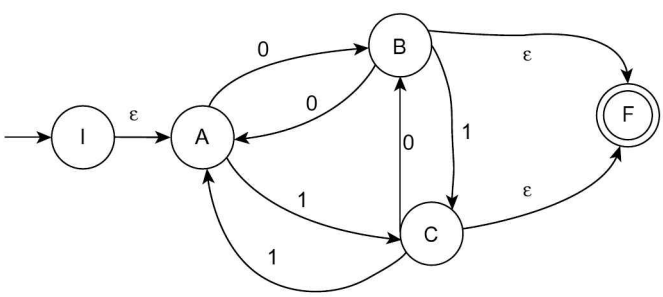


Fig. 3.33 *GNFA*

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Step 2: On elimination of A the GNFA is shown in Figure 3.34

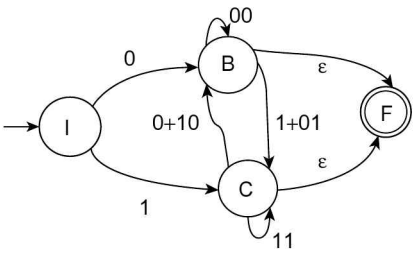


Fig. 3.34 *GNFA After Eliminating A*

Step 3: On elimination of B the GNFA is shown in Figure 3.35

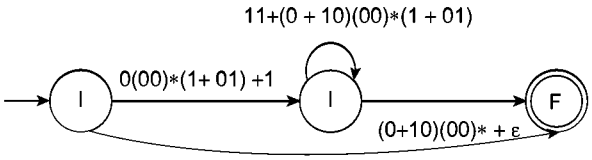


Fig. 3.35 *GNFA After Eliminating B*

Step 4: On elimination of C the GNFA is shown in Figure 3.36

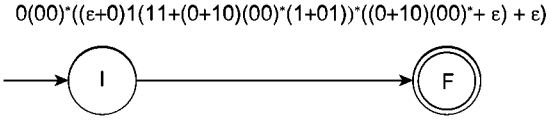


Fig. 3.36 *GNFA After Eliminating C*

Problem 9: State whether L = $\{a^{2n} | n > 0\}$ is regular.

Solution: The given language can be represented as set of all strings whose length is even. We can construct a finite automaton for the given language. Since the language consists of strings {aa, aaaa, aaaaaa, , ...} where the number of a's is even. To accept even strings whose length is greater than or equal to 2, we need three states q_0, q_1 and q_2 where q_0 is initial state and q_2 is final state and is shown in Figure 3.37.

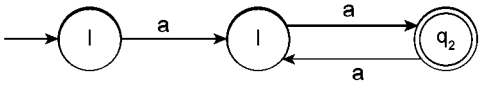


Fig. 3.37 *DFA for Even Number of a's*

Since there exists a finite automaton, we can conclude that the given language is regular.

Problem 10: State whether L = $\{0^{n^2} | n \geq 1\}$ is regular.

Solution: This language consists of strings whose length is a perfect square.

Assume L is regular and let n be the number of states in the automaton. Let Z = 0^{n^2}. By applying pumping lemma, 0^{n^2} may be written as UVW, where $1 \leq |v| \leq n$ and UV^iW is in L

for all i. Consider the value of i = 2. Then the string length would vary between $n^2 \leq |UV^2W| \leq (n^2 + n) < (n + 1)^2$. The length of the string UV^2W lies between n^2 and $(n + 1)^2$, and is thus the length may not be a perfect square. This is contradicting our assumption that the language is regular. Therefore, we conclude that the language is not regular.

Problem 11: State whether $L = \{0^n \mid n \text{ is a prime}\}$ is regular.

Solution: This language consists of strings whose length is a prime number.

Assume L is regular and let n be the number of states in the automaton. Let $Z = 0^n$, and let p be some prime such that $|Z| = p$ and $p \geq n$. By applying the pumping lemma, 0^n may be written as UVW, where $V = 0^m$ where $0 < |m| \leq n$ and UV^iW is in L for all i. Consider the value of i = p. Then, the string length of UV^pW can be represented as

$UV^pW = UVW + V^{p-1}$

$= p + m(p - 1)$

Let i = p + 1. Then $UV^{p+1}W$ can be represented as

$UV^{p+1}W = UVW + V^p$

$= p + mp = p(1 + m)$

When i = p + 1, it is clear that the length cannot be a prime as it is shown that the length is product of p and (1 + m). Hence, L is not regular.

Problem 12: State whether $L = \{a^n \mid n \geq 0\}$ is regular.

Solution: This language resembles a* where there is no condition on the number of a's. We can construct a DFA for all values of i such that UV^iW is in L. Hence L is regular.

Problem 13: State whether $L = \{a^n b^m \mid n, m \geq 0\}$ is regular.

Solution: This language corresponds to regular expression a*b*, which is regular as there exists a DFA to accept all the strings of the language.

Problem 14: Let $\Sigma = \{a, b\}$, $\Delta = \{a, b, c\}$ and let h be defined by h(a) = ab, h(b) = bbc. Find h(aba) and h(aa). If $L = \{aa, aba\}$, find h(L).

Solution:

h(aa) = abab.

h(aba) = abbbcab.

Homomorphic language of $L = \{aa, aba\}$ is computed as

$h(L) = \{abab, abbbcab\}$

Problem 15: Let $\Sigma = \{a, b\}$ and $\Delta = \{b, c, d\}$. Define h by h(a) = dbcc, h(b) = bdc. Find L, if l = h(r) where r = a + b*(aa)*.

Solution:

If L is the regular language denoted by r = (a + b*) (aa)*, then

h(r) = (dbcc + (bdc)*) (dbccdbcc)* denotes the regular language h(L).

Problem 16: Let $\Sigma = \{0, 1\}$ and $\Delta = \{a, b\}$. Define h by h(0) = aa and h(1) = aba.

Find h^1 of L where L = a + b*(aa)*.

Solution:

The language given is

L= {aba, baa, ababa, abbaa, babaa, baaba, ……..}

$h^1(L) = \{x/h^1(x) \in L\}$, $h^1(L)$ is defined as

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$h^1(L) = \{1\}$ (since $h^{-1}(aba) = \{1\}$)

Observe that $h((h^1(L) = h(\{1\}\} = \{aba\} \neq L$

On the other hand, $h(h^1(L)) \subseteq L$ and $h^{-1}(h(L)) \subseteq L$ and $h^{-1}(h(L)) \supseteq L$ for any language L.

Problem 17: Let $\Sigma = \{0, 1, 2\}$ and $\Delta = \{a,b\}$. Define h by $h(0) = a$, $h(1) = ab$, $h(2) = ba$. If $L_1 = \{ababa\}$ and $L_2 = a(ba)^*$, find $h^1(L_1)$ and $h^{-1}(L_2)$.

Solution:

(i) $h^1(L_1) = \{110,102,022\}$.

(ii) $h^{-1}(L_2) = h^{-1}(a(ba)^*) = 02^* + 1^*0 = 1^*02^*$ (since $a(ba)^* = (ab)^*a$).

Problem 18: Let $\Sigma = \{a, b\}$ and $\Delta = \{0, 1\}$. Define a homomorphism h as $h(a) = 01$, $h(b) = 0$. If $L_1 = (10 + 1)^*$, $L_2 = (a + b)^*$ and L_3 is the set of all strings of 0's and 1's with an equal number of 0's and 1's, find $h^1(L_1)$, $h(L_2)$ and $h^{-1}(L_3)$.

Solution:

i) The strings in L_1 are given by the set $\{\varepsilon, 1, 11, 10, 101, 110, 1010,\ldots\ldots\}$. This language does not contain any string that starts with 0. Therefore $h^1(L_1) = \emptyset$

ii) To find $h(L_2)$, substitute $h(a) = 01$ and $h(b) = 0$ in $(a + b)^*$. Therefore, the language is given by $h(L_2) = (01 + 0)^*$

iii) The language L_3 is defined to have all strings of 0's and 1's with an equal number of 0's and 1's. If $h^{-1}(L_3)$ contains b, then L_3 will have strings in which the number of 0's is more than the number of 1's. Hence $h^1(L_3)$ does not contain b's: $h^{-1}(L_3) = a^*$

Problem 19: Prove or disprove the regularity of the language $\{ww^R | w \in \Sigma\}$.

Solution: Let the given language be regular so, it must satisfy the standard form of the pumping lemma, and there must exist a DFA with n states. Choose Z as shown in Figure 3.38:

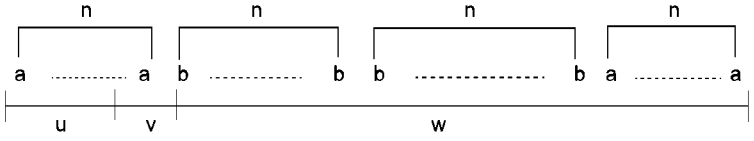


Fig. 3.38 *DFA for Problem 19*

As $|UV| \leq n$ and $|V| \geq 1$, u contains only a's.

Choose $i = 0$ in the string UV^iW.

The string obtained in this fashion has fewer a's on the left than on the right and, hence, it cannot be of the form ww^R. Therefore, L is not regular.

Problem 20: Prove or disprove the regularity of the language $\{a^ib^j/i > j\}$.

Solution: Let the given language be regular. Then, it must satisfy the strong form of the pumping lemma, and there must exist a DFA with n states

Choose $Z \in L$ by

$Z_1UV^iWZ_3 \in L$.

Choose $|V| = d$, $i = 3n$

$a^{2n}b^{(n\,d)+3nd} = a^{2n}b^{(3d+1)n-d}$.

This generated string has more number of b's than a's. So, the given language is not regular.

Problem 21: Show that the language $L = (a^{n!} \mid n \geq 0\}$ is not regular.

Solution: Let the given language be regular. So it must satisfy the standard form of the pumping lemma, and there must exist a DFA with n states.

Let $Z \in L$ and $|x| = n!$.

Let $|V| = k \leq n$, $i = 0$ in the string $UV^iW \in L$.

String with length $n! - k$ is in the language.

So $n! - k$ must be a factorial of some j.

Therefore, $N! - k = j!$.

But this is impossible, since for $n > 2$ and $k \leq n$ we have $n! - k > (n - 1)!$. Therefore, the language is not regular (if $n < 3$ choose z as $a^{3!}$).

Problem 22: Construct NFA for the following grammar: S→aA | bB, A→aA|a. B→bB |b

Solution: The above grammar is right linear, which accepts the strings of the form $(aa^+ + bb^+)$.

Let $M = (Q, \Sigma, \delta, [S], [\varepsilon])$ where $Q = \{[S], [aA], [A], [a], [bB], [B], [b], [\varepsilon])$.

Let $T = \{a, b\}$. Now δ is obtained by

Applying Rule 1

$\delta([S], \varepsilon) = [aA], [bB]$

$\delta([A], \varepsilon) = [aA], [a]$

$\delta([B], \varepsilon) = [bB], [b]$

Applying Rule 2

$\delta([aA], a) = [A]$

$\delta([a], a) = [\varepsilon]$

$\delta([bB], b) = [B]$

$\delta([b], b) = [\varepsilon]$

The resultant NFA is shown in the Figure 3.39

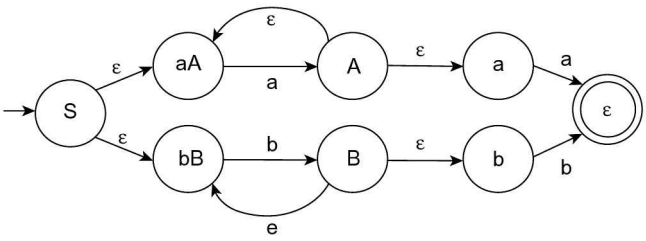


Fig. 3.39 *NFA for Problem 22*

Problem 23: Construct NFA for the following left linear grammar:

S→A0 |B1, A→A0 | 0, B→ B1 | 1.

Solution: The above grammar is left linear, which accepts the language of the form $(00^+ + 11^+)$. It should be converted to right linear grammar.

The converted right linear grammar obtained is

S→0A | 1B, A→0A | 0, B→ 1B | 1.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Let M = (Q, Σ, δ, [S], [ε]) where Q = {[S], [0A], [A], [0], [1B], [B], [1], [ε]).
Let T = {0, 1}. Now δ is obtained by

Applying Rule 1

δ([S], ε) = [0A], [1B]
δ([A], ε) = [0A], [0]
δ([B], ε) = [1B], [1]

Applying Rule 2

δ([0A], 0) = [A]
δ([0], 0) = [ε]
δ([1B], 1) = [B]
δ([1], 1) = [ε]. The NFA is shown in the Figure 3.40.

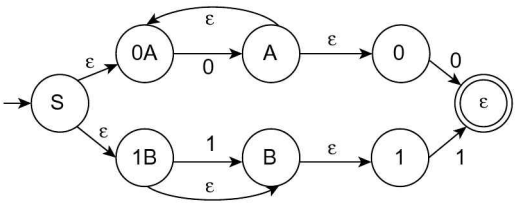


Fig. 3.40 *NFA for RLG*

This is NFA for the right linear grammar. To get the final NFA for the given left linear grammar, we reverse the NFA and the result is shown in Figure 3.41.

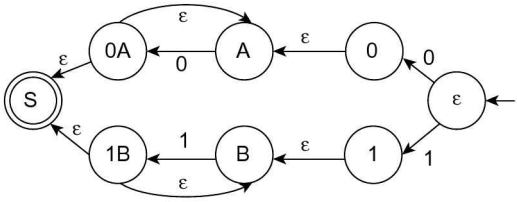


Fig. 3.41 *NFA for LLG*

Problem 24: Find right linear grammar for the DFA in Figure 3.42:

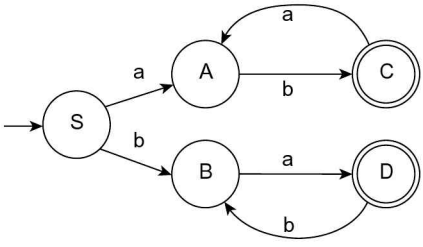


Fig. 3.42 *DFA for Problem 24*

The above DFA accepts strings of a's and b's such that if the string starts with 'a' it ends with 'b' and if it starts' with b' it ends with 'a'.. For the above DFA, the equivalent right linear grammar is

V = {S, A, B, C, D}
T = {a, b}
S = S
P = {S→ aA | bB,
A→ bC | b,
B→ aD | a,
C→ aA | ε,
D→ aB | ε}

Problem 25: Find left linear grammar or the DFA in Figure 3.43:

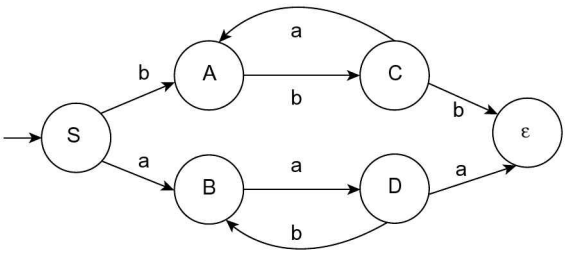


Fig. 3.43 *DFA for Problem 25*

The above DFA accepts strings of a's and b's if they are of length atleast 3, and if they have the following characteristic: (a) if string start with 'a' then it ends with 'aa', and if string start with b then it ends with bb. To find the left linear grammar, first we reverse the DFA by reversing all the edges and change the given initial state to final state and the given final state as initial state as shown in Figure 3.44.

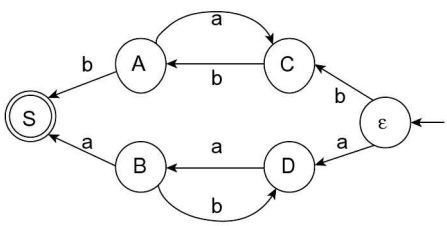


Fig. 3.44 *DFA for LLG*

For the above DFA, the equivalent right linear grammar is

V = {S, A, B, C, D, E}
T = {a, b}
S = E
P = {E→ aD | bC,
D→ aB,
C→ bA,
A→ aC | bS | b,
B→ bD | aS | a,
S → ε}

To get the left linear grammar reverse all the productions of the above grammar. Hence
P = {E→ Da | Cb,

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$D \rightarrow Ba,$
$C \rightarrow Ab,$
$A \rightarrow Ca \mid Sb \mid b,$
$B \rightarrow Db \mid Sa \mid a,$
$S \rightarrow \varepsilon\}$

Summary

1. Regular expression is an algebraic notation for regular languages and regular sets.
2. The basic operations that can be applied to regular expressions are forming the union, concatenating and forming closures.
3. Regular expressions can be simplified by using algebraic laws and identity rules.
4. For every regular expression, we can construct an equivalent finite automaton that accepts the language represented by the expression.
5. For every language accepted by a DFA, there is a corresponding regular expression.
6. Regular grammars are either left linear or right linear. For every right linear grammar, there exists a NFA that accepts the same language.
7. Regular expressions are used in lexical phase of the compilation process and also in (almost every) text processing applications.

Short Answers

1. Let R be any set of regular languages. Is union of R regular? Prove it.
   Answer: Refer to Case 1 of Theorem 1.
2. Verify whether $L = \{a^{2n} \mid n \geq 1\}$ is regular.
   Answer: See Example 19.
3. State the properties of regular languages.
   Answer: The regular languages are closed under
   i. union.
   ii. concatenation.
   iii. intersection.
   iv. kleen closure.
   v. substitution.
   vi. homomorphism.
   vii. complement formation.
   viii. reversing.
   ix. inverse homomorphism.
4. State the pumping lemma and its advantages.
   Answer: Let L be a regular language represented with an automaton with a maximum of n states. Then, there is a word in L such that the length $|Z| \geq n$, we may write $Z = UVW$ in such a way that $|UV| \leq n$, $|V| \geq 1$, and, for all $i \geq 0$, UV^iW is in L. The

advantage of the pumping lemma is that it can be used to check if a language is regular or not.

5. Show that the complement of a regular language is also regular.

   Answer: The class of regular sets are closed under complement formation. That is, if X is a regular set, then $X \subseteq \Sigma^* - L$ is also a regular set. To prove this statement, let $M = \{Q, \Sigma, \delta, q_0, F\}$ be a finite automaton which accepts the language L. To construct the automaton that accepts the language $L` = \Sigma^* - L$, change all final states of M to non-final states and all non-final states to final states, that is, $M` = \{Q, \Sigma, \delta, q_0, (Q - F)\}$.

6. Define a regular set and give an example for a regular expression.

   Answer: Any set represented by a regular expression is called a regular set. $(0 + 1)^*$ is a regular expression that represents the language having 0's and 1's.

7. Write the regular expression for the set of all strings of 0' s and 1's ending in 11.

   Answer: Refer Section 3.2, example 2.

8. Write the regular expression for the set of strings of 0's and 1's with at least two consecutive 1's.

   Answer: Refer Section 3.2, example 6.

9. Find the language accepted by the regular expression $(a + b)^*bbaa(a + b)$.

   Answer: The language generated by the given regular expression is the set of all strings with the substring bbaa; that is, L = {bbaa, abbaa, bbbaa, bbaaa, bbaab .....}.

10. Prove that $\varepsilon + 0^*(1)^*(0^*(1)^*)^* = (0 + 1)^*$.

    Answer: Similar to Solved problems, problem 3.

11. State Arden's theorem and its use.

    Answer: Arden's theorem states that if C and B are two REs over $\sum$, if A is unknown, and if C does not contain ε, then the equation if $A = B + AC$ has a unique solution given by

    $$A = BC^*$$

    The advantage of Arden's theorem is that it can be used to solve the equations for finding the regular expression of deterministic finite automata.

12. List the rules for defining a regular expression?

    Answer: A regular expression is a string that describes the whole set of strings using syntax rules. These expressions are used by many text editors and utilities to search bodies of text for certain patterns, etc.

    Let Σ be an alphabet. Then the regular expression over Σ and the sets they denote are explained in the following:

    i. Φ is a RE and it denotes the empty set.
    ii. ε is a RE and it denotes (the set) $\{\varepsilon\}$.
    iii. For each a in Σ, a is a RE and it denotes (the set) {a}.
    iv. If r and s are RE denoting the languages R and S, respectively, then (r + s), (rs) and (r*) are RE that denote the sets RUS, RS and R*, respectively.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

13. Let L = {0, 11}. Find L^0, L^2.

Answer: $L^0 = \{\varepsilon\}$

$L^2 = \{00, 011, 110, 1111\}$

14. Let L be the set of all strings of length 6 or less over {0.1}. Find out the regular expression corresponding to it.

Answer: The regular expression to generate the strings of length 6 or less than 6 is $(\varepsilon + 0 + 1)(\varepsilon + 0 + 1)(\varepsilon + 0 + 1)(\varepsilon + 0 + 1)(\varepsilon + 0 + 1)(\varepsilon + 0 + 1)(\varepsilon + 0 + 1)$

15. Differentiate L* and L +.

Answer: L* denotes the Kleene closure and is given by

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Example: 0* = {ε, 0, 00, 000,.....}. This language includes null string also.

L^+ denotes the positive closure and it is given by

$$L^+ = \bigcup_{i=0}^{\infty} L^i$$

$0^+ = \{0, 00, 000...\}$ This language doen not contain null string.

16. Write a regular expression to denote a language L that accepts all the strings which begin or end with 00 or 11.

Answer: The regular expression consists of two parts:

L_1 = (00 + 11) (any number of 0's and 1's)

= (00 + 11)(0 + 1)*

L_2 = (any number of 0's and 1's)(00 + 11)

= (0 + 1)*(00 + 11)

Hence, the needed RE $R = L_1 + L_2 = [(00 + 11)(0 + 1)^*] + [(0 + 1)^* (00 + 11)]$

17. Construct a RE for the language that accepts all strings with at least two c's over the set Σ = {c, b}.

Answer: (b + c)* c (b + c)* c (b + c)*

18. Construct a RE for the language over the set Σ = {a, b} in which total number of a's are divisible by 3.

Answer: (b* a b* a b* a b*)*

19. Write the regular set for following Regular Expression.

(i) (0 + 1)* (ii) (01)* (iii) (0 + 1) (iv) (0 + 1) +

Answer:

(i) (0 + 1)* = {ε, 0, 1, 01, 10, 001, 101, 101001,.....}, any combinations of 0's and 1's.

(ii) (01)* = {ε, 01, 0101, 010101,.....},
all combinations with the pattern 01.

(iii) (0 + 1) = 0 or 1, No other possibilities.

(iv) $(0 + 1)^+$ = {0, 1, 01, 10, 1000, 0101, ...}.

20. Give the regular expression denoting a language over $\Sigma = \{1\}$ and having (i) length of string even (ii) length of a string odd

Answer: (i) Even length of string: R = (11)*
(ii) Odd length of the string: R = 1(11)*

21. Give the regular expressions for language with

(i) strings over {0, 1} with the substring 0101,
(ii) strings beginning with 11 and ending with ab,
(iii) set of all strings over {a, b}with three consecutive b's,
(iv) set of all strings that end with 1and has no substring 00.

Answer: (i) (0 + 1)* 0101(0 + 1)*
(ii) 11(1 + a + b)* ab
(iii) (a + b)* bbb (a + b)*
(iv) (1 + 01)* (10 + 11)* 1

22. List out a few applications of regular expressions and finite automata.

Answer: Lexical analyzers and text editors are two applications.

Lexical analyzers: The tokens of the programming language can be expressed using regular expressions. The lexical analyzer scans the input program and separates the tokens.

For example, identifier can be expressed as a regular expression: (letter)(letter + digit)* where letter is [A –Za – z] and digit is [0-9]

If anything in the source language matches with the regular expression given above, it is recognized as an identifier. Thus, regular expressions can be used to identify the tokens in a language.

Text editors: These are programs used for processing the text. For example, UNIX text editors use the regular expressions for substituting the strings such as S/bbb*/b/

This expression indicates to replace multiple blank spaces with single blank. In UNIX text editors, any regular expression is converted to a NFA with ε-transitions. This NFA can then be simulated directly.

23. Give the regular expression for the language that accepts all strings in which a appears tripled over the set $\Sigma = \{a\}$.

Answer: The needed regular expression is (aaa)*.

24. What are the applications of the pumping lemma?

Answer: Pumping lemma is used to check if a language is regular or not.

(i) Assume that the language (L) is regular.
(ii) Select a constant n.
(iii) In L select a string (z) such that $|z| > n$.
(iv) Split the word z into u,v and w such that $|uv| \leq n$ and $|v| \geq 1$.
(v) If we reach a contradiction to the pumping lemma (i.e. if there exists an i such that UV^iW is not in L), then L is not a regular language.

25. Explain the closure property of regular sets?

Answer: The regular sets are closed under union, concatenation and Kleene closure:

$r_1 U r_2 = r_1 + r_2$
$r_1 . r_2 = r_1 r_2$
$(r)^* = r^*$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

The class of regular sets is closed under complementation, substitution, homomorphism and inverse homomorphism.

26. Give the regular expression for the language with the characteristic that every string has at least one a followed by at least one b.
Answer: $R = a^+b^+$

Fill in the Blanks

1. The regular expression for the language that contains the words with alphabets of size 4 is _____.
2. Applying the pumping lemma, we can show that some languages are not regular. (True/False).
3. If r and s are two regular expressions, then the expression r + s is also_____.
4. Every language defined by a finite automaton is a _____ language.
5. If the regular set A represented by A = (ab + b)*, and the regular set B is represented by B = ((ab)*b*)*, then the relation between A and B is _____.
6. The simplified form of regular expression (a(b + a)* + a)* is_____.
7. Language $L = \{a^{2n}/n>1\}$ is _____ .
8. Language L = {w/length(w) is prime} is _____.
9. Language $L = \{uv/u \in L, v \in L^R\}$ is _____.
10. Regular expressions are used in representing text patterns in UNIX-like operating systems. (True/False).
11. Left linear grammars are _____ of CFG.
12. The grammar S→aA | b, A→Bb | c, C→cC | c is regular. (True/False)

Answers

1. $[a - z]^4$
2. True
3. Regular expression
4. Regular
5. Both are equivalent
6. (a + b)*
7. Regular language
8. Not regular
9. Not regular
10. True
11. Subset
12. False

Objective Question Bank

1. Which of the following regular expression identities are true?
   a) (r + s)* = r*
   b) (r*s*)* = (r + s)*
   c) (r + s)* = r* + s*
   d) r*s* = r* + s*
2. Which of the following regular expressions over {0, 1) denote the set of all strings not containing 100 as a substring ?
   a) 0*(1 + 0)*
   b) 0*1010*
   c) 0*1*01
   d) 0*(10 + 1)*0*

3. The string 1101 does not belong to the set represented by
   a) 110*(0 + 1) b) 1(0 + 1)*101
   c) (10)*(01)*(00 + 11)* d) (00 + (11)* 0)*

4. Let r = 1*(1 + 0)*, s = 11*0 and t = 1*0 be three regular expressions. Which of the following is true?
   a) L(s) ⊆ L(r) and L(s) ⊆ L(t) b) L(r) ⊆ L(s) and L(s) ⊆ L(t)
   c) L(t) ⊆ L(s) and L(r) ⊆ L(s) d) L(s) ⊆ L(t) and L(s) ⊆ L(r)

5. Two of the following four regular expressions are equivalent. Which are those two ? (ε is the empty String)
   i) (00)*(ε + 0)
   ii) (00)*
   iii) 0*
   iv) 0(00)*
   a) i and ii b) ii and iii
   c) i and iii d) iii and iv

6. If the regular set A is represented by A = (01 + 1)* and the regular set B is represented by B = ((01)*1*)*, which of the following is true?
   a) A ⊂ B b) B ⊂ A
   c) A and B are in comparable d) A = B

7. Let S and T be languages over Σ = {a,b} represented, respectively, by the regular expressions (a + b*)* and (a + b)*. Which of the following is true?
   a) S ⊂ T b) T ⊂ S c) S = T d) S ∩ T = Ø

8. The regular expression 0*(10*)* denotes the same set as
   a) (0*1)*0* b) 0 + (0 + 10)*
   c) (0 + 1)*10(0 + 1)* d) none of the above.
   State which alternative is true.

9. Which of the following pair of regular expressions are not equivalent?
   a) 1(01)* and (10)*1 b) x(xx)* and (xx)*x
   c) (ab)* and a*b* d) x* and x*x*

10. Consider the following regular expressions:
    i) (a/b)*
    ii) (a*/b*)*
    iii) ((ε/a)b*)*

    Which of the following is correct?
    a) (i), (ii) are equal and (ii), (iii) are not
    b) (i) (ii) are equal and (i) (iii) are not
    c) (ii) (iii) are equal and (i) (iii) are not
    d) all are equal

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

11. How many strings of length less than 4 contains the language described by the regular expression (a + b)*b(a + ab)*?
    a) 7 b) 10 c) 12 d) 11

12. Which of the following is true?
    a) (ab)*a = a(ba)* and (P*Q*)* = (P* + Q*)*
    b) (a + b)*a(a + b)*b(a + b)* = (a + b)*ab(a + b)*
    c) (a + b)*ab(a + b)* + b*a = (a + b)*
    d) (a + b)*ab{(a + b)* + ab(a + b)*+ b*a*} + b*a* = (a + b)*
    e) all of the above

13. Which of the following is false?
    a) (a*b)*a* = a* (ba*)*
    b) (a*bbb)*a* = a* (bbba*)*
    c) (aa)* (ε + a) = a*a
    d) Let R, S and T be three languages and assume that ε is not in S. Then from the premise R = SR + T, we can conclude that R = S*T; and from the premise R = S*T, we can conclude that R = SR + T.

14. Which of the following definitions below generates the language L = [$x^n y^n$ such that $n \geq 1$]
    i. E → x E y/xy ii. xy/(x^+xyy^+) iii. x^+y^+
    a) i only b) i and ii c) ii and iii d) ii only

15. Consider the regular expression (0 + 1)(0 + 1)….., repeated n times. The minimum state finite automaton that recognizes the language represented by this regular expression contains
    a) n states b) n + 1 states c) n + 2 states d) n^2 states

16. What can be said about a regular language L over {a} whose minimal finite state automaton has two states?
    a) L must be {a^n/n is odd}
    b) L must be {a^n/n is a even}
    c) L must be {a^n/n≥ 0}
    d) Either L must be {a^n/n is odd} or it must be {a^n/n is even}

17. Consider the following languages:
    i) {Uv/U∈ L, V∈ L1 where L is regular}
    ii) {$a^n b^l a^k$/k ≥ n + l}
    iii) {$a^n b^l a^k$/n = l or l ≠ k}
    Which of the following is true ?
    a) ii) and (iii) are regular, but not (i)
    b) i) is regular, but not (ii) and (iii)
    c) all are regular sets
    d) none is regular

18. Suppose A is a finite set with n elements. The number of elements in the largest equivalence relation of A is
    a) n b) n^2 c) 1 d) n + 1

19. Choose the correct statements for regular languages.
    a) A class of languages that is closed under union and complementation has to be closed under intersection.
    b) Languages that are closed under intersection and complementation need not be closed under union.
    c) Languages that are closed under union and intersection need not be closed under complementation
    d) All of the above.

20. Read the following statements:
    i. For every NFA with an arbitrary number of final states, there is an equivalent NFA with only one final state.
    ii. Regular sets are closed under infinite union.
    iii. Regular sets are closed under inverse substitution.
    Which of the following is true?
    a) i and iii are the only correct statements
    b) i, ii and iii are correct statements
    c) i is the only correct statement
    d) None of the above is correct

21. Which of the following statement is true?
    a) if R is regular and N is non-regular, there exist R + N, which is regular
    b) if R is regular and N is non-regular, there exist R + N, which is not-regular
    c) $\{a^n/n$ is not a prime$\}$ is regular
    d) if we add a finite set of words to a regular language, the result is regular language

22. Let R_1 and R_2 be regular sets defined over the alphabet Σ. Then
    a) $R_1 \cap R_2$ is not regular
    b) $R_1 \cup R_2$ is not regular
    c) $\Sigma^* - R_1$ is regular
    d) R_1^* is not regular

23. Let $\Sigma = \{0, 1\}$, $L = \Sigma^*$ and $R = \{0^n1^n$ such that $n > 0\}$. Then, the languages L and R are, respectively,
    a) regualr, regular
    b) not regular, regular
    c) regular, not Regular
    d) not regular, not regular

24. Which of the following statements is false?
    a) Every finite subset of a non-regular set is regular
    b) Every subset of a regular set is regular
    c) Every finite subset of a regular set is regular
    d) Intersection of two regular sets is regular

25. Which of the following is regular?
    a) strings of 0's whose length is a perfect square
    b) set of all palindromes made up of 0's and 1's
    c) strings of 0's, whose length is a prime number
    d) strings of odd number of 0's

26. Consider the following statements:
    S_1: $\{0^{2^n}/n \geq 1\}$ is a regular language
    S_2: $\{0^m1^n0^{m+n}/m \geq 1, n \geq 1\}$ is a regular language.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Which of the following alternative(s) is (are) correct:

a) Only S_1 is correct
b) Only S_2 is correct
c) Both S_1 and S_2 are correct
d) None of S_1 and S_2 is correct

27. Which of the following sets can be recognized by DFS automata?
    a) The numbers 1, 2, 4,.....2^n,..... written in binary
    b) The numbers 1, 2, 4,.....2^n,..... written in unary
    c) The set of binary strings in which the number of 0's is same as the number of 1's
    d) The set {1, 101, 11011, 1110111,.....}

28. Let $L \subseteq \Sigma$ *where Σ = {a, b}. Which of the following is true?
    a) L = {x/x has an equal number of a's and b's} is regular
    b) L = {a^nb^n/n ≥ 1} is regular
    c) L = {x/x has more a's than b's} is regular
    d) L {a^mb^n/m ≥ 1, n ≥ 1} is regular

29. Consider the following languages:
    L_1 = {ww/w ∈ {a, b}*}
    L_2 = {ww^R/w ∈ {a, b}*, w^R is the reverse of w}
    L_3 = {a^nb^n/n = 0, 1....$10^{20 \text{ lakh}}$}
    L_4 = {0^i/i is an interger}
    Which of the languages given above are not regular ?
    a) L_1 and L_2 b) L_1, L_2 and L_3 c) L_3 and L_4 d) L_3

30. Consider the following languages:
    i) {a^nb^m/(n + m) is even}
    ii) {a^nb^m/n ≥ 1, m ≥ 1, nm ≥ 3}
    iii) The complement of {a^nb^m/n ≥ 4, m ≤ 3}
    Which of the following is true?
    a) i) and ii) are regular, but not (iii)
    b) ii) and iii) are regular, but not (i)
    c) All are regular sets
    d) (i) and iii) are regular, but not (ii)

31. Read the following statements:
    I. L = {vwv/v, w ∈ {a,b}*, {v} = 2} is regular
    II. The trancate operation defined to retain the right mosr symbols from given strings that are regular.
    Truncate (L) = {a/truncate (w) = a & W∈L}
    Truncate (L) = {truncate (w)/w ∈ L) is regular.,
    III. Let x = a_0a_1.....a_n, y = b_0b_1.....b_n, Z = c_0c_1.....c_n be binary numbers. The
    set of strings of triplets
    where the a_i, b_i, c_i are such that x + y = x is a regular language. State which of the following is true?
    a) I and II are the only correct statements
    b) I and III are the only correct statements
    c) II is the only correct statement
    d) I, II and III are correct statements

Answers

1. b	2. c	3. c	4. d	5. c	6. d	7. c	8. a	9. c	10. d
11. d	12. e	13. c	14. a	15. b	16. c	17. b	18. b	19. a	20. c
21. b	22. c	23. c	24. a	25. d	26. a	27. a	28. d	29. b	30. c
31. d									

Exercises

1. Define a regular expression and find a regular expression for the language L = {w/w ∈ {0,1}*} where w has no pair of consecutive zeroes.
2. Construct NFA for the regular expression (1 + 01 + 001)*(ε + 0 + 00).
3. Show that regular languages are closed under complementation?
4. Construct a NFA that accepts the following languages:
   a. L(aa* + aba*b*)
   b. L(ab(a + ab)*(a + aa))
   c. L(ab*aa + bba*ab)
   d. L(0* + 1*2*)
   e. L(10 + (0 + 11)0*1)
   f. L((a + ba)*bb(b + a)*)
5. Show that regular languages are closed under complementation and intersection?
6. Find the regular expression for the DFA

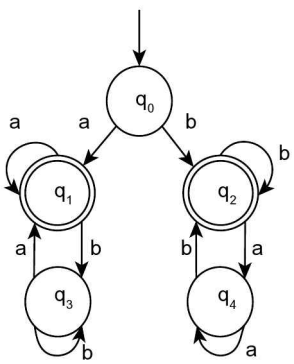


Fig. 3.45

7. Find the regular expression for the DFA

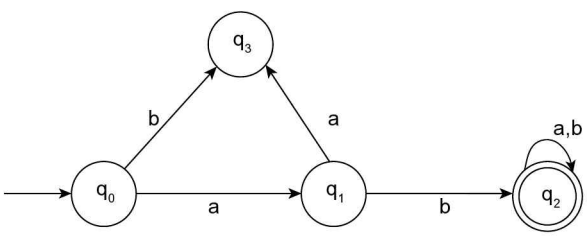


Fig. 3.46

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

8. Find the regular expression for the DFA

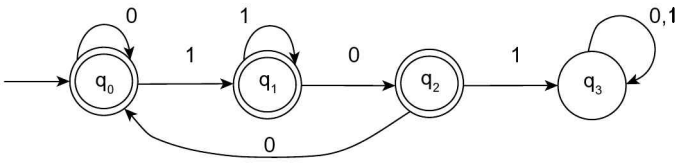


Fig. 3.47

9. Find the regular expression for the DFA

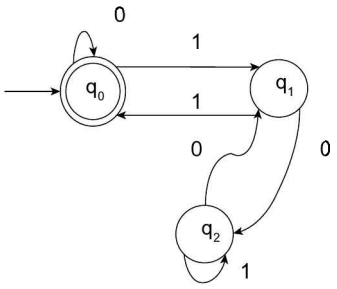


Fig. 3.48

10. State whether the language $L = \{a^i b^{3i} \mid i > 0\}$ is regular or not.
11. State whether the language $L = \{ww^r x \mid x, w \in \{0 + 1)^*\}$ is regular or not.
12. Show that $\{0^i 1^j \mid \gcd(i, j) = 1\}$ is not regular.
    Hint: Identify some integer such that $\gcd(i, j) \neq 1$
13. State whether the language $L = \{wxw^r \mid x, w \in \{0 + 1)^+\}$ is regular or not.
14. List the applications of regular expressions.
15. Obtain the right linear grammar and left linear grammar for the DFA

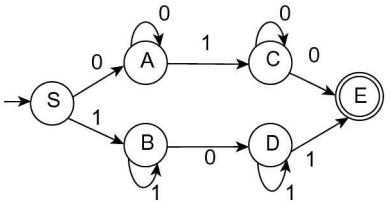


Fig. 3.49

Hint: Similar to Problem 25 in solved problems.

16. Obtain NFA for the following right linear grammar:
    S→ 01A | 10B,
    A→ 01C | 01,
    B→ 10D | 10,
    C→ 01A,
    D→ 10B

17. Obtain NFA for the following left linear grammar:
    S→ Aaa | Bbb,
    A→ Caa | aa,
    B→ Dbb | bb,
    C→ Aaa,
    D→ Bbb
18. For a regular expression of length n, what would be the minimum number of states required in NFA to accept the same language? Justify the answer.

    Hint: Use mathematical induction to prove your statement.
19. Using finite automaton construction, prove that the class of regular sets is closed under substitution.

    Hint: In the FA construction, formulate the procedure to replace the edge labelled a with the FA that accepts the language x where f(a) = x
20. Let p and q be regular expressions. Consider the equation X = pX + q where pX denotes the concatenation of p and X, and + denotes union. Under the assumption that the set denoted by p does not contain ε, find the solution for X and prove that it is unique. What is the solution if L(r) contains ε?

    Hint: Proof is similar to that of Arden's theorem.
21. Give an algorithm for solving the following set of linear equations which are formed from a finite automaton where a_{ij} and c_k are sets of strings denoted by regular expressions, + denotes set union, and multiplication denotes concatenation.

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & .. & a_{1n} \\ a_{21} & a_{22} & .. & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & .. & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}$$

    Hint: The algorithm is similar to solving the linear equations using matrices.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

This page is intentionally left blank

Context Free Grammars and Context Free Languages

4

- Syntax of high-level languages is defined with Context Free Grammars. Hence, these are used as a powerful tool by the parsers in verifying the syntax of any high-level language.

In this chapter, we will discuss (1) context free grammar, (2) Language, (3) derivation tree, (4) leftmost derivation and (5) rightmost derivation. We also discuss ambiguous grammars, simplification of grammars and normal forms.

4.1 Context Free Grammars

For a regular grammar, the productions are restricted in two ways: the left side must be a single variable, while the right side can be any string of terminals and non-terminals. To create grammars that are very powerful, we must ease off some of the restrictions. By permitting anything on the right, but retaining the restrictions on the left side, we get context free grammars.

Definition 1: A grammar G = (V, T, P, S) is said to be context free if all productions in P have the form $A \rightarrow x$ where $A \in V$ and $x \in (V \cup T)^*$.

V, T, P, S are the four important components in the grammatical description of a language.

V – the set of variables, also called ***non-terminals***. Each variable represents a set of strings, with which it can be replaced.

T – the set of terminals, which is the set of symbols that form the strings of the language. T is also called terminal symbols.

P – the finite set of ***productions*** or rules that represent the recursive definition of the language. Each production consists of a variable, production symbol ($\rightarrow$) and a string of terminals and non-terminals. The string is called the ***body*** of production.

S – the start symbol. It is one of the variables that represent the language being defined.

The language generated (defined, derived, produced) by a CFG is the set of all strings of terminals that can be produced from the start symbol S using the productions as substitutions. A language generated by a CFG is called a Context Free Language (CFL).

Example 4.1

terminal:	a
non-terminal:	S

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

productions: $S \rightarrow aS$
$S \rightarrow \varepsilon$
is a simple CFG that defines $L(G) = a^*$
where $V = \{S\}\ T = \{a\}$

Example 4.1 The CFG for defining palindrome over {a or b}.

The productions P are: $S \rightarrow \varepsilon \mid a \mid b$
$S \rightarrow aSa$
$S \rightarrow bSb$
and the grammar is $G = (\{S\}, \{a, b\}, P, S)$

Example 4.1 The CFG for the set of strings with equal number of a's and b's.

The productions P are

$$S \rightarrow SaSbS \mid SbSaS \mid \varepsilon$$

And the grammar $G = (\{S\}, \{a,b\}, P, S)$

Example 4.1 The context free grammar for generating syntactically correct algebraic expressions with the variables x, y and z:

Here the grammar $G = (\{S, T\}, \{+, *, (,), -, x, y, z\}, P, S)$

$$S \rightarrow T + S \mid T - S \mid T$$

$$T \rightarrow T*T \mid T \mid T$$

$$T \rightarrow (S)$$

$$T \rightarrow x \mid y \mid z$$

This grammar can generate the string $(x + y)*x - z*y \mid (x + x)$.

Example 4.1 A context free grammar for the language consisting of all strings (over {a, b}) in which number of a's is different from the number of b's is

$$S \rightarrow U \mid V$$
$$U \rightarrow T\,a\,U \mid T\,a\,T$$
$$V \rightarrow T\,b\,V \mid T\,b\,T$$
$$T \rightarrow a\,T\,b\,T \mid b\,T\,a\,T \mid \varepsilon$$

Here, T can generate all strings in which number of a's is same as the number of b's, U generates all strings with more a's than b's, and V generates all strings with less a's than b's.

Example 4.6 a) Give CFG for RE $(011 + 1)^* (01)^*$.

Solution:

CFG for $(011 + 1)^*$ is $A \rightarrow CA \mid \varepsilon$
$C \rightarrow 011 \mid 1$

CFG for (01)* is
$B \rightarrow DB \mid \varepsilon$
$D \rightarrow 01$

Hence, the final CFG is
$S \rightarrow AB$
$A \rightarrow CA \mid \varepsilon$
$C \rightarrow 011 \mid 1$
$B \rightarrow DB \mid \varepsilon$
$D \rightarrow 01$

Example 4.6 Give CFG for the language $L(G) = a^n b^{2n}$ where $n \geq 1$. Put differently, give the CFG for the language having strings with a's followed by b's where number of b's is twice the number of a's.

Solution: The given language is $a^n (bb)^n$.
Hence it can be defined by

$$S \rightarrow aSbb \mid abb$$

Example 4.6 Give CFG for the language containing all the strings (over $\Sigma = \{0, 1\}$) in which first and last symbols differ.

Solution: The strings should start and end with different symbols (0, 1). But in between, we can have any string on {0, 1}, that is, (0 + 1)*. Hence the language is

$$0(0 + 1)^*1 \mid 1(0 + 1)^*0.$$

The grammar is given by

$$S \rightarrow 0A1 \mid 1A0$$
$$A \rightarrow 0A \mid 1A \mid \varepsilon$$

4.2 Derivation of CFGs

It is a process of defining strings out of a grammar by application of the rules starting from the starting symbol. We can derive the terminal strings, beginning with start symbol and repeatedly replacing a variable/non-terminal by the body of the production. The language of CFG is the set of terminal symbols we can derive using these productions and is called a Context Free Language.

Example 4.6 Derive a^4 from by grammar

Terminal: a
Non-terminal: S
Productions: $S \rightarrow aS$

$$S \rightarrow \varepsilon$$

Solution:
The derivation for a^4 is

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$$\begin{aligned} S &\Rightarrow aS \\ &\Rightarrow aaS \\ &\Rightarrow aaaS \\ &\Rightarrow aaaaS \\ &\Rightarrow aaaa\varepsilon = aaaa \end{aligned}$$

The language has the strings {ε, a, aa, aaa, ...}.

Example 4.10 Derive a^2 from by grammar

Terminal: a
Nonterminal: S
Productions: $S \rightarrow SS$
$S \rightarrow a$
$S \rightarrow \varepsilon$

Solution: Derivation of a^2 can be achieved in many ways as shown.

$$\begin{aligned} S &\Rightarrow SS \\ &\Rightarrow Sa \\ &\Rightarrow aa \text{ (or)} \\ S &\Rightarrow SS \\ &\Rightarrow SSS \\ &\Rightarrow SSa \\ &\Rightarrow SSSa \\ &\Rightarrow SaSa \\ &\Rightarrow \varepsilon aSa \\ &\Rightarrow \varepsilon a\varepsilon a = aa \end{aligned}$$

Example 4.11 Find L(G) and derive the string abbab for the following grammar?

Terminals: a, b
Non-terminals: S
Productions:

$S \rightarrow aS$
$S \rightarrow bS$
$S \rightarrow a$
$S \rightarrow b$

Solution:
More compact notation of the given grammar is

$$S \rightarrow aS \mid bS \mid a \mid b$$

We can derive abbab as follows:

$$\begin{aligned} S &\Rightarrow aS \\ &\Rightarrow abS \\ &\Rightarrow abbS \\ &\Rightarrow abbaS \end{aligned}$$

$$\Rightarrow abbab$$

Context free language generated by the grammar is $(a + b)^+$.

Example 4.10 Find the language and derive abbaaba from the following grammar:

terminals: a, b
non-terminals: S, X
productions:

$$S \rightarrow XaaX$$

$$X \rightarrow aX \mid bX \mid \varepsilon$$

***Solution*:**

$$\text{CFL is } (a + b)^*aa(a + b)^*.$$

We can derive abbaaba as follows:

$$\begin{aligned} S &\Rightarrow XaaX \\ &\Rightarrow aXaaX \\ &\Rightarrow abXaaX \\ &\Rightarrow abbXaaX \\ &\Rightarrow abb\varepsilon aaX = abbaaX \\ &\Rightarrow abbaabX \\ &\Rightarrow abbaabaX \\ &\Rightarrow abbaaba\varepsilon \Rightarrow abbaaba \end{aligned}$$

4.3 Understanding the Language Defined by Grammars

The only way to recognise the language is to try out various strings from the given production rules. Simply by observing the derived strings, one can find out the language generated from the given CFG.

Example 4.11 Give the language defined by grammar

$$G = \{\{S\}, \{a\}, \{S \rightarrow SS\}, S\}$$

Solution: $L(G) = \Phi$. Since there is no terminal that is derived from S.

Example 4.10 Give the language defined by grammar

$G = \{\{S, C\}, \{a, b\}, P, S\}$ where P is given by

$$S \rightarrow aCa$$
$$C \rightarrow aCa \mid b$$

Solution:

$$S \Rightarrow aCa$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$\Rightarrow$ aaCaa
$\Rightarrow$ aaaCaaa
$L(G) = \{a^n ba^n / n \geq 1\}$.

Example 4.15 Give the language defined by grammar

$G = \{\{S\}, \{0, 1\}, P, S\}$ where P is given by
$S \rightarrow 0S1 \mid \varepsilon$

Solution: $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$.

$$L(G) = \{0^n 1^n / n \geq 0\}.$$

4.3.1 Leftmost and Rightmost Derivations

The leftmost non-terminal in a working string is the first non-terminal that we encounter when we scan the string from left to right.

For example, in the string bbabXbaY SbXbY, the leftmost non-terminal is X.

Definition 2: If a word w is generated by a CFG by a certain derivation and at each step in the derivation, a rule of production is applied to the leftmost non-terminal in the working string, then this derivation is called a leftmost derivation (LMD).

Practically, whenever we replace the leftmost variable first in a string, then resulting derivation is called the leftmost derivation. Similarly, **replacing the rightmost variable first at every step gives the rightmost derivation RMD.**

Example 4.15 Consider the CFG ({S, X}, {a, b), P, S)

where productions are

$$S \rightarrow baXaS \mid ab$$

$$X \rightarrow Xab \mid aa$$

Find LMD and RMD for string w = baaaababaab.

Solution: The following is a LMD:

$S \Rightarrow baXaS$ {as $S \rightarrow baXaS$}
$\Rightarrow baXabaS$ {as $X \rightarrow Xab$}
$\Rightarrow baXababaS$ {as $X \rightarrow Xab$}
$\Rightarrow baaaababaS$ {as $X \rightarrow aa$}
$\Rightarrow baaaababaab$ {as $S \rightarrow ab$}

The following is a RMD:

$S \Rightarrow baXaS$ {as $S \rightarrow baXaS$}
$\Rightarrow baXaab$ {as $S \rightarrow ab$}
$\Rightarrow baXabaab$ {as $X \rightarrow Xab$}
$\Rightarrow baXababaab$ {as $X \rightarrow Xab$}
$\Rightarrow baaaababaab$ {as $X \rightarrow aa$}

Any word that can be generated by a given CFG can have LMD|RMD.

Example 4.15 Consider the CFG:

$$S \rightarrow aB \mid bA$$
$$A \rightarrow a \mid aS \mid bAA$$
$$B \rightarrow b \mid bS \mid aBB$$

Find LMD and RMD for (the string) w = aabbabba.

***Solution*:** The following is a LMD:

$$\begin{aligned} S &\Rightarrow aB \Rightarrow aaBB \\ &\Rightarrow aabSB \\ &\Rightarrow aabbAB \\ &\Rightarrow aabbaB \\ &\Rightarrow aabbabS \\ &\Rightarrow aabbabbA \\ &\Rightarrow aabbabba \end{aligned}$$

The following is a RMD:

$$\begin{aligned} S &\Rightarrow aB \Rightarrow aaBB \\ &\Rightarrow aaBbS \\ &\Rightarrow aaBbbA \\ &\Rightarrow aaBbba \\ &\Rightarrow aabSbba \\ &\Rightarrow a\ abbAbba \\ &\Rightarrow aabbabba \end{aligned}$$

4.3.2 Derivation Tree

The derivation process can be shown pictorially as a tree to illustrate how a word is derived from a CFG. These trees are called syntax trees, parse trees or derivation trees. These trees show us clearly how the symbols of the terminal string are grouped into substrings, each of which belongs to the language of one of the variables of the grammar.

For constructing a parse tree for a grammar G = (V, T, P, S),

- root is start symbol
- each node is labeled by either a variable, a terminal or ε.
- each interior node is marked by variable in V
- if an interior node is labeled A, and its children are labeled $X_1, X_2, \ldots, X_k$, from the left, then $A \rightarrow X_1, X_2, \ldots, X_k$ is a production in P.

Example 4.15 CFA with

Terminals: a, b
Non-terminals: S, A

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Production S → AAA | AA

A→ AA | aA | Ab | a | b

String "baaba" has derivation tree: as shown in Figure 4.1

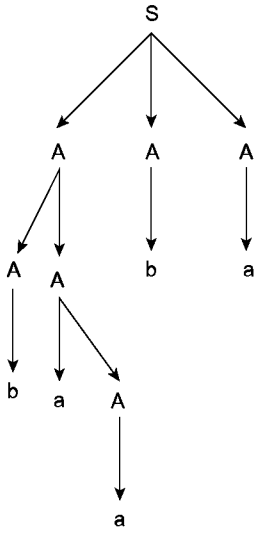


Fig. 4.1 *Derivation Tree for "baaba"*

When we concatenate the leaves of any parse tree from the left, we get a string, which is known as **yield** of the tree. The yield is a string that is always derived from the root variable. There are parse trees whose yields are in the language of the underlying grammar. The importance of tree is:

- The yield is a terminal string. All leaves are labeled with a terminal or a ε.
- The root is labeled by start symbol.
- At any intermediate step, if we concatenate the elements, we get a string in the sentential form.

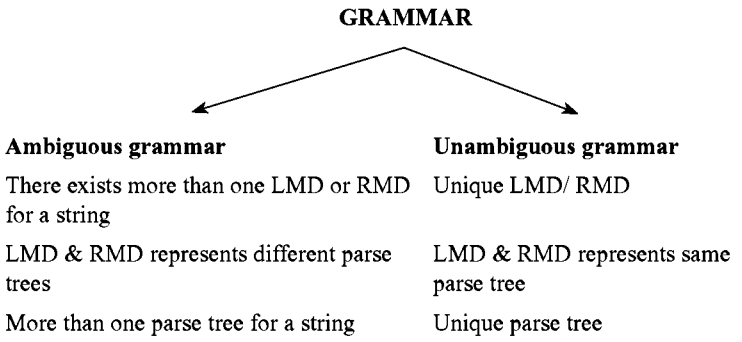
4.3.3 Equivalence of Parse Trees and Derivations

A terminal string is in the language of a grammar iff it is the yield of at least one parse tree. The existence of leftmost derivation, rightmost derivation and parse trees are equivalent, in the sense that each defines exactly the strings in the language of a CFG.

With some CFGs, it may be possible to find a terminal string with more than one parse tree or equivalently, with more than one leftmost derivation or one rightmost derivation. Such a grammar is called **ambiguous**.

4.4 Ambiguous Grammar

Definition 3: A CFG is ambiguous if there exist more than one parse tree, or equivalently, if there exist more than one leftmost derivation and thus there exist more than one rightmost derivation, for at least one word in its CFL.



Example 4.15 Show that the following grammar is ambiguous.

$$E \rightarrow id \mid E + E \mid E * E \mid E - E$$

Solution: LMD: for string id + id*id, the derivation is

$$\begin{aligned} E &\Rightarrow \underline{E} + E \\ &\Rightarrow id + \underline{E} \\ &\Rightarrow id + \underline{E} * E \\ &\Rightarrow id + id * \underline{E} \\ &\Rightarrow id + id * id \end{aligned}$$

string id + id * id, can also be derived as

$$\begin{aligned} &\Rightarrow \underline{E} * E \\ &\Rightarrow \underline{E} + E * E \\ &\Rightarrow id + \underline{E} * E \\ &\Rightarrow id + id * id \end{aligned}$$

Parse trees represented by the above derivation is shown in Figure 4.2:

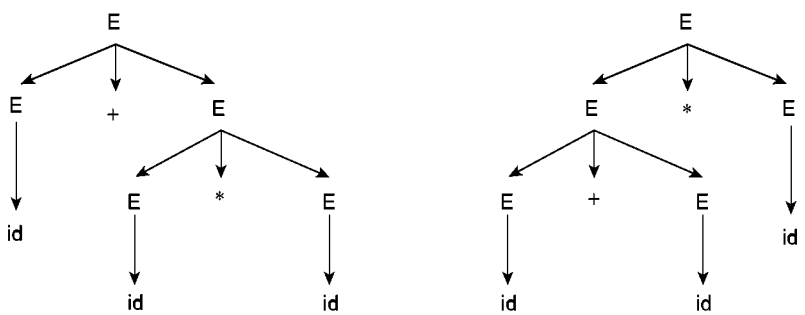


Fig. 4.2 *Parse Tree for 'id+id*id'*

As there are more than one parse tree, the grammar is ambiguous.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example 4.20 Consider the Grammar G with

terminals: a, b
non-terminals: S
productions: S → aS | Sa | a.
Show that G is ambiguous.

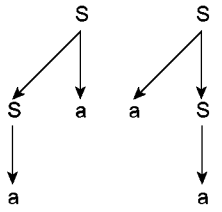


Fig. 4.3 *Parse Tree for 'aa'*

Solution: The word aa can be generated by two different trees as shown in the Figure 4.3. Therefore, grammar G is ambiguous.

Example 4.15 Consider the Grammar G with

terminals: a, b
non-terminals: S, X
productions: S → aS | aSb | X
X → Xa | a
Show that G is ambiguous.

Solution: The word "aa" has two different derivations that correspond to the following two different syntax trees:

S ⇒ X ⇒ Xa ⇒ aa

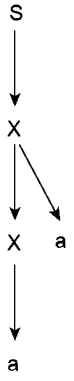


Fig. 4.4(a) *Tree for "aa"*

S ⇒ aS ⇒ aX ⇒ aa

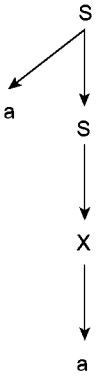


Fig. 4.4(b) *Tree for "aa"*

As there are two parse trees as shown in the Figure 4.4(a) and (b), the grammar is ambiguous.

Example 4.15 The grammar G for PALINDROMES is

S → aSa | bSb | a | b | ε. Check if G is ambiguous.

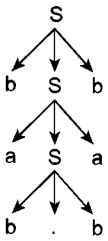


Fig. 4.5 *Parse Tree for 'babbab'*

Solution: The grammar can generate the string 'babbab' as follows:

$$\begin{aligned} S &\Rightarrow bSb \\ &\Rightarrow baSab \\ &\Rightarrow babSbab \\ &\Rightarrow babbab \end{aligned}$$

which has derivation tree shown in the Figure 4.5.

Since there is only one parse tree, the grammar is unambiguous.

Example 4.15 Check whether the following grammar is ambiguous or not.

$$S \rightarrow i\,C\,t\,S \mid i\,C\,t\,S\,e\,S \mid a$$
$$C \rightarrow b$$

Solution: To check the ambiguities consider a suitable input string. Let string be "ibtibtaea".

This has two derivation trees shown in the Figure 4.6.

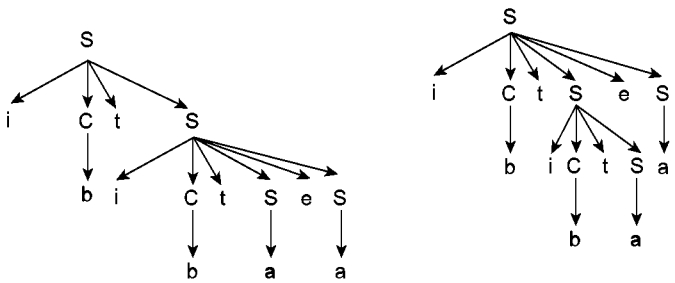


Fig. 4.6 *Tree for "ibtibtaea"*

Since there are two possible parse trees the grammar is ambiguous.

4.4.1 Removing Ambiguity

There is no algorithm that straightaway converts an ambiguous grammar to equivalent unambiguous grammar. But on analysing the grammar, if we identify what is missing in the grammar and why it is unambiguous, then we can write equivalent unambiguous grammars.

For example, consider expression grammar given below:

$$Expr \rightarrow Expr + Expr \mid Expr * Expr \mid id$$

If we take a string id + id * id or id + id + id we get two parse trees.

So if we analyse the grammar with the above two strings, we understand the following.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

1. Precedence of operators is not taken care of; hence you can derive the string either replacing Expr by Expr + Expr or by Expr * Expr.
2. Associative property is also not taken care of. If we take a string id + id + id, we can replace Expr → Expr + Expr either the left Expr or right Expr.

So, write equivalent unambiguous grammar by taking care of precedence and associativity.

To take care of **precedence,** rewrite the grammar by defining rules starting with the lowest precedence to the highest precedence.

For example, in the given grammar id has the highest precedence, then is * and least precedence is for +.

So do not define all of them at the same level. Separate them into different levels by introducing extra non-terminals. Start defining the rules with +, then with * and finally with id:

$$EE \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow id$$

To ensure **associativity**, define the rule as left recursive if the operator is left associative. Define the rule as right recursive if the operator is right associative.

In the given grammar, + and * are left associative. So the rules must be left recursive, that is, A → Aa | b. The equivalent unambiguous grammar is

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow id$$

This procedure can be used for any expression grammar.

4.4.2 Inherent Ambiguity

A CFL is said to be **inherently ambiguous** if it is defined with only ambiguous grammar. It cannot be defined with unambiguous grammar. Following is an example of a grammar for an inherently ambiguous language:

$L = \{a^i b^j c^k d^l \mid i = j, k = 1\}$
$S \rightarrow AB \mid C$
$A \rightarrow aAb \mid ab$
$B \rightarrow cBd \mid cd$
$C \rightarrow aCd \mid aDd$
$D \rightarrow bDc \mid bc$

4.5 Simplification of Grammars

We have seen various, languages can effectively be represented by CFG. All grammars are not always optimized. That means, a grammar may contain some extra/unnecessary symbols. These will increase the length of the grammar. Simplification of the grammar involves removing all these unnecessary symbols. For example, look at following grammar:

$$S \rightarrow AB$$
$$A \rightarrow a$$
$$B \rightarrow b \mid C$$
$$E \rightarrow c \mid \varepsilon$$

we can observe that
Here, C never defines any terminal and E is never reachable from S.

- E and C do not appear in any sentential form.
- $E \rightarrow \varepsilon$ is a null production.
- $B \rightarrow C$ simply replaces B by C.

It is clear that removing these symbols and productions it would not effect the language generated, Hence the grammar can be simplified as follows:

$$S \rightarrow AB$$
$$A \rightarrow a$$
$$B \rightarrow b$$

Simplification of the grammar, generally, includes the following:

1. Elimination of useless symbols
2. Elimination of ε productions
3. Elimination of unit productions of the form $A \rightarrow B$

4.5.1 Elimination of Useless Symbols

Definition 4: A symbol is useless if it cannot derive a terminal or it is not reachable from the start symbol.

To check if the symbol is reachable from the start symbol, we can use dependency graph or by using the following lemmas.

Lemma 1: If the grammar $G = (V, T, P, S)$ with $L(G) \neq \Phi$, we can effectively find an equivalent grammar $G' = (V', T, P', S)$ such that, for each A in V', there is some w in T^* for which $A \overset{*}{\Rightarrow} w$.

Proof: If $A \rightarrow w$ is a production where $w \in T^*$, then A is in V'. If $A \rightarrow X_1X_2 \ldots X_n$ is a production where each X_i is in V' then A is also in V'. The set V' can be computed by the following algorithm:

1. Initialize $V_1 = \Phi$.
2. Include A into V_2 where $A \rightarrow w$ for some w in T^*.
3. Repeat Steps 4 and 5 while $V_1 \neq V_2$.
4. $V_1 = V_2$
5. $V_2 = V_1 \cup \{A\}$ where $A \rightarrow \alpha$ for some α in $(T \cup V_1)^*$
6. $V' = V_2$

From the above algorithm, find all $A \in V'$; now include only those productions which include $V' \cup T$.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Lemma 2: Given a grammar G = (V, T, P, S), we can effectively find an equivalent grammar G′ = (V′, T, P′, S) such that, for each X in V′ ∪ T′, there exist α and β in (V′ ∪ T′)* for which $S \overset{*}{\Rightarrow} \alpha X \beta$.

Proof: Procedure to find G′ is given below.

1. Initially include S into V'.
2. For all productions $A \rightarrow \alpha_1 |\alpha_2| .. | \alpha_n$ then add the variables of $\alpha_1, \alpha_2, \ldots \alpha_n$ to V′ and all terminal to T′.
3. P′ is the set of productions which includes symbols of V′ ∪ T'.

Dependency graph can be drawn to identify the symbols that are reachable. To draw dependency graph all non-terminals are indicated as nodes for each production $A \rightarrow x_1, x_2, \ldots x_n$ place an edge from A to x_i where x_i is non terminal. The set of nodes that have path from start node indicate the non terminals that are reachable.

Example 4.15 Eliminate useless symbols and productions from the following grammar

$$S \rightarrow ABa \mid BC, A \rightarrow aC \mid BCC, C \rightarrow a, B \rightarrow bcc,$$
$$D \rightarrow E, E \rightarrow d, F \rightarrow e$$

Solution:

Step 1: Eliminate useless symbols: All variables are found to be useful as each of them derive a terminal.

Step 2: Elimination of non-reachable variables: Draw the dependency graph as shown in the Figure 4.7.

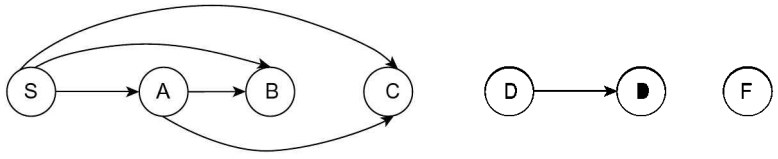


Fig. 4.7 *Graph Representation to Identify Useless Non Terminals*

- D, E and F are non-reachable from S.
- After removing useless symbols, we have

$$S \rightarrow ABa \mid BC$$
$$A \rightarrow aC \mid BCC$$
$$C \rightarrow a$$
$$B \rightarrow bcc$$

Example 4.15 Eliminate useless symbols in the following grammar G:

$$S \rightarrow BC \mid AB \mid CA$$
$$A \rightarrow a$$
$$C \rightarrow aB \mid b$$

Solution: Here B is not defined; hence useless. C and A are reachable and are deriving terminals. Hence, C and A are useful. The reduced grammar is

$$S \rightarrow CA$$
$$A \rightarrow a$$
$$C \rightarrow b$$

Example 4.20 Eliminate useless symbols in the given G:

$$S \rightarrow aAa$$
$$A \rightarrow bBB$$
$$B \rightarrow ab$$
$$C \rightarrow a\,b$$

Solution: Here C is useless, as it is not reachable from start symbol.
So the reduced grammar is

$$S \rightarrow aAa$$
$$A \rightarrow bBB$$
$$B \rightarrow ab$$

Example 4.15 Eliminate useless symbols in the following grammar G:

$$S \rightarrow aS \mid A \mid BC$$
$$A \rightarrow a$$
$$B \rightarrow aa$$
$$C \rightarrow a\,Cb$$

Solution: Here C is useless, as it is not deriving any string. B is not reachable.
So the reduced grammar is

$$S \rightarrow aS \mid A$$
$$A \rightarrow a$$

4.5.2 Elimination of ε-Productions

If some CFL contains the word ε, then the CFG must have an ε-production. However, if a CFG has an ε-production, then the CFL does not necessarily contain ε.
For example,

$$S \rightarrow aX$$
$$X \rightarrow \varepsilon$$

and that defines the CFL {a}.
Nullable variable: In a given CFG, a non-terminal X is nullable if

1. There is a production $X \rightarrow \varepsilon$
2. There is a derivation that starts at X and leads to ε:

$$X \Rightarrow \ldots \Rightarrow \varepsilon, \text{ i.e. } X \overset{*}{\Rightarrow} \varepsilon.$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

For any language L, define the language L_0 as follows:

1. if $\varepsilon \notin L$, then L_0 is the entire language L, that is, $L_0 = L$.
2. if $\varepsilon \in L$, then L_0 is the language $L - \{\varepsilon\}$; that is, L_0 is the set of all words in L except ε.

Note: If L is a CFL generated by a CFG G_1 that includes ε-productions, then there is another CFG G_2 with no ε-productions that generates L_0.

Procedure for Eliminating ε-Productions

(a) Construct V_n, the set of all nullable variables
(b) For each production $B \rightarrow A$, if A is a nullable variable, replace the nullable variable A by ε, and add, all possible combinations of strings on the RHS of production.
(c) Do not add the production $A \rightarrow \varepsilon$

Example 4.28 Eliminate the null production from the following grammar:

$S \rightarrow ABaC$
$A \rightarrow BC$
$B \rightarrow b \mid \varepsilon$
$C \rightarrow D \mid \varepsilon$
$D \rightarrow d$

Solution: Nullable variables are given by $V_n = \{B, C, A\}$
Hence equivalent grammar without null productions is

$S \rightarrow ABaC \mid BaC \mid AaC \mid ABa \mid aC \mid Aa \mid Ba \mid a$
$A \rightarrow BC \mid B \mid C$
$B \rightarrow b$
$C \rightarrow D$
$D \rightarrow d$

Example 4.29 Eliminate the null production from the following grammar:

$S \rightarrow aA$
$A \rightarrow BB$
$B \rightarrow aBb \mid \varepsilon$

Solution: Nullable variables are $V_n = \{A, B\}$
Hence equivalent grammar without null productions is

$S \rightarrow aA \mid a$
$A \rightarrow BB \mid B$
$B \rightarrow aBb \mid ab$

4.5.3 Eliminating Unit Productions

Definition 5: A unit production is a production of the form $A \rightarrow B$.

If a language L is generated by a CFG G_1 that has no ε-productions, then there is also a CFG G_2 for L with no ε-productions and no unit productions. In general elimination of ε-productions, may introduce unit productions in the grammar.

Procedure for Eliminating Unit Productions

- For each pair of non-terminals A and B such that there is a production

$$A \rightarrow B \text{ and the non-unit productions from B are}$$
$$B \rightarrow s_1 \mid s_2 \mid \ldots \mid s_n$$

where $s_i \in (V + T)^*$ are strings of terminals and non-terminals, then create the new productions as

$$A \rightarrow s_1 \mid s_2 \mid \ldots \mid s_n$$

  - Do the same for all such pairs A and B simultaneously.
  - Remove all unit productions.

Example 4.20 Eliminate unit productions in the grammar

$$S \rightarrow A \mid bb$$
$$A \rightarrow B \mid b$$
$$B \rightarrow S \mid a$$

Solution: After eliminating unit productions $S \rightarrow A$, $A \rightarrow B$, $B \rightarrow S$ we get

$$S \rightarrow a \mid b \mid bb, A \rightarrow a \mid b \mid bb, B \rightarrow a \mid b \mid bb$$

Example 4.15 Eliminate unit production from the grammar given below:

$$S \rightarrow Aa \mid B, B \rightarrow A \mid bb, A \rightarrow a \mid bc \mid B$$

Solution: Unit productions are $S \rightarrow B$, $B \rightarrow A$ and $A \rightarrow B$

- A, B and S are derivable
- Eliminating B in the A production gives $A \rightarrow a \mid bc \mid bb$.
- Eliminating A in the B production gives $B \rightarrow a \mid bc \mid bb$.
- Eliminating B in the S production gives $S \rightarrow Aa \mid a \mid bc \mid bb$

- The final set of productions after eliminating unit productions are given below:

$$S \rightarrow Aa \mid a \mid bc \mid bb$$
$$B \rightarrow a \mid bc \mid bb$$
$$A \rightarrow a \mid bc \mid bb$$

Example 4.15 Simplify the following grammar:

$$S \rightarrow aA \mid aBB$$
$$A \rightarrow aAA \mid \varepsilon$$
$$B \rightarrow bB \mid bbC$$
$$C \rightarrow B$$

Solution: Here it is better to eliminate null productions as this may introduce useless symbols and Unit productions. Next eliminate unit productions and at the end eliminate useless symbols.
Removing ε- productions gives resulting grammar as

$$S \rightarrow aA \mid a \mid aBB$$
$$A \rightarrow aAA \mid aA \mid a$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$B \rightarrow bB \mid bbC$
$C \rightarrow B$

Eliminating unit productions, we get the resulting grammar as

$S \rightarrow aA \mid a \mid aBB$
$A \rightarrow aAA \mid aA \mid a$
$B \rightarrow bB \mid bbC$
$C \rightarrow bB \mid bbC$

B and C are identified as useless symbols. Eliminating these, we get

$S \rightarrow aA \mid a$
$A \rightarrow aAA \mid aA \mid a$

Finally, the reduced grammar is $S \rightarrow aA \mid a$, $A \rightarrow aAA \mid aA \mid a$ which defines any number of a's.

4.6 Normal Forms

As we have seen, the grammar can be simplified by reducing the ε productions, removing both useless symbols and unit productions. There is also a need to have the grammar in a specific form. As you have seen in CFG, on the right-hand side of production, there are any number of terminals and non-terminals, in any combination. We need to normalize such a grammar. to standardise the processing of strings That means we want the grammar in some specific format. There should be a fixed number of terminals or non-terminals, in context free grammar with some criteria. There are two important normal forms: The Chomsky Normal Form and the Greibach Normal Form. We will study these two normal forms with help of examples.

4.6.1 The Chomsky Normal Form

Definition 6: A CFG is in Chomsky Normal Form (CNF) if each of its productions has one of the two forms:

1. Non-terminal → string of exactly two Non-terminals, i.e., $A \rightarrow BC$
2. Non-terminal → one terminal, i.e., $A \rightarrow a$

In CNF, number of symbols on the right-hand side of production is strictly limited. The nature of symbols on the right-hand side is also restricted.

Procedure for Converting a Given Grammar to CNF

1. Eliminate null productions and unit productions.
2. Include productions of the form $A \rightarrow BC \mid a$ as it is.
3. Eliminate strings of terminals on the right-hand side of production if they exceed one as follows:

   Suppose we have the production

   $S \rightarrow a_1a_2a_3$ where $a_1a_2a_3$ are terminals

   Then introduce non-terminal C_{ai} for terminal a_i as

   $C_{a1} \rightarrow a_1$, $C_{a2} \rightarrow a_2$, $C_{a3} \rightarrow a_3$

4. To restrict the number of variables on the right-hand side, introduce new variables and separate them as follows:

   Suppose we have the production with n non-terminals as shown below with 5 non-terminals

   $$Y \rightarrow X_1 X_2 X_3 X_4 X_5$$

   Add n-2 new productions using n-2 new non-terminals and modify the production as in the following:

   $$Y \rightarrow X_1 R_1$$
   $$R_1 \rightarrow X_2 R_2$$
   $$R_2 \rightarrow X_3 R_3$$
   $$R_3 \rightarrow X_4 X_5$$

   where the R_i are new non-terminals.

   The language generated by the new CFG is the same as that generated by the original CFG.

Example 4.15 Convert following CFG to CNF.

$S \rightarrow bA \mid aB$
$A \rightarrow bAA \mid aS \mid a$
$B \rightarrow aBB \mid bS \mid b$

***Solution*:** There are no null or unit productions.
$A \rightarrow a$, $B \rightarrow b$ are in the required format, so they are straightaway included.
Other productions are not in CNF. So, replace every terminal by the following variables:

$S \rightarrow C_b A \mid C_a B$
$A \rightarrow C_b AA \mid C_a S \mid a$
$B \rightarrow C_a BB \mid C_b S \mid b$
$C_a \rightarrow a$, $C_b \rightarrow b$

$A \rightarrow C_b AA$ and $B \rightarrow C_a BB$ are the only two productions that are not in CNF. So add two new non-terminals D and E, one for the first production and the other for the second production. And add two new productions as follows:

$S \rightarrow C_b A \mid C_a B$
$A \rightarrow C_b D \mid C_a S \mid a$
$B \rightarrow C_a E \mid C_b S \mid b$
$C_a \rightarrow a$, $C_b \rightarrow b$
$D \rightarrow AA$
$E \rightarrow BB$

This is the CNF.

Example 4.15 Convert following CFG to CNF:

$S \rightarrow AB \mid aB$
$A \rightarrow aab \mid \varepsilon$
$B \rightarrow bbA$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Solution: Eliminate ε productions:

$$S \rightarrow AB \mid B \mid aB$$
$$A \rightarrow aab$$
$$B \rightarrow bbA \mid bb$$

Eliminate unit productions:

$$S \rightarrow AB \mid bbA \mid aB \mid bb$$
$$A \rightarrow aab$$
$$B \rightarrow bbA \mid bb$$

Replace the terminal by a variable:

$$S \rightarrow AB \mid C_b C_b A \mid C_a B \mid C_b C_b$$
$$A \rightarrow C_a C_a C_b$$
$$B \rightarrow C_b C_b A \mid C_b C_b$$
$$C_a \rightarrow a, C_b \rightarrow b$$

Restrict to two variables on the right-hand side of each production:

$$S \rightarrow AB \mid C_b D \mid C_a B \mid C_b C_b$$
$$A \rightarrow C_a E$$
$$B \rightarrow C_b D \mid C_b C_b$$
$$C_a \rightarrow a, C_b \rightarrow b,$$
$$D \rightarrow C_b A,$$
$$E \rightarrow C_a C_b$$

This is the required CNF.

Example 4.15 Convert following CFG to CNF.

$$S \rightarrow ASB \mid \varepsilon, A \rightarrow aAS \mid a, B \rightarrow SbS \mid A \mid bb$$

Solution: **Step 1:** Simplify the grammar with productions P.

Step 1a: Eliminate ε-productions to obtain P_1.

P_1 is

$$S \rightarrow ASB \mid AB$$
$$A \rightarrow aAS \mid a \mid aA$$
$$B \rightarrow SbS \mid A \mid bb \mid Sb \mid bS \mid b$$

Step 1b: Eliminate unit productions to obtain P_2.

P_2 is

$$S \rightarrow ASB \mid AB$$
$$A \rightarrow aAS \mid a \mid aA$$
$$B \rightarrow SbS \mid bb \mid Sb \mid bS \mid b \mid aAS \mid a \mid aA$$

Step 1c: Eliminate useless symbols to obtain P_3.

All variables are generating and all are reachable.

Simplified grammar $G' = (\{S, A, B\}, \{a, b\}, P, S)$ where P is

$$S \rightarrow ASB \mid AB$$
$$A \rightarrow aAS \mid a \mid aA$$
$$B \rightarrow SbS \mid bb \mid Sb \mid bS \mid b \mid aAS \mid a \mid aA$$

Step 2: Convert G′ to CNF.

Step 2a: Add productions of the form A →BC, A →a

P″ is

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b \mid a$

Step 2b: Eliminate terminals from RHS of the other productions:

$A \rightarrow aAS \Rightarrow A \rightarrow C_aAS$ and $C_a \rightarrow a$

$A \rightarrow aA \Rightarrow A \rightarrow C_aA$

$B \rightarrow SbS \Rightarrow B \rightarrow SC_bS$ and $C_b \rightarrow b$

$B \rightarrow bb \Rightarrow B \rightarrow C_bC_b$

$B \rightarrow bS \Rightarrow B \rightarrow C_bS$

$B \rightarrow Sb \Rightarrow B \rightarrow SC_b$

$B \rightarrow aAS \Rightarrow B \rightarrow C_aAS$

$B \rightarrow aA \Rightarrow B \rightarrow C_aA$

Add productions that are in CNF form to P″

$S \rightarrow AB$

$A \rightarrow a \mid C_aA$

$B \rightarrow C_bC_b \mid C_bS \mid SC_b \mid C_aA \mid b \mid a$

$C_a \rightarrow a$

$C_b \rightarrow b$

Step 2c: Reduce the RHS of the remaining productions with more than 2 variables to the required normal form

$S \rightarrow ASB \Rightarrow S \rightarrow AC_4$ and $C_4 \rightarrow SB$

$A \rightarrow C_aAS \Rightarrow A \rightarrow C_aC_1$ and $C_1 \rightarrow AS$

$B \rightarrow SC_bS \Rightarrow B \rightarrow SC_2$ and $C_2 \rightarrow C_bS$

$B \rightarrow C_aAS \Rightarrow B \rightarrow C_aC_3$ and $C_3 \rightarrow AS$

Adding these productions to P″; the complete grammar is

$S \rightarrow AB \mid AC_4$

$A \rightarrow a \mid C_aA \mid C_aC_1$

$B \rightarrow C_bC_b \mid C_bS \mid SC_b \mid C_aA \mid SC_2 \mid C_aC_3 \mid b \mid a$

$C_a \rightarrow a, C_b \rightarrow b, C_1 \rightarrow AS, C_2 \rightarrow C_bS, C_3 \rightarrow AS$

- The grammar in CNF form G″ = (V″, {a, b}, P″, S)
- V″ = {S, A, B, C_a, C_b, C_1, C_2, C_3}
- P″ is

$S \rightarrow AC_4 \mid AB \mid$

$A \rightarrow a \mid C_aA \mid C_aC_1$

$B \rightarrow C_bC_b \mid C_bS \mid SC_b \mid C_aA \mid SC_2 \mid C_aC_3 \mid b \mid a$

$C_a \rightarrow a, C_b \rightarrow b, C_1 \rightarrow AS, C_2 \rightarrow C_bS, C_3 \rightarrow AS, C_4 \rightarrow SB$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

4.6.2 The Greibach Normal Form

Here, we put restriction not only on the length of right sides of production, but also in connection with the positions at which terminals and variables can appear.

Definition 7: A context free language is said to be in the Greibach normal form if all productions are of the form $A \rightarrow a\alpha$ where $a \in T$ and $\alpha \in V^*$.
We may be able to convert a grammar to GNF.

For converting a given grammar to GNF, we use two lemmas.

Lemma1 (Substitution rule)

Let $G = \{V, T, P, S\}$ be a CFG. Let $A \rightarrow B\alpha$ be a production in P if there is a production $B \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \beta_4 \ldots\ldots\ldots$ Then the equivalent grammar can be obtained by substituting B in A. The resulting grammar is

$$A \rightarrow \beta_1\alpha \mid \beta_2\alpha \mid \beta_3\alpha \mid \beta_4\alpha$$

Lemma 2 (Elimination of left recursion)

Grammar of the form $A \rightarrow A\alpha \mid \beta$ is called left-recursive grammar. To eliminate left recursion, rewrite grammar as

$$A \rightarrow \beta A'$$
$$A' \rightarrow \alpha A' \mid \varepsilon.$$

If we eliminate ε production, we get

$$A \rightarrow \beta \mid \beta A'$$
$$A' \rightarrow \alpha \mid \alpha A'$$

We can generalize this grammar. For any CFG given as

$$A \rightarrow A\,\alpha_1 \mid A\,\alpha_2 \mid A\,\alpha_3 \ldots\ldots \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \ldots\ldots$$

the equivalent grammar after eliminating left recursion is

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \beta_3 A' \mid \ldots\ldots \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \ldots\ldots$$
$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid \ldots\ldots \mid \alpha_1 \mid \alpha_2 \mid \alpha_3 \ldots\ldots$$

Procedure to Convert the given Grammar to GNF

1. Eliminate null productions and unit productions, and construct CNF.
2. Rename variables as $A_1\, A_2 \ldots\ldots$ starting with S as A_1
3. For each production of the form $A_i \rightarrow A_j\alpha$, apply the following:
   a) if $j > i$ --- leave the productions as they are
   b) if $j = i$ --- apply Lemma 2
   c) if $j < i$ --- apply Lemma 1
4. For each production of the form $A_i \rightarrow A_j\,\alpha$, where $j > i$, apply substitution lemma if A_j is in GNF to bring A_i to GNF.

Example 4.36 Convert the CFG to GNF.

$$S \rightarrow A\,A \mid a$$
$$A \rightarrow SS \mid b$$

Solution: Rename variables by

$$S = A_1\; A = A_2$$

$A_1 \rightarrow A_2 A_2 | a$.....(1)
$A_2 \rightarrow A_1 A_1 | b$ .....(2)

Apply Lemma 1 for (2):

$$A_2 \rightarrow A_2 A_2 A_1 | a A_1 | b$$

Apply Lemma 2 for this:

$A_2 \rightarrow a A_1 | b | a A_1 Z | b Z$
$Z \rightarrow A_2 A_1 | A_2 A_1 Z$ substitute for A_2 now in Z.
$Z \rightarrow a A_1 A_1 | b A_1 | a A_1 A_1 Z | b A_1 Z | a A_1 Z A_1 Z | a A_1 Z A_1$
$bZ A_1 | b Z A_1 Z$
$A_2 \rightarrow a A_1 | b | a A_1 Z | b Z$
$A_1 \rightarrow a | a A_1 A_2 | b A_2 | a A_1 Z A_2 | b Z A_2$

For any CFL L, the non-ε words of L can be generated by a CFG in GNF. Greibach normal form is useful for proving the equivalence of CFGs and NPDAs. When we discuss converting a CFG to an NPDA, or vice versa, we will use the Greibach normal form.

Example 4.37 Convert the following grammar G into GNF:

$S \rightarrow XA | BB$
$B \rightarrow b | SB$
$X \rightarrow b$
$A \rightarrow a$

Solution:

Step 1: Rewrite G in CNF.
It is already in CNF form.

Step 2: Re-label the variables:

S with A_1
X with A_2
A with A_3
B with A_4
$A_1 \rightarrow A_2 A_3 | A_4 A_4$ .....(1)
$A_4 \rightarrow b | A_1 A_4$ .....(2)
$A_2 \rightarrow b$ .....(3)
$A_3 \rightarrow a$ .....(4)

Step 3: Identify all productions that do not conform to GNF.
Apply Lemma 1 for (2):

$$A_4 \rightarrow A_2 A_3 A_4 | A_4 A_4 A_4 | b$$

Substitute $A_2 \rightarrow b$:

$$A_4 \rightarrow b A_3 A_4 | A_4 A_4 A_4 | b$$

Now apply Lemma 2 on $A_4 \rightarrow A_4 A_4 A_4$:

$$A_4 \rightarrow b A_3 A_4 | b | b A_3 A_4 Z | bZ$$
$$Z \rightarrow A_4 A_4 | A_4 A_4 Z$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Step 4: Now the resulting grammar is

$$A_1 \to A_2 A_3 \mid A_4 A_4$$
$$A_4 \to b A_3 A_4 \mid b \mid bA_3A_4 Z \mid bZ$$
$$Z \to A_4 A_4 \mid A_4 A_4 Z$$
$$A_2 \to b$$
$$A_3 \to a$$

Still the grammar is not in GNF.

Step 5: All productions A_2, A_3, A_4 are in GNF. But A_1 and Z are not in GNF.

For $A_1 \to A_2 A_3 \mid A_4 A_4$

Substitute for A_2 and A_4 to convert it into GNF:

$$A_1 \to b A_3 \mid b A_3 A_4 A_4 \mid b A_4 \mid b A_3 A_4 Z A_4 \mid bZ A_4$$

For $Z \to A_4 A_4 \mid A_4 A_4 Z$

Substitute for A_4 to convert it into GNF:

$$Z \to b A_3 A_4 A_4 \mid b A_4 \mid b A_3 A_4 Z A_4 \mid bZ A_4 \mid b A_3 A_4 A_4 Z$$
$$\mid b A_4 Z \mid b A_3 A_4 Z A_4 Z \mid bZ A_4 Z$$

Step 6: Final grammar is

$$A_1 \to bA_3 \mid b A_3 A_4 A_4 \mid b A_4 \mid b A_3 A_4 Z A_4 \mid bZA_4$$
$$A_4 \to bA_3 A_4 \mid b \mid bA_3 A_4 Z \mid bZ$$
$$Z \to bA_3 A_4 A_4 \mid bA_4 \mid bA_3 A_4 Z A_4 \mid bZ A_4 \mid bA_3 A_4 A_4 Z$$
$$\mid bA_4 Z \mid bA_3 A_4 Z A_4 Z \mid bZ A_4 Z$$
$$A_2 \to b$$
$$A_3 \to a$$

Example 4.36 Convert the CFG to GNF

$$S \to AB A$$
$$A \to aA \mid \varepsilon$$
$$B \to bB \mid \varepsilon$$

Solution: Eliminate ε-productions:

$$S \to ABA \mid AB \mid AA \mid A \mid B$$
$$A \to aA \mid a$$
$$B \to bB \mid b$$

Eliminate unit productions:

$$S \to ABA \mid AB \mid AA \mid aA \mid a \mid bB \mid b$$
$$A \to aA \mid a$$
$$B \to bB \mid b$$

Now substitute for A and B in S:

$$S \to aABA \mid aAB \mid bBA \mid aAA \mid aA \mid a \mid bB \mid b \mid aBA \mid aB \mid aA \mid bA$$
$$A \to aA \mid a$$
$$B \to bB \mid b$$

This is in GNF.

4.7 Pumping Lemma for CFL

The pumping lemma for regular sets states that every sufficiently long string in a regular set contains a short substring that can be pumped. In other words, if a long string is given and if we push or pump any number of substrings any number of times, we always get a regular set. According to pumping lemma for CFLs, there are always two short substrings that are close to each other, and these substrings can be repeated as many times as required.

4.7.1 Lemma

Let L be any context free language. Then there is a constant n, which depends only upon L, such that there exists a string $z \in L$ and $|z| \geq n$, where $z = uvwxy$ such that

1. $|vx| \geq 1$
2. $|vwx| \leq n$ and
3. For all $i \geq 0$ $\mathbf{u}\,\mathbf{v}^i\mathbf{w}\,\mathbf{x}^i\mathbf{y}$ is in L.

Proof: This pumping lemma states that, in a language L that is without unit productions and without ε-productions, there exists a z where $z \in L$. The string z can be derived by a context free grammar G. Let G be a grammar and let it be in the Chomsky normal form. For the string z, we can obtain a parse tree that derives the string z. The depth of the path from root to the leaf node for the yield z is less than or equal to i, then length of the word z is less than or equal to 2^{i-1}. We can prove this by induction.

Basis: If $i = 1$

Let G contain the rules $S \rightarrow a$ where length of the derived string is 1, i.e., $i = 1$. Now according to the rule, the word length should be $\leq 2^{i-1} = 2^0 = 1$. Observe that we have a word which is of length 1. Also observe that the grammar G is in Chomsky's normal form. Hence, the language is regular since $|z| = |uvwxy| = 1$.

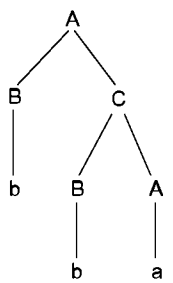


Fig. 4.8 *Tree with Three Distinct Non Terminals and with String Length 3*

Induction step: Let w be a string that is derived by grammar G. Let k be a variable such that $n = 2^k$ and $|z| \geq n$; then $|z| > 2^{k-1}$, when deriving the string w we may get some non-terminals of CFG. G to be repeated any number of times to give the string z as shown in the Figure 4.8. If we pump the substrings to w such that the path length of its newly formed string z′ (z+ pumped string z′) is i and the word length of z′ is 2^{i-1}, then the grammar G deriving z′ is called a regular grammar. The necessary condition is that grammar G is in Chomsky's normal form. Let us consider the following grammar:

$$G = (\{A, B, C\}, \{a\}, \{A \rightarrow BC \mid a, B \rightarrow BA \mid b, C \rightarrow BA\}, A)$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Thus $A \overset{*}{\Rightarrow} bba = w$,
that is, path length $i = 3$

$$|w| \leq 2^{i-1}, \text{ that is, } 3 \leq 2^2$$

If we pump s substrings into w, which satisfies the condition $i \leq |w| \leq 2^{i-1} \leq n$, then the grammar producing the string w is a regular grammar.

Example 4.36 Using pumping lemma, prove that the following language L is NOT a CFL.

$$L = \{0^n 1^n 2^n \mid n \geq 1\}$$

Solution: Consider the string $z = 0^n 1^n 2^n$; Let u be null string then maximum length/string for vwx is n

$$vwx = 0^n$$

it satisfies the condition $|vwx| \leq n$

$$v = 0^1$$
$$w = 0^{(n-2)}$$
$$x = 0^1$$
$$y = 1^n 2^n$$

Since $vx \,! = \varepsilon$, for $i = 0$,
we get $0^{(n-2)} 1^n 2^n$, which is not in L. Hence L is not a CFL.

Example 4.36 $L = \{w\,w \mid w \text{ is a bitstring}\}$. State whether L is CFL or not.

Solution: Consider the string $z = 0^n 1^n 0^n 1^n$.
This string is in L, Now let the substring be

$$vwx = 0^n \text{ it satisfies the condition } |vwx| \leq n$$

Assume u, v, w, x as in the example above.

$$y = 1^n 0^n 1^n$$

For $i = 0$, we get $0^{(n-n)}$

$$0^{(n-2)} 1^n 0^n 1^n \Rightarrow \text{not in L Hence the language is not CFL}$$

Example 4.37 Show that $L = \{a^i b^i c^i \mid i \geq 0\}$ is not a CFL.

Solution: Let z = aabbcc.
Split it as u = a, v = a, w = b, x = b, y = cc. Then,

$$|vx| > 1$$

uv^2wx^2y does not belong to L, that is, $aa^2bb^2cc \notin L$.
Hence, it is not a CFL.

4.8 Decision Algorithms for CFLs

As in regular sets, there are many ways to represent the CFLs. Irrespective of their representation, there are some fundamental questions that need to be answered about the context free language.

- Is the given context free language empty?
- Is the language finite or infinite?
- Does the given word belong to the language?
- Are the two CFGs equivalent?
- Whether the CFL is co-finite?
- Whether the complement of a given CFL is also CFL?
- Whether the given CFG is ambiguous?

Some of these questions can be answered as there exists a well-defined algorithm which can be used, but for some questions no algorithm can answer. There are algorithms that can answer the question whether the given language is empty, finite or infinite.

Emptiness: To check whether the given language is empty, consider a grammar G given by {V, T, P, S}. The language $L(G) \neq \phi$ if every variable A in the grammar G derives a string w with only terminals, that is, if $A \overset{*}{\Rightarrow} W$

4.8.1 Finiteness and Infiniteness

To check whether a given language G is finite or infinite, let us construct a CFG G′ = {V′, T, P′, S} in CNF that has no useless symbols and generates L(G) – ε. L(G′) is finite if and only if L(G) is finite. Finiteness can be checked by constructing a directed graph for variables in the grammar, and if the graph has no cycles, then the language is said to be finite; otherwise, infinite.

Example 4.36 Is the grammar S → AB, A → BB, B → a is finite?

Solution: The directed graph for the given grammar is shown in Figure 4.9:

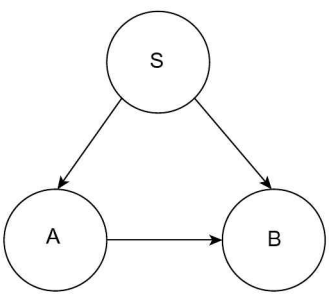


Fig. 4.9 *Graph of Example 4.42*

For the production S → AB, place an edge from S to A and B.

For the production A → BB, place an edge from A to B.

Since B derives only terminals, there is no outgoing edge from B. If we compute the rank (longest path) of each node, we get rank for node S, A and B to be as {2, 1, 0}. Since there is no cycle in the graph, the language is finite for the given grammar.

Example 4.36 Is the grammar S → AB, A → BB, B → AS | a is finite?

Solution: The directed graph for the given grammar is shown in Figure 4.10:

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

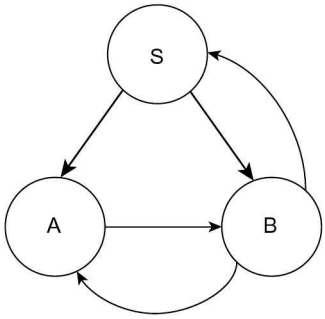


Fig. 4.10 *Graph of Example 4.43*

For the production S → AB, place an edge from S to A and B.
For the production A → BB, place an edge from A to B.
For the production B → AS, place an edge from B to A and S.

Finding rank for each node is not possible as there is a cycle in the graph. Hence, the language is infinite for the given grammar.

4.9 Membership

To check whether the given string x is in the language, we can use Cocke–Younger–Kasami (CYK) algorithm.

CYK Algorithm: CYK algorithm is based on dynamic programming technique and assumes that the grammar is in CNF. The time complexity of the algorithm is cubic time.

Statement of CYK: Let V_{ij} be the set of variables. Let A be variable such that $A \overset{*}{\Rightarrow} X_{i,j}$, where $1 \leq i \leq (n - j + 1)$ and no string is of length greater than $(n - i + 1)$ and not less than i. Then the string X is derived from A using the production $A \rightarrow BC$ such that there are substrings $X_{i,k}$ and $X_{i+k,j-k}$ are derived from B and C by $B \overset{*}{\Rightarrow} X_{i,j}$ and $C \overset{*}{\Rightarrow} X_{i+k,j-k}$, respectively.

Algorithm

Begin

1. For i = 1 to n do
   a. V_{il}: {A | A → a is a production and the ith symbol of x is a}
2. For j = 2 to n do
   a. For i = 1 to n − j + 1 do
      begin
      i. $V_{ij} = \Phi$
      ii. For k = 1 to j − 1 do
      - $V_{ij} = V_{ij} \cup \{A \mid A \rightarrow BC$ is a production, B is in V_{ik} and C is in $V_{i+k,j-k}\}$

      End

End

Example 4.36 Use the CYK algorithm to determine whether aaaaaa is in the following grammar:

$S \rightarrow AB / BC$
$A \rightarrow BA / a$
$B \rightarrow CC / b$
$C \rightarrow AB / a$

Solution: Table 4.1 shows the input string aaaaaa:

To create the entries for the first row use the input string aaaaaa. Then we see that

$$V_{11} = V_{21} = V_{31} = V_{41} = V_{51} = V_{61} = \{A, C\}.$$

This is because $A \rightarrow a$ and $C \rightarrow a$ are the productions using for–loop 1.

To compute V_{ij} for $j > 1$, we use for loop 2.ii. We must match $V_{i,k}$ against $V_{i+k, j-k}$ for $k = 1, 2 \ldots . j - 1$:

$$V_{ij} = V_{ij} \cup \{A / A \rightarrow BC \text{ is a production, B is in } V_{ik} \text{ and C is in } V_{i+k, j-k}\}$$

For $K = 1, 2 \ldots . j - 1$, compute.

$$\mathbf{V_{ij}} \text{ for } \mathbf{j = 2} \text{ to } \mathbf{n, i = 1} \text{ to } \mathbf{n - j + 1} \text{ and } \mathbf{k = 1, 2, 3, 4, 5, 6}$$

Example 4.36 $V_{12} = V_{12} \cup \{A \rightarrow BC \text{ is a production, B is in } V_{11} \text{ and C is in } V_{21}\}$

$$V_{11} V_{21} = \{A, C\}\{A, C\} = \{AA, AC, CA, CC\}$$

Therefore, $V_{12} = \{B\}$ as $B \rightarrow CC$ is a production such that C is in V_{11} and C is in V_{21}.

Table 4.1 *Computation of V_{ij}*

j	i	k	$V_{ik} V_{i+k, j-k}$	$V_{ij} = V_{ij} \cup \{A \mid A \rightarrow BC$ B in V_{ik} and C in $V_{i+k, j-k}$
2	2	1	$V_{21}V_{31} = \{A, C\}\{A, C\} = \{AA, AC, CA, CC\}$	$V_{22} = \{B\}$
2	3	1	$V_{31}V_{41} = \{A, C\}\{A, C\} = \{AA, AC, CA, CC\}$	$V_{32} = \{B\}$
2	4	1	$V_{41}V_{51} = \{A, C\}\{A, C\} = \{AA, AC, CA, CC\}$	$V_{42} = \{B\}$
2	5	1	$V_{51}V_{61} = \{A, C\}\{A, C\} = \{AA, AC, CA, CC\}$	$V_{52} = \{B\}$
3	1	1	$V_{11}V_{22} = \{A, C\}\{B\} = \{AB, CB\}$	$V_{13} = \{S, C\}$
3	1	2	$V_{12}V_{31} = \{B\}\{A, C\} = \{BA, BC\}$	$V_{13} = \{S, C\} \cup \{A\}$ $\therefore V_{13} = \{S, C, A\}$
3	2	1	$V_{21}V_{32} = \{A, C\}\{B\} = \{AB, CB\}$	$V_{23} = \{S, C\}$
3	2	2	$V_{22}V_{41} = \{B\}\{A, C\} = \{BA, BC\}$	$V_{23} = \{S, C\} \cup \{A\}$ $\therefore V_{23} = \{S, C, A\}$
3	3	1	$V_{31}V_{42} = \{A, C\}\{B\} = \{AB, CB\}$	$V_{33} = \{S, C\}$
3	3	2	$V_{32}V_{51} = \{B\}\{A, C\} = \{BA, BC\}$	$V_{33} = \{S, C\} \cup \{A\}$ $\therefore V_{33} = \{S, C, A\}$
3	4	1	$V_{41}V_{52} = \{A, C\}\{B\} = \{AB, CB\}$	$V_{43} = \{S, C\}$

(*continued*)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Table 4.1 *(Continued)*

j	i	k	$V_{ik} V_{i+k, j-k}$	$V_{ij} = V_{ij} \cup \{A \mid A \rightarrow BC \text{ B in } V_{ik} \text{ and C in } V_{i+k, j-k}$
3	4	2	$V_{42}V_{61}$ = {B}{A, C} = {BA, BC}	V_{43} = {S, C} ∪ {A} ∴ V_{43} = {S, C, A}
4	1	1	$V_{11}V_{23}$ = {A, C}{S, A, C}= {AS, AA, AC, CS, CA, CC}	V_{14} = {B}
4	1	2	$V_{12}V_{32}$ = {B}{B} = {BB}	V_{14} = {B} ∪ {ϕ}
4	1	3	$V_{13}V_{41}$ = {S, A, C}{A, C} = {SA, SC, AA, AC, CA, CC}	V_{14} = {B} ∪ {ϕ} ∴V_{14} = {B}
4	2	1	$V_{21}V_{33}$ = {A, C}{S, A, C} = {AS, AA, AC, CS, CA, CC}	V_{24} = {B}
4	2	2	$V_{22}V_{42}$ = {B}{B} = {BB}	V_{24} = {B} ∪ {ϕ}
4	2	3	$V_{23}V_{51}$ = {S, A, C}{A, C} = {SA, SC, AA, AC, CA, CC}	V_{24} = {B} ∪ {ϕ} ∴V_{24} = {B}
4	3	1	$V_{31}V_{43}$ = {A, C}{S, A, C} = {AS, AA, AC, CS, CA, CC}	V_{34} = {B}
4	3	2	$V_{32}V_{52}$ = {B}{B} = {BB}	V_{34} = {B} ∪ {ϕ}
4	3	3	$V_{33}V_{61}$ = {S, A, C}{A, C} = {SA, SC, AA, AC, CA, CC}	V_{34} = {B} ∪ {ϕ} ∴V_{34} = {B}
5	1	1	$V_{11}V_{24}$ = {A, C}{B} = {AB, CB}	V_{15} = {S, C}
5	1	2	$V_{12}V_{33}$ = {B}{S, A, C} = {BS, BA, BC}	V_{15} = {S, C} ∪ {A}
5	1	3	$V_{13}V_{42}$ = {S, A, C}{B} = {SB, AB, CB}	V_{15} = {S, A, C}
5	1	4	$V_{14}V_{51}$ = {B}{A, C} = {BA, BC}	V_{15} = {S, A, C} ∴V_{15} = {S, A, C}
5	2	1	$V_{21}V_{34}$ = {A, C}{B} = {AB, CB}	V_{25} = {S, C}
5	2	2	$V_{22}V_{43}$ = {B}{S, A, C} = {BS, BA, BC}	V_{25} = {S, C} ∪ {A}
5	2	3	$V_{23}V_{52}$ = {S, A, C}{B} = {SB, AB, CB}	V_{25} = {S, A, C}
5	2	4	$V_{24}V_{61}$ = {B}{A, C} = {BA, BC}	V_{25} = {S, A, C} ∴V_{25} = {S, A, C}
6	1	1	$V_{11}V_{25}$ = {A, C}{S, A, C}= {AS, AA, AC, CS, CA, CC}	V_{16} = {B}
6	1	2	$V_{12}V_{34}$ = {B}{B} = {BB}	V_{16} = {B}
6	1	3	$V_{13}V_{43}$ = {S, A, C}{S, A, C} = {SS, SA, SC, AS, AA, AC, CS, CA, CC}	V_{16} = {B}
6	1	4	$V_{14}V_{52}$ = {B}{B} = {BB}	V_{16} = {B}
6	1	5	$V_{15}V_{61}$ = {S, A, C}{A, C} = {SA, SC, AA, AC, CA, CC}	∴ V_{16} = {B}

The table is constructed using the CYK algorithm.

		a	a	a	a	a	a
				i→			
		1	2	3	4	5	6
j ↓	1	A, C	A, C	A, C	A, C	A, C	A, C
	2	B	B	B	B	B	
	3	S, A, C	S, A, C	S, A, C	S, A, C		
	4	B	B	B			
	5	S, A, C	S, A, C				
	6	B					

The given string **aaaaaa** is not accepted as the elements in V_{16} do not contain the start symbol.

From the table, we can say that **aaa** or **aaaaa** strings are accepted.

4.10 Closure Properties of CFLs

CFLs are closed under substitution, union, concatenation, closure and positive closure, reversal homomorphism, and inverse homomorphism.

CFLs are not closed under intersection, difference and complement.

a) CFLs are closed under union

Theorem 1: If L_1 and L_2 are CFLs, then their union $L_1 + L_2$ is a CFL.

Let the grammar CFG_1 define the language L_1. Assume that the non-terminals in CFG_1 are S_1, A_1, B_1, C_1..... Let L_2 be a language defined by CFG_2 and let its non-terminalsbe S_2, A_2, B_2, C_2,.....

Now CFG_1 and CFG_2 have non-intersecting sets of non-terminals.

We create a CFG for $L_1 + L_2$ as follows:

Include all of the non-terminals S_1, A_1, B_1, C_1,... and S_2, A_2, B_2, C_2,....

Include all of the productions from CFG_1 and CFG_2.

Create a new non-terminal S and a new production in CFG s

$$S \rightarrow S_1 | S_2$$

This new production takes care of generating all the strings $x \in L(G_1)$ or $L(G_2)$ as

$$S \Rightarrow S_1 \overset{*}{\Rightarrow} x \quad \text{if } x \in L(a_1)$$

or

$$S \Rightarrow S_2 \overset{*}{\Rightarrow} x \quad \text{if } x \in L(a_2)$$

b) CFLs are closed under concatenation

Theorem 2: If L_1 and L_2 are CFLs, then $L_1 L_2$ is a CFL.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Let the grammar CFG_1 define the language L_1. Assume that the non-terminals in CFG_1 are S_1, A_1, B_1, C_1 ..... Let L_2 be a language defined by CFG_2 and let its non-terminalsbe S_2, A_2, B_2, C_2, ...

Now CFG_1 and CFG_2 have non-intersecting sets of non-terminals. We create a CFG for L_1L_2 as follows:

Include all the non-terminals S_1, A_1, B_1, C_1,..... and S_2, A_2, B_2, C_2,....

Include all the productions from CFG_1 and CFG_2.

Create a new non-terminal S and a production

$$S \rightarrow S_1S_2$$

These rules can derive $S \overset{*}{\Rightarrow} xy$ where $x \in L(G_1)$ with $S_1 \Rightarrow x$ and $y \in L(G_2)$ with $S_2 \Rightarrow y$

c) CFLs are closed under closure

Theorem 3: If L is a CFL, then L* is a CFL.

Since L is a CFL, by definition, there is some CFG that generates L. Suppose CFG for L has non-terminals S, A, B, C, ... Change the non-terminal S to S_1. We create a new CFG for L as follows:

Include all the non-terminals S_1, A, B, C, . . . from the CFG for L.

Include all of the productions from the CFG for L.

Add the new non-terminal S and the new production

$$S \rightarrow S_1S \mid \varepsilon$$

We can repeat last production:

$$S \rightarrow S_1S \rightarrow S_1S_1S \rightarrow S_1S_1S_1S \rightarrow S_1S_1S_1S_1S \rightarrow S_1S_1S_1S_1\varepsilon \rightarrow S_1S_1S_1S_1$$

Note that any word in L* can be generated by the new CFG. To show that any word generated by the new CFG is in L*, note that each S_1 above generates a word in L. Also, there is no interaction between different S_1's.

d) CFLs are not closed under intersection

Theorem 4: Let L_1 and L_2 be two CFLs. then $L_1 \cap L_2$ may or may not be a CFL. This means they are not closed under intersection.

We now give an example showing that the intersection of two CFLs may not be a CFL. To show this, we assume that the language $L_1 = \{a^n b^n a^n : n \geq 1\}$ is not a context free language. L_1 is the set of words with some number of a's, followed by an equal number of b's, and they are ending with the same number of a's.

Let L_2 be generated by the following CFG:

$$S \rightarrow XY$$
$$X \rightarrow aXb \mid \varepsilon$$
$$Y \rightarrow aY \mid \varepsilon$$

Thus, $L_2 = \{a^n b^n a^m : n, m >= 0\}$. Words in this set have a clump of a's, are followed by a clump of b's, and are ending with another clump of a's. The number of a's at the beginning is the same as the number of b's in the middle. The number of a's at the end

of the word is arbitrary, and does not have to be equal with the number of a's and b's that come before it.

Let L_3 be generated by the following CFG:

$$S \rightarrow WZ$$
$$W \rightarrow aW \mid \varepsilon$$
$$Z \rightarrow bZa \mid \varepsilon$$

Thus, $L_3 = \{a^i b^k a^k : i, k \geq 0\}$. Words in this set have a clump of a's, are followed by a clump of b's, and are ending with another clump of a's. The number of b's in the middle is the same as the number of a's at the end. The number of a's at the beginning of the word is arbitrary, and does not have to be equal with the number of b's and a's that come after it.

Note that $L_2 \cap L_3 = L_1$, where $L_1 = \{a^n b^n a^n : n = 0, 1, 2, \ldots\}$, which is not a context free language, as proved by pumping lemma of CFL's. Hence CFL's are not closed under intersection.

e) CFLs are not closed under complement

Theorem 5: Let L be a CFL then $\overline{L}$ may or may not be a CFL.

We first assume that the complement of a CFL may be a CFL.

If L is regular, then $\overline{L}$ is also regular. Also both L and $\overline{L}$ are CFLs.

We now show that the complement of a CFL may not be a CFL (by contradiction):

Suppose that it is always true that if L is a CFL, then $\overline{L}$ is a CFL. Let L_1 and L_2 are CFLs, then by our assumption, L_1 and $\overline{L}_2$ are CFLs. Closure under union implies that $\overline{L}_1 + \overline{L}_2$ is a CFL. Then by our assumption, we must have that complement of ($\overline{L}_1 + \overline{L}_2$) is a CFL. But we know that complement of $(\overline{L}_1 + \overline{L}_2) = L_1 \cap L_2$ by DeMorgan's Law. However, we previously showed that the intersection of two CFLs is not always a CFL, which contradicts the previous two steps. So our assumption that CFLs are always closed under complementation must not be true.

Thus, in general, it is not true that complement of a CFL is CFL.

4.11 Applications of CFG

- Grammars are useful in specifying the syntax of programming languages. They are mainly used in the design of programming languages.
- They are also used in natural language processing.
- Tamil poetry called Venpa is described by a context free grammar.
- CFGs are used in speech recognition and also in processing spoken words.
- The expressive power of CFG is too limited to adequately capture all natural language phenomena. Therefore, extensions of CFGs are of interest in computational linguistics.

Example 4.36 As an example, CFG for pascal statements are given below.

Stmt → begin optional_ stmts end
optional_ stmts → list_of_stmt | ε

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

list_of_stmt	→ list_of_stmt; Stmt | Stmt
Stmt	→ if Expr then Stmt
	| if Expr then Stmt else Stmt
	| while Expr do Stmt
	| id = Expr
Expr	→ Expr + Term | Term
Term	→ Term * Fctr | Fctr
Fctr	→ num | id

Solved Problems

Problem 1: Give the CFG that generates all positive even integers up to 998.

***Solution*:** We need to generate positive integers with one or two or three digits.
Of one digit are 0, 2, 4, 6, 8. Two and three digit positive integers can have any number in 10's place and so also in 100's place. Hence, we can define grammar as follows:

S → A	single digit numbers
| BA	double digit numbers
| BCA	Three digit numbers
A →0 | 2 | 4 | 6 | 8	
B → 1 | 2 | 3 9	
C →0 | B	

Problem 2: Give a CFG for the RE (a + b)*cc(a + b)*.

Solution: The grammar for all the strings on a, b, i.e., is, (a + b)*, is A → aA | bA | ε. the middle of any string of a's, and b's on either side, substring cc is occurring.

So the grammar for given regular expression is

$$S \rightarrow AccA$$

$$A \rightarrow aA \mid bA \mid \varepsilon$$

Problem 3: Give CFG for the language L = $\{0^m1^n \mid m, n \geq 0\}$.

Solution: The regular expression is 0*1*.
The grammar for strings a* is A → aA | ε.

Hence, the grammar for the given regular expression is

$$S \rightarrow AB$$
$$A \rightarrow 0A \mid \varepsilon$$
$$B \rightarrow 1B \mid \varepsilon$$

Problem 4: Give CFG for the language L = $\{0^{2m}1^n/m, n \geq 0\}$.

Solution: The given RE has zero or more strings of 00's and 1's.
The language is (00)*1*. So final grammar is

$$S \rightarrow AB$$

$$A \rightarrow 00A \mid \varepsilon$$
$$B \rightarrow 1B \mid \varepsilon$$

Problem 5: Find language defined by the following CFG:

$$S \rightarrow aB \mid bA$$
$$A \rightarrow a \mid aS \mid bAA$$
$$B \rightarrow b \mid bS \mid aBB$$

Solution: To find the language defined by the given grammar, list the words defined by grammar. This gives

$$S \Rightarrow aB \Rightarrow ab$$
$$S \Rightarrow bA \Rightarrow ba$$
$$S \Rightarrow aB \Rightarrow abS \Rightarrow abaB \Rightarrow abab$$
$$S \Rightarrow bA \Rightarrow bbAA \Rightarrow bbaSA \Rightarrow bbabAa \Rightarrow bbabaa$$

Hence, the language defined by the given grammar is equal number of a's and b's.

Problem 6: Find LMD and RMD for the string 00101 in grammar given below:

$$S \rightarrow B \mid A$$
$$A \rightarrow 0\,A \mid \varepsilon$$
$$B \rightarrow 1B \mid 0B \mid \varepsilon$$

Solution: Here LMD, RMD are same as there is only one nonterminal on R.H.S of production.

$$S \Rightarrow B \Rightarrow 0B \Rightarrow 00B \Rightarrow 001B \Rightarrow 0010B \Rightarrow 00101B$$
$$\Rightarrow 00101\varepsilon$$
$$\Rightarrow 00101$$

Problem 7: Find LMD, RMD and derivation tree for the string 00110101 in the grammar given below:

$$S \rightarrow 0B \mid 1A$$
$$A \rightarrow 0 \mid 0S \mid 1AA$$
$$B \rightarrow 1 \mid 1S \mid 0BB$$

Solution: LMD

$$S \Rightarrow 0\underline{B} \Rightarrow 00\underline{B}B \Rightarrow 001\underline{S}B \Rightarrow 0011\underline{A}B$$
$$\Rightarrow 00110\underline{S}B$$
$$\Rightarrow 001101\underline{A}B$$
$$\Rightarrow 0011010B$$
$$\Rightarrow 00110101$$

RMD

$$S \Rightarrow 0B \Rightarrow 00\underline{B}B$$
$$\Rightarrow 00\underline{B}1$$
$$\Rightarrow 001\underline{S}1$$
$$\Rightarrow 0011\underline{A}1$$
$$\Rightarrow 00110\underline{S}1$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$$\Rightarrow 001101\underline{A}1$$
$$\Rightarrow 00110101$$

derivation tree is shown in the Figure 4.11

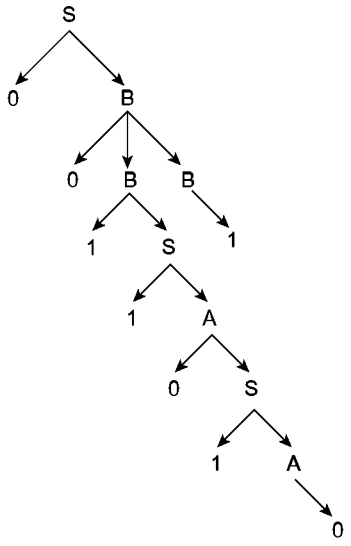


Fig. 4.11 *Derivation Tree for "00110101"*

Problem 8: Check whether the following grammar is ambiguous:

$$S \rightarrow 0S1 \mid SS \mid \varepsilon$$

Solution: Let us consider the string 01. This string can be generated in two ways:

i. $S \Rightarrow 0S1 \Rightarrow 01$
ii. $S \Rightarrow SS \Rightarrow 0S1S \Rightarrow 01S \Rightarrow 01$

Hence, the given grammar is an ambiguous grammar.

Problem 9: Check whether the following grammar is ambiguous for w = ab.

$$S \rightarrow aB \mid ab$$
$$A \rightarrow aAB \mid a$$
$$B \rightarrow AB\ b \mid b$$

Solution: The string ab can be derived using LMD as follows:

1. $S \Rightarrow ab$
2. $S \Rightarrow a\underline{B} \Rightarrow ab$

Since there are two possible ways to derive the string, the grammar is ambiguous for w.

Problem 10: Check whether the following grammar is ambiguous for w = aab.

$$S \rightarrow AB \mid aaB$$
$$A \rightarrow a \mid Aa$$
$$B \rightarrow b$$

Solution: The string aab can be derived using LMD as depicted in the following:

1. $S \Rightarrow AB \Rightarrow AaB \Rightarrow aaB \Rightarrow aab$
2. $S \Rightarrow aaB \Rightarrow aab$

Since there are two possible ways to derive the string, the grammar is ambiguous for w.

Problem 11: Check whether the following grammar is ambiguous for w = abababa:

$$S \rightarrow SbS \mid a$$

Solution: The string "abababa" can be derived using LMD as depicted in the following:

1. $S \Rightarrow SbS \Rightarrow SbSbS \Rightarrow SbSbSbS$
   $\Rightarrow abSbSbS \Rightarrow ababSbS \Rightarrow abababS \Rightarrow abababa$
2. $S \Rightarrow SbS \Rightarrow abS \Rightarrow abSbS \Rightarrow ababS \Rightarrow ababSbS$
   $\Rightarrow abababS \Rightarrow abababa$

Since there are two possible ways to derive the string, the grammar is ambiguous for w.

Problem 12: Eliminate useless symbols in the following G:

$S \rightarrow a\,Aa$
$A \rightarrow Sb \mid bCc \mid DaA$
$C \rightarrow abb \mid DD$
$E \rightarrow a\,C$
$D \rightarrow a\,DA$

Solution: Here D is useless as it is not deriving any string. E is not reachable.
The reduced grammar is the one without useless symbols.

$S \rightarrow aAa$
$A \rightarrow Sb \mid bCc$
$C \rightarrow abb$

Problem 13: Eliminate useless symbols in the following grammar.

$S \rightarrow a\,A \mid bB$
$A \rightarrow aA \mid a$
$B \rightarrow bB$
$D \rightarrow ab \mid Ea$
$E \rightarrow aC \mid d$

Solution: Here B is useless as it is in a loop, not deriving any string. There is no rule for C. So C is also useless. E and D are not reachable from S. Hence

The reduced grammar is

$S \rightarrow a\,A$
$A \rightarrow aA \mid a$

Problem 14: Eliminate null production in the given grammar.

$S \rightarrow aS \mid AB$
$A \rightarrow \varepsilon$
$B \rightarrow \varepsilon$
$D \rightarrow b$

Solution: Nullable variables V_n are given by {A, B, S}.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Hence, the equivalent grammar without null productions is

$S \rightarrow aS \mid a \mid AB \mid A \mid B$
$D \rightarrow b$

Problem 15: Eliminate null production in the grammar

$S \rightarrow AaB \mid aaB$
$A \rightarrow \varepsilon$
$B \rightarrow bbA \mid \varepsilon$

Solution: Nullable variables V_n = are given by {A, B}.
Hence, the equivalent grammar without null productions is

$S \rightarrow AaB \mid aaB \mid aB \mid A\,a \mid aa \mid a$
$B \rightarrow bbA \mid bb$

Problem 16: Convert to CNF

$S \rightarrow ABa, A \rightarrow aab, B \rightarrow Ac$

Solution: There are no null/unit productions.
Replace every terminal by variable:

$C_a \rightarrow a, C_b \rightarrow b, C_c \rightarrow c$
$S \rightarrow ABC_a$
$A \rightarrow C_a C_a C_b$
$B \rightarrow A\,C_c$

Now restrict to only two variables on the right-hand side:

$C_a \rightarrow a, C_b \rightarrow b, C_c \rightarrow c$
$S \rightarrow AD, D \rightarrow B\,C_a$
$A \rightarrow C_a E, E \rightarrow C_a C_b$
$B \rightarrow A\,C_c.$

This is the CNF.

Problem 17: Convert following grammar to CNF:

$S \rightarrow aAbB, A \rightarrow aA \mid a, B \rightarrow bB \mid b$

Solution: Given grammar is

$S \rightarrow aAbB$
$A \rightarrow aA \mid a$
$B \rightarrow bB \mid b$

Convert the above to CNF form, that is, $A \rightarrow BC \mid a$

CNF form is
$C \rightarrow a\ D \rightarrow b$
$S \rightarrow CE,$
$E \rightarrow AF$
$F \rightarrow DB$
$A \rightarrow CA \mid a$
$B \rightarrow DB \mid b$

Problem 18: Let G be given by the following grammar:

$S \rightarrow ASA \mid aB$

$A \rightarrow B \mid S$
$B \rightarrow b \mid \varepsilon.$

Convert this to CNF.

Solution: We will convert this to the Chomsky normal form by following the steps in the algorithm.

i. Add new start variable. This is accomplished by adding the new rule

$$S_0 \rightarrow S.$$

ii. Now we eliminate the rule $B \rightarrow \varepsilon$. We must make a copy of each rule where B occurs on the right-hand side (underlined below). Therefore, the grammar is

$S_0 \rightarrow S$
$S \rightarrow ASA \mid aB$
$A \rightarrow B \mid S$
$B \rightarrow b \mid \varepsilon$
on eliminating ε the grammar is transformed to
$S_0 \rightarrow S$
$S \rightarrow ASA \mid aB \mid a$
$A \rightarrow B \mid S \mid \varepsilon$
$B \rightarrow b$

Notice that, for example, we do not drop $A \rightarrow B$; instead we keep it and add $A \rightarrow \varepsilon$. So, we have dropped one ε-rule and added another.

iii. Eliminate $A \rightarrow \varepsilon$. This yields the following grammar:

$S_0 \rightarrow S$
$S \rightarrow ASA \mid AS \mid SA \mid S \mid aB \mid a$
$A \rightarrow B \mid S$
$B \rightarrow b$

We have now finished eliminating ε-rules and can move to eliminating unit rules.

iv. Eliminate $S \rightarrow S$. This illustrates a special case: When asked to eliminate a rule $V \rightarrow V$, the rule may simply be dropped without any more thought. Thus we have the grammar

$S0 \rightarrow S$
$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$A \rightarrow B \mid S$
$B \rightarrow b$

v. Eliminate $S_0 \rightarrow S$. In this case, the algorithm means that wherever there is a rule $S \rightarrow w$, we will add $S_0 \rightarrow w$. Thus we have

$S_0 \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$A \rightarrow B \mid S$
$B \rightarrow b$

vi. Eliminate $A \rightarrow B$. In this case, wherever there is a rule $B \rightarrow w$, we will add $A \rightarrow w$. Thus we have

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$S0 \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$A \rightarrow S \mid b$
$B \rightarrow b$

vii. Eliminate $A \rightarrow S$. In this case, wherever there is a rule $S \rightarrow w$, we will add $A \rightarrow w$. Thus we have

$S0 \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$S \rightarrow ASA \mid AS \mid SA \mid aB \mid a$
$A \rightarrow ASA \mid AS \mid SA \mid aB \mid a \mid b$
$B \rightarrow b$

That finishes the elimination of unit rules. Now we map the grammar to binary form.

viii. The rule $S \rightarrow ASA$ needs to be split, which is accomplished by adding a new rule

$A_1 \rightarrow SA,$

and replacing all occurrences of ASA by AA_1:

$S0 \rightarrow AA_1 \mid AS \mid SA \mid aB \mid a$
$S \rightarrow AA_1 \mid AS \mid SA \mid aB \mid a$
$A \rightarrow AA_1 \mid AS \mid SA \mid aB \mid a \mid b$
$A_1 \rightarrow SA$
$B \rightarrow b$

ix. The grammar is still not in final form: Right-hand sides such as aB are not in the correct format. This is taken care of by adding a new rule $U \rightarrow a$ and propagating its definition to all binary rules with the terminal a on the right-hand side. This gives us the final grammar in the Chomsky normal form:

$S_0 \rightarrow AA_1 \mid AS \mid SA \mid UB \mid a$
$S \rightarrow AA_1 \mid AS \mid SA \mid UB \mid a$
$A \rightarrow AA_1 \mid AS \mid SA \mid UB \mid a \mid b$
$A_1 \rightarrow SA$
$B \rightarrow b$
$U \rightarrow a$

As we can see, conversion to the Chomsky normal form (CNF) can lead to bulky and awkward grammars. However, a grammar G in CNF has various advantages. One of them is that every step in a derivation using G makes demonstrable progress towards the final string because either the sentential form gets strictly longer (by 1), or a new terminal symbol appears.

Problem 19: Consider the language of balanced parentheses, generated by the following grammar

$$S \rightarrow \varepsilon \mid (S) \mid SS.$$

Convert this to CNF.

Solution: This grammar is not in the Chomsky normal form. But the following steps will achieve that:

Let new start symbol be S_0. $S_0 \rightarrow S$

$$S \rightarrow \varepsilon \mid (S) \mid SS$$

- Eliminate ε-productions $S \rightarrow \varepsilon$:

$$S_0 \rightarrow S \mid \varepsilon$$
$$S \rightarrow (S) \mid () \mid S \mid SS$$

- Drop $S \rightarrow S$:

$$S_0 \rightarrow S \mid \varepsilon$$
$$S \rightarrow (S) \mid () \mid SS$$

- Eliminate $S_0 \rightarrow S$, and $S_0 \rightarrow \varepsilon$. Now eliminate unit productions and ε productions, then we get the following grammar

$$S_0 \rightarrow \varepsilon \mid (S) \mid ( ) \mid SS$$
$$S \rightarrow (S) \mid ( ) \mid SS$$

- Put in binary rule format. We add two rules for deriving the opening and closing parentheses:

$$L \rightarrow ($$
$$R \rightarrow)$$

Then the final grammar in CNF is

$$S_0 \rightarrow LA \mid LR \mid SS$$
$$S \rightarrow LA \mid LR \mid SS$$
$$A \rightarrow SR$$
$$L \rightarrow ($$
$$R \rightarrow)$$

Summary

1. A Context free grammar mainly defines the syntax of the programming language and push down automata is used to recognize the language.
2. A context free grammar can be simplified by eliminating useless symbols null productions and unit productions.
3. An equivalent grammar can be constructed for any language without null strings by eliminating null productions.
4. Context free grammars can be represented in standard form using the Chomsky normal form or the Greibach normal form.

Short Answers

1. What are the applications of context free languages?

   Answer: Context free languages are used in:

   - Defining programming languages
   - Formalizing the notion of parsing

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

- Translation of programming languages
- String-processing applications

2. What are the uses of context free grammars?

Answer:

- Construction of compilers
- Simplifying the definition of programming languages
- Describing the arithmetic expressions with arbitrary nesting of balanced parenthesis {(,)}
- Describing block structure in programming languages
- Modeling neural nets

3. Define a context free grammar

Answer: A context free grammar G is defined by G = (V, T, P, S), where V and T are finite set of variables and terminals, respectively. V and T are disjoint. P is a finite set of productions; each is of the form $A \rightarrow \alpha$, where A is a variable and α is a string of symbols from $(V \cup T)^*$.

4. What is the language generated by CFG or G?

Answer: The language generated by G (denoted by L(G)) is $\{w \mid w \text{ is in } T^* \text{ and } S \overset{*}{\Rightarrow} w\}$. That is, a string is in L(G) if

1. the string consists solely of terminals, and
2. the string can be derived from S.

5. What is a (a) CFL? What is a (b) sentential form?

Answer: L is a context free language (CFL) if it is L(G) for some CFG G.
A string of terminals and variables α is called a sentential form:
if $S \overset{*}{\Rightarrow} \alpha$, where S is the start symbol of the grammar and $\alpha \in (V \cup T)^*$.

6. Describe is the language generated by the grammar G = (V, T, P, S), where

$$P = \{S \rightarrow aSb, S \rightarrow ab\}$$

Answer: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \Rightarrow a^n b^n$
Thus the language $L(G) = \{a^n b^n \mid n \geq 1\}$. The language has strings with equal number of a's and b's.

7. Discuss the following:

(a) derivation, (b) derivation/parse tree and (c) subtree.
Answer: (a) Let G = (V, T, P, S) be the context free grammar. If $A \rightarrow \beta$ is a production of P. and α and β are any two strings in $(V \cup T)^*$, then $\alpha A \Rightarrow \alpha\beta$.
(b) A tree is a parse\derivation tree for G if:

(i) Every vertex has a label which is a symbol of $V \cup T \cup \{\varepsilon\}$.
(ii) The label of the root is S.
(iii) If a vertex is interior and has a label A, then A must be in V.
(iv) If n has a label A and vertices $n_1, n_2, \ldots\ldots n_k$ are the sons of the vertex n in order from left with labels $X_1, X_2, \ldots\ldots\ldots X_k$, respectively, then $A \rightarrow X_1 X_2 \ldots\ldots X_k$ must be in P.
(v) If vertex n has label ε, then n is a leaf and it is the only son of its father.

(c) A subtree of a derivation tree is a particular vertex of the tree together with all its descendants, the edges connecting them and their labels. The label of the root may not be the start symbol of the grammar.

8. If $S \rightarrow aSb \mid aAb, A \rightarrow bAa, A \rightarrow ba$. Find out the CFL.

Answer: $S \rightarrow aAb \Rightarrow abab$

$S \rightarrow aSb \Rightarrow a\ aAb\ b \Rightarrow a\ a\ ba\ b\ b$ (sub $S \rightarrow aAb$)

$S \rightarrow aSb \Rightarrow a\ aSb\ b \Rightarrow a\ a\ aAb\ b\ b \Rightarrow a\ a\ a\ ba\ b\ bb$

Thus $L = \{a^n b^m a^m b^n$, where $n, m \geq 1\}$

9. What is an ambiguous grammar?

Answer: A grammar is said to be ambiguous if it has more than one derivation tree for a sentence or, in other words, if it has more than one leftmost derivation or more than one rightmost derivation.

10. Check whether the grammar $P = \{S \rightarrow aS \mid aSbS \mid \varepsilon\}$ is ambiguous by constructing two: (a) leftmost derivations and (b) rightmost derivation.

Answer: Consider a string aab:

(a)

(i) $S \Rightarrow aS$
$\Rightarrow aaSbS$
$\Rightarrow aabS$
$\Rightarrow aab$

(ii) $S \Rightarrow aSbS$
$\Rightarrow aaSbS$
$\Rightarrow aabS$
$\Rightarrow aab$

(b)

(i) $S \Rightarrow aS$
$\Rightarrow aaSbS$
$\Rightarrow aaSb$
$\Rightarrow aab$

(ii) $S \Rightarrow aSbS$
$\Rightarrow aSb$
$\Rightarrow aaSbS$
$\Rightarrow aaSb$
$\Rightarrow aab$

11. Find the grammar for the language $L = \{a^{2n}bc$, where $n > 1\}$.

Answer: Let $G = (\{S, A, B\}, \{a, b, c\}, P, \{S\})$ where $P: S \rightarrow Abc, A \rightarrow aaA \mid \varepsilon$.

12. Find the language generated by $S \rightarrow 0S1 \mid 0A \mid 0 \mid 1B \mid 1$.

Answer: $A \rightarrow 0A \mid 0, B \rightarrow 1B \mid 1$. The minimum string is $S \rightarrow 0 \mid 1$

$S \rightarrow 0S1 \Rightarrow 001$ $S \rightarrow 0S1 \Rightarrow 011$ $S \rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 0000A111$
$\Rightarrow 00000111$

Thus $L = \{0^n 1^m \mid m \neq n$, and $n, m \geq 1\}$.

13. Construct the grammar for the language $L = \{a^n\ ba^n \mid n \geq 1\}$.

Answer: The grammar has the production P as: $S \rightarrow aAa\ A \rightarrow aAa \mid b$

The grammar is thus: $G = (\{S, A\}, \{a, b\}, P, S)$.

14. Construct a grammar for the language L that has all the strings that are palindromes over $\Sigma = \{a, b\}$, $G = (\{S\}, \{a, b\}, P, S)$ and $P: \{S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon\}$

Answer: The language has palindrome strings with a's and b's.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

15. Give an example of a non-deterministic CFL.
Answer: The language $L = \{ww^R/w \in \{a, b\}^+\}$ is a non-deterministic CFL.

16. Find a CFG with no useless symbols equivalent to $S \rightarrow AB \mid CA$, $B \rightarrow BC \mid AB$, $A \rightarrow a$, $C \rightarrow aB \mid b$.
Answer: For the given set of productions, a symbol X is useful if $S \Rightarrow \alpha X\beta \Rightarrow w$.
The variable B cannot generate terminals as $B \rightarrow BC$ and $B \rightarrow AB$.
Thus, B is useless symbol and, hence, remove B from all productions.
Hence, useful productions are $S \rightarrow CA$, $A \rightarrow a$, $C \rightarrow b$.

17. Find the grammar with no null productions for the grammar given below:
$S \rightarrow a/Ab/aBa$
$A \rightarrow b/\varepsilon$, $B \rightarrow b/A$
Answer: In the given set of productions, $A \rightarrow \varepsilon$ is the only empty production.
Remove the empty production and in the $S \rightarrow Ab$, put $A \rightarrow \varepsilon$ and, hence, $S \rightarrow b$.
If $B \rightarrow A$ and $A \rightarrow \varepsilon$ then $B \rightarrow \varepsilon$. Hence $S \rightarrow aBa$ becomes $S \rightarrow aa$.
Thus $S \rightarrow a \mid Ab \mid b \mid aBa \mid aa$, $A \rightarrow b$, $B \rightarrow b$
Finally, the productions are: $S \rightarrow a \mid Ab \mid b \mid aBa \mid aa$, $A \rightarrow b$, $B \rightarrow b$.

18. What are the three ways to simplify a context free grammar?
Answer:
- By removing the useless symbols from the set of productions
- By eliminating the empty productions
- By eliminating the unit productions

19. What are the properties of the CFL generated by a CFG?
Answer:
- Each variable and each terminal of G appear in the derivation of some word in L.
- There are no productions of the form $A \rightarrow B$ where A and B are variables.

20. What are the closure properties of CFL?
Answer: CFL are closed under union, concatenation and Kleene closure. CFL are closed under substitution, homomorphism. CFL are not closed under intersection, complementation. Closure properties of CFLs are used to prove that certain languages are not context free.

21. State the pumping lemma for CFLs.
Answer: Let L be any CFL. Then there is a constant n, depending only on L, such that if z is in L and $|z| \geq n$, then $z = uvwxy$ such that:
(i) $|vx| \geq 1$
(ii) $|vwx| \leq n$ and
(iii) for all $i \geq 0$ uv^iwx^iy is in L.

22. What is the main application of the pumping lemma in CFLs?

Answer: The pumping lemma can be used to prove a variety of languages are not context free. Some examples are

$L_1 = \{a^i b^i c^i \mid i \geq 1\}$ is not a CFL.

$L_2 = \{a^i b^j c^i d^j \mid i \geq 1 \text{ and } j \geq 1\}$ is not a CFL.

23. What are the properties of CFL?

Answer: A grammar G = (V, T, P, S) is said to be context free if all productions in P have the form $A \rightarrow x$ where $A \in V$ and $x \in (V \cup T)^*$. The language defined by context free grammar is CFL. The CFLs are recognized by PDA.

Fill in the Blanks

1. Syntax of any high-level language is defined with _____
2. The language defined by $S \rightarrow SS$ is _____
3. A non-terminals is useless if it is _____
4. A variable that derives ε is called _____ variable.
5. Left linear grammars are _____ of CFG
6. CFGs are _____of CSG
7. If there is a unique LMD, the grammar is _____ .
8. The grammar

   $S \rightarrow bbba \mid \varepsilon$

   $A \rightarrow \varepsilon$

   is CFG. State whether this claim is _____ (True/False).
9. Context free languages are described by type _____ grammars.
10. In Type 1 grammars, for the production $\alpha \rightarrow \beta$ the relation between α and β is _____ .
11. To simplify the grammar $S \rightarrow ST \mid \varepsilon$, $T \rightarrow abT \mid ab$ we apply elimination of _____ .
12. Is the grammar $S \rightarrow abA \mid bbc$, $A \rightarrow aA \mid ab$ GNF? (True/False).
13. If the grammar is $S \rightarrow aA \mid b$, $A \rightarrow aA \mid ab$, then its equivalent grammar in CNF is _____ .
14. _____ a context free grammar to generate expression with balanced parenthesis.
15. Grammar $S \rightarrow aSb \mid SS \mid \varepsilon$ is _____ grammar.
16. For every ambiguous grammar, there is an equivalent grammar for the same language which is unambiguous. (True/False).
17. _____ Languages are the subset of context free languages.
18. Every context free language is context sensitive language. (True/False).
19. Elimination of null productions results in simplified grammar with no unit productions and useless symbols. (True/False)
20. If a grammar has two different LMD's or RMD's, then it is ambiguous. (True/False)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Answers

1. CFG
2. $\varnothing$
3. not deriving any terminal
4. Nullable
5. Subset
6. Subset
7. unambiguous
8. True
9. Type 2
10. $|\alpha| \leq |\beta|$ and $|\alpha| = 1$.
11. Null rule
12. False
13. $S \rightarrow C_aA \mid b, A \rightarrow C_aA \mid C_aC_bC_a \rightarrow a, C_b \rightarrow b$
14. $S \rightarrow (S) \mid \varepsilon$
15. Ambiguous
16. False
17. Regular
18. True
19. False
20. True

Objective Question Bank

1. A context free grammar is ambiguous if
   a) The grammar contains useless non-terminals
   b) The grammar produces more than one parse tree for some sentence
   c) Some production has two non-terminals side by side on the right-hand side
   d) None of the above

2. FORTRAN is a
   a) Regular language
   b) Context free language
   c) Context-sensitive language
   d) None of the above

3. If G is a context free grammar and w is a string of length 1 in L(G), how long is a derivation of w in G, if G is in the Chomsky normal form ?
   a) $2l$
   b) $2l+1$
   c) $2l-1$
   d) l

4. What is the language generated by the the following CFG:

   $S \rightarrow aSb/aAb/aBb$
   $A \rightarrow aA / a$
   $B \rightarrow Bb / b$

   Here V = {S, A, B) and T = {a, b}
   a) $\{a^n b^m, m > 0, |n - m| > 2\}$
   b) $\{a^n b^m, n,m > 1, |n - m| \geq 1\}$
   c) $\{a^n b^m, m > 0, |n - m| > 1\}$
   d) $\{a^n b^m, m > 0, |n - m| > 0\}$

5. If L_1 is context free language and L_2 is regular language, which of the following is/are false?
   a) $L_1 - L_2$ is not context free
   b) $L_1 \cap L_2$ is context free
   c) L_1 complement is context free
   d) L_{21} is regular

6. Let L denote the language generated by the grammar $S \rightarrow 0S0 \mid 00$. Which of the following is true?
   a) $L = 0^+$
   b) L is regular, but not 0^+
   c) L is context free, but not regular
   d) L is not context free

7. Aliasing in the context free programming languages refers to
   a) Multiple variables having the same memory location
   b) Multiple variables having the same values
   c) Multiple variables having the same memory identifier
   d) Multiple uses of the same variable

8. Let G = ({S}), {a, b}, P, S) be a context free grammar where the rule set P
   $S \rightarrow a\,S\,b / S\,S / \varepsilon$
   Which of the following is true?
   a) G is not ambiguous
   b) Strings x, y ∈ L(G) such that xy ∉ L(G).
   c) There is a deterministic PDA that accepts L(G)
   d) We can find a DFA that accepts L(G)

9. PDA can recognize
   a) any grammar
   b) only CFG
   c) any unambiguous grammar
   d) only regular grammar

10. The following CFG S→ aS / bS / a / b is equivalent to the regular expression
    a) $(a^* + b)^*$ b) $(a + b)^+$ c) $(a + b)(a + b)^*a$ d) $(a + b)^*(a + b)a$

11. Any string of terminals that can be generated by the CFG
    $S \rightarrow XY,\ X \rightarrow aX / bX / a,\ Y \rightarrow Ya / Yb / a$
    a) has at least one b
    b) should end in a 'a'
    c) has no consecutive a's or b's
    d) has at least two a's

12. The CFG
    $S \rightarrow aB / bA, \quad A \rightarrow a / aS / bAA, \quad B \rightarrow b / bS / aBB$
    generates strings of terminals that have
    a) equal number of a's and b's
    b) odd number of a's and odd number of b's
    c) even number of a's and even number of b's
    d) odd number of a's and even number of b's

13. The set $\{a^n b^n / n = 1, 2, 3....\}$ can be generated by the CFG
    a) $S \rightarrow ab / aSb$
    b) $S \rightarrow aaSbb / ab$
    c) $S \rightarrow ab / aSb / \varepsilon$
    c) $S \rightarrow aaSbb / aSb / aabb$

14. Choose the correct statements:
    a) All languages can be generated by CFG
    b) Any regular language has an equivalent CFG
    c) Some non-regular languages cannot be generated by any CFG
    d) Some regular languages cannot be generated by any CFG

15. The set $A = \{a^n b^n a^n / n = 1, 2, 3....\}$ is an example of a grammar that is
    a) regular b) context free c) not context free d) none of the above

16. Let $L_1 = \{a^n b^n a^m / n, m = 1, 2, 3.....\}$
    $L_2 = \{a^n b^m a^m / n, m = 1, 2, 3....\}$
    $L_3 = \{a^n b^n a^n / n = 1, 2, 3.....\}$
    Choose the correct answer:
    a) $L_3 = L_1 \cap L_2$
    b) L_1 and L_2 are CFLs, but L_3 is not CFL
    c) L_1 and L_2 are not CFLs, but L_3 is CFL
    d) L_1 is a subset of L_3

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

17. $L = \{a^n b^n c^n / n = 1, 2, 3....\}$ is an example of a language that is
    a) context free
    b) not context free
    c) not context free, but its complement is CF
    d) context free, but its complement is not CF

18. The intersection of a CFL and a regular language
    a) need not be regular
    b) need not be context free
    c) is always regular
    d) is always CFL

19. Let $L_1 = \{a^i b^j c^k / i < j < k\}$
    $L_2 = \{a^i b^j / j = i^2\}$
    $L_3 = \{a^i / i \text{ is prime}\}$
    L_4 = the set of strings a's, b's and c's with an equal number of each
    $L_5 = \{a^n b^n c^m / n \leq m \leq 2n\}$
    Which of the following statements is correct?
    a) L_1 and L_3 are not CFLs, but rest are CFLs
    b) L_3 and L_4 are not CFLs, but rest are CFLs
    c) All are not CFLs
    d) None of the above

20. Consider the following CFG. G defined by productions
    $S \rightarrow aSbS / bSaS / \varepsilon$
    The language generated by this CFG is
    a) the set of all strings that contain even number of a's and even number of b's
    b) the set of all strings that contain odd number of a's and even number of b's
    c) the set of all strings that contain odd number of a's and odd number of b's
    d) the set of all strings with an equal number of a's and b's

21. Consider the following grammar productions:
    $S \rightarrow AB$,
    $A \rightarrow BB / a$,
    $B \rightarrow AB / b$,
    Choose the incorrect statement
    a) aabbb can be derived from the above grammar
    b) aabb can be derived from the above grammar
    c) ababab can be derived from the above grammar
    d) aaabb can be derived from the above grammar

22. Consider the following context free grammars:
    1. $S \rightarrow aSbb / a$
    2. $S \rightarrow aSA / a$, $A \rightarrow bB$, $B \rightarrow b$
    Which of the following is correct?
    a) The language generated by 1 is a subset of 2
    b) The language generated by 2 is a subset of 1
    c) The language generated by both the grammars 1 and 2 are equal
    d) None of the above

23. Which of the following is incorrect?
    a) Every regular language is a non-deterministic, context free language
    b) If L_1 is a deterministic context free and L is regular, then the language $L_1 \cup L_2$ is deterministic and context free
    c) If L is a deterministic context free and L_2 is regular, then the language $L_1 \cap L_2$ is deterministic and context free
    d) For every deterministic and context free language, the reverse also is context free.

24. A context free grammar is said to be ambiguous if it has
    a) $w \in L(G)$ that has at least two distinct derivative trees
    b) $w \in L(G)$ that has at least two leftmost derivations
    c) $w \in L(G)$ that has at least two rightmost derivations
    d) none of the above

Answers

1. b	2. b	3. c	4. b	5. a, c	6. b	7. a	8. c	9. b	10. b
11. d	12. a	13. a	14. b, c	15. c	16. a	17. c	18. c	19. c	20. d
21. d	22. a	23. b	24. d						

Exercises

1. Define leftmost and rightmost derivations. Give example.

2. Consider the grammar G: $S \rightarrow S + S \mid S*S \mid (S) \mid a$. For the given string a + a*a show
   a. Parse trees
   b. Leftmost derivations
   c. Find the unambiguous grammar G' equivalent to G and show that L(G) = L(G') and G' is unambiguous.
   Hint: Eliminate left recursion.

3. Convert the following CFG to an equivalent unambiguous grammar:
   $E \rightarrow E + E$
   $E \rightarrow E*E$
   $E \rightarrow a \mid b \mid (E)$

4. Check whether the grammar given below is ambiguous or not.
   (a) $S \rightarrow 0S1 \mid SS \mid \varepsilon$ w = '0011'
   (b) $S \rightarrow AB \mid aaB, A \rightarrow a \mid AA, B \rightarrow b$ w = 'aab'
   (c) $S \rightarrow SbS \mid a$ w = 'abababa'
   (d) $S \rightarrow aSb \mid ab$
   (e) $R \rightarrow R + R \mid RR \mid R* \mid a \mid b \mid c$ w = a + b*c

5. Construct the reduced grammar from regular grammar given below:
   $S \rightarrow Aa \mid bs \mid \varepsilon$
   $A \rightarrow aA \mid bB \mid \varepsilon$
   $B \rightarrow aA \mid bc \mid \varepsilon$
   $C \rightarrow aC \mid bc$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

6. Find a CFG without ε productions, unit production and useless productions equivalent to the grammar defined by

   $S \rightarrow ABaC$
   $A \rightarrow BC$
   $B \rightarrow b \mid \varepsilon$
   $C \rightarrow D \mid \varepsilon$
   $D \rightarrow d.$

7. Eliminate the useless symbol from the following grammar. Also eliminate the useless productions.

   $S \rightarrow aB \mid aCD \mid aE$
   $B \rightarrow bC$
   $C \rightarrow aB \mid b$
   $D \rightarrow aE$
   $E \rightarrow bCD$

8. Eliminate useless symbols from following grammar:

   $S \rightarrow a\,A \mid a \mid Bb \mid cC$
   $A \rightarrow aB$
   $B \rightarrow a \mid Aa$
   $C \rightarrow cCD$
   $D \rightarrow ddd$

9. Remove ε productions from the following grammar:

   $S \rightarrow XYX$
   $X \rightarrow 0X \mid \varepsilon$
   $Y \rightarrow 1Y \mid \varepsilon$

10. Eliminate the useless symbols and productions from the following grammar:

    $G = (V, T, P, S)$ where $V = \{S, A, B, C\}$, $T = \{a, b\}$ and productions P is given by

    $S \rightarrow ab \mid A \mid C, A \rightarrow a, B \rightarrow ac, C \rightarrow aCb$

11. Remove ε productions from the following grammar:

    $S \rightarrow A0B \mid BB \mid 0C \mid CBA$
    $A \rightarrow C\,0 \mid 1C \mid CC \mid CBA$
    $B \rightarrow B\,0 \mid AB \mid \varepsilon$
    $C \rightarrow 0A \mid BBB$

12. Remove unit productions from the grammar

    $S \rightarrow 0A \mid 1B \mid C$
    $A \rightarrow 0S \mid 00$
    $B \rightarrow 1 \mid A$
    $C \rightarrow 01$

13. Remove unit productions from the grammar

    $S \rightarrow AB$
    $A \rightarrow a$
    $B \rightarrow C \mid b$

$C \rightarrow D$
$D \rightarrow E \mid bC$
$E \rightarrow d \mid Ab$

14. Optimize the following grammar:

$S \rightarrow A \mid 0C1$
$A \rightarrow B \mid 01 \mid 10$
$C \rightarrow CD \mid \varepsilon$

15. Optimize the following grammar:

$S \rightarrow aAa$
$A \rightarrow Sb \mid bCc \mid DaA$
$C \rightarrow abb \mid DD$
$E \rightarrow aC$
$D \rightarrow aDA$

16. Convert the CFG (to GNF)

$S \rightarrow AB$
$A \rightarrow BS \mid b$
$B \rightarrow SA \mid a$

17. Convert the CFG (to GNF)

$E \rightarrow E + T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow id \mid (E)$

18. What are CNF and GNF of a context free grammar? Give examples. Obtain the following CFG in GNF notations:

$S \rightarrow AB$
$A \rightarrow aA \mid bB \mid b$

19. What are CNF and GNF notations of a grammar? Obtain the CNF notation for the following grammar:

$S \rightarrow ASB$
$A \rightarrow aAS \mid a$
$B \rightarrow SbS \mid aAS \mid a \mid bb$

20. If L_1 and L_2 are two context free languages, then prove that $L_1 \cup L_2$, L_1, L_2 and L_1^* are context free languages.

21. Convert the following grammar (to CNF)

$S \rightarrow S + S \mid S*S \mid a \mid b$

22. Simplify the following CFG

$S \rightarrow ASB \mid \in, A \rightarrow aAS \mid a, B \rightarrow SbS \mid A \mid bb$
and convert it to CNF.

23. Convert the following grammar (to CNF):

$S \rightarrow S + S \mid S*S \mid a \mid b$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

24. Convert the following grammar (to CNF):
    $S \rightarrow \sim S \mid [S \cap S] \mid a \mid b$
25. Convert following grammar (to GNF):
    $S \rightarrow AB, A \rightarrow BS \mid b, B \rightarrow SA \mid a$
26. Convert following grammar (to GNF):
    $E \rightarrow E + E \mid T, T \rightarrow T*F \mid F, F \rightarrow (E) \mid a$
27. Convert following grammar (to GNF):
    $S \rightarrow AB1 \mid 0, A \rightarrow 00A \mid B, B \rightarrow 1A1$
    Hint: For Problems 26 and 27, first convert to CNF and then to GNF.
28. Write equivalent, unamabiguous grammar for the following CFG:
    $R \rightarrow R + R \mid RR \mid R^* \mid a \mid b$
29. Write equivalent unamabiguous grammar for the following CFG:
    Bexpr $\rightarrow$ Bexpr and Bexpr | Bexpr or Bexpr | not R Bexpr | a | b
30. Convert the grammar (to GNF)
    $S \rightarrow AaAb \mid BbBa, A \rightarrow \varepsilon, B \rightarrow \varepsilon$
31. Convert the grammar (to CNF)
    $S \rightarrow AaAb \mid BbBa, A \rightarrow \varepsilon, B \rightarrow \varepsilon$
32. Construct the CFG generating the set $\{w\$w^r\$ \mid w \text{ is in } (0 + 1)+\}$.
    Hint: Grammar is similar to an odd palindrome where the mid-element is \$. Use a new start symbol that derives the odd palindrome followed by \$.
33. Show that for each CFG G, there is a constant c such that if w is in L(G), and $w \neq \varepsilon$, then w has a derivation of no more than $c|w|$ steps.
    Hint: Assume the grammar is in CNF.
34. Show that conversion to CNF can square the number of productions in a grammar.

Push Down Automata

5

- The concept PushDown Automata (PDAs) is useful in designing parsers or syntax analysers. A parser verifies the syntax of the text. Parsing is a part of the compilation process. So, knowledge of PDAs is useful in compiler design.

In this chapter, we introduce PushDown Automata (PDAs) and discuss how the language is accepted. Theorems that show the equivalence of PDA and context free languages are discussed. Equivalence of PDA that accepts language using empty stack and reaching to final state is explained. Finally, how to use PDA as a parser is described.

The automata that we have discussed so far have very limited memory capabilities. It cannot recognize all CFLs. We understand that this is because Finite Automata have strictly finite memories; whereas recognition of a CFL may require storing an unbounded amount of information. For example, scanning a string from the language $L = \{a^n b^n\}$. We must not only check that all a's precede the first b, but also match the number of a's with the number of b's. Since n is unbounded, this counting cannot be done with a finite memory. The difficulty can be avoided by adding an auxiliary memory in the form of a stack. All the a's are read and added to a stack. This is how they are remembered. When b's are read, match the number of a's with the number of b's by taking out an a from the stack for each b. This is how we can strengthen the power of automaton, by adding a stack structure.

We will now consider a new notion of automata, PDAs. PDAs are finite automatons a with a stack, i.e, a data structure that can be used to store an arbitrary number of symbols (and hence PDAs have an infinite set of states) but that can be accessed only in a last-in–first-out (LIFO) fashion. The languages that can be recognized by PDA are precisely the context free languages.

5.1 Pushdown Automata

As Figure 5.1 indicates, a PDA consists of three components: 1) An input tape, 2) a finite control and 3) a stack structure. The input tape consists of a linear configuration of cells each of which contains a character from the input alphabet. This tape can be moved one cell at a time to the left. The stack is also a sequential structure that has a first element and grows in either direction from the other end. The control unit has some pointer (head) which points the current symbol that is to be read. The head positioned over the current stack element can read and write special stack characters from that position. The current stack element is always the top element of the stack; hence, the name 'stack'. The control unit contains both tape head and stack head and finds itself at any moment in a particular state.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

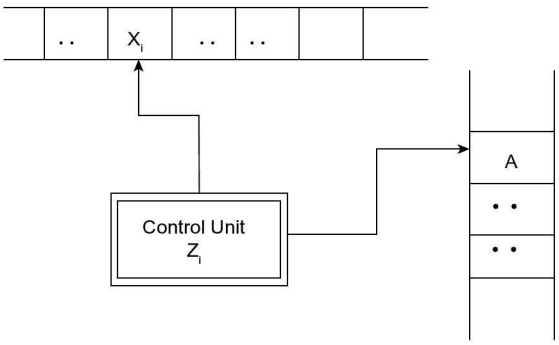


Fig. 5.1 *Conceptual Model of a Pushdown Automation*

Definition 1: A finite state pushdown automation is a 7-tuple $M = (Q, \Sigma, \delta, \tau, q_0, Z_0 F)$ where

Q: a finite set of states

Σ: a finite set of input alphabet

Γ: a finite set of stack alphabet

q_0: the start state. $q_0 \in Q$

$F \subseteq Q$: a set of final state, and

δ: a transition function from

$Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma) \rightarrow Q \times \Gamma^*$

Z_0 is the initial stack symbol. $Z_0 \in \Gamma$.

PDA has 2 alphabets:

a) An input alphabet Σ (for input strings)

b) A stack alphabet Γ (stored on the stack)

A move on PDA may indicate:

i) An element may be added to the stack

ii) An element may be deleted from the stack:
$(q, a, Z_0) = (q, \varepsilon)$ and

iii) There may or may not be change of state

Example 5.1

i) $\delta(q, a, Z_0) = (q, aZ_0)$ indicates that in the state q on seeing a, a is pushed onto stack. There is no change of state.

ii) $\delta(q, a, Z_0) = (q, \varepsilon)$ indicates that in the state q on seeing a, the current top symbol Z_0 is deleted from the stack.

iii) $\delta(q, a, Z_0) = (q_1, aZ_0)$ indicates that a is pushed onto the stack and the state is changed to q_1.

5.1.1 Graphical Representation of PDA

Let $M = (Q, \Sigma, \tau, \delta, q_0, Z_0, F)$ be a PDA where $Q = \{p, q\}$, $\Sigma = \{a, b, c\}$, $\tau = \{a, b\}$, $q_0 = q$, $F = \{p\}$, and δ is given by the following equatios:

$$\delta(q, a, \varepsilon) = \{(q, a)\}$$

$$\delta(q, b, \varepsilon) = \{(q, b)\}$$

$$\delta(q, c, \varepsilon) = \{(p, \varepsilon)\}$$

$$\delta(p, a, a) = \{(p, \varepsilon)\}$$

$$\delta(p, b, b) = \{(p, \varepsilon)\}$$

It is very hard to visualize what M does. The transition diagram in Figure 5.2 helps us to understand. We use the notation a, b → c on a transition between states q_i and q_j to mean

$$\delta(q_i, a, b) = \{(q_j, c)\}.$$

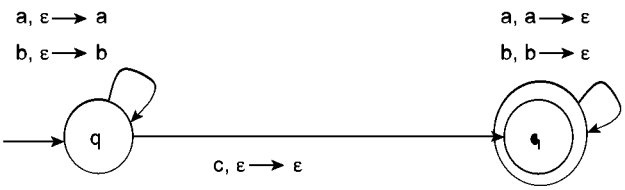


Fig. 5.2

In words, M works as follows: It stays in state q pushing input symbols 'a' and 'b' onto the stack (irrespective of what is at the top of the stack) until it encounters a 'c'. Then it moves to state p, in which it repeatedly pops a and b symbols off the stack provided the input symbol is identical to that on top of the stack. If at the end of the string, the stack is empty, then the machine accepts. However, notice that failure in processing a string is not explicitly represented. For example, what happens if the input string has no c in it? In that case, M will never leave state q. Once the input is finished, we find that we are not in a final state and so cannot accept the string.

5.1.2 Instantaneous Description of PDA

During processing, the PDA moves from one configuration to another configuration. At any given instance, the configuration of PDA is expressed by the state of finite control, the content of stack and the input. The configuration is expressed as a triple (q, x, γ), where

1. q is the state.
2. x is the input string to be processed.
3. γ is the content of the stack where the leftmost symbol corresponds to top of stack, and the rightmost is the bottom element.

Example 5.1 When string ababcbcb is processed, the instantaneous description is as shown below.

$$(q, ababcbab, \varepsilon) \Rightarrow (q, babcbab, a) \Rightarrow (q, abcbab, ba) \Rightarrow (q, bcbab, aba) \Rightarrow (q, cbab, baba) \Rightarrow (p, bab, baba) \Rightarrow (p, ab, aba)$$

$$\Rightarrow (p, b, ba) \Rightarrow (p, \varepsilon, a)$$

At this point, the input string is exhausted and the computation stops. We cannot accept the original string although we are in an 'accept' state because the stack is not empty.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Thus we see that $L(M) = \{wcw^R \mid w \in \{a, b\}^*\}$. This example used c as a marker telling the machine when to change states. It turns out that such an expedient is not needed because we have non-determinism at our disposal.

5.1.3 Language Acceptance by PDA

A language can be accepted by a PDA using two approaches:

1. **Acceptance by final state:** The PDA accepts its input by consuming it and finally it enters the final state.
2. **Acceptance by empty stack:** On reading the input string from the initial configuration for some PDA, the stack of PDA becomes empty.

Example 5.1 Design a PDA which accepts the language $L = \{a^n b^n / n \geq 1\}$.

Solution: PDA can be constructed as follows:

Let q_0 be initial state, q_f final state and Z_0 bottom of the stack. Read each 'a' and push it onto the stack. Then read each b and pop out from stack for matching with a's. When all b's are read, if the stack is empty, then string is valid. If any a's are left over on stack or b's on input type then the string is rejected.

Suppose the input is aaabbb. The steps involved are shown in Figure 5.3.

The PDA moves are the following:

$\delta(q_0, a, Z_0) = (q_0, aZ_0)$ push a
$\delta(q_0, a, a) = (q_0, aa)$ push a
$\delta(q_0, b, a) = (q_1, \varepsilon)$ pop a and change the state
$\delta(q_1, b, a) = (q_1, \varepsilon)$ pop a
$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0)$ change to final state q_f and halt

The moves of PDA can be shown as transition diagram: as in Figure 5.3.

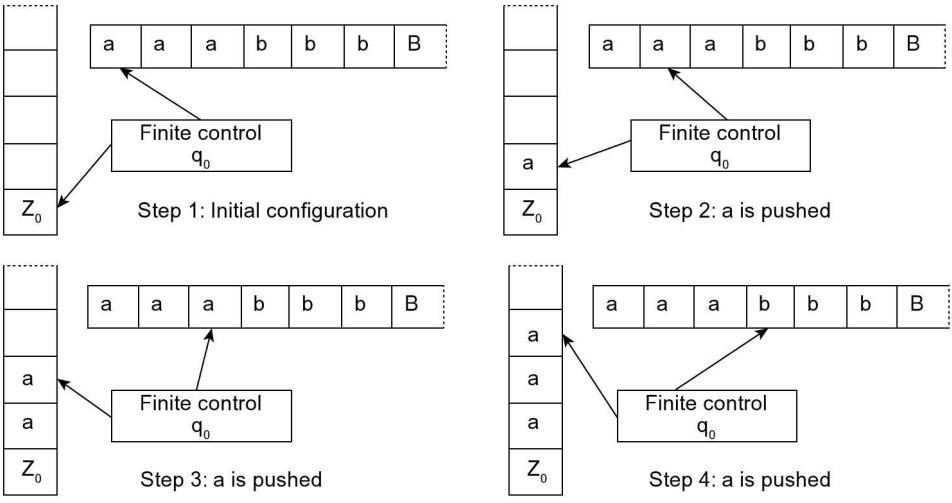


Fig. 5.3 *PDA for Example 5.3*

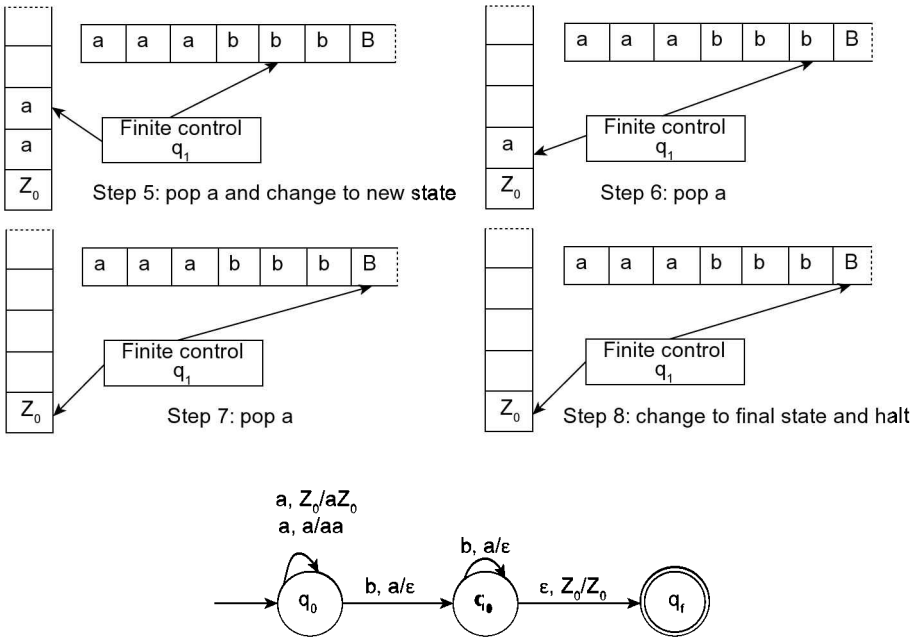


Fig. 5.3 *Continued*

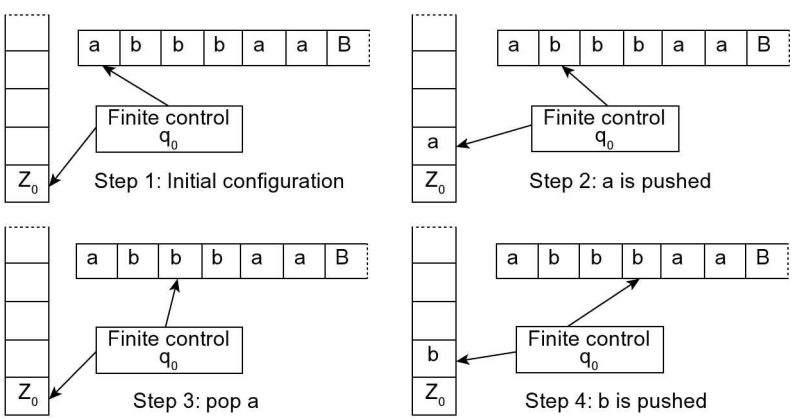
Let us slightly modify the above example.

Example 5.1 Design a PDA which accepts equal number of a's and b's over $\Sigma = \{a, b\}$.

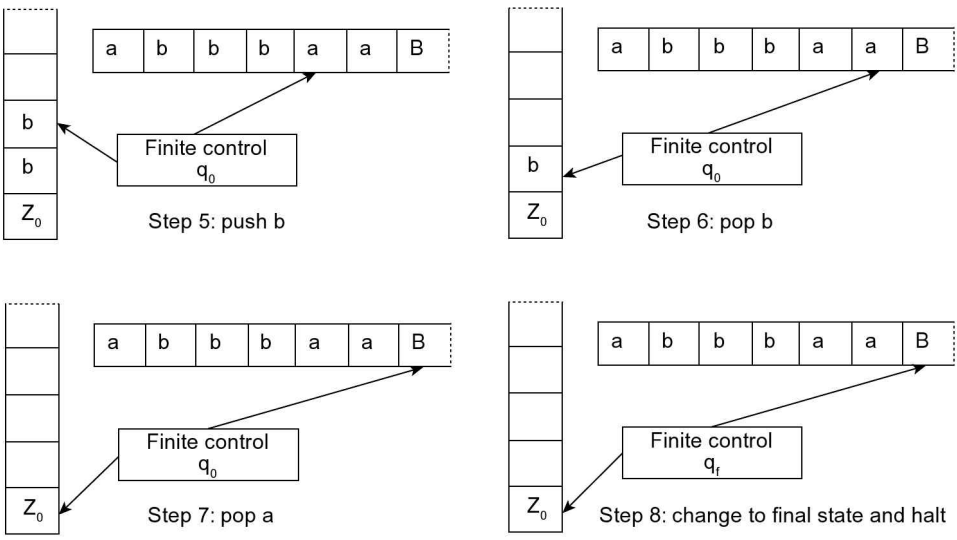
Solution: The input can start with either a or b, and they can occur in any order, as bbaaba or bababa or babbaa and so on are all valid in the language.

To design such a PDA, read the first symbol either a or b push it on the stack. Then read the next symbol. If the next symbol and top of stack are same, push it onto the stack. Whenever top of stack and tape symbol are different, pop off the stack. When entire tape is read, if the stack becomes empty, then string is accepted.

The PDA can be given as follows:



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]



Let q_0 be initial state, q_f be final state and Z_0 be the bottom of the stack and is shown in the Figure 5.4.

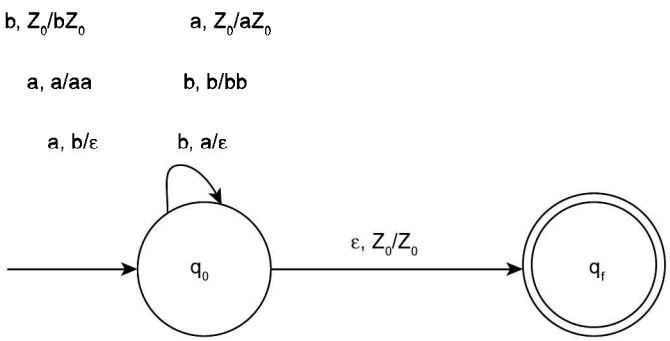


Fig. 5.4 *Sample PDA*

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, b) = (q_0, bb)$$

$$\delta(q_0, a, b) = (q_0, \varepsilon)$$

$$\delta(q_0, b, a) = (q_0, \varepsilon)$$

$$\delta(q_0, \varepsilon, Z_0) = (q_f, Z_0)$$

PDA can also be shown using special symbols explained in Figure 5.5:

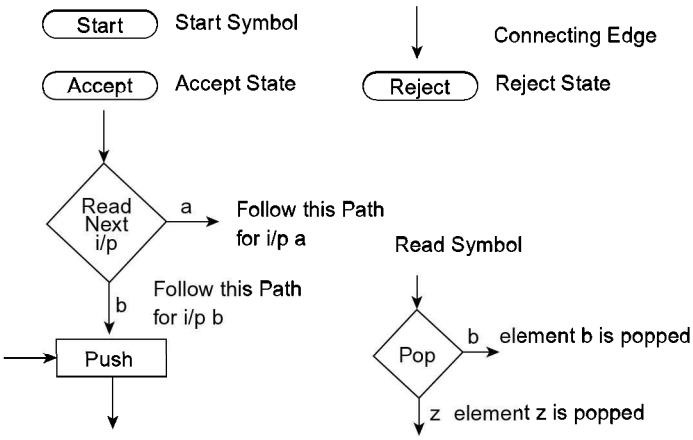


Fig. 5.5 *Symbols for PDA*

Example 5.5 Design a PDA that accepts $L = \{0^n 1^{2n} \mid n \geq 1\}$.

Solution: Here, we have to match each '0' with two 1's. So, Read the 0's and push it on the stack. After that, read one '1' and change to different state. Then read second 1. Now pop off from the stack and reset the state to initial state to continue the same process with each '1'. When input is read completely, if the stack becomes empty, then it is successful.

The PDA can be given, as flow chart shown in Figures 5.6 and 5.7

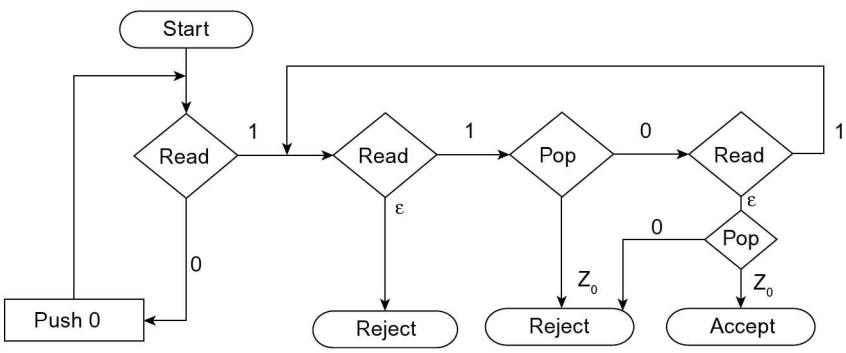


Fig. 5.6 *Sample of Flow Chart for PDA*

Let q_0 be initial state, q_f be final state and Z_0 be the bottom of the stack.

$$\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 1, 0) = (q_1, 0)$$

$$\delta(q_1, 1, 0) = (q_2, \varepsilon)$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$$\delta(q_2, 1, 0) = (q_1, 0)$$

$$\delta(q_2, \varepsilon, Z_0) = (q_f, Z_0)$$

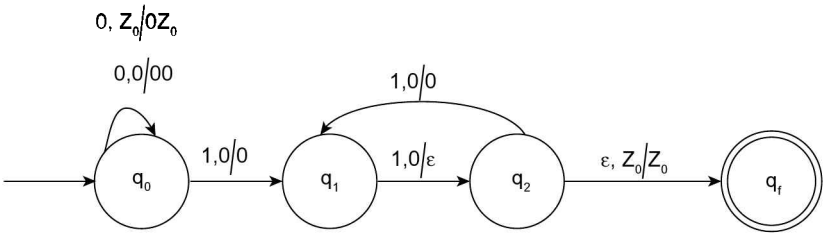


Fig. 5.7 *PDA for Example 5.5*

Example 5.5 Design a PDA that accepts $L = \{a^3b^nc^n \mid n \geq 0\}$.

Here, ensure that exactly three a's are read initially. Then read all b's onto the stack and match them with c's by poping off. To ensure three a's, read each 'a' and change the state and is shown in Figures 5.8 and 5.9.

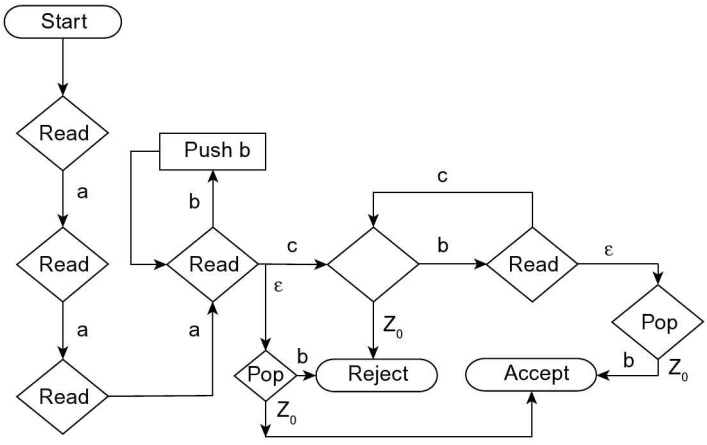


Fig. 5.8 *Flow Chart for Example 5.6*

$$\delta(q_0, a, Z_0) = (q_1, Z_0)$$

$$\delta(q_1, a, Z_0) = (q_2, Z_0)$$

$$\delta(q_2, a, Z_0) = (q_3, Z_0)$$

$$\delta(q_3, \varepsilon, Z_0) = (q_f, Z_0)$$

$$\delta(q_3, b, Z_0) = (q_4, bZ_0)$$

$$\delta(q_4, b, b) = (q_4, bb)$$

$$\delta(q_4, c, b) = (q_5, \varepsilon)$$

$$\delta(q_5, c, b) = (q_5, \varepsilon)$$

$$\delta(q_5, \varepsilon, Z_0) = (q_f, Z_0)$$

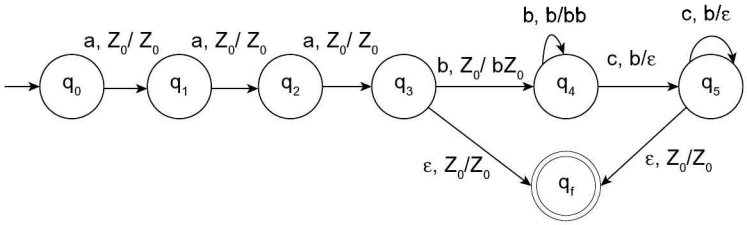


Fig. 5.9 *PDA for Example 5.6*

Example 5.1 Design a PDA which accepts $L = \{wcw^r | w \in (a + b)^*\}$.

Read the string w and push it onto the stack untill it encounters c. After that, read each symbol, if it matches with the top of the stack, pop off the symbol. When input is read completely, if stack becomes empty, then string is accepted.

The PDA can be given as follows:

Let q_0 be initial state, q_f be final state and Z_0 be the bottom of the stack as shown in Figure 5.10.

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, a, b) = (q_0, ab)$$

$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$

$$\delta(q_0, b, a) = (q_0, ba)$$

$$\delta(q_0, b, b) = (q_0, bb)$$

$$\delta(q_0, c, Z_0) = (q_1, Z_0)$$

$$\delta(q_0, c, a) = (q_1, a)$$

$$\delta(q_0, c, b) = (q_1, b)$$

$$\delta(q_1, a, a) = (q_1, \varepsilon)$$

$$\delta(q_1, b, b) = (q_1, \varepsilon)$$

$$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0)$$

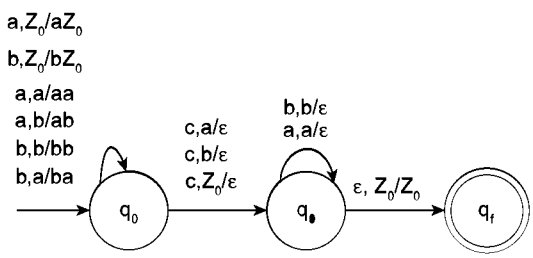


Fig. 5.10 *PDA for Example 5.7*

The final PDA is given by $M = \{(q_0, q_1, q_f), (a, b, c), (a, b, Z_0), \delta, q_0, Z_0, \{q_f\}\}$.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

5.2 Equivalence of Acceptance of Final State and Empty Stack

Theorem 1: If L is $L(M_1)$ for some PDA M_1 that accepts using final state, then L is $L(M_2)$ for some PDA M_2 that accepts using the empty stack.

Proof: Let $M_1 = \{Q_1, \Sigma_1, \tau_1, \delta_1, q_1, Z_1, q_{f1}\}$ where the PDA M_1 accepts the strings by entering into final state, ie,

$$(q_1, x, Z_1) \overset{*}{\Rightarrow} (q_{f1}, \varepsilon, \gamma)$$

We need to construct a PDA M_2 such that it accepts the strings of L.
$\therefore$ we need to find

$$M_2 = \{Q_2, \Sigma_2, \tau_2, \delta_2, q_2, Z_2, \phi\} \text{ such that } L(M_2) = L.$$

Let $Q_2 = Q_1 \cup \{q_2, q_e\}$,
$\Sigma_2 = \Sigma_1 \cup \{\varepsilon\}$,
$\tau_2 = \tau_1 \cup \{Z_2\}$,
q_2 – initial state,
Z_2 – initial stack symbol,
δ_2 is defined as follows.

- Add transition in M_2, to move to initial state of M_1 to process the strings.
  $\delta_2 (q_2, \varepsilon, Z_2) = (q_1, Z_1 Z_2)$
- Simulate the moves of M_1 on M_2:
  $\delta_2 (q, a, X) = \delta_1(q, a, X)$
  for all $q \in \{Q_1\} - q_{f1}$, $a \in \Sigma_1$ and $X \in \tau_1$.
- If M_1 enters into final state, then moves in M_2 are defined to move to q_e state, which will empty the stack contents.

$$\delta_2 (q_{f1}, \varepsilon, X) = (q_e, \varepsilon, \varepsilon)$$

$$\delta_2 (q_e, \varepsilon, X) = (q_e, \varepsilon, \varepsilon)$$

For all $X \in \tau$.

To show $x \in L(M_2)$

$$(q_2, x, Z_2) \Rightarrow (q_1, x, Z_1 Z_2) \overset{*}{\Rightarrow} (q_{f1}, \varepsilon, \gamma) \Rightarrow (q_e, \varepsilon, \gamma) \overset{*}{\Rightarrow} (q_e, \varepsilon, \varepsilon)$$

Theorem 2: If L is $L(M_3)$ for some PDA M_3 which accepts using empty stack, then L is $L(M_4)$ for some PDA M_4 that accepts using final state.

Proof: Let $M_3 = \{Q_3, \Sigma_3, \tau_3, \delta_3, q_3, Z_3, \phi\}$, where the PDA M_3 accepts the strings by emptying the stack, ie,

$$(q_3, x, Z_3) \overset{*}{\Rightarrow} (q, \varepsilon, \varepsilon) \; \forall \; q \in Q_3$$

We need to construct PDA M_4 such that it accepts string of L. $\therefore$ we need to find

$$M_4 = \{Q_4, \Sigma_4, \tau_4, \delta_4, q_4, Z_4, q_{f4}\} \text{ such that } L(M_4) = L.$$

Let $Q_4 = Q_3 \cup \{q_4, q_{f4}\}$,
$\Sigma_4 = \Sigma_3 \cup \{\varepsilon\}$,
$\tau_4 = \tau_3 \cup \{Z_4\}$,
q_4 – initial state,
Z_4 – initial stack symbol,
q_{f4} – final state,
δ_4 is defined as follows.

- Add transition in M_4, to move to initial state of M_3, to process the strings:

  $\delta_4(q_4, \varepsilon, Z_4) = (q_3, Z_3Z_4)$
- Simulate the moves of M_3 on M_4:

  $\delta_4(q, a, X) = \delta_3(q, a, X)$. For all $q \in \{Q_3\}$, $a \in \Sigma_3$ and $X \in \tau_3$
- Even if M_3 empties the stack content, there would be Z_4 on the stack. Now move to final state:

  $\delta_4(q, \varepsilon, Z_4) = (q_{f4}, \varepsilon, Z_4)$. For all $q \in Q_3$.

To show $x \in L(M_4)$,

$$(q_4, x, Z_4) \Rightarrow (q_3, x, Z_3 Z_4) \overset{*}{\Rightarrow} (q, \varepsilon, Z_4) \Rightarrow (q_{f4}, \varepsilon, Z_4)$$

5.3 Types of PDAs

5.3.1 Deterministic PDA

A PDA that has at most one choice of move in any state is called a deterministic PDA. Non-deterministic PDA (NPDA) provides non-determinism in the moves defined. Deterministic PDAs (DPDAs) are very useful in programming languages. For example, parsers used in Yet Another Compiler Compiler (YACC) are deterministic PDA's (DPDA).

Definition 2: A PDA $P = (Q, \Sigma, \tau, \delta, q_0, Z_0, F)$ is deterministic if and only if

1. $\delta(q, a, X)$ has at most one move for $q \in Q$, $a \in \Sigma \cup \{\varepsilon\}$, and $X \in \tau$.
2. If $\delta(q, a, X)$ is not empty for some $a \in \Sigma$, then $\delta(q, \varepsilon, X)$ must be empty.

DPDA is less powerful than NPDA. The context free languages could be recognized by NPDAs.

Deterministic Context Free Language (DCFL)

The class of languages accepted by DPDA is in between that of regular languages and CFLs. Such a language is called a deterministic context free language (DCFL) and is a subset of the languages accepted by NPDA. NPDA can be constructed for accepting language of palindromes, but not by DPDA.

The syntax of programming languages can be described by DCFLs. Compiler writing system requires a restricted form of context free grammar, also known as DCFG. LR grammars are restricted grammars that generate DCFL.

It is useful for the compiler designer to find out whether the given grammar is suitable for defining the syntax of the language. To find whether the language is a CFL, we can use this pumping lemma; but this pumping lemma cannot be used to find whether the

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

grammar is DCFL. To identify whether the language is DCFL or not, we can use the closure properties of DCFL as they are not closed on all properties as CFLs.

Note: Most of the examples we have discussed so far are DPDAs.

5.3.2 Closure Properties of DCFL

DCFL are closed under

- Union
- Concatenation
- Kleen's closure.
- Complementation.

Note: CFLs are not closed under complement, but DCFLs are closed. Hence, if the CFL is found to be closed under complement, then it is a DCFL.

Normal forms for DPDAs: Any PDA is said to deterministic if it satisfies the following properties.

1) Whenever $\delta(q, a, X)$ is non-empty for some a in Σ, $\delta(q, \varepsilon, X)$ is empty, and
2) For each q in Q, a in $\Sigma \cup \{\varepsilon\}$ and X in Γ, $\delta(q, a, X)$ contains at most one element.

The first rule prevents us from considering the next input. The second rule prevents moving to two different IDs for the same input symbol.

The DPDA can be represented in normal form by imposing the following two lemmas on transition to restrict the opertation:

Lemma 1: Every DCFL is L(M) for a DPDA $M = (Q, \Sigma, \tau, \delta, q_0, Z_0, F)$ such that if $\delta(q, a, X) = (p, \gamma)$, then $|\gamma| \leq 2$.

This lemma would restrict pushing more than one symbol onto the stack.

Lemma 2: Every DCFL is L(M) for a DPDA $M = (Q, \Sigma, \tau, \delta, q_0, Z_0, F)$ such that if $\delta(q, a, X) = (p, \gamma)$, then γ is either ε (pop operation), X (no stack operation) of YX (push of single element).

By applying these two lemmas, the DPDA's would be in normal form where the only stack operations performed are to erase the top symbol or to push one symbol or only change of state.

Closure of DCFL's under complementation

To show that the complement of DCFL is also a DCFL, we can use the approach used in the case of regular languages, but with some modifications. This is because in the case of regular languages, to show that the complement language is regular, we could interchange the final states and non-final states such that if string w is accepted in L, then it is rejected in L'. There are two difficulties with this approach.

1. The original DPDA never moves beyond some point on an input string w.

Reasons: This could happen, on reading the input w, if it reaches an ID where no move is possible or moves are defined with ε, which forces it to make infinite moves without considering the next input.

Problem: In either case, original DPDA does not accept any input with w as a prefix. If a DPDA to accept the complement language is defined by simply changing final and

non-final states, it would accept every string with prefix w without going beyond w. Hence, the resulting DPDA still would accept strings with w as prefix without processing the complete string.

2. After processing the sentence x, the DPDA may take several moves on ε-input.

Reason: The DPDA may be in final states after some of these moves and in non-final states after others.

Problem: By interchanging the final and non final states in DPDA for complement language, it would still accept x.

These difficulties can be overcome by forcing

1. The DPDA to enter an ID only after processing the entire input. This takes care of accepting or rejecting the input only after processing the complete input.
2. Modify the DPDA by adding the second component to the state to record whether a final state of the original DPDA is reached or not. This takes care of rejecting sentence x by DPDA constructed for complement language; it is accepted by original DPDA.

Theorem 3: The complement of a DCFL is a DCFL.

Proof: Let $M = (Q, \Sigma, \tau, \delta, q_0, Z_0, F)$ be a DPDA satisfying the first condition.

Let $M' = (Q', \Sigma, \tau, \delta', q'_0, Z_0, F')$ be a DPDA simulating M where

$Q' = \{[q, k] \mid q \in Q \text{ and } k = 1, 2 \text{ or } 3\}$

$F' = \{[q, 3] \mid q \in Q\}$ and

$$q'_0 = \begin{cases} [q_0, 1] & \text{if } q_0 \text{ is in F} \\ [q_0, 2] & \text{if } q_0 \text{ is not in F} \end{cases}$$

The purpose of k in [q, k] is to record, between true inputs, whether it has entered an accepting state.

- $k = 1 \Rightarrow$ M has entered an accepting state since the last true input.
- $k = 2 \Rightarrow$ M has not entered an accepting state since the last true input.
- M′ simulates the move of M and changes k to 1 or 2 when M reads a true input, k = 1, and irrespective of the new state, whether it is final or non-final.
- If k = 2, M′ first changes k to 3 and then simulates the move of M, changing k to 1 or 2, depending on whether the new state of M is or is not in F.
- For q and p in Q and a in Σ, we define δ′ as follows:
  - If $\delta(q, \varepsilon, Z) = (p, \gamma)$, then for k = 1 or 2, $\delta'([q, k], \varepsilon, Z) = ([p, k'], \gamma)$ where k′ = 1 if k = 1 or if p is in F; otherwise k′ = 2.
  - If $\delta(q, a, Z) = (p, \gamma)$, for a in Σ, then $\delta'([q, 2], \varepsilon, Z) = ([p, 3], Z)$ and $\delta'([q, 1], a, Z) = \delta'([q, 3], a, Z) = ([p, k], \gamma)$ where k = 1or 2 for p in F, or not in F, respectively.

We can show that L(M′) is the complement of L(M).

Suppose $a_1, a_2, a_3 \ldots a_n$. is in L(M), then M enters a final state on a_n. State of M′ while simulating this string will be 1 before it is possible for M′ to use a true input after a_n. Therefore, M′ does not accept, while a_n was the last true input used.

Suppose $a_1, a_2, a_3 \ldots a_n$ is not in L(M). M′ will some time after reading a_n, have no ε-moves to make and will have to use a true input symbol. The second component of M′

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

state is 2, since $a_1, a_2, a_3 \ldots a_n$ is not in L(M). M′ will accept before attempting to use a true input symbol.

5.3.3 Decision Properties of DCFLs

Let L be a DCFL and R, a regular set, then the problems that are decidable are

1. $L = R$.
2. $R \subseteq L$.
3. $\bar{L} = \varnothing$.
4. $\bar{L}$ is a CFL.
5. L is regular.

If L and L′ are arbitrary DCFL's, then the following problems are undecidable:

1. $L \cap L' = \varnothing$.
2. $L \subseteq L'$.
3. $L \cap L'$ is DCFL.
4. $L \cup L'$ is DCFL.
5. $L \cap L'$ is CFL.

5.3.4 DPDA and Regular Languages

The class of languages DPDA accepts, is in between regular languages and CFLs. The DPDA language include all regular languages. The two modes of acceptance are not same for DPDA.

To accept with final state

If L is a regular language, L = L(P) for some DPDA P. PDA surely includes a stack, but the DPDA used to simulate a regular language does not use the stack. The stack is inactive always. If A is the FA for accepting the language L, then $\delta_p(q, a, Z) = \{(p, Z)\}$ for all p, $q \in Q$ such that $\delta_A(q, a) = p$.

To accept with empty stack

Not every regular language is N(P) for some DPDA P. A language L = N(P) for some DPDA P, if and only if L has prefix property. Definition of prefix property of L states that if x, $y \in L$, then x should not be a prefix of y, or vice versa. Non-regular language $L = wcw^R$ could be accepted by DPDA with empty stack, because if you take any x, $y \in L(wcw^R)$, x and y satisfy the prefix property. But the language L = {0*} could be accepted by DPDA with final state, but not with empty stack, because strings of this language do not satisfy the prefix property. So, N(P)s are properly included in CFL L, i.e, $N(P) \subseteq L$

5.3.5 DPDA and Ambiguous Grammar

DPDA is most suitable for design of structure of programming language using DPDA because language accepted by DPDA are unambiguous grammar. But all unambiguous

grammars are not accepted by DPDA. For example, $S \rightarrow 0S0|1S1|\varepsilon$ is an unambiguous grammar corresponds to the language of palindromes. This language is accepted by only NPDA. If L = N(P) for DPDA P, then surely L has an unambiguous CFG.

If L = L(P) for DPDA P, then L has unambiguous CFG. Convert L(P) to N(P) to have prefix property by adding an end marker \$ to strings of L. Then convert N(P) to CFG G′. From G′, we have to construct G to accept L by getting rid of \$. So add a new production $\$\rightarrow\varepsilon$ as a production in G.

Definition 3 Non-deterministic PDA (NPDA): The PDA that has more than one choice of moves in any state is called a non-deterministic PDA (NPDA).

Example 5.8 Design a PDA which accepts $L = \{ww^r \mid w \in (a + b)^*\}$.

Read the string w and push it on to the stack. After that, read each symbol. If it matches with top of the stack, pop off the symbol. When input is read completely, if stack becomes empty, it is successful.

The PDA can be given as follows:

Let q_0 be initial state, q_f be final state and Z_0 the bottom of the stack as shown in Figure 5.11.

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa), (q_1, \varepsilon)$$

$$\delta(q_0, b, b) = (q_0, bb), (q_1, \varepsilon)$$

$$\delta(q_0, a, b) = (q_0, ab)$$

$$\delta(q_0, b, a) = (q_0, ba)$$

$$\delta(q_1, a, a) = (q_1, \varepsilon)$$

$$\delta(q_1, b, b) = (q_1, \varepsilon)$$

$$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0)$$

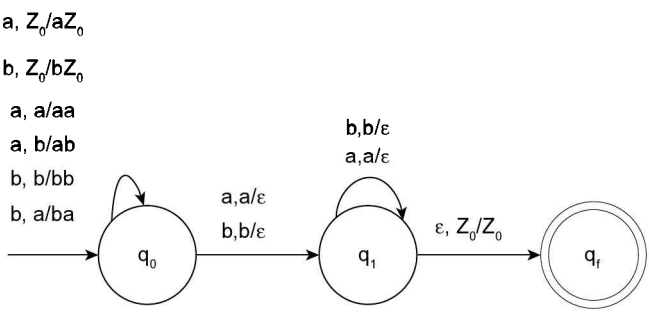


Fig. 5.11 *PDA for Example 5.8*

The final PDA is given by $M = \{(q_0, q_1, q_f), (a, b), (a, b, Z_0), \delta, q_0, Z_0, q_f\}$.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

5.4 Equivalence of PDA's and CFG's

The relationship between PDA's and CFG's is similar to that between DFA's and regular expressions namely the languages accepted by PDA's are just those that can be generated by CFGs.

The proofs of these theorems take a familiar approach: given an arbitrary grammar, we construct the corresponding PDA; and given an arbitrary PDA, we construct the corresponding grammar.

5.4.1 Constructing PDA for given CFG

Theorem 4: Suppose L is a context free language. Then there is a PDA M such that L = N(M).

Proof: The basic idea in the construction is to build M so that it simulates the leftmost derivation of strings using G. The machine we construct uses the terminals and non-terminals of the grammar as stack symbols. What we conceptually want to do is to use the stack to hold the sentential form that evolves during a derivation. At each step, the topmost variable in the stack will get replaced by the RHS of some grammar rule. Of course, there are several problems with implementing this concept. For one, the PDA can only access the top of its stack, it cannot find a variable below the top. For another, even if the PDA could find such a variable, it could not fit the RHS into a single stack slot. But these are not insurmountable. We simply have to arrange things so that the PDA always has the leftmost variable of the sentential form on top of the stack. If that can be set up, the PDA can use the technique of using extra states to push multiple symbols 'all at once'.

The other consideration is that we are constructing a PDA; so it needs to consume the input string and give a verdict. This fits in nicely with our other requirements. In brief, the PDA will use ε – transitions to push the RHS of rules into the stack, and will use 'normal' transitions to consume input. In consuming input, we will be able to remove non-variables from the top of the stack, always guaranteeing that a variable is at the top of the stack.

We assume ε is not in L(G). Let G be given as (V, T, P, S) be a context free grammar in the Greibach normal form. It is required to construct $M = (\{q\}, T, V, \delta, q_1, S, \Phi)$ where $\delta(q, a, A)$ contains (q, γ) whenever $A \rightarrow a\gamma$ is in P.

The PDA M simulates leftmost derivations of G. Since G is in the Griebach normal form, each sentential form in a leftmost derivation consists of terminals x followed by a string of variables α. M stores the suffix α of the left sentential form on its stack after processing the prefix x. Formally we show that

$S \Rightarrow x\alpha$ by a leftmost derivation if and only if $(q, x, S) \overset{*}{\underset{M}{\vdash}} (q, \varepsilon, \alpha)$

Example 5.9 Construct a PDA equivalent to the following grammar:

$$S \rightarrow aAA, A \rightarrow aS \mid bS \mid a$$

Solution: The grammar is in GNF. Hence we can apply the rule as follows:

$S \rightarrow aAA$	corresponds $\delta(q, a, S) = (q, AA)$
$A \rightarrow aS$	corresponds $\delta(q, a, A) = (q, S)$
$A \rightarrow bS$	corresponds $\delta(q, b, A) = (q, S)$
$A \rightarrow a$	corresponds $\delta(q, a, A) = (q, \varepsilon)$

Show that both accept the same language, consider the string 'aaa' accepted by the grammar and the change of state is shown in the Figure 5.12.

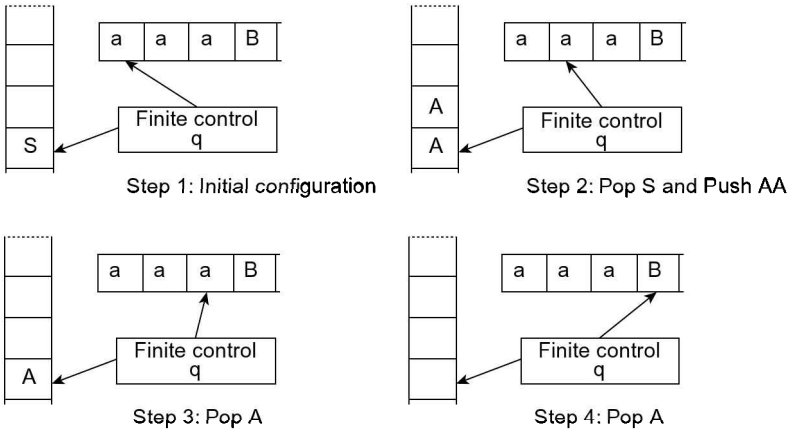


Fig. 5.12 *PDA for Example 5.9*

5.4.2 Constructing CFG for the Given PDA

Theorem 5: Suppose M is a PDA. Then there is a grammar G such that L(G) = L(M), i.e., L(M) is context free.

Proof: The previous construction spelled out in full would look messy, but is in fact quite simple. Going in the reverse direction, i.e, converting a PDA to a CFG, is more difficult. The basic idea is to consider any two states p, q of PDA M and think about what strings could be consumed in executing M from p to q. Those strings will be represented by a variable [p, a, q] in G, the grammar we are building. By design, the strings generated by [p, a, q] would be just those substrings consumed by M in going from p to q. Thus S, the start variable, will stand for all strings consumed in going from q_0 to an accept state. This is clear enough, but as always for PDA's, we must consider the stack, hence, the stack will be more involved; for example, we will use funky variables of the form [p, A, q], where A represents the top of the stack.

The construction goes as follows: given PDA M = {Q, Σ, τ, δ, q_0, Z_0, Ø}, we will construct a grammar G such that L(G) = L(M).

To convert the PDA to CFG, we use the following three rules:

R1: The productions for start symbol S are given by

$$S \rightarrow [q_0, Z_0, q] \text{ for each state } q \text{ in } Q.$$

R2: Each move that pops a symbol from stack with transition as

$$\delta(q, a, Z_i) = (q_1, \varepsilon)$$

induces a production as

$$[q, Z_i, q_1] \rightarrow a \text{ for } q_1 \text{ in } Q.$$

R3: Each move that does not pop symbol from stack with transition as

$$\delta(q, a, Z_0) = (q_1, Z_1Z_2\, Z_3Z_4 \ldots..)$$

induces a production as

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$$[q, Z_0, q_m] \rightarrow a[q_1, Z_1 q_2] [q_2, Z_2 q_3] [q_3, Z_3 q_4] [q_4, Z_4 q_5] \ldots [q_{m-1}, Z_m q_m]$$

for each q_i in Q, where $1 \leq i \leq m$.

After defining all the rules, apply simplification of grammar to get reduced grammar.

Example 5.10 Give the equivalent CFG for the following PDA M = $\{\{q_0, q_1\}, \{a, b\}, \{Z, Z_0\}, \delta, q_0, Z_0, \emptyset\}$ where δ is defined by

$$\delta(q_0, b, Z_0) = (q_0, ZZ_0)$$
$$\delta(q_0, \varepsilon, Z_0) = (q_0, \varepsilon)$$
$$\delta(q_0, b, Z) = (q_0, ZZ)$$
$$\delta(q_0, a, Z) = (q_1, Z)$$
$$\delta(q_1, b, Z) = (q_1, \varepsilon)$$
$$\delta(q_1, a, Z_0) = (q_0, Z_0)$$

Solution: The states are q_0 and q_1, and the stack symbols are Z and Z_0. The states are $\{S, [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1], [q_0, Z, q_0], [q_0, Z, q_1], [q_1, Z, q_0], [q_1, Z, q_1]\}$.

S- Productions are given by Rule 1

$$S \rightarrow [q_0, Z_0, q_0] \mid [q_0, Z_0, q_1]$$

(1) The CFG for $\delta(q_0, b, Z_0) = (q_0, ZZ_0)$ is obtained by rule 3

$[q_0, Z_0, q_0] \rightarrow b\ [q_0, Z, q_0] [q_0, Z_0, q_0]$
$[q_0, Z_0, q_0] \rightarrow b\ [q_0, Z, q_1] [q_1, Z_0, q_0]$
$[q_0, Z_0, q_1] \rightarrow b\ [q_0, Z, q_0] [q_0, Z_0, q_1]$
$[q_0, Z_0, q_1] \rightarrow b\ [q_0, Z, q_1] [q_1, Z_0, q_1]$

(2) The CFG for $\delta(q_0, \varepsilon, Z_0) = (q_0, \varepsilon)$ is obtained by rule 2

$[q_0, Z_0, q_0] \rightarrow \varepsilon$

(3) The CFG for $\delta(q_0, b, Z) = (q_0, ZZ)$ is obtained by rule 3

$[q_0, Z, q_0] \rightarrow b\ [q_0, Z, q_0] [q_0, Z, q_0]$
$[q_0, Z, q_0] \rightarrow b\ [q_0, Z, q_1] [q_1, Z, q_0]$
$[q_0, Z, q_1] \rightarrow b\ [q_0, Z, q_0] [q_0, Z, q_1]$
$[q_0, Z, q_1] \rightarrow b\ [q_0, Z, q_1] [q_1, Z, q_1]$

(4) The CFG for $\delta(q_0, a, Z) = (q_1, Z)$ is obtained by rule 3

$[q_0, Z, q_0] \rightarrow a\ [q_1, Z, q_0]$
$[q_0, Z, q_1] \rightarrow a[q_1, Z, q_1]$

(5) The CFG for $\delta(q_1, b, Z) = (q_1, \varepsilon)$ is obtained by rule 2

$[q_1, Z, q_1] \rightarrow b$

(6) The CFG for $\delta(q_1, a, Z_0) = (q_0, Z_0)$ is obtained by rule 2

$[q_1, Z_0, q_0] \rightarrow a\ [q_0, Z_0, q_0]$
$[q_1, Z_0, q_1] \rightarrow a[q_0, Z_0, q_1]$

Simplifying grammar: In the above grammar, first identify the non-terminals that are not defined and eliminate the productions that refer to these productions. Similarly, use the procedure of eliminating the useless symbols and useless productions. Hence the complete grammar is as follows:

$$S \rightarrow [q_0, Z_0, q_0]$$
$$[q_0, Z_0, q_0] \rightarrow b\,[q_0, Z, q_1]\,[q_1, Z_0, q_0]$$
$$[q_0, Z_0, q_0] \rightarrow \varepsilon$$
$$[q_0, Z, q_1] \rightarrow b\,[q_0, Z, q_1]\,[q_1, Z, q_1]$$
$$[q_0, Z, q_1] \rightarrow a[q_1, Z, q_1]$$
$$[q_1, Z, q_1] \rightarrow b$$
$$[q_1, Z_0, q_0] \rightarrow a\,[q_0, Z_0, q_0]$$

Example 5.11 Construct PDA to accept if–else of a C program and convert it to CFG. (This does not accept nested if statements).

Let the PDA $P = (\{q\}, \{i, e\}, \{X, Z\}, \delta, q, Z, \emptyset)$, where δ is given by

$$\delta(q, i, Z) = \{(q, XZ)\},$$
$$\delta(q, e, X) = \{(q, \varepsilon)\} \text{ and}$$
$$\delta(q, \varepsilon, Z) = \{(q, \varepsilon)\}$$

Solution:

Non-terminals are $\{S, [qXq], [qZq]\}$,

Equivalent productions are

$$S \rightarrow [qZq]$$
$$[qZq] \rightarrow i[qXq][qZq]$$
$$[qXq] \rightarrow e$$
$$[qZq] \rightarrow \varepsilon$$

If [qZq] is renamed to A and [qXq] is renamed to B, then the CFG can be defined by

$G = (\{S, A, B\}, \{i, e\}, P, S)$ where P is

$$S \rightarrow A$$
$$A \rightarrow iBA \mid \varepsilon$$
$$B \rightarrow e$$

Example 5.10 Convert PDA to CFG. PDA is given by $P = \{\{p, q\}, \{0, 1\}, \{X, Z\}, \delta, q, Z, \emptyset\}$.

Transition function δ is defined by

$$\delta(q, 1, Z) = \{(q, XZ)\}$$
$$\delta(q, 1, X) = \{(q, XX)\}$$
$$\delta(q, \varepsilon, X) = \{(q, \varepsilon)\}$$
$$\delta(q, 0, X) = \{(p, X)\}$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$$\delta(p, 1, X) = \{(p, \varepsilon)\}$$

$$\delta(p, 0, Z) = \{(q, Z)\}$$

Solution: Non-terminals are {S, [pXp], [pXq], [qXp], [qXq], [pZp], [pZq], [qZp], [qZq]}.

Add productions for start variable

$S \rightarrow [qZq] \mid [qZp]$

For $\delta(q, 1, Z) = \{(q, XZ)\}$

$$[qZq] \rightarrow 1[qXq][qZq]$$

$$[qZq] \rightarrow 1[qXp][pZq]$$

$$[qZp] \rightarrow 1[qXq][qZp]$$

$$[qZp] \rightarrow 1[qXp][pZp]$$

For $\delta(q, 1, X) = \{(q, XX)\}$

$$[qXq] \rightarrow 1[qXq][qXq]$$

$$[qXq] \rightarrow 1[qXp][pXq]$$

$$[qXp] \rightarrow 1[qXq][qXp]$$

$$[qXp] \rightarrow 1[qXp][pXp]$$

For $\delta(q, \varepsilon, X) = \{(q, \varepsilon)\}$

$$[qXq] \rightarrow \varepsilon$$

For $\delta(q, 0, X) = \{(p, X)\}$

$$[qXq] \rightarrow 0[pXq]$$

$$[qXp] \rightarrow 0[pXp]$$

For $\delta(p, 1, X) = \{(p, \varepsilon)\}$

$$[pXp] \rightarrow 1$$

For $\delta(p, 0, Z) = \{(q, Z)\}$

$$[pZq] \rightarrow 0[qZq]$$

$$[pZp] \rightarrow 0[qZp]$$

Renaming the variables

[qZq] to A, [qZp] to B, [pZq] to C, [pZp] to D, [qXq] to E [qXp] to F, [pXp] to G and [pXq] to H, the equivalent CFG can be defined by

$$G = (\{S, A, B, C, D, E, F, G, H\}, \{0, 1\}, R, S).$$

The productions of R also are to be renamed accordingly:

$$S \rightarrow A \mid B$$

$$A \rightarrow 1EA \mid 1FC$$

$$B \rightarrow 1EB \mid 1FD$$

$$E \rightarrow 1EE \mid 1FH$$

$$F \rightarrow 1EF \mid 1FG$$

$$E \rightarrow \varepsilon \mid 0H$$

$$F \rightarrow 0G$$

$$G \rightarrow 1$$

$$C \rightarrow 0A$$

$$D \rightarrow 0$$

On simplifying the above grammar, we get

$$S \rightarrow B$$

$$B \rightarrow 1EB \mid 1FD$$

$$E \rightarrow 1EE$$

$$F \rightarrow 1EF \mid 1FG$$

$$E \rightarrow \varepsilon$$

$$F \rightarrow 0G$$

$$G \rightarrow 1$$

$$D \rightarrow 0$$

Example 5.11 Give the equivalent CFG for the following PDA M = $\{\{q_0, q_1\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \emptyset\}$ where δ is defined by

$$\delta(q_0, \varepsilon, Z_0) = (q_1, \varepsilon)$$

$$\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 1, 0) = (q_0, 10)$$

$$\delta(q_0, 1, 1) = (q_0, 11)$$

$$\delta(q_0, 0, 1) = (q_1, \varepsilon)$$

$$\delta(q_1, 0, 1) = (q_1, \varepsilon)$$

$$\delta(q_1, 0, 0) = (q_1, \varepsilon)$$

$$\delta(q_1, \varepsilon, Z_0) = (q_1, \varepsilon)$$

Solution: The states are q_0 and q_1; and the stack symbols are Z and Z_0. The states are

$\{S, [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1], [q_0, 0, q_0], [q_0, 0, q_1], [q_1, 0, q_0], [q_1, 0, q_1], [q_0, 1, q_0], [q_0, 1, q_1], [q_1, 1, q_0], [q_1, 1, q_1]\}$

S- productions are given by rule 1:

$$S \rightarrow [q_0Z_0q_0] \mid [q_0Z_0q_1]$$

(1) The CFG for $\delta(q_0, \varepsilon, Z_0) = (q_1, \varepsilon)$ is

$[q_0Z_0q_1] \rightarrow \varepsilon$

(2) The CFG for $\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$ is

$[q_0Z_0q_0] \rightarrow 0[q_0 0q_0][q_0Z_0q_0] \mid 0[q_0 0q_1][q_1Z_0q_0]$

$[q_0Z_0q_1] \rightarrow 0[q_0 0q_0][q_0Z_0q_1] \mid 0[q_0 0q_1]\,[q_1Z_0q_1]$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

(3) The CFG for $\delta(q_0, 0, 0) = (q_0, 00)$ is

$[q_0 0 q_0] \rightarrow 0\ [q_0 0 q_0][q_0 0 q_0] \mid 0[q_0 0 q_1][q_1 0 q_0]$
$[q_0 0 q_1] \rightarrow 0\ [q_0 0 q_0][q_0 0 q_1] \mid 0[q_0 0 q_1]\ [q_1 0 q_1]$

(4) The CFG for $\delta(q_0, 1, 0) = (q_0, 10)$ is

$[q_0 1 q_0] \rightarrow 1[q_0 1 q_0][q_0 0 q_0] \mid 1[q_0 1 q_1][q_1 0 q_0]$
$[q_0 1 q_1] \rightarrow 1[q_0 1 q_0][q_0 0 q_1] \mid 1[q_0 1 q_1][q_1 0 q_1]$

(5) The CFG for $\delta(q_0, 1, 1) = (q_0, 11)$ is

$[q_0 1 q_0] \rightarrow 1[q_0 1 q_0][q_0 1 q_0] \mid 1[q_0 1 q_1][q_1 1 q_0]$
$[q_0 1 q_1] \rightarrow 1[q_0 1 q_0][q_0 1 q_1] \mid 1[q_0 1 q_1][q_1 1 q_1]$

(6) The CFG for $\delta(q_0, 0, 1) = (q_1, \varepsilon)$ is

$[q_0 1 q_1] \rightarrow 0$

(7) The CFG for $\delta(q_1, 0, 1) = (q_1, \varepsilon)$ is

$[q_1 1 q_1] \rightarrow 0$

(8) The CFG for $\delta(q_1, 0, 0) = (q_1, \varepsilon)$ is

$[q_1 0\ q_1] \rightarrow 0$

(9) The CFG for $\delta(q_1, \varepsilon, Z_0) = (q_1, \varepsilon)$ is

$[q_1 Z_0 q_1] \rightarrow \varepsilon$

5.5 Two-stack PDA

Some languages that are not CFL can be accepted by a two-stack PDA.

Definition 4: Let us define a two-stack pushdown automaton to be a six-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$
Q – is a finite set of states,
Σ – is an input alphabet,
Γ – is a stack of symbols,
$q_0 \in Q$ is the initial state,
$F \subseteq Q$ is the set of final states, and
δ, the transition relation, is a finite subset of $(Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma^* \times \Gamma^*) \Rightarrow (Q \times \Gamma^* \times \Gamma^*)$,

where in the left-hand side, the third parameter indicates the first stack symbol, the fourth parameter indicates the second stack symbol. On the right-hand side, the second parameter indicates the operation on the first stack, and the third parameter indicates the operation on the second stack.

Example: Design a two-stack PDA for the language $L = \{a^n b^n c^n \mid n \in N\}$.

Solution: This problem is not solvable with a normal PDA, but can be solved with a two-stack PDA.

The PDA M is defined by $M = \{\{q_0, q_1, q_2, q_3\}, \{a, b, c\}, \{a, b\}, \delta, q_0, q_3\}$
where δ is defined by

$$\delta(q_0, a, \varepsilon, \varepsilon) = (q_0, a, \varepsilon)$$
$$\delta(q_0, a, a, \varepsilon) = (q_0, aa, \varepsilon)$$

$$\delta(q_0, \varepsilon, \varepsilon, \varepsilon) = (q_1, \varepsilon, \varepsilon)$$
$$\delta(q_1, b, a, \varepsilon) = (q_1, a, b)$$
$$\delta(q_1, b, a, b) = (q_1, a, bb)$$
$$\delta(q_1, c, a, b) = (q_2, \varepsilon, \varepsilon)$$
$$\delta(q_2, c, a, b) = (q_2, \varepsilon, \varepsilon)$$
$$\delta(q_2, \varepsilon, \varepsilon, \varepsilon) = (q_3, \varepsilon, \varepsilon)$$

Example 5.10 Design a two-stack PDA for the language $L = \{a^n b^n a^n b^n | \ n \in N\}$.

Solution: This is another problem that is not solvable with a normal PDA, but solvable by a two-stack PDA.

The PDA M is defined by $M = \{\{q_0, q_1, q_2, q_3, q_4\}, \{a, b, c\}, \{a, b\}, \delta, q_0, q_4\}$ where δ is defined by

$$\delta(q_0, a, \varepsilon, \varepsilon) = (q_0, a, \varepsilon)$$
$$\delta(q_0, a, a, \varepsilon) = (q_0, aa, \varepsilon)$$
$$\delta(q_0, \varepsilon, \varepsilon, \varepsilon) = (q_1, \varepsilon, \varepsilon)$$
$$\delta(q_1, b, a, \varepsilon) = (q_1, a, b)$$
$$\delta(q_1, b, a, b) = (q_1, a, bb)$$
$$\delta(q_1, a, a, b) = (q_2, \varepsilon, b)$$
$$\delta(q_2, a, a, b) = (q_2, \varepsilon, b)$$
$$\delta(q_2, b, \varepsilon, b) = (q_3, \varepsilon, \varepsilon)$$
$$\delta(q_3, b, \varepsilon, b) = (q_3, \varepsilon, \varepsilon)$$
$$\delta(q_3, \varepsilon, \varepsilon, \varepsilon) = (q_4, \varepsilon, \varepsilon)$$

5.6 Applications of PDA

PDAs can be used in compilation process in checking for the syntax of the input program. It is also used in converting the given infix expression to postfix expression, or in evaluation of the input expression.

5.6.1 PDA as a Parser

During compilation it is required to check the input validation according to the syntax of the language. The syntax for writing arithmetic statements is given below:

$$S \rightarrow i\ A\ E\ T$$
$$E \rightarrow i\ O\ E \mid i$$
$$O \rightarrow + \mid - \mid * \mid /$$
$$A \rightarrow =$$
$$T \rightarrow ;$$

Using these rules, we can frame a statement in the source program as

$$i = i + i;$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

This is validated by PDA defined as follows:

S→ i A E T	corresponds	δ(q, i, S) = (q, AET)
E→ i O E	corresponds	δ(q, i, E) = (q, OE)
E→ i	corresponds	δ(q, i, E) = (q, ε)
O→ + | − | * | /	corresponds	δ(q, +, O) = (q, ε)
		δ(q, −, O) = (q, ε)
		δ(q, *, O) = (q, ε)
		δ(q, /, O) = (q, ε)
A→ =	corresponds	δ(q, =, A) = (q, ε)
T→ ;	corresponds	δ(q, ;, T) = (q, ε)

$$(q, i = i + i_j;, S) \Rightarrow (q, = i + i;, AET) \Rightarrow (q, i + i;, ET) \Rightarrow$$
$$(q, + i;, OE) \Rightarrow (q, i;, E) \Rightarrow (q, \varepsilon, \varepsilon)$$

5.6.2 Top-down Parser Using the PDA

The goal of a top-down parser is to produce a leftmost derivation starting with the start symbol. It processes the entire input and states whether it is valid or invalid. This task is accomplished using the PDA, where the stack is used for validating input.

The PDA is initialized with the configuration where the stack is initialized with symbol S (starting non-terminal), and the reading head points the leftmost symbol in the input. Given the current input symbol and the current non-terminal symbol, the parser refers to the parsing table and if there is an entry corresponding to it, it replaces the top symbol with the right-hand side of the production in reverse. If the top of the stack is a terminal symbol and matches the input, then it just pops that symbol and continue with the rest of the input string. A simplified process is given in the following algorithm.

Algorithm:

set ip to point to the first symbol of w$.
  repeat
    let X be the top stack symbol and a the symbol pointed to by ip.
    if X is a terminal of $ then
      if X = a then
        pop X from the stack and advance ip
      else error()
    else
      if M[X, a] = $X \rightarrow Y_1 Y_2 Y_3 \ldots Y_k$ then begin
        pop X from the stack;
        push $Y_k, Y_{k-1} \ldots Y_1$ onto the stack, with Y1 on top;
        output the production $X \rightarrow Y_1 Y_2 \ldots Y_k$
      end
      else error()
  until X = $

The processing of the arithmetic statement is shown below. Let the grammar to generate the arithmetic expression be represented as

$$E \rightarrow TA$$
$$A \rightarrow + TA \mid \varepsilon$$

$$T \rightarrow FB$$

$$B \rightarrow *FB \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id}$$

This grammar is processed and a parsing table is constructed shown below.

Non-terminal	(	id	+	*	)	$
E	T A	T A				
A			+ T A		ε	ε
T	F B	F B				
B			ε	*F B	ε	ε
F	**(*E*)**	**id**				

To parse the expression (id + id).

Note the end marker is pushed as initial stack symbol onto the stack. Then when the expression has been processed, the end marker '$' of the input string is matched against the initial stack symbol and the string is accepted.

Input	Stack content	Production used
(id + id)$	E$	
(id + id)$	TA$	E→ TA
(id + id)$	FBA$	T→ FB
(id + id)$	(E)BA$	F→ (E)
(id + id)$	(E)BA$	T→ FB
id + id)$	E)BA $	Pop (and read next input
id + id)$	TA)BA $	E→ TA
id + id)$	FBA)BA $	T→ FB
id + id)$	idBA)BA $	F → id
+ id)$	BA)BA $	Pop id and read next input
+ id)$	BA)BA $	B → ε, pop B
+ id)$	A)BA $	
+ id)$	+ TA)BA $	A→ + TA
id)$	TA)BA $	Pop +, read next symbol
id)$	FBA)BA $	T → FB
id)$	idBA)BA $	F → id
)$	BA)BA $	Pop id and read next input
)$	BA)BA $	B → ε, pop B
)$	A)BA $	A → ε, pop A
)$	)BA $	pop), read next input
$	BA $	B → ε, pop B
$	A $	A → ε, pop A
$	$	Accepted

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

The previous table shows the contents of stack and of the tape of PDA while it is processing the input.

Solved Problems

Problem 1: Design a PDA which accepts the set of balanced parenthesis.

({{( )}})

Solution: Read all open braces/parentheses onto the stack. Then whenever a closed parentheses is seen, match it by popping off:

$$\delta(q_0, (, Z_0) = (q_0, (Z_0)$$

$$\delta(q_0, \{, Z_0) = (q_0, \{Z_0)$$

$$\delta(q_0, (, (\ ) = (q_0, (( )$$

$$\delta(q_0, \{, \{\ ) = (q_0, \{\{)$$

$$\delta(q_0, (, \ \{) = (q_0, (\{)$$

$$\delta(q_0, \{, (\ ) = (q_0, \{( )$$

$$\delta(q_0,), (\ ) = (q_0, \varepsilon)$$

$$\delta(q_0, \}, \{\ ) = (q_0, \varepsilon)$$

$$\delta(q_0, \varepsilon, Z_0) = (q_f, Z_0)$$

Problem 2: Design a PDA which accepts $L = \{a^p b^q c^m \mid p + m = q\}$.

Solution: Read all a's and push onto the stack. Then read each 'b', match it with 'a' by popping off.

Once all a's are exhausted from the stack, read the remaining b's and store them on the stack. Read each 'c' and match it with b's on stack, by popping off.

$$\delta(q_0, a, Z_0) = (q_0, a, Z_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, \varepsilon)$$

$$\delta(q_1, b, a) = (q_1, \varepsilon)$$

$$\delta(q_1, b, Z_0) = (q_1, bZ_0)$$

$$\delta(q_0, b, Z_0) = (q_1, bZ_0) - \text{if input starts with b's like } b^m c^m$$

$$\delta(q_1, b, b) = (q_1, bb)$$

$$\delta(q_1, c, a) = (q_3, \varepsilon)$$

$$\delta(q_3, c, a) = (q_3, \varepsilon)$$

$$\delta(q_3, \varepsilon, z_0) = (q_f, z_0)$$

$$\delta(q_1, c, b) = (q_2, \varepsilon)$$

$$\delta(q_2, c, b) = (q_2, \varepsilon)$$

$$\delta(q_2, \varepsilon, Z_0) = (q_f, Z_0)$$

$$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0) - \text{if input is } a^m b^m$$

Problem 3: Design a PDA which accepts $L = \{0^n 1^m 0^n \mid m, n \geq 1\}$.

Solution: Read all initial 0's and push them onto the stack. Read all 1's; ignore them. Then read next '0' and match it top of satck with '0' and pop it.

$$\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 1, 0) = (q_1, 0)$$

$$\delta(q_1, 1,0) = (q_1, 0)$$

$$\delta(q_1, 0,0) = (q_2, \varepsilon)$$

$$\delta(q_2, 0, 0) = (q_2, \varepsilon)$$

$$\delta(q_2, \varepsilon, Z_0) = (q_f, Z_0)$$

Problem 4: To accept the language $L = \{a^i b^j c^k \; i + j = k; \; i \geq 0, j \geq 0\}$.

Solution: PDA $M = (\{q_0, q_1, q_2, q_3, q_f\}, \{a, b, c\}, \{a, b, Z_0\}, \delta, q_0, Z_0, \{q_f\})$, where δ is defined by the following rules:

$$\delta(q_0, a, Z_0) = \{(q_0, aZ_0)\},$$

$$\delta(q_0, a, a) = \{(q_0, aa)\}$$

$$\delta(q_0, b, a) = \{(q_1, ba)\},$$

$$\delta(q_0, b, Z_0) = \{(q_1, bZ_0)\}$$

$$\delta(q_0, c, a) = \{(q_3, \varepsilon)\},$$

$$\delta(q_1, b, b) = \{(q_1, bb)\}$$

$$\delta(q_1, c, b) = \{(q_2, \varepsilon)\},$$

$$\delta(q_2, c, b) = \{(q_2, \varepsilon)\}$$

$$\delta(q_2, c, a) = \{(q_3, \varepsilon)\}$$

$$\delta(q_2, \varepsilon, z_0) = \{(q_f, \varepsilon)\}$$

$$\delta(q_3, c, a) = \{(q_3, \varepsilon)\},$$

$$\delta(q_3, \varepsilon, Z_0) = \{(q_f, \varepsilon)\}$$

$$\delta(q_0, \varepsilon, Z_0) = \{(q_f, \varepsilon)\},$$

$$\delta(q, x, Y) = \phi \text{ for all other possibilities}$$

Problem 5: To accept the language $L = \{a^n b^{2n} \mid a, b \in \Sigma, n \geq 1\}$, by final state.

Solution: PDA $M = (\{q_0, q_1, q_2, q_f\}, \{a, b\}, \{a, Z_0\}, \delta, q_0, Z_0, \{q_f\})$, where δ is defined by the following rules:

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$$\delta(q_0, a, Z_0) = \{(q_0, aZ_0)\},$$
$$\delta(q_0, a, a) = \{(q_0, aa)\}$$
$$\delta(q_0, b, a) = \{(q_1, a)\},$$
$$\delta(q_1, b, a) = \{(q_2, \varepsilon)\}$$
$$\delta(q_2, b, a) = (q_1, a)$$
$$\delta(q_2, \varepsilon, Z_0) = \{(q_f, Z_0)\}$$

Problem 6: Convert CFG to PDA.

$$I \rightarrow a \mid b, S \rightarrow aA, A \rightarrow aABC \mid bB \mid a, B \rightarrow b, C \rightarrow c$$

Solution: PDA accepted by empty stack is given by M = ({q}, {a, b, c}, {a, b, c, S, A, B, C, I}, {δ, q, S, ϕ}) where transition function δ is given by

$$\delta(q, a, I) = \{(q, \varepsilon)\}$$
$$\delta(q, b, I) = \{(q, \varepsilon)\}$$
$$\delta(q, a, S) = \{(q, A)\}$$
$$\delta(q, a, A) = \{(q, ABC), (q, \varepsilon)\}$$
$$\delta(q, b, A) = \{(q, B)\}$$
$$\delta(q, b, B) = \{(q, \varepsilon)\}$$
$$\delta(q, c, C) = \{(q, \varepsilon)\}$$

Problem 7: Convert CFG to PDA.

$$S \rightarrow aBB,\ B \rightarrow bS \mid c$$

Solution: PDA accepted by empty stack M = ({q}, {a, b, c}, {a, b, c, S, B}, δ, q, s, ϕ), where transition function δ is given below:

$$\delta(q, a, S) = \{(q, BB)\}$$
$$\delta(q, b, B) = \{(q, S)\}$$
$$\delta(q, c, B) = \{(q, \varepsilon)\}$$

Problem 8: Convert the following CFG to a PDA:

$$S \rightarrow aAA$$
$$A \rightarrow aS \mid bS \mid a$$

Solution: Let M = (Q, Σ, τ, δ, q_0, Z_0, F) be a PDA where

$$Q = \{q\}, \Sigma = \{a, b\}, \tau = \{a, b, S, A\}$$
$$q_0 = q\ F = \{\}, Z_0 = S$$

and the transition function δ is defined by

$$\delta(q, a, S) = \{(q, AA)\}$$

$$\delta(q, a, A) = \{(q, S), (q, \varepsilon)\}$$

$$\delta(q, b, A) = \{(q, S)\}$$

Summary

1. PDA is used to recognize CFLs.
2. PDA is a FA with an auxiliary memory, that is, Stack.
3. Stack allows PDA to recognize some non-regular languages.
4. PDA may be deterministic or non-deterministic.
5. NPDA and DPDA are not equivalent in power unlike NFA and DFA.
6. There are two ways to check the language accepted by a PDA: acceptance by empty stack and acceptance by reaching the final state.
7. For every context free language, there exists a PDA that accepts the same language.
8. DPDA's accept the language that is a subset of CFG's, and RL's are a subset of DCFL's.
9. Not every NPDA has a deterministic DPDA.
10. For every PDA that accepts using empty stack, there is an equivalent PDA that accepts using final-state method.

Short Answers

1. Define pushdown automaton

   Answer: A finite state **pushdown automaton** (abbreviated PDA) is a 7-tuple $M = (Q, \Sigma, \delta, \tau, q_0, Z_0 F,)$ where

   Q: a finite set of states

   Σ: a finite set of input alphabet

   Γ: a finite set of stack alphabet

   q_0: the start state. $q_0 \in Q$

   $F \subseteq Q$: a set of final state, and

   δ: a transition function

   $Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma) \rightarrow Q \times \Gamma^*$

   Z_0 is the **initial stack symbol**. $Z_0 \in \tau$.

2. Compare NFA and PDA.

   Answer:

NFA	PDA
The language accepted by NFA is the regular language	The language accepted by PDA is a context free language
NFA has no memory	PDA is essentially a NFA with a stack (memory)
It can store only a limited amount of information	Amount of information it can store is unlimited
A language/string is accepted only by reaching the final state	It accepts a language either by an empty stack or by reaching a final state

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

3. Specify the two types of moves in PDA.

   Answer:

   1. $\delta(q, a, Z_0) = (q, aZ_0)$ indicates that in the state q on seeing 'a', 'a' is pushed onto the stack. There is no change of state.
   2. $\delta(q, a, Z_0) = (q, \varepsilon)$ indicates that in the state q on seeing 'a', the current top symbol Z_0 is deleted from the stack.
   3. $\delta(q, a, Z_0) = (q_1, aZ_0)$ indicates that 'a' is pushed onto the stack and the state is changed to q_1.

4. What are the different types of languages accepted by a PDA and define them.

   Answer: A language can be accepted by a PDA using two approaches:

   1. Acceptance by final state: The PDA accepts its input by reaching to final state an reading the input string
   2. Acceptance by empty stack: On reading the input string from initial configuration for some PDA, the stack of PDA becomes empty.
      For a PDA $M = (Q, \Sigma, \delta, \tau, q_0, Z_0 F,)$ we define the the language accepted by final state L(M) as follows: {w | (q0, w, Z0) |--- (p, ε, Z0) for some p in F and in Q*}. Language accepted by empty / null stack N(M) is {w | (q0, w, Z0) |----(p, ε, ε) for some p in Q}.

5. Are the languages accepted by a PDA by empty stack and by final states are different languages.

   Answer: No, because the languages accepted by PDA's by final state are exactly the languages accepted by PDA's by empty stack.

6. Define deterministic PDAs.

   Answer: A PDA $P = (Q, \Sigma, \tau, \delta, q_0, Z_0, F)$ is deterministic if and only if

   1. $\delta(q, a, X)$ has at most one member for $q \in Q$, $a \in (\Sigma \cup \{\varepsilon\})$, and $X \in \tau$.
      If $\delta(q, a, X)$ is not empty for some $a \in \Sigma$ then $\delta(q, \in, X)$ must be empty

   Example: The PDA accepting $\{wcw^R \mid w \text{ in } (0 + 1)^*\}$ is deterministic.

7. What is the significance of a PDA?

   Answer: Finite automaton is used to model regular expressions and cannot be used to represent non-regular languages. Thus, to model a context free language, a pushdown automaton is needed.

8. When is a string accepted by a PDA?

   Answer: The input string is accepted by the PDA if, on reading the string
   (1) The final state is reached; (2) The stack is empty.

9. Give examples of languages handled by PDAs.

   Answer: (1) $L = \{a^n b^n \mid n >= 0\}$. Here, n is unbounded. Hence, counting cannot be done by finite memory. So, we require a PDA, a machine that can count without limit.

   (2) $L = \{ww^R \mid w \in \{a, b\}^*\}$. To handle this language, we need unlimited counting capability.

10. Are NPDA (non-deterministic PDA) and DPDA (deterministic PDA) equivalent?

    Answer: The languages accepted by NPDA and DPDA are not equivalent. For example, ww^R is accepted by NPDA and not by any DPDA.

11. State the equivalence of acceptance by final state and empty stack.

Answer: If L is $L(M_1)$ for some PDA M_1 that accepts using final state, then L is $L(M_2)$ for some PDA M_2 that accepts using empty stack. Here $L(M_1)$ = language accepted by PDA M_1 by reaching a final state. $L(M_2)$ = language accepted by PDA M_2 by empty stack.

12. State the equivalence of PDA and CFL.

Answer: If L is a context free language, then there exists a PDA M such that L = N(M). If L is N(M) for some PDA, then L is a context free language.

13. Compare NPDA and DPDA.

Answer:

DPDA	NPDA
The PDA that has at most one choice of move in any state is called a deterministic PDA	NPDA provides non-determinism to PDA
Deterministic PDA's (DPDAs) are very useful in programming languages. For example, parsers are deterministic.	DCFLs are recognized by NPDA. Syntax of most of the programming languages is described by DCFLs.
Strings such as $\{wcw^R \mid w \text{ is in } (a + b)^*\}$ can be recognized by DPDA	Strings such as $\{ww^R \mid w \text{ is in } (a + b)^*\}$ cannot be recognized by DPDAs.

14. What are the components of PDA ?

Answer: The PDA usually consists of four components:
A control unit, a read unit, an input tape and a memory unit.

15. What is the informal definition of a PDA?

Answer: A PDA is a computational machine to recognize a context free language. Computational power of a PDA is between that of a finite automaton and a Turing machine. The PDA has a finite control, and the memory is organized as a stack.

Fill in the Blanks

1. For every NPDA, there is an equivalent DPDA accepting the same language. (True | False).
2. For the language $L = \{ww \mid w \in (a + b)^*\}$, there is a_____ PDA which accepts the language.
3. We can construct a PDA for the language $L_1 \cap L_2$ where $L_1 = \{a^i b^j c^k \mid i = j\}$ and $L_2 = \{a^i b^j c^k \mid j = k\}$ (True/False).
4. For a language that contains a set of all strings with even palindrome, we can construct _____PDA.
5. For every regular language, we can construct _____ to accept the language.
6. If PDA accepts the language using empty stack method, then we do not have _____ state.
7. For every PDA, we can construct a context free grammar that accepts the _____ language.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

8. In stack, elements can be removed only from_____.
9. Stack is also called _____.
10. In PDA, stack provides _____memory.

Answers

1.	False	2.	Non-deterministic	3.	False	4.	Non-deterministic	5.	PDA
6.	Final	7.	Context free	8.	Top	9.	LIFO	10.	Unlimited

Objective Question Bank

1. Let $Q = \{\{q_1, q_2\}, \{a, b\}, \{a, b, z\}, \delta, q_1, z, \Phi\}$ be a PDA accepting by empty stack, for the language which is the set of all non-empty even palindromes over the set {a, b}. Given below is an incomplete specification of the transition function δ. The top of stack is assumed to be at the right end of the string representing the stack contents and
   1. $\delta(q_1, a, z) = \{(q_1, za)\}$
   2. $\delta(q_1, b, z) = \{(q_1, zb)\}$
   3. $\delta(q_1, a, a) = \{\mathbf{1}, \mathbf{2}\}$
   4. $\delta(q_1, b, b) = \{\mathbf{3}, \mathbf{4}\}$
   5. $\delta(q_2, a, a) = \{(q_2, \varepsilon)\}$
   6. $\delta(q_2, b, b) = \{(q_2, \varepsilon)\}$
   7. $\delta(q_1, \varepsilon, z) = \{(q_2, \varepsilon)\}$

   Now boxes 1, 2, 3, and 4 are.

   a) $\delta(q_1, bb), \delta(q_2, \varepsilon), \delta(q_1, aa), \delta(q_2, \varepsilon)$
   b) $\delta(q_1, aa), \delta(q_2, \varepsilon), \delta(q_1, bb), \delta(q_2, \varepsilon)$
   c) $\delta(q_1, aa), \delta(q_2, bb), \delta(q_1, bb), \delta(q_2, aa)$
   d) none of the above

2. Which of the following languages over {a, b, c} is accepted by a DPDA?
   a) $\{wcw^R / w \in \{a, b\}^*\}$ b) $\{ww^R / w \in \{a, b, c\}^*\}$
   c) $\{a^n b^n c^n / n \geq 0\}$ d) {w / w is a palindrome over {a, b, c}}

3. What is the language accepted by the following PDA:

   $\delta(q_0, 1, z_0) = \{(q_0, Xz_0)\}$
   $\delta(q_0, 1, X) = \{(q_0, XX)\}$
   $\delta(q_0, 0, X) = \{(q_1, X)\}$
   $\delta(q_1, 1, X) = \{(q_1, \varepsilon)\}$
   $\delta(q_1, \varepsilon, z_0) = \{(q_f, z_0)\}$

   a) $\{0^n 10^n 1 / n > 1)$ b) $\{1^n 01^n / n > 1\}$
   c) $\{1^n 01^n 0 / n > 1\}$ d) $\{1^n 01^n 01^n / n > 1\}$

4. Let L_D be the set of all languages accepted by a PDA by final state and L_E, the set of all languages accepted by empty stack. Which of the following is true?
   a) $L_D = L_E$ b) $L_D \subset L_E$ c) $L_E \subset L_D$ d) None of the above

5. Which of the following statements is true?
   a) If a language is context free, it can always be accepted by a deterministic PDA
   b) The Union of two CFLs is context free
   c) The intersection of two CFLs is context free
   d) The complement of a two CFLs is context free

6. The language accepted by a PDA in which the stack is limited to 10 items is best described as
   a) Context free
   b) regular
   c) deterministic context free
   d) recursive

7. Which of the following CFGs cannot be simulated by a FSM?
   a) $S \rightarrow Sa / a$
   b) $S \rightarrow abX\ X \rightarrow cY \quad Y \rightarrow d / aX$
   c) $S \rightarrow aSb / ab$
   d) None of the above

8. A PDM behaves like a FSM when the number of auxiliary memory it has is
   a) 0 b) 1 c) 2 d) None of the above

9. Choose the correct statements
   a) The power of DFSM and NDFSM are same
   b) The power of DFSM and NDFSM are different
   c) The power of DPDM and NDPDM are different
   d) The power of DPDPM and NDPDM are same

10. Which of the following is accepted by a NDPDM, but not by a DPDM?
    a) All strings in which a given symbol is present at least twice
    b) Even palindromes (i.e. palindromes made up of even number of symbols)
    c) Strings ending with a particular alphabet
    d) None of the above

11. Consider the PDA $\{(q_0, q_1, q_2, q_3, q_f), (a, b), (Z, A, B), \delta, q_0, Z, \{q_f\}\}$.
    1. $\delta(q_0, b, z_0) = (q_f, z_0)$
    2. $\delta(q_0, a, z_0) = (q_0, Az_0)$
    3. $\delta(q_0, a, A) = (q_0, AA)$
    4. $\delta(q_0, b, A) = (q_1, A)$
    5. $\delta(q_1, b, A) = (q_2, \varepsilon)$
    6. $\delta(q_2, b, z_0) = (q_f, z_0)$
    6. $\delta(q_2, \varepsilon, z_0) = (q_f, z_0)$

    The language accepted by this PDA is
    a) $\{a^n b^{2n} / n \geq 0\}$
    b) $\{a^n b^{2n+1} / n \geq 0\}$
    c) $\{a^{n+1} b^{2n} / n \geq 0\}$
    d) $\{a^{n+1} b^{2n+1}, n \geq 0\}$

12. Choose the correct statement.
    a) For any CFL L, there exist a DPDA M such that L = L(M)
    b) For any CFL L, there exist a NPDA M such that L = L(M)
    c) For any non-regular language L, there exists a NPDA M such that L = L(M)
    d) For any regular language L, there exist a DPDA M such that. L = L(M)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

13. Consider the following two grammars:

G_1: $S \rightarrow abAB / ba$
$A \rightarrow aaa$
$B \rightarrow aA / bb$

G_2: $S \rightarrow abAaA / abAbb / ba$,
$A \rightarrow aaa$.

Choose the correct statement:

a) The language generated by G_1 is a subset of G_2
b) The language generated by G_2 is a subset of G_1
c) There is no relation between G_1 is and G_2
d) None of the above

14. Which of the following machines accepts regular as well as non-regular languages?

(a) NFA (b) DFA
(c) PDA (d) all

15. Which of the following statements is true?

a) PDA accepts all regular, all context free and some non-regular languages.
b) PDA accepts some regular, all context free and some non-regular languages.
c) PDA accepts all regular languages, all languages that are not context free and some non-regular languages.
d) PDA accepts all regular, all context free and all non-regular languages.

16. The stack in PDA has

a) unlimited memory b) read only
c) limited memory d) has no memory

17. In PDA, in the stack, elements can be added

a) on the top b) from bottom
c) in the middle d) anywhere

18. Which of the following accepts only regular languages:

a) NFA b) DFA
c) PDA d) All

19. Which of the following is true?

a) NPDA is more powerful than DPDA
b) TM is more powerful than PDA
c) both a) and b)
d) none

Answers

1.	b	2.	a	3.	b	4.	a	5.	b	6.	b	7.	c	8.	a
9.	a, c	10.	b	11.	b	12.	d	13.	d	14.	c	15.	a	16.	a
17.	a	18.	d	19.	c										

Exercises

1. Convert (to PDA) the following CFG whose productions are
   a) $A \rightarrow aAA, A \rightarrow aS \mid bS \mid a$
   b) $S \rightarrow SS \mid (S) \mid \varepsilon$
   c) $S \rightarrow aAS \mid bAB \mid aB, A \rightarrow bBB \mid aS \mid a, B \rightarrow bA \mid a$
2. Convert (to CFG) the following PDA whose transition functionis given by
$$\delta(q, 0, Z) = \{(q, XZ)\}$$
$$\delta(q, 0, X) = \{(q, XX)\}$$
$$\delta(q, 1, X) = \{(q, X)\}$$
$$\delta(q, \varepsilon, X) = \{(p, \varepsilon)\}$$
$$\delta(p, 1, X) = \{(p, XX)\}$$
$$\delta(p, \varepsilon, X) = \{(p, \varepsilon)\}$$
$$\delta(p, 1, Z) = \{(p, \varepsilon)\}$$
3. Give the equivalent CFG for the following PDA:
$$\delta(q_0, 0, Z_0) = (q_0, X\, Z_0)$$
$$\delta(q_0, 0, X) = (q_0, XX)$$
$$\delta(q_0, 1, X) = (q_1, \varepsilon)$$
$$\delta(q_1, 1, X) = (q_1, \varepsilon)$$
$$\delta(q_1, \varepsilon, X) = (q_1, \varepsilon)$$
$$\delta(q_1, \varepsilon, Z_0) = (q_1, \varepsilon)$$
   Hint: Language accepted is $\{a^n b^m \mid n > m \text{ and } n \geq 1\}$.
4. Construct a DPDA for accepting strings in which the number of a's is more than number of b's.
5. Construct a DPDA for accepting $\{0^n 1^m \mid n \geq m\}$.
6. Give the formal definition of a PDA. Discuss the languages accepted by a PDA. Design a NPDA for the language $L = \{a^n b^{2n} \mid n \geq 1\}$.
7. Construct a NPDA that accepts the language accepted by the following grammar:
   $S \rightarrow 0S1/A \qquad A \rightarrow 1AO/S/\in$
8. With a neat diagram show the working of a PDA and define the language accepted by a PDA.
9. Obtain a PDA equivalent to the following grammar:
   $S \rightarrow aA \qquad A \rightarrow aA \mid bA \mid a \mid b$
   a) Explain the following terms:
   b) Languages.
   c) Instantaneous description of a PDA.
10. Design the PDA for the language $L = \{W \mid W \in (a + b)^* \; n_a(w) > n_b(w)\}$.
11. Convert the following grammar to an equivalent PDA:
$$S \rightarrow aABC$$
$$A \rightarrow aB \mid a$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$$B \rightarrow bA \mid b$$
$$C \rightarrow a$$

12. Define PDA and discuss its applications. Suppose the PDA P = ({q, p},{0, 1}, $\{Z_0, X\}$, δ, q, Z_0, {p}) has the following transition function:
    i) $\delta(q, 0, Z_0) = \{(q, X, Z_0)\}$ ii) $\delta(q, 0, X) = \{(q, X_1, X)\}$
    iii) $\delta(q, 1, X_0) = \{(q, X)\}$ iv) $\delta(q, \varepsilon, X) = \{(p, \varepsilon)\}$
    v) $\delta(P, \varepsilon, X) = \{(p, \varepsilon)\}$ vi) $\delta(P, 1, X) = \{(P, X, X)\}$
    vii) $\delta(P, 1, Z_0) = \{(P, \varepsilon)\}$
    a) Find the language accepted.
    b) Starting from initial ID (q, w, Z_0), show all the reachable IDs when the input w is
        i) 0011
        ii) 010

    Hint: $L = \{a^n b^m \mid n, m \ 0\}$.

13. Prove that DPDA is not equivalent to NPDA.

    Hint: Prove by an example. Give a language that is accepted by NPDA but not by a DPDA.

14. Let G be a context free grammar in the CNF.
    a) Give an algorithm to determine the number of distinct derivations of a string x.
    b) Associate a cost with each production of G. Give an algorithm to produce a minimum-cost parse of a string x. The cost of a parse is the sum of the costs of the productions used.

    Hint: Modify the CYK algorithm.

15. Show that CFLs are closed under CYCLE operation.

Turing Machines 6

- Turing machine is a yardstick for computations that can be carried out on a digital computer. Turing machines, first described by Alan Turing in (Turing 1937), are simple, abstract, computational devices intended to help investigate the extent and limitations of what can be computed.

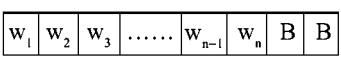
In this chapter, we discuss Turing machines (TMs) and their applications. Turing, writing before the invention of the modern digital computer, was interested in the question of what it means to be computable? Intuitively, a task is computable if one can specify a sequence of instructions which when followed will result in the completion of the task. Such a set of instructions is called an *effective procedure*, or *algorithm*, for the task. This intuition must be made precise by defining the capabilities of the device that is to carry out the instructions. Devices with different capabilities may be able to complete different instruction sets, and therefore may result in different classes of computable tasks.

6.1 Turing Assumptions

Turing proposed a set of assumptions that lead to a formal notion of computation that we will call *Turing computability*.

- No creativity implies that each step in the calculation must be fully spelled out.
- The list of instructions followed must be finite; i.e, programmes must be finite objects.
- Each individual step in the calculation must take a finite amount of time to complete.
- Intermediate results may need to be calculated, so a scratch-pad area is needed.
- There has to be a way to keep track of the current step of the calculation.
- There has to be a way to view the complete current state of the calculation.

To support the above assumptions, Alan modelled a TM as a machine with a finite number of control states and an infinite tape, bounded at the left and stretching off to the right. The tape is divided into cells, each of which can hold one symbol. The input of the machine is a string $w = w_1 w_2 \ldots w_n$ initially written on the leftmost portion of the tape, followed by an infinite sequence of blanks B.



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

The machine is able to move a read/write head left and right over the tape as it performs its computation. It can read and write symbols on the tape as it pleases. These considerations led Turing to the following formal definition.

Definition 1: A Turing Machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where

Q is a finite set of states.

Σ is the input alphabet, which never includes blanks.

Γ is the tape alphabet, which always includes blanks. Moreover, every input symbol is in Γ.

δ: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function, where L and R are directions, telling the machine head in which direction to go in a step.

$q_0 \in Q$ is the start state.

B is a special symbol indication of a blank cell.

F is a set of final states, where the TM halts on reaching these states.

The program that a TM executes is embodied in the transition function δ. Conceptually, the following happens when a transition $\delta(q_i, a) = (q_j, b, d)$ is made by a TM M:

- M writes b to the current tape cell, overwriting a.
- The current state changes from q_i to q_j.
- The tape head moves to the left or right by one cell, depending on whether d is L or R.

A sequence of linked transitions starting from the initial state is an execution of the given task.

6.1.1 Instantaneous Description

The Instantaneous Description (ID) of a turing machine is the configuration of the system in triple $<l, q, r>$ where q is the current state, l is a string denoting the tape contents to the left of the tape head, and r is a string to the right of the tape head. Since the tape is infinite, there is a point past which the tape is nothing but blanks. By convention, these are not included in r. The leftmost symbol of r is the current tape cell. A turing machine starts with the initial configuration $<\varepsilon, q_0, w>$ and repeatedly makes transitions until it halts.

Note: A machine may sometimes enter into an infinite loop where the processing is recursively repeated and then it never stops. Such programs are useless, but they are undeniably part of what we understand by computation.

Moves: At any given time, the move of a turing machine M is dependent on the input and the current state. Let us suppose that the current ID of the turing machine is $\alpha a q_i a \beta$.

- if the transition is defined as $\delta(q_i, a) = (q_j, b, L)$, then the next ID is $\alpha q_j a b \beta$.
- if the transition is defined as $\delta(q_i, a) = (q_j, b, R)$, then the ID is $\alpha a b q_j \beta$.

It is clear that at any given instance, the configuration at i + 1 instance is dependent on the transition made on configuration i.

Turing Machine will terminate execution on reaching to accepting configuration ($<u, q, w>$ $\forall$ q is a halting state), or may enter into a state where the next moves are not defined. Such states are termed rejecting states.

6.1.2 Turing Machine as Language Accepter

The language of a Turing Machine M, L(M), is the set of strings accepted by M.

L(M) = {x | M halts and accepts x}. Turing machines are defined so that they can accept by halting on given input or by entering into recursive loop for invalid input string.

M accepts w iff the execution of M on w is terminating and ends in the accept state: $<\varepsilon, q_0, w> \vdash <l, q, r> \forall$ q declared as accepting state.

M rejects w iff the execution of M on w is terminating and ends in the non-accepting state: $<\varepsilon, q_0, w> \vdash <l, q, r> \forall$ q declared as non-accepting state.

M does not accept w iff M rejects w or M loops on w.

But most programs compute functions, i.e, deliver answers beyond just 'yes' or 'no'.

6.2 Turing Machine as a Computational Machine

A turing machine M computes a function f if, when given input w in the domain of f, the machine halts in its accept state with f(w) written (leftmost) on the tape. To use TM as a computational machine, it is required to place the integer numbers as 0^m. Suppose it is required to add two numbers; that is, f(m, n) = m + n, then the numbers m and n are to be placed on the tape as 0^m10^n where 1 is a separator for the numbers m and n. Once processing is completed and the TM halts, the tape would have the contents as $0^{(m+n)}$, which is the required result of computation.

Example 6.1 Design a TM to accept strings belonging to the language $(0 + 1)^*$.

Solution: To design a TM for the given language, we have to identify (1) the required states, (2) inputs to be processed, (3) steps of the process and (4) when to halt.

To accept strings of language, it is required to move right end with inputs either '0' or '1' and rewrite the same symbol until it encounters B. The states of the machine will just be $\{q_0, q_A\}$. The input alphabet $\Sigma = \{0, 1\}$, for simplicity. The tape alphabet $\Gamma = \Sigma \cup \{B\}$ includes blanks, but is, otherwise, the same as the input alphabet. All that is left to specify is the transition function. The machine simply moves right, along the tape, until it hits a blank, and then it halts. Thus, at each step, it just writes back the current symbol, remains in q_0, and moves right, by one cell. The transitions can be defined by

$$\delta(q_0, 0) = (q_0, 0, R)$$

$$\delta(q_0, 1) = (q_0, 1, R)$$

Once the machine hits a blank, it moves one cell to the left and stops:

$$\delta(q_0, B) = (q_A, B, L)$$

Notice that if the input string is B, the first step the machine makes is moving left from the leftmost cell. This is not possible and so the tape head just stays in the leftmost cell.

TMs can be represented by *transition diagrams*.

A transition $\delta(q_i, a) = (q_j, b, d)$ between states q_i and q_j can be drawn as an edge labelled as a/b, d between q_i and q_j. This edge indicates if the machine is in state q_i and the current cell has an 'a' symbol, then the current cell is updated to have a 'b' symbol, the tape head moves one cell to the left (right) if d = L (R), and the current state becomes q_j. The Figure 6.1

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

is a transition diagram for a TM that travels right, enters an accepting state and halts on seeing a blank.

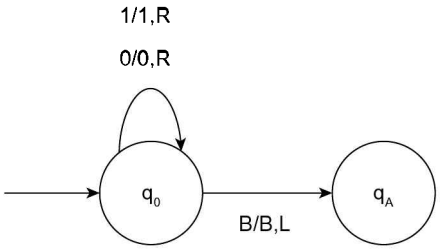


Fig. 6.1 *Simple TM*

For the current example, the transition diagram is quite simple:

TM can also be represented as a transition table. The following is the table representation of the language (0 + 1)*

	a = 0	a = 1	a = B
$\rightarrow q_0$	$(q_0, 0, R)$	$(q_0, 0, R)$	(q_A, B, L)
	----	a = 1	----

Example 6.1 Design a TM for finding 1's complement of a given binary number.

Solution: If the given number is 100110, then its 1's complement is 011001.

If the given number is 11110, then its 1's complement is 00001.

From the above examples, it is clear that the TM has to be designed such that while moving to right, if the input is 1, it should be changed to 0 and if the input is 0 it should be changed to 1 and halt on B. The transition diagram is shown in the Figure 6.2.

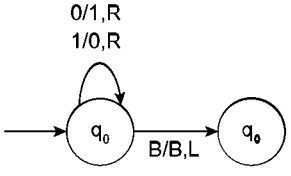


Fig. 6.2 *TM for 1's Complement*

	a = 0	a = 1	a = B
$\rightarrow q_0$	$(q_0, 1, R)$	$(q_0, 0, R)$	(q_A, B, L)
$\rightarrow q_0$	----	a = 1	----

Example 6.1 Design a TM to add two numbers a and b.

Solution: Let the numbers be 2 and 3. The addition of these numbers using simple logic is explained. The numbers are placed as $B0^2 10^3 B$. After processing, the tape content would be $B0^5 B$. The simple logic that can be used is: replace the occurrence of 0 by B and move to

right and replace 1 to 0, so that it is in required form as B0^5B. Sequence of steps is given for understanding and Figure 6.3 shows the details.

1. In initial state, 0's is replaced by B and the state is changed to a new state:

$$\delta(q_0, 0) = (q_1, B, R)$$

2. In q_1 state, move to right on '0'. When the input is '1' replace it by '0'.

$$\delta(q_1, 0) = (q_1, 0, R)$$

$$\delta(q_1, 1) = (q_1, 0, R)$$

$$\delta(q_1, B) = (q_A, B, R)$$

B•001000B ⊢ B•01000B ⊢ B0•1000B ⊢ B00•000B ⊢ B000•00B ⊢ B0000•0B ⊢B00000•B ⊢ B00000B

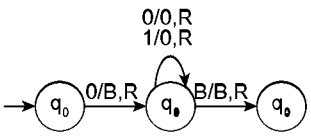


Fig. 6.3 *TM to Add Two Numbers*

	0	1	B
$\rightarrow q_0$	(q_1, B, R)	---	---
q_1	$(q_1, 0, R)$	$(q_1, 0, R)$	(q_A, B, R)
q_A	---	---	---

6.3 Techniques for Turing Machine Construction

Describing a Turing Machine transitions is not a simple task. To make this task easy, there are some high-level conceptual tools which can be used in the construction process.

6.3.1 Storage in Finite Control

A Turing Machine has a finite number of states in its CPU. However, the number of states is not always small. Like a Pentium chip, we can store values in it as long as there is only a finite number of states. For example, all real computers have registers, but there are only a fixed number of them, and each register can only hold one of a fixed (and finite) number of bits. Similarly, we define a state as a pair which stores the details of control and other stores the symbol. To account for this modification, we can define the Turing Machine as M = (Q, Σ, Γ, δ, [q_0, B], B, F) where Q is of the form [q, a] where q is a state, and a ∈ Σ. The transitions are defined by ([QXΣ], Γ) → ([QXΣ], Γ, {R/L}). For example, the transition δ([q, a], b) = ([p, b], c, R)indicates that the control is in state q, and a is stored in finite control. On the input symbol b, it moves to p state, changes the symbol in finite control to b, changes the cell content as c and moves one cell right.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example 6.1 Design a TM that accepts strings of the ab* or ba*.

Solution: To design this system, it is required to define the transitions such that on seeing 'a' it should move to a state and be in same state for input 'b' until 'B' is encountered. On 'B', move to final state. Similarly, if the first symbol is 'b', move to a different state and be in that state on 'a', and finally move to final state on 'B'. Transition diagram is shown in Figure 6.4.

The same problem can be solved with two states, but changing the data stored in finite control.

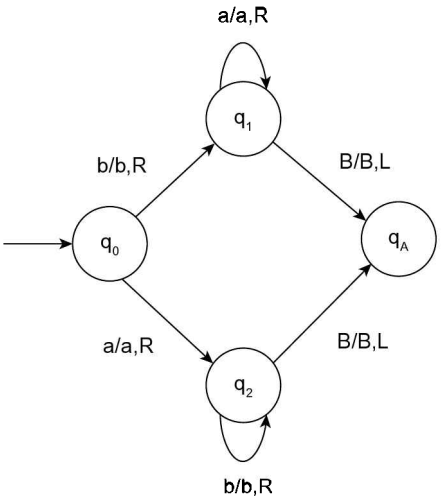


Fig. 6.4 *TM for ab*+ba**

Let the state be defined as $\{q_0, q_A\} \times \{a, b, B\}$. Hence the states are $[q_0, a]$, $[q_0, b]$, $[q_0, B]$, $[q_A, a]$, $[q_A, b]$ and $[q_A, B]$, where $[q_A, B]$ is the final state.

The transitions are defined as follows:

1. Let the initial configuration be $[q_0, B]$.
2. On seeing eithier 'a' or 'b' symbol, store it in finite control, and move right.

$$\delta([q_0, B], a) = ([q_0, a], a, R)$$

$$\delta([q_0, B], b) = ([q_0, b], b, R)$$

3. In $[q_0, a]$ state, move right on seeing 'b' and enter the final state $[q_A, B]$ on seeing 'B'.

$$\delta([q_0, a], b) = ([q_0, a], b, R)$$

$$\delta([q_0, a], B) = ([q_A, B], B, L)$$

4. In $[q_0, b]$ state, move right on seeing 'a' and enter the final state $[q_A, B]$ on seeing 'B'.

$$\delta([q_0, b], a) = ([q_0, b], a, R)$$

$$\delta([q_0, b], B) = ([q_A, B], B, L)$$

6.3.2 Multi-track Tape

The tape is imagined as divided into cells where the input to be processed is placed. We can further imagine that the tape is divided into k tracks for some finite number k as shown in Figure 6.5.

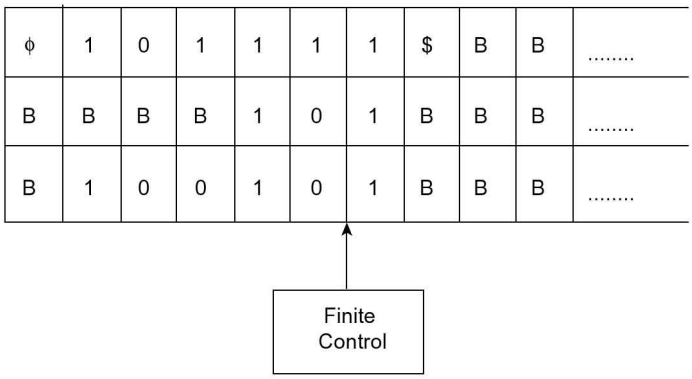


Fig. 6.5 *Multi-track Tape*

The reading head considers k symbols belonging to different tracks in same column and processes it. There are two special symbols Φ and $ used in the first track and they indicate the boundary of the input. The other tracks are used to place the intermediate results and the final result of the processing. The blank input is identified as 'all B's in all tracks', that is, as [B, B, B]. The input at the current position of the reading head is [1, 1, 1].

Example 6.1 Design a TM to find whether the given number is prime or not.

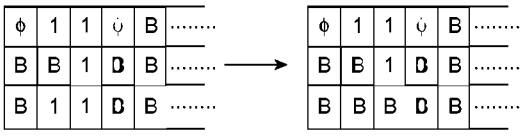
Solution: To design a TM to identify a given number is prime or not, we have to find whether the number has a factor other than 1 and itself.

1. Let us place the given number on the first track in binary form bounded by Φ and $. For example, 47 is represented as Φ101111$.
2. On the second track, write 2 in binary form, 10.v
3. Copy the number on the first track to the third track.
4. Perform repeated subtraction of the number on third track with the number on the second track until the number on the third track is either 0 or less than the number on the second track.
5. If the number on the third track is zero and number on the second is not equal to the number on the first track, then the number on the first track is not prime, otherwise prime.
6. If the number on the third track in non-zero then increase the number on the second track by one.
7. Repeat the Steps 4–6 until the number on the second track is equal to number on the first.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Illustration with examples is shown in the Figure 6.6.

a) Find whether 6 is a prime or not



b) Find whether 7 is a prime or not

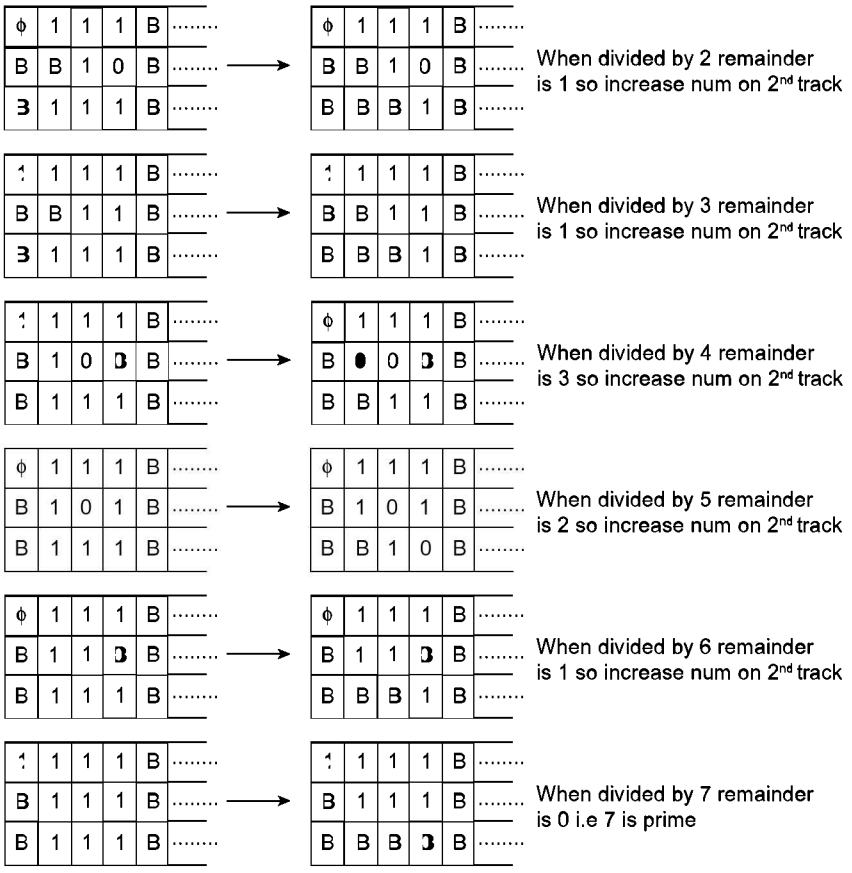


Fig. 6.6 *Example Using Multitrack Tape*

6.3.3 Checking off Symbols

This is one useful technique that can be used to visualize how TM would recognize the languages. This technique uses an extra track that indicates the symbol on the other track is processed. The languages which have repeated strings or some conditions relating to other part of string can be solved with this procedure. Such languages are listed below.

a) $\{ww \mid w \text{ in } \Sigma^*\}$

b) $\{ww^R \mid w \text{ in } \Sigma^*\}$

c) $\{a^ib^i \mid i \geq 1\}$

d) $\{a^ib^jc^k \mid i \neq j \text{ or } j \neq k\}$

e) $\{wcw \mid w \text{ in } \Sigma^*\}$

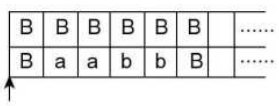
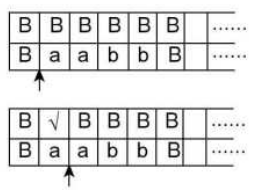
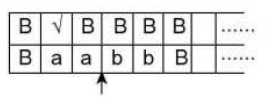
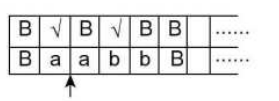
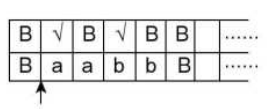
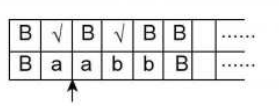
For the languages mentioned above, we can use the tape with two tracks where on one track, we place the given input, and on the other track, we place either B or √. If the upper track symbol is B, it indicates the symbol on lower track is not considered. If the symbol on upper track is √, it indicates that the symbol on the lower track is considered.

Example 6.6 Design a Turing machine for the language $\{a^ib^i \mid i \geq 1\}$.

Solution: Let the TM be defined such that

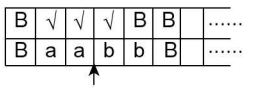
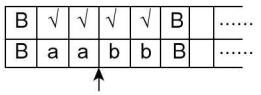
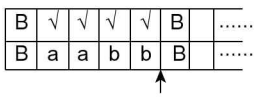
$$Q = \{[q, d] \mid q = q_0, q_1, \ldots \text{ and } d = a, b \text{ or } B\}$$
$$\Sigma = \{[B, d] \mid d = a, b\}$$
$$\Gamma = \{[X, d] \mid X = B \text{ or } \surd \text{ and } d = a, b \text{ or } B\}$$
$$q_0 = [q_0, B] \text{ and } F = \{[q_A, B]\}$$

Initially the input is placed as shown in the figure.

1. [B, B] is identified as blank symbol. Let the transition on blank be defined by $\delta([q_0, B], [B, B]) = ([q_0, B], [B, B], R)$	
2. On [B, a], mark it as considered and change to a new state and move right: $\delta([q_0, B], [B, a]) = ([q_1, a], [\surd, a], R)$.	
3. In $[q_1, a]$ state, travel right to see [B, b]: $\delta([q_1, a], [B, a]) = ([q_1, a], [B, a], R)$.	
4. On [B, b], mark it as considered and change to a new state and move left: $\delta([q_1, a], [B, b]) = ([q_2, B], [\surd, b], L)$	
5. In $[q_2, B]$ state, travel left to see [√, a]: $\delta([q_2, B], [B, a]) = ([q_2, B], [B, a], L)$	
6. In $[q_2, B]$, change to $[q_1, B]$ on [√, a] and move right: $\delta([q_2, B], [\surd, a]) = ([q_1, B], [\surd, a], R)$	

(continued)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

7. Define recursive transitions for $[q_1, a]$ and $[q_2, B]$ as follows: $\delta([q_1, a], [\surd, b]) = ([q_1, a], [\surd, b], R)$ $\delta([q_2, B], [\surd, b]) = ([q_2, B], [\surd, b], L)$	
8. Repeat the process until $[\surd, b]$ on $[q_1, B]$ is seen: $\delta([q_1, B], [\surd, b]) = ([q_3, B], [\surd, b], R)$	
9. In $[q_3, B]$, ignore $[\surd, b]$; and on $[B, B]$, move to final state: $\delta([q_3, B], [\surd, b]) = ([q_3, B], [\surd, b], R)$ $\delta([q_3, B], [B, B]) = ([q_A, B], [B, B], R)$	

6.3.4 Subroutines

A Turing machine can simulate any type of subroutines found in programming languages, including recursive procedures, and any of the known parameter passing mechanisms. We can design a TM program to serve as a subroutine. It has a designated initial state and a return state. These states are used to indicate the call to a subroutine and return to caller subroutine. To design a call to subroutine, a new set of states, are defined which are used to enter the initial state of the subroutine and return from the return state of subroutine. As an example, a TM designed to accept strings with balanced parenthesis.

Example 6.6 Design a TM that accepts only the strings with balanced parenthesis.

Solution: To solve this, we need to frame an algorithm that can be used to match every occurrence of "(" for every occurrence of ")". At the end, if any parenthesis is unmatched, then the given string is declared not balanced.

Procedure:

1. First search for the occurrence of ")".

   For this process, in the initial state q_0 ignore all "(" until ")" is seen. The transitions are as follows:

   $$\delta(q_0, (\ ) = (q_0, (, R)$$

2. On finding ")" replace it by X, change to new state and travel left for the first occurrence of "(". We consider this state as the initial state of the subroutine that is used to find the corresponding "(" for ")". While travelling back the possible elements, it can see is X.

   Transitions required are as follows:

   $$\delta(q_0,)\ ) = (q_1, X, L)$$

   $$\delta(q_1, X) = (q_1, X, L)$$

3. If "(" is found, replace it by X. If X is not found, enter into rejecting state. In this example, q_1 acts as both initial state and return state of the subroutine:

   $$\delta(q_1, (\ ) = (q_0, X, R)$$

   $$\delta(q_1, B) = (q_3, B, R)$$

4. Repeat Steps 1 and 2 until a B is encountered:

$$\delta(q_0, X) = (q_0, X, R)$$

$$\delta(q_0, B) = (q_2, B, L)$$

5. If B is encountered, enter into a new state and check if there is no "(" unbalanced:

$$\delta(q_2, X) = (q_2, B, L)$$

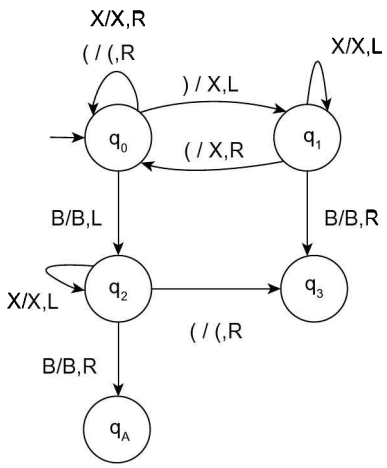
$$\delta(q_2, (\ ) = (q_3, (, R)$$

$$\delta(q_2, B) = (q_A, B, R)$$

The following example explains the procedure with the string (())(). Let • indicate the current position of the reading head where the right element is the current input.

•(())()B ⊢ (•())()B ⊢ ((•))()B ⊢ (•(X)() B ⊢ (X•X)() B ⊢ (XX•)()B ⊢ (X•XX()B ⊢ (•XXX() B ⊢•(XXX()B ⊢ X•XXX()B ⊢ XX•XX()B ⊢ XXX•X()B ⊢ XXXX•()B ⊢ XXXX(•)B ⊢ XXXX•(XB ⊢ XXXX•XXB ⊢ XXXXX•XB ⊢⊢ XXXXXX•B

Since there are some steps that are performed repeatedly as shown in the above example, let us define the states as q_0 as start state which is used to look for the occurrence of ")" while travelling right. Once it encounters ")", it enters into q_1 state. In q_1 state, it should travel left until it encounters "(". On seeing "(", it is again replaced by X and enters into q_0 state and travels right. This process is repeated until it encounters B in q_0 state, and then it enters q_2 state. In q_2 state, travel to extreme left. If no "(" is encountered, then on seeing B, enter to accepting state and halt. If "(" is seen, enter into rejecting state.



Transition table for the above problem is as follows:

	(	)	X	B
$\rightarrow q_0$	$(q_0, (, R)$	(q_1, X, L)	(q_0, X, R)	(q_2, B, L)
q_1	(q_0, X, R)	---	(q_1, X, L)	(q_3, B, R)
q_2	$(q_3, (, R)$	---	(q_2, X, L)	(q_A, B, R)
q_3	---	---	---	---
q_A	---	---	---	---

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

6.3.5 Shifting Over

A turing machine can make space on its tape by shifting non-blank symbols by a finite number of cells to the right. To perform this operation, we can use the state with a small amount of storage. This storage space is used to store the symbols and then replacing the current cell on tape by blank, and to move right. Read the right symbol and replace it with the symbol stored in the finite control. To perform this operation without loosing the date, it requires storage capacity to store at least two symbols. The following example is given to explain the procedure.

Example 6.6 Design the turing machine which can shift the data on the tape by two spaces.

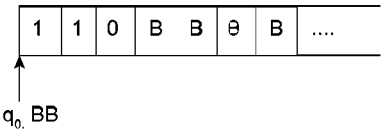
Solution: To design the turing machine which creates two spaces on the left, we need to use a finite control that can store two symbols. Let the state be defined as $[q, A_1, A_2]$ where $q = q_0, q_1, q_A$, and A_1 and A_2 can hold the symbols of Σ or B. Let X denote a special symbol to indicate the created blank spaces on the left of the input.

The TM starts with initial state $[q_0, B, B]$.

1. In $[q_0, B, B]$ state, the transition is defined to store the current symbol encountered in the A_1 and to replace the cell content with X:

$$\delta([q_0, B, B], A_1) = ([q_0, B, A_1], X, R)$$

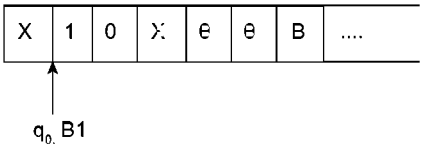
$$\text{i.e, } \delta([q_0, B, B], 1] = ([q_0, B, 1], X, R)$$



2. In $[q_0, B, A_1]$ state, the transition is defined to move content of A_1 left, store the current symbol encountered in A_2 and replace the cell content with X:

$$\delta([q_0, B, A_1], A_2) = ([q_0, A_1, A_2], X, R)$$

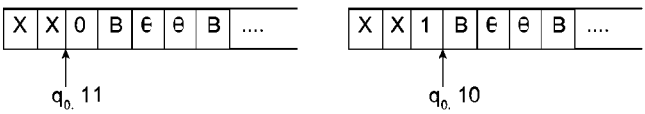
$$\text{i.e, } \delta([q_0, B, 1], 1) = ([q_0, 1, 1], X, R)$$



3. In $[q_0, A_1, A_2]$ state, the transition is defined to move the content of A_2 to A_1, store the current symbol encountered in A_2 and replace the cell content with A_1:

$$\delta([q_0, A_1, A_2], A_3) = ([q_0, A_2, A_3], A_1, R).$$

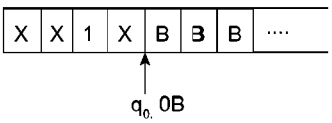
$$\text{i.e, } \delta([q_0, 1, 1], 0) = ([q_0, 1, 0], 1, R).$$



4. If B is encountered in $[q_0, A_1, A_2]$ state, the transition is defined to move the content of A_2 to A_1, store B in A_2 and replace the cell content with A_1:

$$\delta([q_0, A_1, A_2], B) = ([q_0, A_2, B], A_1, R).$$

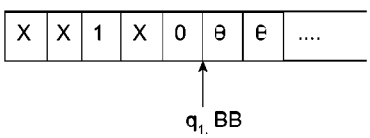
i.e, $\delta([q_0, 1, 0], B) = ([q_0, 0, B], 1, R).$



5. If B is encountered in $[q_0, A_1, B]$ state, the transition is defined to replace the cell content with A_1 and change to new state as the entire string is shifted to right:

$$\delta([q_0, A_1, B], B) = ([q_1, B, B], A_1, R).$$

i.e, $\delta([q_0, 0, B], B) = ([q_1, B, B], 0, R).$



6.4 Types of Turing Machines

There are a number of other types of Turing machines in addition to the one we have seen, such as Turing machines with multiple tapes, one tape but with multiple heads, two-dimensional tapes, non-deterministic TMs, etc. It turns out that computationally all these TMs are equally powerful. That is, what one type can compute any other type can also compute. However, the efficiency of computation, that is, how fast they can compute, may vary.

6.4.1 Non-deterministic Turing Machines

A non-deterministic turing machines is a machine for which like non-deterministic finite automata, at any current state and for the tape symbol it is reading, there may be different possible actions to be performed. Here an action means a combination of writing a symbol on the tape, moving the tape head and going to a next state. One action could be just changing the state without modifying the cell content. One action could be not changing the state and changing the cell content. One action could be changing f both the state and the cell content. In all actions, it may move right or left.

For example, let us consider the language $L = \{ww \mid w \in \{a, b\}^*\}$. Given a string x, a non-deterministic TM that accepts this language L would first guess the midpoint of x, which is the place where the second half of x starts. Then it would compare the first half of x with the

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

second half by comparing the ith symbol of the first half with the ith symbol of the second half for i = 1, 2, ... A deterministic Turing Machine, on the other hand, cannot guess the midpoint of the string x. It must find the midpoint by, pairing off symbols from the two ends of x. Formally, a non-deterministic TM is a TM whose transition function takes values that are subsets of $\{(Q \cup \{h\}) \times (\Gamma \cup \{\Delta\}) \times (R, L, H)\}$. As in the case of a NFA, it is understood that a non-deterministic TM, at any configuration, selects one combination of next state, tape symbol and head movement out of the set of triples without following any specific predetermined rule. It can be shown that a non-deterministic TM is also as powerful as a deterministic Turing machine.

Theorem: Any language accepted by a non-deterministic Turing machine is also accepted by some deterministic Turing machine.

Proof: Let T_N denote a non-deterministic Turing machine. Given a string x, T_N starts at the initial configuration and goes through a sequence of configurations until it, reaches a halt configuration, goes into an infinite loop or aborts. At any point in the process, T_N is in some configuration and has a finite set of configurations to choose from for its next configuration. The set of all possible computations that T_N can perform for a given string x can be represented by a rooted tree as follows: The root of the tree is the initial configuration and it is the only vertex of level 0. All possible configurations that are reachable by applying the transition function of T_N once form the children of the initial configuration. They form level 1. In general, for each vertex of level i, all possible configurations that are reachable by applying the transition function of T_N are its children. The children of all the vertices of level i form level i + 1. Note that the number of children for a vertex in this tree is finite because the number of states is finite and there are a finite number of tape symbols.

6.4.2 Turing Machines with Two-dimensional Tapes

Turing machines with *two-dimensional tape* is a kind of TM that has one finite control, one read-write head and one two-dimensional tape. The cells in the tape is two dimensional, i.e, the tape has the top end and the left end, but extends indefinitely to the right and down. It is divided into rows of small squares. For any TM of this type, there is an equivalent TM with a one-dimensional tape that is equally powerful; i.e, the former can be simulated by the latter. To simulate a two-dimensional tape with a one-dimensional tape, first we map the squares of the two-dimensional tape to those of a one-dimensional tape diagonally as shown in the following tables:

Two-dimensional tape

v	v	v	v	v	v	v	...	...
h	1	2	6	7	15	16	...	...
h	3	5	8	14	17	26	...	...
h	4	9	13	18	25	...	...	...
h	10	12	19	24	...	...	...	...
h	11	20	23	...	...	...	...	...
h	21	22	.<.>	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Here the numbers indicate the correspondence between the squares of the two tapes: square numbered i in the two-dimensional tape is mapped to square numbered i in the one-dimensional tape. Symbols h and v are not in the tape alphabet and they are used to mark the left and the top end of the tape, respectively.

Equivalent one-dimensional tape

v	v	v	v	v	h	v	v	h	v	v	h	v	10	h	11	. . .	. . .

The head of a two-dimensional tape moves one square up, down, left or right. Let us simulate this head move with a one-dimensional tape. Let i be the head position of the two-dimensional tape. If the head moves down from i, then move the head of the one-dimensional tape to right until it hits h or v counting the number of squares it has visited after i. Let k be the number of squares visited by the head of the one-dimensional tape. If h was hit first, then from h move the head of the one-dimensional tape further right to the kth square from h. That is the square corresponding to the square below i in the two-dimensional tape. If v was hit first, then $(k + 1)^{th}$ square to the right from v is the new head position. For example, suppose that the head is positioned at position numbered 8 in the table representing the two-dimensional tape; that is, i = 8. In the two-dimensional table, if the head moves down it reaches the square numbered 13. For equivalence on the one-dimensional tape, the head has to make the following moves: From position 8 move to the right, then it meets h first, and that is the third square from 8. In this circumstance, from h, move 3 positions to the right. By such a move the head position of the one-dimensional tape will be at the cell corresponding to 13 on the two-dimensional tape. If i = 5 and the head moves down on the other hand, then on the one-dimensional tape, the head moves to the right and it hits v first, which is the second square from i = 5. Thus this time, the third square is the head position of the one-dimensional tape corresponding to 9 on the two-dimensional tape. Thus some TMs with a one-dimensional tape can simulate every move of TM with a two-dimensional tape. Hence, they are at least as powerful as TMs with a two-dimensional tape. Since TMs with a two-dimensional tape obviously can simulate TMs with a one-dimensional tape, it can be said that they are equally powerful.

6.4.3 Turing Machines with Multiple Tapes

This kind of TM has one finite control and more than one tape each with its own read/write head. It is denoted by a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$. Its transition function is a partial function:

$$\delta: QX(\Gamma \cup \{B\})^n \rightarrow (Q \cup \{h\})X(\Gamma \cup \{B\})^n X\{R, L, S\}^n$$

A configuration for this kind of Turing machine must show the current state the machine is in and the state of each tape. It can be proved that any language accepted by a n-tape Turing Machine can be accepted by a one-tape TM and that any function computed by a n-tape TM can be computed by a one-tape TM. Since the converses are obviously true, one can say that one-tape TMs are as powerful as n-tape TMs.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

6.4.4 Turing Machines with Multiple Heads

This kind of Turing Machines has one finite control and one tape, but more than one read/write heads. In each state, only one of the heads is allowed to read and write. It is denoted by a 5-tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$. The transition function is a partial function:

$$\delta: QX\{H_1, H_2, \ldots H_n\}X(\Gamma \cup \{B\}) \rightarrow (Q \cup \{h\})X(\Gamma \cup \{B\})X\{R, L, S\}$$

where H_1, H_2 ..., H_n denote the tape heads. It can be easily seen that this type of Turing machines are as powerful as one-tape TMs.

6.4.5 Turing Machines with Infinite Tape

This is a kind of TM that have one finite control and one tape which extends infinitely in both directions. It turns out that this type of Turing machines are also as powerful as one-tape Turing Machines whose tape has a left end.

Two-way infinite tapes

In all our formulations, we specified that the tape had a left end and stretched infinitely far to the right. Relaxing this stipulation to allow the tape to stretch infinitely far to right and left results in a new formulation of Turing Machines equivalent to the original. That is, for any Turing Machine using a two-way tape, there is a Turing Machine with a one-way infinite tape with the same input–output behaviour, and vice versa.

One can simulate a Turing Machine with two-way infinite tape on a Turing Machine with one-way infinite tape. Let the two-way infinite tape have the contents as shown below:

A_{-4}	A_{-4}	A_{-4}	A_{-1}	A_0	A_1	$A_\bullet$ $A_\bullet$	A_4	....

Let the reading head be left of A_0. This can be simulated by a Turing machine with one way infinite tape with two tracks. The contents on tape is placed in such a way that, content on right of A_0 is placed on upper track and the content on left of A_0 is placed on lower track in reverse order. The left end cell contains on upper track, and on the lower track, the left end cell contains a special symbol Φ:

A_0	A_1	A_2	A_1	A_2	....
Φ	A_1	A_2	A_1	A_2	....

The moves are simulated such that when the reading head is to the right of A_0, the moves are implemented as if they are reading symbols from upper track. If the reading head is to the left of A_0, then symbols are read from lower track head, but the direction of the reading head would be in opposite direction in which it moves on the two-way infinite tape. That is, if it moves left then on one way infinite tape, it moves right on the two-way infinite tape, and vice versa.

Many other variations of the Turing machine are possible. However, it has been shown that none of them exceed the capability of basic deterministic TM as far as accepting languages is concerned. In fact, Church's thesis conjectures that any so-called computation done by humans or computers can be performed by a basic deterministic TM.

6.5 Church's Thesis

The Church–Turing thesis (formerly commonly known simply as Church's thesis) says that any real-world computation can be translated into an equivalent computation involving a TM. In Church's original formulation (Church 1935, 1936), the thesis says that real-world calculation can be done using the lambda calculus, which is equivalent to using general recursive functions.

The Church–Turing thesis can be applied to any kind of computations, those involving cellular automata, combinators, register machines and substitution systems. It is also applicable to other kinds of computations found in theoretical computer science, such as quantum computing and probabilistic computing.

There are conflicting points of view about the Church–Turing thesis. One says that it can be proven, and the other says that it serves as a definition for computation. Church–Turing thesis has not been proved, but the support for its validity comes from the fact that every realistic model of computation, so far discovered, has been shown to be equivalent. As long as our method of computation places no bound on the number of steps or on the amount of storage, it seems that the partial recursive functions are intuitively computable. Random Access Machine (RAM) is an abstract model that makes use of partial recursive functions.

RAM consists of an infinite number of memory words, numbered 0, 1, …., where each word can hold any integer; and a finite number of arithmetic registers capable of holding any integer. These integers can be decoded into the form of a computer instruction. To simulate RAM by a TM, we use multi-tapes. One of the tapes holds the words of the RAM, where each word is separated by #, as shown below.

$$\#0*V_0\ \#1*V_1\#2*V_2\#3*V_{3\ldots}\ \#i*V_{i\ldots}$$

V_i is the contents, in binary, of the ith word. At any given time, there will be a finite number of words of the RAM stored on the tape; so, it is required to keep record of the largest numbered word that had been used so far.

RAM has finite number of arithmetic registers. One tape is used to hold each register's contents, and one tape is used to hold the location counter that indicates the number of the next word to be taken. One tape is used as a memory address register on which the number of a memory word may be placed.

Suppose that each word has the first 10 bits indicating the operation to be performed such as LOAD, STORE, ADD and so on, and let the remaining bits denote the address of the operand. Suppose the location counter holds number i in binary. The processing is done as below.

1. The reading head searches the first tape for the pattern #i*.
2. If not found, there is no instruction in the word i in the RAM. So, it halts.
3. If the pattern #i* is found, then the bits followed by * till next # are examined. Suppose the first 10 bits correspond ADD to Register 2, and suppose the remaining bits are some number j in binary.
4. M adds 1 to i on location counter tape.
5. Copies j into memory address tape.
6. M searches for #j* on the first tape, again starting from the left.
7. If #j* is not found, we assume j holds 0 and go to next instruction.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

8. If #j* is found, V_j is added to the contents of Register 2, which is stored on its own tape.
9. Repeat Steps 1–8 for every instruction.

6.6 Turing Machines as Enumerators

So far we have seen that a TM can be used as a language recognizer or as a computational machine. Here we would like to discuss the other capability of TM: It can also be used as a language generator. Consider a TM M that uses one tape as an output tape, on which a symbol once written can never be changed, and whose tape head never moves left. Suppose that TM writes on the output tape some strings over some alphabet Σ, separated by a marker $. These set of enumerated strings can form a language where the TM would halt for all the strings. We can define G(M) as the language generated by M having strings enclosed in pairs of $'s. Such languages are said to be enumerable languages.

Theorem 1: A language is Turing recognizable if and only if some enumerator enumerates it.

Proof of (IF):

Here we have to show that if enumerator E enumerates a language L then, there is a Turing machine M that recognizes L.

Given an input string x, we design M in the following way :

1. Run the enumerator E to enumerate the next string y of L. Compare it with x.
2. If x = y accept x, else go to 1

Proof of (ONLY IF): Here we have to show that if a TM M recognizes a language L, there is an enumerator for L.

Let s_1, s_2, s_3..... (be a list of all possible strings in Σ^*. One can readily construct an enumerator E_s that generates such a sequence. An enumerator E for L(M) works as follows:

Repeat the following for i = 1, 2, 3, ...

1. Run turing machine M for i steps on each input s_1, s_2, s_3.....
2. If any computation is accepted, print out the corresponding s_j

If turing machine M accepts a particular string s, it eventually appears on the list generated by E_s, after which it is printed out.

6.7 Universal Turing Machine

A universal TM M_u is an automaton that, given as input the description of any TM M and a string w, can simulate the computation of M for input w. To construct such a M_u, we first choose a standard way of describing TMs. We may, without loss of generality, assume that $M = (Q, \{0, 1\}, \{0, 1, B\}, d, q_1, B, q_2)$ where $Q = \{q_1, q_2, \ldots q_n\}$, q_1 the initial state, and q_2 the single final state. The alphabet $\{0, 1, B\} \in \Gamma$ are represented as a_1, a_2 and a_3. The directions left and right are represented as D_1 and D_2, respectively. The transitions of TM are encoded in a special binary representation where each symbol is separated by 1. For example, if there is a transition

$$\delta (q_i, a_j) = (q_k, a_l, D_m)$$

the binary representation for the transition is given as

$$0^i10^j10^k10^l10^m$$

The binary code for the Turing machine M that has transitions $t_1, t_2, t_3 \ldots t_n$ is represented as

$$111\ t_1\ 11\ t_2\ 11\ t_3\ 11\ \ldots\ 11\ t_n\ 111$$

Note: The transitions need not be in any particular order.

If a string has to be verified, then the problem is represented as a tuple <M, w> where M is the definition of TM, and w is the input string.

Example 6.6 Let M = $(\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$ have moves defined as

$$\delta(q_1, 1) = (q_3, 0, R)$$
$$\delta(q_3, 0) = (q_1, 1, R)$$
$$\delta(q_3, 1) = (q_2, 0, R)$$
$$\delta(q_3, B) = (q_3, 1, L)$$

Give the problem representation for the string w = 1011.

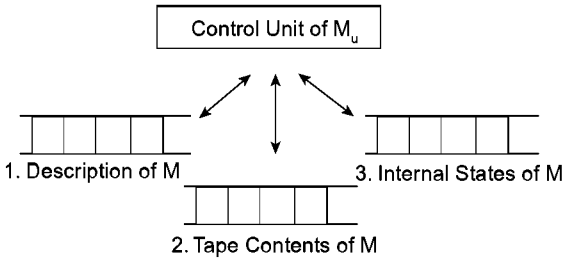
Solution: Let binary representation for states $\{q_1, q_2, q_3\}$ be {0, 00, 000}, for alphabet {0, 1, B} be {0, 00, 000} and for directions {R, L} be {0, 00}. The transitions are represented as follows:

Transition	Binary representation
$\delta(q_1, 1) = (q_3, 0, R)$	010010001010
$\delta(q_3, 0) = (q_1, 1, R)$	000101010010
$\delta(q_3, 1) = (q_2, 0, R)$	0001001001010
$\delta(q_3, B) = (q_3, 1, L)$	00010001000100100

The problem instance <M, 1011> is represented as

111 010010001010 **11** 000101010010 **11** 0001001001010 **11** 00010001000100100 **111** 1011

The following figure shows the organization of a universal TM that has a control unit and three tapes:



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

For any input M and w, Tape 1 will keep an encoded definition of M, Tape 2 will contain the tape contents of M and Tape 3, the internal state of M. M_u looks first at the contents of Tapes 2 and 3 to determine the configuration of M. The behaviour of the M is as follows.

1. Check the format of Tape 1 for the validations of the TM model.
   a. No two transitions should begin with 0^i10^j1 for the same i and j.
   b. Check that if $0^i10^j10^k10^l10^m$ represents a transition, then $1 \le j \le 3$, $1 \le l \le 3$, and $1 \le m \le 3$.
2. Initialize Tape 2 to contain w. Initialize Tape 3 to hold a single 0 representing initial state q_1. For all the tapes, the tape heads are positioned at the left end and these symbols are marked to identify the starting position.
3. When Tape 3 holds 00, it is said to reach the final state, and the machine can halt.
4. Let, at any given time, a_j be the symbol currently scanned by tape head 2 and let 0^i, the contents of Tape 3 (which indicates state). Scan Tape 1 from the left end to the second 111 looking for a substring beginning with 110^i10^j1.
   a. if no such string is found, then halt and reject.
   b. if found, then let the suffix be $0^k10^l10^m11$. Put 0^k on Tape 3, print a_l on the tape cell scanned by head 2 and move the head in direction D_m.

It is clear that M_u accepts <M, w> if and only if M accepts w. It is also true that if M runs forever on w, M_u runs forever on <M, w> and if M halts on w without accepting, M_u also halts on w without accepting.

6.8 Counter Machine

A counter machine is an abstract machine used in formal logic and theoretical computer science to model computation. A counter machine comprises a set of one or more unbounded **registers**, each of which can hold a single non-negative integer, and a list of arithmetic and control instructions for the machine to follow.

***Definition*:** A counter machine consists of

1. A finite set of registers $r_0 \ldots r_n$, where each register is labelled and can hold any single non-negative integer.
2. A special register to identify the current instruction to be executed is maintained and is called the **state register**.
3. A finite set of instructions $I_0 \ldots I_m$, where each instruction is in the same physical space as the registers.

Equivalence of Counter machines and Turing machine

Every turing machine can be simulated by a two counter machine. The turing machine consists of a Finite State machine and infinite tape that can be divided and used as a stack. The proof is explained in the following three steps.

1. Simulate turing machine by a finite-state machine (FSM) equipped with two stacks.
2. Simulate two stacks by four counters.
3. Simulate four counters by two counters.

Step 1: Simulating turing machine by two stacks
The tape is initially filled with zeroes, which can be modified with ones and zeroes. At any time, the read/write head of the machine points to one cell on the tape. This tape is conceptually cut into half and each half is treated as a stack. The top is the cell nearest the read/write head, and the bottom is some distance away from the head, with all zeroes on the tape beyond the bottom. Accordingly, a turing machine can be simulated by a FSM plus two stacks. Moving the head left or right is equivalent to popping a bit from one stack and pushing it onto the other. Writing is equivalent to changing the bit before pushing it.

Step 2: Simulate two stacks by four counters
A stack containing zeroes and ones can be simulated by two counters. The bits on the stack are assumed to be represented in binary, with the top being the least significant bit.

0 is pushed onto the stack to indicate the doubling of the number 1, to indicate doubling and adding 1. Popping the top is equivalent to dividing by 2, where the remainder is the bit that was popped. Two counters can simulate this stack: One counter holds a number whose binary representation represents the bits on the stack, and the other counter is used as a scratch pad. To double the number in the first counter, the FSM can initialize the second counter to zero, then repeatedly decrement the first counter once and increment the second counter twice. This continues until the first counter reaches zero. At that point, the second counter will hold the doubled number. To divide by 2, repeat decrementing in Counter 1 by twice and increment the other by one until the first counter reaches zero. The remainder can be determined by whether it reached zero after an even or an odd number of tries.

Step 3: Simulate four counters by two counters
In Step 2, one counter is used as scratch pad, the other is a real counter that holds an integer whose prime factorization is $2^a3^b5^c7^d$. The exponents a, b, c, and d can be thought of as four virtual counters that are being simulated.

- To set all virtual counters to zero, the real counter is set to zero and then incremented once.
- To double or make it half, increment or decrement a.
- To multiply or divide by 3, increment or decrement b.

As a result, a FSM with two counters can simulate four counters, which are in turn simulating two stacks, which are simulating a TM. Therefore, a FSM plus two counters is at least as powerful as a TM. A TM can easily simulate a FSM with two counters. Therefore, the two machines have equivalent power.

6.9 Recursive and Recursively Enumerable Languages

There are *three* possible outcomes of executing a TM over a given input. The TM may

- halt and accept the input;
- halt and reject the input; or
- never halt.

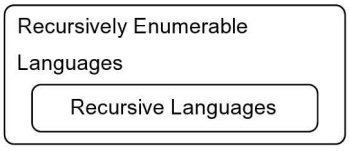
Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

A language is said to be recursive if there exists a TM that accepts every string of the language and rejects every string (over the same alphabet) that is not in the language.

Note: If a language L is recursive, then its complement $\bar{L}$ must also be recursive.

A language is said to be recursively enumerable if there exists a TM that accepts every string of the language, and does not accept strings that are not in the language. For strings that are not in the language the turing machine may or may not halt.

Note: Every recursive language is also recursively enumerable. It is not obvious whether every recursively enumerable language is also recursive:



6.10 Linear Bound Automata and Context Sensitive Language

A non-deterministic TM is called linear bound automata (LBA) if

- Its input alphabet includes two special symbols ∅ and $ as left and right end markers.
- It has no moves beyond these end markers, i.e, no left move from ∅ and no right move from $.
- It never changes the symbols ∅ and $.

A linear bound automaton is defined using 8-tuple form by $M = (Q, \Sigma, \Gamma, \delta, q_0, \varnothing, \$, F)$, where $Q, \Sigma, \Gamma, \delta, q_0, F$ are same as for non-deterministic TM, and ∅ and $ are left and right end markers. The language accepted by M is defined as L(M) and is given by

$$\{w \mid w \in (\Sigma - \{\varnothing, \$\})^* \text{ and } q_0\varnothing w\$ \overset{*}{\underset{M}{\longmapsto}} aqb \text{ for some } q \text{ in } F\}$$

6.10.1 Equivalence of LBA's and CSG's

We can show that if L is a context sensitive language (CSL), then there exists a linear bound automaton M such that $L(M) = L - \varepsilon$.

Theorem 3: If L is a CSL, then L is accepted by some LBA.

Proof: Let us construct a linear bound automaton M with two-track tape, to recognize L. The first track holds the input string w as ∅w$. The second track is used while the input is processed.

- LBA initializes the second track with S just below the leftmost symbol of w.
- If $w = \varepsilon$, then the system halts without accepting.
- Otherwise, it repeatedly guesses a production and a position in the sentential form which is on the second track.

- If the sentential form expands, then it shifts the portion of string from the current position to right.
- If the new sentential form of is longer than w, then the system halts without accepting.

Since the right side of all the productions are at least as long as left side, there would not be any derivation as $S \overset{*}{\Rightarrow} \alpha \overset{*}{\Rightarrow} w$, where α is longer than w. Hence, the LBA accepts a string if and only if $S \overset{*}{\Rightarrow} w$, where w is a word generated by CSG.

Theorem 4: If L = L(M) for LBA M = (Q, Σ, Γ, δ, q_0, ∅, $, F), then L – {ε} is a CSL.

Proof: Let L be accepted by a LBA M. Construct a grammar G that generates string w which is modified to incorporate the end markers ∅ and $. The reason for this is that if the CSG simulates the LBA using separate symbols for the end markers or state, it could not erase these symbols. This would necessitate shortening the sentential form, and the right side of every CSG production to be as long as the left side.

The sequential pair are generated with two components where first component is a terminal string of the form $a_1, a_2, \ldots a_n$, and second component is the linear bound automation tape to incorporate the production rule.

1. $A_1 \rightarrow [a, q_0 \varnothing a] A_2$
2. $A_1 \rightarrow [a, q_0 \varnothing a\$]$
3. $A_2 \rightarrow [a, a] A_2$
4. $A_2 \rightarrow [a, a\$]$

   For all a in Σ – {∅, $}
5. q[a, X] → [a, Y]p for each a in Σ ∪ {ε} and each q in Q and X, Y in Γ, such that δ(q, X) = (p, Y, R)
6. [b, Z]q[a, X] → p[b, Z] [a, Y] for each a and b in Σ ∪ {ε} and each q in Q and X, Y and Z in Γ, such that δ(q, X) = (p, Y, L)
7. [a, aqb] → a for all a in Σ – {∅, $} and q in F, and all possible α and β, which includes {∅, $}
8. [a, α]b → ab and b[a, α] → ba for any a and b in Σ – {∅, $} and all possible a's.

Solved Problems

Problem 1: Design a TM to accept strings formed with 0 and 1 that have the substring 000.

Solution: To accept strings with substring 000, the TM would be similar to the finite automaton constructed for the same language.

1. In state q_0, define transition to ignore the 1's encountered. If 0 is seen, then change to new state which indicates one 0 is present:

$$\delta(q_0, 1) = (q_0, 1, R)$$
$$\delta(q_0, 0) = (q_1, 0, R)$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

2. In q_1 state, if input is 0, change to new state to indicate the substring 00; if input is 1, go back to initial state:

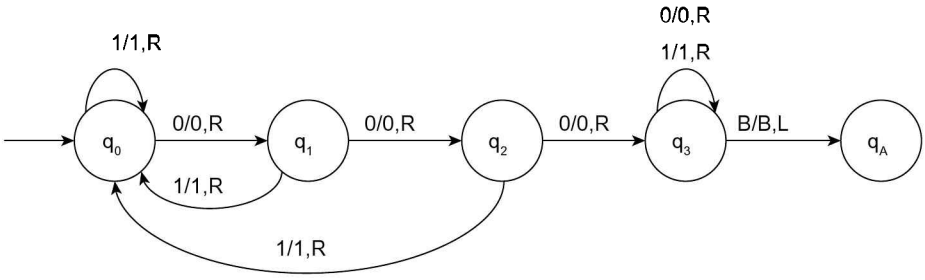
$$\delta(q_1, 1) = (q_0, 1, R)$$
$$\delta(q_1, 0) = (q_2, 0, R)$$

3. In q_2 state, if input is 0, change to new state to indicate substring 000; if input is 1, go back to initial state:

$$\delta(q_2, 1) = (q_0, 1, R)$$
$$\delta(q_2, 0) = (q_3, 0, R)$$

4. In q_3 state, if input is 0 or 1, be in same state, and on B move to final state:

$$\delta(q_3, 1) = (q_3, 1, R)$$
$$\delta(q_3, 0) = (q_3, 0, R)$$
$$\delta(q_3, B) = (q_A, B, R)$$



	0	1	B
$\rightarrow q_0$	$(q_1, 0, R)$	$(q_0, 1, R)$	---
q_1	$(q_2, 0, R)$	$(q_0, 1, R)$	---
q_2	$(q_3, 0, R)$	$(q_0, 1, R)$	---
q_3	$(q_3, 0, R)$	$(q_3, 1, R)$	(q_A, B, R)
q_A	---	---	---

Problem 2: Design a TM to accept strings formed on {0, 1} that end with 000.

Solution: To accept strings ending with 000, the TM would be similar to the one in the example above, but the only difference is, in q_3 state, define moves such that on 0, it is in same state, and on 1, it changes to q_0 state.

1. In state q_0, define transition to ignore the 1's encountered. If 0 is seen, then change to new state, which indicates one 0 is present:

$$\delta(q_0, 1) = (q_0, 1, R)$$
$$\delta(q_0, 0) = (q_1, 0, R)$$

2. In q_1 state, if input is 0, change to new state to indicate substring 00; if input is 1, go back to initial state:

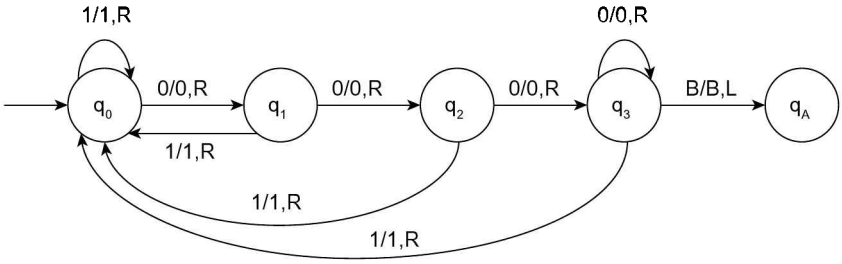
$$\delta(q_1, 1) = (q_0, 1, R)$$
$$\delta(q_1, 0) = (q_2, 0, R)$$

3. In q_2 state, if input is 0, change to new state to indicate substring 000; if input is 1, go back to initial state:

$$\delta(q_2, 1) = (q_0, 1, R)$$
$$\delta(q_2, 0) = (q_3, 0, R)$$

4. In q_3 state, if input is 0 or 1 be in same state, and on B move to final state:

$$\delta(q_3, 1) = (q_0, 1, R)$$
$$\delta(q_3, 0) = (q_3, 0, R)$$
$$\delta(q_3, B) = (q_A, B, R)$$



	0	1	B
$\rightarrow q_0$	$(q_1, 0, R)$	$(q_0, 1, R)$	---
q_1	$(q_2, 0, R)$	$(q_0, 1, R)$	---
q_2	$(q_3, 0, R)$	$(q_0, 1, R)$	---
q_3	$(q_3, 0, R)$	$(q_0, 1, R)$	(q_A, B, R)
q_A	---	---	---

Problem 3: Design a TM for accepting strings of the language defined as $\{ww^r \mid w \in (0+1)^*\}$.

Solution: The given language contains strings which are even palindromes.

For example, abba. To accept such strings, first find the symbol encountered and travel extreme right; and if the same symbol is found, then replace it by blank. Repeat until no more symbol is found.

1. In initial state, if 1 is encountered, change to q_1 state:.

$$\delta(q_0, 1) = (q_1, B, R)$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

2. In q_1 state, travel extreme right until B is seen. On B, change to q_2 state and move left:

$$\delta(q_1, 1) = (q_1, 1, R)$$

$$\delta(q_1, 0) = (q_1, 0, R)$$

$$\delta(q_1, B) = (q_2, B, L)$$

3. In q_2 state, if the input is 1, replace it by B and change to q_5 state:

$$\delta(q_2, 1) = (q_5, B, L)$$

4. In q_5, move left until to extreem left until it finds 'B':

$$\delta(q_5, 1) = (q_5, 1, L)$$

$$\delta(q_5, 0) = (q_5, 0, L)$$

5. In initial state, if input is 0, go q_3 state:

$$\delta(q_0, 0) = (q_3, B, R)$$

6. In q_3 state, travel extreme right until B is seen. On B, change to q_4 state and move left:

$$\delta(q_3, 1) = (q_3, 1, R)$$

$$\delta(q_3, 0) = (q_3, 0, R)$$

$$\delta(q_3, B) = (q_4, B, L)$$

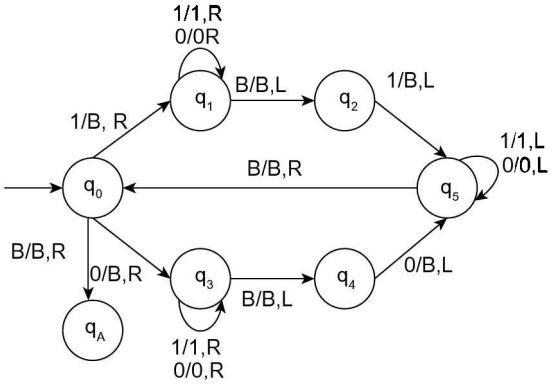
7. In q_4 state, if the input is 0, replace it by B and change to q_5 state:

$$\delta(q_4, 0) = (q_5, B, L)$$

8. Repeat Steps 1–3, or Steps 5–7 and Step 4 till it encounters B:

$$\delta(q_5, B) = (q_0, B, R)$$

$$\delta(q_0, B) = (q_A, B, R)$$



	0	1	B
$\rightarrow q_0$	(q_1, B, R)	(q_3, B, R)	(q_A, B, R)
q_1	$(q_1, 0, R)$	$(q_1, 1, R)$	(q_2, B, L)
q_2	---	(q_5, B, L)	---
q_3	$(q_3, 0, R)$	$(q_3, 1, R)$	(q_4, B, L)
q_4	(q_5, B, L)	---	---
q_5	$(q_5, 0, L)$	$(q_5, 1, L)$	(q_0, B, R)
q_A	---	---	---

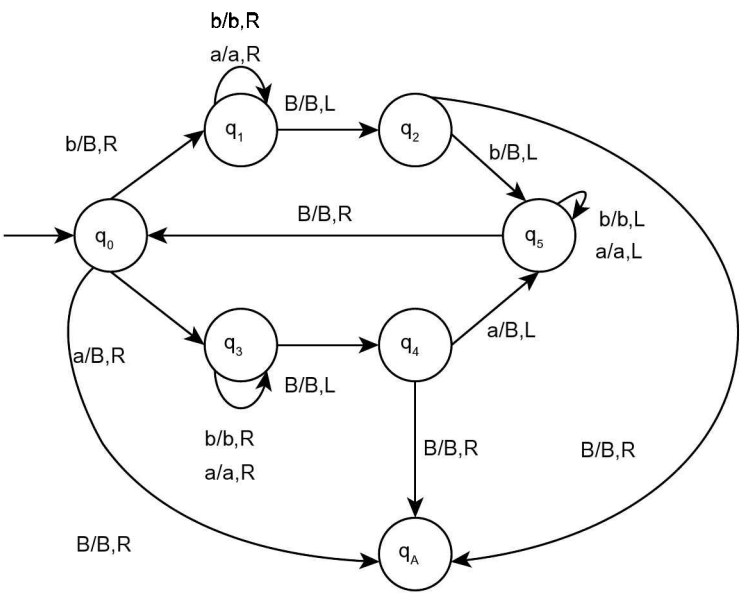
Problem 4: Design a Turing machine for palindrome strings over $\{a, b\}$.

Solution: In this case, size of strings could be even or odd. This problem is similar to the above, and it need to accept strings that are odd and for this the above TM has to be slightly modified.

In state q_2 and q_4, extra transitions are to be added as follows:

$$\delta(q_2, B) = (q_A, B, R)$$

$$\delta(q_4, B) = (q_A, B, R)$$



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

	a	b	B
$\rightarrow q_0$	(q_1, B, R)	(q_3, B, R)	(q_A, B, R)
q_1	(q_1, a, R)	(q_1, b, R)	(q_2, B, L)
q_2	---	(q_5, B, L)	(q_A, B, R)
q_3	(q_3, a, R)	(q_3, b, R)	(q_4, B, L)
q_4	(q_5, B, L)	---	(q_A, B, R)
q_5	(q_5, a, L)	(q_5, b, L)	(q_0, B, R)
q_A	---	---	---

Problem 5: Design a Turing Machine to accept strings of the language defined by $\{a^n b^n \mid n \geq 1\}$.

Solution: Here if the strings are formed with a's followed by b's and if the number of both a's and b's are equal, then they are accepted. The processing can be illustrated with the example as below.

1. In initial state, if a is encountered, change to q_1 state:

$$\delta(q_0, a) = (q_1, x, R)$$

2. In q_1 state, travel extreme right until b is seen. On B, change to q_2 state, change the symbol to y, and move left:

$$\delta(q_1, a) = (q_1, a, R)$$

$$\delta(q_1, b) = (q_2, y, L)$$

3. In q_2, move left until it encounters x:

$$\delta(q_2, a) = (q_2, a, L)$$

$$\delta(q_2, x) = (q_0, x, R)$$

4. Repeat until in q_0 state it encounters y and then change to state q_3:

$$\delta(q_0, y) = (q_3, y, R)$$

5. In q_3, move right until it encounters B:

$$\delta(q_3, y) = (q_3, y, R)$$

$$\delta(q_3, B) = (q_A, B, R)$$

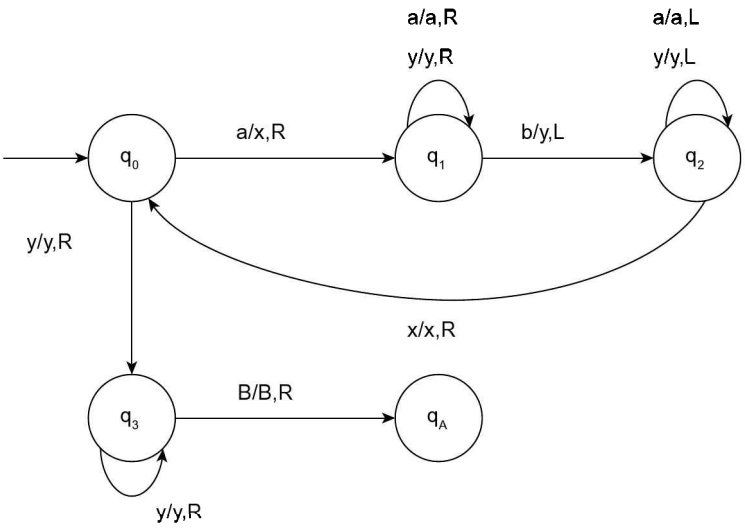
6. To support the repetition, some transitions have to be add at q_1 and q_2:

$$\delta(q_1, y) = (q_1, y, R)$$

$$\delta(q_2, y) = (q_2, y, L)$$

B•aabbB ⊢ Bx•abbB ⊢ Bxa•bbB ⊢ Bx•aybB ⊢ B•xaybB ⊢ Bx•aybB ⊢ Bxx•ybB ⊢ Bxxy•bB ⊢ Bxx•yyB ⊢ Bx•xyyB ⊢ Bxxy•yB ⊢ Bxxyy•B

If the TM halts replacing all a's by x and all b's by y, then the string is accepted. To design this, we need five states as depicted in the following figure:



	a	b	X	Y	B
$\rightarrow q_0$	(q_1, x, R)	---	---	(q_3, y, R)	---
q_1	(q_1, a, R)	(q_2, y, L)	---	(q_1, y, R)	---
q_2	(q_2, a, L)	---	(q_0, x, R)	(q_2, y, L)	---
q_3	---	---	---	(q_3, y, R)	(q_A, B, R)
q_A	---	---	---	---	---

Problem 6: Design a Turing machine where the number of a's and number of b's are equal or, put differently, the language is defined as $L = \{w \mid n_a(w) = n_b(w)\}$.

Solution: In the set of strings given, the number of a's and b's are equal and the elements are in any order. In this case, it is important to remember the element encountered; and if the other element is seen, then it should be replaced. The corresponding TM is shown below.

1. In initial state, if b is encountered, change to q_1 state:

$$\delta(q_0, b) = (q_1, B, R)$$

2. In q_1 state, travel right until a is seen. On a, change to q_3 state and move left:

$$\delta(q_1, x) = (q_1, x, R)$$

$$\delta(q_1, b) = (q_1, b, R)$$

$$\delta(q_1, a) = (q_3, x, L)$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

3. In q_3, move left until it encounters B:

$$\delta(q_3, x) = (q_3, x, L)$$
$$\delta(q_3, b) = (q_3, b, L)$$
$$\delta(q_3, a) = (q_3, a, L)$$
$$\delta(q_3, B) = (q_0, B, R)$$

4. In initial state, if input is a, go q_2 state:

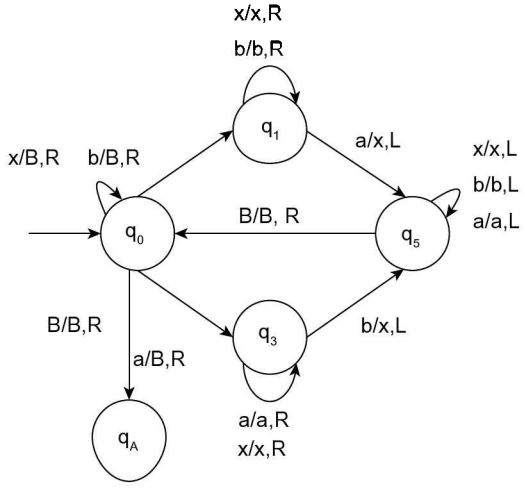
$$\delta(q_0, a) = (q_2, B, R)$$

5. In q_2 state, travel right until b is seen. On b, change to q_3 state and move left:

$$\delta(q_2, x) = (q_2, x, R)$$
$$\delta(q_2, a) = (q_2, a, R)$$
$$\delta(q_2, b) = (q_3, x, L)$$

6. Repeat Steps 1–2 or Steps 4–5 and Step 3 till it encounters B:

$$\delta(q_0, B) = (q_A, B, R)$$



	a	b	x	B
$\rightarrow q_0$	(q_2, B, R)	(q_1, B, R)	(q_0, B, R)	(q_A, B, R)
q_1	(q_3, a, L)	(q_1, b, R)	(q_2, x, R)	---
q_2	(q_2, a, R)	(q_3, x, L)	(q_2, x, R)	---
q_3	(q_3, a, L)	(q_3, b, L)	(q_3, x, L)	(q_0, B, R)
q_A	---	---	---	---

Problem 7: Design a TM to accept strings of the language defined as $\{a^{2n}b^n \mid n \geq 0\}$.

Solution: Here if the strings are formed with a's followed by b's and if the number of a's are twice the number of b's, then they are accepted. The processing can be illustrated with an example below.

1. In initial state, if a is encountered, change to q_1 state:

$$\delta(q_0, a) = (q_1, x, R)$$

2. In q_1 state, if a is encountered, change to q_2 state:

$$\delta(q_1, a) = (q_2, x, R)$$

3. In q_2 state, travel extreme right until b is seen. On B, change to q_3 state, change the symbol to y, and move left:

$$\delta(q_2, a) = (q_2, a, R)$$

$$\delta(q_2, b) = (q_3, y, L)$$

4. In q_3, move extream left until it encounters x:

$$\delta(q_3, a) = (q_3, a, L)$$

$$\delta(q_3, x) = (q_0, x, R)$$

5. In q_0 state, repeat until it encounters y and change to state q_4:

$$\delta(q_0, y) = (q_4, y, R)$$

6. In q_4 move extream right until it encounters B:

$$\delta(q_4, y) = (q_4, y, R)$$

$$\delta(q_4, B) = (q_A, B, R)$$

7. To support the repetition, some transitions have to be add to q_2 and q_3 states:

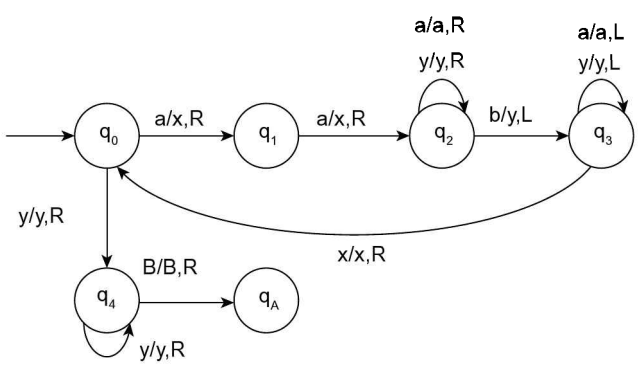
$$\delta(q_2, y) = (q_2, y, R)$$

$$\delta(q_3, y) = (q_3, y, L)$$

B•aaaabbB ⊢ Bx•aaabbB ⊢ Bxx•aabbB …..⊢ Bxxaa•ybB ……⊢ Bx•xaaybB ⊢ Bxx•aaybB ⊢ Bxxx•aybB….. ⊢ Bxxxxy•bB ⊢ Bxxxx•yyB ⊢ Bxxx•xyyB ⊢ Bxxxxy•yB ⊢ Bxxxxyy•B

If the TM halts replacing all a's by x and all b's by y, then the string is accepted.

To design this, we need six states as depicted in the following figure:



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

	a	b	X	Y	B
$\rightarrow q_0$	(q_1, x, R)	---	---	(q_4, y, R)	---
q_1	(q_2, x, R)	---	---	---	---
q_2	(q_2, a, R)	(q_3, y, L)	---	(q_2, y, R)	---
q_3	(q_3, a, L)	---	(q_0, x, R)	(q_3, y, L)	---
q_4	---	---	---	(q_4, y, R)	(q_A, B, R)
q_A	---	---	---	---	---

Problem 8: Let T be the TM defined by the following five-tuple representation where q_0 is initial state and q_A is the accepting state:

$$(q_0, 0, q_0, 1, R)$$
$$(q_0, 1, q_0, 0, R)$$
$$(q_1, B, q_1, B, L)$$
$$(q_1, 0, q_2, 1, R)$$
$$(q_1, 1, q_1, 0, L)$$
$$(q_1, B, q_1, 0, L)$$
$$(q_2, 0, q_2, 0, R)$$
$$(q_2, B, q_A, B, H)$$

For each of the following initial tape, determine the final tape when T halts, assuming that T begins in initial state.

(a) 0000
(b) 10010

***Solution*:** The processing of input is shown using the IDs.

(a) 0000

$Bq_0 0000B \vdash B1q_0000B \vdash B11q_000B \vdash B111q_00B \vdash B1111q_0B \vdash B1111q_1BB \vdash B111q_110BB \vdash B11q_1100BB \vdash B1q_11000BB \vdash Bq_11000BB \vdash Bq_1B0000BB \vdash Bq_1B0000BB$

It is clear that the TM never halts and the string is modified to infinite number of 0's.

(b) 10010

$Bq_010010B \vdash B0q_00010B \vdash B01q_0010B \vdash B011q_010B \vdash B0110q_00B \vdash B01101q_0B \vdash B0110q_11B \vdash B011q_100B \vdash B0111q_20B \vdash B01110q_2B \vdash B01110q_AB$

It computes 2's complement and halts in final state.

Problem 9: Design a TM to accept the strings of the language $L = \{a^i b^j \mid i < j\}$.

Solution: In the strings, a's are followed by b's with number of a's less than the number of b's.

The TM halts replacing all a's by x and all b's by y. On the initial state, if it sees y, then change to new state and in that state if it sees at least one b, then the string is accepted.

The transitions would be similar to those in Example 9, but the modification to be done is at state q_3 is as follows:

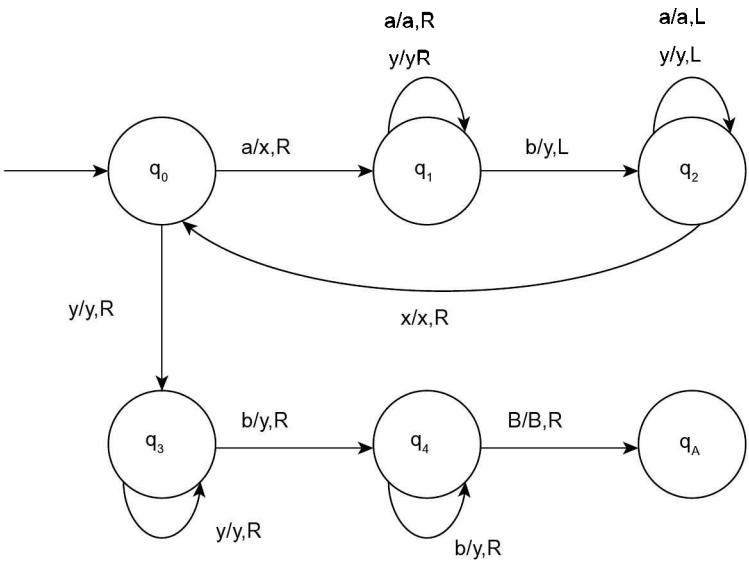
$$\delta (q_3, y) = (q_3, y, R)$$

$$\delta (q_3, b) = (q_4, y, R)$$

and, for q_4, transitions are to be added as

$$\delta (q_4, b) = (q_4, y, R)$$

$$\delta (q_3, B) = (q_A, B, R)$$



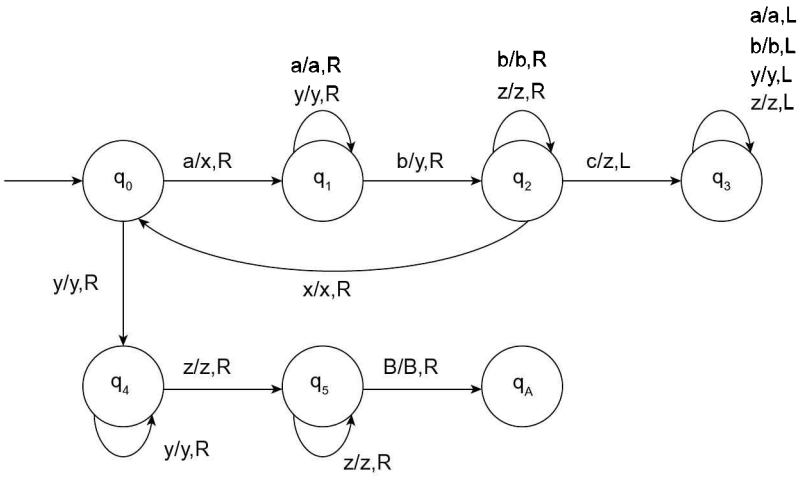
	a	b	X	Y	B
$\rightarrow q_0$	(q_1, x, R)	---	---	(q_3, y, R)	---
q_1	(q_1, a, R)	(q_2, y, L)	---	(q_1, y, R)	---
q_2	(q_2, a, L)	---	(q_0, x, R)	(q_2, y, L)	---
q_3	---	(q_4, y, R)	---	(q_3, y, R)	---
q_4	---	(q_3, y, R)	---	---	(q_A, B, R)
q_A	---	---	---	---	---

Problem 10: Design a TM for accepting strings of the language $L = \{a^n b^n c^n \mid n \geq 0\}$.

Solution: In the strings, a's are followed by b's and these are followed by c's. The number of a's, b's and c's are equal.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

To design this, we need 7 states as the present procedure is similar to that for $\{a^n b^n \mid n \geq 0\}$ with extra state added to take care of the number of c's. The corresponding transition diagram is shown below:

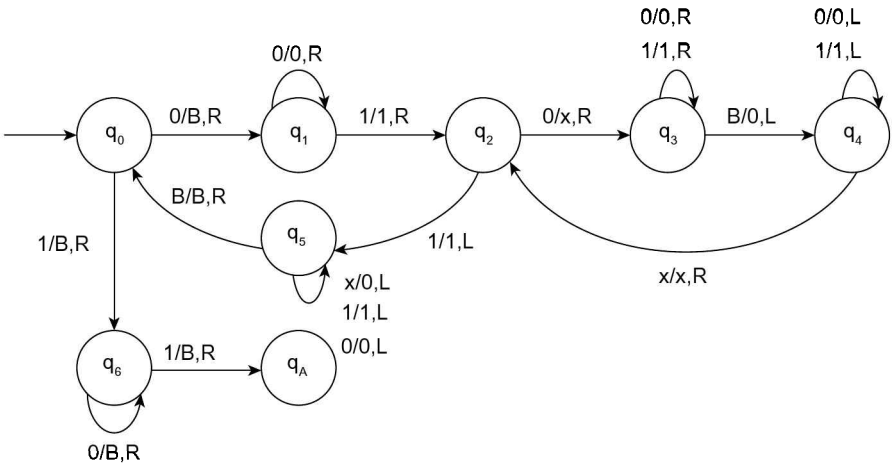


	a	b	c	x	Y	z	B
$\rightarrow q_0$	(q_1, x, R)	---	---	---	(q_4, y, R)	---	---
q_1	(q_1, a, R)	(q_2, y, R)	---	---	(q_1, y, R)	---	---
q_2	---	(q_2, b, R)	(q_3, z, L)	---	---	(q_2, z, R)	---
q_3	(q_3, a, L)	(q_3, b, L)	---	(q_4, y, R)	(q_3, y, L)	(q_3, z, L)	---
q_4	---	---	---	---	(q_4, y, R)	(q_5, z, R)	---
q_4	---	---	---	---	---	(q_5, z, R)	(q_A, B, R)
q_A	---	---	---	---	---	---	---

Problem 11: Design a TM for multiplication of two integers.

Solution: For multiplying two numbers m and n, place $0^n 10^m 1$ on the input tape and design the system such that it first replaces first occurrence of '0' by blank and for each occurrence of '0' of second integer write '0' at the end. Once all 0's of second integer are replaced with X, now remodify all X's to 0's and move extream left to find 'B'. Repeat this process until all 0's of first integr are replaced with B. Now replace all 0's of second number by blank and halt. The processing is illustrated with example 2×3 in the following:

•0010001B ⊢ B•010001B ⊢ ....B01•0001B ⊢ B01x•001B ⊢..... B01x00•10B ⊢..... B01• x0010B ⊢ B01x•0010B ⊢ B01xx•010B...⊢ B01xx01•00B....⊢ B01xxx•1000B ⊢.... B0•10001000B ⊢ ....•B010001000B .. ⊢ BB•10001000B ⊢... BB1xxx•1000000B ⊢... B•B10001000000B ⊢ ..... BBB•B000000B



	0	1	x	B
$\rightarrow q_0$	(q_1, B, R)	(q_6, B, R)	---	---
q_1	$(q_1, 0, R)$	$(q_2, 1, R)$	---	---
q_2	(q_3, x, R)	$(q_5, 1, L)$	---	---
q_3	$(q_3, 0, R)$	$(q_3, 1, R)$	---	$(q_4, 0, L)$
q_4	$(q_4, 0, L)$	$(q_4, 1, L)$	(q_2, x, R)	---
q_5	$(q_5, 0, L)$	$(q_5, 1, L)$	$(q_5, 0, L)$	(q_0, B, R)
q_6	(q_3, B, R)	(q_A, B, R)	---	---
q_A	---	---	---	---

Problem 12: Draw a TM for the function $f(x) = x^2$.

Solution: The solution for this problem is the same as multiplication where the input would be $0^x\ 10^x1$.

Problem 13: Draw a TM for the function $f(x) = 2x$.

Solution: The solution for this problem is the same as multiplication where the input would be $0^2 10^x 1$

Problem 14: Draw a TM that computes the function $f(x) = x + 2$.

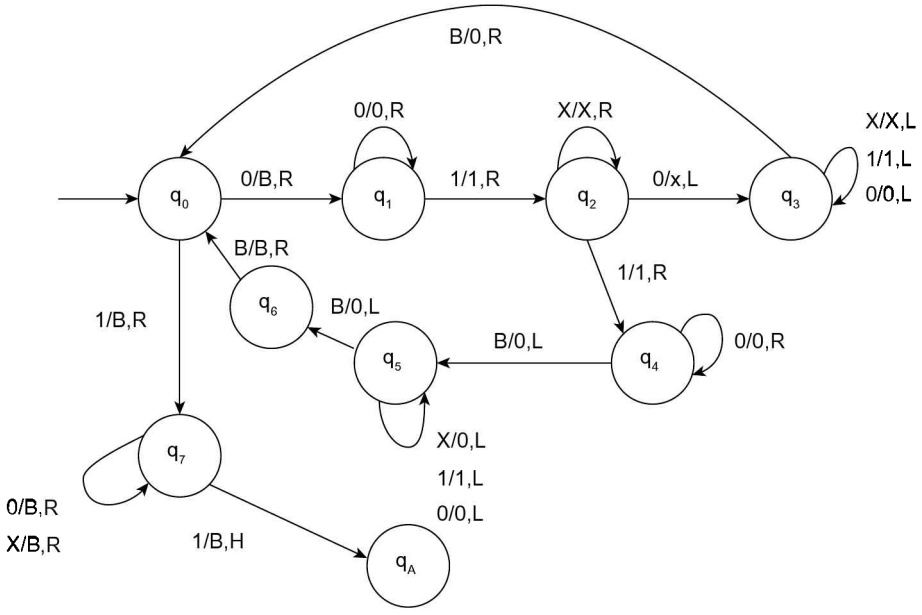
Solution: To compute $x + 2$, place the input as $0^2 10^x$ and use the procedure defined for the addition of two numbers as in Example 4.

Problem 15: Draw a TM that computes integer division defined as $f(x) = x/2$.

Solution: To compute the division function m/n, place the input as $0^m 10^n 1$ where m is dividend and n is divisor. First replace 0 of first integer by B and replace the corresponding 0

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

by x in second integer. Repeat this process until all 0's of second integer are replaced by x, Once this is done, replace the first occurrence of the right blank by 0. Move to the left end and replace the left blank by zero, and repeat the complete procedure until, in initial state, 1 is encountered.



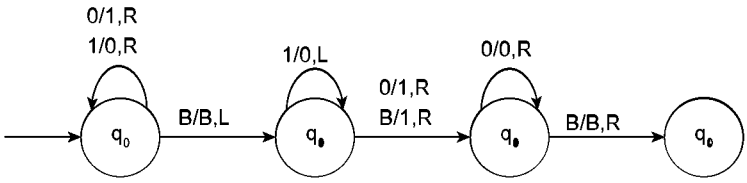
	0	1	X	B
$\rightarrow q_0$	(q_1, B, R)	(q_7, B, R)	---	---
q_1	$(q_1, 0, R)$	$(q_2, 1, R)$	---	---
q_2	(q_3, X, L)	$(q_4, 1, R)$	(q_2, X, R)	---
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	(q_3, X, L)	$(q_0, 0, R)$
q_4	$(q_4, 0, R)$	---	---	$(q_5, 0, L)$
q_5	$(q_5, 0, L)$	$(q_5, 1, L)$	(q_5, X, L)	$(q_6, 0, L)$
q_6	---	---	---	(q_0, B, R)
q_7	(q_7, B, R)	(q_A, B, H)	(q_7, B, R)	---
q_A	---	---	---	---

Problem 16: Draw a TM that gives two's complement for the given binary representation.

Soution: Two's complement is computed by first computing one's complement and then adding 1 to it. For example,

Binary number	One's complement	Two's complement
1001	0110	0111
0011	1100	1101
1000	0111	1000

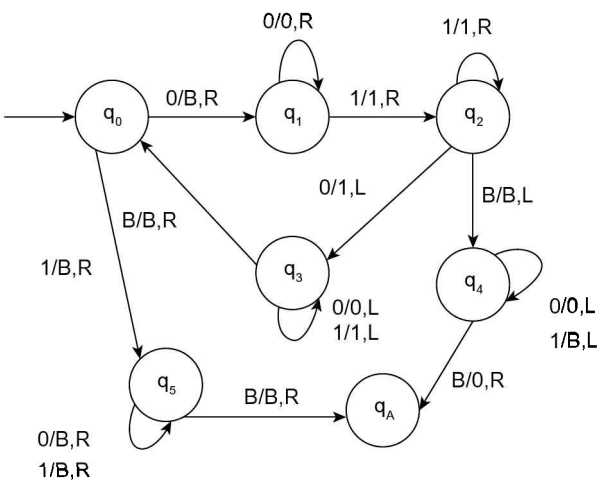
By seeing these few examples, it is clear that the machine has to be designed such that it first complements every bit moving left to right, travels back complementing each bit until it sees first occurrence of 0, and then goes to the right end and halts.



	a = 0	a = 1	a = B
q_0	$(q_0, 1, R)$	$(q_0, 0, R)$	(q_1, B, L)
q_1	$(q_2, 1, R)$	$(q_1, 0, L)$	$(q_2, 1, R)$
q_2	$(q_A, 0, R)$	---	(q_A, B, R)
q_A	----	----	----

Problem 17: Draw a TM for the function f (m, n) = m − n for m ≥ n and zero for m < n.

Solution: To compute this function, the TM starts with the tape content as 0^m10^n. After processing, the TM halts with the 0^{m-n} on its tape. The process is depicted with the following diagram:



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

	0	1	x	B
q_0	(q_1, B, R)	(q_5, B, R)	---	---
q_1	$(q_1, 0, R)$	$(q_2, 1, R)$	---	---
q_2	$(q_3, 1, L)$	$(q_2, 1, R)$	---	(q_4, B, L)
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	---	(q_0, B, R)
q_4	$(q_4, 0, L)$	(q_4, B, L)	---	$(q_A, 0, R)$
q_5	(q_5, B, R)	(q_A, B, R)	(q_5, B, R)	---
q_6	(q_6, x, L)	(q_6, x, L)	(q_6, x, L)	(q_5, B, R)
q_A	---	---	---	---

Summary

1. TM is an abstract machine with the power of a real computer. It consists of a finite control and an infinite tape to hold the input. The contents on the tape can be modified by the reading head.
2. TMs are used as language recognizer, language enumerator and as a computing machine.
3. Language generated by TM is a recursively enumerable language.
4. Multi-tape systems can be used to simulate working process of a RAM in a computer.
5. Counter machine can be implemented using Multi-stack system.
6. Non-deterministic TMs are more powerful than deterministic TM and have a finite number of choices at the next move.
7. Storage in finite control technique can be used when there is a need to remember small amount of information in finite control while processing the input.
8. Multi-track system is a useful technique where processing requirements are complex and many components are to be remembered at the same time.

Short Answers

1. What are the various techniques for TM construction?

   Answer:

   - storage in finite control
   - multiple tracks
   - checking off symbols
   - shifting over
   - subroutines

2. What are different types of TM?

   Answer:

   - two-way finite tape TM
   - multitape TM
   - non-deterministic TM

- multi-dimensional TM
- multi-head TM

3. Design a TM to perform proper subtraction.
   Answer: See Problem 17.

4. Design a TM to accept the language $L = \{0^n1^n \mid n >= 1\}$.
   Answer: Similar to Problem 05.

5. Explain how a TM can be used to determine whether a given number is prime or not?
   Answer: Explained in Example 6.5.

6. What is a recursively enumerable language?
   Answer: The language for which the TM halts for the words of the language, and may or may not halt for the words that are not in the language is said to be a recursively enumerable language.

7. What is a multi-tape TM?
   Answer: A multi-tape TM consists of a finite control with k-tape heads and k-tapes; each tape is infinite in both directions. On a single move, depending on the state of finite control and symbol scanned by each of the tape heads, this machine can change state, print a new symbol on each cell scanned by the tape head, move each of its tape head independently one cell to the left or right, or remain stationary.

8. What is a multi-dimensional TM?
   Answer: This device has a finite control, but the tape consists of a k-dimensional array of cells infinite in all 2k directions, for some fixed k. Depending on the state and symbol scanned, the device changes state, prints a new symbol and moves its tape head in one of the 2k directions, either positively or negatively, along one of the k-axes.

9. When is a recursively enumerable language said to be recursive? Is it true that the language accepted by a non-deterministic TM is different from the recursively enumerable language?
   Answer: A language L is recursively enumerable (r.e.) if there is a TM that accepts L, and it is recursive if there is a TM that recognizes L. Thus r.e. language is Turing acceptable, and a recursive language is Turing decidable.
   The language accepted by non-deterministic TM is the same as recursively enumerable language.

10. What is Church's hypothesis?
    Answer: The notion that a computable function can be identified with the class of partial recursive functions is known as Church's hypothesis or Church–Turing thesis. The TM is equivalent to the digital computer, in computing power.

11. Define a Turing machine?
    Answer: A TM is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where

    Q is a finite set of states.

    Σ is the input alphabet, which never includes blanks.

    Γ is the tape alphabet, which always includes blanks. Moreover, every input symbol is in Γ.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

δ: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function, where L and R are directions, telling the machine head in which direction to go in a step.

$q_0 \in Q$ is the start state.

B is a special symbol indication, for a blank cell.

F is the set of final states and the TM halts on reaching these states.

Fill in the Blanks

1. The languages accepted by a TM are _____.
2. Well-structured problems are always decidable (True/False).
3. The string of the form $\alpha q \beta$ gives _____ of the system where the next input is _____.
4. Algorithms written for the TM that always halts are said to be _____.
5. _____ languages are a subset of recursive languages.
6. _____ machine can accept languages that are accepted by two-stack pushdown automaton.
7. _____ machines influenced the concept of stored program.
8. Verification of the syntax of a statement is done faster with TM than with PDA (True/False).

Answers

1. Recursively enumerable language
2. True
3. Instantaneous description, leftmost symbol of β
4. Decidable
5. Recursively enumerable
6. Turing
7. Universal Turing
8. False

Objective Question Bank

1. Choose the correct statements:
   a) A is recursive if both A and its complement are accepted by TMs
   b) The r.e. sets are closed under complementation.
   c) The recursive sets are closed under complementation
   d) The r.e. sets and recursive sets are closed under union.

2. The language $L = \{a^n b^n \mid n >= 1\}$ is not
   a) regular set b) CFL c) CSL d) r.e. set

3. The following languages are closed under infinite union:
   a) Recursive sets b) Recursively enumerable sets
   c) All formal languages d) none of the above

4. Recursive languages are
   a) a proper superset of CFLs b) always recognizable
   c) also called type 0 languages d) recognizable by Turing machines

5. In which of the cases stated below is the following statement true? 'For every non-deterministic machine M_1, there exists an equivalent deterministic machine M_2 recognizing the same language'.
   a) M_1 is a finite automaton b) M_1 is a PDA.
   c) M_1 is a Turing machine d) For no machine M_1, the above statement is true

6. Which of the following statements are true?
   a) The union of two recursive languages is recursive
   b) The infinite union of recursive languages is recursive
   c) The language $\{a^n/n \text{ prime}\}$ is not regular
   d) Regular sets are closed under infinite union and so are DCFLS

7. Choose the incorrect statements:
   a) Every subset of a countable set is countable
   b) The class of TMs is countable
   c) The class of LBA is countable
   d) The class of DPDA is not countable

8. Which of the following conversion is not possible (algorithmically)?
   a) Regular grammar to Context free grammar
   b) Non-deterministic FSA to deterministic FSA
   c) Non-deterministic PDA to deterministic PDA
   d) Non-deterministic TM to deterministic TM

9. Consider the grammar with the following productions:
   $S \rightarrow aab / bac / aB$
   $S \rightarrow aS / b$
   $S \rightarrow abb / ab$
   $aB \rightarrow bdb$
   The above grammar is
   a) context free b) regular c) context sensitive d) LR(K)

10. Which of the following statements is false:
    a) The halting problem of TMs is undecidable
    b) Determining whether a CFG is ambiguous is undecidable
    c) Given two arbitrary CFGs G_1 and G_2, it is undecidable whether $L(G_1) = L(G_2)$
    d) Given two regular grammars G_1 and G_2, it is undecidable whether $L(G_1) = L(G_2)$

11. If set A is the set of recursive languages and B is the set of recursively enumerable languages, then
    a) A is subset of B
    b) B is subset of A
    c) A and B are same sets
    d) A and B are disjoint sets

12. Regarding the power of recognition of languages, which of the following statements is false?
    a) The NDFA are equivalent to DFA.
    b) NPDA are equivalent to DPDA

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

c) NDTM is equivalent to DTMs
d) Multi-tape TMs are equivalent to single-tape TMs

13. Choose the correct statements.
a) The problem as to whether a TM M accepts w is decidable.
b) It is decidable of a TM M if it starts on blank
c) It is undecidable if a TM ever prints a symbol
d) It is undecidable if a TM ever enters a particular state

14. Consider the language L defined as follows:

$$L = \begin{cases} (0+1)^* & \text{if } P = NP \\ \pi & \text{otherwise} \end{cases}$$

Which of the following statements is true?
a) L is recursive
b) L is recursively enumerable, but not recursive
c) L is not recursively enumerable
d) Whether L is recursive or not will be known after we find out if P = NP.

15. A single tape TM M has two states q_0 and q_1, of which q_0 is the starting state. The tape alphabet of M is {0, 1, B}, and its input alphabet is {0, 1}. The symbol B is the blank symbol used to indicate the end of an input string. The transition function of M is described in the following table:

	a = 0	a = 1	a = B
q_0	$(q_0, 1, R)$	$(q_0, 1, R)$	Halt
q_1	$(q_1, 1, R)$	$(q_0, 1, L)$	(q_0, B, L)

Which of the following statements is true about M?
a) M halts on any string in (0 + 1)
b) M does not halt on any string (00 + 1)*
c) M does not halt on all strings ending in a 0.
d) M does not halt on all strings ending in a 1

16. A FSM can be considered to be a TM of
a) finite tape length, rewinding capability and unidirectional tape movement
b) finite tape length, without rewinding capability and unidirectional tape movement
c) finite tape length, without rewinding capability and bi-directional tape movement.
d) finite tape length, rewinding capability and bi-directional capability.

17. TM is more powerful than FA because
a) tape movement is confined to one direction
b) it has no finite state control
c) it has the capability to remember arbitrarily long sequences of input symbols
d) none of the above.

18. A PDM behaves like a FA when the number of auxiliary memory it has, is
a) 0 b) 1 c) 2 d) none of the above

19. A PDM behaves like a TM when the number of auxiliary memories it has, is
    a) 0 b) 1 or more
    c) 2 or more d) none of the above

20. Choose the correct statements.
    a) A FSM with one stack is more powerful than a FSM with no stack
    b) A FSM with two stacks is more powerful than a FSM with one stack
    c) A FSM with three stacks is more powerful than a FSM with n stacks
    d) All of the above comments are true

21. A language is said to be recursive if
    a) both the language and its complement are not recursively enumerable.
    b) it is recursively enumerable, while its complement is not recursively enumerable.
    c) it is not recursively enumerable, while its complement is recursively enumerable.
    d) both the language and its complement are recursively enumerable.

22. If there exists a language L for which there exists a TM, T, that accepts every word in L, and either rejects or loops for every word that is not in L, then the language is said to be
    a) recursive b) recursively enumerable
    c) NP hard d) none of the above

23. Choose the incorrect statements:
    a) $L = \{a^n b^n a^n\} / n = 1, 2, 3.....\}$ is r.e.
    b) Recursively enumerable languages are closed under union
    c) Every recursive language is recursively enumerable
    d) Recursive languages are closed under intersection

24. Choose the correct statements:
    a) Set of recursively enumerable languages is closed under union
    b) If a language and its complement are both regular, then the language must be recursive
    c) Recursive languages are closed under complementation
    d) All the above.

25. Pick the correct answers.
    Universal TM influenced the concept of
    a) stored-program computers
    b) interpretive implementation of programming languages
    c) computability
    d) none of the above

26. The statement 'A TM cannot solve halting problem' is
    a) true b) false
    c) still an open question d) none of the above

27. If there exists a TM which when applied to any problem in a class of recurssively enumerable sets, terminates if the computer answer is yes, and may or may not terminate otherwise, is said to be
    a) stable b) unsolvable c) partially solvable d) unstable

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

28. The vernacular language English, if considered as formal language, is a
    a) regular language
    b) context free language
    c) context sensitive language
    d) none of the above
29. Which of the following statements is incorrect?
    a) Every recursively enumerable language is recursive.
    b) Complement of a recursive language is recursive.
    c) Recursive languages are said to be decidable.
    d) If L is a recursive language, then TM halts for $w \in L$ and rejects $w \notin L$.

Answers

1. a, c	2. a	3. d	4. d	5. a, c	6. a, c	7. d	8. c	9. c	10. b
11. a	12. b	13. a	14. a	15. a	16. b	17. c	18. a	19. c	20. d
21. d	22. b	23. d	24. d	25. a	26. c	27. c	28. b	29. a	

Exercises

1. What is a Turing machine? With a neat sketch, explain the working of a Turing machine.
2. What are Turing machine and multi-tape Turing machine? Explain the general structure of a multi-tape Turing machine.
3. Design a Turing machine to accept the language $L = \{a^n. b^n. c^n: n >= 1\}$. Also give the graphical representation and ID for the input 'aabbcc'.
4. Prove that every language accepted by a multi-tape Turning machine is recursively enumerable.
5. Discuss how to use a computer to stimulate a Turing machine, and compare the running times of the computer and the Turing machine.
6. Design a Turing machine for N * (N – 1) given N.
   Hint: Same as multiplication.
7. Design a Turing machine for recognizing the language $\{0^n1^{3n} \mid n > 1\}$.
8. Design a Turing machine for recognizing the language $\{wcw^R \mid w \in (0 + 1)^*\}$.
9. Design a Turing machine for recognizing the language $\{0^i 0^j 0^k \mid i < j < k\}$.
10. Design a Turing machine for recognizing the language $\{0^i 0^{2j} 0^{4k} \mid \text{where } i = j = k\}$.
    Hint: The length of the string should correspond to $i + 2i + 4i = 7i$ for all values of i. Hence check if total length is multiple of 7.
11. Design a Turing machine for computing the function defined by $f(x, y) = x(x + y)$
    Hint: Input is placed as $0^x10^x10^y$.
12. Design a Turing machine for finding N!
    Hint: Place the input as 0^n10^{n-1}. On multiplying we get as $0^{n-1}10^{nm}$. Modify this as $0^{mn}10^{n-1}$.

Undecidability and Computability

7

- Decidability and computability are the basic factors to analyse the algorithms.

In this chapter, we try to understand the importance and procedure to check whether the problem is solvable or not. As we know, computers have capabilities and limitations, and it becomes essential to have a mechanism by which we can identify an unsolvable problem, so that the problem can be altered or simplified before finding an algorithmic solution.

In the computability theory and the computational complexity theory, a **decision problem** is like a question in some formal system with a yes or no answer, depending on the values of some input parameters. For example, In two given numbers x & y, does x evenly divide y?' is a decision problem. The answer can be either 'yes' or 'no', and it depends upon the values of x and y.

7.1 Decision Problems

Decision problems are closely related to function problems, which can have answers that are more complex than a simple 'yes' or 'no'. A corresponding function problem is 'given two numbers x and y, how to program for x divided by y?' This is related to optimization problems, which is concerned with finding the **best** answer to a particular problem.

A method for solving a decision problem given in the form of an algorithm is called a **decision procedure** for that problem. A decision procedure for the decision problem 'given two numbers x and y, does x evenly divide y?' would give the steps for determining whether x evenly divides y. One such algorithm is by long division, taught to many school children. If the remainder is zero, the answer produced is 'yes'; otherwise, it is 'no'. A decision problem that can be solved by an algorithm, such as the example of divisibility discussed above, is called **decidable**.

The field of Computational Complexity categorizes the decidable decision problems depending on difficulty with which they are solved. 'Difficulty', in this sense, is described in terms of the computational resources needed by the most efficient algorithm for a certain problem.

The field of Recursion Theory, categorizes undecidable decision problems by Turing degree, which is a measure of the non-computability inherent in any solution.

Definition 1: A decision problem is any arbitrary yes-or-no question on an infinite set of inputs. Because of this, it is usual to define the decision problem equivalently as follows:

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] konkurcomputer.ir

The set of inputs for which the problem returns **yes**. These inputs can be natural numbers, but values of some other kind, such as strings of a formal language, can also be inputs.

Using some encoding, such as **Gödel numbers** (Gödel numbering is a function that assigns to each symbol and well-formed formula of some formal language, a unique natural number, called its Gödel number), the strings can be encoded as natural numbers. Thus, a decision problem informally phrased in terms of a formal language is also equivalent to a set of natural numbers. To keep the formal definition simple, it is phrased in terms of subsets of the natural numbers.

Formally, a **decision problem** is a subset of the natural numbers. The corresponding informal problem is that of deciding whether a given number is in the subset.

A classic example of a decidable decision problem is the set of prime numbers. It is possible to effectively decide whether a given natural number is a prime by testing every possible non-trivial factor.

7.2 Decidability and Decidable Languages

A decision problem 'A' is called **decidable** or **effectively solvable** if 'A' is a recursive set. A problem is called **partially decidable, solvable**, or **provable** if 'A' is a recursively enumerable set. Partially decidable problems and other problems that are not decidable are called **undecidable**.

In this section, we give examples of languages that are decidable by algorithms.

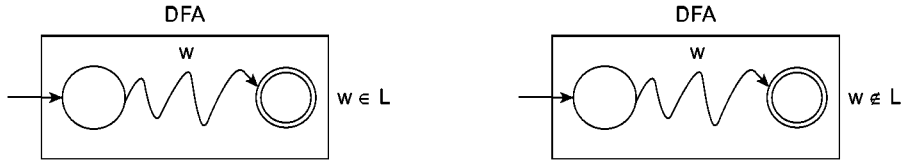
7.2.1 Decidable Problems Concerning Regular Languages

We can find some algorithms for testing whether a finite automaton accepts a string, whether the language of a finite automaton is empty and whether two finite automata given are equivalent.

Example 7.1 A DFA defines a decidable language.

Solution: We need to construct a turing machine M such that M = <B, a> where B is the description of DFA and w is a string to be processed.

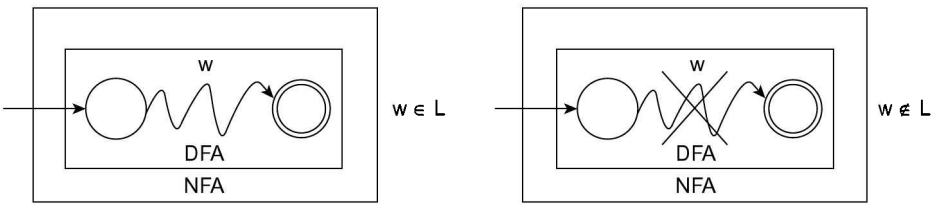
1. Simulate B on input w.
2. If the simulation ends in an accept state, accept. If it ends in a non-accepting state, reject.



To construct such a turing machine, the five components of DFA Q, Σ, δ, q_0 and F can be used in the TM. When M receives its input, M first determines whether machine properly represents a DFA B and a string w. If not, M rejects. Then M carries out simulation directly. It keeps track of B's current state and B's current position in the input w by writing the information

down on its tape. Initially, B's current state is q_0, and B's current input position is the leftmost symbol of w. The states are updated according to a specified transition function δ. When M finishes processing the last symbol of w, M accepts the input if B is in an accepting state, and M rejects the input if B is in a non-accepting state.

Example 7.1 Language accepted by NFA is decidable.



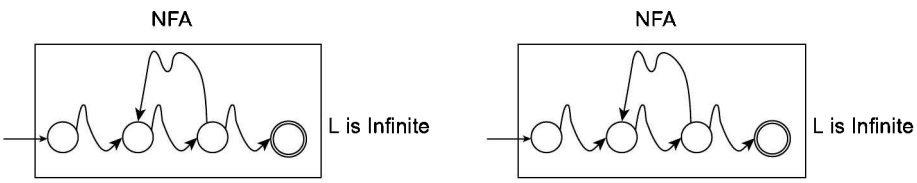
Solution: We need to construct a turig machine N such that N = <B, a>, where B is the description of NFA and w is a string.

1. Convert Nondeterministic FA B to an equivalent DFA C, using NFA-to-DFA-conversion procedure.
2. Run turing machine M to simulate C on input w, as explained in previous example.
3. If M accepts the string w; then B accepts w, otherwise rejects.

Example 7.1 Language generated by regular expression is a decidable language.

Solution: We can construct a turing machine P such that P = <R, w>, where R is a regular expression and w is the string.

1. Convert regular expression R to an equivalent NFA A, by using RE-to-NFA- conversion procedure.
2. Run turing machine M on input <A, w>.
3. If M accepts w then ρ accepts, otherwise rejects.

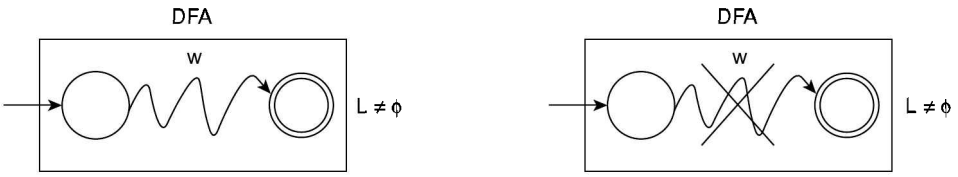


Example 7.4 Emptiness of DFA is a decidable language.

Solution: We can construct a turing machine T such that T = <A, w> where A is a DFA.

1. Mark the start state of A.
2. Mark any state that has a transition coming into it from any state that is already marked.
3. Repeat step-2 until no new states get marked.
4. If no accept state is marked, then accept; otherwise reject.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]



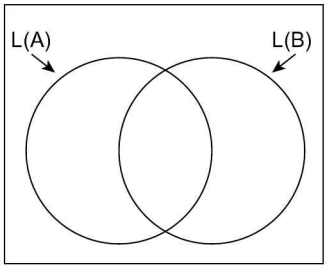
Example 7.4 Equivalence of DFA's is a decidable language.

Solution: To prove this, we construct a new DFA C from A and B, where C accepts only those strings that are accepted by either A or B, but not by both. Thus if A and B recognize the same language, C will accept nothing. The language of C is

$$L(C) = (L(A) \cap (L(B))') \cup ((L(A))' \cap L(B))$$

We can construct a turing machine F such that F = <A, B> where A and B are DFAs.

1. Construct a DFA C as described. Mark the start state of A.
2. Run TM T as explained in previous example on input <C>.
3. If T accepts, then accept; otherwise reject.



7.2.2 Decidable Problems Concerning Context Free Languages

In this section, we describe the algorithms that can be used to determine whether the given CFG can generate the given string or to find whether the given CFG is empty.

Example 7.4 Language of context free grammar G is a decidable.

Solution: We need to find whether the given string can be generated by the grammar G. One approach that can be used is to generate all the strings and check whether the given string w is generated. But this approach does not work, sometimes the turing machine may not halt. It may enter into an infinite loop trying to recognize rather than decide and report whether it is valid or not.

Second procedure is to convert the given grammar to CNF, so that for a string w of length n, there would be at most 2^{n-1} steps to generate the string. Hence, it ensures that

the turing machine would halt after generating strings using at most 2^{n-1} steps. It can also decide and say whether the string is valid or not.

TM for this can be defined as S = <G, w> where G is the given grammar and w is the string given:

1. Convert the G to CNF.
2. List all derivations of the grammar starting from 1 to a maximum of 2^{n-1} steps, where n is the length of w.
3. If any of these derivations correspond to w, then accept; otherwise, reject.

Example 7.4 Emptiness of Context free grammar G, is decidable.

Solution: Consider the given grammar G and to check if the grammar is empty simple approach is to write an algorithm that may go through all possible w's one by one. But this may go into an infinite loop.

Emptiness of the language can be found out in finite amount of time. Start with the start symbol and check whether it can generate the string of terminals. To do this in simple way, first mark all the terminals, then scan all the rules and mark the variables that can be replaced by terminal or by a variable that is already marked. Repeat until no new symbols are marked. If the start symbol is marked, then the language of the grammar is not empty.

Let R be the TM given by R = <G> where G is the given context free grammar.

1. Mark all terminal symbols of the grammar.
2. Repeat until no new variables are marked.
3. Mark any variable A of G if the rule for A is of the form $A \rightarrow X_1X_2X_3 \ldots X_n$ where all X_i's are already marked.
4. If the start symbol is not marked, then accept; otherwise, reject.

7.3 Halting Problem

In the previous section, we stated that the problems are said to be decidable if there exists an algorithm that can solve the problem in finite amount of time. There are some specific problems that are algorithmically unsolvable. Although the computers are so powerful, there are some limitations. For example, computers cannot easily avoid crashing, or rather, they cannot predict when they are about to crash and avoid it. In this section, we try to understand a few problems that are unsolvable and learn techniques for proving unsolvability.

Turing machines

A Turing machine is an extremely powerful model of a computing device, and could be programmed to solve many problems. The TM is a theoretical computer, which works by storing data on a tape of infinite length and moving back and forth along it according to the program. Even the program itself can be stored on the tape, and these machines are called Universal TM. Turing programs either halt when they have arrived at the results or else they continue for ever. No Turing program can be said to have produced any result until it has halted.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

7.3.1 The Halting Problem for Turing Machines

Halting problem of the TM is formulated as follows: Given an arbitrary TM and an arbitrary input for the machine, will the given machine halt on the given input?

In other words, Turing wondered whether it would be possible to write a TM program that would take two inputs <program P, input i> and would answer 'Yes' if the TM halts when executing program P on input i, and would answer 'No' if it wouldn't halt, that is, if it would loop for ever.

Solution can be found by giving a description number to every possible TM program, so that no possible programs is left out. One way to figure out if there is a program that can solve the halting problem would be to look through all the whole numbers, interpreting each as the description number of a TM program, and checking to see if it is the program that solves the halting problem.

Of course, this is completely impractical. But Turing realized that if we could prove that no whole number was the right one, then we would know that no program to solve the halting problem exists.

Theorem 1: A TM is recognizable, but not decidable. A recognizer of a TM is called the Turing Universal Machine U, where U = <M, w>, where M is a TM and w is a string

1. Simulate M on w;
2. If M ever enters its accept state, *accept*; if M ever enters its reject state, *reject*'.

Note: U is a universal because it simulates any other TM from its description.
Important points to understand here are

1. All problems may be solvable, but all are not decidable.
2. Since a TM is undecidable, to solve this problem, we need to expand the problem-solving methodology by a new method for proving undecidability.

7.4 Diagonalization Method

The method used to prove the undecidability of the halting problem is proposed by Georg Cantor in 1873, and this method is called diagonalization. Suppose there are two sets to be compared. Suppose set one is the set of even numbers less than 20. Let the other set be the set of multiples of 2 till 10. To find whether these sets are equal, simple method is to count the number of elements in both the sets and find the answer. For finite sets, it is easy to find the answer, but for infinite sets counting the elements is a difficult task.

Cantor proposed a solution to this problem. The two sets would be of same size if every element of one set can be paired with the elements of the other set. This kind of solution does not require counting of elements in either set. The same procedure can be extended to infinite sets.

Definition 2: Assume two sets A and B, and let function f: A $\rightarrow$ B. If f is one-to-one and onto, i.e, if f(a) $\neq$ f(b) or if every element of B is mapped with element of A such that f(a) = b $\forall$ a $\in$ A,b $\in$ B. This correspondence can be used to find solutions to problems corresponding to size.

Example 7.3 Let N be the set of natural numbers {1, 2, 3, 4,.....}, and let E be the set of even numbers {2, 4, 6,.....}. Are these sets of same size

Solution: Using Cantor's definition of size, we can say that these two sets are of same size. The relation between N and E can be given by a function F(n) = 2n. It appears that E is a subset of N, but pairing each member of N with its own member if E is possible. Hence, these sets are of same size.

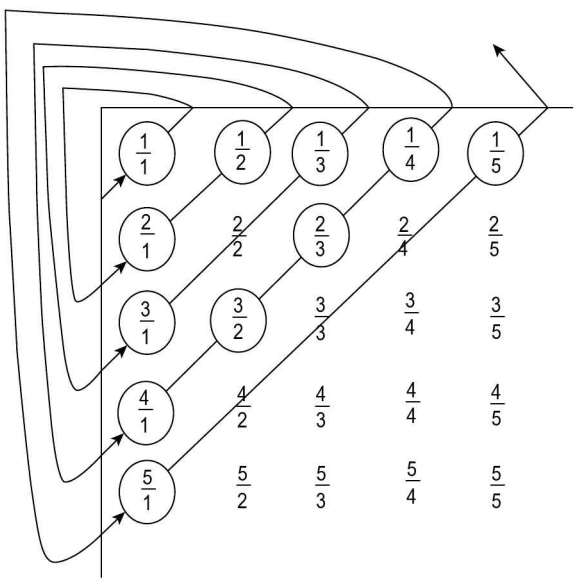
Example 7.4 Can the natural numbers set be compared with set of rational numbers

$Q = \left\{ \frac{m}{n} \mid m, n \in N \right\}$, Justify.

Solution: The set Q appears to be larger than N, but these two sets are of same size according to Cantor's definition. We give a correspondence with N to show that Q is countable. To show this

- Pair the first element on the list with number 1 in N.
- Pair each consecutive element on the list with successive elements in N.
- Ensure that every member of Q appears only once on the list.

To construct a unique list Q, we make use of the infinite matrix containing all positive elements as shown below.



In the above infinite matrix, the ith row contains all numbers with numerator as I, and the jth column has all the numbers with denominator as j. So he number i/j is in ith row and jth column.

List is formed by reading the elements diagonally skipping the elements which give same value.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

In first diagonal $\frac{1}{1}$ is selected

In second diagonal $\frac{2}{1}$ and $\frac{1}{2}$ are selected

In third diagonal $\frac{3}{1}$ and $\frac{1}{3}$ are selected, $\frac{2}{2}$ is skipped as it is same as $\frac{1}{1}$

Similarly the list Q is formed. Now it is easy to show correspondence of N with Q.

From the above two examples, it is clear that if we can show the correspondence, it is easy to find the solution. But for some cases, it is difficult even to show this correspondence. Such sets are called uncountable. Set of real numbers is an example of uncountable sets.

7.4.1 Undecidable Problems

M = {<M, w>| M is a TM, w is a string, M accepts w} Assume a TM is decidable, which halts and says accepted or rejected. Let H be a machine for a TM <M, w>. Then, H halts and accepts, if M accepts w; or H rejects, if M fails to accept w. To put more formally,

H(<H, w>) = {accept if M accept w.
= {reject if M does not accept w.

Construct a new truing machine D with H as subroutine. D calls H to find what M does when input to M is its own description <M>. that is, D is running a machine as its own description. It is just like a compiler written and compiled in the same language. D gets information and complements the action.
D is defined as <M> where M is a TM.

1. Runs H on input <M, <M>>.
2. If H accepts, it rejects, if H rejects, it accepts.

In summary,

$$D(<M>) = \begin{cases} \text{accept if M does not accept } <M> \\ \text{reject if M accepts } <M> \end{cases}$$

When we run D with its own description <D> as input? In that case, we get

$$D(<D>) = \begin{cases} \text{accept if D does not accept } <D> \\ \text{reject if D accepts } <D> \end{cases}$$

It is forced to do opposite to what D does. Thus neither TM D nor TM H exists.

7.5 Post's Correspondence Problem

Definition 3: Given an alphabet S, one instance of Post's Correspondence Problem (PCP) of size s is a finite set of pairs of strings (g_i, h_i) ($i = 1 \ldots s$ $s \geq 1$) over the alphabet S. A solution of length $n \geq 1$ to this instance is a sequence $i_1 i_2 \ldots i_n$ of selections such that the strings $g_{i1} g_{i2} \ldots g_{in}$ and $h_{i1} h_{i2} \ldots h_{in}$ formed by concatenation are identical.

Width of a PCP instance is the length of the longest string in gi and h_i (i = 1, 2, ..., s). We use *Pair* i as a short name for pair (g_i, h_i), where g_i and h_i are the top string and bottom string of the pair, respectively. Mostly, people are interested in *optimal solution*, which has the shortest length over all possible solutions to an instance. The corresponding length is called *optimal length*. We use the word *hard* or *difficult* to describe instances whose optimal lengths are very large. For simplicity, we restrict the alphabet S to {0, 1}, and it is easy to transform other alphabets to their equivalent binary format.

To describe subclasses of Post's Correspondence Problem, we use PCP[s] to represent the set of all PCP instances of size s, and PCP[s, w] to represent the set of all PCP instances of size s and width w.

For convenience, we use a matrix of 2 rows and s columns to represent instances of PCP[s], where string g_i is located at (i, 1) and h_i at (i, 2). The following is the matrix representation of the instance {{100, 1}, {0, 100}, {1, 00}} in PCP [3, 3].

i	g_i	h_i
1	100	1
2	0	100
3	1	00

Let us consider the results of selections of sequence {1, 3, 1, 1, 3, 2, 2} accordingly. They are shown in the following table and each selection is shown in the table.

Solution sequence	1	3	1	1	3	2	2
String G	100	1	100	100	1	0	0
String H	1	00	1	1	00	100	100

After the elimination of blanks and concatenation of strings in the top and bottom rows separately, we get

1001100100100

1001100100100

Now, the string in the top is identical to the one in the bottom. Therefore, these selections form a solution to the PCP problem.

Example 7.10 Consider the following sequence and find whether it has solution or not.

I	g_i	h_i
1	01	0
2	110010	0
3	1	1111
4	11	01

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

The tuple $(i_1, i_2, i_3, i_4, i_5, i_6) = (1, 3, 2, 4, 4, 3)$ is a witness for a positive solution because $x_1x_3x_2x_4x_4x_3 = y_1y_3y_2y_4y_4y_3 = 01111001011111$. The positive solution has also the witnesses (1, 3, 2, 4, 4, 3, 1, 3, 2, 4, 4, 3), (1, 3, 2, 4, 4, 3, 1, 3, 2, 4, 4, 3, 1, 3, 2, 4, 4, 3), etc.

Example 7.11 Consider the following sequence and find whether it has a solution or not.

i	g_i	h_i
1	0	10
2	01	1

This problem has no solution as the strings formed by g's would always start with 0, and the strings formed by h's would always start with 1.

7.5.1 The Undecidability of Post's Correspondence Problem

Post's correspondence problem is very useful for showing the undecidability of many other problems by means of reducibility. Its undecidability follows from its capacity for simulating the computations of TMs, as exhibited indirectly in the following proof through derivations in Type 0 grammars.

7.5.2 Modified Version of PCP

We can show that PCP is decidable if the modified version of PCP is decidable. To show it is decidable, we would have an algorithm for L_u.

The Modified Post's Correspondence Problem (MPCP) is described as given lists G and H of k-strings each from Σ^*, say

$$G = g_1, g_2, \ldots g_k \text{ and } H = h_1, h_2, \ldots . H_k,$$

does there exist a sequence of integers, $i_1, i_2 \ldots . i_r$ such that

$$g_1, g_{i1}, g_{i2} \ldots . g_{ir} = h_1, h_{i1}, h_{i2} \ldots h_{ir}$$

The difference between the MPCP and PCP is that in the MPCP, a solution is required to start with the first string on each list.

If we have a problem instance represented in MPCP, then it can be reduced to PCP. If there is a solution for PCP instance, then there exists a solution for MPCP instance.

Lemma 1: If PCP were decidable, then MPCP would be decidable, that is MPCP reduces to PCP.

Procedure to convert MPCP to PCP:

1. Let the lists G and H be the given instance of MPCP.
2. Let Σ be the smallest alphabet containing all the symbols in the lists G and H.
3. Consider two special symbols $\{\theta, \$\}$ not present in Σ, and find two new lists X from G and Y from H using the following rules.

4. x_i of list X is obtained from g_i by inserting $ symbol after each character of g_i.
5. y_i of list Y is obtained from h_i by inserting $ symbol before each character of h_i.
6. Create new words as follows:

$$x_0 = \$g_1, \qquad y_0 = h_1$$
$$x_{k+1} = \theta, \qquad y_{k+1} = \$\theta$$

Example 7.10 Consider the following MPCP instance and find whether it has a solution.

i	g_i	h_i
1	100	1
2	0	100
3	1	00

Solution: This problem can be converted to MPCP by applying the above procedure.

i	x_i	y_i
0	$1$0$0$	$1
1	1$0$0$	$1
2	0$	$1$0$0
3	1$	$0$0
4	θ	$θ

PCP problem would have solution if there is a sequence in MPCP as $0, i_1, i_2, i_3 \ldots i_r, k+1$, which is solution with the list X and Y. Then there is sequence as $i_1, i_2, i_3 \ldots i_r$ which is a solution with lists G and H.
Let the sequence be 0, 3, 1, 1, 3, 2, 2, 4

Solution sequence	0	3	1	1	3	1	2	4
String G	$1$0$0$	3	1$0$0$	1$0$0$	3	0$	2	[illegible]
String G	[illegible]	$0$0	1	1	$0$0	$1$0$0	$1$0$0	$θ

String formed from list X: $1$0$0$1$1$0$0$1$0$0$1$0$0$θ
String formed from list Y: $1$0$0$1$1$0$0$1$0$0$1$0$0$θ
Since there is solution for MPCP, the solution for PCP is 3, 1, 1, 3, 2, 2, 4.

Example 7.11 Does the following PCP problem have a solution.

$$X = \{b, babbb, ba\}\ Y = \{bbb, ba, a\}$$

Solution:

String formed from list X: babbbbbba
String formed from list Y: babbbbbba

There is a solution for PCP and is 2, 1, 1, 3.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Example 7.10 Explain how PCP can be treated as a game of dominoes.

Solution: In the game of dominoes, upper half and lower half have strings say A_i B_i as shown below:

A_i: $a_1a_2a_3$	Upper Half
B_i: $b_1b_2b_3$	Lower Half

To win a game, the same string must appear in A_i and B_i, that is, $a_1a_2a_3 = b_1b_2b_3$. Winning the game is equivalent to getting solution for PCP.

Example 7.11 Consider the PCP system with two strings A and B with A = {1, 0, 010, 11} and B = {10, 10, 01, 1}. Find whether PCP problem has a solution or not.

Solution:

String formed from list X: 10101001011
String formed from list Y: 10101001011

There is solution for PCP and it is 1, 2, 1, 3, 3, 4.

Theorem 2: The PCP is an undecidable problem.

Proof: From the above lemma, it is sufficient to show that if MPCP were decidable, then it would be decidable whether a TM accepts a given word. We reduce the Lu to MPCP, which is reduced to PCP. For each M and w, we construct an instance of MPCP that has a solution if and only if M accepts w. We do this by constructing an instance of MPCP that, if it has a solution, has one that starts with $\#q_0w\#\alpha_1q_1\beta_1\#.....\#\alpha_kq_k\beta_k\#$ where strings between successive #'s are successive IDs in a computation of M with input w, and q_k is a final state. Procedure to convert decidability of TM to MPCP:

	List X	List Y
Basic string to be added	#	$\#q_0w\#$
Group I	X	X
For each X in τ add strings to both x & y list	#	#
Group II		
For each q in Q – F, p in Q, and X, Y and Z in τ		
If $\delta(q, X) = (p, Y, R)$	qX	Yp
If $\delta(q, X) = (p, Y, L)$	ZqX	pZY
If $\delta(q, B) = (p, Y, R)$	q#	Yp#
If $\delta(q, B) = (p, Y, L)$	Zq#	pZY#
Group III	XqY	q
For each q in F, and X and Y in τ	Xq	q
	qY	q
Group IV	q##	#
For each q in F		

Let us say that (x, y) is a partial solution to MPCP with lists X and Y if s is a prefix of y, and x and y are the concatenation of corresponding strings of lists A and B, respectively. If xz = y, then call z the remainder of (x, y).

Example 7.10 Consider the following Turing machine defined as

$$M = (\{q_0, q_1, q_A\}, \{a, b\}, \{a, b, B\}, \delta, q_0, B, \{q_A\})$$

	a	b	B
q_0	(q_1, b, R)	(q_1, a, L)	(q_1, b, L)
q_1	(q_A, a, L)	(q_0, a, R)	(q_1, a, R)
q_A	---	---	---

State whether for the string w = ab, Turing machine halts.

Solution: First let us convert this problem instance to MPCP form:

	List X	List Y
Basic string to be added	#	$\#q_0ab\#$
Group I For each X in τ add strings as	a b #	a b #
Group II For each q in Q – F, p in Q, and X, Y and Z in τ $\delta(q_0, a) = (q_1, b, R)$ $\delta(q_0, b) = (q_1, a, L)$	 q_0a aq_0b bq_0b	 bq_1 q_1aa q_1ba
$\delta(q_0, B) = (q_1, b, L)$ $\delta(q_1, a) = (q_A, a, L)$ $\delta(q_1, b) = (q_0, a, R)$ $\delta(q_1, B) = (q_1, a, R)$	$aq_0\#$ $bq_0\#$ aq_1a bq_1a q_1b $q_1\#$	$q_1ab\#$ $q_1ab\#$ q_Aaa q_Aba aq_0 $aq_1\#$
Group III For each q in F, and X and Y in τ	aq_Aa bq_Aa aq_Ab bq_Ab q_Aa q_Ab aq_A bq_A	q_A q_A q_A q_A q_A q_A q_A q_A
Group IV For each q in F	q_A ##	#

To find a solution for the instance w = ab, we start with a partial solution (#, $\#q_0ab\#$).
Choose the pair (q_0a, bq_1) that would result in partial solution as ($\#q_0a$, $\#q_0ab\#$ bq_1).

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Similarly, make the choices in the following order:

Choice used	Partial solution
(b, b)	(#$q_0$ab, #$q_0$ab#bq_1b)
(#, #)	(#$q_0$ab#, #$q_0$ab#bq_1b#)
(b, b)	(#$q_0$ab#b, #$q_0$ab#bq_1b#b)
(q_1b, aq_0)	(#$q_0$ab#bq_1b, #$q_0$ab#bq_1b#baq_0)
(#, #)	(#$q_0$ab#bq_1b#, #$q_0$ab#bq_1b#baq_0 #)
(b, b)	(#$q_0$ab#bq_1b#b, #$q_0$ab#bq_1b#baq_0 #b)
(aq_0#, $q_1$ab#)	(#$q_0$ab#bq_1b#baq_0#, #$q_0$ab#bq_1b#baq_0#b$q_1$ab#)
(bq_1a, $q_A$ba)	(#$q_0$ab#bq_1b#baq_0#bq_1a, #$q_0$ab#bq_1b#baq_0#b$q_1$ab#$q_A$ba)
(b, b)	(#$q_0$ab#bq_1b#baq_0#b$q_1$ab, #$q_0$ab#bq_1b#baq_0#b$q_1$ab#$q_A$ba)
(#, #)	(#$q_0$ab#bq_1b#baq_0#b$q_1$ab#, #$q_0$ab#bq_1b#baq_0#b$q_1$ab#$q_A$ba#)
(q_Ab, q_A)	(#$q_0$ab#bq_1b#baq_0#b$q_1$ab#q_Ab, #$q_0$ab#bq_1b#baq_0#b$q_1$ab#$q_A$ba#q_A)
...	
...	
(q_Ab, q_A)	(#$q_0$ab#bq_1b#baq_0#b$q_1$ab#$q_A$bab#$q_A$ab#, #$q_0$ab#bq_1b#baq_0#b$q_1$ab#$q_A$bab#$q_A$ab#q_Ab#)
(q_A##, #)	(#$q_0$ab#bq_1b#baq_0#b$q_1$ab# $q_A$bab#$q_A$ab#q_Ab#q_A##, #$q_0$ab#bq_1b#baq_0#b$q_1$ab# $q_A$bab#$q_A$ab#q_Ab#q_A##)

Thus, the shortest word that can be composed of corresponding strings from lists X and Y, starting with pair 1 is #$q_0$ab#bq_1b#baq_0#b$q_1$ab# $q_A$bab#$q_A$ab#q_Ab#q_A##.

7.6 Reducibility

In Computability Theory and Computational Complexity Theory, a **reduction** is a transformation of one problem into another problem. Problem A is reducible to problem B if solutions to B exist and give solutions to A whenever A has solutions. Thus, solving A cannot be harder than solving B. We write $A \leq B$, usually with a subscript on the $\leq$ to indicate the type of reduction being used.

A quick way of solving the new problem is to transform each instance of the new problem into instances of the old problem, solve the instance of problem using our existing solution, and then use the solutions to obtain our final solution. This is perhaps the most obvious use of reductions. Reducibility can be used to measure the complexity of the problem depending on the kind of transformation used.

Another, more subtle use is the following: We have a problem that is proved to be hard to solve, we have a similar new problem or suspect that the new problem is hard to solve. If our hunch is correct, we can try to prove it by contradiction. For this, we suppose the new problem is easy to solve. Then, if we can show that every instance of the old problem can be solved easily by transforming it into instances of the new problem and solving those, then we have a contradiction. This way we can establish that the new problem is also hard.

Example 7.11 Let us assume that we know addition, subtraction, division by two or squaring. Using this knowledge, we can multiply two numbers. For instance, multiplying 10 and 8 can be carried out as follows:

a) Repeated addition of 10 by 8 times.
b) Square 10 and subtract 10 twice.
c) If both are even numbers, then compute $((a + b)/2)^2 - (a - (a + b)/2)^2$.

$$((a + b)/2)^2 = ((10 + 8)/2)^2 = 9^2 = 81$$
$$(a - (a + b)/2)^2 = (10 - (10 + 8)/2))^2 = (10 - 8)^2 = 1$$
$$((a + b)/2)^2 - (a - (a + b)/2)^2 = 81 - 1 = 80$$

We can propose many possible solutions. Problem comes when there is constraint on number of operations or on the type of operation to be applied. The reduction becomes much harder if we add the restriction that we can only use the squaring function one time only, and only at the end. This shows that the reduced problem is hard to solve. This kind of reduction corresponds to Turing reduction.

Definition 4: Given two subsets A and B of **N** and a set of functions F: $\mathbf{N} \rightarrow \mathbf{N}$ which is closed under composition, A is called **reducible** to B under F if

$$\exists\, f \in F, \forall\, x \in N.\ x \in A \Leftrightarrow f(x) \in B$$

Then we write $A \leq_F B$. Let S be a subset of $\mathbf{P}(\mathbf{N})$ and let $\leq$ be a reduction. Then S is called **closed** under $\leq$ if

$$\forall\, s \in S.\ A \in P(N).\ A \leq s \Rightarrow A \in S.$$

A subset A of N is called hard for S if

$$\forall\, s \in S.\ s \leq A$$

A subset A of N is called complete for S if A is hard for S and A is in S.

7.6.1 Properties

A reduction is a pre-ordering (i.e. a reflexive and transitive relation) on $\mathbf{P}(\mathbf{N})$ $\mathbf{P}(\mathbf{N})$, where $\mathbf{P}(\mathbf{N})$ is the power set of the natural numbers.

Detailed Example

The following example shows how to use reduction from the halting problem to prove that a language is undecidable.

Suppose H(M, w) is the problem of determining whether a given turing machine M halts (by accepting or rejecting) on input string w. This language is known to be undecidable. Suppose E(M) is the problem of determining whether the language a given turing machine M accepts is empty (in other words, whether M accepts any strings at all). We show that E is undecidable by a reduction from H.

To obtain a contradiction, suppose R is a decider for E. We will use this to produce a decider S for H (which we know does not exist). Given input M and w (a TM and some input string), define S(M, w) with the following behaviour: S creates a TM N that accepts only if the input string to N is w and M halts on input w, and does not halt otherwise.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

The decider S can now evaluate R(N) to check whether the language accepted by N is empty. If R accepts N, then the language accepted by N is empty. So, in particular, M does not halt on input w. So, S can reject. If R rejects N, then the language accepted by N is non-empty. So M does halt on input w, so S can accept. Thus, if we had a decider R for E, we would be able to produce a decider S for the halting problem H(M, w) for any machine M and input w. Since we know that such a S cannot exist, it follows that the language E is also undecidable.

7.6.2 Mapping Reducibility

We have various techniques to prove that some problems are undecidable (PCP in chapter 7.5). To prove certain languages are not Turing recognizable, we can use reducibility in a refined way. There are many ways, and the choice depends on the application. One such technique is **mapping reducibility,** also called as **many–one reducibility**.

Reducing problem A to problem B by using mapping reducibility means that there exists a computable function that converts every instance of A to a unique instance of B. If such conversion function is available, then A can be reduced to B, and A can be solved with a solver for B.

7.6.3 Formal Definition of Mapping Reducibility

Definition 5: Language A is mapping reducible to language B, written $A \leq_m B$, if there is a computable function f: $\Sigma^* \rightarrow \Sigma^*$, where for every w, $w \in A \Leftrightarrow f(w) \in B$. The function f is called the reduction of A to B.

To test whether $w \in A$, we use reduction f to map w to f(w) and test whether $f(w) \in B$. The term mapping reduction comes from the function of mapping that provides the means of doing reduction. If one problem is mapped to another solved problem, then solution can be obtained for the first problem.

Theorem 3: If $A \leq_m B$ and B is decidable, then A is decidable.

Proof: Let M be a decider for B and let f be the reduction from A to B. We describe a decider N for A as follows.
N = 'On input w:

1. Compute f(w).
2. Run M on input f(w) and output whatever M outputs'.

Clearly, if $w \in A$, then $f(w) \in B$ because f is a reduction from A to B. Thus M accepts f(w) whenever $w \in A$. Therefore, N works as desired.

7.7 Recursion Theorem

In mathematics and computer science, **Recursion** is a method of defining functions in which the function being defined is applied within its own definition. The term is also used more generally to describe a process of repeating objects in a self-similar way.

Simple examples

1. Person's ancestors:
   a) One's parents are one's ancestors (*base case*).
   b) The parents of one's ancestors are also one's ancestors (*recursion step*).
2. Mathematical functions – Fibonacci sequence
   a) Fib(0) is 1 [base case]
   b) Fib(1) is 1 [base case]
   c) For all integers n > 1: Fib(n) is (Fib(n – 1) + Fib(n – 2)) [recursive definition]

7.7.1 Applications and Uses of Recursion

Recursion in language

In 5th century BC an ancient Indian linguist Pāṇini used recursion in defining the grammar rules of Sanskrit. Linguist Noam Chomsky theorizes that unlimited extension of a language such as English is possible only by the recursive definitions of grammar rules to enable embedding sentences in sentences. Recursion in linguistics enables 'discrete infinity' by embedding phrases within phrases of the same type in a hierarchical structure. Without recursion, language does not have 'discrete infinity' and cannot embed sentences into infinity.

Functional recursion

A function may be partly defined in terms of itself. A familiar example is the Fibonacci number sequence: $F(n) = F(n-1) + F(n-2)$. For such a definition to be useful, it must lead to values that are non-recursively defined, in the present case $F(0) = 0$ and $F(1) = 1$.

A famous recursive function is the Ackermann function which, unlike the Fibonacci sequence, cannot be expressed without recursion (see Section 7.9).

Recursion in computer science

Recursion in computer programming is exemplified when a function is defined in terms of itself. One example of application of recursion is in parsers for programming languages. The great advantage of recursion is that an infinite set of possible sentences, designs or other data can be defined, parsed or produced by a finite computer program.

Recurrence relations are equations to define one or more sequences recursively. Some specific kinds of recurrence relation can be 'solved' to obtain a non-recursive definition.

An example of recursion is the definition of the factorial function, given here in C code:

```
unsigned int factorial(unsigned int n)
{
    if (n <= 1) return 1;
    else
    {
    return n * factorial(n-1);
    }
}
```

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

The function calls itself recursively on a smaller version of the input (n – 1) and multiplies the result of the recursive call by n, until reaching the base case, analogous to the mathematical definition of factorial.

Use of recursion in an algorithm has both advantages and disadvantages. The main advantage is usually simplicity. The main disadvantage is often that the algorithm may require large amounts of memory if the depth of the recursion is very large. It has been claimed that recursive algorithms are easier to understand because the code is shorter and is closer to a mathematical definition, as seen in these factorial examples.

It is often possible to replace a recursive call with a simple loop, as the following example of factorial shows:

```
unsigned int factorial(unsigned int n) {
    if (n <= 1) return 1;
    unsigned int result = n;
    while (-n) result *= n;
    return result;
}
```

It should be noted that on most CPUs, the above examples give correct results only for small values of n, due to arithmetic overflow.

7.8 Rice's Theorem

Rice's theorem is an important result in the theory of recursive functions. A property of partial functions is *trivial* if it holds for all partial recursive functions or for none. At any given time

- Programs p and q are equivalent if they compute the same partial function.
- A set of strings S is *non-trivial* if S is not the empty set and S does not contain all strings.
- A set of programs S *respects equivalence* if, whenever programs p and q are equivalent, either both p and q are members of S or both p and q are non-members of S.

Theorem 4: Every non-trivial set of programs that respects equivalence is undecidable.

Proof: Let S be a non-trivial set of programs that respects equivalence. We prove that S is undecidable using proof by contradiction.

Assume that S is decidable. Then there must be a program called P_{DECIDE} that decides membership in S.
This program has two answers.

$$P_{DECIDE}(p) = \text{yes} \quad \text{whenever } p \in S.$$
$$P_{DECIDE}(p) = \text{no} \quad \text{whenever } p \notin S.$$

We prove by contradiction showing if the program P_{DECIDE} exists then there exists a member P belong to S. We prove by a contradiction from the assumption that this program P_{DECIDE} exists. (Or, if you prefer, we will find a program p on which P_{DECIDE} gives the wrong answer. That is the contradiction).

Write a program called P_{LOOP} that loops for ever on all inputs. It is either in S or not. We can assume without loss of generality that P_{LOOP} is not in S. If P_{LOOP} is in S, then if we consider the complement of S The complement of S is decidable if and only if S is decidable.

S is non-trivial. So it must have a member. Choose any member of S, and call it P_{GEORGE}. Now we have one member of S (P_{GEORGE}) and one non-member of S (P_{LOOP}).

In order to show that P_{DECIDE} does not work, we need to write two programs. The first we write has another program r built into it. For any r, you can build program Q_r. It goes like this.

Program $Q_r(x)$: Run the interpreter on input (r, r). This is equivalent to running program r on itself.

Run program P_{GEORGE} on input x, and produce the same result that P_{GEORGE} produces.

Notice, by inspecting the text of the program, that Q_r has the following properties. We are making use of the fact that S respects equivalence here.

Observation Q1: For every r:

$P_r(r)$ terminates

$\Rightarrow Q_r$ is equivalent to P_{GEORGE}

$\Rightarrow Q_r$ is a member of S

Observation Q2: For every r:

$P_r(r)$ does not terminate

$\Rightarrow Q_r$ is equivalent to P_{LOOP}

$\Rightarrow Q_r$ is not a member of S

Now write a second program called P_{FOOL} as follows.

P_{FOOL} (r): Build program Q_r by building parameter r into Q.

Run P_{DECIDE} on input Q_r, and call its answer y. (We are asking P_{DECIDE} whether Q_r is in S.)

If y = 'yes', then loop for ever, else stop.

Now we are ready to prove that P_{DECIDE} does not get the correct answer on every input. That is the contradiction that we are looking for (since we presumed that P_{DECIDE} got the right answer on every input.)

Once the following claim is proved, we are done.

Claim: P_{DECIDE} gets the wrong answer on input P_{FOOL}.

Proof of Claim: Imagine running P_{FOOL} on input P_{FOOL}. The following are evident.

$P_{DECIDE}(Q_{FOOL})$ = 'yes'

$\Rightarrow P_{FOOL}(P_{FOOL})$ does not terminate [by inspecting the definition of P_{FOOL}]

$\Rightarrow Q_{FOOL}$ is not a member of S [by Observation Q2]

So if P_{DECIDE} answers yes on input Q_{FOOL}, it got the wrong answer. (The correct answer is no, since Q_{FOOL} is not a member of S.)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$P_{DECIDE}(Q_{FOOL})$ = 'no'

$\Rightarrow P_{FOOL}(P_{FOOL})$ terminates [by inspecting the definition of P_{FOOL}]

$\Rightarrow Q_{FOOL}$ is a member of S [by Observation Q1]

So if P_{DECIDE} answers 'no' on input Q_{FOOL}, it got the wrong answer. (The correct answer is yes, since Q_{FOOL} is a member of S.)

Regardless of what answer P_{DECIDE} gives on input Q_{FOOL}, the answer is wrong.

Note: Program Q_r needs to be constructed by program P_{FOOL}. One mechanism for doing that is to use parameter fixing. Write Q as a program with two parameters, rr and x. Then what P_{FOOL} needs to do is fix the parameter rr to the value of P_{FOOL}'s parameter r. That can be done by the computable function that fixes a parameter of a program to some particular value. That is, we use function P_{PARAM} where $P_{PARAM}(Q, r) = T$ where $P_T(x) = Q(r, x)$. That is, P_{PARAM} has fixed the first parameter of Q to a given value, r, yielding new program T.

7.9 Ackermann's Function

Ackermann's function is a simple example of a total function that is computable, but not primitive recursive. The function $f(x) = A(x, x)$ grows much faster than polynomials or exponentials. A(x, y) is defined by

1. If $x = 0$, then $A(x, y) = y + 1$
2. If $y = 0$, then $A(x, y) = A(x - 1, 1)$
3. Otherwise, $A(x, y) = A(x - 1, A(x, y - 1))$

To compute Ackermann's function, A(x, y), for $x = 3$ and $y = n$, is given as

$A(3, n) = 2^{(n+3)} - 3$ and it requires at least $4^{(n+1)}$ function calls, and reaches a recursive depth of $2^{(n+3)} - 1$.

Thus, computation of it stresses your compiler's ability to do deep recursion efficiently. In particular, it tests your code generation for function calls. If you happened to implement tail-call elimination, you will see a very large performance gain.

Example 7.18 Compute A(1, 1) and A(1, 2).

Solution:

$$
\begin{aligned}
A(1, 1) &= A(1 - 1, A(1, 1 - 1)) && \text{by 3}\\
&= A(0, A(1, 0)) && \text{by 2}\\
&= A(0, A(0, 1)) && \text{by 1}\\
&= A(0, 2) && \text{by 1}\\
&= 3
\end{aligned}
$$

$$
\begin{aligned}
A(1, 2) &= A(1 - 1, A(1, 2 - 1))\\
&= A(0, A(1, 1)) && \text{by 3}\\
&= A(0, 3) && \text{by 2}\\
&= 4 && \text{by 1}
\end{aligned}
$$

The values for A(m, n) where $0 \le m \le 5$ and $0 \le n \le 5$ are shown in the following table.

A(m, n)	n = 0	n = 1	n = 2	n = 3	n = 4	n = 5
m = 0	1	2	3	4	5	6
m = 1	2	3	4	5	6	7
m = 2	3	5	7	9	11	13
m = 3	5	$2^{3+1} - 3 = 13$	$2^{3+2} - 3 = 29$	$2^{3+3} - 3 = 61$	$2^{3+4} - 3 = 125$	$2^{3+5} - 3 = 253$
m = 4	$2^4 - 3 = 13$	65533	$2^{65536} - 3$	$2^{265536} - 3$	A(3, $2^{265536} - 3$)	A(3, A(4, 4))
m = 5	65533	A(4, 65533)	A(4, A(4, 65533))	A(4, A(5, 2))	A(4, A(5, 3))	A(4, A(5, 4))

Ackermann's function, while Turing computable, grows faster than any primitive recursive function. The reason can be seen from the table: The *index* for the next value is also growing.

An equivalent definition is:

$A(0, n) = n + 1$

$A(1, n) = 2 + (n + 3) - 3$

$A(2, n) = 2 \times (n + 3) - 3$

$A(3, n) = 2^{n+3} - 3$

$A(4, n) = 2^{2^{\cdot^{\cdot^{\cdot^{2}}}}} - 3$ $(n + 3$ terms$)$

...

Remember that $2^{2^{2^2}} = 2^{(2^{(2^2)})} = 65536$, and not $((2^2)^2)^2 = 2^{2\times2\times2} = 256$

Solved Problems

Problem 1: Consider the following sequence and find whether it has solution or not.

i	g_i	h_i
1	0	10
2	01	1

Solution: This problem has no solution as the strings formed by g's would always start with 0 and the strings formed by h's would always start with 1.

Problem 2: Prove that the PCP with two lists a = {01, 1, 1} and b = {111, 10, 11} has no solution.

Solution: Each string in a is smaller in length than each string in b. Hence, the sequence generated by strings of a will be always shorter than the sequence generated by b. Hence, the PCP has no solution.

Problem 3: Find A(2, 1).

Solution: $A(2, 1) = A(1 + 1, 0 + 1)$
$= A(1, A(2, 0))$
$= A(1, A(1, 1))$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$= A(1, 3)$
$= A(0 + 1, 2 + 1)$
$= A(0, A(1, 2))$
$= A(0, 4) = 4 + 1 = 5$

Problem 4: Find A(2, 2).

Solution: $A(2, 2) = A(1 + 1, 1 + 1)$
$= A(1, A(2, 1))$
$= A(1, 5)$
$= A(0 + 1, 4 + 1)$
$= A(0, A(1, 4)) = 1 + A(1, 4)$
$= 1 + A(0 + 1, 3 + 1) = 1 + A(0, A(1, 3))$
$= 1 + 1 + A(1, 3) = 1 + 1 + 1 + A(1, 2)$
$= 1 + 1 + 1 + 4 = 7$

Summary

- The languages accepted by TM are called recursively enumerable, and the subset languages are recursive languages for which a TM always halts.
- The languages that are recursive are decidable languages. The languages that are not recursive are undecidable languages.
- Rice's theorem can be used to show problem of deciding if a language is empty is undecidable.
- Post's correspondence problem has a solution if the concatenated strings of xi's list and yi's list are same.
- Any TM problem can be reduced to PCP form, and if the problem has solution then that problem is decidable.
- Any PCP can be converted to an MPCP, and solution to the problem instance is computed with some starting string.

Short Answers

1. When do we say a problem is decidable? Give an example of an undecidable problem?

   Answer: A problem whose language is recursive is said to be decidable. Otherwise, the problem is said to be undecidable. Decidable problems have an algorithm that takes as input an instance of the problem and determines whether the answer to that instance is 'yes' or 'no'. An example of an undecidable problem is the Halting problem of the TM.

2. Give examples of decidable problems.

   Answer:
   1. Given a DFSM M and string w, does M accept w?
   2. Given a DFSM M, is $L(M) = \Phi$?

3. Given two DFSMs M_1 and M_2, is $L(M_1) = L(M_2)$?
4. Given a regular expression α and a string w, does α generate w?
5. Given a NFSM M and string w, does M accept w?

3. Give examples of recursive languages?

Answer:

i) The language L defined by L = {'M', 'w': M is a DFSM that accepts w} is recursive.

ii) L defined by {'M_1' ∪ 'M_2': DFSMs M_1 and M_2 and $L(M_1) = L(M_2)$} is recursive.

4. Differentiate recursive and recursively enumerable languages.

Answer:

Recursive languages	Recursively enumerable languages
1. A language is said to be recursive iff there exists a membership algorithm for it.	1. A language is said to be r.e. if there exists a TM that accepts it.
2. A language L is recursive iff there is a TM that decide languages have algorithms.	2. L is recursively enumerable iff there is a TM that partially decidable L. (Turing-acceptable languages). TMs that partially decides languages are not algorithms.

5. What are UTMs or Universal Turing machines?

Answer: Universal TMs are TMs that can be programmed to solve any problem that can be solved by any Turing machine. A specific Universal Turing machine U is the following: Input to U: The encoding 'M' of a TM M and encoding 'w' of a string w. Behaviour: U halts on input 'M' 'w' if and only if M halts on input w.

6. What are the crucial assumptions for encoding a TM?

Answer: There are no transitions from any of the halt states of any given TM. Apart from the halt state, a given TM is total.

7. What properties of recursive enumerable seta are not decidable?

Answer:

- Emptiness
- Finiteness
- Regularity
- Context freedom.

8. Define L_l.When is l a trivial property?

Answer: L_l is defined as the set {<M> | L(M) is in l}.

l is a trivial property if l is empty or it consists of all r.e. languages.

9. What is a universal language L_u?

Answer: The universal language consists of a set of binary strings in the form of pairs (M, w) where M is a TM encoded in binary, and w is the binary input string.: L_u = {<Mw> | M accepts w}.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

10. What is a diagonalization language L_d?

    Answer: The diagonalization language consists of all strings w such that the TM M whose code is w does not accept when w is given as input.

11. What properties of r.e. sets are recursively enumerable?

    Answer:

    - $L \neq \Phi$
    - L contains at least 10 members.
    - w is in L for some fixed w.
    - $L \cap L_u \neq \Phi$

12. What properties of r.e. sets are not r.e.?

    Answer:

    - $L = \Phi$.
    - $L = \Sigma^*$.
    - L is recursive.
    - L is not recursive.
    - L is singleton.
    - L is a regular set.
    - $L - L_u \neq \Phi$.

13. Show that AMBIGUITY problem is undecidable.

    Answer: Consider the ambiguity problem for CFGs. Use the 'yes–no' version of AMB. An algorithm for FIND is used to solve AMB. FIND requires producing a word with two or more parses if one exists and answers 'no' otherwise. By the reduction of AMB to FIND, we conclude there is no algorithm for FIND and hence there is no algorithm for AMB.

14. State the halting problem of TMs.

    Answer: The halting problem for TMs is the following:

    Given any TM M and an input string w, does M halt on w? This problem is undecidable as there is no algorithm to solve this problem.

15. Define PCP or Post's Correspondence Problem.

    Answer: An instance of PCP consists of two lists, $A = w_1, w_2, \ldots w_k$ and $B = x_1, \ldots x_k$, of strings over some alphabet Σ.

    This instance of PCP has a solution if there is any sequence of integers $i_1, i_2, \ldots i_m$ with $m \geq 1$ such that

    $$w_{i1}, w_{i2}, \ldots w_{im} = x_{i1}, x_{i2}, \ldots x_{im}$$

    The sequence $i_1, i_2, \ldots i_m$ is a solution to this instance of PCP.

16. Define MPCP (or Modified PCP).

    Answer: The MPCP is the following: Given lists A and B of K strings from Σ^*, say

    $$A = w_1, w_2, \ldots w_k \text{ and } B = x_1, x_2, \ldots x_k$$

does there exist a sequence of integers i1, i2,...ir such that

$$w_1 w_{i1} w_{i2} \ldots\ldots w_{ir} = x_1 x_{i1} x_{i2} \ldots x_{ir}?$$

17. What is the difference between PCP and MPCP?

Answer: The difference between MPCP and PCP is that in the MPCP, a solution is required to start with the first string on each list.

Fill in the Blanks

1. The language generated by an unrestricted grammar is_____.
2. _____ problems will always have two possible solutions.
3. A decision problem that can be solved by an algorithm is said to be _____.
4. A decision problem A is called decidable if A is _____.
5. A partial decision problem A is called undecidable if A is _____.
6. _____ problem of TM is formulated as "given an arbitrary input the Turing machine will halt".
7. _____ method is used to prove the undecidability of halting problem.
8. The other name for many--one reducibility is _____.
9. _____ theorem characterizes the sets of partial recursive functions.
10. _____ theorem says that all non-trivial properties about the language of a TM are undecidable.

Answers

1. xxxx
2. Decision
3. Decidable
4. Recursive set
5. Recursively enumerable set
6. Halting
7. Diagonalization
8. Mapping reducibility
9. Extended Rice's theorem
10. Rice's

Objective Question Bank

1. Which one of the following is not decidable:
   a) Given a TM M, a string s and an integer k, M accepts s within k steps.
   b) Equivalence of two given TMs
   c) Language accepted by a given FSM is non-empty.
   d) Language generated by a CFG is non-empty.

2. Consider the following decision problems:
   (P_1) does a given FSM accept a given string.
   (P_2) does a given CFG generate an infinite number of strings.
   Which of the following statements is true?
   a) Both (P_1) and (P_2) are decidable
   b) Neither (P_1) nor (P_2) is decidable
   c) Only (P_1) is decidable
   d) Only (P_2) is decidable

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

3. Consider the following problem X:
   Given a TM M over the input alphabet E, any state q of M and a word $w \in \Sigma^*$, does the computation of M on w visit the state. Which of the following statements about X is correct?
   a) X is decidable
   b) X is undecidable, but partially decidable
   c) X is undecidable and not even partially decidable
   d) X is not a decision problem

4. Which of the following are true?
   a) The member ship problem of CFLs is undecidable
   b) It is undecidable if a CFL is regular
   c) The halting problem of finite automata is undecidable
   d) The membership problem for type languages is decidable

5. Which of the following is the strongest correct statement about a finite language over some finite alphabet Σ?
   a) It could be undecidable b) It is TM recognizable c) It is a CSL
   d) It is a regular language e) None of the above

6. Consider two languages L_1 and L_2, each on the alphabet Σ, f: $\Sigma^* \rightarrow \Sigma^*$ be a polynomial time computable bisection such that $(\forall x)$ $\{x \in L_1$ iff $f(x) \in L_2\}$. Further let f^{-1} be also polynomial time computable. Which of the following cannot be true?
   a) $L_1 \in P$ and L_2 is finite
   b) $L_1 \in NP$ and $L_2 \in P$
   c) L_1 is undecidable and L_2 is decidable
   d) L_1 is recursively enumerable and L_2 is recursive

7. Define language L_0 and L_1 as follows:
   $L_0 = \{<M, w, 0> / M$ halts on $w\}$, $L_1 = \{<M, w, 1> / M$ halt on $w\}$
   Here <M, w, 1> is a triplet, whose first component, M, is an encoding of a TM; second component, w, is a string, and third component, i, is a bit. Let $L = L_0 \cup L_1$. Which of the following is true?
   a) L is recursively enumerable, but L is not.
   b) L is recursively enumerable, but L is not
   c) Both L and L are recursive
   d) Neither L, nor L, is recursively enumerable

8. If PCP is decidable then MPCP is
   a) decidable b) undecidable
   c) cannot say d) none of these

9. The value of A(2, 1) is
   a) 1 b) 2 c) 3 d) 4

10. The value of A(4, 3) is
    a) 2^{16} b) 6 c) 7 d) 8

Answers

1. b 2. c 3. b 4. b 5. d 6. b 7. d 8. a 9. d 10. a

Exercises

1. Given an instance of TM show that this instance has no solution.

	LIST A	LIST B
i	W_i	X_i
1	10	101
2	011	11
3	101	011

2. Write short notes on Post's Correspondence Problem. Explain how it can be used to prove whether a given problem is decidable or not.
3. Find whether there is a solution for the following PCP instance:
   A = {ba, aba, baa, b} B = {bab, baa, a, aba}.
4. What do you mean by 'decidable problems' and 'undecidable problems'? Give examples.
5. What is a halting problem? Is halting problem solvable?
6. What is a modified version of PCP? State whether the following has a solution and whether it is decidable.

	LIST A	LIST B
i	W_i	X_i
1	11	111
2	100	001
3	111	11

7. Prove that whether TM halts on all inputs is undecidable.
   Hint: Prove that some TM for ever remains in a loop on some inputs.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

This page is intentionally left blank

Non-deterministic Polynomial Completeness

8

- NP-hard and NP-complete define the computational complexity of algorithms.

In this chapter, we introduce the theory of intractability, that is, discuss whether a problem can be solved in polynomial time or not, that is, P, Non-deterministic Polynomial (NP) Problems are discussed. The classification of problems into NP-hard problems and NP-complete (NPC) problems is explained with examples.

8.1 NP-hard and NP-complete

8.1.1 Classification of Problems

The subject of *computational complexity theory* is focused on classifying problems by how hard they are. There are many different classifications depending on the time taken by the problem. The following are the types of classification.

- P problems are those that can be solved by a Turing machine (TM) (deterministic) in polynomial time. ('P' stands for polynomial). P problems form a class of problems that can be solved efficiently.
- NP problems are those that can be solved by non-deterministic TM in polynomial time. A problem is in NP if you can quickly (in polynomial time) test whether a solution is correct (without worrying about how hard it might be to find the solution). NP problems are a class of problems that cannot be solved efficiently. *NP does not stand for 'non-polynomial'*. There are many complexity classes that are much harder than NP.
- **Undecidable problems:** For some problems, we can prove that there is no algorithm that always solves them, no matter how much time or space is allowed. One very uninformative proof of this is based on the fact that there are as many problems as there real numbers, and only as many programs as there are integers. So, there are not enough programs to solve all the problems. But we can also define explicit and useful problems which cannot be solved.

8.2 P Problems

In computational complexity theory, P, also known as PTIME or DTIME, is one of the most fundamental complexity classes. It contains all decision problems that can be solved by a deterministic TM using a polynomial amount of computation time or, simply put,

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

polynomial time. P is known to contain many natural problems, including the decision versions of linear programming calculating the greatest common divisor and finding a maximum matching.

8.3 NP Problems

In computational complexity theory, NP is one of the most fundamental complexity classes.

Intuitively, NP is the set of all decision problems for which the 'yes' answers have simple proofs because of the fact that the answer is indeed 'yes'. More precisely, these proofs have to be verifiable in polynomial time by a deterministic TM. In an equivalent formal definition, NP is the set of decision problems solvable in polynomial time by a non-deterministic TM. The Figure 8.1 indicates the relation between P and NP problems.

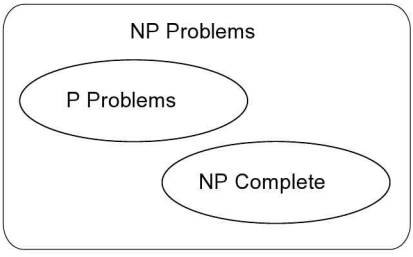


Fig. 8.1 *Class of P and NP Problems*

The complexity class P is contained in NP, but NP contains many important problems, called NPC problems, for which no polynomial-time algorithms are known. The most important open question in complexity theory, the P = NP problem, asks whether such algorithms actually exist for NPC problems. It is widely believed that this is not the case.

8.4 Tractable Problems

The problems that can be solved in polynomial time are called tractable problems. The problems that cannot be solved in polynomial time are called intractable problems. It is believed that no efficient algorithm exists for any of the decision problems. So they are considered to be intractable.

8.5 NP-complete

In computational complexity theory, the complexity class NPC is a class of problems having two properties:

- Solution to the problem in this class can be verified quickly (in polynomial time); the set of problems with this property is called NP.
- If the problem in this class can be solved quickly (in polynomial time), then so can every problem in NP.

The most notable characteristic of NPC problems is that no fast solution to them is known; that is, the time required to solve the problem using any currently known algorithm increases very quickly as the size of the problem grows. As a result, the time required to solve even moderately large versions of many of these problems easily reaches billions or trillions of years, using any amount of computing power available today. As a consequence, determining whether or not it is possible to solve these problems quickly is one of the principal unsolved problems in Computer Science today.

While a method for computing the solutions to NPC problems using a reasonable amount of time remains undiscovered, computer scientists and programmers still frequently encounter NPC problems. An expert programmer should be able to recognize an NPC problem so that he or she does not unknowingly waste time trying to solve a problem that so far has eluded generations of computer scientists. Hence, NPC problems are often addressed using approximation algorithms in practice.

NPC is a subset of NP, the set of all decision problems whose solutions can be verified in polynomial time; NP may be equivalently defined as the set of decision problems that can be solved in polynomial time on a non-deterministic TM. A problem p in NP is also in NPC if and only if every other problem in NP can be quickly transformed into p. (The word 'quickly' in these statements refers to polynomial time; a more formal definition of NPC is given below. NPC can also be used as an adjective: Problems in the class NPC are known as NPC problems.

NPC problems are studied because the ability to quickly verify solutions to a problem (NP) seems to correlate with the ability to quickly solve that problem (P). It is not known whether every problem in NP can be quickly solved – this is called the P = NP problem. But if *any single problem* in NPC can be solved quickly, then *every problem in NP* can also be quickly solved, because the definition of an NPC problem states that every problem in NP must be quickly reducible to every problem in NP-complete. Because of this, it is often said that the NPC problems are 'harder' or 'more difficult' than NP problems in general.

A decision problem C is NPC if

1. C is in NP, and
2. Every problem in NP is reducible to C.

C can be shown to be in NP by demonstrating that a candidate solution to C can be verified in polynomial time.

A problem K is reducible to problem C if there is a polynomial-time many–one reduction, a deterministic algorithm that transforms instances $k \in K$ into instances $c \in C$, such that the answer to *c* is YES if and only if the answer to k is YES. To prove that an NP problem A is in fact an NPC problem, it is sufficient to show that an already known NPC problem reduces to A.

Note that a problem satisfying Condition 2 is said to be NP-hard, whether or not it satisfies Condition 1.

A consequence of this definition is that if we had a polynomial-time algorithm (on a UTM, or any other Turing-equivalent abstract machine) for C, we could solve all problems in NP in polynomial time.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

8.6 NP-hard

Venn diagram for P, NP, NP-complete, and NP-hard set of problems is shown in Figure 8.2.

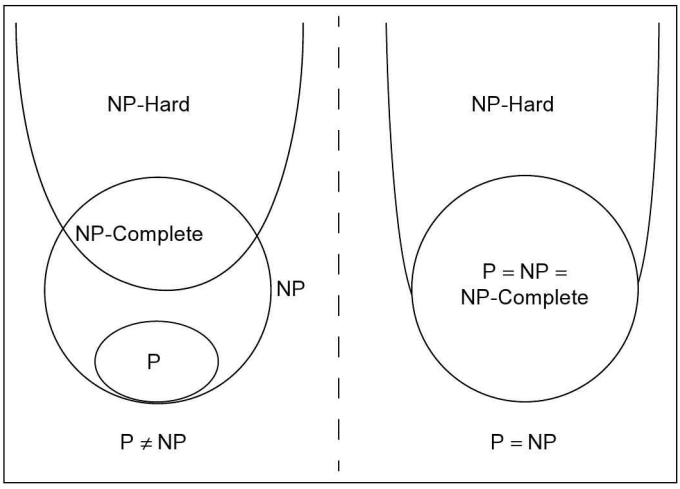


Fig 8.2 *Relation Between NP-hard and NP Complete*

NP-hard (non-deterministic polynomial-time hard), also called computational complexity theory, is a class of problem that are informally 'at least as hard as the hardest problem in NP'. A problem H is NP-hard if and only if there is an equivalent NPC problem L that is polynomial time Turing-reducible to H, that is, $L \leq_T H$. In other words, L can be solved in polynomial time by an oracle machine with an oracle for H. Informally, we can think of an algorithm that can call such an oracle machine as subroutine for solving H, and solves L in polynomial time if the subroutine call takes only one step to compute. NP-hard problems may be of any type: decision problems, search problems, optimization problems. We note

- Problem H is at least as hard as L, because H can be used to solve L.
- Since L is NP-complete, and hence the hardest in class NP, also problem H is at least as hard as NP, but H does not have to be in NP and hence does not have to be a decision problem (even if it is a decision problem, it need not be in NP).
- Since NPC problems transform to each other by polynomial-time many–one reduction (also called polynomial transformation), all NPC problems can be solved in polynomial time by a reduction to H. Hence, all problems in NP reduce to H; note however, that this involves combining two different transformations: from NPC decision problems to NPC problem L by polynomial transformation, and from L to H by polynomial Turing reduction.
- If there is a polynomial algorithm for any NP-hard problem, then there are polynomial algorithms for all problems in NP, and hence P = NP.
- If P ≠ NP, then NP-hard problems have no solutions in polynomial time, while P = NP does not resolve whether the NP-hard problems can be solved in polynomial time.
- If an optimization problem H has an NPC decision version L, then H is NP-hard.

- If H is in NP, then H is also NPC because in this case the existing polynomial Turing transformation fulfils the requirements of polynomial-time transformation.

A common mistake is to think that the 'NP' in 'NP-hard' stands for 'non-polynomial'. Although it is widely suspected that there are no polynomial-time algorithms for these problems, this has never been proved.

An example of an NP-hard problem is the following decision problem called SUBSET-SUM: Given a set of integers, does any non-empty subset of them add up to zero? That is a *yes–no* question, and happens to be NP-complete. Another example of an NP-hard problem is the optimization problem of finding the least-cost route through all nodes of a weighted graph. This is commonly known as the Travelling Salesman Problem (TSP).

There are also decision problems that are NP-hard, but not NP-complete; for example, the halting problem. This is the following problem: 'Given a program and its input, decide if it will run forever?' That's a *yes–no* question. So this is a decision problem. It is easy to prove that the halting problem is NP-hard, but not NP-complete. For example, the Boolean satisfaction problem can be reduced to the halting problem by transforming it to the description of a TM that tries all truth value assignments and when it finds one that satisfies the formula, it halts; otherwise, it goes into an infinite loop. It is also easy to see that the halting problem is not in NP since all problems in NP are decidable in a finite number of operations, while the halting problem, in general, is not. But not all NP-Hard problems that are not NPC are undecidable. For instance, the language of True quantified Boolean formulas is decidable in polynomial space, but probably not in non-deterministic polynomial time.

An alternative definition of NP-hard that is often used restricts NP-Hard to decision problems and then uses polynomial-time many–one reduction instead of Turing reduction. So, formally, a language L is NP-hard if $\forall L' \in$ NP, $L' \leq_p L$. If it is also the case that L is in NP, then L is called NP-complete.

8.7 Examples of Problems in Different Classes

Example 8.1 Long simple paths

A *simple path* in a graph is just one without any repeated edges or vertices. To describe the problem of finding long paths in terms of complexity theory, we need to formalize it as a yes-or-no question: Given a graph G, vertices s and t, and a number k, does there exist a simple path from s to t with at least k edges? A solution to this problem would then consist of such a path.

Why is this in NP? If you are given a path, you can quickly look at it and add up the length, double-checking that it really is a path with length at least *k*. This can be done in linear time; so certainly it can be done in polynomial time.

However, we do not know whether this problem is in P, as no procedure exists to find a best path (with time polynomial in m, n, and K). As one has to explore all possible ways from every possible node treating it as a possible start state or a possible end state, we believe that no such algorithm exists. This makes the problem NP-complete.

There are algorithms that solve the problem above. For instance, the following is such an algorithm: List all 2^m subsets of edges, and check whether any of them solves the problem. But as far as we know, there is no algorithm that runs in polynomial time.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example 8.1 Cryptography

Suppose we have an encryption function, for example,

Code = RSA (key, text)

The 'RSA' encryption works by performing some simple integer arithmetic on the code and the key, which consists of a pair (p, q) of large prime numbers. One can perform the encryption only knowing the product pq; but to decrypt the code you instead need to know a different product, $(p - 1)(q - 1)$.

A standard assumption in cryptography is the 'known plaintext attack': We have the code for some message, and we know (or can guess) the text of that message. We want to use that information to discover the key, so we can decrypt other messages sent using the same key.

Formalized as an NP problem, we simply want to find a key for which code = RSA (key, text). If you are given a key, you can test it by doing the encryption yourself: so this is in NP.

The hard question is, how do you find the key? For the code to be strong, we hope it is not possible to do much better than a brute force search.

Another common use of RSA involves 'public key cryptography': A user of the system publishes the product pq, but does not publish p, q or $(p - 1)(q - 1)$. That way anyone can send a message to that user by using the RSA encryption, but only the user can decrypt it. Breaking this scheme can also be thought of as a different NP problem: Given a composite number pq, find a factorization into smaller numbers.

One can test a factorization quickly (just multiply the factors back together again). So the problem is in NP. Finding a factorization seems to be difficult, and we think it may not be in P. However, there is some strong evidence that it is not NPC either. It seems to be one of the (very rare) examples of problems between P and NPC in difficulty.

Example 8.1 Chess

We have seen in the news recently a match between the world chess champion Gary Kasparov and a very fast chess computer Deep Blue. The computer lost the match, but won one game and tied others.

What is involved in chess programming? Essentially, the sequences of possible moves form a tree: The first player has a choice of 20 different moves (most of which are not very good), after each of which, the second player has a choice of many responses, and so on. Chess-playing programs work by traversing this tree finding what the possible consequences of each different move would be.

The tree of moves is not very deep–a typical chess game might last 40 moves, and it is rare for one to reach 200 moves. Since each move involves a step by each player, there are at most 400 positions involved in most games. If we traversed the tree of chess positions only to that depth, we would need enough memory to store the 400 positions on a single path at a time. This much memory is easily available on the smallest computers you are likely to use.

So perfect chess playing is a problem in PSPACE. (Actually one must be more careful in definitions. There are only a finite number of positions in chess. So, in principle you could write down the solution in constant time. But that finite number would be very large. Generalized versions of chess on larger boards are in PSPACE).

The reason this deep game-tree search method cannot be used in practice is that the tree of moves is very bushy, so that, even though it is not deep, it has an enormous number of vertices. We will not run out of space if we try to traverse it, but we will run out of time before we get through even a small fraction of the way. Some pruning methods, notably 'alpha–beta search', can help reduce the portion of the tree that needs to be examined, but they are not enough to solve this difficulty. For this reason, actual chess programs, instead, only search a much smaller depth (such as up to 7 moves), at which point they do not have enough information to evaluate the true consequences of the moves and are forced to guess by using heuristic 'evaluation functions' that measure simple quantities such as the total number of pieces left.

Example 8.1 Halting problem

Suppose you are working on a lab for a programming class, suppose you have written your program, and also suppose you started to run it. After five minutes, it is still going. Does this mean it is in an infinite loop, or is it just slow?

It would be convenient if your compiler could tell you that your program has an infinite loop. However, this is an undecidable problem: There is no program that will always correctly detect infinite loops.

Some people have used this idea as evidence that people are inherently smarter than computers, since it shows that there are problems computers cannot solve. However, it is not clear to me that people can solve them either. Here's an example:

```
Main ( )
{
int x = 3;
for (;;) {
    for (int a = 1; a <= x; a ++)
    for (int b = 1; b <= x; b ++)
    for (int c = 1; c <= x; c ++)
    for (int i = 3; i <= x; i ++)
        if (pow(a, i) + pow(b, i) == pow(c, i))
        exit;
    x++;
}
}
```

This program searches for solutions to Fermat's last theorem. Does it halt? (You can assume I am using a multiple-precision integer package instead of built-in integers, so you do not have to worry about arithmetic overflow complications.) To be able to answer this, you have to understand the recent proof of Fermat's last theorem. There are many similar problems for which no proof is known; so we are clueless whether the corresponding programs halt.

8.8 NP-completeness

The theory of NP-completeness is a solution to the practical problem of applying complexity theory to individual problems. NPC problems are defined in a precise sense as the

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

hardest problems in P. Even though we do not know whether there is any problem in NP that is not in P, we can point to an NPC problem and say that if there are any hard problem in NP, that problem is one of the hard ones.

(Conversely if everything in NP is easy, then all problems are easy. So NP-completeness can be thought of as a way of making the big P = NP question equivalent to smaller questions about the hardness of individual problems.)

So if we believe that P and NP are unequal, and we prove that some problem is NP-complete, we should believe that it does not have a fast algorithm.

For unknown reasons, most problems we have looked at in NP turn out either to be in P or NP-complete. So the theory of NP-completeness turns out to be a good way of showing that a problem is likely to be hard, because it applies to a lot of problems. But there are problems that are in NP, that are not known to be in P, and are not likely to be NP-complete. Example: The halting problem given before.

8.9 Reduction

Formally, NP-completeness is defined in terms of 'reduction', which is just a complicated way of saying one problem is easier than another.

We say that A is easier than B (and write A < B) if we can write down an algorithm for solving A that uses a small number of calls to a subroutine for B (with everything outside the subroutine calls being fast, polynomial time). There are several minor variations of this definition depending on the detailed meaning of 'small'–it may be a polynomial number of calls, a fixed constant number or just one call.

Then if A < B and B is in P, so is A: We can write down a polynomial algorithm for A by expanding the subroutine calls to use the fast algorithm for B.

So, 'easier' in this context means that if one problem can be solved in polynomial time, so can the other. It is possible for the algorithms for A to be slower than those for B, even though A < B.

As an example, consider the Hamiltonian cycle problem: Does a given graph have a cycle visiting each vertex exactly once? Here is a solution, using the longest path as a subroutine:

```
for each edge (u, v) of G
if there is a simple path of length n – 1 from u to v
   return yes // path + edge form a cycle
   return no
```

This algorithm makes m calls to a longest path subroutine, and does O (m) work outside those subroutine calls. So it shows that Hamiltonian cycle < longest path. (It does not show that Hamiltonian cycle is in P, because we do not know how to solve the longest path subproblems quickly.)

As a second example, consider a polynomial-time problem such as the minimum spanning tree. For every other problem B, B < minimum spanning tree, since there is a fast algorithm for minimum spanning trees using a subroutine for B. (We do not actually have to call the subroutine, or we can call it and ignore its results.)

Example 8.1 Fifteen puzzle problem

The n-puzzle is a sliding puzzle that consists of a frame of numbered square tiles in random order with one tile missing. If the size of puzzle is 3×3, it is called the 8-puzzle or 9-puzzle; and if size is 4×4, the puzzle is called the 15-puzzle or 16-puzzle. For the puzzle with initial configuration in Figure 8.3, the object of the puzzle is to place the tiles in order as shown in Figure 8.4, by making sliding moves that use the empty space.

14	2	3	
5	6	15	8
9	7	11	12
13	4	1	10

Fig. 8.3 *Initial Configuration*

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Fig. 8.4 *Final Configuration*

The n-puzzle is a classical problem for modelling algorithms involving heuristics. A simple heuristics that can be used for estimating the time to solve this problem are

1. Count the number of misplaced tiles
2. Find the sum of the Manhattan distances between each block and its position in the goal configuration

Both heuristics are admissible, that is, they never overestimate the number of moves left, which ensures optimality for certain search algorithms.

A simple parity argument shows that half of the starting states for the n-puzzle are impossible to resolve, no matter how many moves are made. This is because some states may be having invalid moves or some times the states are repeated. The possible states can be divided into two equivalence classes and they can be labelled as reachable or unreachable states.

The time complexity includes the moves made in permutations of all 16 squares and the distance moved by the empty square. Each move changes the parity of the permutation and the parity distance. If the empty square is not moved, the permutation of the remaining pieces must be even. The same works for all rectangular boards or, more generally, for all boards with no odd cycles.

For larger versions of the n-puzzle, finding a solution is easy, but the problem of finding the shortest solution is NP-hard. For the 15-puzzle, lengths of optimal solutions range from 0 to 80 moves; the 8-puzzle can be solved in 31 moves or fewer.

Example 8.6 Knapsack problem

Knapsack problem is defined as a selection problem where one should choose boxes to fill the bag so as to maximize the amount of money, maintaining the overall weight within capacity of the bag. It is a multiple-constraint problem which considers weight, volume,

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

shape, size and cost. It is also a decision problem as the bag needs to be filled with most useful items without exceeding a given weight W.

In Figure 8.5, we have n kinds of items, 1 through n. Each kind of item has a value p_j and a weight w_j. We usually assume that all values and weights are non-negative. The maximum weight that we can carry in the bag is W.

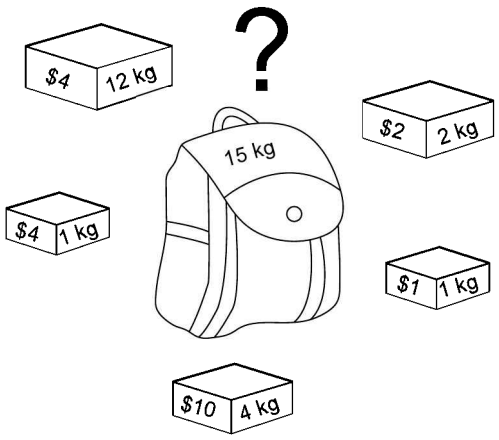


Fig. 8.5 *Instance of Knapsack Problem*

In general, the problem of 0–1 knapsack is formulated as follows:

$$\text{Maximize } \sum_{j=1}^{n} p_j x_j$$

$$\text{subject to } \sum_{j=1}^{n} w_j x_j \leq w,\ x_j \in \{0, 1\}$$

The bounded knapsack problem restricts the number x_j of copies of each kind of item to a maximum integer value b_j. Mathematically the bounded knapsack problem can be formulated as follows:

$$\text{Maximize } \sum_{j=1}^{n} p_j x_j$$

$$\text{subject to } \sum_{j=1}^{n} w_j x_j \leq w,\ x_j \in \{0,1,.....,b_j\}$$

Note: Unbounded knapsack problem places no upper bound on the number of copies of each kind of item.

Of particular interest is the special case of the problem with the following properties:

- it is a decision problem,
- it is a 0–1 problem,
- for each kind of item, the weight equals the value: $w_j = p_j$.

8.9.1 Computational Complexity

The knapsack problem is interesting from the perspective of computer science because

- Using Dynamic Programming approach, it takes pseudo-polynomial time.
- There is a fully polynomial-time approximation scheme, which uses the pseudo-polynomial time algorithm as a subroutine
- The problem is NPC to solve exactly. Hence, it is expected that no algorithm can be both correct and fast on all cases.
- Many cases that arise in practice and 'random instances' from some distributions, can nonetheless be solved exactly.

One theme in the research literature is to identify what the 'hard' instances of the knapsack problem look like, or, viewed another way, to identify the properties of instances in practice that might make them more amenable than their worst-case NPC behaviour suggests.

Several algorithms are freely available to solve knapsack problems, based on dynamic programming approach, on branch and bound approach or on hybrids of the two approaches.

Solutions: If all weights (w_1, ..., w_n and W) are non-negative integers, the knapsack problem can be solved in pseudo-polynomial time using dynamic programming. The following describes a dynamic programming solution for the *unbounded* knapsack problem.

We assume that all weights are strictly positive ($w_j > 0$) and wish to maximize the total value, subject to the constraint that total weight is less than or equal to a fixed *W*.

For each weight $Y \leq W$, define A(Y) to be the maximum value that can be attained with total weight less than or equal to Y. The properties of A(Y) are as follows:

- $A(0) = 0$ (the sum of zero items, i.e. the summation of the empty set)
- $A(Y) = \max \{p_j + A(Y - w_j) \mid w_j \leq Y\}$
  where p_j is the value of the jth kind of item.

Solution is obtained by calculating the values from A(0) up through A(W). Since the calculation of A(Y) involves examining n items, and there are W values of A(Y) to calculate, the running time of the dynamic programming solution is O(nW).

Dividing w_1, ..., w_n, W by their greatest common divisor is an obvious way to improve the running time.

The O(nW) complexity does not contradict the fact that the knapsack problem is NP-complete, since W, unlike n, is not polynomial in the length of the input to the problem. The length of the input to the problem is proportional to the number of bits in W, not to W itself.

8.9.2 0–1 Knapsack Problem

A similar dynamic programming solution for the 0–1 *knapsack problem* also runs in pseudo-polynomial time. As above, we define A(j, Y) to be the maximum value that can be attained with weight less than or equal to Y using items up to j.

We can define A(j, Y) recursively as follows:

- $A(0, Y) = 0$
- $A(j, 0) = 0$

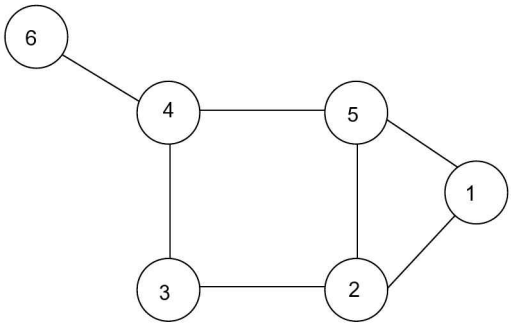
Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

- $A(j, Y) = A(j - 1, Y)$ if $w_j > Y$
- $A(j, Y) = \max \{A(j - 1, Y), p_j + A(j - 1, Y - w_j)\}$ if $w_j \leq Y$.

The solution can then be found by calculating A(n, W). To do this efficiently, we can use a table to store previous computations. This solution will therefore run in O(nW) time and O(nW) space, although with some slight modifications we can reduce the space complexity to O(W).

Example 8.7 Clique problem

A clique in a graph is a set of pair wise adjacent vertices or, in other words, an induced subgraph that is a complete graph. In the graph shown below, the vertices 1, 2 and 5 form a clique as the subgraph formed with these nodes is a complete graph.



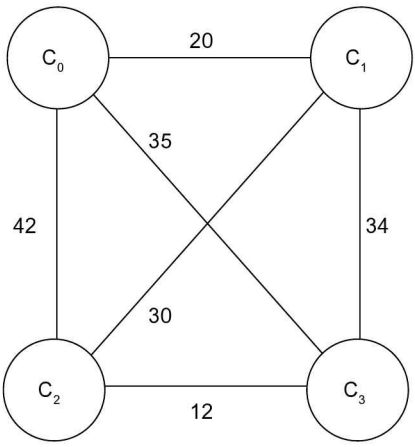
The clique problem is the problem of determining whether a graph contains a clique of size at least k. Once we have located k or more vertices, it is trivial to verify if they form a clique. This is why the clique problem is in NP. The corresponding optimization problem, the maximum clique problem, is to find the largest clique in a graph.

In computational complexity theory, the clique problem is a graph-theoretic NPC problem. The NP-completeness of the clique problem follows trivially from the NP-completeness of the independent set problem, because there is a clique of size at least *k* if and only if there is an independent set of size at least *k* in the complement graph. This is easy to see, since if a subgraph is complete, its complement subgraph has no edges at all.

A brute force algorithm to find a clique in a graph is to examine each subgraph with at least k vertices and check if it forms a clique. This algorithm is polynomial if k is the number of vertices, or a constant less than this, but not if k is, say, half the number of vertices.

A heuristic is to start by considering each node to be a clique of size one, and to merge cliques into larger cliques until there are no more possible merges. Two cliques A and B may be merged if each node in clique A is adjacent to each node in clique B. This requires only linear time (linear in the number of edges), but may fail to find a large clique because two or more parts of the large clique have already been merged with nodes that are not in the clique. The algorithm can be implemented most efficiently using the disjoint-set data structure.

Example 8.8 Travelling salesman problem



The Travelling Sales Person (TSP) is stated as a problem to find a shortest possible tour that visits each city exactly once in the given set of cities and the distance between a pair of cities. This problem is formulated as an optimization problem represented as a graph and is used as a benchmark for many optimization methods.

TSP can be modelled as a graph, where the cities are the vertices, graph is considered as and the edges are the distances between the two connected cities a complete graph. The optimal TSP tour is the shortest Hamiltonian cycle. The no edge exists between two cities, then a new edge with arbitrarily long distance is added without affecting the optimal tour. This can also be formulated as either directed or undirected graph depending on whether there exists path in one or both directions.

8.9.3 Computational Complexity

Travelling Sales Person (TSP) problem is defined to be NP-hard for the case when the cities are in the plane with Euclidean distances, as well as in a number of other restrictive cases. Removing the condition of visiting each city 'only once' does not remove the NP-hardness, since it is easily seen in the planar case, that there is an optimal tour that visits each city only once.

The problem when formulated as a decision problem where 'given the costs and a number x, decide whether there is a round-trip route cheaper than x' would be an NPC problem.

In the theory of computational complexity, the decision version of TSP belongs to the class of NPC problems. Thus, it is assumed that there is no efficient algorithm for solving TSPs. In other words, it is likely that the worst-case running time for any algorithm for TSP increases exponentially with the number of cities. So, even some instances with only hundreds of cities will take many CPU years to solve exactly.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Solved Problems

Problem 1: Show that the travelling salesman problem is in class NP.

Solution: The input to the travelling salesperson problem (TSP) is a graph with integer weights on the edges as shown in the Figure 8.6 and a weight limit w.

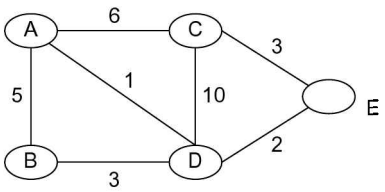


Fig. 8.6 *Simple Weighted Graph*

The question is whether the graph has a Hamiltonian circuit of total weight w. A Hamiltonian circuit is a set of edges that connects the nodes into a single cycle with each node appearing exactly once. Note that the number of edges on a Hamiltonian cycle must be equal to the number of nodes in the graph. If there was a real computer that was non-deterministic, no branch would use more than O(n) steps if the input was of length n. On a multiple tape TM we can guess a permutation in $O(n^2)$ steps and check its total weight in a similar amount of time; thus, a single-tape NTM can solve the TSP in at most $O(n^4)$ time. We conclude that TSP belongs to the complexity class NP.

Problem 2: Show that Kruskal's algorithms is in class P.

Solution: These algorithms were developed by Joseph Kruskal. Kruskal algorithms create a minimum spanning tree T by adding the edges one at a time to T. A minimum cost spanning tree is built edge by edge. We start with the edge of minimum cost. However, if there are several edges with the same minimum cost, then we select one of them and add it to the spanning tree T provided its addition does not form a cycle. We then add with the next lower cost, and so on. We repeat this process until we have selected N–1 edges to form the complete minimum cost spanning tree. This algorithm selects the edges for addition in the minimum spanning tree in the increasing order of their cost. We have to remember here that the edges can only be added if it does not form a cycle. Let us take the following graph to find out the MST using Kruskal algorithms. We can organize the whole process in a tabular form with three rows and the number of columns equal to the number of edges. The first row contains the edges in the descending order of their cost, the second row contains the cost and the third contains A if the corresponding edge is added. This can be shown for the fiven graph as shown in Figure 8.7.

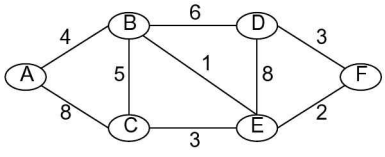


Fig. 8.7 *Simple Weighted Graph*

Edges	EB	EF	DF	CE	AB	BC	BD	DE	AC
Cost	1	2	3	3	4	5	6	8	8
Add	A	A	A	A	A				

The complexity of finding the MST using prims's method id $O(n^2)$, where n is number of vertices, for Kruskal method, the complexity is O(e2), where e is the number of edges. So, the problem of finding the MST belongs to class P.

Problem 3: Show that the satisfiability problem is in class NP.

Solution: The Boolean Satisfiability problem (SAT) is a decision problem considered in complexity theory. Suppose we have a Boolean expression which is made up of the variables (x_1, x_2, x_3 … x_n), parentheses and Boolean operators $\vee$, $\wedge$ and $\neg$ where these operators are for logical OR, AND and NOT respectively. A truth assignment for a Boolean expression depends on the values to the variables so that the whole expression is true. Now the question that arises is as follows. Do there exist values of the logical variables (x_1, x_2, x_3… x_n) to make a given Boolean expression true?. Thus SAT is used to determine whether there exists a true or false assignment to the variables such that all clauses are evaluated to be true making entire expression true? The Boolean expression is said to be satisfied if truth values can be assigned to its free variables in such a way that the formula becomes true. SAT clearly belongs to the complexity class NP because we can guess a truth assignment and verify that it satisfies the Boolean expression in polynomial time.

Summary

1. The problems which have Yes or No answer are called decision problems.
2. Problems that have an efficient algorithm to solve it are called decidable problems.
3. Problems that cannot be solved with an algorithm are undecidable problems.
4. The class of decision problems that can be solved in non-deterministic polynomial are called NP problems.
5. A problem is said to be NPC if it is contained in the class NP and all other problems in NP can be polynomially reduced to it.
6. A language is said to NPC if, L is in NP and for every language L' in NP, there is a polynomial-time reduction of L' to L.
7. If any NPC problem is in P, then P = NP.
8. *P* is the set of languages L such that L = L(M) for some non-deterministic TM M of time complexity T(n) where T(n) is a polynomial.
9. NP is the set of languages L such that L = L(M) for some non-deterministic TM M, where on any input of length n, there are no sequences of more than T(n) moves of M, where T(n) is a polynomial.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Short Answers

1. Can P and NP complete problems be NP hard.
   Answer: All NP complete problems are NP hard, but all P problems are not NP hard.
2. What is the relation between NP hard and NP complete.
   Answer: All NP – complete problems are NP hard but some NP hard problems are known not to be NP complete.
3. Give an example for NP-hard problem and justify the statement.
   Answer: SUM of SUBSET problem is said to be NP-Hard as it is a decision problem where given set of integers where and need to find the sum of them that adds up to zero.
4. What is the time complexity of Hamiltonian cycle and classify the type of problem.
   Answer: The time complexity of Hamiltonian cycle is O(m*n) where m is the number of edges in the graph and n is the number of nodes in the graph.
5. Is travelling sales person problem is NP Complete?
   Answer: Travelling sales person problem is NP Complete as the time complexity of any algorithm for TSP increases exponentially as number of cities increase.

Fill in the Blanks

1. _____ are those problems that can be solved by a Turing machine in polynomial time.
2. _____ are those problems that can be solved in non-deterministic Turing machine in polynomial time.
3. Search problems are _____problems.
4. Sum of subsets problem is _____ problem.
5. If P = NP, then NP-complete problem is in _____.
6. NP-complete problem is the class of problems which are NP-hard and belong to NP. (State True/False)
7. Every NP problem reduces to SAT. (True/False)
8. If any NP-complete problem is polynomial time solvable, then P = NP. (True/False)
9. Travelling salesman problem is NP complete problem. (True/False)
10. A is undecidable if A is reduced to B and B is undeciadable. (True/False)

Answers

1. P problems	2. NP	3. NP-hard	4. NP-hard	5. P
6. False	7. True	8. True	9. True	10. True

Objective Question Bank

1. An NP hard problem is a problem
   a) to which all the NP problems are polynomially reducible.
   b) which is harder than NP problem.
   c) which is not in NPC.
   d) which is in NP.

2. Which of the following statements is incorrect?
   a) P is a class of polynomial time or quick calculations.
   b) NP is the class of search problems.
   c) NP-complete is the hardest search problems.
   d) None of the above.

3. Which of the following statements is correct?
   a) If any NP-complete problem is contained in P, then we can conclude that P = NP.
   b) If any NP-complete problem is contained in P, then we can conclude that P ≠ NP.
   c) A problem is said to be computable if there exists an algorithm to solve it.
   d) A problem is said to be tractable if there exists an efficient algorithm to solve it.

4. Which of the following problems belongs to the NP-complete complexity class?
   a) Hamiltonian cycle.
   b) Satisfaction problem (SAT).
   c) 3-colouring.
   d) All the above.

5. If A is reducible to B and B is undecidable, then
   a) A is decidable.
   b) A is undecidable.
   c) B is decidable.
   d) None of these.

6. Which of the following problems is NP-complete?
   a) travelling salesman problem.
   b) 3-colouring.
   c) set partitioning.
   d) All the above.

7. Which of the following statements is not correct?
   a) If any problem in NP is not polynomial-time solvable, then all NP-complete problems are polynomial-time solvable.
   b) If any problem in NP is not polynomial-time solvable, then all NP-complete problems are not polynomial-time solvable.
   c) If any NP-complete problem is polynomial-time solvable, then P ≠ NP.
   d) All the above.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

8. Which of the following statements is not correct?
   a) NP = P iff the satisfibility problem is a problem.
   b) P is the class of problems which can be solved by a deterministic polynomial algorithm.
   c) The circuit satisfaction is NP hard.
   d) If any NPC problem can be solved in polynomial time, then all NP problems can be solved in polynomial time.
9. Problem A is NP complete iff
   a) A is NP-hard.
   b) A is NP.
   c) Both a and b are true.
   d) A is in complexity class of NP.
10. Intractable problems are
   a) Not solvable.
   b) Not in NP.
   c) Not in P.
   d) In NP-complete.

Answers

1. a 2. d 3. b 4. d 5. b 6. d 7. a 8. c 9. c 10. c

Exercises

1. Determine whether the following problems are P, NP or NP complete.
   a) Satisfibility Problem.
   b) Travelling Salesman Problem.
   c) Knapsack Problem.
   d) Set Partitioning.
   e) Hamiltonian Cycle.
2. Explain the different complexity classes.
3. What is the difference between NP-hard problems and NP-complete problems.
4. Show that finding minimum spanning tree is in Class P.
5. How are P problems different from NP-problems?

LR(k) and LL(1) Grammars 9

- The most widely used top-down parsers are constructed with LL(1) grammars. The most powerful bottom-up parsers are constructed with LR(k) grammars. The language defined by LR(k) grammars is DCFL.

In this chapter, we introduce a restricted type of CFG called LR(0) Grammar. We discuss the question what are LL(1), LR(0) and LR(1) grammars. Given a grammar, we discuss how to test whether it is LL(1) or LR(0) or LR(1).

9.1 LL(1) Grammar

A grammar that is suitable for LL(1) parser construction is called LL(1) grammar.

Given a grammar, to check whether it is LL(1) or not, we use two functions called First( ) and Follow( ).

Let us now understand the functions First( ) and Follow( ).

The function **First (x)** (where x is a grammar symbol) gives set of terminals that may be the first symbol in any string derived from x.

The function First (x) is evaluated as follows.

1. If x is a terminal, then First (x) is x itself.

   Ex: First(+) = {+}, First(id) = {id}.

2. If x is a non-terminal
   a) & is defined with ε-rules, that is, $X \to \varepsilon$, then First(X) = {ε}.
   b) & is defined with non-ε-rules, that is, $X \to A_1A_2A_3$ then

$$\text{First}(x) = \text{First}(A_1A_2A_3) = \text{First}(A_1) \text{ if } A_1 \overset{*}{\Rightarrow} \varepsilon$$
$$= \text{First}(A_1) - \{\varepsilon\} \cup \text{First}(A_2A_3) \text{ if } A_1 \overset{*}{\Rightarrow} \varepsilon$$

Example 8.7 Find First(·) of each non-terminal in the grammar given:

$$S \to AB$$
$$A \to a \mid \varepsilon$$
$$B \to b \mid \varepsilon$$

Solution: First(S) = First(AB) = First(A)

First(A) is {a, ε}

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Since there is ε, add a, then continue with First(B).
First(B) is{b, ε}.
Since there is ε, add b, then ε.
Hence, First(S) = {a, b, ε}.

Example 8.7 Find First(·) of each non-terminal in the grammar given:

$$S \rightarrow ABCDE$$
$$A \rightarrow a \mid \varepsilon$$
$$B \rightarrow b \mid \varepsilon$$
$$C \rightarrow c \mid \varepsilon$$
$$D \rightarrow d \mid \varepsilon$$
$$E \rightarrow e$$

Solution: First(S) = First(ABCDE) = First(A)
First(A) is {a, ε}
Since there is ε, add a, then continue with First(B).
First(B) is{b, ε}.
Since there is ε, add b, then continue with First(C).
First(C) is{c, ε}.
Since there is ε, add c, then continue with First(D).
First(D) is{d, ε}.
Since there is ε, add d, then continue with First(E).
First(E) is{e} i.e terminal only hence evaluation stops here.
Hence, First(S) = {a, b, c, d, e}.

When A is a non-terminal, Follow(A) gives the set of terminals that may follow immediately to the right of A while deriving any string from the grammar.

Follow(A) is evaluated by using following rules:

- If A is a start symbol, then Follow(A) is {$}.
- If $S \rightarrow \alpha A \beta$ is in G, then Follow(A) = First(β) – {ε}
- If $S \rightarrow \alpha A$ or $S \rightarrow \alpha A \beta$ where $\beta \overset{*}{\Rightarrow} \varepsilon$, then Follow(A) = Follow(S).

Example 8.7 Find Follow(·) of each non-terminal in the grammar

$$S \rightarrow aABb$$
$$A \rightarrow c \mid \varepsilon$$
$$B \rightarrow d \mid \varepsilon$$

Solution: Follow(S) = {$} as S is start symbol.
Follow (A) = First(Bb)
First(Bb) is First(B) = {d, ε}
Since there is ε, add d, then continue with First(b).
Hence, First(A) = {d, b}.
Follow (B) = First(b) = {b}

Example 8.7 Find Follow(·) of each non-terminal in the grammar

$$S \rightarrow ABCDE$$
$$A \rightarrow a \mid \varepsilon$$

$$B \rightarrow b \mid \varepsilon$$
$$C \rightarrow c \mid \varepsilon$$
$$D \rightarrow d \mid \varepsilon$$
$$E \rightarrow e \mid \varepsilon$$

Solution: Follow(S) = {$} as S is start symbol.
Follow (A) = First(BCDE) = First(B)
First(B) is {b, ε}.
Since there is ε, add b, then continue with First(C).
First(C) is {c, ε}.
Since there is ε, add c, then continue with First(D).
First(D) is {d, ε}.
Since there is ε, add d, then continue with First(E).
First(E) is {e, ε}.
Since there is ε, add Follow(S), that is, $
Hence, Follow(A) = {b, c, d, e, $}.
Similarly, Follow(B) = {c, d, e, $}
Follow(C) = {d, e, $}
Follow(D) = {e, $}
Follow(E) = {$}

9.2 Rules for Verifying Whether the Given Grammar Is LL(1) or Not

1. A grammar without null rules is LL(1) provided for each production of the form $A \rightarrow \beta_1 \mid \beta_2 \mid \beta_3$, First($\beta_1$), First($\beta_2$), First($\beta_3$),. must be mutually disjoint, that is,
   i) First(β_i) $\cap$ First(β_j) = $\varnothing$
2. A grammar with null rules is LL(1) provided for each production of the form
   a) $A \rightarrow \beta \mid \varepsilon$, First($\beta$) and Follow(A) must be mutually disjoint, that is,
   i) First(β) $\cap$ Follow(A) = $\varnothing$
3. An ambiguous grammar cannot be LL(1).
4. Left recursive grammar is not LL(1).

Given any grammar, by using the above rules, we can verify for LL(1).

Example 8.7 Check whether the following grammar is LL(1) or not.

$$S \rightarrow aB \mid \varepsilon$$
$$B \rightarrow bC \mid \varepsilon$$
$$C \rightarrow cS \mid \varepsilon$$

Solution: For each production, apply Rule 2.

$S \rightarrow aB \mid \varepsilon$ … First(aB) $\cap$ Follow(S) is {a} $\cap$ {$} = $\varnothing$.

Hence, the condition is satisfied.

$B \rightarrow bC \mid \varepsilon$ … First(bC) $\cap$ Follow(B) is
{b} $\cap$ {$} = $\varnothing$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Hence, the condition is satisfied.

$$C \rightarrow cS|\ \varepsilon \ldots \text{First}(c) \cap \text{Follow}(C) \text{ is}$$
$$\{c\} \cap \{\$\} = \varnothing$$

Hence, the condition is satisfied.
So, the given grammar is LL(1).

Example 8.8 Check whether the following grammar is LL(1) or not.

$$S \rightarrow A \mid a$$
$$A \rightarrow a$$

Solution: This is an ambiguous grammar. The sentence a can be derived in two ways. $S \rightarrow A \rightarrow a$ or $S \rightarrow a$

So, the grammar is ambiguous. According to Rule 3, it is not LL(1).

Example 8.7 Check whether the following grammar is LL(1) or not.

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L, S \mid S$$

Solution: This is a left recursive grammar. According to Rule 4, it is not LL(1).

Example 8.8 Check whether the following grammar is LL(1) or not.

$$S \rightarrow aB \mid b$$
$$B \rightarrow bC \mid b$$
$$C \rightarrow cS \mid d$$

Solution: For each production apply Rule 2.

$S \rightarrow aB \mid b \ldots \text{First}(aB) \cap \text{First}(b)$ is

$$\{a\} \cap \{b\} = \varnothing$$

Hence, the condition is satisfied.

$B \rightarrow bC \mid b \ldots \text{First}(bC) \cap \text{First}(b)$ is

$$\{b\} \cap \{b\} \neq \varnothing$$

Hence, the condition is not satisfied.
So, the given grammar is **not** LL(1)

9.3 LR(K) Grammars

LR(k) grammars play an important role in the study of programming languages and designing of compilers. It stands for left-to-right scanning of input string producing a right-most derivation using k-symbol look ahead in the input string.

For any context free grammar defined as G = {V, T, P, S}, there are strings valid in the language defined as $L(G) = \{w \mid w \in T^*\}$. To find the production applied in the last step to get w, we can represent w through substrings, $\alpha\beta\gamma$, which is obtained by substituting $A \rightarrow \beta$ in the last step. If it can be shown that $A \rightarrow \beta$ is a production substituted in the last step by looking ahead of k symbols, then G is called an LR(k) grammar.

Example 8.8 Let G be $S \to AB, A \to aAb \mid \varepsilon, B \to Bb \mid b$.

It is easy to see that $L(G) = \{a^m b^n \mid n > m \geq 1\}$. Some sentential forms of G obtained by right-most derivation are AB, ABb^k, $a^mAb^mb^k$, a^mb^{m+k}, where $k \geq 1$. AB appears as the RHS of $S \to AB$. So AB may be a handle for AB or ABb^k. If we apply the handle to AB, we get $S \underset{R}{\Rightarrow} AB$. If we apply the handle to ABb^k, we get $Sb^k \underset{R}{\Rightarrow} ABb^k$. But Sb^k is not a sentential form. So to decide whether AB can be a handle, the input has to be scanned to the right of AB. If ε can be replaced by B, then AB serves as handle. By looking ahead of one symbol, we can decide whether AB is a handle. Similarly we can see that the correct handle can be determined by looking ahead of one symbol for various sentential forms.

Definition 1: Let $G = \{V, T, P, S\}$ be a CFG in which $S \overset{n}{\Rightarrow} S$ only when $n = 0$. G is an LR(k) grammar ($k \geq 0$) if

a) $S \underset{R}{\overset{*}{\Rightarrow}} \alpha Aw \underset{R}{\Rightarrow} \alpha\beta w$, where $\alpha, \beta \in V^*, w \in T^*$.

b) $S \underset{R}{\overset{*}{\Rightarrow}} \alpha' A' w' \underset{R}{\Rightarrow} \alpha' \beta' w'$, where $\alpha' \beta' \in V^*, w' \in T^*$ and

c) the first $|\alpha\beta| + k$ symbols of $\alpha\beta w$ and $\alpha' \beta' w'$ coincide. Then $\alpha = \alpha', A = A', \beta = \beta'$.

9.4 Properties of LR(k) Grammars

Every LR(k) grammar G is unambiguous.

1. If G is in LR(k) grammars, there exists a deterministic pushdown automaton A accepting L(G).
2. If A is a deterministic pushdown automaton A, there exists an LR(k) grammar G such that L(G) = N(A).
3. If G is an LR(k) grammar, where $k > 1$, then there exists an equivalent grammar G1 which is LR(1). In so far as languages are concerned, it is enough to study the languages generated by LR(0) grammars and LR(1) grammars.
4. The class of deterministic languages is a proper subclass of the class of CFL.
5. DCFL is closed under complementation, but not under union and intersection.
6. A CFL is generated by an LR(0) grammar if and only if it is accepted by a DPDA and has the prefix property.
7. There is an algorithm to decide whether a given CFG is LR(k) for given natural number k.

Example 9.10 Show that the grammar $S \to 0A2, A \to 1A1 \mid 1$ is not an LR(0) grammar.

Solution: Consider the string 01112 belonging to the language. To derive the string,

0 1 1 1 2	Look ahead of 1 symbol is needed to select A.
0 1 A 1 2	No look ahead is needed to select A
0 A 2	No look ahead is needed to select S.
S	

Since we need at least one symbol to be looked ahead for proper substitution, it is not LR(0) grammar.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Example 9.11 Is the grammar S → C | D, C → aC | b, D → aD | C is LR(0)?

Solution: Consider the string aaaab belonging to the language. To derive the string,

a a a b	No look ahead is needed to select C.
a a a C	No look ahead is needed to select C
a a C	No look ahead is needed to select C
a C	No look ahead is needed to select C
S	No look ahead is needed to select S
	(Or)
a a a b	No look ahead is needed to select C.
a a a C	No look ahead is needed to select C
a a a D	No look ahead is needed to select D
a a D	No look ahead is needed to select D
a D	No look ahead is needed to select D
D	No look ahead is needed to select D
S	No look ahead is needed to select S

Since we do not need any look ahead of symbol for proper substitution, it is LR(0) grammar.

9.5 Construction of LR(0) Items for Context Free Grammars

A context free grammar is a quadruplet G = (V, T, P, S), where V, T and P are finite sets of non-terminal symbols, terminal symbols and production rules, respectively, and S ∈ V is the initial symbol of the grammar. The production rules in P are of the form A→ α with A ∈ V, α ∈ (N ∪ T)*. We represent the non-terminal symbols with capital letters and terminal symbols with lower-case letters. For any two strings v and w ∈ (N ∪ T)$^+$, we write V $\Rightarrow_G$w if w can be derived from V by substituting a non-terminal A in V by α applying the rule A → α in G.

The language L(G) generated by the grammar G is the set:

L(G) = {w ∈ T* | S $\Rightarrow_G^*$ w}

Given a context free grammar G, an LR(0) item for G is a production rule with a position marked in its right-hand side. For instance, given a production rule A → a B c, we can form four items: [A → •a B c], [A → a •B c], [A → a B• c] and [A → a B c•]. When no confusion is possible, we will drop the square brackets.

Intuitively, the meaning of an item A → α•β is that the parsing has already recognized α. The symbol that follows the marker is called the *goal symbol* and represents the next symbol to be processed by the parser. Once this symbol is recognized, the marker will be advanced.

9.6 Definition of LR(0) Grammar

A grammar is said to be LR(0) grammar if

- Its start symbol does not appear on the right-hand side of any other production.
- If the closure set of a item has a production of the form A → α•, then there is no production of the form B → β• or B → β•γ

Items

The construction of the parsing tables is based on the notion of LR(0) *items* (simply called *items* here) which are grammar rules with a special dot added somewhere in the right-hand side. For example, the rule E → E + B has the following four corresponding items:

$$E \rightarrow \bullet E + B$$
$$E \rightarrow E \bullet + B$$
$$E \rightarrow E + \bullet B$$
$$E \rightarrow E + B \bullet$$

Rules of the form A → ε have only a single item A → •. These rules will be used to denote the state of the parser. The item E → E• + B, for example, indicates that the parser has recognized a string corresponding with E on the input stream and now expects to read a + followed by another string corresponding with B.

Item sets

It is usually not possible to characterize the state of the parser with a single item because it may not know in advance which rule it is going to use for reduction. For example, if there is also a rule E → E * B, then the items E → E • + B and E → E• * B will both apply after a string corresponding with E has been read. Therefore, we will characterize the state of the parser by a set of items, in this case, the set {E → E• + B, E → E• * B}.

Closure of item sets

An item with a dot in front of a non-terminal, such as E → E + •B, indicates that the parser expects to parse the non-terminal B next. To ensure the item set contains all possible rules the parser may need in the midst of parsing, it must include all items describing how B itself will be parsed. This means that if there are rules such as B → 1 and B → 0, then the item set must also include the items B → •1 and B → •0. In general, this can be formulated as follows:

If there is an item of the form A → v • Bw in an item set and in the grammar there is a rule of the form B → w′, then the item B → • w′ should also be in the item set.

Any set of items can be extended such that it satisfies this rule: This is because we can simply continue to add the appropriate items until all non-terminals preceded by dots are accounted for. The minimal extension is called the *closure* of an item set and written as **closure**(I) where I is an item set. It is these closed item sets that we will take as the states of the parser, although only the ones that are actually reachable from the begin state will be included in the tables.

Augmented grammar

Before we start determining the transitions between the different states, the grammar is always augmented with an extra rule:

$$(0)\ S \rightarrow E$$

where S is a new start symbol and E is the old start symbol. The parser will use this rule for reduction exactly when it has accepted the input string.

Example 9.10 State whether the following grammar is an LR(0) grammar.

$$A \rightarrow aAa \mid B,\ B \rightarrow b$$

Solution: Add a new production as S → A.

Construct LR(0) items.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

I_0: S → •A 1
A → •a A a 1
A → •B 3
B → •b 4

I_1: S → A •

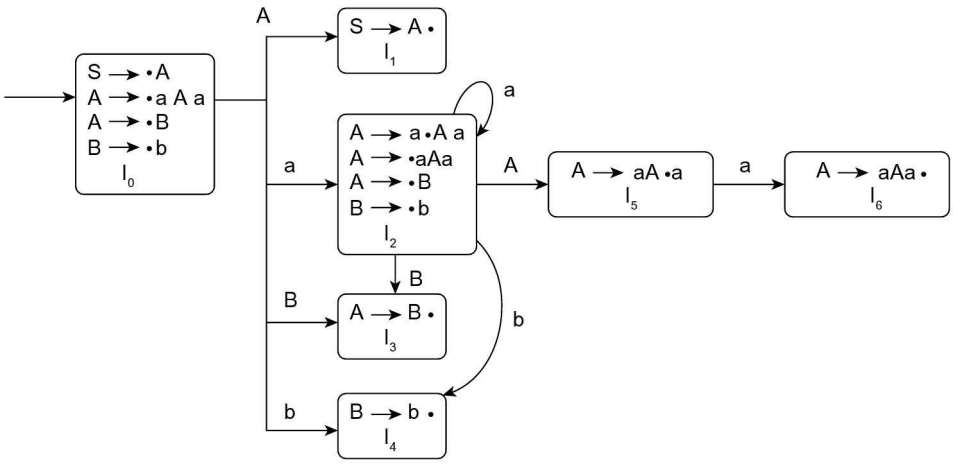
I_2: A → a• A a 5
A → •a A a 1
A → •B 3
B → •b 4

I_3: A → B•

I_4: B → b•

I_5: A → aA• a 6

I_6: A → aAa•



The given grammar is LR(0) grammar as no item set that terminates with dot has any other production.

Example 9.13 State whether the following grammar is LR(0).

(0) S → E
(1) E → E * B
(2) E → E + B
(3) E → B
(4) B → 0
(5) B → 1

It is for this augmented grammar that we will determine the item sets and the transitions between them.

I_0: S → •E 1
E → •E * B 1
E → •E + B 1
E → •B 2

$B \rightarrow \bullet 0$ 3
$B \rightarrow \bullet 1$ 4

I_1: $S \rightarrow E\bullet$
$E \rightarrow E\bullet * B$ 5
$E \rightarrow E\bullet + B$ 6

I_2: $E \rightarrow B\bullet$

I_3: $B \rightarrow 0\bullet$

I_4: $B \rightarrow 1\bullet$

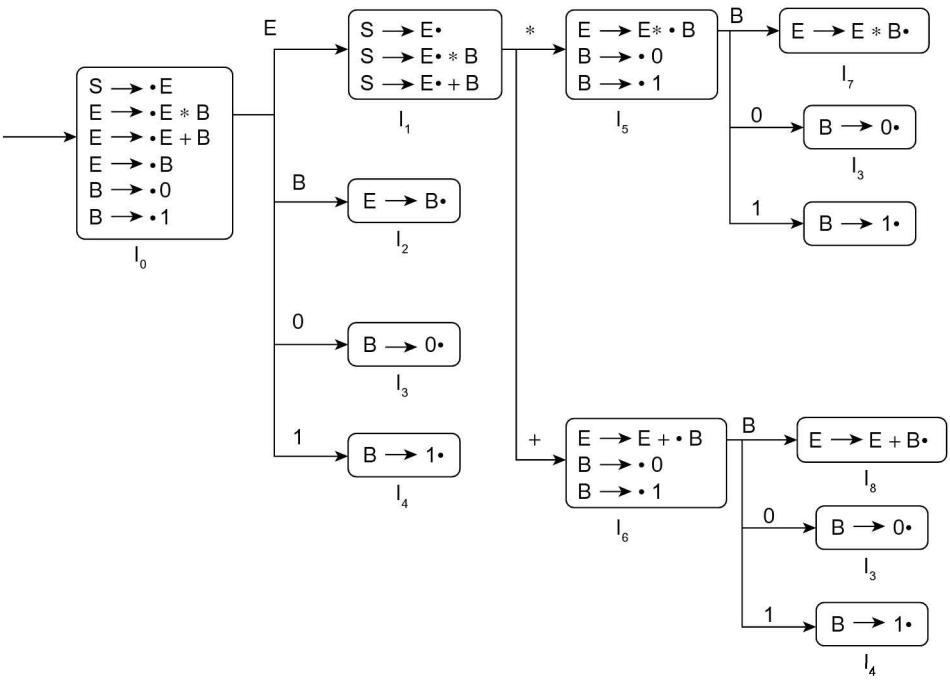
I_5: $E \rightarrow E * \bullet B$ 7
$B \rightarrow \bullet 0$ 3
$B \rightarrow \bullet 1$ 4

I_6: $E \rightarrow E + \bullet B$ 8
$B \rightarrow \bullet 0$ 3
$B \rightarrow \bullet 1$ 4

I_7: $E \rightarrow E * B\bullet$

I_8: $E \rightarrow E + B\bullet$

DFA for the item set is as follows.



This grammar is not LR (0), as in state I_1, there is one production ending with • and in the same set, there are other items with • in other positions. Hence, this cannot be LR (0) grammar.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

9.7 LR(1) Grammar

LR(1) grammar consists items which are LR(0) items with a look ahead element, which is either a terminal or a $ (a special symbol which is the right-end marker of the string).

LR(1) item would be of the form

$$A \rightarrow \alpha \bullet \beta, \{a_1, a_2, \ldots a_m\}$$

This is a valid item for a viable prefix γ if there is a right-most derivation

$S \underset{rm}{\overset{*}{\Rightarrow}} \delta Ay \underset{rm}{\Rightarrow} \delta\alpha\beta y$, where $\delta\alpha = \gamma$, and either

- a is the first symbol of y, or
- $y = \varepsilon$ and a is $.

Closure of LR(1) items is computed by the following steps:

1. Add a new transition $S \rightarrow \bullet A, \{\$\}$ where S is new root symbol and A is the starting symbol of the grammar.
2. If there is a transition $A \rightarrow \alpha\bullet B\beta, \{a_1, a_2, \ldots a_m\}$ in State I, then there would a transition $A \rightarrow \alpha B\bullet \beta, \{a_1, a_2, \ldots a_m\}$ in the next state on B.
3. If there is a transition $A \rightarrow \alpha\bullet B\beta, \{a_1, a_2, \ldots a_m\}$ in State I and if there exists a production $B \rightarrow \gamma$, then add a transition $B \rightarrow \bullet\gamma, T$ in the same set, where T is set of terminals b such that
   - β derives a terminal string beginning with b or
   - $\beta \overset{*}{\Rightarrow} \varepsilon$, and b is a_i for some $1 \le i \le n$.

The grammar is said to LR(1) if it satisfies the following constraints.

- The start symbol does not appear on the right side of any other production.
- In any set of items I, if there is a production $A \rightarrow \alpha\bullet, \{a_1, a_2, \ldots a_m\}$, then
  - No a_i appears immediately to the right of the dot in any item of I.
  - If $B \rightarrow \beta\bullet, \{b_1, b_2, \ldots b_k\}$ is another complete item in I, then $a_i \neq b_j$ for any $1 \le i \le m$ and $1 \le j \le k$.

Example 9.13 Find whether the following grammar is LR(1).

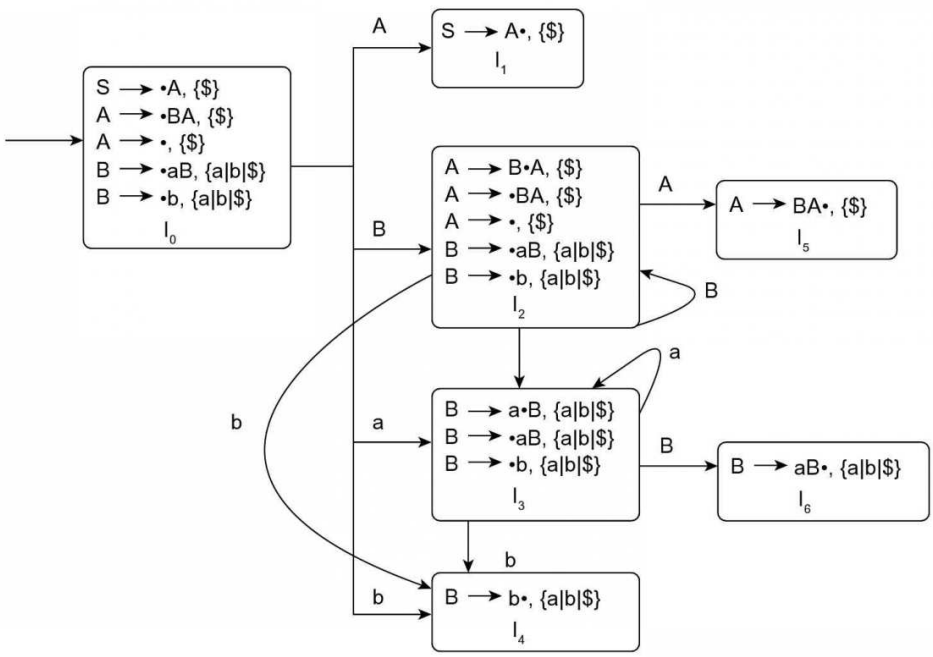
$$A \rightarrow BA \mid \varepsilon, B \rightarrow aB \mid b$$

Solution: First add a new production $S \rightarrow A$.

Computing LR(1) items.

I_0	$S \rightarrow \bullet A, \{\$\}$	1
	$A \rightarrow \bullet BA, \{\$\}$	2
	$A \rightarrow \bullet, \{\$\}$	
	$B \rightarrow \bullet aB, \{a \mid b \mid \$\}$	3
	$B \rightarrow \bullet b, \{a \mid b \mid \$\}$	4
I_1	$S \rightarrow \bullet A, \{\$\}$	
I_2	$A \rightarrow B\bullet A, \{\$\}$	5

	A → •BA, {$}	2
	A→ •, {$}	
	B → •aB, {a | b | $}	3
	B → •b, {a | b | $}	4
I_3	B → a•B,{a | b | $}	6
	B → •aB, {a | b | $}	3
	B → •b, {a | b | $}	4
I_4	B → b•, {a | b | $}	
I_5	A → BA•, {$}	5
I_6	B → aB•, {a | b | $}	6



DFA for LR(1) items is shown below.
This grammar is LR(1) grammar as it satisfies the required rules of LR(1) grammar.

Solved Problems

Problem 1: Find First () and Follow() sets for each non-terminals.

$$A \rightarrow a\ A\ B\ e$$
$$B \rightarrow c \mid \varepsilon$$
$$C \rightarrow d \mid \varepsilon$$

Solution: First(A) = First(aABe)
This is same as first of first symbol, that is, First(A) = {a}

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

First(B) = {c, ε} as First(terminal) is {terminal}
and First(B) is {ε} Since B→ ε
Similarly First(C) = {d, ε}.

Problem 2: Check whether the following grammar is LL(1).

$$S \rightarrow Aa\ A\ b \mid BaBb$$
$$A \rightarrow \varepsilon$$
$$B \rightarrow \varepsilon$$

Solution: This grammar is an ambiguous grammar.
The string ab can be derived in two ways. An ambiguous grammar is not LL(1).
So, this is not LL(1).

Problem 3: Check whether the following grammar is LL(1):

$$S \rightarrow Aa\ A\ b \mid BbBa$$
$$A \rightarrow \varepsilon$$
$$B \rightarrow \varepsilon$$

Solution: The rule for checking is

- A grammar without null rules is LL(1), for each production of the form $A \rightarrow \beta_1 \mid \beta_2 \mid \beta_3$, First($\beta_1$), First($\beta_2$), First($\beta_3$), ... must be mutually disjoint, that is,
  i) First(β_i) ∩ First(β_j) = ∅

Hence, the given grammar is LL(1) if
First(AaAb) ∩ First(BbBa) = ∅
That is, {a} ∩ {b} = ∅. So condition is satisfied.
The 2nd and 3rd production in the given grammar has only one rule on r.h.s. So there is no need to check any condition.
Hence, this is LL(1).

Problem 4: Check whether the following grammar is LL(1):

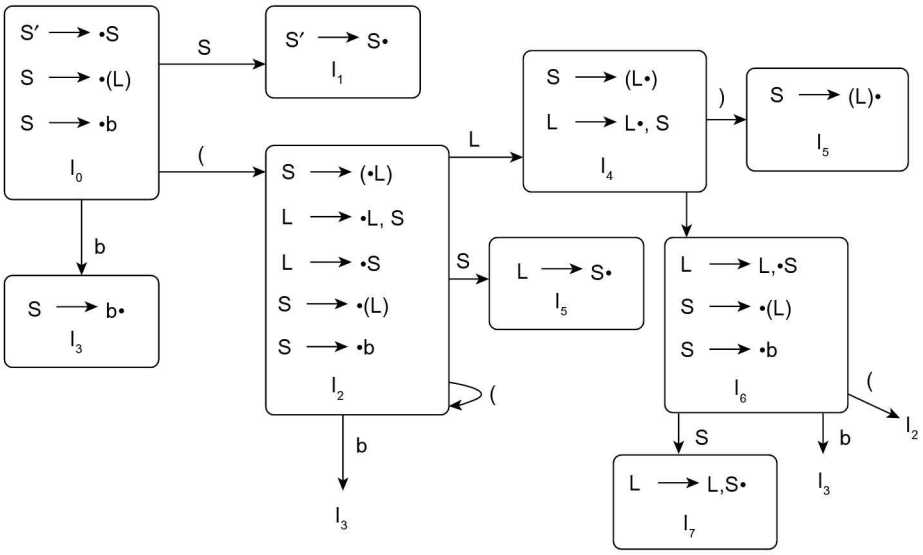
$$S \rightarrow (L) \mid b$$
$$L \rightarrow L, S \mid S$$

Solution: This grammar is a left recursive grammar because of rule L → L, S | S. Any left Grammar is not LL(1).
Hence, it is not LL (1).

Problem 5: Check whether the following grammar is LR(0)

$$\boldsymbol{S \rightarrow (L) / b}$$
$$\boldsymbol{L \rightarrow L, S / S}$$

Solution: Draw the DFA with LR(0) items by using Closure( ) and goto( ) functions.

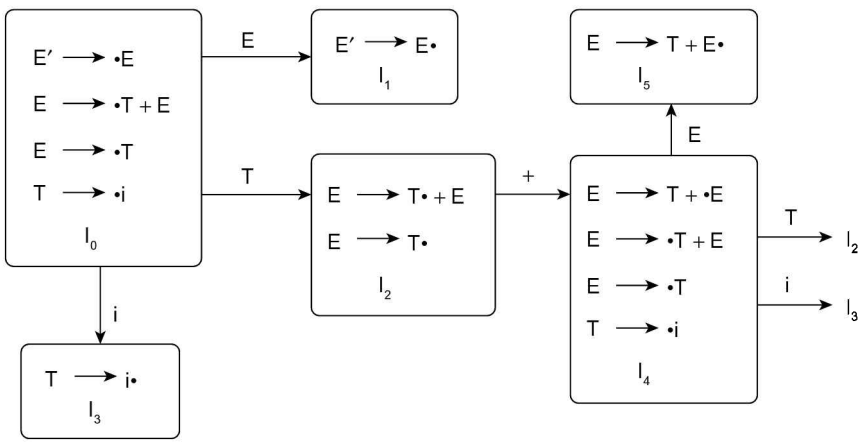


There are no final items with one more non-final item or another final item. Hence, grammar is LR(0).

Problem 6: Check whether the following grammar is LR(0).

$$E \rightarrow T + E \mid T$$
$$T \rightarrow i$$

Solution: Draw the DFA with LR(0) items by using Closure( ) and goto( ) functions.



State I_2 has final item with one more non-final item. Hence, grammar is not LR(0).

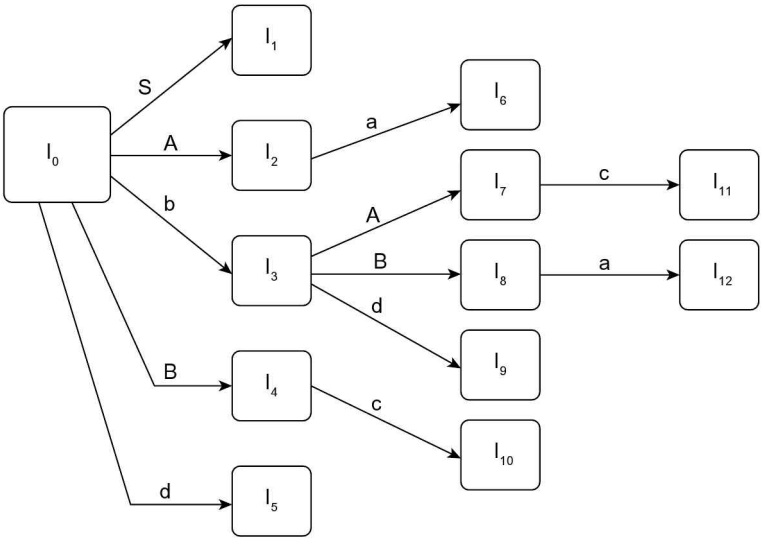
Problem 7: Check whether the following grammar is LR(1):

$$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$$
$$A \rightarrow d$$
$$B \rightarrow d$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Solution: Draw the DFA with LR(1) items by using Closure( ) and goto( ) functions.

$\mathbf{I}_0$	S → •S, $	1
	S → •Aa, $	2
	S → •bAc, $	3
	S → •Bc, $	4
	S → •bBa, $	3
	A → •d, a	5
	B → •d, c	5
$\mathbf{I}_1$	S → S•, $	
$\mathbf{I}_2$	S → A•a, $	6
$\mathbf{I}_3$	S → b•Ac, $	7
	S → b•Ba, $	8
	A → •d, c	9
	B → •d, a	9
$\mathbf{I}_4$	S → B•c, $	10
$\mathbf{I}_5$	A → d•, a	
	B → d•, c	
$\mathbf{I}_6$	S → Aa•, $	
$\mathbf{I}_7$	S → bA•c, $	11
$\mathbf{I}_8$	S → bB•a, $	12
$\mathbf{I}_9$	A → d•, c	
	B → d•, a	
$\mathbf{I}_{10}$	S → Bc•, $	
$\mathbf{I}_{11}$	S → bAc•, $	
$\mathbf{I}_{12}$	S → bBa•, $	



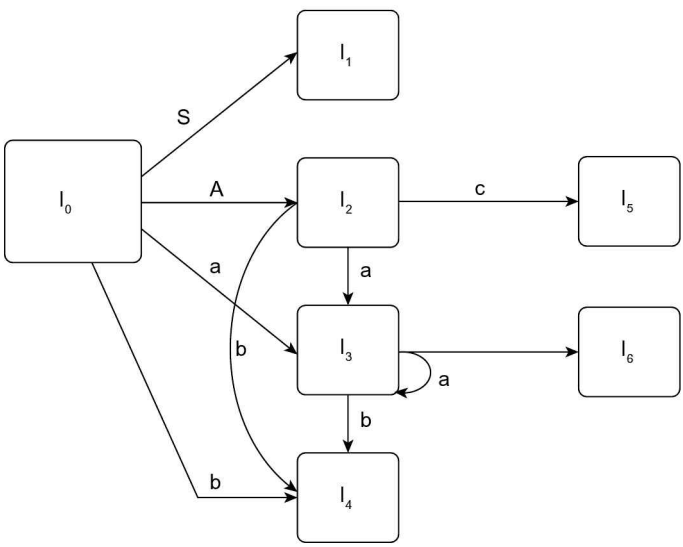
There are no final items with one more non-final item or another final item. Hence, grammar is LR(1).

Problem 8: Check whether the following grammar is LR(0).

$$S \rightarrow AA$$
$$A \rightarrow a\,A \mid b$$

Solution: Draw the DFA with LR(0) items by using Closure( ) and goto( ) functions.

$\mathbf{I}_0$	$S \rightarrow \bullet S$	1
	$S \rightarrow \bullet AA$	2
	$A \rightarrow \bullet aA$	3
	$A \rightarrow \bullet b$	4
$\mathbf{I}_1$	$S \rightarrow S\bullet$	
$\mathbf{I}_2$	$S \rightarrow A\bullet A$	5
	$A \rightarrow \bullet aA$	3
	$A \rightarrow \bullet b$	4
$\mathbf{I}_3$	$A \rightarrow a\bullet A$	6
	$A \rightarrow \bullet aA$	3
	$A \rightarrow \bullet b$	4
$\mathbf{I}_4$	$A \rightarrow b\bullet$	5
$\mathbf{I}_5$	$S \rightarrow AA\bullet$	4
$\mathbf{I}_6$	$A \rightarrow aA\bullet$	



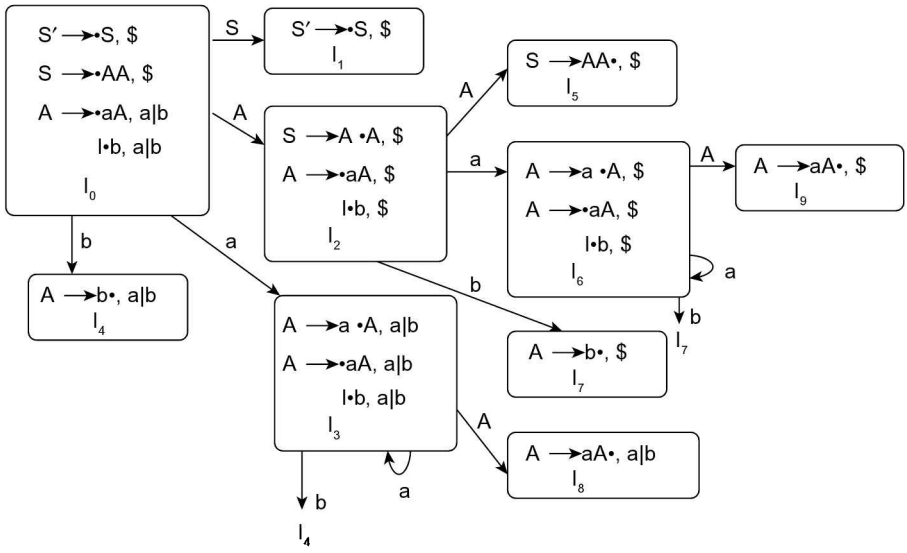
There are no final items with one more non-final item or another final item. Hence, the grammar is LR(0).

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Problem 9: Check whether the following grammar is LR(1):

$$S \rightarrow AA$$
$$A \rightarrow a\,A \mid b$$

Solution: Draw the DFA with LR(1) items by using Closure( ) and goto( ) functions.



There are no final items with one more non-final item or another final item. Hence, the given grammar is LR(1).

Problem 10: Check whether the following grammar is LL(1) or not.

$$S \rightarrow aBCe$$
$$B \rightarrow b \mid \varepsilon$$
$$C \rightarrow c \mid \varepsilon$$

Solution: For each production, apply Rule 2.

$S \rightarrow aBCe$ when there is only one production for non-terminal no need to check any condition.

$B \rightarrow b \mid \varepsilon$ .... First(b) $\cap$ Follow(B) is

$\{b\} \cap \{c, e\} = \varnothing$ the condition is satisfied.

$C \rightarrow c \mid \varepsilon$ .... First(c) $\cap$ Follow(C) is

$\{c\} \cap \{e\} = \varnothing$ the condition is satisfied.

So the given grammar is LL(1).

Problem 11: Check whether the following grammar is LL(1) or not.

$$S \rightarrow aSbS \mid bSaS \mid \varepsilon$$

Solution: For each production, apply Rule 1.

$S \rightarrow aSbS \mid bSaS$ .... First(aSbS) $\cap$ Follow(bSaS) is

$\{a\} \cap \{b\} = \varnothing$

Hence, the condition is satisfied.

$S \rightarrow aSbS \mid \varepsilon$ .... First(aSbS) $\cap$ Follow(S) is
$\{a\} \cap \{a, b, \$\}^1 \neq \varnothing$

Hence, the condition is not satisfied.
So the given grammar is not LL(1).

Problem 12: State whether the following grammar is LR(0).

(0) $S` \rightarrow S$
(1) $S \rightarrow AS$
(2) $S \rightarrow b$
(3) $A \rightarrow SA$
(4) $A \rightarrow a$

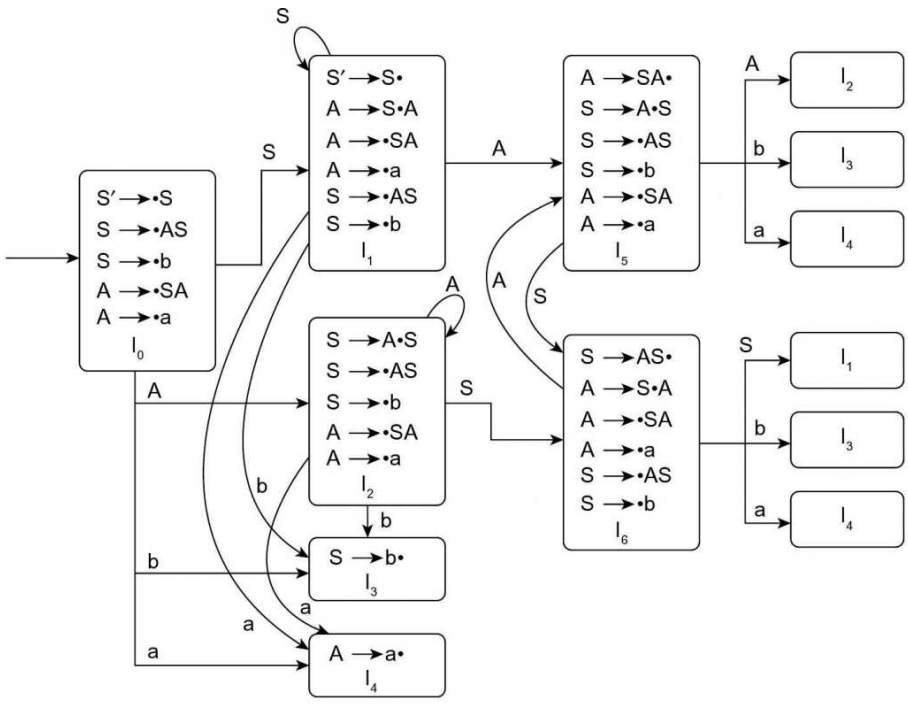
Solution: In the given grammar (0) is augumented rule. The LR items can be computed without adding any new grammar rule. The list of items are listed below.

I_0: $S` \rightarrow \bullet S$	1
$S \rightarrow \bullet AS$	2
$S \rightarrow \bullet b$	3
$A \rightarrow \bullet SA$	1
$A \rightarrow \bullet a$	4
I_1: $S` \rightarrow S\bullet$	
$A \rightarrow S\bullet A$	5
$A \rightarrow \bullet SA$	1
$A \rightarrow \bullet a$	4
$S \rightarrow \bullet AS$	5
$S \rightarrow \bullet b$	3
I_2: $S \rightarrow A\bullet S$	6
$S \rightarrow \bullet AS$	2
$S \rightarrow \bullet b$	3
$A \rightarrow \bullet SA$	6
$A \rightarrow \bullet a$	4
I_3: $S \rightarrow b\bullet$	
I_4: $A \rightarrow a\bullet$	
I_5: $A \rightarrow SA\bullet$	
$S \rightarrow A\bullet S$	6
$S \rightarrow \bullet AS$	2
$S \rightarrow \bullet b$	3
$A \rightarrow \bullet SA$	6
$A \rightarrow \bullet a$	4

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

I_6: S → AS•
A → S•A 6
A → •SA 2
A → •a 3
S → •AS 6
S → •b 4

DFA for the item set is as follows.



The given grammar is not LR(0) as in the States I_1, I_5, I_6, there is a production ending with • and there are other productions which have • in other positions.

Problem 13: State whether the following grammar is LR(0).

(0) S` → S
(1) S → iSeS
(2) S → iS
(3) S → a

Solution: It is for this augmented grammar that we will determine the item sets and the transitions between them.

I_0: S` → •S 1
S → •iSeS 2

$S \rightarrow \bullet iS$ 2
$S \rightarrow \bullet a$ 3

I_1: $S` \rightarrow S\bullet$

I_2: $S \rightarrow i\bullet SeS$ 4
$S \rightarrow i\bullet S$ 4
$S \rightarrow \bullet iSeS$ 2
$S \rightarrow \bullet iS$ 2
$A \rightarrow \bullet a$ 3

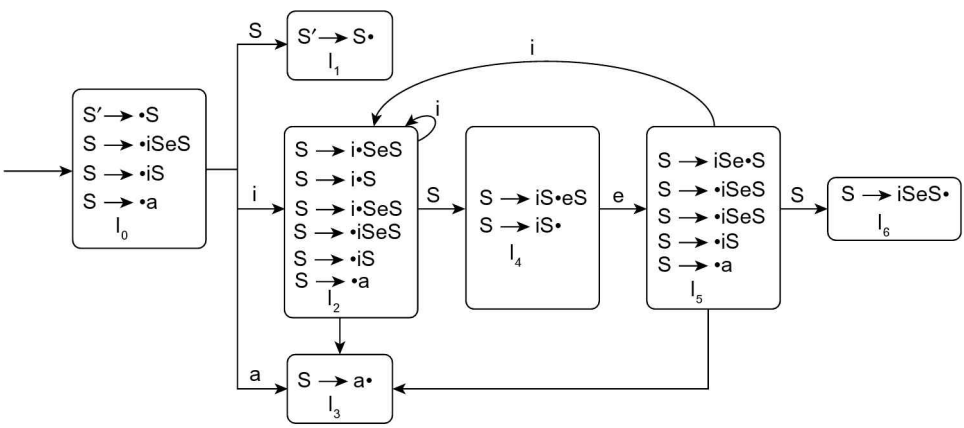
I_3: $S \rightarrow a\bullet$

I_4: $S \rightarrow iS\bullet eS$ 5
$S \rightarrow iS\bullet$

I_5: $S \rightarrow iSe\bullet S$ 4
$S \rightarrow \bullet iSeS$ 2
$S \rightarrow \bullet iS$ 2
$S \rightarrow \bullet a$ 3

I_6: $S \rightarrow iSeS\bullet$

DFA for the item set is as follows.



The given grammar is not an LR(0) grammar as in State I_4, there is a production ending with • and there are other productions with • in other position.

Problem 14: State whether the following grammar is LR(1).

(0) $S` \rightarrow S$
(1) $S \rightarrow iSeS$
(2) $S \rightarrow iS$
(3) $S \rightarrow a$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Solution: It is for this augmented grammar that we will determine the item sets and the transitions between them.

I_0: S` → •S, $ 1
S → •iSeS, $ 2
S → •iS, $ 2
S → •a, $ 3

I_1: S` → S•, $

I_2: S → i•SeS, $ 4
S → i•S, $ 4
S → •iSeS, e/$ 5
S → •iS, e/$ 5
S → •a, e/$ 6

I_3: S → a•, $

I_4: S → iS•eS, $ 7
S → iS•, $

I_5: S → i•SeS, e/$ 8
S → i•S, e/$ 8
S → •iSeS, e/$ 5
S → •iS, e/$ 5
S → •a, e/$ 6

I_6: S → a•, e/$

I_7: S → iSe•S, $ 9
S → •iSeS, $ 2
S → •iS, $ 2
S → •a, $ 3

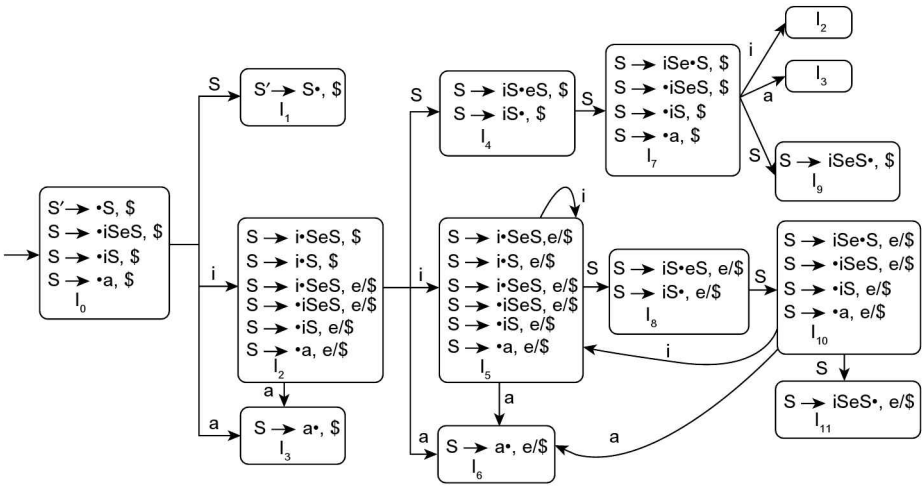
I_8: S → iS•eS, e/$ 10
S → iS•, e/$

I_9: S → iSeS•, $

I_{10}: S → iSe•S, e/$ 11
S → •iSeS, e/$ 5
S → •iS, e/$ 5
S → •a, e/$ 6

I_{11}: S → iSeS•, e/$

The DFA for the LR(1) items is shown below.



The given grammar is not LR(1) as in State I_8, there is a production ending with •, and the symbol e listed for the production as look ahead is to the right of • in other production.

Problem 15: State whether the following grammar is LR(1).

(0) S` → S
(1) S → AA
(2) S → aA
(3) S → b

Solution: It is for this augmented grammar we will determine the item sets and the transitions between them.

I_0: S` → •S, $ 1
S → •AA, $ 2
A → •aA, a/$ 3
A → •b, a/$ 4

I_1: S` → S•, $

I_2: S → A•A, $ 5
A → •aA, $ 6
A → •b, $ 7

I_3: A → a•A, a/b 8
A → •aA, a/b 3
A → •b, a/b 4

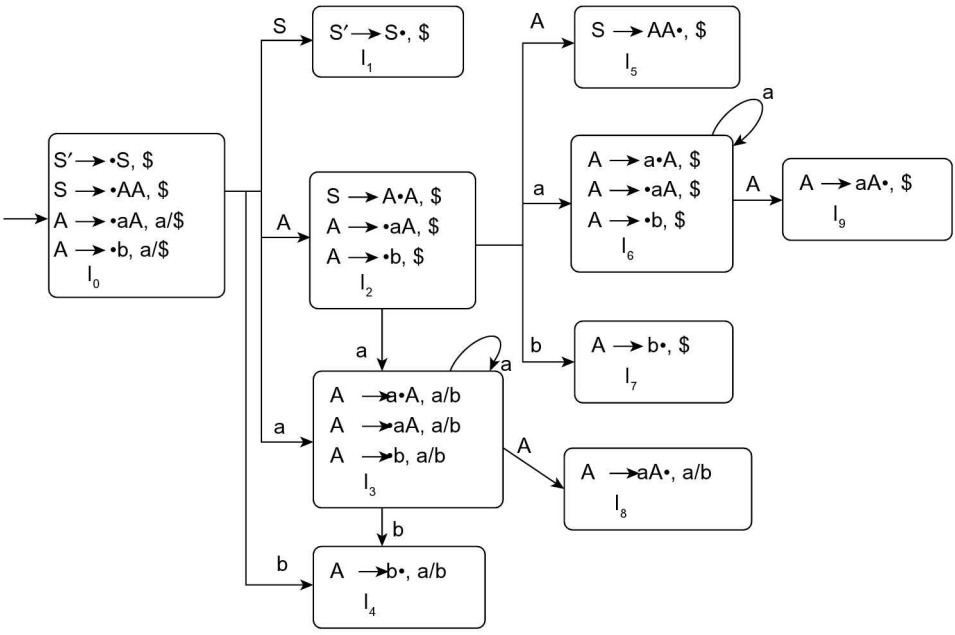
I_4: A → b•, a/b

I_5: S → AA•, $

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

I_6: A → a•A, $ 9
A → •aA, $ 6
A → •b, $ 7
I_7: A → b•, $
I_8: A → aA•, a/b
I_9: A → aA•, $

The DFA for the LR(1) items is shown below.



The grammar is LR(1) grammar as it satisfies the rules.

Summary

1. LR(K) stands for left-to-right scan of input string using the right-most derivation with a look ahead of K symbols.
2. Every LR(K) grammar is an unambiguous grammar.
3. For every LR(K) grammar, there exists a deterministic push down automaton.
4. The CFL generated by LR(0) grammar has strings which satisfy the prefix property.
5. For any LR grammar, the first root non-terminals should not appear on the right side of any other production.
6. LR(0) grammars form a subset of LR(1) grammars.

Short Answers

1. List the rules to find first

   Answer: i. If x is a terminal, then first(x) is x itself.
   Ex: first (+) = {+}, First(id)={id}.
   ii. If x is a nonterminal
   (a) & is defined with ε-rules i.e. $x \rightarrow \varepsilon$, then First (x) ={ε}.
   (b) & is defined with non ε-rules i.e. $x \rightarrow A_1A_2A_3$ then
   First (x) = First $(A_1A_2A_3)$ = First (A_1) if $A_1 =*> \varepsilon$
   = First $(A_1) - \{\varepsilon\} \cup$ First $(A_2 A_3))$ if $A_1 =*> \varepsilon$

2. Compute the first (S) the given grammar $S \rightarrow Ab, A \rightarrow a \mid \varepsilon$

   Answer: First(S) = {a, b}

3. Compute the first (S) the given grammar

   $S \rightarrow AB|CD|E, A \rightarrow a|\varepsilon, B \rightarrow b, C \rightarrow c|\varepsilon, D \rightarrow d, E \rightarrow e$

   Answer: First(S) = {a, b, c, d, e}

4. Define Follow

   Answer: Follow of any non terminal is the element that appears first immediately after the last element derived by that non terminal.

5. Find follow of C in the given grammar

   $S \rightarrow aB|b, B \rightarrow bC|b, C \rightarrow cS|d$

   Answer: Follow of (C) = {$}

6. How do you say the given grammar is LR(0)?

   Answer: A grammar is said to be LR(0) grammar if, Its start with a symbol does not appear on right hand side of any other production and if the closure set of item has a production of the form $A \rightarrow \alpha \bullet$ then there is no production of the form $B \rightarrow \beta \bullet$ or $B \rightarrow \beta \bullet \gamma$

Fill in the Blanks

1. In LR(K) grammars, k stands for _____.
2. LR(0) grammars are more powerful than LR(1) grammars. (True/False)
3. The deterministic pushdown automaton uses _____ grammars.
4. The class of languages that is generated by LR(0) grammar is also called _____.
5. Regular grammars form a superset of LR(0) grammars. (True|False)
6. _____ property states that if $w \in L$, then proper prefix of w Ï L.
7. _____ grammars are unambiguous.
8. The string obtained by the right-most derivation that is in sentential form is called _____.
9. The string obtained by either right-most derivation or left most derivation that has only terminals is called _____.
10. All CFGs are either LR(0) or LR(1) grammars.(True|False)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Answers

1. Look ahead	2. False	3. LR	4. deterministic CFGs	5. False
6. Prefix	7. LR(K)	8. Handle	9. Yield	10. False

Objective Question Bank

1. An LL(1) grammar is used for as below parser
   a) Top-down b) Bottom-up c) Both d) None
2. An LR(0) grammar defines
   a) CSL b) recursive
   c) recursively enumerable d) DCFL
3. The item A→•b,{a/b/$} is a/an
   a) LR(3) b) LR(1) c) LR(0) d) Grammar unite
4. The item B→•b is a/an
   a) LR(3) b) LR(1) c) LR(0) d) Grammar unite
5. An LR(1) grammar is suitable for
   a) Top-down b) Bottom-up parser
   c) Both d) None
6. S → SS/(S)/a is not suitable for LL(1) because the grammar is
   a) ambiguous b) left recursive
   c) right recursive d) None
7. S → S + A/A, A→a is not suitable for LL(1) because the grammar is
   a) ambiguous b) left recursive
   c) right recursive d) None
8. The statement "Every unambiguous grammar is LL(1)"
   a) true b) false c) cannot say d) None
9. The statement "Every LR(0) is LL(1)" is
   a) true b) false c) cannot say d) None
10. The statement "Every LR(0) is LR(1)" is
    a) true b) false c) cannot say d) None
11. The following grammar is S→A | a, A → a
    a) LL(1) b) LR(0) c) LR(1) d) None
12. The LR(0) grammar define
    a) CSL's b) CFL's c) DCFL's d) None
13. Consider the following grammar
    A → BCD |a, B→ a | ε, C→ b | ε, D→ c | ε
    First(A) is
    a) {a, b, c, ε} b) {a, b, c} c) {a, b, c} d) None

14. Consider the following grammar
    $A \rightarrow BCD \mid a, B \rightarrow a \mid \varepsilon, C \rightarrow b \mid \varepsilon, D \rightarrow c \mid \varepsilon$
    Follow (A) is
    a) {b, c, d, ε} b) {b, c} c) {b, c, d, $} d) None
15. Which of the following is true?
    a) LL(1) ⊂ LR(1) b) LR(0) ⊂ LR(1)
    c) LR(1) ⊂ LL(1) d) None

Answers

1. A	2. D	3. B	4. C	5. B	6. A	7. B	8. B
9. C	10. A	11. D	12. C	13. A	14. C	15. B	

Exercises

1. Check whether the following grammar is LL(1) or not. $S \rightarrow aS \mid Sa \mid a$
2. Check whether the following grammar is LL(1) or not.

$$S \rightarrow AaAb \mid BbBa$$
$$A \rightarrow \varepsilon$$
$$B \rightarrow \varepsilon$$

3. List LR(0) items for the following grammar:

$$S \rightarrow (S) \mid \{S\} \mid SS \mid a$$

4. Find whether the above grammar is LR(0) or not.
5. Show that the following grammar is not LR(0).

$$S \rightarrow S \text{ sub } S \text{ sup } S \mid S \text{ sub } S \mid S \text{ sup } S \mid a \mid b$$

6. Is the following grammar LR(0)?

$$S \rightarrow bA \mid aB, A \rightarrow aBB \mid aS \mid a, B \rightarrow bAA \mid bS \mid b$$

7. List LR(1) items for the following grammar:

$$E \rightarrow E + E \mid E*E \mid E - E \mid E/E \mid (E) \mid a$$

8. State whether the above grammar is LR(1) or not.
9. Find whether the following grammar is LR(1).

$$S \rightarrow S \text{ sub } S \text{ sup } S \mid S \text{ sub } S \mid S \text{ sup } S \mid a \mid b$$

10. Check whether the following grammar is LL(1) or not.

$$S \rightarrow Aa \mid bAc \mid dc \mid bda$$
$$A \rightarrow d$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

This page is intentionally left blank

Appendix A

PROPOSITION AND PREDICATE LOGIC

In this chapter, we introduce proposition and logical connectives. Normal forms for well-formed formulas are given. Introduction to predicates and rules of inference for propositional calculus and predicate calculus is given.

A.1 Propositions

A proposition is a declarative sentence that is either true or false. If the sentence is true then its truth value (T_v) is T, otherwise it is F.

Example:

S_1 – Square of 6 is 36. $T_v(S_1) = T \Rightarrow$ proposition

S_2 – Delhi is the capital of America. $T_v(S_2) = F \Rightarrow$ proposition

S_3 – On Friday it will rain.

(This statement has two possible truth values depending on whether it rains or not on Friday).

$T_v(S_3) = F$ if it does not rain.

$T_v(S_3) = T$ if it rains.

$\Rightarrow$ proposition

S_4 – Get me glass of water.

S_5 – Shut the Door!

The sentences S_4 and S_5 are not declarative sentences. Hence, they are not propositions.

A.2 Connectives

Connectives are words used to get a new proposition from the given set of propositions. New sentence formed by connecting two propositions will be a proposition only if it has a truth value T or F, but not both.

Different connectives are

- Negation (NOT)
- Conjunction (AND)
- Disjunction (OR)
- Implication (IF … THEN …)
- If and only if

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

i) Negation (NOT)

If P is a proposition, then its negation NOT P (denoted as $\neg P$ (read as 'NOT P')) is also a proposition whose truth value is T if P is F, otherwise F.

P	$\neg P$
T	F
F	T

ii) Conjunction (AND)

If P and Q are two propositions, then the conjunction of P and Q (denoted as $P \wedge Q$ (read as 'P and Q')) is a proposition whose truth values are given in the following truth table:

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

iii) Disjunction (OR)

If P and Q are two propositions, then the disjunction of P or Q (denoted as $P \vee Q$ (read as 'P OR Q')) is a proposition whose truth values are given in the following table:

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

iv) Implication or Conditional (IF ... THEN ...)

If P and Q are two propositions, then the proposition 'if P then Q' (denoted as $P \rightarrow Q$ (read as 'IF P THEN Q')) is a proposition whose truth values are given in the following table:

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Note: The truth value of $P \rightarrow Q$ is always T if the value of P is F.

P is called antecedent and q is called consequent.

v) Biconditional

If P and Q are two propositions, then biconditional connective is stated as if P then Q (denoted as $P \leftrightarrow Q$ read as 'P only if Q'). It is a proposition whose truth values are given in the table.

P	Q	$P \leftrightarrow Q$
T	T	T
T	F	F
F	T	F
F	F	T

Note: The truth value of $P \leftrightarrow Q$ is T if the truth values of both P and Q are T or F i.e identical.

Example A.1 Represent the following sentences in symbols P and Q, if P represents 'This garland is big' and Q represents 'This is cheap'.

a) This garland is big and cheap.
b) This garland is small but cheap.
c) This garland is costly but big.
d) This garland is neither big nor cheap.
e) This garland is either big or cheap.

Solution:

a) $P \wedge Q$ b) $(\neg P) \wedge Q$
c) $(\neg Q) \wedge P$ d) $(\neg P) \wedge (\neg Q)$ e) $P \vee Q$

A.3 Well-Formed Formula

When we have two propositions P and Q connected with any connective, if the order of the formula is changed, the truth tables would remain identical. For example, $P \wedge Q$ is same as $Q \wedge P$. This can be expressed by the concept of Well Formed Formulae (w.f.f.).

Definition 1: A propositional variable is a symbol representing any proposition.

Definition 2: A well-formed formula (w.f.f.) is defined recursively as follows.

i) If P is a propositional variable. then it is a w.f.f.
ii) If α is a w.f.f., then $\neg\alpha$ is a w.f.f.
iii) If α and β are well – formed formulas, then $(\alpha \wedge \beta)$, $(\alpha \vee \beta)(\alpha \rightarrow \beta)$ and $(\alpha \leftrightarrow \beta)$ are well – formed formulas.
iv) A string of symbols is a w.f.f. if and only if it is obtained by finitely many applications of (i)–(iii).

Note: A w.f.f. is not a proposition, but if we substitute a proposition in place of propositional variable, we get a proposition. For example,

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

i) $(\neg(P \vee Q) \wedge (\neg Q \wedge R) \rightarrow Q)$ is a w.f.f.

ii) $(\neg P \wedge Q) \Leftrightarrow Q$ is a w.f.f.

Parenthesis can be dropped when there is no ambiguity.

A.3.1 Truth Table for a Well-formed Formula

In formula α, if we replace the propositional variables by propositions, we get a proposition which has connectives. The truth table of α is obtained by replacing the propositional variables by arbitrary propositions. For α with n propositional constants, there would be 2^n possible combinations of truth values of proportions replacing the variables.

Example A.1 Obtain the truth table for $\alpha = (P \vee Q) \wedge (P \rightarrow Q) \wedge (Q \rightarrow P)$.

Solution: The truth table of the given w.f.f. is as follows.

P	Q	$P \vee Q$	$P \rightarrow Q$	$(P \vee Q) \wedge (P \rightarrow Q)$	$(Q \rightarrow P)$	$(P \vee Q) \wedge (P \rightarrow Q) \wedge (Q \rightarrow P)$
T	T	T	T	T	T	T
T	F	T	F	F	T	F
F	T	T	T	T	F	F
F	F	F	T	F	T	F

Definition 3: A tautology or a universally true formula is a well-formed formula whose truth values is T for all possible assignments of truth values to the propositional variables.

For example, $P \vee \neg P$ has the truth value T irrespective of the truth value of P. Such formula is called a tautology.

Example A.1 Find whether the following are tautology or not.

i) $(P \wedge Q) \rightarrow P$.

ii) $((P \rightarrow Q) \wedge (Q \rightarrow R)) \rightarrow (P \rightarrow R)$

Solution: $(P \wedge Q) \rightarrow P$. The truth table for the present formula (i) is the following:

P	Q	$P \wedge Q$	$(P \wedge Q) \rightarrow P$
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	T

In the truth table of the formula (i) we arrived at, we see that the formula is always true (i.e., the last column is has only T's). Hence, the given relation is a tautology.

ii) $\alpha = ((P \rightarrow Q) \wedge (Q \rightarrow R)) \rightarrow (P \rightarrow R)$

The truth table for the present formula is given below.

P	Q	R	$P \to Q$	$Q \to R$	$(P \to Q) \wedge (Q \to R)$	$P \to R$	α
T	T	T	T	T	T	T	T
T	T	F	T	F	F	F	T
T	F	T	F	T	F	T	T
T	F	F	F	T	F	F	T
F	T	T	T	T	T	T	T
F	T	F	T	F	F	T	T
F	F	T	T	T	T	T	T
F	F	F	T	T	T	T	T

The truth value of the given formula (i.e., entries in the last column) is always T. Hence, the given formula (ii) is a tautology.

A.4 Logical Identities

Some formulas can be expressed in simple form without changing the truth values of the formula using the logical identity rules.

1. Idempotent laws:
   $P \vee P \equiv P$ $\quad P \wedge P \equiv P$
2. Commutative laws:
   $P \vee Q \equiv Q \vee P$ $\quad P \wedge Q \equiv Q \wedge P$
3. Associative laws:
   $P \vee (Q \vee R) \equiv (P \vee Q) \vee R$ $\quad P \wedge (Q \wedge R) \equiv (P \wedge Q) \wedge R$
4. Distributive laws:
   $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$ $\quad P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$
5. Absorption laws:
   $P \vee (P \wedge Q) \equiv P$ $\quad P \wedge (P \vee Q) \equiv P$
6. De Morgan's laws:
   $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ $\quad \neg(P \wedge Q) \equiv \neg P \vee \neg Q$
7. Double negation:
   $P \equiv \neg(\neg P)$ $\quad P \vee (\neg P) \equiv T$
   $P \wedge (\neg P) \equiv F$ $\quad P \vee T \equiv T$
   $P \wedge T \equiv P$ $\quad P \vee F \equiv P$
   $P \wedge F \equiv F$ $\quad (P \to Q) \wedge (P \to \neg Q) \equiv \neg P$
8. Contrapositive Laws:
   $P \to Q \equiv \neg Q \to \neg P$ $\quad P \to Q \equiv (\neg P \vee Q)$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Example A.4 Simplify the formula $(P \wedge Q) \vee (P \wedge \neg Q)$.

Solution: $(P \wedge Q) \vee (P \wedge \neg Q) \equiv P \wedge (Q \vee \neg Q)$ {using $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$}
$\equiv P \wedge T$ {using $P \vee (\neg P) \equiv T$}
$\equiv P$ {using $P \wedge T \equiv P$}

A.5 Normal Forms of Well-formed Formals

So far we discussed well – formed formulas in terms of two propositional variables P and Q. The number of distinct truth tables for formulas in P and Q is 2^4. The possible combinations of truth values for two variables are 4 {TT, TF, FT, FF}. The truth table of any formula in P and Q has 4 rows. Thus there are only 16 distinct formulas and any formula in P and Q is equivalent to one of these 16 formulas. 'Normal form' is a method of reducing a given formula to an equivalent form. For any propositional variable P, we use 'sum' for disjunction, 'product' for conjunction, and 'literal' for either P or $\neg P$.

Definition: An elementary product is a product of literals. An elementary sum is a sum of literals.

For example, $P \wedge \neg Q$, $\neg P \wedge \neg Q$, $P \wedge Q$, $\neg P \wedge Q$ are elementary products. $P \vee \neg Q$, $P \vee \neg R$ are elementary sums.

Definition: A formula is in disjunctive normal form if it is a sum of elementary products.

For example, $P \vee (Q \wedge R)$ and $P \vee (\neg Q \wedge R)$ are in disjunctive normal form. $P \wedge (Q \vee R)$ is not in disjunctive normal form.

A.5.1 Construction to Obtain a Disjunctive Normal Form of a Given Formula

Step 1: Eliminate $\rightarrow$ and $\Leftrightarrow$ using logical identities

Step 2: Use De Morgan's laws to eliminate $\neg$ before sums or products. The resulting formula has $\neg$ only before propositional variables. It involves sum, products and literals.

Step 3: Apply distributive laws repeatedly to eliminate product of sums. The resulting formula will be a sum of products of literals, that is, a sum of elementary products.

Example A.1 Obtain a disjunctive normal form of $P \vee (\neg P \rightarrow (Q \vee (Q \rightarrow \neg R)))$.

Solution:

$P \vee (\neg P \rightarrow (Q \vee (Q \rightarrow \neg R)))$
$\equiv P \vee (\neg P \rightarrow (Q \vee (\neg Q \vee \neg R)))$ {using $P \rightarrow Q \equiv (\neg P \vee Q)$}
$\equiv P \vee (\neg(\neg P) \vee (Q \vee (\neg Q \vee \neg R)))$ {using $P \rightarrow Q \equiv (\neg P \vee Q)$}
$\equiv P \vee (P \vee (Q \vee (\neg Q \vee \neg R)))$ {using $P \equiv \neg(\neg P)$}
$\equiv P \vee P \vee Q \vee \neg Q \vee \neg R$ {using $P \vee (Q \vee R) \equiv (P \vee Q) \vee R$}
$\equiv P \vee Q \vee \neg Q \vee \neg R$ {using $P \vee P \equiv P$}

The $P \vee Q \vee \neg Q \vee \neg R$ is disjunctive normal form of the given formula.

A.6 Principal Disjunctive Normal Form

For a given formula, we can get a normal forms named as principal disjunctive normal form or sum of products canonical form. The advantages of constructing principal disjunctive normal form are the following:

i) For a given formula, its principal disjunctive normal form is unique.
ii) Two formulas are equivalent if and only if their principal disjunctive normal forms coincide.

Definition: A min term in n propositional variables $P_1, P_2 \ldots P_n$ is $Q_1 \wedge Q_2 \ldots Q_n$, where each Q_i is either P_i or $\neg P_i$.

Example: The min terms in P and Q are $P \wedge Q$, $\neg P \wedge Q$, $P \wedge \neg Q$, $\neg P \wedge \neg Q$. The number of min terms in n variables is 2^n.

Definition: A formula α is in principal disjunctive normal form if α is a sum of min terms.

A.6.1 Construction to Obtain the Principal Disjunctive Normal Form of a Given Formula

Step 1: Obtain a disjunctive normal form.

Step 2: Drop elementary products which are contradictions (such as $P \wedge \neg P$).

Step 3: If Pi and $\neg P_i$ are missing in an elementary product α, replace α by $(\alpha \wedge P_i) \vee (\alpha \wedge \neg P_i)$.

Step 4: Repeat Step 3 until all elementary products are reduced to sum of min terms. Use idempotent laws to avoid repetition of min terms.

Example A.4 Obtain the canonical sum of products form of $\alpha = P \vee (\neg P \wedge \neg Q \wedge R)$.

Solution: The given formula is already in disjunctive normal form. Only Step 3 has to be applied.

Definition: A formula is in conjunctive normal form if it is a product of elementary sums.

Definition: A max term in n propositional variables $P_1, P_2, \ldots P_3$ is $Q_1 \vee Q_2 \vee \ldots Q_n$, where each Q_i is either P_i or $\neg P_i$.

Definition: A formula α is in principal conjunctive normal form if α is a product of max terms. For obtaining the principal conjunctive normal form of α, we can construct the principal disjunctive normal form of $\neg\alpha$ and apply negation ($\neg$).

Example:
Rules of inference for propositional calculus (Statement calculus)

In logical reasoning, some propositions are assumed to be true, and these propositions are used to derive other propositions using rules of inferences. The propositions that are assumed to be true are called hypotheses or premises, and the propositions that are derived

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

using rules of inferences are called conclusions. The process of deriving conclusions based on assumptions is called valid argument.

The rules of inferences are tautologies in the form of implications and are listed in the table.

Rules of inference	Implication form
RI_1: Addition $\frac{P}{\therefore P \vee Q}$	$P \Rightarrow (P \vee Q)$
RI_2: Conjunction $\frac{Q}{\therefore P \wedge Q}$	$Q \Rightarrow (P \wedge Q)$
RI_3: Simplification $\frac{P \wedge Q}{\therefore P}$	$P \wedge Q \Rightarrow P$
RI_4: Modus ponens P $\frac{P \rightarrow Q}{\therefore \quad Q}$	$(P \wedge (P \rightarrow Q)) \Rightarrow Q$
RI_5: Modus tollens $\wedge Q$ $\frac{P \rightarrow Q}{\therefore \neg P}$	$(\neg Q \wedge (P \rightarrow Q)) \Rightarrow \neg P$
RI_6: Disjunctive syllogism $\neg P$ $\frac{P \vee Q}{\therefore Q}$	$(\neg P \wedge (P \vee Q)) \Rightarrow Q$
RI_7: Hypothetical syllogism $P \rightarrow Q$ $\frac{Q \rightarrow R}{\therefore P \rightarrow R}$	$((P \rightarrow Q) \wedge (Q \rightarrow R) \Rightarrow (P \rightarrow R)$
RI_8: Constructive dilemma $(P \rightarrow Q) \wedge (R \rightarrow S)$ $\frac{P \vee R}{\therefore Q \vee S}$	$(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R) \Rightarrow (Q \vee S)$
RI_9: Destructive dilemma $(P \rightarrow Q) \wedge (R \rightarrow S)$ $\frac{\neg Q \vee \neg S}{\therefore \neg P \vee \neg R}$	$(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (\neg Q \vee \neg S) \Rightarrow (\neg P \vee \neg R)$

Example A.1 Can we conclude S from the following hypotheses?

i) $P \rightarrow Q$.
ii) $P \rightarrow R$.
iii) $\neg(Q \wedge R)$.
iv) $S \vee P$.

Solution: The rules if inferences can be used to deduce S from the given four hypotheses.

1.	$P \rightarrow Q$.	hypothesis (i)
2.	$P \rightarrow R$.	hypothesis (ii)
3.	$(P \rightarrow Q) \wedge (P \rightarrow R)$	using lines 1 and 2 and inference rule RI_2.
4.	$\neg(Q \wedge R)$	hypothesis (iii)
5.	$\neg Q \vee \neg R$	Using line 4 and De Morgan's laws.
6.	$\neg P \vee \neg P$	Using 3, 5 and Destructive dilemma RI_9.
7.	$\neg P$	Using Idempotent law.
8.	$S \vee P$	Using hypothesis (iv)
9.	S	Using 7 and 8 and Disjunctive syllogism RI_6

Thus we can conclude S from the given hypotheses.

A.7 Predicate Calculus

The common feature expressed by a statement is called predicate. Consider statements 'Mango is a fruit' and 'Apple is a fruit'. Although these statements are not related, they share common feature. These statements can be represented by a single statement as 'x is a fruit' where x can have many values and each substitution creates a statement. Predicate calculus deals with sentences involving predicates. These statements occur in mathematics and programming languages. Following is a list of predicates in mathematics and C language.

$2x + 3y = 4z$	in mathematics
If (x >= 0) go to 30	in C language

Definition: A part of a declarative sentence describing the properties of objects or relation among objects is called a predicate.

Sentence that involve predicates that describe the property of objects are denoted by P(x), where P is predicate and x is variable denoting the object.

Example:

1. If P(x) is 'x is a fruit'
   Then, P denotes 'is a fruit' and x is a variable.
2. If P(x, y) is 'x is father of y'
   Then, P denotes 'is father of' and x, y are variables.

Note: Although predicate looks like a proposition, it is not a proposition, as it involves a variable x. It cannot be assigned a truth value. But if x is replaced by an object, we get a proposition which has truth value.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Example: If x is replaced by 'apple' then truth value is T, but if x is replaced by name of a person 'Rama', then the truth value is F.

The following definitions can be used to assign values to variables.

Definition: For a declarative sentence involving a predicate, the universe of discourse, or simply the universe, is the set of all possible values that can be assigned to variables.

Example: The universe of discourse for P(x) where 'x is a fruit' can be taken as the set of all fruit names. The universe of discourse for E(n) where 'n is as even integer' can be taken as the set of all integers.

A.8 Universal and Existential Quantifier

The phrase 'for all' (denoted $\forall$) is called the universal quantifier. And the phrase 'there exists' (denoted $\exists$) is called the existential quantifier.

Example:

'*For all* x, $x^2 = (-x)^2$' can be written as $\forall x\ R(x)$ *is* '$x^2 = (-x)^2$'.

'*There exists x such that* $x^2 = 5$' can be written as $\exists xR(x)$, *where* R(x) *is* '$x^2 = 5$'.

When we use a quantifier, we should specify the universe of discourse. If we change the universe of discourse, the truth value may change. For example, consider $\exists xR(x)$, where R(x) is $x^2 = 5$. If the universe of discourse is the set of all integers, then $\exists xR(x)$ is false. If the universe of discourse is the set of all real numbers, then $\exists xR(x)$ is true (when $x = \pm (5)^{1/2}$, $x^2 = 5$).

The logical connectives involving predicates can be used for declarative sentences involving predicates.

Example A.4 Express the following sentences involving predicates in symbols.

1. All students are clever.
2. Some students are not successful.
3. Every clever student is successful.
4. There are some successful students who are not clever.
5. Some students are clever and successful.

Solution: To give the solution, let us define the universe of discourse as the set of all students.

Let C(x) denote 'x is clever'.

Let S(x) denote 'x is successful'.

Then symbolic representations of the sentences given above are the following:

1. $\forall xC(x)$.
2. $\exists x(\neg S(x))$
3. $\forall x(C(x) \rightarrow S(x))$
4. $\exists x(S(x) \wedge \neg C(x))$
5. $\exists x(C(x) \wedge S(x))$

A.9 Well-formed Formulas of Predicate Calculus

A well-formed formula (w.f.f.) of predicate calculus is a string of variables (such as x_1, x_2, x_3, …… x_n), connectives, parentheses and quantifiers defined recursively by the following rules:

i) $P(x_1, x_2, \ldots x_n)$ is w.f.f., where P is a predicate involving n variables $x_1, x_2, \ldots x_n$.
ii) If α is a w.f.f., then $\neg\alpha$ is a w.f.f.
iii) If α and β are w.f.f.'s, then $\alpha \vee \beta$, $\alpha \wedge \beta$, $\alpha \rightarrow \beta$, $\alpha \Leftrightarrow \beta$ are also w.f.f.'s.
iv) If α is a w.f.f. and x is a variable, then $\forall x(\alpha)$, $\exists x(\alpha)$ are w.f.f.'s.
v) A string is a w.f.f. if and only if it is obtained by finitely many applications of rules (i)–(iv).

Note: A proposition can be viewed as a sentence involving a predicate with 0 variables. So propositions are w.f.f.'s of predicate calculus by rule (i). w.f.f.'s of predicate calculus are also called predicate formulas.

Definition: Let α and β be two predicate formulas in variables $x_1, x_2, \ldots x_n$ and let U be a universe of discourse for α and β. Then α and β are equivalent to each other over U if, for every possible assignment of values to each variable in α and β, the resulting statements have the same truth values. We can write $\alpha \equiv \beta$ over U.

Definition: If a formula of the form $\exists xP(x)$ or $\forall xP(x)$ occurs as part of a predicate formula α, then such part is called an x-bound part of α, and the occurrence of x is called a bound occurrence of x. An occurrence of x is free if it is not a bound occurrence. A predicate variable in α is free if its occurrence is free in any part of α.

Definition: A predicate formula is valid if for all possible assignments of values from any universe of discourse to free variables, the resulting propositions have truth value T.

Definition: A predicate formula is satisfied if, for all possible assignment of values to predicate variables, the resulting proposition has truth value T.

A predicate formula is unsatisfiable if, for all possible assignments of values from any universe of discourse to predicate variables, the resulting propositions have truth value F.

We note that valid predicate formulas correspond to tautologies among proposition formulas, and unsatisfiable predicate formulas correspond to contradictions.

A.10 Rules of Inference for Predicate Calculus

The rules of inference for proposition formulas are also applicable for predicate calculus:

- The proposition formulas are also called predicate formulas.
- Predicate formulas where all variables are quantified are proposition formulas.

Note: To derive some conclusions from a given set of hypotheses involving quantifiers, we may have to eliminate the quantifiers before applying the rules of inference for proposition formulas.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Example A.9 Discuss the validity of the following argument: All graduates can read and write. Ram can read and write. Therefore, Ram is a graduate.

Solution: Let P(x) denote 'x is a graduate'. Let Q(x) denote 'x can read and write'. Let R denote Ram.

The hypotheses are $\forall x(P(x) \rightarrow Q(x))$ and $Q(R)$.

The conclusion is $P(R)$.

$((P(R) \rightarrow Q(R)) \wedge Q(R)) \rightarrow P(R)$ is not a tautology.

So, we cannot derive P(R). For example, a school boy can read and write and he is not a graduate.

Summary

- Representation and meaning of logical connectives.

Table A.1 *Logical Connectives*

Connective	Resulting proposition	Read as
Negation $\neg$	$\neg P$	NOT P
Conjunction $\wedge$	$P \wedge Q$	P AND Q
Disjunction $\vee$	$P \vee Q$	P OR Q
Implication $\rightarrow$	$P \rightarrow Q$	If P THEN Q
IF AND ONLY IF $\Leftrightarrow$	$P \Leftrightarrow Q$	P IF AND ONLY IF Q

Appendix B

FREQUENTLY ASKED UNIVERSITY QUESTIONS WITH SOLUTIONS

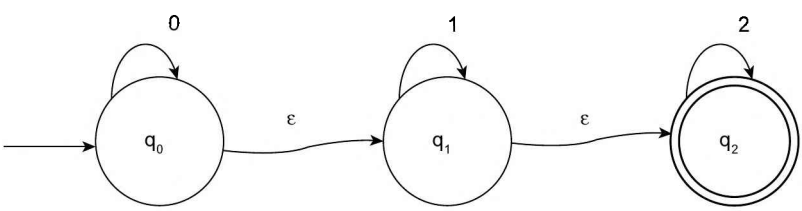
PART A - Brief Questions

1. **What is the proof by contradiction?**

   Ans: In a proof by contradiction we assume, along with the hypotheses, the **logical negation** of the result we wish to prove, and then reach some kind of contradiction. Principle of Contradiction is as follows.
   If we want to prove "If P, Then Q"
   (1) We assume P and Not Q.
   (2) We arrive at some conclusion contradicting one of our assumptions, or something obviously untrue for Not Q.
   (3) This contradicts our assumption for P and Not Q.

2. **Define ε - closure(q) with an example.**

   Ans: Epsilon Closure of a state is simply the set of all states we can reach by following the transition function from the given state that are labeled ε. This can be expressed as either $\hat{\varepsilon}(q)$ or ε-closure (q).

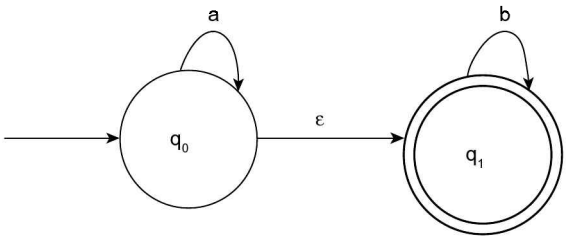


ε-closure $(q_0) = \{q_0, q_1, q_2\}$
ε-closure $(q_1) = \{q_1, q_2\}$
ε-closure $(q_2) = \{q_2\}$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

3. **Construct NFA for the regular expression a*b*.**

Ans:



4. **Is regular set closed under complementation? Justify.**

Ans: The **compliment** of two regular languages is regular.
If L is a regular language over alphabet Σ, then Σ*-L is also regular language.

5. **Specify the use of Context free grammar.**

Ans:
- Construction of compilers.
- Simplified the definition of programming languages.
- Describes the arithmetic expressions with arbitrary nesting of balanced parenthesis { (,) }.
- Describes block structure in programming languages.
- Model neural nets.

6. **Define parse tree with an example.**

Ans: A tree is a parse \ derivation tree for G if:
(i) Every vertex has a label which is a symbol of VU TU{∅}.
(ii) The label of the root is S.
(iii) If a vertex is interior and has a label A, then A must be in V.
(iv) If n has a label A and vertices $n_1, n_2, \ldots n_k$ are the sons of the vertex n in order from left with labels $X_1, X_2, \ldots X_k$ respectively then $A \rightarrow X_1X_2 \ldots X_k$ must be in P.
(v) If vertex n has label ε, then n is a leaf and is the only son of its father.

7. **State pumping lemma for CFL.**

Ans: Let L be any CFL. Then there is a constant n, depending only on L, such that if Z is in L and $|Z| >= n$, then $Z = UVWXY$ such that:
(i) $|VX| >= 1$
(ii) $|VWX| <= n$ and
(iii) for all $i >= 0$ UV^iWX^iY is in L.

8. **What is Chomsky normal Form?**

Ans: A CFG is in Chomsky Normal Form (CNF) if each of its productions has one of the two forms:
(1) Non terminal → string of exactly two Nonterminals i.e. A → BC
(2) Non terminal → one terminal i.e A → a

9. Mention the difference between P and NP problems.

Ans:

- **P.** Problems are those that can be solved by a Turing Machine (deterministic) in polynomial time. ("P" stands for polynomial). P problems are class of problems which can be solved efficiently.
- **NP.** Problems are those that can be solved by nondeterministic Turing machine in polynomial time. A problem is in NP if you can quickly (in polynomial time) test whether a solution is correct (without worrying about how hard it might be to find the solution).

10. What is recursively enumerable language?

Ans:

1. A language is said to be r.e if there exists a TM that accepts it.
2. L is recursively enumerable iff there is a TM that semi-decides L. (Turing acceptable languages).

11. What is meant by DFA?

Ans: Finite Automata is a mathematical model of a system consisting of finite set of states, set of transition from state to state that occurs on input symbols from alphabet. DFA has moves well defined on the current state and current input symbol. Deterministic finite automata can be defined using 5 tuple form as $M = (Q, \Sigma, \delta, q_0, F)$

Where Q = Non empty finite set of states

Σ = input alphabet

q_0 = initial start state

F = set of final states

δ = transition function that takes two arguments a state and input symbol and returns output as state i.e δ: $Q \times \Sigma \rightarrow Q$

Ex: $\delta(q_1, a) = q_2$

12. Define the term Epsilon transition.

Ans: Epsilon Closure of a state is simply the set of all states we can reach by following the transition function from the given state that are labeled ε. This can be expressed as either $\hat{\varepsilon}(q)$ or ε-closure (q).

13. What is a regular expression?

Ans: Regular Expression describes the language accepted by finite state automata. The basic operations performed on regular expressions are union, concatenation and Kleene's closure. Among the three, closure has highest precedence, next highest is for Concatenation and least is for union.

Example: an identifier in any high level language is given by R.E

An identifier starts with a letter followed by any number of letters or digits. The regular expression can be given as l(l|d)*.

14. Name any four closure properties of Regular languages.

Ans: The principal closure properties of regular languages are:

(1) The **union** of two regular languages is regular.

If L and M are regular languages, then so is $L \cup M$.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

(2) The **intersection** of two regular languages is regular.
If L and M are regular languages, then so is $L \cap M$.

(3) The **compliment** of two regular languages is regular.
If L is a regular language over alphabet Σ, then Σ^*-L is also regular language.

(4) The **difference** of two regular languages is regular.
If L and M are regular languages, then so is L – M.

15. What is a CFG?

Ans: A context free grammar (CFG) is denoted as G = (V, T, P, S) where V and T are finite set of variables and terminals respectively. V and T are disjoint. P is a finite set of productions each is of the form A->α where A is a variable and α is a string of symbols from (V U T)*.

16. Define the term Ambiguity in grammars.

Ans: A grammar is said to be ambiguous if it has more than one derivation trees for a sentence or in other words if it has more than one leftmost derivation or more than one rightmost derivation.

17. What is meant by Greibach Normal Form?

Ans: A context free language is said to be in Greibach Normal Form if all productions are of the form $A \rightarrow a\alpha$, where $a \in T$ and $\alpha \in V^*$.

18. List the closure properties of Context Free Languages.

Ans: CFL's are closed under union, concatenation, closure. These languages are not closed under intersection and compliment. If the context free languages are deterministic then they are closed even under intersection.

19. What is meant by recursively enumerable language?

Ans: The language for which the Turing machine halts for the words of the language and may or may not halt for the words that are not in the language is said to be a recursively enumerable language.

20. Define the class NP problem.

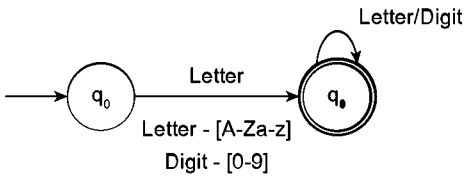
Ans: **NP.** Problems are those that can be solved by nondeterministic Turing machine in polynomial time. A problem is in NP if you can quickly (in polynomial time) test whether a solution is correct (without worrying about how hard it might be to find the solution). NP problems are class of problems which cannot be solved efficiently.

21. Draw the transition diagram (automata) for an identifier.

Ans: Identifier is a token which has a pattern "An alphabet followed by any number of alphanumeric characters". The corresponding regular expression is

[A-Za-z] [A-Za-z0-9]*.

When the expression is specified using regular expression then the corresponding Finite automata generated is as



22. What is a non deterministic finite automation?

Ans: Nondeterministic finite automata can be defined as quintuple M = (Q, Σ, δ, q_0, F)

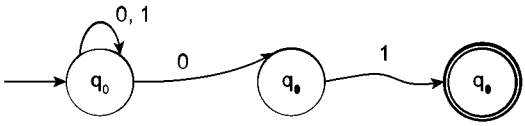
Where Q = Non empty finite set of states
Σ = input alphabet
q_0 = initial start state
F = set of final states
δ = transition function that takes two arguments a state and input symbol and returns output as state i.e δ: Q X $\Sigma \rightarrow 2^Q$

23. State pumping lemma for regular languages.

Ans: If L is a Regular Language represented with automaton with maximum of n states, then there is a word in L such that the length $|Z| \geq n$, we may write Z = UVW in such a way that $|UV| \leq n$, $|V| \geq 1$, and for all $i \geq 0$, UV^iW is in L.

24. Construct NFA equivalent to the regular expression: (0 + 1)*01.

Ans: We can have any string on 0 or 1 but should end with 0 followed by 1.



25. Write the CFG for the language L = {a^n b^n | n ≥ 1}.

Ans: The Given language is $a^n(bb)^n$ where n is greater than or equal to 1. It implies the language contains strings of the form {ab, aabb, aaabbb, ...}. Hence it can be defined as

$$S \rightarrow aSb \mid ab$$

26. Compare NFA and PDA.

Ans: NFA is Non – deterministic Automata and has moves well defined on the current state and current input symbol and has no storage. The language accepted by NFA is Regular Language and is defined as N = {Q, Σ, δ, q_0, F}where strings are as {w | (q_0, w) $\xrightarrow{*}$ q_f} i.e it reaches to at least one final state in the resultant

PDA is push down Automata and has moves well defined on the current state, current input symbol and element present on top of the stack. The language accepted by PDA is Context Free Language and is defined as P = {Q, Σ, Γ, δ, q_0, Z_0, F}where strings are as {w | (q_0, w, Z_0) $\xrightarrow{*}$ (q_f, ε, Z_0)} i.e it reaches to at least one final state.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

27. What are the closure properties of CFL?

Ans: CFL's are closed under Union, Concatenation, closure, substitution, homomorphism. CFL's are not closed under intersection and complement.

28. List out the different techniques for Turing machine construction.

Ans: The different techniques for Turing machines are

1. Storage in Finite Control
2. Check off symbol
3. Shifting over
4. Multi track tape
5. Subroutine

29. What are (a) recursively enumerable languages (b) recursive sets?

Ans:

1. A language is said to be r.e if there exists a TM that accepts it.
2. L is recursively enumerable iff there is a TM that semi-decides L. (Turing acceptable languages). Recursive sets consists set of strings that are accepted by the Turing machine.

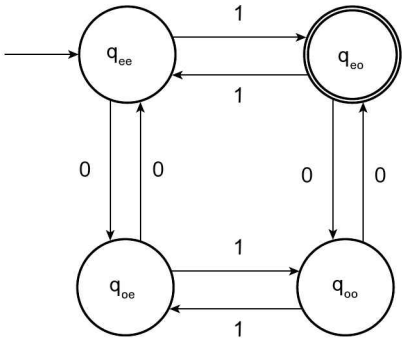
30. What is Universal Turing machines?

Ans: A universal Turing machine, M_u is an automata that, given as input the description of any Turing machine M and a string w, can simulate the computation of M for input w. To construct such an M_u we first choose a standard way of describing Turing machines. We may, without loss of generality, assume that M = (Q, {0, 1}, {0, 1, B}, δ, q_1, B, q_2) where

Q = {q_1, q_2, ...q_n} and q_1 the initial state, q_2 the single final state. The alphabet {0, 1, B} $\in \Gamma$ are represented as a_1, a_2 and a_3. The direction left and right are represented as D_1 and D_2 respectively. The transitions of TM are encoded in special binary representation where each symbol is separated by 1.

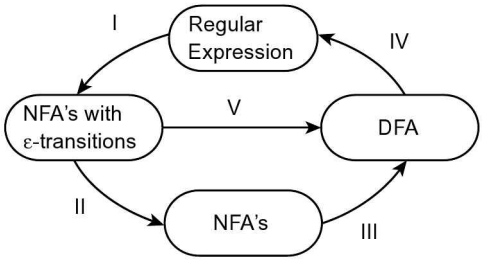
31. Construct deterministic finite automata to recognize odd number of 1's and even number of 0's?

Ans:



32. State the relations among regular expression, deterministic infinite automaton, non deterministic finite automaton and finite automaton with epsilon transition.

Ans:



NFA – Non deterministic Finite Automata.
DFA – Deterministic Finite Automata.

33. Let $L = \{w: w \in \{0, 1\}^*$ w does not contain 00 and is not empty$\}$. Construct a regular expression that generates L.

Ans: Given: $L = \{w: w \in \{0, 1\}^*$ w does not contain 00 and is not empty$\}$

Solution:

RE $= 1^*01^*$

34. Prove or disprove that the regular languages are closed under concatenation and complement.

Ans: Concatenation: Let $A = (Q, \Sigma, \delta, q_0, F)$ where

$Q = Q_1 \cup Q_2$, $\Sigma = \Sigma_1 \cup \Sigma_2$, $q_0 = q_1$, $F = F_2$ and δ is defined as follows:

$\delta(q, a) = \{\delta_1 (q, a)\}$ for each q in $Q_1 - F_1$

$\delta(q, a) = \{\delta_1 (q, a)\}$ for each q in F_1

and $\delta(q, a) = \{\delta_2 (q, a)\}$ for each q in Q_2.

A word w is in $L_1 L_2$ if $w = w_1 w_2$ where $w_1 \in L_1$ and $w_2 \in L_2$.

This is equivalent to $\delta_1(q_1, w_1)$ being F_1 and $\delta_2(q_2, w_2)$ being F_2 which is equivalent to $w_1 w_2$ being in $T(A)$. Thus $L_1 L_2 = T(A)$.

Theorem

The class of regular sets is closed under complementation. That is if X is a regular set and $X \subseteq \Sigma^* - L$ is a regular set.

Proof

Let X be $X(M)$ for DFA $M = (Q, \Sigma_1, \delta, q_0, F)$ and let $X \subseteq \Sigma^*$. First we assume $\Sigma_1 = \Sigma$, if there are symbols in Σ_1 not in Σ, we may delete all transitions of M on symbols not in Σ. The fact that $X \subseteq \Sigma^*$ assures us that we shall not thereby change the language of M. If there are symbols in Σ not in Σ_1, then none of these symbols appear in words of X. We may therefore introduce a dead state d into M with $\delta(d, a) = d$ for all in Σ and $\delta(a, a) = d$ for all q in Q and a in $\Sigma - \Sigma_1$. Now to accept $\Sigma^* - X$, complement the final states of M. That is let $M' = (Q, \Sigma, \delta, q_0, Q - F)$.

The M' accepts a word w if and only if $\delta(q_0, w)$ is in $Q - F$, that is, w is in $\Sigma^* - X$. Note that it is essential to the proof that M is deterministic and without y moves.

35. Consider the following grammar G with productions

$S \rightarrow ABC \mid BaB$ $A \rightarrow aA \mid BaC \mid aaa$ $B \rightarrow bBb \mid a$ $C \rightarrow CA \mid AC$

Give a CFG with no useless variables that generates the same language.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Ans: The variables A and B are generating and derives a terminal string. Therefore C is a useless variable. After removing 'C', the CFG is:

$S \rightarrow BaB$
$A \rightarrow aA \mid aaa$
$B \rightarrow bBb \mid a$

36. State the definition for Pushdown automata.

Ans: A push down automaton M is defined by $(Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$, where

- Q is *a finite set of states.*
- Σ is an alphabet called the *input alphabet* – is a finite alphabet of tape symbols.
- Γ is an alphabet called *stack alphabet* – is a finite alphabet of stack symbols.
- $q_0 \in Q$ is the *start state* (or) *initial state*
- z_0 in Γ is a particular stack symbol called *start symbol.*
- $F \subseteq Q$ is the set of final (or) *favorable states.*
- δ is the *transition relation.*

  i.e. δ is a subset of $(Q \times (\Sigma \cup \{e\}) \times \Gamma^*) \rightarrow (2^{Q \times \Gamma^*})$

37. Convert the following grammer G in greibach normal form.
S→ABb | a A → aaA | B B → bAb

Ans: **GNF (Solution):**

Replace S with A_1, A with A_2 and B with A_3.

$A_1 \rightarrow A_2 A_3 b \mid a$
$A_2 \rightarrow aaA_2 \rightarrow A_3$
$A_3 \rightarrow b A_2 b$

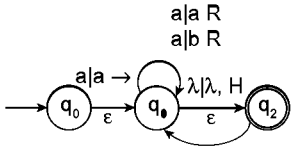
Here A_2 and A_3 are useless symbols. Therefore the required GNF form is:

$A_1 \rightarrow a$

38. Design a Turing machine with no more than three states that accepts the language.
a(a + b)*. Assume Σ = {a, b}.

Ans: TM for the language L = a(a + b)*

(i.e.) {*a, aa, ab, aab, abb*⊃}



39. State Rice's theorem.

Ans: Any nontrivial property about the language recognized by a Turing maching is undecidable.

A property about Turing machines can be represented as the language of all Turing Machines, encoded as strings, that satisfy that property. The property P is *about the language recognized by Turing machines* if whenever L(M) = L(N) then P contains (the encoding of) M iff it contains (the encoding of) N. The property is non-trivial if there is at least one Turing Machine that has the property, and at least one that hasn't.

40. Show that the collection of all Turing machines is countable.

Ans: First, observe that the set of all strings in Σ is countable for any alphabet. We create a listing of the strings by writing down all strings of length 1, followed by all strings of length 2, etc. Since each TM can be described by a finite string we can list all TM's by listing only those strings in Σ that correspond to valid TM representations.

41. What do you mean be expressive power of a grammar?

Ans: Expressive power of a grammar is a measure of how concisely grammar can be expressed in a particular formalism by introducing the set of rules. In original motivation grammar was the description of language and comparison of various terminologies from the literature with respect to expressiveness.

42. Construct a PDA to accept the language $\{(ab)^n \mid n \geq 1\}$ empty stack.

Ans: L $= \{ab; abab, ababab \ldots\}$

$\delta(q_0, a, z_0) = (q_1, az_0)$

$\delta(q_1, b, a) = (q_0, \varnothing)$

$\delta(q_0, \lambda, z_0) = (q_2, \lambda)$

43. When does a Turing machine become an algorithm?

Ans: If a TM decides a language or computes a function it can be reasonably thought of as an algorithm that performs correctly and reliably some computational task.

44. Give a semi-Thue grammar generating (a^i|*i* is a positive power of 2).

Ans: Give a semi-The grammar generating $\{a^i \mid$ i is a positive power of 2).

$S \rightarrow ACaB,\ Ca \rightarrow aaC,\ CB \rightarrow DB,\ CB \rightarrow E,\ aD \rightarrow Da,$

$AD \rightarrow AC,\ aE \rightarrow Ea,\ AE \rightarrow \varepsilon$

45. What is {10, 11}* (Write atleast first seven terms)

Ans: (ε, 10, 11, 1011, 101011, 101111, 111110, ...)

46. State Greback Normal From.

Ans: A context free grammar G is in GNF if every production is of the form $A \rightarrow a\alpha$ where $\alpha \in N^*$ and $a \in T$ (α may be λ) and $S \rightarrow \lambda$ is in G if $\lambda \in L(G)$, where S does not appear on the RHS of any production.

47. For a PDA M $= (Q, \Sigma, \Upsilon, \delta, q_0, Z_0, F)$, define the language accepted by final state.

Ans: Let M $= (Q, \Sigma, \in, \delta, q_0, Z_0, F)$ be a PDA. Then, the language accepted by M is the set of strings such that

$L(M) = (w \mid q0, w, z0) \mid -(P, \varepsilon, \gamma), P \in F, \gamma \in \Gamma^*)$. i.e., PDA M consumes *w* from the input and enters an accepting state.

48. When do you say a problem is NP-Hard?

Ans: It is obvious the $P \subseteq NP$. It is not known that whether there exists some language *L* in NP, which is also accepted by deterministic TM.

A language *L* is said to be NP-hard if $L_1 \leq PL$ for every $L_1 \subseteq NP$. The language *L* is NP-complete if $L \subseteq NP$ and *L* is NP-hard.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

49. State pumping lemma for context-free languages.

Ans: Let L be an infinite context-free Language. Then there exists some positive integar m such that any $w \in L$ with $|w \geq m$ can be decomposed as

$$w = uvxyz \quad (1)$$

with

$$|vxy| \leq m \quad (2)$$

and

$$|vy| \geq 1 \quad (3)$$

Such that

$$uv^i xy^i z \in L \quad (4)$$

for all $i = 0, 1, \ldots$ This is known as pumping lemma for context free languages.

50. State two languages, which are not recursively enumerable.

Ans: State two languages, which are not recursively enumerable.

(i) The diagonalization language L_d is the set of string w_i, where $w_i \notin L(M_i)$

(ii) The language NSA (Not Self Accepting)

51. List any four ways of theorem proving

Ans:

(i) Deductive proofs

(ii) Proof about sets

(iii) Proof by contradiction

(iv) Proof by counter example

52. Show that the complement of a regular language is also regular.

Ans: Let $L = L(A)$ for the DFA $A = (\alpha, \Sigma, \delta, q_0, F)$

then $\overline{L} = L(B)$ where B is a DFA and $B = (\alpha, \Sigma, \delta, q_0, F)$

$B = (\alpha, \Sigma, \delta, q_0, Q-F)$

DFA B is similar *h* DFA *A* but the final states of *A* have become the non-final states of DFA B and the final states of DFA B have become the non-final states of DFA *A*.

$\therefore$ ω is in L(B) iff $\delta^*(q, \omega)$ is in Q–F, which occurs iff ω is not in L(*A*)

53. What is meant by equivalent states in DFA?

Ans: The states 'p' and 'q' are said to be equivalent if for all input strings $\omega \in \Sigma^*$, $\delta^*(p, \omega)$ is a final stae iff $\delta^*(q, \omega)$ is a final state.

54. State pumping lemma and its advantage.

Ans: Let L be a regular language. Then there exists a constant '*n*', the number of states that accepts L such that if z is any string in L and z can be written as such that $z = \mu\nu\omega$ in a way

$|\mu\nu| \leq n$

$|\nu| \geq 1$ and

for all $i \geq 0$, $\mu\nu^i \in \Sigma$

The pumping language is used to check whether the certain language is regular or not.

55. Consider the alphabet Σ = (a, b, (,), + , *, · ∅) construct a CFG that generates all strings in Σ* that are regular expressions over the alphabet {a, b}.

Ans: $E \rightarrow E + E \mid E^*E \mid E \cdot E \mid (E) \mid a \mid b \mid \in$

56. Find whether the language $\{a^m, b^m, c^m / m \geq 0\}$ is CFL or not.

Ans: Let $z = a^n b^n c^n$

Then $|z| = |a^n b^n c^n| = 3^n$

Let 'n' be the number defined in pumping lemma.

$|z| = 3^n > n$

As $0 \leq |vx| \leq n$, 'v' or '*x*' cannot contain all the three symbols of a, b, c

So, 'v' or '*x*' is of the form

(i) a's
(ii) b's
(iii) c's
(iv) $a^i b^j$
(v) $b^i c^j$

Then $uv^i ux^i y$

Let $i = 0$, $u = a^i$, $v = b^j$

$z = uv^0 wx^c y - uwy$

∴ uwy contains atmost a^i, b^{n-j}, c^n, which is not of the form $uv^i wx^i y$

∴ $uwy \notin L$

∴ L is not content free.

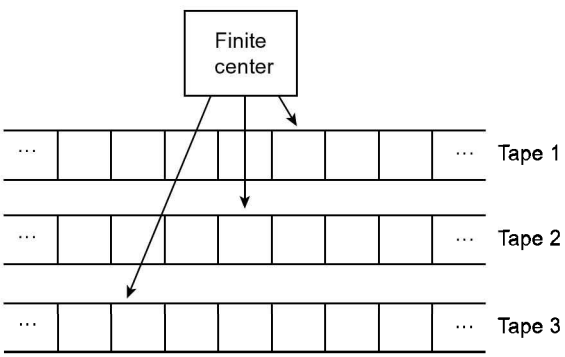
57. Define the language recognized by the PDA using empty stack.

Ans: Let $M = (\mu, \Sigma, \Pi, \delta, q, z, F = \phi)$ be a PDA, then the language accepted by empty stack or null stack is denoted by N(M) defined as

$N(M) = \{w/(q, w, z) \overset{*}{\vdash} (p, \in, e)\ \phi \wedge (p, \in, z)$ for sum p in a$\}$

58. What is meant by multitape Turing Maching?

Ans: A multitape Turing Machine has a finite control with some finite number of tapes. Each tape is infinite in both direction. It has its own initial state and some final states.



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

59. What are useless symbols in a grammar?

Ans: A symbol X is said to be useful if there is a derivation such that

$$S \overset{*}{\Rightarrow} \alpha \times \beta \overset{*}{\Rightarrow} \omega,$$

for some $\alpha, \beta, \in (VUT)^* w \in T^*$
otherwise X is said to be useless.

60. Define diagonal language

Ans: The language L_d consists of all those strings 'w' such that the Turing Machine represented by 'w' does not accept the input 'w'

$$L_d = \{w_c \mid w_c \notin L(M_i)\}$$

61. What is a finite automation? Give the examples.

Ans: A finite automation (FA) is a mathematical model/computational model of a system with discrete inputs and outputs. It consists of a finite set of states and set of transitional from one state to another state that depends in input symbols.

E.g.,

(i) Control mechanism of Elevator
(ii) Working Principle of Digital Logic Circuit.
(iii) Operation of washing Machine.

62. Enumerate the differences between DFA and NFA.

Ans:

DFA	NDFA
The future/behaviour of the system can be determined uniquely.	The future/behaviour of the system can't be determined uniquely
$\delta : Q \times \Sigma \to C\alpha$	$\delta : C\alpha \times \Sigma \to 2^{C\alpha}$
$\delta(q, a) = p \quad \forall a \in \Sigma$ $\forall q, p \in C\alpha$	$\delta(q, a) = \{p\} \quad \forall a \in \Sigma$ $\forall p, q \in C\alpha$
Results in a single state on leading an input symbol	Results in a multiple state on leading an input symbol

63. Verify whether $L = \{a^{2n} / n \geq I\}$ is regular.

Ans:

(i) Assume that L is regular.
Let 'n' be the number of states in FA accepting L.

(ii) Let $z = a^{2n}$.
Then $|z| = 2x > x$

By PL, $z = uvw$ with $|uv| \leq x$ & $|v| > 0$

(iii) Let $I = 2$ is $uv^i w$

Then $|uv^2 w| = |uvw \cdot v|$

As $|uv^2w| = |uvw| + |v|$

$= 2x + x$

$= 3x \neq 2x$

i.e., 'a' increases is proves of $2x$ not in $3x$

$\therefore$ $uv^2w \in L$, which is a contradiction.

Hence L is not regular.

64. Mention the closure properties of regular languages.

Ans:

- Union
- Intersection
- Complement
- Difference
- Reversal
- Closure

65. Let the properties of a grammar be S → OB|1A; A → O|OS|1AA, B → 1|1S|OAA. For the string 0110, find a right most deviation.

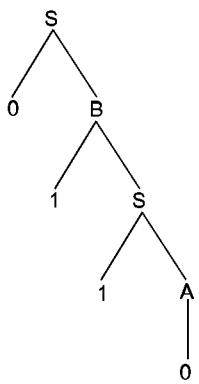
Ans: $S \Rightarrow OB$

$\underset{RMD}{\Rightarrow} 01S$

$\underset{RMD}{\Rightarrow} 011A$

$\underset{RMD}{\Rightarrow} 0110$

$\therefore S \underset{RMD}{\Rightarrow} 0110$



66. Define the languages generated by a PDA Using the two method of accepting a language.

Ans: Let $M = (Q, \Sigma, \Gamma, \delta, q_\epsilon, Z_0 F)$ be a PDA. Then the language accepted by a final store is denoted by L(M), defined as

$$L(M) = \{\omega / (q_0, \omega, z_0) \overset{*}{\rightarrow} (P, \in, v) \text{ for serve } P \in F, v \in \Gamma^*\}$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_e, F = \phi)$ be PDA. Then the language accepted by a empty stack/null stack is denoted by $N(M)$, is defined as,

$$N(M) = \{\omega / (q_0, \omega, z_0) \overset{*}{\leftarrow} (P, \in, \in) / (P, \in, Z_0) \text{ for serve } P \in Q\}$$

67. State pumping lamina for content-free language.

Ans: Let L be any central free language. Then there is a constant 'n' depending on 'L' bench that YZ is in L and $|Z| \geq x$, then $z = uvuxy$ such that

$$|vx| \geq I$$
$$|v\omega x| \leq x$$

For all $i \geq 0$, $uv^i wx^i y \in L$

68. Define a Turning Machine.

Ans: A Turning Machine $M = (\alpha, \Sigma, \Gamma, \delta, q_0, B_1 F)$

Where

Q - finite set of states.

Γ - set of tape symbols, tape alphabet

Σ - set of input symbols, input alphabet

$q_0 \in c\alpha$ the initial state

$B \in \Gamma$ called black symbol.

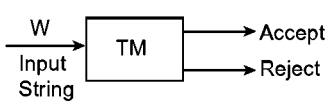
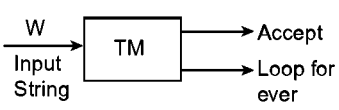
$F \leq c\alpha$ the set of final state

δ is a partial function (transition function) mapping from $Q \times \Gamma$ to finite sets or $Q \times \Gamma \times \{L, R\}$

i.e. $f : Q \times \Gamma \rightarrow$ finite subsets q $Q \times \Gamma \times \{L, R\}$ where L – Left direction R – Right direction.

69. Differentiate between recursive and recursively enumerate languages.

Ans:

Recursive	Recursively Enumerate
A language is recursive if there exists a Tuning Machine M that accepts L and goes to halt state or else rejects L.	A language is recursively enumerable (RE) if there exists a Tuning Machine that accepts every string of L and does not accepts strings that are not in the language.
	

70. Mention any two undesirability properties of recursively enumerable languages.

Ans:

- Emptiness
- Finiteness
- Regularity
- Content free.

PART B - Detailed Questions

1. Prove by induction on n that $\sum_{i=0}^{i=n} i = \frac{n(n+1)}{2}$

Ans: Many theorems are proved by mathematical induction. Suppose we have a statement P(n) about a non negative integer n. A commonly chosen example is to take P(n) to be

$$1 + 2 + 3 + \ldots\ldots\ldots\ldots\ldots\ldots n = \frac{n(n+1)}{2}$$

The principle of mathematical induction is that P(n) follows from

(a) P(0) and

(b) P(n-1) implies P(n) for N ≥ 1

condition (a) is an inductive proof, called the 'basis'.

condition (b) is called the inductive step.

The L.H.S of (b) that is P(n-1) is called the inductive hypothesis.

Ex:

Basis of induction:

n = 1 than L.H.S = 1

R.H.S = n(n + 1)/2 = 2/2 = 1

Induction hypothesis:

We assume n = k. Then equation becomes

1 + 2 + 3 + ……..+ K = K(K + 1)/2

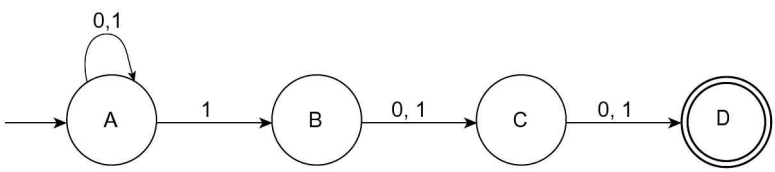
Inductive step: We assume that equation is true for n = k. And then check if it is also true for n = K + 1 or not.

L.H.S = 1 + 2 + 3……K + (K + 1)

$$\text{R.H.S} = \frac{K(K+1)}{2} + (K+1) = \frac{K^2 + 3K + 1}{2} = \frac{(K+1)(K+2)}{2}$$

2. Construct a DFA accepting binary strings such that the third symbol from the right end is 1.

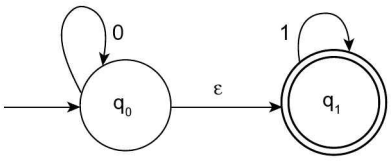
Ans: Since third symbol from right end is 1, we can have minimum 4 states.
The first transition should be on 1, remaining can be either 0 or 1.



- Every language that can be described by NFA can be described by some DFA
- DFA in practice has more states than NFA
- Equivalent DFA can have at most 2^n states where as NFA has only n states.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

3. Construct NFA without ε - transitions for the NFA given below.



Ans: The transition table is

	a = 0	a = 1	a = ε
q_0	q_0	∅	q_1
$*q_1$	∅	q_1	∅

Step 1: Find ε–closure of each state.
ε-closure $(q_0) = \{q_0, q_1\}$
ε-closure $(q_1) = \{q_1\}$
Step 2: Find the transition on each state for each element.

$\hat{\delta}(q_0, 0)$ = ε-closure $(\delta(\text{ε-closure } (q_0), 0))$
= ε-closure $(\delta(\{q_0, q_1\}, 0))$
= ε-closure $(\{q_0\}, \cup\{\varnothing\})$
= $\{q_0, q_1\}$

$\hat{\delta}(q_0, 1)$ = ε–closure $(\delta(\text{ε-closure } (q_0), 1))$
= ε-closure $(\delta(\{q_0, q_1\}, 1))$
= ε-closure $(\{\varnothing\}, \cup\{q_1\})$
= $\{q_1\}$

$\hat{\delta}(q_1, 0)$ = ε-closure $(\delta(\text{ε-closure } (q_1), 0))$
= ε-closure $(\delta(\{q_1\}, 0))$
= ε-closure $(\{\varnothing\})$
= $\{\varnothing\}$

$\hat{\delta}(q_1, 1)$ = ε-closure $(\delta(\text{ε-closure } (q_1), 1))$
= ε-closure $(\delta(\{q_1\}, 1))$
= ε-closure $(\{q_1\})$
= $\{q_1\}$

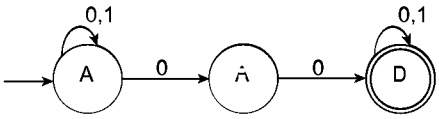
NFA without – ε transitions is

	a = 0	a = 1
$\rightarrow *q_0$	$\{q_0, q_1\}$	$\{q_1\}$
$*q_1$	∅	$\{q_1\}$

4. Construct NFA accepting binary strings with two consecutive 0's.

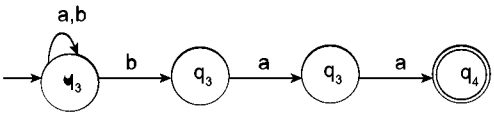
Ans: The language has strings of the form {00, 000, 100, 001, 1100, 0011, 0000...}

On two consecutive 0's go to state D using A. In A state and D state define moves to be in the same state on 0/1 as shown in the figure.

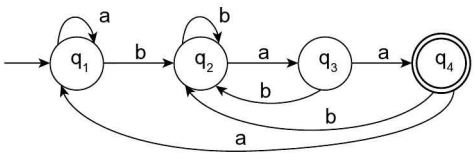


5. Obtain minimized finite automata for the regular expression (b/a)*baa.

Ans: The strings accepted are ending with baa. The NFA for it can be drawn as follows.



The DFA for it is as shown in the following figure



Minimization of the DFA
Let us represent the DFA as transition table.

	a = a	a = b
→q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_4	q_2
*q_4	q_1	q_2

(a) Initially we identify 0 – equivalence as $\prod_0 = \{Q_1^0, Q_2^0\}$ Where Q_1^0 is set of final states & $Q_2^0 = Q - Q_1^0$ is set of non final states.
$Q_1^0 = \{q_4\}$
$Q_2^0 = \{q_1, q_2, q_3\}$

(b) Construct $\prod_1$ from $\prod_0$ identifying the equivalent states in $\{ Q_1^0, Q_2^0 \}$
Q_1^0 cannot be divided as it has only one state.
Q_2^0 has four states, we need to identify whether they are 1- equivalent.
Compare q_1, q_2 on input a and b
$\delta (q_1, a) = q_1$
$\delta (q_2, a) = q_3$ both resultant states belong to Q_2^0.
$\delta(q_1, b) = q_2$
$\delta(q_2, b) = q_2$ both resultant states belong to Q_2^0.
$\Rightarrow q_1$ is 1- equivalent to q_2
Compare q_1, q_3 on input a and b
$\delta(q_1, a) = q_1$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$\delta(q_3, a) = q_4$ both resultant states belong to different sets in Π_0.
$\delta(q_1, b) = q_2$
$\delta(q_3, b) = q_2$ both resultant states belong to Q_2^0.
$\Rightarrow q_1$ is not 1- equivalent to q_3
$\therefore \Pi_1 = \{Q_1^1, Q_2^1, Q_3^1\}$
Where
$Q_1^1 = \{ q_4\}$
$Q_2^1 = \{ q_1, q_2 \}$
$Q_3^1 = \{ q_3\}$

(c) Construct Π_2 from Π_1 identifying the equivalent states in $\{Q_1^1, Q_2^1, Q_3^1\}$
Q_1^1 and Q_3^1 cannot be divided as these have only one state.
Q_2^1 has two states, we need to identify whether they are equivalent.
Compare q_1, q_2 on input a and b
$\delta(q_1, a) = q_1$
$\delta(q_2, a) = q_3$ both resultant states belong to different sets in Π_1.
$\delta(q_1, b) = q_2$
$\delta(q_2, b) = q_2$ both resultant states belong to Q_2^0.
$\Rightarrow q_1$ is not 2- equivalent to q_2
$\therefore \Pi_2 = \{Q_1^2, Q_2^2, Q_3^2, Q_4^2 \}$

Where
$Q_1^2 = \{ q_4\}$
$Q_2^2 = \{ q_1 \}$
$Q_3^2 = \{ q_2\}$
$Q_3^2 = \{ q_3\}$

(d) We see that Π_2 is equal not equal to Π_1, and each set has only single state

	a = a	a = b
$\rightarrow q_1$	q_1	q_2
q_2	q_3	q_2
q_3	q_4	q_2
$*q_4$	q_1	q_2

6. Prove that there exists NFA with ε- transition that accepts the regular expression y.

Ans: This statement can be proved by **mathematical induction.**

To show that it is true let k = ∅. The corresponding FA can be showed as follows.

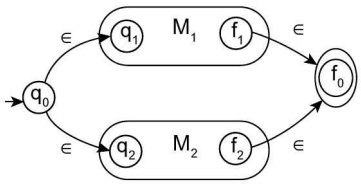
Let k = 0. The string with length 0 is ε and its corresponding FA is shown in table

Let k = 1. The string of length 1 in Σ defined as {a, b} is either a or b and its corresponding FA is shown in table

Let us assume that it is true for all k = 2, 3, 4, ...i-1. To show that it is true for k = i there are three possible operations applied on regular expression which has corresponding FA.

Case 1:

Let r_1 and r_2 be two regular expressions with less than i operations that has FA M_1 and M_2 given as $M_1 = \{Q_1, \Sigma_1, \delta_1, q_1, \{f_1\}\}$ and $M_2 = \{Q_2, \Sigma_2, \delta_2, q_2, \{f_2\}\}$ To show that r formed using union operation as $r = r_1 + r_2$ has automaton M which accepts the language $L(M_1) \cup L(M_2)$ and is shown below.



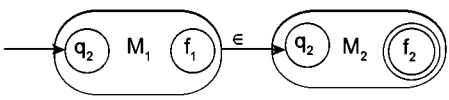
$M = \{\{Q_1 \cup Q_2 \cup \{q_0, f_0\}\}, \{\Sigma_2 \cup \Sigma_2\}, \delta, q_0, \{f_0\}\}$ where δ is defined as

(1) $\delta(q_0, \varepsilon) = \{q_1, q_2\}$
(2) $\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$.
(3) $\delta(q, a) = \delta_2(q, a)$ for q in $Q_2 - \{f_2\}$ and a in $\Sigma_2 \cup \{\varepsilon\}$.
(4) $\delta(f_1, a) = \delta_2(f_2, a) = \{f_0\}$

By inductive hypothesis there are no transitions out of the final states. Thus all moves of M_1 and M_2 are present in M. Any string x valid in M_1 or M_2 must be valid in M. For such a string there exists a path from q_0 to f_0. To prove this observe rule 1 which shows path from q_0 to either q_1 or q_2. Since "x" is valid in either M_1 or in M_2 there exists path from q_1 to f_1 or from q_2 to f_2. By rule 2 and 3 all these transitions are also present in M. By rule there is a path from f_1 and f_2 to f_0. Hence there is path from initial state to final state which indicates "x" is also valid in M.

Case 2:

Let r_1 and r_2 be two regular expressions with less than i operations that has FA M_1 and M_2 given as $M_1 = \{Q_1, \Sigma_1, \delta_1, q_1, \{f_1\}\}$ and $M_2 = \{Q_2, \Sigma_2, \delta_2, q_2, \{f_2\}\}$ To show that r formed using concatenation operation as $r = r_1 . r_2$ has automaton M which accepts the language $L(M_1) . L(M_2)$ as shown below.



$M = \{\{Q_1 \cup Q_2\}, \{\Sigma_1 \cup \Sigma_2\}, \delta, q_1, \{f_2\}\}$ where δ is defined as

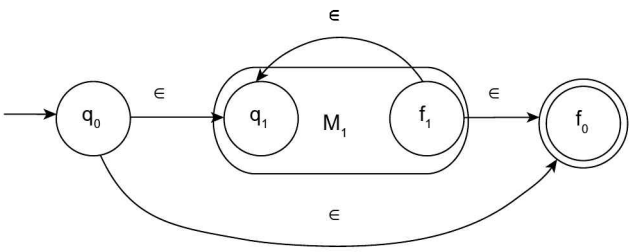
(1) $\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$.
(2) $\delta(f_1, \varepsilon) = \{q_2\}$
(3) $\delta(q, a) = \delta_2(q, a)$ for q in Q_2 and a in $\Sigma_2 \cup \{\varepsilon\}$.

The construction of M is as shown in the figure. Any string w = xy is valid if x is in M1 and y is in M2. For such a string there exists a path from q_1 to f_2 where q_1 is initial state and f_2 is final state in M. To prove this observe rule 1 which shows path from q_1 to f_1 on "x" as is valid in M_1 By rule 2 there is path from f_1 to q_2 on ε. Rule 3 includes path from q_2 to f_2 on string "y" Hence there is path from initial state q_1 to final state f_2 which is the concatenated string "x" and "y". Since w = x.y, w is valid string in M.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Case 3:

Let r_1 regular expression with less than i operations that has FA M_1 given as $M_1 = \{Q_1, \Sigma_1, \delta_1, q_1, \{f_1\}\}$. To show that r formed using concatenation operation as $r = r_1^*$ has automaton M which accepts the language $L(M_1)^*$ as shown below.



$M = \{\{Q_1 \cup \{q_0, f_0\}\}, \Sigma_1, \delta, q_0, \{f_0\}\}$ where δ is defined as

(1) $\delta(q_0, \varepsilon) = \delta(f_1, \varepsilon) = \{q_1, f_0\}$

(2) $\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$.

The construction of M is as shown in the figure. Any string $w = \{\varepsilon, x, xx, \}$is valid if x is in M_1 For such a strings there exists a path from q_0 to f_0 . If string is ε then there is path from q_0 to f_0 using rule 1. For string's x or xx the path is from q_0 to q_1 on ε followed by path on x from q_1 to f_1 using rule 2(if the string x is repeated then followed by a path on ε from f_0 to q_1 to repeat the string) followed by path on ε from f_1 to f_0 using rule 1. Hence $L(M) = L(M_1)^*$.

7. Which of the following languages is regular? Justify.

(a) $L = \{a^n b^m \mid n, m \geq 1\}$

Ans: This language corresponds to regular expression a*b* which is regular as there exists a DFA to accept all strings of the language.

(b) $L = \{a^n b^n \mid n \geq 1\}$

Ans: Let us assume that the language is regular. According to Pumping lemma choose the string $Z = a^n b^n$ where the length is 2n. Let n be the number of states such that $|Z| \geq n$. Now let us represent the string Z as UVW and if the language is regular then for all i, $UV^iW \in L$.

To check whether it is regular or not, it is required to consider three possible cases i) where the string V is formed with only a's or ii) with only b's or iii) with the combination of a's and b's.

Case 1: V is a string such that it contains only a's.

$V = a^x$ Such that $x \geq 1$.

Let $i = 0$, then the string formed is UW, the string would be of the form $a^{n-x}b^n \notin L$ as the number of a's is less than the number of b's.

Case 2: V is a string such that it contains only b's.

$V = b^x$ Such that $x \geq 1$.

Let $i = 0$, then the string formed is UW, the string would be of the form $a^n b^{n-x} \notin L$ as the number of b's is less than the number of a's.

Case 3: V is a string such that it contains combination of a's and b's.

$V = a^x b^x$ Such that $x \geq 1$.

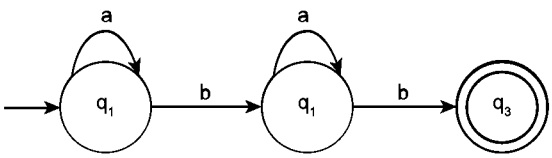
If $i = 0$, then the string formed is UW, the string would be of the form $a^{n-x}b^{n-x} \in L$ as the number of a's is equal to number of b's.

If $i = 1$, then the string formed is UVW, the string would be of the form $a^{n-x}a^x b^x b^{n-x} \in L$ as the number of a's is equal to number of b's.

Now if $i = 2$, then the string formed is UV^2W, the string would be of the form $a^{n-x}a^x b^x a^x b^x b^{n-x} \notin L$, the language is defined as all a's followed by all b's, but the string generated is a's followed by b's then by a's and then by b's which is invalid string according to the language.

Since in all the three possible cases there exists value of i such that the string is not in L. Hence the language is not regular.

8. Obtain the regular expression for the finite automata.



Ans: The set of equations that can be framed are

$$q_1 = q_1 a + \varepsilon \quad -1$$
$$q_2 = q_1 b + q_2 a \quad -2$$
$$q_3 = q_2 b \quad -3$$

Equation 1 is in the required form to apply the Arden's theorem.

$$q_1 = q_1 a + \varepsilon$$
$$q_1 = \varepsilon (a)^*$$

Substituting the expression of q_1 in equation 2 we get

$$q_2 = (a)^* b + q_2 a$$
$$q_2 = (a)^* b (a)^*$$

Substituting the expression of q_2 in equation 3 we get

$$q_3 = (a)^* b (a)^* b$$

The regular expression for the given DFA is **(a)*b (a)*b**

9. Is the grammar $E \rightarrow E + E \mid E * E \mid id$ is ambiguous? Justify your answer.

Ans: **LMD:** for string id + id * id is

$$\mathbf{E \Rightarrow E + E}$$
$$\mathbf{\Rightarrow id + E}$$
$$\mathbf{\Rightarrow id + E * E}$$
$$\mathbf{\Rightarrow id + id * E}$$
$$\mathbf{\Rightarrow id + id * id}$$

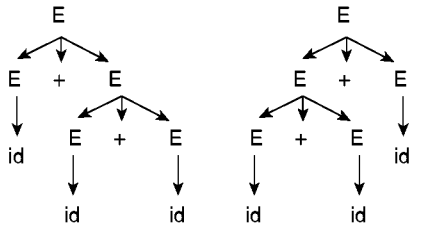
RMD: for string id + id * id is

$$\mathbf{E \Rightarrow E * E}$$
$$\mathbf{\Rightarrow E * id}$$
$$\mathbf{\Rightarrow E + E * id}$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$$\Rightarrow \mathbf{E + id * id}$$
$$\Rightarrow \mathbf{id + id * id}$$

Parse trees represented by above LMD and RMD are as follows:



As there are more than one parse tree, grammar is ambiguous.

10. Find the context free language for the following grammars

(1) $S \rightarrow aSbS \mid bSaS \mid \varepsilon$

Ans: The language generated by this grammar is set of all strings such that the number of a's is equal to number of b's.

$$L = \{w \mid N_a(w) = N_b(w)\}$$

This can be proved by mathematical induction.

(2) $S \rightarrow aSb \mid ab$

Ans: The language generated by this grammar is set of all strings such that the number of a's is equal to number of b's and all b's follow a's.

$$L = \{a^n b^n \mid n \geq 1\}$$

This can be proved by mathematical induction.

11. Construct the PDA for $L = \{ww^r \mid w \text{ is in } (a + b)^*\}$

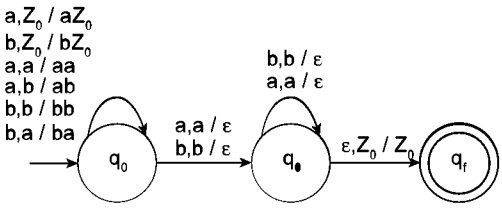
Ans: Read the string w and push it on to the stack. After that read each symbol, if it matches with top of the stack pop off the symbol. When input is read completely, then if stack becomes empty, then it is successful.

The PDA can be given as follows:

Let q_0 be initial state, q_f be final state and Z_0 be bottom of the stack.

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$
$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$
$$\delta(q_0, a, a) = (q_0, aa), (q_1, \varepsilon)$$
$$\delta(q_0, b, b) = (q_0, bb), (q_1, \varepsilon)$$
$$\delta(q_0, a, b) = (q_0, ab)$$
$$\delta(q_0, b, a) = (q_0, ba)$$
$$\delta(q_1, a, a) = (q_1, \varepsilon)$$
$$\delta(q_1, b, b) = (q_1, \varepsilon)$$
$$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0)$$

The final PDA is given by M = $\{(q_0, q_1, q_f), (a, b), (a, b, Z_0), \delta, q_0, Z_0, q_f\}$.



12. Discuss the equivalence between PDA and CFG.

Ans: Suppose L is a context-free language. Then there is a PDA M such that L = N(M).

Proof: The basic idea in the construction is to build M so that it simulates the leftmost derivation of strings using G. The machine we construct uses the terminals and non-terminals of the grammar as stack symbols. What we conceptually want to do is to use the stack to hold the sentential form that evolves during a derivation. At each step, the topmost variable in the stack will get replaced by the RHS of some grammar rule. Of course, there are several problems with implementing this concept. For one, the PDA can only access the top of its stack it can't find a variable below the top. For another, even if the PDA could find such a variable, it could not fit the RHS into a single stack slot. But these are not insurmountable. We simply have to arrange things so that the PDA always has the leftmost variable of the sentential form on top of the stack. If that can be set up, the PDA can use the technique of using extra states to push multiple symbols 'all at once'.

The other consideration is that we are constructing a PDA, so it needs to consume the input string and give a verdict. This fits in nicely with our other requirements. In brief, the PDA will use ε-transitions to push the RHS of rules into the stack, and will use 'normal' transitions to consume input. In consuming input, we will be able to remove non-variables from the top of the stack, always guaranteeing that a variable is at the top of the stack.

We assume that ε is not in L(G) and let G be given as (V, T, P, S) be a context free grammar in Greibach normal form. It is required to construct M = ({q}, T, V, δ, S, Φ) where δ(q, a.A) contains (q, γ) whenever A→aγ is in P.

The PDA M simulates leftmost derivations of G. Since G is in Griebach normal form, each sentential form in a leftmost derivation consists of terminals x followed by a string of variables α. M stores the suffix α of the left sentential form on its stack after processing the prefix x. Formally we show that

$S \Rightarrow x\alpha$ by a leftmost derivation if and only if $(q, x, S) \overset{*}{\underset{M}{\vdash}} (q, \varepsilon, \alpha)$

13. Find GNF for the grammar

S→AA | 1
A→SS | 0

Ans:

$$S \rightarrow AA \mid 1 \quad \ldots\ldots(1)$$
$$A \rightarrow SS \mid 0 \quad \ldots\ldots(2)$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Apply lemma 1 for (2)

$$A \rightarrow AAS \mid 0S \mid 1$$

Apply lemma 2 for this

$$A \rightarrow 1S \mid 0 \mid 1SZ \mid 0Z$$

$Z \rightarrow AS \mid ASZ$ substitute for A now in Z.

$$Z \rightarrow 1SS \mid 0S \mid 1SSZ \mid 0SZ \mid 1SZSZ \mid 1SZS \mid 0ZS \mid 0ZSZ$$

The grammar in GNF is as follows

$$S \rightarrow 1 \mid 1SA \mid 0A \mid 1SZA \mid 0ZS$$
$$A \rightarrow 1S \mid 0 \mid 1SZ \mid 0Z$$
$$Z \rightarrow 1SS \mid 0\,S \mid 1SSZ \mid 0SZ \mid 1SZSZ \mid 1SZS \mid 0ZS \mid 0ZSZ$$

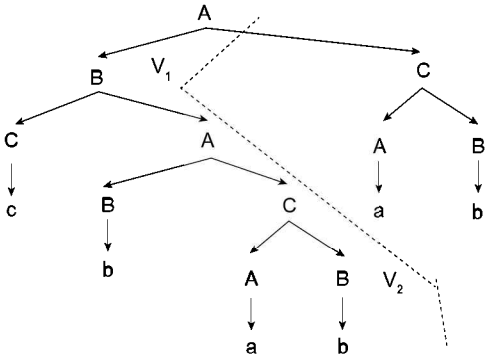
14. Explain any two higher level techniques for Turing machine construction

***Ans:* Storage in Finite Control**

A Turing machine has a finite number of states in its CPU. However, the number of states is not always small. Like a Pentium chip we can store values in it as long as there is only finite number of states. For example all real computers have registers but there are only a fixed number of them, AND each register can only hold one of a fixed (and finite) number of bits. Similarly we define a state as a pair which stores the details of control and other stores the symbol. To account this modification we can define the turing machine as $M = (Q, \Sigma, \Gamma, \delta, [q_0, B], B, F)$ where Q is of the form [q, a] where q is a state and $a \in \Sigma$, the transitions are defined as $([QX\Sigma], \Gamma) \rightarrow ([QX\Sigma], \Gamma, \{R, L\})$. For example the transition $\delta([q,a],b) = ([p,b],c,R)$ indicates that the control is in state q and a is stored in finite control. On the input symbol b it moves to p state and changes the symbol in finite control to b, changes the cell content as c and moves one cell right.

Multi-tape tracks

The tape is imagined as divided into cells where input to be processed is placed. We can further imagine that the tape is divided into k tracks for some finite number k as shown below.

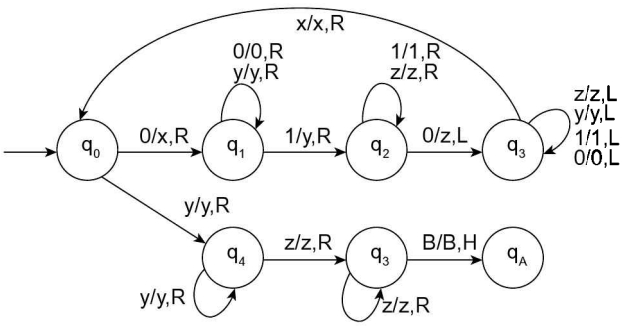


The reading head considers k symbols each belonging to different track in same column and processes it. There are two special symbols ϕ and $ used in the first track which indicates the boundary of the input. The other tracks are used to place the intermediate results and the final result of the processing. The blank input is identified as the all B's in all tracks as [B, B, B]. The input at the current position of the reading head is [1, 1, 1].

15. Construct Turing machine for L = {$1^n 0^n 1^n$ | n≥1}

Ans: The strings of 1's followed by 0's and followed by 1's with number of 1's, 0's and 1's are equal.

To design this we need 7 states as procedure is similar to {1^n0^n | n ≥ 0} with extra state added to take care of number of 1's. The corresponding transition diagram is shown below.



The transition table for the same is shown below.

	1	0	x	Y	z	B
q_0	(q_1, x, R)	---	---	(q_4, y, R)	---	---
q_1	(q_1, 1, R)	(q_2, y, R)	---	(q_1, y, R)	---	---
q_2	(q_3, z, L)	(q_2, 0, R)	---	---	(q_2, z, R)	---
q_3	(q_3, 1, L)	(q_3, 0, L)	(q_0, x, R)	(q_3, y, L)	(q_3, z, L)	---
q_4	---	---	---	(q_4, y, R)	(q_5, z, R)	---
q_5	---	---	---	---	(q_5, z, R)	(q_A, B, R)
q_A	---	---	---	---	---	---

16. Discuss the closure properties of CFL's

Ans: CFLs are closed under substitution, union, concatenation, closure and positive closure, reversal homomorphism, inverse homomorphism

CFLs NOT closed under - intersection, difference, complement

(a) CFLs are closed under union

If L_1 and L_2 are CFLs, then their union $L_1 + L_2$ is a CFL.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Let the grammar CFG_1 defines language L_1. Assume that the nonterminals in CFG_1 are S_1, A_1, B_1, C_1..... Let L_2 is a language defined by CFG2 and its nonterminals are S_2, A_2, B_2, C_2, ...

Now CFG_1 and CFG_2 have nonintersecting sets of nonterminals.

We create a CFG for $L_1 + L_2$ as follows:

Include all of the nonterminals $S_1, A_1, B_1, C_1, \ldots$ and $S_2, A_2, B_2, C_2, \ldots$.

Include all of the productions from CFG_1 and CFG_2.

Create a new nonterminal S and a new production in CFG as

$S \rightarrow S_1 \mid S_2$

(b) CFLs are closed under concatenation

If L_1 and L_2 are CFLs, then L_1L_2 is a CFL.

Let the grammar CFG_1 defines language L_1. Assume that the nonterminals in CFG_1 are S_1, A_1, B_1, C_1 ..... Let L_2 is a language defined by CFG_2 and its nonterminals are S_2, A_2, B_2, C_2, ...

Now CFG_1 and CFG_2 have nonintersecting sets of nonterminals.

We create a CFG for L_1L_2 as follows:

Include all of the nonterminals S_1, A_1, B_1, C_1, ... and $S_2, A_2, B_2, C_2, \ldots$.

Include all of the productions from CFG_1 and CFG_2.

Create a new nonterminal S and a production

$S \rightarrow S_1S_2$

(c) CFLs are closed under closure

If L is a CFL, then L* is a CFL.

Since L is a CFL, by definition there is some CFG that generates L.

Suppose CFG for L has nonterminals S, A, B, C, ... Change the nonterminal S to

S1.We create a new C CFG for L as follows:

Include all the nonterminals S_1, A, B, C, . . . from the CFG for L.

Include all of the productions from the CFG for L.

Add the new nonterminal S and the new production

$S \rightarrow S_1S \mid e$

We can repeat last production

$S \rightarrow S_1S \rightarrow S_1S_1S \rightarrow S_1S_1S_1S \rightarrow S_1S_1S_1S_1S \rightarrow S_1S_1S_1S_1e \rightarrow S_1S_1S_1S_1$

Note that any word in L* can be generated by the new CFG. To show that any word generated by the new CFG is in L*, note that each of the S_1 above generates a word in L. Also, there is no interaction between the different S_1's.

(d) CFLs are not closed under intersection

Let L_1 and L_2 are two CFLs. Then $L_1 \cap L_2$ maybe a CFL or may not be a CFL. That means it is not closed under intersection.

Proof: We now will give an example showing that the intersection of two CFLs may not be a CFL. To show this, we assume that the language $L_1 = \{a^nb^na^n: n \geq 1\}$ is a non context free language. L_1 is the set of words with some number of a's, followed by an equal number of b's, and ending with the same number of a's.

Let L_2 be generated by the following CFG:

$S \rightarrow XY$

$X \rightarrow aXb \mid \varepsilon$

$Y \rightarrow aY \mid \varepsilon$

Thus, $L_2 = \{a^n b^n a^m: n, m> = 0\}$, which is the set of words that have a clump of a's, followed by a clump of b's, and ending with another clump of a's, where the number of a's at the beginning is the same as the number of b's in the middle. The number of a's at the end of the word is arbitrary, and does not have to equal the number of a's and b's that come before it.

Let L_3 be generated by the following CFG:

$S \rightarrow WZ$

$W \rightarrow aW \mid \varepsilon$

$Z \rightarrow bZa \mid e$

Thus, $L_3 = \{a^i b^k a^k: i, k> = 0\}$, which is the set of words that have a clump of a's, followed by a clump of b's, and ending with another clump of a's, where the number of b's in the middle is the same as the number of a's at the end. The number of a's at the beginning of the word is arbitrary, and does not have to equal the number of b's and a's that come after it.

Note that $L_2 \cap L_3 = L_1$, where $L_1 = \{a^n b^n a^n: n = 0, 1, 2, \ldots\}$, which is a non context free language.

(e) CFLs are not closed under Complement

If L is a CFL, then $\bar{L}$ may or may not be a CFL.

We first show that the complement of a CFL may be a CFL:

If L is regular, then $\bar{L}$ is also regular. Also both L and $\bar{L}$ are CFLs.

We now show that the complement of a CFL may not be a CFL by contradiction: Suppose that it is always true that if L is a CFL, then $\bar{L}$ is a CFL. Suppose that L_1 and L_2 are CFLs. Then by our assumption, we must have that $\bar{L}_1$ and $\bar{L}_2$ are CFLs. Closure under union implies that $\bar{L}_1 + \bar{L}_2$ is a CFL. Then by our assumption, we must have that compliment of $(L_1 + \bar{L}_2)$ is a CFL. But we know that compliment of $(\bar{L}_1 + \bar{L}_2) = L_1 \cap L_2$ by DeMorgan's Law. However, we previously showed that the intersection of two CFLs is not always a CFL, which contradicts the previous two steps. So our assumption that CFLs are always closed under complementation must not be true.

Thus, in general, we cannot say if the complement of a CFL is a CFL.

17. Explain undecidability with respect to post correspondence problem.

Ans: Given an alphabet S, one instance of Post's correspondence problem of size *s* is a finite set of pairs of strings (*gi*, *hi*) ($i = 1 \ldots s$ $s> = 1$) over the alphabet S. A solution of length $n> = 1$ to this instance is a sequence *i1 i2 ... in* of selections such that the strings *gi1gi2 ... gin* and *hi1hi2 ... hin* formed by concatenation are identical.

Width of a PCP instance is the length of the longest string in *gi* and *hi* ($i = 1, 2, ..., s$). *Pair i* is the short name for pair (*gi*, *hi*), where *gi* and *hi* are the top string and bottom string of the pair respectively. Mostly, people are interested in *optimal solution*, which has the shortest length over all possible solutions to an instance. The corresponding length is called *optimal length*. We use the word *hard* or *difficult* to describe instances whose optimal lengths are very large. For simplicity, we restrict the alphabet S to {0, 1}, and it is easy to transform other alphabets to their equivalent binary format.

To describe subclasses of Post's Correspondence Problem, we use PCP[*s*] to represent the set of all PCP instances of size *s*, and PCP[*s*, *w*] the set of all PCP instances of size *s* and width *w*.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

For convenience, we use a matrix of 2 rows and *s* columns to represent instances of PCP[*s*], where string *gi* is located at (*i*, 1) and *hi* at (*i*, 2). The following is the matrix representation of the instance {{100, 1}, {0, 100}, {1, 00}} in PCP[3, 3].

i	g_i	h_i
1	100	1
2	0	100
3	1	00

Let's consider the result of selections of pair 1, 3, 1, 1, 3, 2, 2 accordingly. They can be shown in the following table with each selection assigned a different color. After the elimination of blanks and concatenation of strings in the top and bottom separately, it turns to:

1001100100100

1001100100100

Now, the string in the top is identical to the one in the bottom; therefore, these selections form a solution to PCP problem.

18. Discuss the properties of recursive languages.

Ans: There are *three* possible outcomes of executing a Turing machine over a given input. The Turing machine may

- Halt and accept the input;
- Halt and reject the input; or
- Never halt.

A language is *recursive* if there exists a Turing machine that accepts every string of the language and rejects every string (over the same alphabet) that is not in the language. Note: If a language L is recursive, then its complement L must also be recursive.

A language is *recursively enumerable* if there exists a Turing machine that accepts every string of the language, and does not accept strings that are not in the language. (Strings that are not in the language may be rejected or may cause the Turing machine to go into an infinite loop.)

19. Explain any two undecidable problems with respect to Turing machine.

Ans: TM = {<M, w>| M is a TM, w is a string, M accepts w}

Assume a TM is decidable which halts and says accept or rejected. Let H be a machine for A Turing Machine <M, w>, H halts and accepts if M accepts w, or rejects if M fails to accept w.

i.e H(<H, w>) = {accept if M accept w.

= {reject if M does not accept w.

Construct new TM, D with H as subroutine. D calls H to find what M does when input to M is its own description <M>. i.e running a machine as its own decription. It is just like a compiler written and compiled in same language.

D gets information and complements the action.

D is defined as <M> where M is a Turing Machine.

(1) Runs H on input <M, <M>>.
(2) If H accepts it rejects, if H rejects it accepts.

$$\text{In summary } D(<M>) = \begin{cases} \text{accept} & \text{if M does not accept } <M> \\ \text{reject} & \text{if M accepts } <M> \end{cases}$$

When we run D with its own description <D> as input? In that case we get

$$D(<D>) = \begin{cases} \text{accept} & \text{if D does not accept } <D> \\ \text{reject} & \text{if D accepts } <D> \end{cases}$$

It is forced to do opposite to what D does. Thus neither TM D nor TM H exists.

20. Discuss the difference between NP-complete and NP Hard problems.

Ans: The subject of *computational complexity theory* is focused on classifying problems by how hard they are. There are many different classifications depending the time taken by the problem. The following are the types of classification.

(a) **P**. Problems are those that can be solved by a Turing Machine (deterministic) in polynomial time. ("P" stands for polynomial). P problems are class of problems which can be solved efficiently.

(b) **NP**. Problems are those that can be solved by nondeterministic Turing machine in polynomial time. A problem is in NP if you can quickly (in polynomial time) test whether a solution is correct (without worrying about how hard it might be to find the solution). NP problems are class of problems which cannot be solved efficiently.*NP does not stand for "non-polynomial"*. There are many complexity classes that are much harder than NP.

(c) **Undecidable**. For some problems, we can prove that there is no algorithm that always solves them, no matter how much time or space is allowed. One very uninformative proof of this is based on the fact that there are as many problems as there real numbers, and only as many programs as there are integers, so there are not enough programs to solve all the problems. But we can also define explicit and useful problems which can't be solved.

21. Construct DFA to accept the language L = {w / w is of even length and begins with 11}

Ans: This automaton should accept strings that start with 11 and followed by any other elements such that the total length must be even. It should reject strings that start with 0. Hence on seeing 0 in the initial state it should enter into dead state from which there is no path to final state. After 1 it should find 1, hence in the second state on seeing 0 it should enter into dead state. Once it sees 11 it is final state and on seeing either 0 or 1 it goes to next state and comes back to this state on seeing either 0 or 1, which maintains the length to be even.

Δ / Σ	0	1
$\rightarrow q_0$	D	q_1
q_1	D	q_2

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

* q_2	q_3	q_3
q_3	q_2	q_2

22. Write a note on NFA and compare with DFA

Ans: DFA has moves well defined on the current state and current input symbol. NFA the moves are not well defined i.e on an input it may go different set of states or there may be state change with out input symbol.

Language of DFA A = $\{Q, \Sigma, \delta, q_0, F\}$is defined as $\{w \mid \delta`(q_0, w) = P$ for some P in F$\}$

Language of NFA A = $\{Q, \Sigma, \delta, q_0, F\}$is defined as $\{w \mid \delta`(q_0, w) \cap F \neq \varnothing\}$ i.e there is at least one final state in the resultant

23. Convert the following NFA to DFA

δ	a	B
→p	{p}	{p, q}
q	{r}	{r}
*r	{Φ}	{Φ}

Ans: start with initial state, find transitions. Whenever you get new set of states, add it to set of states and find transitions on input symbols.

δ	A	b
→[p]	[p]	[pq]
[pq]	[pr]	[pqr]
*[pr]	[p]	[pq]
*[pqr]	[pr]	[pqr]

24. Discuss on the relation between DFA and minimal DFA.

Ans: **Let M and M^1 be two FA s over Σ. We construct a comparison table consisting of n + 1 columns where n is the no of input symbols.**

(1) 1st column consisting of pair of nodes of form (q, q^1) where q ∈ M & q^1 ∈ M^1.

(2) If (q, q^1) appears in same row of 1st column then corresponding entry in a column (a ∈ Σ) is (qa, qa^1) where (qa, qa^1) are reachable from q & q^1 on a .

(3) Table is constructed by starting with pair of initial vertices q_{in}, $q_{in}{}^1$ of M & M^1. We complete construction by considering the pairs in 2nd & subsequent column which are not in 1st column.

(i) if we reach a pair (q, q^1) such that q is final states of M & q^1 is non final state of M^1 ⇒ terminate construction and conclude that M & M^1 are not equivalent.

(ii) if construction is terminated when no new element appears in second & subsequent columns which are not on 1st column. Conclude that M & M^1 are equivalent.

25. Discuss on regular expressions.

Ans: Any terminal symbol / element of Σ is R.E
Ex: ϕ, $\in$, a in Σ
is a regular expression and denotes the empty set.
$\in$ is a regular expression and denotes the set $\{\in\}$.
a is a regular expression and denotes the set {a}.

- Union of two regular expressions R_1 & R_2 is Regular expression R. ($R = R_1 + + R_2$)
  Ex: Let "a" be regular expression R_1
  "b" be regular expression R_2
  (a + b) is also a regular expression R having the elements {a, b}.
- Concatenation of two regular expression R_1 & R_2 written as R_1 R_2 is also Regular Expression R ($R = R_1 . R_2$)
  Ex: Let "a" be regular expression R_1
  "b" be regular expression R_2
  (a.b) is also a regular expression R having the elements {ab}.
- Iteration (Closure) of a regular expression R written as R* is also a regular expression.
  Let "a" be regular Expression
  Then $\in$, a, aa … are also regular expression.
  If L is a language represented by the regular expression R then the Kleene closure of L is denoted as L^* and is given as

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

The positive closure of L, denoted L +, is the set.

$$L^+ = \bigcup_{i=1}^{\infty} L^i$$

- If R is a regular expression, then $(R)^*$ is also a regular expression
- Regular Expression over Σ is precisely those obtained recursively by the application of the above rules once or several times.

26. Discuss in detail about the closure properties of regular languages.

Ans: The principal closure properties of regular languages are:

(1) The **union** of two regular languages is regular. If L and M are regular languages, then so is $L \cup M$.
(2) The **intersection** of two regular languages is regular. If L and M are regular languages, then so is $L \cap M$.
(3) The **compliment** of two regular languages is regular. If L is a regular language over alphabet Σ, then Σ^*-L is also regular language.
(4) The **difference** of two regular languages is regular. If L and M are regular languages, then so is L – M.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

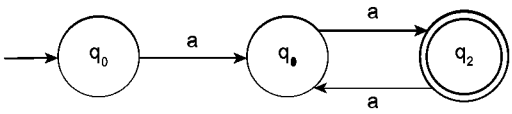
(5) The **reversal** of a regular language is regular. The reversal of a string means that the string is written backward, i.e. reversal of abcde is edcba. The reversal of a language is the language consisting of reversal of all its strings, i.e. if L = {001, 110} then L(R) = {100, 011}.
(6) The **closure** of a regular language is regular. If L is a regular language, then so is L*.
(7) The **concatenation** of regular languages is regular. If L and M are regular languages, then so is L M.
(8) The **homomorphism** of a regular language is regular. A homomorphism is a substitution of strings for symbol. Let the function h be defined by h(0) = a and h(1) = b then h applied to 0011 is simply aabb. If h is a homomorphism on alphabet Σ and a string of symbols w = abcd...z then h (w) = h(a)h(b)h(c) h(d)...h (z) The mathematical definition for homomorphism is h: $\Sigma^* \rightarrow \Gamma^*$ such that " x, y ∈ Σ* and h(x), h(y) ∈ τ*. A homomorphism can also be applied to a language by applying it to each of strings in the language. Let L be a language over alphabet Σ, and h is a homomorphism on Σ, then h(L) = {h(w) | w is in L} The theorem can be stated as " If L is a regular language over alphabet Σ, and h is a homomorphism on Σ, then h(L) is also regular ".
(9) The **inverse homomorphism** of two regular languages is regular. Suppose h be a homomorphism from some alphabet Σ to strings in another alphabet T and L be a language over T then h inverse of L, h′ (L) is set of strings w in Σ* such that h(w) is in L. The theorem states that "If h is a homomorphism from alphabet Σ to alphabet T, and L is regular language on T, then h′(L) is also a regular language.

27. Prove that the following languages are not regular

(a) $\{0^{2n} / n \geq 1\}$

(b) $\{a^m b^n a^{m+n} / m \geq 1$ and $n \geq 1\}$

Ans: a) The given language can be represented as set of all strings whose length is even. We can construct a finite automaton for the given language. Since the language consists of strings {aa, aaaa, aaaaaa, , ...} where the number of a's is even. To accept even strings whose length is greater than or equal to 2 we need three states q_0, q_1 and q_2 where q_0 is initial state and q_2 is final state. Since there exists a finite automaton we can conclude that the given language is Regular.



(b) $\{a^m b^v a^{m+n} / m \geq 1$ and $n \geq 1\}$

Ans: Let us assume that the language is regular. According to Pumping lemma choose the string $Z = a^m b^n a^{m+n}$ where the length is 2(m + n). Let n be the number of states such that $|Z| \geq n$. Now let us represent the string Z as UVW and if the language is regular then for all i, $UV^iW \in L$.

To check whether it is regular or not, it is required to consider three possible cases i) where the string V is formed with only a's or ii) with only b's or iii) with the combination of a's and b's.

Case 1: V is a string such that it contains only a's.

$V = a^x$ Such that $x \geq 1$.

Let i = 0, then the string formed is UW, the string would be of the form $a^{m-x}b^n a^{n+m} \notin L$ as the number of a's and b's is less than the number of a's appearing after b's.

Case 2: V is a string such that it contains only b's.

$V = b^x$ Such that $x \geq 1$.

Let i = 0, then the string formed is UW, the string would be of the form $a^m b^{n-x} a^{m+n} \notin L$ as the number of a's and b's is less than the number of a's appearing after b's

Case 3: V is a string such that it contains combination of a's and b's.

$V = a^x b^x$ Such that $x \geq 1$.

If i = 0, then the string formed is UW, the string would be of the form $a^{m-x}b^{n-x}a^{m+n} \in L$ as the number of a's and b's is not equal to number of a's appearing after b's.

If i = 1, then the string formed is UVW, the string would be of the form $a^{n-x}a^x b^x b^{n-x} a^{m+n} \in L$ as the number of a's and b's is equal to number of a's.

Now if i = 2, then the string formed is UV^2W, the string would be of the form $a^{n-x}a^x b^x a^x b^x b^{n-x} a^{m+n} \notin L$, the language is defined as all a's followed by all b's and then by a's, but the string generated is a's followed by b's then by a's and then by b's followed by a's which is invalid string according to the language.

Since in all the three possible cases there exists value of i such that the string is not in L. Hence the language is not regular.

28. Discuss on equivalence and minimization of automata.

Ans: For any given Deterministic Automation with more number of states we can construct its equivalent DFA with minimum number of states. To minimize the automata we construct equivalence classes using the following procedure.

- Initially construct 0 – equivalence class as $\prod_0 = \{ Q_1^0, Q_2^0 \}$ Where Q_1^0 is set of final states & $Q_2^0 = Q - Q_1^0$ is set of non final states.
- Construct $\prod_{K+1}$ from $\prod_K$ further partitioning as follows:
  - (i) Let Q_1^K be any subset in $\prod_K$. if q_1 & q_2 are in Q_1^K they are (K + 1) equivalent provided $\delta(q_1, a)$ & $\delta(q_2, a)$ are K – equivalent.
  - (ii) Find out whether $\delta(q_1, a)$ and $\delta(q_2, a)$ are in same equivalence class in $\prod_K$ for every $a \in \Sigma$. If so, q_1 and q_2 are (k + 1) equivalence. This way Q_i^k is further divided into (K + 1) equivalence classes. Repeat this for every Q_i^k in $\prod_K$ to get all the elements of $\prod_{K+1}$.
- Construct $\prod_n$ for n = 1, , 2, 3, … until $\prod_n = \prod_{n+1}$.
- For required minimum state automation, states are equivalent classes obtained finally.

29. Explain about Parse trees, for the following grammar

$S \rightarrow aB \mid bA$ $A \rightarrow a \mid aS \mid bAA$ $B \rightarrow b \mid bS \mid aBB$

For the string "aaabbabbba", Find LMD, RMD and Parse tree

Ans: The following is a LMD:

$S \Rightarrow aB \Rightarrow aaBB$

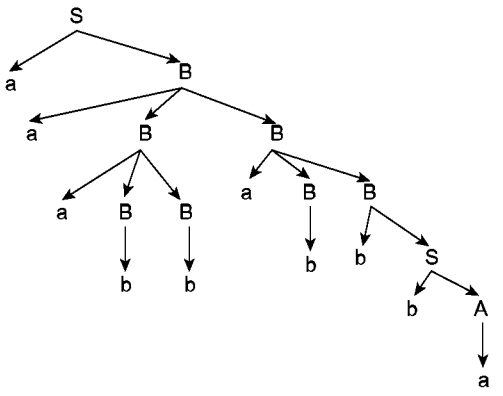
Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$\Rightarrow$ aaaBBB
$\Rightarrow$ aaabBB
$\Rightarrow$ aaabbB
$\Rightarrow$ aaabbaBB
$\Rightarrow$ aaabbabB
$\Rightarrow$ aaabbabbS
$\Rightarrow$ aaabbabbbA
$\Rightarrow$ aaabbabbba

The following is a RMD:

S $\Rightarrow$aB $\Rightarrow$ aaBB
$\Rightarrow$ aaBaBB
$\Rightarrow$ aaBaBbS
$\Rightarrow$ aaBaBbbA
$\Rightarrow$ aaBaBbba
$\Rightarrow$ aaBabbba
$\Rightarrow$ aaaBBabbba
$\Rightarrow$ aaaBbabbba
$\Rightarrow$ aaabbabbba

(c) Derivation tree/parse tree



30. Construct PDA for the language L = {ww^r | w is in (a + b)*}.

Ans: Read the string w and push it on to the stack. After that read each symbol, if it matches with top of the stack pop off the symbol. When input is read completely, then if stack becomes empty, then it is successful.

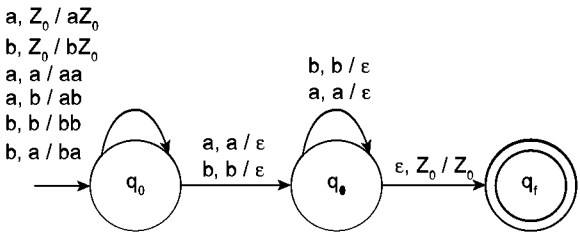
The PDA can be given as follows:

Let q_0 be initial state, q_f be final state and Z_0 be bottom of the stack.

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$
$$\delta(q_0, b, Z_0) = (q_0, bZ_0)$$
$$\delta(q_0, a, a) = (q_0, aa), (q_1, \varepsilon)$$
$$\delta(q_0, b, b) = (q_0, bb), (q_1, \varepsilon)$$
$$\delta(q_0, a, b) = (q_0, ab)$$
$$\delta(q_0, b, a) = (q_0, ba)$$

$$\delta(q_1, a, a) = (q_1, \varepsilon)$$
$$\delta(q_1, b, b) = (q_1, \varepsilon)$$
$$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0)$$

The final PDA is given by $M = \{(q_0, q_1, q_f), (a, b), (a, b, Z_0), \delta, q_0, Z_0, q_f\}$.



31. Explain in detail about equivalence of Pushdown automata and CFG.

Ans: The equivalence of PDA and CFG can be proved by showing the language accepted by PDA is the language generated by the CFG.

Suppose L is a context-free language. Then there is a PDA M such that L = N(M).

We assume that ε is not in L(G) and let G be given as (V, T, P, S) be a context free grammar in Greibach normal form. It is required to construct $M = (\{q\}, T, V, \delta, S,)$ where $\delta(q, a.A)$ contains (q, γ) whenever $A \rightarrow a\gamma$ is in P.

The PDA M simulates leftmost derivations of G. Since G is in Griebach normal form, each sentential form in a leftmost derivation consists of terminals x followed by a string of variables α. M stores the suffix α of the left sentential form on its stack after processing the prefix x. Formally we show that

$S \Rightarrow x\alpha$ by a leftmost derivation if and only if $(\vdash_M^* q, x, S) (q, \varepsilon, \alpha)$

Suppose M is a PDA. Then there is a grammar G such that L(G) = L(M), i.e., L(M) is context-free.

The previous construction, spelled out in full would look messy, but is in fact quite simple. Going in the reverse direction, i.e., converting a PDA to a CFG, is more difficult. The basic idea is to consider any two states p, q of PDA M and think about what strings could be consumed in executing M from p to q. Those strings will be represented by a variable [p, a, q] in G, the grammar we are building. By design, the strings generated by [p, a, q] would be just those substrings consumed by M in going from p to q. Thus S, the start variable, will stand for all strings consumed in going from q0 to an accept state. This is clear enough, but as always for PDA's, we must consider the stack, hence the story will be more involved; for example, we will use funky variables of the form [p, A, q], where A represents the top of the stack.

The construction goes as follows: given PDA $M = \{Q, \Sigma, \tau, \delta, q_0, Z_0, \varnothing\}$ we will construct a grammar G such that L(G) = L(M).

To convert the PDA to CFG we use the following three rules.

R1: The productions for start symbol S are given by

$S \rightarrow [q_0, Z_0, q]$ for each state q in Q.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

R2: For each move that pops a symbol from stack with transition as
$\delta(\mathbf{q}, a, Z_0) = (\mathbf{q1}, \varepsilon)$ induces a production as
$[\mathbf{q}, Z_0, \mathbf{q1}] \rightarrow a$ for $\mathbf{q1}$ in Q.

R3: For each move that does not pop symbol from stack with transition as
$\delta(\mathbf{q}, a, Z_0) = (\mathbf{q1}, Z_1 Z_2 Z_3 Z_{4.....})$ induces a production as
$[\mathbf{q}, Z_0, \mathbf{q^m}] \rightarrow a[\mathbf{q1}, Z_1\ q_2]\ [q_2, Z_2\ q_3]\ [q_3, Z_3\ q_4]\ [q_4, Z_4\ q_5] \ldots [q_{m-1}, Z_m\ \mathbf{q^m}]$ for each $\mathbf{q^m}$ in Q.

After defining all the rules apply simplification of grammar to get reduced grammar.

32. Construct the following grammar in CNF

$A \rightarrow BCD \mid b \qquad B \rightarrow Yc \mid d \qquad C \rightarrow gA \mid c \qquad D \rightarrow dB \mid aY \rightarrow f$

Ans: The grammar is said to be in CNF if all the productions have either two non terminals or a single terminal on the right side of the production.

The first production after converting to CNF the resultant productions are

$$A \rightarrow BX_1 \mid bX_1 \rightarrow CD$$

The second production after converting to CNF the resultant productions are

$$B \rightarrow YX_2 \mid dX_2 \rightarrow c$$

The third production after converting to CNF the resultant productions are

$$C \rightarrow X_3A \mid cX_3 \rightarrow g$$

The fourth production after converting to CNF the resultant productions are

$$D \rightarrow X_4B \mid aX_4 \rightarrow d$$

And the last is in CNF.

The final grammar in CNF is

$$\begin{aligned} A &\rightarrow BX_1 \mid b \\ X_1 &\rightarrow CD \\ B &\rightarrow YX_2 \mid d \\ X_2 &\rightarrow c \\ C &\rightarrow X_3A \mid c \\ X_3 &\rightarrow g \\ D &\rightarrow X_4B \mid a \\ X_4 &\rightarrow d \end{aligned}$$

33. Discuss about programming techniques for turing machines

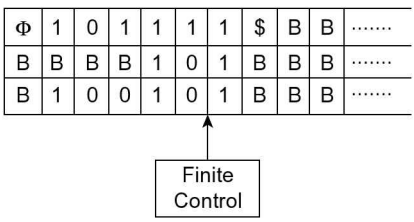
Ans: **Storage in Finite Control**

A Turing machine has a finite number of states in its CPU. However, the number of states is not always small. Like a Pentium chip we can store values in it as long as there is only finite number of states. For example all real computers have registers but there are only a fixed number of them, AND each register can only hold one of a fixed (and finite) number of bits. Similarly we define a state as a pair which stores the details of control and other stores the symbol. To account this modification we can define the

turing machine as $M = (Q, \Sigma, \Gamma, \delta, [q_0, B], B, F)$ where Q is of the form [q, a] where q is a state and $a \in \Sigma$, the transitions are defined as $([Q \times \Sigma], \Gamma) \rightarrow ([Q \times \Sigma], \Gamma, \{R, L\})$. For example the transition $\delta([q, a], b) = ([p, b], c, R)$ indicates that the control is in state q and a is stored in finite control. On the input symbol b it moves to p state and changes the symbol in finite control to b, changes the cell content as c and moves one cell right.

Multi-tape tracks

The tape is imagined as divided into cells where input to be processed is placed. We can further imagine that the tape is divided into k tracks for some finite number k as shown below.



The reading head considers k symbols each belonging to different track in same column and processes it. There are two special symbols Φ and $ used in the first track which indicates the boundary of the input. The other tracks are used to place the intermediate results and the final result of the processing. The blank input is identified as the all B's in all tracks as [B, B, B]. The input at the current position of the reading head is [1, 1, 1].

Checking off symbols

This is one useful trick that can be used to visualize how TM would recognize the languages. This technique uses an extra track which indicates the symbol on the other track is processed. The languages which have repeated strings or some conditions relating to other part of string can be solved with this procedure. Such languages are listed below.

(a) $\{ ww \mid w \text{ in } \Sigma^* \}$
(b) $\{ ww^R \mid w \text{ in } \Sigma^* \}$
(c) $\{ a^i b^i \mid i \geq 1 \}$
(d) $\{ a^i b^j c^k \mid i \neq j \text{ or } j \neq k \}$
(e) $\{ wcw \mid w \text{ in } \Sigma^* \}$

For the languages mentioned above we can use the tape with two tracks where on one track we place the given input and on the other track we place either B or √. If the upper track symbol is B it indicates the symbol on lower track is not considered. If the symbol on upper track is √ it indicates that the symbol on lower track is considered.

Subroutines:

A turing machine can simulate any type of subroutine found in programming languages, including recursive procedures and any of the known parameter passing

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

mechanisms. We can design a TM program to serve as a subroutine, which has a designated initial state and a designated return state which temporarily has to move and which will be used to effect a return to the calling routine. To design a TM to call the subroutine a new set of states are defined which are used to enter the initial state of the subroutine and return from the return state of subroutine. As an example a TM is designed to accept strings with balanced parenthesis.

34. Explain about the closure properties of CFL

Ans: CFLs are closed under substitution, union, concatenation, closure and positive closure, reversal homomorphism, inverse homomorphism

CFLs NOT closed under - intersection, difference, complement

(a) CFLs are closed under union

If L_1 and L_2 are CFLs, then their union $L_1 + L_2$ is a CFL.

Let the grammar CFG_1 defines language L_1. Assume that the nonterminals in CFG_1 are S_1, A_1, B_1, C_1..... Let L_2 is a language defined by CFG2 and its nonterminals are S_2, A_2, B_2, C_2, ...

Now CFG_1 and CFG_2 have nonintersecting sets of nonterminals.

We create a CFG for $L_1 + L_2$ as follows:

Include all of the nonterminals S_1, A_1, B_1, C_1, . . . and S_2, A_2, B_2, C_2, ...

Include all of the productions from CFG_1 and CFG_2.

Create a new nonterminal S and a new production in CFG as

$S \rightarrow S_1 \mid S_2$

(b) CFLs are closed under concatenation

If L_1 and L_2 are CFLs, then L_1L_2 is a CFL.

Let the grammar CFG_1 defines language L_1. Assume that the nonterminals in CFG_1 are S_1, A_1, B_1, C_1 ..... Let L_2 is a language defined by CFG_2 and its nonterminals are S_2, A_2, B_2, C_2, ...

Now CFG_1 and CFG_2 have nonintersecting sets of nonterminals.

We create a CFG for L_1L_2 as follows:

Include all of the nonterminals S_1, A_1, B_1, C_1, ... and S_2, A_2, B_2, C_2, ...

Include all of the productions from CFG_1 and CFG_2.

Create a new nonterminal S and a production

$S \rightarrow S_1S_2$

(c) CFLs are closed under closure

If L is a CFL, then L* is a CFL.

Since L is a CFL, by definition there is some CFG that generates L.

Suppose CFG for L has nonterminals S, A, B, C, ... Change the nonterminal S to

S1.We create a new C CFG for L as follows:

Include all the nonterminals S_1, A, B, C, . . . from the CFG for L.

Include all of the productions from the CFG for L.

Add the new nonterminal S and the new production

$S \rightarrow S_1S \mid \varepsilon$

We can repeat last production

$S \rightarrow S_1S \rightarrow S_1S_1S \rightarrow S_1S_1S_1S \rightarrow S_1S_1S_1S_1S \rightarrow S_1S_1S_1S_1\varepsilon \rightarrow S_1S_1S_1S_1$

Note that any word in L* can be generated by the new CFG. To show that any word
generated by the new CFG is in L*, note that each of the S_1 above generates a word in
L. Also, there is no interaction between the different S_1's.

(d) CFLs are not closed under intersection

Let L_1 and L_2 are two CFLs. Then $L_1 \cap L_2$ maybe a CFL or may not be a CFL. That means it is not closed under intersection.

Proof: We now will give an example showing that the intersection of two CFLs may not be a CFL. To show this, we assume that the language $L_1 = \{a^n b^n a^n: n \geq 1\}$ is a non context free language. L_1 is the set of words with some number of a's, followed by an equal number of b's, and ending with the same number of a's.

Let L_2 be generated by the following CFG:

$$S \rightarrow XY$$
$$X \rightarrow aXb \mid \varepsilon$$
$$Y \rightarrow aY \mid \varepsilon$$

Thus, $L_2 = \{a^n b^n a^m: n, m > = 0\}$, which is the set of words that have a clump of a's, followed by a clump of b's, and ending with another clump of a's, where the number of a's at the beginning is the same as the number of b's in the middle. The number of a's at the end of the word is arbitrary, and does not have to equal the number of a's and b's that come before it.

Let L_3 be generated by the following CFG:

$S \rightarrow WZ$
$W \rightarrow aW \mid \varepsilon$
$Z \rightarrow bZa \mid e$

Thus, $L_3 = \{a^i b^k a^k: i, k > = 0\}$, which is the set of words that have a clump of a's, followed by a clump of b's, and ending with another clump of a's, where the number of b's in the middle is the same as the number of a's at the end. The number of a's at the beginning of the word is arbitrary, and does not have to equal the number of b's and a's that come after it.

Note that $L_2 \cap L_3 = L_1$, where $L_1 = \{a^n b^n a^n: n = 0, 1, 2, \ldots\}$, which is a non context free language.

(e) CFLs are not closed under Complement

If L is a CFL, then $\bar{L}$ may or may not be a CFL.

We first show that the complement of a CFL may be a CFL:

If L is regular, then $\bar{L}$ is also regular. Also both L and $\bar{L}$ are CFLs.

We now show that the complement of a CFL may not be a CFL by contradiction: Suppose that it is always true that if L is a CFL, then $\bar{L}$ is a CFL. Suppose that L_1 and L_2 are CFLs. Then by our assumption, we must have that $\bar{L}_1$ and $\bar{L}_2$ are CFLs. Closure under union implies that $\bar{L}_1 + \bar{L}_2$ is a CFL. Then by our assumption, we must have that compliment of $(L_1 + L_2)$ is a CFL. But we know that compliment of $(\bar{L}_1 + \bar{L}_2) = L_1 \cap L_2$ by DeMorgan's Law. However, we previously showed that the intersection of two CFLs is not always a CFL, which contradicts the previous two steps. So our assumption that CFLs are always closed under complementation must not be true.

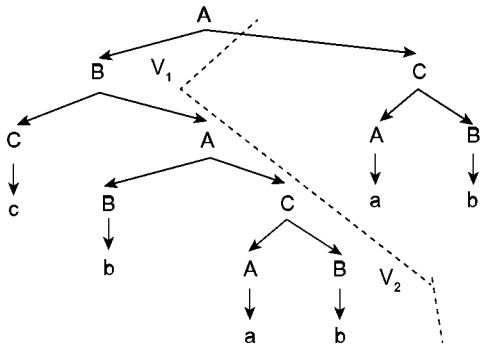
Thus, in general, we cannot say if the complement of a CFL is a CFL.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

35. Explain in detail about Pumping lemma for CFL.

Ans: Let L be any context free language, Then there is a constant n, which depends only upon L, such that there exists a string Z ∈ L and | Z | ≥ n where Z = UVWXY such that

(1) |VX | ≥ 1
(2) |VWX | ≤ n and
(3) For all i ≥ 0 $\mathbf{UV^iWX^iY}$ is in L.

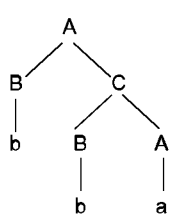


The string Z can be derived by a context free grammar G. The G be a grammar which is in Chomsky Normal Form. The grammar G generates language L. For the string z, we can obtain a parse tree which derives the string Z. Then if the length of the path to Z is less than or equal to i then length of word Z is less than or equal to 2^{i-1}. We can prove this by induction step.

Basis: If $i = 1$
Let G contains the rules S → a where length of the derived string is 1. i.e i = 1. Now according to the rule the word length should be ≤ 2^{i-1}. i.e $2^0 = 1$. Observe that we have a word which is of length 1. Also observe that the grammar G is in Chomsky's normal form. This language is regular since | Z | = | | = 1.

Induction step: Let w be a string which is derived by grammar G.Let k be a variable such that $n = 2^k$, |z| ≥ n then |z| > 2^{k-1} while deriving w string we may get some nonterminals of CFG. G can be repeated for any number of times and will give the string z. If we pump the substrings to w such that the path length of its newly formed string z'(z + pumped string z') is i and the word length of z' is 2^{i-1} then the grammar G deriving z' is called a regular grammar. The necessary condition is that grammar G is in Chomsky's normal form.

Let us consider a grammar
G = ({A, B, C}, {a}, {A→BC |a, B→BA|b, C→BA}, A)



Thus $A \overset{*}{\Rightarrow} bba = w$
i.e path length $i = 3$
$|w| \leq 2^{i-1}$ i.e $3 \leq 2^2$
If we pump s substring into w which satisfies the condition as $i \leq |w| \leq 2^{i-1} \leq n$ the grammar producing string w is a regular grammar.

36. Explain about "A language that is Recursively Enumerable".

Ans: There are *three* possible outcomes of executing a Turing machine over a given input. The Turing machine may

- Halt and accept the input;
- Halt and reject the input; or
- Never halt.

A language is *recursive* if there exists a Turing machine that accepts every string of the language and rejects every string (over the same alphabet) that is not in the language. Note: If a language L is recursive, then its complement L must also be recursive.

A language is *recursively enumerable* if there exists a Turing machine that accepts every string of the language, and does not accept strings that are not in the language. (Strings that are not in the language may be rejected or may cause the Turing machine to go into an infinite loop.)

37. Discuss on undecidable problem about Turing Machine.

Ans: TM = {<M, w>| M is a TM, w is a string, M accepts w}

Assume a TM is decidable which halts and says accept or rejected. Let H be a machine for A Turing Machine <M, w>, H halts and accepts if M accepts w, or rejects if M fails to accept w.
i.e H(<H, w>) = {accept if M accept w.
= {reject if M does not accept w.
Construct new TM, D with H as subroutine. D calls H to find what M does when input to M is its own description <M>. i.e running a machine as its own decription. It is just like a compiler written and compiled in same language.
D gets information and complements the action.
D is defined as <M> where M is a Turing Machine.
(1) Runs H on input <M, <M>>.
(2) If H accepts it rejects, if H rejects it accepts.

$$\text{In summary } D(<M>) = \begin{cases} \text{accept} & \text{if M does not accept } <M> \\ \text{reject} & \text{if M accepts } <M> \end{cases}$$

When we run D with its own description <D> as input? In that case we get

$$D(<D>) = \begin{cases} \text{accept} & \text{if D does not accept } <D> \\ \text{reject} & \text{if D accepts } <D> \end{cases}$$

It is forced to do opposite to what D does. Thus neither TM D nor TM H exists.

38. Explain about the PCP.

Ans: Given an alphabet S, one instance of Post's correspondence problem of size *s* is a finite set of pairs of strings (g_i, h_i) $(i = 1...s\ s> = 1)$ over the alphabet S.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

A solution of length $n >= 1$ to this instance is a sequence $i_1\ i_2 \ldots in$ of selections such that the strings $g_{i1}g_{i2} \ldots gin$ and $h_{i1}h_{i2} \ldots hin$ formed by concatenation are identical.

Width of a PCP instance is the length of the longest string in g_i and h_i ($i = 1, 2, \ldots, s$). *Pair i* is the short name for pair (g_i, h_i), where g_i and h_i are the top string and bottom string of the pair respectively. Mostly, people are interested in *optimal solution*, which has the shortest length over all possible solutions to an instance. The corresponding length is called *optimal length*. We use the word *hard* or *difficult* to describe instances whose optimal lengths are very large. For simplicity, we restrict the alphabet S to {0, 1}, and it is easy to transform other alphabets to their equivalent binary format.

To describe subclasses of Post's Correspondence Problem, we use PCP[s] to represent the set of all PCP instances of size s, and PCP[s, w] the set of all PCP instances of size s and width w.

For convenience, we use a matrix of 2 rows and s columns to represent instances of PCP[s], where string g_i is located at (i, 1) and hi at (i, 2). The following is the matrix representation of the instance {{100, 1}, {0, 100}, {1, 00}} in PCP[3, 3].

i	g_i	h_i
1	100	1
2	0	100
3	1	00

Let's consider the result of selections of pair 1, 3, 1, 1, 3, 2, 2 accordingly. They can be shown in the following table with each selection assigned a different color. After the elimination of blanks and concatenation of strings in the top and bottom separately, it turns to:

1001100100100
1001100100100

Now, the string in the top is identical to the one in the bottom; therefore, these selections form a solution to PCP problem.

39. Describe the following:

(a) Alphabet, String, Language, Empty String.
(b) NFA.
(c) Transition Diagram.
(d) δ in NFA with ε (Epsilon) moves

Ans:

(a) Alphabet, String, Language, Empty String:
Symbol is an abstract entity. It cannot be formerly defined as points in geometry.
Example: Letters, digits or special symbols like $, @, # etc.,
Alphabet Finite collection of symbols denoted by Σ.

String /word Set of symbols from alphabet
Example: 001, 110, 1111 strings from binary alphabet.
a01 is not a string from binary alphabet.
Language: Set of words formed with alphabet
Example: { 0, 1, 00, 01, 10, 11 000, ... } are the strings of the language $(0 + 1)+$

(b) NFA:
Definition 3: Nondeterministic finite automata can be defined as quintuple $M = (Q, \Sigma, \delta, q_0, F)$
Where Q = Non empty finite set of states
Σ = input alphabet
q_0 = initial start state
F = set of final states
δ = transition function that takes two arguments a state and input symbol and returns output as state i.e δ: $Q \times \Sigma \rightarrow 2^Q$

(c) Transition Diagram
A transition graph contains
(a) Set of states as circlesStart state q_0 with arrow → 

Final state by double circle 

(b) A finite set of transitions (edges | labels) that show how to go from some state to other.

(d) δ in NFA with ε (Epsilon) moves:
Epsilon Closure of a state is simply the set of all states we can reach by following the transition function from the given state that are labeled ε. This can be expressed as either $\hat{\varepsilon}$ (q) or ε-closure (q) and contains all the states that are reachable even without any input.

40. Write an algorithm to minimize a given FA

Ans: The minimization algorithm for the given FA based on Π construction is explained here.

(1) Initially construct 0 – equivalence class as $\Pi_0 = \{ Q_1^0, Q_2^0 \}$ Where Q_1^0 is set of final states & $Q_2^0 = Q - Q_1^0$ is set of non final states.
(2) Construct Π_{K+1} from Π_K further partitioning as follows:
(a) Let Q_1^K be any subset in Π_K. if q_1 & q_2 are in Q_1^K they are $(K + 1)$ equivalent provided $\delta(q_1, a)$ & $\delta(q_2, a)$ are K – equivalent.
(b) Find out whether $\delta(q_1, a)$ and $\delta(q_2, a)$ are in same equivalence class in Π_K for every $a \in \Sigma$. If so, q_1 and q_2 are $(k + 1)$ equivalence. This way Q_i^k is further divided into $(K + 1)$ equivalence classes. Repeat this for every Q_i^k in Π_K to get all the elements of Π_{K+1}.
(3) Construct Π_n for $n = 1, 2, 3, \ldots$ until $\Pi_n = \Pi_{n+1}$.
(4) For required minimum state automation, states are equivalent classes obtained finally.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

41. Minimize the following FA

S	0	1
→A0	A0	A3
A1	A2	A5
A2	A3	A4
A3	A0	A5
A4	A0	A6
A5	A1	A4
*A6	A1	A3

Ans: Any two final states are 0 – equivalent and any two non final states are also 0 – equivalent.

$$\Pi_0 (1, 2) = \{\{A6\}, \{A0, A1, A2, A3, A4, A5\}\}$$

	A0	A1	A2	A3	A4	A5
0	2	2	2	2	2	2
1	2	2	2	2	1	2

From the above table we find A0, A1, A2, A3 and A5 are 1-equivalent and hence Π_1 is as follows.

$$\Pi_1 (1, 3, 4) = \{ \{A6\}\{ A0, A1, A2, A3, A5\} \{A4\} \}$$

Using the new classes we find whether they are 2-equivalent.

	A0	A1	A2	A3	A5
0	3	3	3	3	3
1	3	3	4	3	4

From the above table we find A0, A1 and A3 are 2-equivalent and A2 and A5 are 2 – equivalent. hence Π_2 is as follows.

$$\Pi_2 (1, 4, 5, 6) = \{ \{A6\}, \{A4\}, \{ A0, A1, A3\}, \{A2, A5\} \}$$

Using the new classes we find whether they are 3-equivalent.

	A0	A1	A3	A2	A5
0	4	6	4	5	5
1	4	6	6	4	4

From the above table we find A0, A1 and A3 are not 3-equivalent and A2 and A5 are 3 – equivalent. hence Π_3 is as follows.

$$\Pi_3 (1, 4, 6, 7, 8, 9) = \{\{A6\}, \{A4\}, \{A2, A5\}, \{A0\}, \{A1\}, \{A3\}\}$$

Using the new classes we find whether they are 4-equivalent.

	A2	A5
0	9	8
1	4	4

From the above table we find A2 and A5 are 4 – equivalent. hence Π_4 is as follows.

$$\Pi_4 (1, 4, 7, 8, 9, 10, 11) = \{\{A6\}, \{A4\}, \{A0\}, \{A1\}, \{A3\}, \{A2\}, \{A5\}\}$$

Since all the sets contain single element the minimized automata is same as given automata. That is the automata cannot be minimized.

42. Design a Moore Machine to determine the residue mod 4 for each binary string treated as integer.

Ans: The integer number when it is represented as binary the residue mod 4 for each input is given below.

Integer number	Binary representation	output	Expected state
1	1	1	q_1
2	10	2	q_2
3	11	3	q_3
4	100	0	q_0
5	101	1	q_1
6	110	2	q_2
7	111	3	q_3
8	1000	0	q_0
9	1001	1	q_1

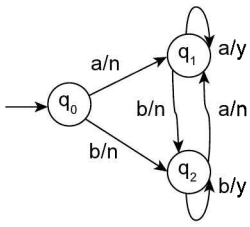
The constructed moore machine is given as transition table as below.

Current State	Next State		Out Put
	a = 0	a = 1	
$\rightarrow q_0$	q_0	q_1	0
q_1	q_2	q_3	1
q_2	q_0	q_1	2
q_3	q_2	q_3	3

43. Design a Mealy machine that uses its state to remember the last symbol read and emits output 'y' whenever current input matches to previous one, and emits n otherwise.

Ans: The example of Mealy machine is designed to generate the output as "***y***" if current input matches with the previous input and output "***n***" Consider the language formed with a's and b's on {a|b}*. For input abb the final output would be "***y***" and for input aba the output would be "***n***". the mealy machine designed is as follows.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir



44. Construct the Left Linear Grammar for the following Regular Expressions:

(a) (11 + 0)* (00 + 1)*

(b) 10 + (0 + 11)0*1

Ans: for the regular expression the left linear grammar is as follows.

(a) (11 + 0)* (00 + 1)*

Corresponding Left Linear grammar is

S→ A00 | A1 | ε,

A→ A00 | A1 | B,

B→ B11 | B0 | ε

(b) 10 + (0 + 11)0*1

Corresponding Left Linear grammar is

S→ 10 | A1

A→ B0 |C

B → 11 | 0

45. Design DPDA for the language L = { $a^n b^{2n}$ / n>0}

Ans: The strings accepted in this language are all a's followed by b's where the number of b's is twice the number of a's. PDA M = ($\{q_0, q_1, q_2\}$, {a, b}, {a, Z_0}, δ, q_0, Z_0, $\{q_f\}$), where δ is defined by following rules:

$$\delta(q_0, a, Z_0) = \{(q_0, aZ_0)\}$$

$$\delta(q_0, a, a) = \{(q_0, aa)\}$$

$$\delta(q_0, b, a) = \{(q_1, a)\}$$

$$\delta(q_1, b, a) = \{(q_2, \varepsilon)\}$$

$$\delta(q_2, b, a) = \{(q_1, a)\}$$

$$\delta(q_2, \varepsilon, Z_0) = \{(q_f, Z_0)\}$$

To show that aabbbb is accepted by the PDA ID's of the system while processing the string are as follows.

$(q_0, aabbbb, Z_0) \Rightarrow (q_0, abbbb, aZ_0) \Rightarrow (q_0, bbbb, aaZ_0) \Rightarrow (q_2, bb, aZ_0) \Rightarrow (q_1, b, aZ_0)$ $\varepsilon, Z_0) \Rightarrow (q_f, \varepsilon, Z_0)$

46. Explain in brief the properties of recursive and recursively enumerable languages

Ans: There are *three* possible outcomes of executing a Turing machine over a given input. The Turing machine may

- Halt and accept the input;
- Halt and reject the input; or
- Never halt.

A language is *recursive* if there exists a Turing machine that accepts every string of the language and rejects every string (over the same alphabet) that is not in the language. Note: If a language L is recursive, then its complement L must also be recursive.

A language is *recursively enumerable* if there exists a Turing machine that accepts every string of the language, and does not accept strings that are not in the language. (Strings that are not in the language may be rejected or may cause the Turing machine to go into an infinite loop.)

47. Prove that PCP is undecidable

Ans: Given an alphabet S, one instance of Post's correspondence problem of size *s* is a finite set of pairs of strings (g_i, h_i) $(i = 1...s\ s> = 1)$ over the alphabet S. A solution of length $n > = 1$ to this instance is a sequence $i_1\ i_2 \ldots in$ of selections such that the strings $gi1gi2 \ldots gin$ and $h_{i1}h_{i2} \ldots hin$ formed by concatenation are identical.

Width of a PCP instance is the length of the longest string in g_i and h_i $(i = 1, 2, ..., s)$. *Pair i* is the short name for pair (g_i, h_i), where *gi* and h_i are the top string and bottom string of the pair respectively. Mostly, people are interested in *optimal solution*, which has the shortest length over all possible solutions to an instance. The corresponding length is called *optimal length*. We use the word *hard* or *difficult* to describe instances whose optimal lengths are very large. For simplicity, we restrict the alphabet S to {0, 1}, and it is easy to transform other alphabets to their equivalent binary format.

To describe subclasses of Post's Correspondence Problem, we use PCP[*s*] to represent the set of all PCP instances of size *s*, and PCP[*s*, *w*] the set of all PCP instances of size *s* and width *w*.

For convenience, we use a matrix of 2 rows and *s* columns to represent instances of PCP[*s*], where string *gi* is located at (*i*, 1) and h_i at (*i*, 2). The following is the matrix representation of the instance {{100, 1}, {0, 100}, {1, 00}} in PCP[3, 3].

i	g_i	h_i
1	100	1
2	0	100
3	1	00

Let's consider the result of selections of pair 1, 3, 1, 1, 3, 2, 2 accordingly. They can be shown in the following table with each selection assigned a different color. After the elimination of blanks and concatenation of strings in the top and bottom separately, it turns to:

1001100100100
1001100100100

Now, the string in the top is identical to the one in the bottom; therefore, these selections form a solution to PCP problem.

48. Design Turing Machine over Σ_ = {1} to accept the language L = {1^m / m is odd}

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

49. Write about:

a) Multi tape Turing Machine b) NP Hard and NP Complete problem

***Ans:* (a) Multitape Turing machine**

It is a kind of Turing machines that has one finite control and more than one tapes each with its own read-write head. It is denoted by a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$. Its transition function is a partial function

$$\delta : QX(\Gamma \cup \{B\})^n \rightarrow (Q \cup \{h\})X(\Gamma \cup \{B\})^n X\{R, L, S\}^n$$

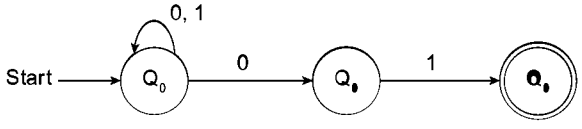
A configuration for this kind of Turing machine must show the current state the machine is in and the state of each tape. It can be proved that any language accepted by an n-tape Turing machine can be accepted by a one tape Turing machine and that any function computed by an n-tape Turing machine can be computed by a one tape Turing machine. Since the converses are obviously true, one can say that one tape Turing machines are as powerful as n-tape Turing machines.

(b) NP Hard and NP Complete Problems

The subject of *computational complexity theory* is focused on classifying problems by how hard they are. There are many different classifications depending the time taken by the problem. The following are the types of classification.

- P. Problems are those that can be solved by a Turing Machine (deterministic) in polynomial time. ("P" stands for polynomial). P problems are class of problems which can be solved efficiently.
- NP. Problems are those that can be solved by nondeterministic Turing machine in polynomial time. A problem is in NP if you can quickly (in polynomial time) test whether a solution is correct (without worrying about how hard it might be to find the solution). NP problems are class of problems which cannot be solved efficiently. NP does not stand for "non-polynomial". There are many complexity classes that are much harder than NP.
- Undecidable. For some problems, we can prove that there is no algorithm that always solves them, no matter how much time or space is allowed. One very uninformative proof of this is based on the fact that there are as many problems as there real numbers, and only as many programs as there are integers, so there are not enough programs to solve all the problems. But we can also define explicit and useful problems which can't be solved.

50. Explain the steps in conversion of NFA to DEA convert the following NFA to DFA.



Ans:

$Q = 2^3 = 8$ states = all subsets of q_0, q_1, q_2

$= \{\varnothing, [q_0], [q_1], [q_2], [q_0, q_1], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$

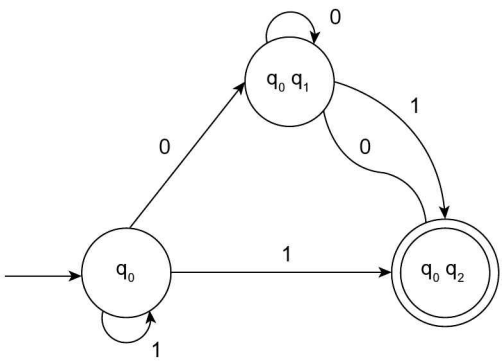
$\Sigma = 0, 1$

$q_0 = [q_0]$

$F = \{[q_2], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$
δ is given by $\delta_D([q_1\ q_2], a) = \delta_n(q_1, a) \cup \delta_n(q_2, a)$
when δ_n is transition function of NFA

	0	1
$\varnothing$	$\varnothing$	$\varnothing$
$\rightarrow[q_0]$	$[q_0, q_1]$	$[q_0]$
$[q_1]$	$\varnothing$	$[q_2]$
$[q_2]$	$\varnothing$	$\varnothing$
$[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_2]$
$*[q_0, q_2]$	$[q_0, q_1]$	$[q_0]$
$[q_1, q_2]$	$\varnothing$	$[q_2]$
$[q_0, q_1, q_2]$	$[q_0, q_1]$	$[q_0, q_2]$

The states $[\varnothing]$, $[q_1]$, $[q_2]$, $[q_1, q_2]$ and $[q_0, q_1, q_2]$ are not reachable from start stated hence cannot define any strings. So they can be thrown away. Hence the DFA can be simplified as follows in Fig 2.28:



To get this **Simplified DFA** construct the states of DFA as follows

(i) Start with initial state. Do not add all subsets of states as there may be unnecessary states.

(ii) After finding the transition on this initial state, include only the resultant states into the list until no new state is added to the list. For example if $\delta(q_0, a) = \{q_0, q_1\}$ say then add this as new state in DFA. and find transition from this state on input symbol.

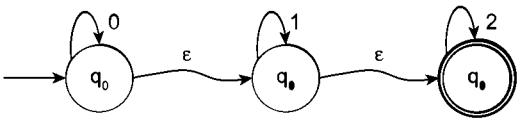
(iii) Declare the states as final if it has at least one final state of NFA.

51. Prove that, if L is accepted by an NFA with ∅ transactions, then L is accepted by NFA without ∅ transactions.

Ans: For each state compute ε-closure(q) on each input symbol $a \in \Sigma$. If the ε-closure of a state contains a final state then make the state as final.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Let the following be NFA with ε – transitions



The transition table is

	a = 0	a = 1	a = 2	a = ε
q_0	q_0	∅	∅	q_1
q_1	∅	q_1	∅	q_2
*q_2	∅	∅	q_2	∅

Step 1: Find ε-closure of each state.
ε-closure (q_0) = {q_0, q_1, q_2}
ε-closure (q_1) = {q_1, q_2}
ε-closure (q_2) = {q_2}

Step 2: Find the transition on each state for each element.

$\hat{\delta}$ (q_0, 0) = ε-closure (δ(ε-closure (q_0), 0))
= ε-closure (δ({q_0, q_1, q_2}, 0))
= ε-closure ({q_0}, ∪{∅}, ∪{∅})
= {q_0, q_1, q_2}

$\hat{\delta}$ (q_0, 1) = ε-closure (δ(ε-closure (q_0), 1))
= ε-closure (δ({q_0, q_1, q_2}, 1))
= ε-closure ({∅}, ∪{q_1}, ∪{∅})
= {q_1, q_2}

$\hat{\delta}$ (q_0, 2) = ε-closure (δ(ε-closure (q_0), 2))
= ε-closure (δ({q_0, q_1, q_2}, 2))
= ε-closure ({∅}, ∪{∅}, ∪{q_2})
= {q_2}

$\hat{\delta}$ (q_1, 0) = ε-closure (δ(ε-closure (q_1), 0))
= ε-closure (δ({q_1, q_2}, 0))
= ε-closure ({∅})
= {∅}

$\hat{\delta}$ (q_1, 1) = ε-closure (δ(ε-closure (q_1), 1))
= ε-closure (δ({q_1, q_2}, 1))
= ε-closure ({q_1}, ∪{∅})
= {q_1, q_2}

$\hat{\delta}$ (q_1, 2) = ε-closure (δ(ε-closure (q_1), 2))
= ε-closure (δ({q_1, q_2}, 2))
= ε-closure ({∅}, ∪{q_2})
= {q_2}

$\hat{\delta}(q_2, 0)$ = ε-closure (δ(ε-closure (q_2), 0))
= ε-closure (δ({q_2}, 0))
= ε-closure ({∅})
= {∅}

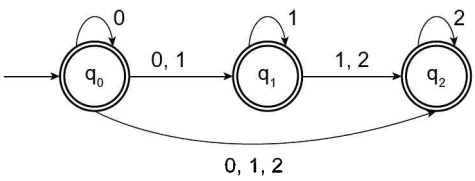
$\hat{\delta}(q_2, 1)$ = ε-closure (δ(ε-closure (q_2), 1))
= ε-closure (δ({q_2}, 1))
= ε-closure ({∅})
= {∅}

$\hat{\delta}(q_2, 2)$ = ε-closure (δ(ε-closure (q_2), 2))
= ε-closure (δ({q_2}, 2))
= ε-closure ({q_2})
= {q_2}

NFA without – ε transitions is

	a = 0	a = 1	a = 2
→*q_0	{q_0, q_1, q_2}	{q_1, q_2}	{q_2}
*q_1	∅	{q_1, q_2}	{q_2}
*q_2	∅	∅	{q_2}

Transition diagram of NFA without ε-transitions



52. Prove the equivalence of NFA and DFA using subset construction.

Ans: Let $M_N = (Q_N, \Sigma_N, \delta_n, q_{ON}, F_N)$ be given NFA to construct equivalent DFA M_D define M_D as follows.

(i) $Q_D = 2^{Q_N}$. If NFA has n states. DFA at most can have 2^n states.
(ii) $\Sigma_n = \Sigma_D$
(iii) $[q_0] = \{q_0\}$
(iv) F_D = Set of all states of Q_D that contains at least one final states of F_N.
(v) $\delta_D((q_1, q_2, q_3), a) = \delta_n(q_1, a) \cup \delta_n(q_2, a) \cup \delta_n(q_3, a)$
$= \{P_1, P_2, P_3\}$ say

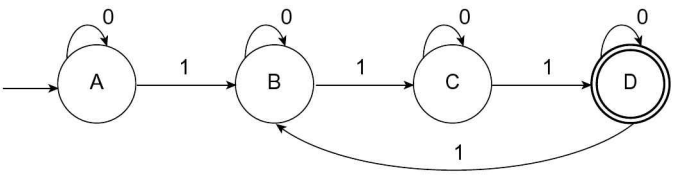
Add state [P_1, P_2, P_3] to Q_D if it is not there.

53. Give Deterministic finite automata accepting the following language over the alphapet.

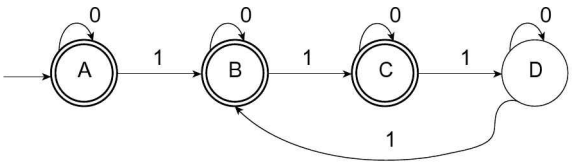
(1) Number of 1's is a multiples of 3

Ans: To construct DFA to satisfy the given conditions we need to have the number of 1's as multiple of three.

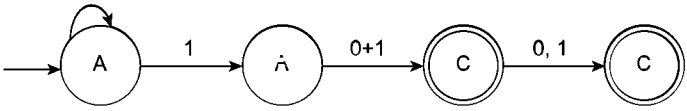
Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir



(2) Number of 1's is not a multiples of 3



54. Convert the following NFA into a regular expression.



Ans: the strings generated are the third symbol from the right end is 1. The regular expression is $(1 + 0)^*1(1 + 0)(1 + 0)$

55. Discuss the closure properties of regular languages.

Ans: The principal closure properties of regular languages are:

1. The **union** of two regular languages is regular. If L and M are regular languages, then so is $L \cup M$.
2. The **intersection** of two regular languages is regular. If L and M are regular languages, then so is $L \cap M$.
3. The **compliment** of two regular languages is regular. If L is a regular language over alphabet Σ, then Σ^*-L is also regular language.
4. The **difference** of two regular languages is regular. If L and M are regular languages, then so is L - M.
5. The **reversal** of a regular language is regular. The reversal of a string means that the string is written backward, i.e. reversal of abcde is edcba. The reversal of a language is the language consisting of reversal of all its strings, i.e. if L = {001, 110} then L(R) = {100, 011}.
6. The **closure** of a regular language is regular. If L is a regular language, then so is L*.
7. The **concatenation** of regular languages is regular.If L and M are regular languages, then so is L M.
8. The **homomorphism** of a regular language is regular.A homomorphism is a substitution of strings for symbol. Let the function h be defined by h(0) = a and h(1) = b then h applied to 0011 is simply aabb. If h is a homomorphism on alphabet Σ and a string of symbols w = abcd...z then h (w) = h (a) h (b) h(c)

h (d)...h (z) The mathematical definition for homomorphism is h: $\Sigma^* \rightarrow \Gamma^*$ such that $\forall$ x, y $\in \Sigma^*$ and h(x), h(y) $\in \tau^*$. A homomorphism can also be applied to a language by applying it to each of strings in the language. Let L be a language over alphabet Σ, and h is a homomorphism on Σ, then h (L) = {h(w) | w is in L} The theorem can be stated as " If L is a regular language over alphabet Σ, and h is a homomorphism on Σ, then h(L) is also regular ".

9. The **inverse homomorphism** of two regular languages is regular. Suppose h be a homomorphism from some alphabet Σ to strings in another alphabet T and L be a language over T then h inverse of L, h′ (L) is set of strings w in Σ^* such that h(w) is in L. The theorem states that " If h is a homomorphism from alphabet Σ to alphabet T, and L is regular language on T, then h′(L) is also a regular language".

56. Discuss the application of regular languages.

Ans: The regular languages are useful in defining the tokens in a language which can be recognized during Lexical analyzers and by Text editors.

Lexical analyzers: The tokens of the programming language can be expressed using regular expressions. The lexical analyzer scans the input program and separates the tokens. For eg identifier can be expressed as a regular expression as: (letter)(letter + digit)* If anything in the source language matches with this reg exp then it is recognized as an identifier. The letter is{A, B, C, ...Z, a, b, c....z} and digit is {0, 1, ...9}. Thus reg exp identifies token in a language.

Text editors: These are programs used for processing the text. For example UNIX text editors uses the reg exp for substituting the strings such as: S/bbb*/b/
Gives the substitute a single blank for the first string of two or more blanks in a given line. In UNIX text editors any reg exp is converted to an NFA with $\in$ –transitions, this NFA can be then simulated directly.

57. Using pumping lemma for regular sets prove that the language

$$L = \{0^m 1^n 0^{m-n} \mid m \geq 1 \text{ and } n \geq 1\} \text{ is not regular.}$$

Let us assume that the language is regular. According to Pumping lemma choose the string $Z = 0^m 1^n 0^{m+n}$ where the length is 2(m + n). Let n be the number of states such that $|Z| \geq n$. Now let us represent the string Z as UVW and if the language is regular then for all i, $UV^iW \in L$.

To check whether it is regular or not, it is required to consider three possible cases i) where the string V is formed with only 0's or ii) with only 1's or iii) with the combination of 0's and 1's.

Case 1: V is a string such that it contains only 0's.

$V = 0^x$ Such that $x \geq 1$.

Let i = = 0, then the string formed is UW, the string would be of the form $0^{m-x} 1^n 0^{m+n} \notin L$ as the number of 0's is less than the number of 1's and 0's after it.

Case 2: V is a string such that it contains only 1's.

$V = 1^x$ Such that $x \geq 1$.

Let i = 0, then the string formed is UW, the string would be of the form $0^m 1^{n-x} 0^{m+n} \notin L$ as the number of 1's is less than the number of 0's.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Case 3: V is a string such that it contains combination of 0's and 1's.

$V = 0^x1^x$ Such that $x \geq 1$.

If $i = 0$, then the string formed is UW, the string would be of the form $0^{m-x}\,1^{n-x}\,0^m + {}^n \notin L$ as the number of 0's and number of 1's is not equal to number of 0's.

Since in all the three possible cases there exists value of i such that the string is not in L. Hence the language is not regular.

58. Convert the following grammar into GNF.

S →XY1/0
X →00X/Y
Y→1X1

Ans: Step 1: Since there are unit productions first eliminate it and the resulting grammar is

S → XY1/0
X → 00X/1X1
Y → 1X1

Step 2: Converting the grammar to CNF we get

S → XP/0
P → YB
X → AQ/BR
Q → AX
R → XB
Y → BR
A → 0
B → 1

Step 3: Converting the grammar to GNF we get

S → 0QP | 1RP |0
P → 1RB
X → 0Q/1R
Q → 0X
R → 0QB | 1RB
Y → 1R
A → 0
B → 1

59. Give formal pushdown automata that accepts $\{wcw^R \mid w \text{ in } (0 + 1)^*\}$ by empty stack.

Ans: Read the string w and push it on to the stack till it encounters 'c'. After that read each symbol, if it matches with top of the stack pop off the symbol. When input is read completely, then if it enters the final state, then it is successful.

The PDA can be given as follows:

Let q_0 be initial state, q_f be final state and Z_0 be bottom of the stack.

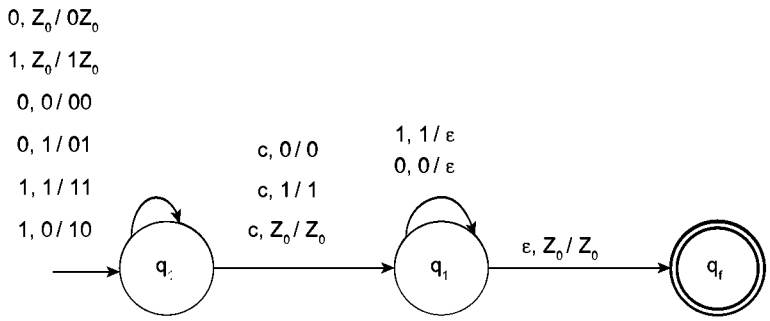
$$\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 0, 1) = (q_0, 01)$$

$$\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$$

$$\delta(q_0, 1, 0) = (q_0, 10)$$
$$\delta(q_0, 1, 1) = (q_0, 11)$$
$$\delta(q_0, c, Z_0) = (q_1, Z_0)$$
$$\delta(q_0, c, 0) = (q_1, 0)$$
$$\delta(q_0, c, 1) = (q_1, 1)$$
$$\delta(q_1, 0, 0) = (q_1, \varepsilon)$$
$$\delta(q_1, 1, 1) = (q_1, \varepsilon)$$
$$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0)$$



The final PDA is given by $M = \{(q_0, q_1, q_f), (a, b, c), (a, b, Z_0), \delta, q_0, Z_0, \{q_f\}\}$.

60. Show that the following grammars are ambiguous.

$\{S \rightarrow aSbS / bSaS / \varepsilon\}$ and

Ans: The string abab can be derived using LMD as follows

1. S => aSbS =>abS => abaSbS => ababS => abab
2. S => aSbS => abSaSbS => abaSbS => ababS => abab

Since there are two possible ways to derive the string it is ambiguous for w

$\{S \rightarrow AB / aaB, A \rightarrow a/Aa, B \rightarrow b\}$

Ans: The string aab can be derived using LMD as follows

1. S => AB =>AaB => aaB => aab
2. S => aaB => aab

Since there are two possible ways to derive the string it is ambiguous for w

(ii) Prove the equivalence of PDA and CFL.

Suppose L is a context-free language. Then there is a PDA M such that L = N(M).

Proof: The basic idea in the construction is to build M so that it simulates the leftmost derivation of strings using G. The machine we construct uses the terminals and non-terminals of the grammar as stack symbols. What we conceptually want to do is to use the stack to hold the sentential form that evolves during a derivation. At each step, the topmost variable in the stack will get replaced by the RHS of some grammar rule. Of course, there are several problems with implementing this concept. For one,

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

the PDA can only access the top of its stack it can't find a variable below the top. For another, even if the PDA could find such a variable, it could not fit the RHS into a single stack slot. But these are not insurmountable. We simply have to arrange things so that the PDA always has the leftmost variable of the sentential form on top of the stack. If that can be set up, the PDA can use the technique of using extra states to push multiple symbols 'all at once'.

The other consideration is that we are constructing a PDA, so it needs to consume the input string and give a verdict. This fits in nicely with our other requirements. In brief, the PDA will use ε-transitions to push the RHS of rules into the stack, and will use 'normal' transitions to consume input. In consuming input, we will be able to remove non-variables from the top of the stack, always guaranteeing that a variable is at the top of the stack.

We assume that ε is not in L(G) and let G be given as (V, T, P, S) be a context free grammar in Greibach normal form. It is required to construct $M = (\{q\}, T, V, \delta, S, \phi)$ where $\delta(q, a.A)$ contains (q, γ) whenever $A \rightarrow a\gamma$ is in P.

The PDA M simulates leftmost derivations of G. Since G is in Griebach normal form, each sentential form in a leftmost derivation consists of terminals x followed by a string of variables α. M stores the suffix α of the left sentential form on its stack after processing the prefix x. Formally we show that

$S \Rightarrow x\alpha$ by a leftmost derivation if and only if $(q, x, S) \overset{*}{\underset{M}{\vdash}} (q, \varepsilon, \alpha)$

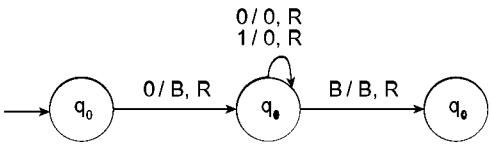
61. Explain Turing machine as a computer of integer functions with an example.

Ans: A TM M computes a function f if, when given input w in the domain of f, the machine halts in its accept state with f(w) written (leftmost) on the tape. To use Turing Machine as a computational machine it is required to place the integer numbers as 0^m. Suppose it is required to add two numbers, ie. $f(m, n) = m + n$ then the numbers m and n are to be placed on the tape as $0^m 1 0^n$ where 1 is separator for the numbers m and n. Once processing is completed and the turing machine halts then the tape would have the contents as $0^{(m+n)}$ which is the required result of computation.

Example: The addition of these numbers using simple logic is explained as below. The numbers are placed as $B0^2 1 0^3 B$. After processing the tape content would be $B0^5 B$. The simple logic that can be used is replace the occurrence of 1 by 0 and move to right end and replaces the last 0 to B so that it is in required form as $B0^5 B$. Sequence of steps is given for understanding.

1. In initial state 0's is replaced by B and change to new state.
   $\delta(q0, 0) = (q1, B, R)$
2. If this state travel right until it encounters 1. replace this 1 by 0 be in the same state and on seeing B halt.
   $\delta(q1, 0) = (q1, 0, R)$
   $\delta(q1, 1) = (q1, 0, R)$
   $\delta(q1, B) = (qA, B, R)$

B•001000B ⊢ B•01000B ⊢ B0•1000B ⊢ B00•000B ⊢ B000•00B ⊢ B0000•0B ⊢ B00000•B ⊢ B00000B



	0	1	B
q_0	(q_1, B, R)	---	---
q_1	$(q_1, 0, R)$	$(q_1, 0, R)$	(q_A, B, R)
q_A	---	---	---

62. Write the procedure to remove ∈ productions from the given grammar.

Ans: If some CFL contains the word ε, then the CFG must have a ε-production. However, if a CFG has a ε-production, then the CFL does not necessarily contain ε. e.g.

$S \rightarrow aX$

$X \rightarrow \varepsilon$

which defines the CFL {a}.

Nullable variable: In a given CFG, a nonterminal X is nullable if

1. There is a production $X \rightarrow \varepsilon$
2. There is a derivation that starts at X and leads to ε:

   $X => \ldots => \varepsilon$ i.e., $X \overset{*}{\Rightarrow} \varepsilon$.

For any language L, define the language L_0 as follows:

1. if $\varepsilon \notin L$, then L_0 is the entire language L, i.e., $L_0 = L$.
2. if $\varepsilon \in L$, then L_0 is the language L - {ε}, so L_0 is all words in L except ε.

   ⇒Note: If L is a CFL generated by a CFG G1 that includes ε-productions, then there is another CFG G2 with no ε-productions that generates L_0.

Procedure for eliminating ε productions:

(a) Construct V_n set of all nullable variables

(b) For each production $B \rightarrow A$, if A is nullable variable, replace nullable variable by ε and add with all possible combinations on the RHS.

(c) Do not add the production

$A \rightarrow \varepsilon$

63. Write short notes on the following:

(i) Two-way infinitentape TM.

Ans: In all our formulations we specified that the tape had an left end and stretched infinitely far to the right. Relaxing this stipulation to allow the tape to stretch infinitely far too right and left results in a new formulation of Turing machines equivalent to the original. That is for any Turing machine using a two way tape there is a Turing machine with a one-way infinite tape with the same input-output behavior, and vice versa.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

One can simulate a Turing machine with two way infinite tape on a Turing machine with one way infinite tape. Let the two way infinite tape have the contents as shown below.

A_{-4}	A_{-3}	A_{-2}	A_{-1}	A_0	A_1	A_2	A_3	A_4	....

Let the reading head be left of A_0. This can be simulated by a Turing machine with one way infinite tape with two tracks having the contents placed such that the contents of the tape right of A_0 are placed on the upper track and the contents of the tape to the left of A_0 are placed on the lower track in reverse order. The left end cell contains on upper track and a special symbol Φ on the lower track as shown below.

A_0	A_1	A_2	A_3	A_4	....
Φ	A_{-1}	A_{-2}	A_{-3}	A_{-4}	....

The moves are simulated such that when the reading head is to the right of A_0 the moves are implemented as they are reading symbols from upper track. If the reading head is to the left of A_0 then symbols are read from lower head but the direction of reading head would be in opposite direction to the direction in which it moves on two way infinite tapes. That is if it moves left then on one way infinite tape it moves right or vice versa.

Many other variations of Turing machine are possible. However, it has been shown that none of them exceed the capability of basic deterministic Turing machine as far as accepting languages is concerned. In fact the Church's thesis conjectures that any so called computation done by humans or computers can be performed by a basic deterministic Turing machine.

(ii) Multiple tracks TM.

Ans: The tape is imagined as divided into cells where input to be processed is placed. We can further imagine that the tape is divided into k tracks for some finite number k as shown below.

	1	0	1	1	1	1	$	B	B	....
B	B	B	B	1	0	1	B	B	B	....
B	1	0	0	1	0	1	B	B	B	.....



The reading head considers k symbols each belonging to different track in same column and processes it. There are two special symbols ϕ and $ used in the first track which indicates the boundary of the input. The other tracks are used to place the intermediate results and the final result of the processing. The blank input is identified as the all B's in all tracks as [B, B, B]. The input at the current position of the reading head is [1, 1, 1].

Example: To design a TM to identify the number as prime or not it is required to find whether the number has factors other than itself.

1. Let us place the given number on first track in binary form bounded by Φ and $. For example 47 is represented as Φ101111$.

2. On the second track write 2 in binary form as 10.W
3. Copy the number on first track to third track.
4. Perform repeated subtraction of number on third track with number on second track until the number on third track is either 0 or less than number on second track.
5. If the number on third track is zero and number on second is not equal to number on first track then the number on first track is not prime, otherwise not prime.
6. If the number on third track in nonzero and increase number on second track by one.
7. Repeat the steps 4-6 until the number on second is equal to number on first.

64. Write the classes and definition of NP problems.

Ans: The subject of *computational complexity theory* is focused on classifying problems by how hard they are. There are many different classifications depending the time taken by the problem. The following are the types of classification.

(a) **P.** Problems are those that can be solved by a Turing Machine (deterministic) in polynomial time. ("P" stands for polynomial). P problems are class of problems which can be solved efficiently.

(b) **NP.** Problems are those that can be solved by nondeterministic Turing machine in polynomial time. A problem is in NP if you can quickly (in polynomial time) test whether a solution is correct (without worrying about how hard it might be to find the solution). NP problems are class of problems which cannot be solved efficiently.*NP does not stand for "non-polynomial"*. There are many complexity classes that are much harder than NP.

(c) **Undecidable.** For some problems, we can prove that there is no algorithm that always solves them, no matter how much time or space is allowed. One very uninformative proof of this is based on the fact that there are as many problems as there real numbers, and only as many programs as there are integers, so there are not enough programs to solve all the problems. But we can also define explicit and useful problems which can't be solved.

65. Prove that if a language is recursive if and only if it and its complement are both recursively enumerable.

Ans: A language L is recursively enumerable if there is a TM that accepts L and recursive if there is a TM that recognizes L. Thus r.e language is Turing acceptable and recursive language is Turing decidable languages. No, the language accepted by non-deterministic Turing machine is same as recursively enumerable language.

Recursive languages	Recursively enumerable languages
1. A language is said to be recursive if and only if there exists a membership algorithm for it.	1. A language is said to be r.e if there exists a TM that accepts it.
2. A language L is recursive iff there I (Turing decidable languages). TMs that decide languages are algorithms. s a TM that decides L.	2. L is recursively enumerable iff there is a TM that semi-decides L. (Turing acceptable languages). TMs that semi-decides languages are not algorithms.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

(ii) Example about undecidability of PCP.

Given an alphabet S, one instance of Post's correspondence problem of size s is a finite set of pairs of strings (g_i, h_i) $(i = 1...s\ s> = 1)$ over the alphabet S. A solution of length $n> = 1$ to this instance is a sequence $i_1\ i_2 \ldots i_n$ of selections such that the strings $g_{i1}g_{i2} \ldots g_{in}$ and $h_{i1}h_{i2} \ldots h_{in}$ formed by concatenation are identical.

Width of a PCP instance is the length of the longest string in g_i and h_i $(i = 1, 2, ..., s)$. *Pair i* is the short name for pair (g_i, h_i), where g_i and h_i are the top string and bottom string of the pair respectively. Mostly, people are interested in *optimal solution*, which has the shortest length over all possible solutions to an instance. The corresponding length is called *optimal length*. We use the word *hard* or *difficult* to describe instances whose optimal lengths are very large. For simplicity, we restrict the alphabet S to {0, 1}, and it is easy to transform other alphabets to their equivalent binary format.

To describe subclasses of Post's Correspondence Problem, we use PCP[s] to represent the set of all PCP instances of size s, and PCP[s, w] the set of all PCP instances of size s and wisdth w.

For convenience, we use a matrix of 2 rows and s columns to represent instances of PCP[s], where string gi is located at $(i, 1)$ and hi at $(i, 2)$. The following is the matrix representation of the instance {{100, 1}, {0, 100}, {1, 00}} in PCP[3, 3].

i	g_i	h_i
1	100	1
2	0	100
3	1	00

Let's consider the result of selections of pair 1, 3, 1, 1, 3, 2, 2 accordingly. They can be shown in the following table with each selection assigned a different color:

Solution sequence	1	3	1	1	3	2	2
String G	100	1	100	100	1	0	0
String H	1	00	1	1	00	100	100

After the elimination of blanks and concatenation of strings in the top and bottom separately, it turns to:

1001100100100
1001100100100

Now, the string in the top is identical to the one in the bottom; therefore, these selections form a solution to PCP problem.

66. Describe the fundamental differences in the rules for forming DFA and NFA. Are these differences important in terms of the languages they can recognize? Give a reason for your answer. (8)

Ans: **Properties of Transition Function (δ)**

(i) $\delta(q, \varepsilon) = q$

This means the state of the system can be changed only by an input symbol else remains in original state.

(ii) For all strings w and input symbol a
$\delta(q, aw) = \delta(\delta(q, a), w)$
similarly $\delta(q, wa) = \delta(\delta(q, w), a)$

(iii) The transition function δ can be extended to $\bar{\delta}$ (or) $\hat{\delta}$ that operates on states and strings (as opposed to states and symbols)

Basis: $\bar{\delta}(q, \varepsilon) = q$

Induction: $\bar{\delta}(q, xa) = \delta\left(\bar{\delta}(q, x), a\right)$

Language of a DFA

A string x is said to be accepted by DFAM $= (Q, \Sigma, S, F, \delta)$, if $\delta(q_0, x) = P$ for some P in F.

Method: A finite automata accepts a string $w = a_1 a_2 \ldots a_n$ if there is a path in the transition diagram which begins at a start state ends at an accepting state with the sequence of labels $a_1 a_2 \ldots a_n$

- The Language accepted by finite automata (A) is
  $L(A) = \{w : \bar{\delta}(q_0, w) \in F\}$ where F is a final state.
- The language accepted by finite automata's are called "*regular language*"

Extended Transition Function $(\bar{\delta})$

Basis: $\bar{\delta}(q, \varepsilon) = \{q\}$

Induction:

$$\bar{\delta}(q, wa) = \bigcup_{P \in \bar{\delta}(q, w)} \delta(P, a) \text{ for each } w \in \Sigma^*, a \in \Sigma \text{ and } P \in \bar{\delta}(q, w)$$

Language of a NFA

Language accepted by NFA is $L(A) = \left\{w : \bar{\delta}(q_0, w) \cap F \neq \phi\right\}$

Transition function and language of ε–NFA

The transition function $\hat{\delta}$ is defined as:

(i) $\hat{\delta}(q, \varepsilon) = \varepsilon - \text{CLOSURE}(q)$

(ii) For w in Σ^*, and a in Σ
$\hat{\delta}(q, wa) = \varepsilon - \text{CLOSURE}(P)$
Where $P = \{p \mid \text{for some } r \text{ in } \hat{\delta}(q, w), p \text{ in } \delta(r, a)\}$

(iii) $\hat{\delta}(R, a) = \bigcup_{q \text{ in } R} \delta(q, a)$

(iv) $\hat{\delta}(R, w) = \bigcup_{q \text{ in } R} \hat{\delta}(q, w)$

The language accepted by NFA with ε – move is defined as:

$L(M) = \left\{w \middle| \hat{\delta}(q_0, w) \text{ contains a state in } F\right\}$

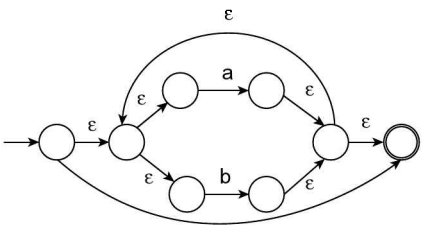
In each model the transition rules are defined as per the model, but the language acceptance will be same for all the models.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

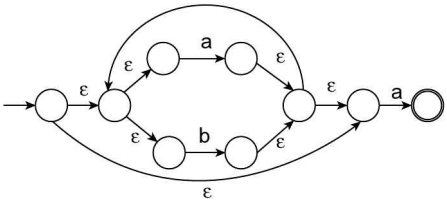
67. Construct an NFA for the following regular expression:

(a + b)* a + b **(8)**

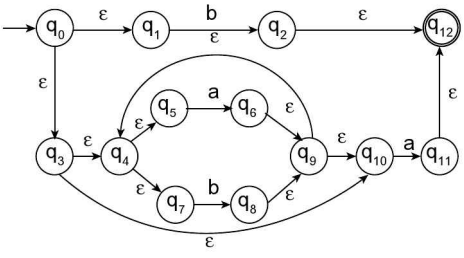
Ans: ⇒ (a + b)*



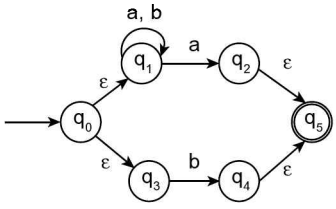
⇒ (a + b)*.a



⇒ (a + b)*.a + b



NFA (simplified)



68. Consider the alphabet A = {a, b} and the language L = {bb, bab, baab, baaab, ...} over A.

(i) Is A* finite or infinite? Give a brief reason for your answer. **(2)**

(ii) Write down a regular expression that represents the above language L. **(4)**

(iii) Write down a regular grammar which describes the above language L. **(4)**

(iv) Draw the DFA corresponding to the above language L. **(6)**

Ans: Given:

A = {a, b}

L = {bb, bab, baab, baaab, ...}

Solution:

(i) A* is infinite, since it is a combination of all possible a's and b's including ·

$A^* = \{\varepsilon, a, b, aa, bb, ab, ba, \ldots.\}$

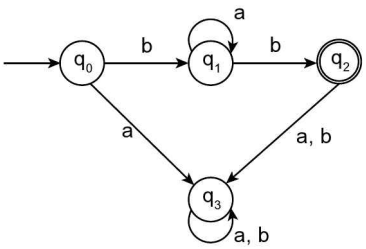
(ii) RE = ba*b

(iii) Regular grammar

$S \rightarrow bS_1$

$S \rightarrow b \mid aS_1$

(iv)



69. Find an equalities for the following regular expression and prove for the same.

(1) b + ab* + aa*b + aa*ab*

(2) a*(b + ab*)

(3) a(a + b)* + aa(a + b)* + aaa(a + b)* **(9)**

Ans: (1) b + ab* + aa*b + aa*ab*

= (b + ab*) + aa*(b + ab*)

= (ε + aa*) (b + ab*)

(2) a*(b + ab*)

= a*b + a*ab* = a* b + a + b*

(3) a (a + b)* + aa (a + b)* + aaa(a + b)*

= (a + aa + aaa) (a + b)*

70. State and prove using an example, the properties of regular language. **(7)**

Ans: If a class of languages is closed under a particular operation we call that fact a closure property of the class of languages.

In this section we discuss the closure properties of regular sets under (a) union (b) concatenation (c) closure (d) complementation (e) intersection (f) transpose (g) substitution (h) homomorphism.

Theorem

The regular sets are closed under union, concatenation and closure.

Proof

Let L_1 and L_2 be regular sets such that

$L_1 = T(A_1)$ and $L_2 = T(A_2)$ where

$A_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ and $A_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$

We shall assume that $Q_1 \cap Q_2 = \phi$

(a) **Union:** Let $A = (Q, \Sigma, \delta, q_0, F)$, where

$Q = Q_1 \cup Q_2 \cup \{q_0\}$, $q_0 \in Q_1 \cup Q_2$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

q_0 is the start date, $\Sigma = \Sigma_1 \cup \Sigma_2$, $F = F_1 \cup F_2$ and $\delta\cdot$ is defined as follows:
$\delta(q_0, \lambda) = \{q_1, q_2\}$
If $q \in \delta_1(p, a)$ then $q \in \delta(p, a)$
If $q \in \delta_2(p, a)$ then $q \in \delta(p, a)$
It is clear that $T(A) = (L_1 \cup L_2)$

(b) **Concatenation:** Let $A = (Q, \Sigma, \delta, q_0, F)$ where
$Q = Q_1 \cup Q_2$, $\Sigma = \Sigma_1 \cup \Sigma_2$, $q_0 = q_1$, $F = F_2$ and
δ is defined as follows:
$\delta(q, a) = \{\delta_1(q, a)\}$ for each q in $Q_1 - F_1$
$\delta(q, a) = \{\delta_2(q, a)\}$ for each q in F_1
and $\delta(q, a) = \{\delta_2(q, a)\}$ for each q in Q_1

A word w is in L_1, L_2 if $w = w_1 w_2$ where $w_1 \in L_1$and $w_2 \in L_2$. This is equivalent to $\delta_1(q_1, w)$ being in F_1 which is equivalent to $w_1 w_2$ being in T(A). Thus $L_1 L_2 =$ T(A).

(c) **Closure:** Let $L = T(A)$be a regular set where
$A = (Q, \Sigma, \delta, q_0, F)$
Let $B = (Q, \Sigma, \delta', q_0, F)$,
Where δ' is defined as follows:
$\delta'(q, a) = \{\delta(q, a)\}$ if q is in Q – F and
$\delta'(q, a) = \{\delta(q, a), \delta(q_0, a)\}$ if q is in F

It is easy to see that $L^* = T(B)$. If λ is in L, then it is accepted by B. Otherwise, we can modify B to accept λ.

(d) **Theorem**
The class of regular sets is closed under complementation. That is if X is a regular set and $X \subseteq \Sigma^* - L$ is a regular set.

Proof
Let *X* be *X(M)* for DFA $M = (Q, \Sigma_1, \delta, q_0, F)$ and let $X \subseteq^*$. First we assume $\Sigma_1 = \Sigma$, if there are symbols in Σ_1 not in Σ, we may delete all transitions of *M* on symbols not in $\Sigma\cdot$. The fact that $X \subseteq \Sigma\cdot^*$ assures us that we shall not thereby change the language of *M*. If there are symbols in Σ not in Σ_1, then none of these symbols appear in words of *X*. we may therefore introduce a dead state *d* into *M* with $\delta(d, a) = d$ for all a in Σ and $\delta(q, a) = d$ for all q in Q and a in $\Sigma - \Sigma_1$. Now to accept $\Sigma^* - X$, complement the final states of *M*. That is let $M' = (Q, \Sigma, \delta, q_0, Q - F)$.

The *M'* accepts a word *w* if and only if $\delta(q_0, w)$ is in Q – F, that is, w is in Σ^*–*X*. Note that it is essential to the proof that *M* is deterministic and without y moves.

(e) **Theorem**
If *X* and *Y* are regular sets over Σ, then $X \cap Y$ is also regular.

Proof
$X \cap Y = X \cap Y$ by Demorgan's Law where–denotes complementation with respect to an alphabet including the alphabets of *X* and *Y*. Closure under intersection then follows from closure under union and complementation.

(f) **Therorem**
If *L* is regular then L^R is also regular.

Proof

As L is regular, we can construct aFA

$M = (Q, \Sigma_1, \delta, q_0, F)$ such that $T(M) = L$.

We can construct a transition system (or transition diagram or transition graph) by M' by starting with the state diagram of M, and reversing the direction of the directed edges. The set of initial states of M' is defined as the set F, and q_0 defined as the (only) final state of M' (i.e.) $M' = (Q, \Sigma_1, \delta, F, \{q_0\})$.

If $w \in T(M)$, we have a path from q_0, to some final state in F with value w. By reversing the edegs we get a pair in M from some final state in F to q_0. Its path value is w^R. So $w^R \in T(M)$. In a similar way, we can see that if $w_1 \in T(M')$, then $w_1^R \in T(M)$. Thus from the state diagram it is easy to see that $T(M') = T(M)^R$. We can prove that $w \in T(M')$ if $w^R \in T(M')$ by induction on $|w|$. Since $T(M')$ is regular it follows that $T(M)^R$ is regular.

(g) **Theorem**

The class or regular sets is closed under substitution.

Proof

Let $R \subseteq \Sigma^*$ be a regular set and each for each a in Σ let $Ra \subseteq \Delta^*$ be a regular set. Let f: $\Sigma \rightarrow \Delta^*$ be the substitution defined by $f(a) = Ra$. Select regular expressions denoting R and each Ra. Replace each occurrence of the symbol a in the regular expression for R by the regular expression denotes $f(R)$, observe that the substitution of a union, product, or closure of the substitution.

A simple induction on the number of operators in the regular expression completes the proof.

(h) **Theorem**

The class of regular sets is closed under homomorphisms and inverse homomorphisms.

Proof

Closure under homomorphisms follows immediately from closure under substitution, since every homomorphism is a substitution in which $h(a)$ has one member.

To show closure under inverse homomorphism, let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA accepting L, and let h be a homomorphism from Δ to Σ^*. We construct a DFA M' that accepts $h(L)$ by reading symbol a in Δ and simulating M on $h(a)$. Formally let $M' = (Q, \Delta, \delta', q_0, F)$ and define $\delta'(q, a)$ for q in Q and a in Δ to be $\delta(q, h(a))$. Note that $h(a)$ may be a long string or λ but δ is defined on all strings by extension. It is easy to show by induction on $|x|$ that $\delta'(q_0, x) = \delta(q_0, h(x))$. Therefore M' accepts x if and only if M accepts $h(x)$. That is $L(M') = h^{-1}L(M)$.

71. State the algorithm for minimization of a DFA. Construct a minimized DFA for the regular expression (a + b)(a + b)* and trace for the string baaaab. (16)

Ans: His minimization algorithm finds a DFA M' equivalent to the DFA $M' = (Q, \Sigma, \delta, q_0, F)$ with reduced number of states.

Steps

- Mark the pair of inequivalent states (p, q) with 'X'
  (a) Initially 'X' is placed by considering one final state and one non-final state, where $\delta(p, x) \in F$ and $\delta(q, x) \in F$.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

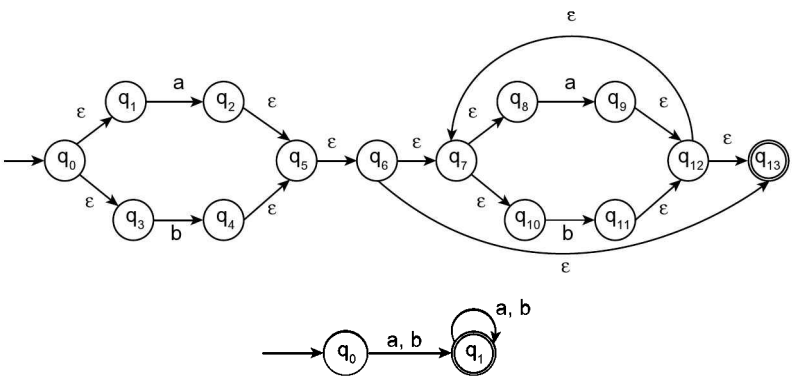
(b) If $\delta(p, a) = r$ and $\delta(q, a) = s$ for input symbol a and the states r, s are already distinguishable by some string x, then p, q are distinguished by ax, otherwise (p, q) is placed in a list associated with (r, s) entry.

- From the unmarked states, each pair of equivalent states are identified.
- New states of DFA has pair of equivalent states and the states which are not in the equivalent pairs are individual states.
- Construct DFA transition table with reduced number of states.

Algorithm for marking pairs of inequivalent states

Begin
for p in F and q in Q–F do mark (p, q):
  for each pair of distinct states(p, q) in F x F or (Q – F) x (Q – F) do
    if for some input symbol a, $(\delta(p, a), \delta(q, a))$ is marked then
      begin
        mark (p, q);
          Recursively mark all unmarked pairs on the list for (p, q) and on the lists of other pairs that are marked at this step.
      end
      else/* no pair (δ(p, a), δ(q, a)) is marked*/
        for all input symbols a **do**
          put(p, q) on the list for (δ(p, a), δ(q, a)) unless
          (δ(p, a) = δ(q, a)
end

εNFA for RE = (a + b)(a + b)*


ε
a
ε
ε
ε

In this representation q_0 is the start date and q_1 is the final state. Therefore it can't be reduced further.

Given $w = baaab$

$\delta(q_0, baaab)$
$= \delta(\delta(q_0, b), aaab)$
$= \delta(q_1, aaab)$
$= \delta(\delta(q_1, a), aab)$
$= \delta(q_1, aab)$
$= \delta(\delta(q_1, a), ab)$
$= \delta(q_1, ab)$
$= \delta(\delta(q_1, a), b)$
$= \delta(q_1, b)$
$= q_1 \in F$
String is accepted

72. Consider the grammar:

(i) S → I C t S

(ii) S → I C t S e S

(iii) S → a

(iv) C → b

where I, t, and, e stand for if, then, and else, and C and S for "conditional" and "statement" respectively.

(1) Construct a leftmost derivation for the sentence w = i b t i b t a e a.

(2) Show the corresponding parse tree for the above sentence.

(3) Is the above grammar ambiguous? If so, prove it.

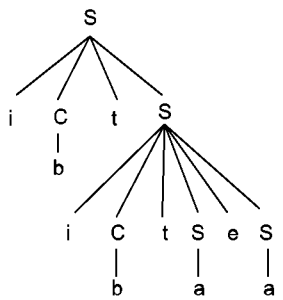
(4) Remove ambiguity if any and prove that both the grammar produces the same language. **(16)**

Ans: (1) w = i b t i b t a e a

Leftmost derivation 1:

S ⇒ i C t S
⇒ i b t S (C→b)
⇒ i b t i C t SeS (S→i C t SeS)
⇒ i b t i bt SeS (C→b)
⇒ i b t i bt aeS (S→a)
⇒ i b t i bt aea (S→a)

(2)



Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

(3) Leftmost derivation 2:

$S \Rightarrow i\ C\ t\ S\ e\ S$
$\Rightarrow i\ b\ t\ S\ E\ S\ (C \rightarrow b)$
$\Rightarrow i\ b\ t\ i\ C\ t\ S\ e\ S\ (S \rightarrow iCtS)$
$\Rightarrow i\ b\ t\ i\ b\ t\ S\ e\ S\ (C \rightarrow b)$
$\Rightarrow i\ b\ t\ i\ b\ t\ a\ e\ S\ (S \rightarrow a)$
$\Rightarrow i\ b\ t\ i\ b\ t\ a\ e\ a\ (S \rightarrow a)$

For the string w = *i b t i b t a e a,* the given grammar has two leftmost derivations. Therefore it is ambiguous.

73. Consider the GNF CFG G = ({S, T, C, D}, {a, b, c, d}, S, P) where P is:

$S \rightarrow cCD \mid dTC \mid \in$ $C \rightarrow a\ T\ D \mid c$
$T \rightarrow cDC \mid cST \mid a$ $D \rightarrow d\ C \mid d$

Present a pushdown automaton that accepts the language generated by this grammar.

Your PDA must accept by empty store, it must start with S on its stack, and it must be based on the above grammar. (16)

Ans: PDA is defined as $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, \phi)$

where:

$Q = \{q_0, q_1, q_2\}$
$\Sigma = \{a, b, c, d\}$
$\Gamma = \{S, T, C, D, z_0\}$

δ is defined as:

$\delta(q_0, \lambda, z_0) = \{(q_1, Sz_0)\}$
$\delta(q_1, \lambda, S) = \{(q_1, cCD), (q_1, dTC), (q_1, \varepsilon)\}$
$\delta(q_1, \lambda, T) = \{(q_1, cCD), (q_1, cST), (q_1, a)\}$
$\delta(q_1, \lambda, C) = \{(q_1, aTD), (q_1, c)\}$
$\delta(q_1, \lambda, D) = \{(q_1, dC)\ (q_1, d)\}$
$\delta(q_1, a, a) = \{(q_1, \lambda)\}$
$\delta(q_1, b, b) = \{(q_1, \lambda)\}$
$\delta(q_1, c, c) = \{(q_1, \lambda)\}$
$\delta(q_1, \lambda, z_0) = \{(q_2, \lambda)\}$

74. Define Pumping Lemma for Context Free Languages. Show that $L = \{a^i b^j c^k: i < j < k\}$ is not context-free. (6)

Ans: Let *n* be the pumping –lemma constant and consider string $z = a^n b^{(n+1)} c^{(n+2)}$

Write $z = uvwxy$, where *v* and *x*, may be "pumped", and a $|vwx| \leq n$.

If *vwx* does not have c's, then uv^3wx^3y has at least n + 2 a's or b's, and thus could not be in the language.

If *vwx* has a c, then it could not have an a, because its length is limited to n.

Thus, *uwy* has n a's, but no more than 2n + 2 b's and c's in total.

Thus, it is not possible that *uwy* has more b's than a's and also has more c's than b's.

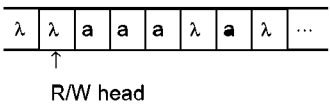
We conclude that *uwy* is not in the language, and now have a contradiction no matter how z is broken into *uvwxy.*

75. Construct a Turing Machine ™ to move an input string over the alphabet A = {a} to the right one cell. Assume that the tape head starts somewhere on a blank cell to the left of the input string. All other cells are blank, labeled by ^. The machine must move the entire string to the right one cell, leaving all remaining cells blank. (10)

Ans: Given: A = {a}

Solution:

The format of the string in the tape is:



The required transition rules are:

$\delta(q_0, \lambda) = (q_1, \lambda, R)$

$\delta(q_1, a) = (q_2, \lambda, R)$

$\delta(q_2, a) = (q_2, a, R)$

$\delta(q_2, \lambda) = (q_3, a, R)$ where $q_3 \in F$

76. Convert the following grammar into an equivalent one with no unit productions and no useless symbols. Convert to Chomsky Normal Form (CNF). (16)

S → A | CB
A → C | D
B → 1B | 1
C → 0C | 0
D → 2D | 2

Ans: Given

S→A | CB
A→C | D
B→1B | 1
C→0C | 0
D→2D | 2

Solution:

1. No null productions, the unit productions are exist in the form of chain productions.

S → A
A → C
C → 0C | 0 can be rewritten as
S → 0C | 0
Similarly
S → A
A → D
D → 2D | 2 can be rewritten as
S → 2D | 2
The new set of productions are:
S → 0C | 2D | CB | 0 | 2
B → 1B | 1

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$C \rightarrow 0C \mid 0$
$D \rightarrow 2D \mid 2$

2. Let $G_1 = (N_1, \{0, 1, 2\}, S, P_1)$ where P_1 is:
$S \rightarrow 0 \mid 2 \mid CB$
$B \rightarrow 1$ $\quad C \rightarrow 0$ $\quad D \rightarrow 2$ are added to P_1
$S \rightarrow 0C \mid 2D$, $\quad B \rightarrow 1B$ $\quad C \rightarrow 0C$
$D \rightarrow 2D$ yield
$S \rightarrow A_1C \mid A_3D$, $\quad B \rightarrow A_2B$, $\quad C \rightarrow A_1C$
$D \rightarrow A_3D$, $\quad$ WHERE $A_1 \rightarrow 0$, $\quad A_2 \rightarrow 1$, $\quad A_2 \rightarrow 2$
$\therefore N_1 = \{S_1 A_1, A_2, A_3, B, C, D\}$
Here G_1 is in CNF for the given grammar

77. Consider the language of all TMs that given no input eventually write a non-blank symbol on their tapes. Explain why this set is decidable. Why does this not conflict with the halting problem? (8)

Ans: Consider the language of all TMs that given no input eventually write a non-blank symbol on their tape. Since the controller is finite, there are only a finite number of states possible before the TM is forced to either loop or write a symbol, so we simply run the Turing machine for that number of steps and then accept if it has written a symbol and reject otherwise.

This does not conflict with the halting problem because we are considering only a subset of Turing machines (those roughly equivalent to DFAs) and not really addressing the halting problem for Turing machines.

78. Prove that the Post Correspondence Problem is decidable for strings over the alphabet (0). (8)

Ans: We can simplify this problem by assigning each domino a value which is the number of 0s on the top row minus the number of zeros on the bottom row. Our goal, then, is to choose dominos whose values sum to zero. We consider a few cases:

- If the set contains a "0" domino, that domino alone is a legal solution, so we accept.
- If the set contains a "+ m" and "–n" domino, we construct a solution from n" + m"s and m " + n"s and accept.
- If the set contains only " + " dominos or "–" dominos we cannot add them to zero, so we reject.

Since we can accept or reject based on a quick examination of the set of dominos, the problem is decidable.

79. Prove that the problem of determining if the languages generated by two CFGs are equal is undecidable. (8)

Ans: We show this by reducing the problem of determining whether a CFG accepts everything to the problem of determining if the languages generated by tow CFGs are equal by taking our input CFG and comparing it to the CFG that generates everything. Since we know from class that the "everything" problem is undecidable, the "equal" probably must also be.

80. Prove that the Punchcard Puzzle is NP-complete. (8)

Ans: We show that the problem is in NP by showing that it is verifiable in polynomial time. To do this, we simply stack the cards according to the answer presented in the certificate to determine if they cover all the holes. This can be accomplished easily in polynomial time.

To show that it is NP-complete, we reduce 3–SAT to it. We create cards $\{x_1, x_2, \ldots\}$ for each variable in the 3–SAT formula and create a hole position in each column fo reach term of the formula. Then, we punch holes in the left column of the card in every position which corresponds to a term that does not contain that card's variable and in the right column for every term which does not contain that card's variable's complement. The Punchcard problem is only satisfiable if every hole can be covered by one of the card which implies that every term in 3–SAT problem is satisfiable.

81. Prove that for any language L recognized by an NFA with ε-transitions, there exists an NFA without ε-transitions to recognize L. (8)

Ans: **Theorem**

If L is accepted by NFA with ε-transitions, than L is accepted by an NFA without ε-transitions.

Proof

Let $M = (Q, \Sigma, \delta, q_0, F^1)$ be an NFA with ε-transitions. Construct M^1 which is NFA without ε-transitions.

$M^1 = (Q, S, \delta^1, q_0, F^1)$ where

$F1 = F \{q_0\}$ if ε-CLOSURE (q_0) contains a state of F.

F otherwise

By induction: $\begin{cases} \delta 1 \text{ and } \hat{\delta} \text{ are same} \\ \delta \text{ and } \hat{\delta} \text{ are different} \end{cases}$

Let x be any string

$\delta^1(q_0, x) = \delta(q_0, x)$

This statement is not ture if $x = \varepsilon$ because $\delta^1(q_0, \varepsilon) = \{q_0\}$ and

$\hat{\delta}(q_0, \varepsilon) = \varepsilon\text{-CLOSURE}(q_0)$

Basic step

$|x| = 1$ x is a symbol whose value is a

$\delta^1(q_0, a) = \delta(q_0, a)$ (because by definition of $\hat{\delta}$)

Induction step

let $x = wa$ where a is in Σ

$\delta^1(q_0, wa) = \delta^1(\delta^1(q_0, w), a)$

$= \delta^1(\delta(q_0, w), a)$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$= \hat{\delta}(p,a)$[because by inductive hypothesis

$\delta(q_0,w) = \hat{\delta}(q_0,w) = p$(say)]

Now we must show that

$\delta^1(p,a) = \delta(q_0,wa)$

But

$$\delta^1(p,a) = \bigcup_{q \text{ in } P} \delta^1(q,a) = \bigcup_{q \text{ in } P} \delta(q,a)$$

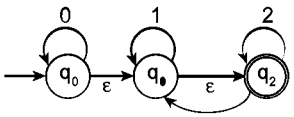
$$= \delta(\delta^1(q_0,w),a)$$

$$= \delta(q_0,wa)$$

$$= \hat{\delta}(q_0,x)$$

Hence $\delta^1(q_0,x) = \hat{\delta}(q_0,x)$

82. Construct an NFA without ε - transitions for the following NFS. (8)



Ans: NFA with ε-transition table (M):

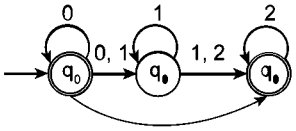
State	Inputs			
	0	1	2	ε
q_0	q_0	–	–	q_1
q_1	–	q_1	–	q_2
q_2	–	–	q_2	–

NFA without ε-transition table (M^1):

State	Inputs		
	0	1	2
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	–	$\{q_1, q_2\}$	$\{q_2\}$
q_2	–	–	$\{q_2\}$

The set of final states of M1 $= F \cup \{q_0\} = \{q_0, q_2\}$ since ε-CLOSURE $\{q_0\} = \{q_0, q_1, q_2\}$

Transition diagram:



83. For a given regular expression r, prove that there exists an NFA with transitions that accepts L(r). (8)

***Ans:* Theorem (Conversion of R.E. to FSA)**

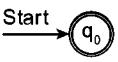
For every regular expression r there exists a NFA with ε-transitions that accepts L(r).

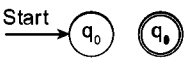
Proof

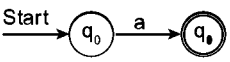
We prove by induction on the number of operators in the regular expression r that there is an NFA M with ε-transistors, having one final state and no transitions out of this final state such that L(M) = L(r).

Basis step (Zero operators)

Suppose r is ε, ϕ or a for some $a \in \Sigma$. Then the equivalent NFA's are:

(i) $r = \varepsilon$ 

(ii) $r = \phi$ 

(iii) $r = a$ 

Induction (One or more operators)

Assume the theorem is true for r having fewer that i operators, $i \geq 1$. Let r have i operators. We discuss three cases depending on the form of r.

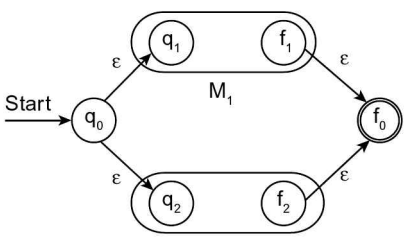
Case 1:

Let $r = r_1 + r_2$. Both r_1 and r_2 must have fewer that i operators. Thus there are NFA's $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, \{f_1\})$ and $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, \{f_2\})$ with $L(M_1) = L(r_1)$ and $L(M_2) = L(r_2)$. Assume Q_1 and Q_2 are disjoint.

Let q_0, f_0 be a new initial and final state respectively.

$\therefore$ $M = (Q_1 \cup Q_2 \cup \{q_0, f_1\}, \Sigma_1 \cup \Sigma_2, \delta, q_0, \{f_0\})$ where δ is defined by

(i) $\delta(q_0, \varepsilon) = \{q_1, q_2\}$
(ii) $\delta(q, a) = \delta_1(q, a)$ if $q \in Q_1 - \{f_1\}$, $a \in \Sigma_1 \cup \{\varepsilon\}$
(iii) $\delta(q, a) = \delta_2(q, a)$ if $q \in Q_2 - \{f_2\}$, $a \in \Sigma_2 \cup \{\varepsilon\}$
(iv) $\delta_1(f_1, \varepsilon) = \delta_2(f_2, \varepsilon) = \{f_0\}$



All the moves of M_1 and M_2 are present in M. Any path in the transition diagram of M from q_0 to f_0 must being by going to either q_1 or q_2 on ε. If the path goes to q_1, it may follow any path in M_1 to f_1 and then goto f_0 on ε.

Similarly paths that begin by going to q_2, may follow any path in M_2 to f_2 and then go to f_0 on ε. These are the only paths from q_0 to f_0. That is, there is a path labeled x in M_1 from q_1 to f_1 or a path in M_2 from q_2 to f_2. Hence $L(M) = L(M_1) \cup L(M_2)$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

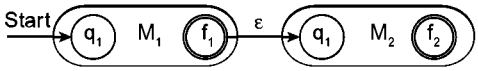
Case 2:

Let $r = r_1 r_2$. Let M_1 and M_2 be as in case 1. Construct $M = (Q_1 \cup Q_2, \Sigma_1 \cup \Sigma_2, \delta, \{q_1\}, \{f_2\})$, where δ is given by

(i) $\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$

(ii) $\delta(f_1, \varepsilon) = \{q_2\}$

(iii) $\delta(q, a) = \delta_2(q, a)$ for q in Q_2 a in $\Sigma_2 \cup \{\varepsilon\}$



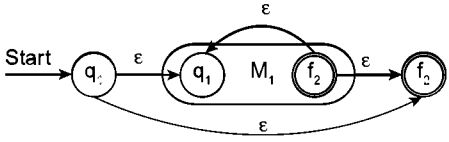
Every path in M from q_1 to f_2 is a path labeled by some string x from q_1 to f_1, followed by the edge from f_1 to q_2 labeled ε, followed by a path labeled by some string y from q_2 to f_2.

Thus $L(M) = \{xy | x \text{ is in } L(M_1) \text{ and } y \text{ is in } L(M_2)\}$
and $L(M) = L(M_1).L(M_2)$.

Case 3:

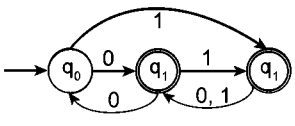
Let $r = r_1^*$. Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, \{f_1\})$ and $L(M_1) = r_1$. Construct $M = (Q_1 \cup \{q_0, f_0\}, \Sigma_1, \delta, q_0, \{f_0\})$, where δ is given by

$\delta(q_0, \varepsilon) = \delta(f_1, \varepsilon) = \{q_1, f_0\}$
$\delta(q, a) = \delta_1(q, a)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\varepsilon\}$



Any path from q_0 to f_0 consists either of a path from q_0 to f_0 on ε or a path from q_0, to q_1 on ε followed by some number of paths from q_1 to f_1, then back to q_1 on ε. There is a path in M from q_0 to f_0 labeled x if and only if we write $x = x_1 x_2 \ldots\ldots x_j$ fro some $j \geq 0$ such that each $x_i \in L(M_1)$. Hence $L(M) = L(M_1)^*$

84. Find the regular expression corresponding to the following automata. (8)



Ans: We will find $R_{ij}^{(0)}$ where $K = 0$

$R_{11}^{(0)} = \varepsilon$	because from state q_1 to q_1 can be achieved only by ε transition.
$R_{12}^{(0)} = 0$	because from q_1 we can reach q_2 by '0' input.
$R_{13}^{(0)} = 1$	because from q_1 we can reach q_3 by '1' input $\delta(q_1, 1) = q_3$
$R_{21}^{(0)} = 0$	because $\delta(q_2, 0) = q_1$
$R_{22}^{(0)} = \varepsilon$	because from q_2 we can reach q_2 only on ε transition
$R_{23}^{(0)} = 1$	because $\delta(q_2, 1) = q_3$
$R_{31}^{(0)} = \phi$	because from q_3 to reach q1 no such path exists
$R_{32}^{(0)} = 0 + 1$	because from q_3 to reach q_2 we can give either '0' or '1' as input symbol.
$R_{33}^{(0)} = \varepsilon$	to remain in q_3 it needs ε transition.

Tabulation is as follows for K = 0:

$R_{11}^{(0)}$	ε
$R_{12}^{(0)}$	0
$R_{13}^{(0)}$	1
$R_{21}^{(0)}$	0
$R_{22}^{(0)}$	ε
$R_{23}^{(0)}$	1
$R_{31}^{(0)}$	ϕ
$R_{32}^{(0)}$	0 + 1
$R_{33}^{(0)}$	ε

We need to apply the following simplification.

- $(\varepsilon + R)^* = R^*$
- $R + RS^* = RS^*$
- $\phi R = R\phi = \phi$ (Annihilation)
- $\phi + R = R + \phi = R$ (Identity)

Now we will go for K = 1
$n = 3$ (number of states is 3 $\{q_1, q_2, q_3\}$)
$R_{ij}^{(K)} = R_{ij}^{(K-1)} + Ri_K^{(K-1)}(R_{KK}^{(K-1)})^* R_{Kj}^{(K-1)}$

Therefore for K = 1

$R_{11}^{(1)} = R_{11}^{(0)} + R_{11}^{(0)}(R_{11}^{(0)})^* R_{11}^{(0)}$
$= \varepsilon + \varepsilon(\varepsilon)^*\varepsilon$
$= \varepsilon$

$R_{12}^{(1)} = R_{12}^{(0)} + R_{11}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)}$
$= 0 + \varepsilon(\varepsilon)^*0$
$= 0 + 0 = 0$

$R_{13}^{(1)} = R_{13}^{(0)} + R_{11}^{(0)}(R_{11}^{(0)})^* R_{13}^{(0)}$
$= 1 + \varepsilon(\varepsilon)^*1$
$= 1 + 1 = 1$

$R_{21}^{(1)} = R_{21}^{(0)} + R_{21}^{(0)}(R_{11}^{(0)})^* R_{13}^{(0)}$
$= 0 + 0(\varepsilon)^*\varepsilon$
$= 0 + 0 = 0$

$R_{22}^{(1)} = R_{22}^{(0)} + R_{21}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)}$
$= \varepsilon + 0(\varepsilon)^*0$
$= \varepsilon + 00$

$R_{23}^{(1)} = R_{23}^{(0)} + R_{21}^{(0)}(R_{11}^{(0)})^* R_{13}^{(0)}$
$= 1 + 0(\varepsilon)^*1$
$= 1 + 01$

$R_{31}^{(1)} = R_{31}^{(0)} + R_{31}^{(0)}(R_{11}^{(0)})^* R_{11}^{(0)}$
$= \phi + \phi(\varepsilon)^*\varepsilon$
$= \phi + \phi$
$= \phi$

$R_{32}^{(1)} = R_{32}^{(0)} + R_{31}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)}$
$= 0 + 1 + \phi(\varepsilon)^*)$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

$= 0 + 1 + \phi$

$= 0 + 1$

$R_{33}^{(1)} = R_{33}^{(0)} + R_{31}^{(0)}(R_{11}^{(0)})^* R_{13}^{(0)}$

$= \varepsilon + \phi(\varepsilon)^*1$

$= \varepsilon$

Tabulation is as follows for K = 0:

$R_{11}^{(1)}$	ε
$R_{12}^{(1)}$	0
$R_{13}^{(1)}$	1
$R_{21}^{(1)}$	0
$R_{22}^{(1)}$	$\varepsilon + 00$
$R_{23}^{(1)}$	$1 + 01$
$R_{31}^{(1)}$	ϕ
$R_{32}^{(1)}$	$0 + 1$
$R_{33}^{(1)}$	ε

Now for K = 2

$R_{ij}^{(1)} = R_{ij}^{(2-1)} + R_{i2}^{(2-1)}(R_{22}^{(2-1)})^* R_{2i}^{(2-1)}$

$= R_{ij}^{(1)} + R_{i2}^{(1)}(R_{22}^{(1)})^* R_{2i}^{(1)}$

Therefore

$R_{11}^{(2)} = R_{11}^{(1)} + R_{12}^{(1)}(R_{22}^{(1)})^* R_{21}^{(1)} = \varepsilon + 0(\varepsilon + 00)^*0$

$= \varepsilon + 0(00)^*0 = (00)^*$

$R_{12}^{(2)} = R_{12}^{(1)} + R_{12}^{(1)}(R_{22}^{(1)})^* R_{22}^{(1)} = 0 + 0(\varepsilon + 00)^*(\varepsilon + 00)$

$= 0 + 0(00)^* = 0(00^*)\ (\therefore R + RS^* = RS^*)$

$R_{13}^{(2)} = R_{13}^{(1)} + R_{12}^{(1)}(R_{22}^{(1)})^* R_{23}^{(1)} = 1 + 0(\varepsilon + 00)^*(1 + 01)$

Here $(\varepsilon + 00)^* = (00)^*$

and $(1 + 01) = (\varepsilon + 0)1$

and so $R_{13}^{(2)} = 1 + 0(00)^*(\varepsilon + 0)1$

Here $(00)^*(\varepsilon + 0) = 0^*$

Hence $0(00)^*(\varepsilon + 0)1 = 0(0^*)1$

$\therefore R_{13}^{(2)} = 1 + 00^*1 = 0^*1$

$R_{21}^{(2)} = R_{21}^{(1)} + R_{21}^{(1)}(R_{12}^{(1)})^* R_{21}^{(1)}$

$= 0 + (\varepsilon + 00)(\varepsilon + 00)^*0$

$= 0 + 00(00)^*0$

$= 0(00)^*$

$R_{22}^{(2)} = R_{22}^{(1)} + R_{22}^{(1)}(R_{22}^{(1)})^* R_{22}^{(1)}$

$= (\varepsilon + 00) + (\varepsilon + 00)(\varepsilon + 00)^*(\varepsilon + 00)$

$= (00)^*$

$R_{23}^{(2)} = R_{23}^{(1)} + R_{22}^{(1)}(R_{22}^{(1)})^* R_{23}^{(1)}$

$= (1 + 01) + (\varepsilon + 00)(\varepsilon + 00)^*(1 + 01)$

$= (\varepsilon + 0)1 + (00)^*(1 + 01)$

$= (\varepsilon + 0)1 + (00)^*(\varepsilon + \ + \ 0)1$

$= (\varepsilon + 0)1 + 0^*1$ (because $(00)^*(\varepsilon + 0)1 = 0^*1$)

$= 0^*1$

$$R_{31}^{(2)} = R_{31}^{(1)} + R_{32}^{(1)}(R_{22}^{(1)})^* R_{21}^{(1)}$$
$$= \phi + (0+1)(\varepsilon+00)^*0$$
$$= \phi + (0+1)(00)^*0$$
$$= (0+1)(00)^*0 \text{ (because } (\varepsilon+00)^* = (00)^*)$$
$$R_{32}^{(2)} = R_{32}^{(1)} + R_{32}^{(1)}(R_{22}^{(1)})^* R_{22}^{(1)}$$
$$= (0+1) + (0+1)(\varepsilon+00)^*(\varepsilon+00)$$
$$= (0+1) + (0+1)(00)^*$$
$$= (0+1)(00)^* \text{ (because } R + RS^* = RS^*)$$
$$R_{33}^{(2)} = R_{33}^{(1)} + R_{32}^{(1)}(R_{22}^{(1)})^* R_{23}^{(1)}$$
$$= \varepsilon + (0+1)(\varepsilon+00)^*(1+01)$$
$$= \varepsilon + (0+1)(00)^*(\varepsilon+0)1 \text{ (because } (\varepsilon+00)^* = (00)^* \text{ and } (1+01) = ((\varepsilon+0)1))$$
$$= \varepsilon + (0+1)0^*1 \text{ (because } (00)^*(\varepsilon+0) = 0^*)$$
$$= \varepsilon + (0+1)0^*1$$

Tabulation is as follows for K = 0:

$R_{11}^{(2)}$	$(00)^*$
$R_{12}^{(2)}$	$0(00)^*$
$R_{13}^{(2)}$	0^*1
$R_{21}^{(2)}$	$0(00)^*$
$R_{22}^{(2)}$	$(00)^*$
$R_{23}^{(2)}$	0^*1
$R_{31}^{(2)}$	$(0+1)(00)^*0$
$R_{32}^{(2)}$	$(0+1)(00)^*$
$R_{33}^{(2)}$	$\varepsilon + (0+1)0^*1$

No we will find regular expression L(M)

$$L(M) = \bigcup_{q_j \text{ in } F} R_{1j}^{(n)}$$

i.e., $R_{1j1}^{(n)} + R_{1j2}^{(n)} + \ldots + R_{1jp}^{(n)}$

Where F is set of final sets

$$F = \{q_{j1}, q_{j2}, \ldots q_{jp}\}$$

In our example, the set of final states

$$F = \{q_2, q_3\}$$

$$\therefore L(M) = R_{12}^{(3)} + R_{13}^{(3)} \text{ (where n=3 states)}$$

$$\therefore R_{12}^{(3)} = R_{12}^{(2)} + R_{13}^{(2)}(R_{33}^{(2)})^* R_{32}^{(2)}$$
$$= 0(00)^* + 0^*1(\varepsilon + (0+1)0^*1)^*(0+1)(00)^*$$
$$= 0(00)^* + 0^*1((0+1)0^*1)^*(0+1)(00)^* \text{ (because } (\varepsilon+R)^* = R^*)$$

$$R_{13}^{(3)} = R_{13}^{(2)} + R_{13}^{(2)}(R_{33}^{(2)})^* R_{33}^{(2)}$$
$$= 0^*1 + 0^*1(\varepsilon + 0 + 1)0^*1^*(\varepsilon + (0+1)0^*1)$$

Similarly
$$= 0^*1[\varepsilon + (\varepsilon + (0+1)0^*1)^*(\varepsilon + (0+1)0^*1)]$$
$$= 0^*1(\varepsilon + (0+1)0^*1)^* \text{ (because } \varepsilon + R^* = R^*)$$
$$= 0^*1((0+1)0^*1)^* \text{ (because } \varepsilon + R)^* = R^*)$$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Hence regular expression is

$$R_{12}^{(3)} = R_{13}^{(2)} = 0*1((0+1)0*1)*(0+1)(00)*+0(00)* \\ +0*1((0+1)0*1)* \\ = 0*1((0+1)0*1)*(0+1)(00)*+0(00)*(\text{because } R+RS*=RS*)$$

$\therefore$ The regular expression for the finite automata is
$L(M) = 0*1((0+1)0*1)(0+1)(00)* + 0(00)*$

85. Prove that a CFL can be recognized by a PDA by empty stack. **(9)**

Ans: Theorem
For any context free language L, there exists an *pda* M such that L = L(M)

Proof
Let G = (V, T, P, S) be a grammar. There exists a Greibach Normal Form then we can construct *pda* which simulates left most derivations in this grammar.

M = (Q, Σ, Υ, δ, q0, z, F), where
Q = $\{q_0, q_1, q_f\}$ = set of states
Σ = terminals of grammar G
Υ = V$\cup${z} where V is the variables in grammar G
F = $\{q_f\}$ = final state.

The transition function will include
$\delta(q_0, \lambda, z) = \{q_1, Sz\}$, so that after the first move of M, the stack contains the start symbol S of the derivation. (The stack symbol z is a marker to allow us to detect the end of the derivation)

In addition, the set of transition rules is such that
(i) $\delta(q_1, \lambda, A) = \{(q, \alpha)\}$ for each $A \to \alpha$ in P
(ii) $\delta(q, a, a) = \{(q, \lambda)\}$ for each $a \in \Sigma$

86. Construct a PDA equivalent to the following grammar.

$S \to aAA$ $A \to aS \mid bS \mid a$ **(7)**

Ans: *S→aAA*
A→aS/bS/a
The PDA equivalent of the given grammar is:
M = ({*q*}, (*a*, *b*), {*S*, *a*, *b*, *A*}, δ, *q*, *S*, ϕ) where δ is defined as:

$R_1 : \delta(q, \lambda, S) = \{(q, aAA)\}$
$R_2 : \delta(q, \lambda, A) = \{(q, aS), (q, bS), (q, a)\}$
$R_3 : \delta(q, a, a) = \{(q, \lambda)\}$
$R_4 : \delta(q, b, b) = \{(q, \lambda)\}$

Test whether some string *abaaaa* is in N(M).

(*q*, *abaaaa*, *S*) $\vdash$(*q*, *a baaaa*, *aAA*) by R_1
$\vdash$(*q*, *baaaa*, *AA*) by R_1
$\vdash$(*q*, *baaaa*, *bSA*) by R_2
$\vdash$(*q*, *aaaa*, *SA*) by R_4
$\vdash$(*q*, *aaaa*, *aAAA*) by R_1
$\vdash$(*q*, *aaa*, *AAA*) by R_3
$\vdash$(*q*, *aaa*, *aAA*) by R_2

$\vdash(q, aa, AA)$ by R_3
$\vdash(q, aa, aA)$ by R_2
$\vdash(q, a, A)$ by R_1
$\vdash(q, a, a)$ by R_2
$\vdash(q, \lambda, \lambda)$ by R_3

87. Prove that every language recognized by a PDA is Context-Free. (8)

Ans: Theorem

If L is N(M) for some PDA M, then L is CFL.

Proof

1. It has single final state q_f iff the stack is empty.
2. All transitions must have the form
   $\delta(q_i, a, A) = \{C_1, C_2 \ldots . C_n\}$, where
   $\delta(q_i, a, A) = (q_j, \lambda)$ (1)
   $\delta(q_i, a, A) = (q_j, BC)$ (2)

That is, each move either increases or decreases the stack content by a single symbol.

Given $M = (Q, \Sigma, \Upsilon, \delta, q_0, z_0, \{q_f\})$ satisfies the condition (1) and (2)

G = (V, T, P, S)

V = elements of the form $[a, A, p]$, a and p in Q and A in Υ

$T = \Sigma$

S = start symbol

$S \rightarrow [q_0, z_0, q]$ for each q in Q.

P consist of: $u, v \in \Sigma^*$

$A, X \in \Upsilon^*, q_i, q_j \in Q$

$(q_i, uv, AX) \overset{*}{\vdash} (q_j, v, X)$

Implies that $(q_i, A, q_j) \rightarrow u$

Consider $[q_i, A, q_k] \rightarrow a[q_j, B, q_i]\,[q_i, C, q_k]$

The corresponding transition for PDA is

$\delta(q_i, a, A) = \{(q_j, B\ C) \ldots\ldots\}$

Similarly if $[q_i, A, q,] \rightarrow a$ then the corresponding transition is $\delta(q_i, a, A) = \{(q_j, \lambda)\}$

For all sentential forms leading to a terminal string, the argument holds true.

The conclusion is

$(q_0, w, z_0) \vdash^* (q_j, \lambda, \lambda)$ is true iff $(q_0 z_0 q_f) \overset{*}{\Rightarrow} w$

Consequently L(M) = L(G)

88. Construct a PDA for the set of palindrome over the alphabet {a, b}. (8)

Ans:

$M = (Q, \Sigma, \Gamma, d, q_0, z, F)$ where

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$\Gamma = \{z, a, b\}$

$F = \{q_2\}$

The transition function has several parts :

(i) set to push w on to stack.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

$\delta(q_0, a, a) = \{(q_0, aa)\} \rightarrow (1)$

$\delta(q_0, b, a) = \{(q_0, ba)\} \rightarrow (2)$

$\delta(q_0, a, b) = \{(q_0, ab)\} \rightarrow (3)$

$\delta(q_0, b, b) = \{(q_0, bb)\} \rightarrow (4)$

$\delta(q_0, a, z) = \{(q_0, az)\} \rightarrow (5)$

$\delta(q_0, b, z) = \{(q_0, bz)\} \rightarrow (6)$

(ii) To find middle of the string, where *npda* switches from q_0 to state q_1.

$\delta(q_0, \lambda, a) = \{(q_1, a)\} \rightarrow (7)$

$\delta(q_0, a, b) = \{(q_1, b)\} \rightarrow (8)$

(iii) Set to match w^R against contents of the stack.

$\delta(q_1, a, a) = \{(q_1, \lambda)\} \rightarrow (9)$

$\delta(q_1, b, b) = \{(q_1, \lambda)\} \rightarrow (10)$

and finally

$\delta(q_1, \lambda, z) = \{(q_2, z)\} \rightarrow (11)$

To recognize successful match.

The processing of string $w = abba$

$(q0, abba, z) \vdash (q_0, bba, az) \rightarrow$ by rule 5

$\vdash (q_0, ba, baz) \rightarrow$ by rule 2

$\vdash (q_1, ba, baz) \rightarrow$ by rule 8

$\vdash (q_1, a, az) \rightarrow$ by rule 10

$\vdash (q_1, a, az) \rightarrow$ by rule 9

$\vdash (q_1, \lambda, z) \rightarrow$ by rule 9

$\vdash (q_2, \lambda) \rightarrow$ by rule 11

Hence string is accepted.

Note:

At 3rd move, to locate middle of the string, (q_0, bc, baz) we have two choices for next move

(i) $\delta(q_0, b, b) = \{(q_0, bb)\}$ (or)

(ii) $\delta(q_0, \lambda, b) = \{(q_1, b)\}$

89. Prove that every non-empty CFL is generated by a CFG with no useless symbols. (9)

***Ans:* Theorem**

If G is a CFG such that $L(G) = \phi$, we can find an equivalent grammar G^1, such that each variable in G^1 derives some terminal string.

Proof:

Let $G = (N, T, S, P)$ and $G^1 = (N^1, T^1, S, P^1)$

(a) ***Construction of N^1***

We define $W_1 \subseteq N$ by recursion. $W_1 = \{A \in N |$ there exists a production $A \rightarrow w$ where

$w \in T^*\}$. (If W1 $= \phi$, some variable will remain after the application of any production, and so L(G) $= \phi$).

$W_{i+1} = W_i \cup \{A \in N |$ there exists some production $A \to \alpha$ with $\alpha \in (T \cup W_i)^*\}$

(b) ***Construction of i***

Pi $= \{A \to \alpha \mid A, \alpha \in (Ni \cup T)^*\}$

We can define Gi $=$ (Ni, T, S, Pi), S is in Ni. We can prove that every variable In Ni defines some terminal string. So $S \in Ni$, L(G) $= \phi$. Now we prove that Gi is the required grammer.

(i) If each $A \in$ Ni than $A \overset{*}{\underset{G^1}{\Rightarrow}} w$ for sme $w \in T^*$; conversely, if $A \overset{*}{\underset{G^1}{\Rightarrow}} w$ then $A \in Ni$

(ii) L(Gi) $=$ L(G)

To prove (i) we note that $W_{k+1} W_2 \cup \cup W_{k.We}$ prove by induction on i

That for $i = 1, 2, \ldots k$, $A \in Wi$ implies $A \overset{*}{\underset{G^1}{\Rightarrow}} w$ for some $w \in T^*$. If $A \in W_1$, then $A \underset{G}{\Rightarrow} w$. So the production $\overset{G^1}{\to} w$ is in P^1.Therefore, $A \overset{*}{\underset{G^1}{\Rightarrow}} w$. Thus there is basis for induction.

Let us assume the result for i. Let $A \in W_{i+1}$. Then either $A \in w_i$, in which case, $A \overset{*}{\underset{G^1}{\Rightarrow}} w$ for some $w \in T^*$ by induction hypothesis. Or, there exists a production $A \to \alpha$ is P^1. e can write $\alpha = X_1 X_2 \ldots X_m$, where $X_j \in T \cup W_i$. If $X_j \in W_i$ by induction hypothesis, $X_j \overset{*}{\underset{G^1}{\Rightarrow}} W_j$ for some $W_j \in X_j$). By induction the result is true for $i = 1, 2, \ldots .k$.

The converse part can be proved in a similar way by induction on the number of steps in the derivation $A \overset{*}{\underset{G^1}{\Rightarrow}} w$. We see immediate;y that $L(G_1) \subseteq L(G)$ as $N_1 \subseteq N$ and $P_1 \subseteq P$.

To prove $L(G) \subseteq L(G^1)$, we need an auxillary result.

$A \overset{*}{\Rightarrow} w$ if $A \overset{*}{\Rightarrow} w$ for some $w \in T^* \quad \to \quad 1$

We prove the above step by induction on the number of steps in the derivation $A \overset{*}{\underset{G^1}{\Rightarrow}} w$ If $A \underset{G}{\Rightarrow} w$, then $A \to w$ is in P and $A \in W^1 \subseteq N^1$. As $A \in N^1$ and $w \in T^*$, $A \to w$ is in P^1. So $A \underset{G^1}{\Rightarrow} w$, and there is a basis for induction. Assume $A \overset{*}{\Rightarrow} w$ derivation in atmost k steps. Let $A \overset{k+1}{\Rightarrow} w$. We can split this as

$A \underset{G}{\Rightarrow} X_1 X_2 \ldots X_m \overset{k}{\underset{G}{\Rightarrow}} w_1 w_2 \ldots wm$ such that $X_j \overset{*}{\underset{G^1}{\Rightarrow}} w_j$. If $X_j \in T$ then $w_j = x_j \to 2$

If $X_j \in N$ then by equation 1 above, $X_j \in N^1$. As $X_j \overset{*}{\underset{G^1}{\Rightarrow}} w_j$ is atmost k steps, $X_j \overset{*}{\underset{G^1}{\Rightarrow}} w_j$.

Also, $X_1, X_2, \ldots \ldots X_m \in (T \cup N^1)^*$ implies $A \to X_1, X_2, \ldots X_m$ is in P^1. Thus $A \underset{G^1}{\Rightarrow} X_1, X_2 \ldots \ldots X_m \overset{*}{\underset{G^1}{\Rightarrow}} w_1 w_2 \ldots w_m$. Hence by induction, equation 1 is true for all derivations. In particular, $S \overset{*}{\underset{G}{\Rightarrow}} w$ implies $S \overset{*}{\underset{G^1}{\Rightarrow}} w$. This prove that $L(G) \subseteq L(G^1)$, and equation 2 is completely proved.

90. State and prove Chomsky normal form for CFL. **(7)**

Ans: **Theorem**

For every context free grammar, there is an equivalent grammar in Chomsky Normal Form (CNF)

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Proof

Step 1: Elimination of null productions. We then apply theorem to eliminate chain productions. Let the grammar thus obtained to G = (N, T, S, P)

Step 2: Elimination of terminals on R.H.S. We define G1 = (N^1, T, S, P_1) where P_1 and N^1 are constructed as follows:

(i) All the productions in P of the form A→a or A→BC are included in P_1, All the variables in N are included in N^1.

(ii) Consider A→X_1, X_2.....X_n with some terminal on R.H.S. If X_i is a terminal, say a_i, add a new variable C_{ai} to N^1 and C_{ai}→$_{ai}$ to P_1. In production A→X_1 X_2....X_n, every terminal on R.H.S. is repaced by the corresponding new variable and the variables on the R.H.S. are retained. The resulting production is added to P_1. Thus we get G_1 = (N^1, T, P_1, S).

Step 3: Restricting the number of variables on R.H.S. For any production in P_1, the R.H.S. consists of either a single terminal (or λ in S→λ) or two or more variables. We define G_2 = (N″, T, P_2, S) as follows:

All the productions in P_1 are added to P_2 if they are in the required form. All the variables in N1 are added to N″.

Consider A→A_1 A_2....A_m, where m≥3. We introduce new productions A→A_1C_1, C_1→A_2C_2, ...C_{m-2}→$A_{m-1}A_m$, and new variables C_1, C_2, ...C_{m-2}. These are added to P″ and N″ respectively.

Thus we get G_2 in Chomsky Normal Form.

Step 4: To complete the proof we have to show that L(G) = L(G_1) = L(G_2).
To show that L(G)⊆L(G_1), we start with w∈L(G). If A → X_1, X_2 .....X_n is used in the derivation of w, the same effect can achieved by using the corresponding production in P, and the productions involving the new variables. Hence A $\overset{*}{\Rightarrow}$ X_1X_2....X_n. Thus L(G)⊆L(G_1).

Let w ∈ L(G_1). To show that w∈L(G_1), it is enough to prove the following.

$$A \underset{G}{\overset{*}{\Rightarrow}} w \text{ if } A \in N, A \underset{G}{\overset{*}{\Rightarrow}} w \tag{1}$$

We prove 1 by induction on the number of steps in A $\underset{G^1}{\overset{*}{\Rightarrow}}$ w. If A $\underset{G^1}{\overset{*}{\Rightarrow}}$ w, the A→w is a production in P_1. By construction of P_1, w is a single terminal. So A→w is in P i.e., A $\underset{G}{\overset{*}{\Rightarrow}}$ w. This is basis for induction.

Let us assume (1) for derivations in at most k steps. Let A $\underset{G^1}{\overset{k+1}{\Rightarrow}}$ w. we can split this derivation as A $\underset{G^1}{\Rightarrow}$ A_1A_2....A_m $\underset{G^1}{\overset{k}{\Rightarrow}}$ w_i, w_m = w such that $A_i \underset{G_1}{\overset{*}{\Rightarrow}} w_i$. Each A_i is either in N or a new variable, say Ca_i, When A_i∈N, $A_i \underset{G_1}{\overset{*}{\Rightarrow}} w_i$ is a derivation in atmost k steps, and so by induction hypotheses, $A_i \underset{G_1}{\overset{*}{\Rightarrow}} w_i$. Thus (1) is true for all derivations. Therefore L(G) = L(G_1).

The effect of applying A→A_1A_2....Am in a derivation for w ∈ L(G_1) can be achieved by applying the production A→A_1C_1, C_1→A_2C_2, ...C_{m-2}→$A_{m-1}A_m$ in P_2. Hence it is easy to see that L(G_1)⊆L(G_2).

To prove $L(G_2) \subseteq L(G_1)$, we can prove an auxillary result.

$$A \underset{G_1}{\overset{*}{\Rightarrow}} w \text{ if } A \in N^1, A \underset{G_2}{\overset{*}{\Rightarrow}} w \quad (2)$$

Condition (2) can be proved by induction on the number of steps $A \underset{G_2}{\overset{*}{\Rightarrow}} w$. Applying (1) to S, we get $L(G_2) \subseteq L(G_1)$.

Thus $L(G) = L(G_1) = L(G_2)$

91. State and prove pumping lemma for context free languages (10)

Ans: **Theorem**

Let be an infinite context-free language. Then there exists some positive integar m such that any $w \in L$ with $|w| \geq m$ can be decomposed as

$w = uvxyz$ (1)

with

$|vxy| \leq m$ (2)

and

$|vy| \geq 1$ (3)

such that

$uv^i xy^i z \in L$ (4)

for all $i = 0, 1, \ldots\ldots$ This is known as pumping lemma for context free languages.

Proof

Consider the context free grammar G without unit productions (or)λ-productions. L−{λ} is the language which is generated by G. The length of the string on the right hand side of any production is bounded. Since L is infinite, there exists arbitrarily long derivations and corresponding derivation trees of arbitrary height.

Consider a high derivation tree from root to leaf. Since the number of variables in G in finite, there must be some variables that repeats on this path.

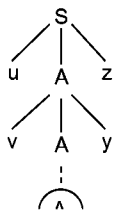
Consider the derivation

$S \Rightarrow^* uAz \Rightarrow^* uvAyz \Rightarrow^* uvxyz$

Where u, v, x, y, z are all strings of terminals.

$A \Rightarrow^* uAy$ and $A \Rightarrow^* x$.

If this derivations are repeated, we can generate all strings $uv^i xy^i z$, $i = 0, 1\ldots.$



We can assume that no variables repeats (repeating variable-A). The length of the strings v, x, and y depends only on productions of the grammar and can be bounded independently of w (condition (2) holds). Since there is no λ-productions, v and y cannot be empty string (condition (3) holds).

92. Using pumping lemma prove that the language ($a^i b^i c^i \mid \geq 1$) is not context free.(6)

Ans: The given language L = $\{a^n b^n c^n\}$

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir]

Let z be any string that belongs to L

Let $z = a^P b^P c^P \in L$

According to pumping lemma, if z is in L and $|z|>n$, z can be written as

$z = uvwxy$

$z = a^P b^P c^P$ as

u, vwx and y respectively, we get

$u = a^P$

$vwx = b^P$ where $|vwx| \leq n$

$vx = b^{P-m}$ where $|vx| \geq 1$

$y = c^P$

Substituting these values in uv^1wx^1y

$= uv^{i-1}vwx\, x^{i-1}y$ (uv^iwx^iy is expressed in this form)

$= uvwx(vx)^{i-1}y$

$= a^P b^P (b^{P-m})^{i-1} c^P$

$= a^P b^P b^{P1-mi-P} + {}^m c^P \notin L$ for all values of i

Let $i = 0$.

$uv^{i-1}\, vwx\, x^{i-1}\, y = a^P b^P b^{P(0)-m(0)-P} + {}^{mi}c^P$

$= a^P b^P b^{m-P} c^P$

$= a^P b^m c^P\ L$

Hence L is not a context free grammar.

References

Davis M., Sigal R., Weyuker E.J. (1994). *Computability, Complexity, and Languages: Fundamentals of Theoretical Computer Science* (2nd Edn). Academic Press.

Eitan G (1989). *An Introduction to the Theory of Computation*. Computer Science Press.

Harrison M.A. (1978). *Introduction to Formal Language Theory*. Addison-Wesley.

Hein J.L. (1996). *Theory of Computation*. Sudbury, MA: Jones & Bartlett.

Hopcroft J.E., Rajeev M., Ullman J.D. (2000). *Introduction to Automata Theory, Languages, and Computation* (2nd Edn). Pearson Education.

Hopcroft J.E., Ullman J.D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd Edn). Reading, MA: Addison-Wesley.

Lewis F.D. (2007). *Essentials of Theoretical Computer Science*.

Schmeiser J.P., Barnard D.T. (1995). *Producing a Top-Down Parse Order with Bottom-Up Parsing*. North-Holland: Elsevier.

Sipser M. (1997). Part One: Automata and Languages, Chapters 1–2, pp.29–122. Section 4.1: Decidable Languages, pp.152–159. Section 5.1: Undecidable Problems from Language Theory, pp.172–183. In: *Introduction to the Theory of Computation*. PWS Publishing.

Sipser M. (2006). *Introduction to the Theory of Computation* (2 Edn). PWS Publishing.

Taylor R.G. (1998). *Models of Computation and Formal Languages*. New York: Oxford University Press.

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

This page is intentionally left blank

Index

A
Acceptance by empty stack, 204
Acceptance by final state, 204
Ackermann's function, 300
Alphabet, 12
Ambiguous, 156
Ambiguous grammar, 156, 214
Applications of CFG, 181
Applications of PDA, 223
Applications of regular expressions, 121
Arden's theorem, 111
Automata theory, 1

B
Binary function, 10
Binary relation, 7
Binary tree, 18

C
Cardinality, 3
Cartesian product, 6
Checking off symbols, 244
Chess, 314
Chomsky normal form, The, 166
Church's thesis, 253
Church–Turing thesis, 253
Classification of problems, 309
Clique problem, 320
Closure properties of CFLs, 179
Closure properties of DCFL, 212
Closure properties of regular sets, 120
Complement, 4
Complement of a relation, 7
Complementation, 212
Composition, 11
Computability, 281
Computability theory, 294
Computable function, 296
Computational complexity, 281
Computational complexity theory, 294
Computational machine, 239
Concatenation, 101, 212
Constructing CFG for the given PDA, 217
Context free grammars, 14, 149
Context Sensitive Language (CSL), 258
Conversion from NFA to DFA, 123
Counter machine, 256
Cryptography, 314
CSGs, 258

D
DCFL, 211, 212
Decidability, 281
Decidable problems, 284
Decidable, 281
Decision algorithms for CFLs, 174
Decision problem, 281
Decision procedure, 281
Decision properties of DCFLs, 214
Decision properties of regular languages, 123
Derivation tree, 155
Deterministic context free language, 211
Deterministic Finite Automata (DFA), 37
Deterministic PDA, 211
Diagonalization method, 286
Difference, 4
Directed graph, 16
Domain, 7

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

DPDA, 212, 231
DTIME, 309

E
ε-closure, 47
Elimination of ε-productions, 163
Emptiness membership and equivalence, 124
Emptiness of context free grammar G, 285
Empty set, 2
Empty stack, 214
Empty word, 12
Enumerators, 254
Equivalence of acceptance of final state and empty stack, 210
Equivalence of counter machines are turing machine, 256
Equivalence of PDA's and CFG's, 216
Equivalent relation, 8

F
Fibonacci number sequence, 297
Fibonacci sequence, 297
Fifteen puzzle problem, 317
Final state, 214
Finding patterns, 122
Finite automata, 1
Finite automaton, 31
Finite control, The, 32
Finiteness and infiniteness, 175
Finite-state system, 31
Formal languages, 1
Full-adder, 65
Functions, 1

G
Gödel numbers, 282
Graph, 7, 16
Graphical representation of PDA, 202
Greibach normal form, The, 170

H
Halting problem, 285, 315
Hamiltonian cycle, 321

I
Identity, 3
Indistinguishable states, 53
Infinite tape, 252
Inherent ambiguity, 160
Input alphabet, 32
Instantaneous description, 238
Instantaneous description of PDA, 203
Integer division, 271
Into function, 11
Inverse of a relation, 8
Irreflexive, 8
Iteration (Closure), 102

K
Kleen's closure, 12, 212
Knapsack problem, 317
K-tuple, 6

L
Language accepter, 239
Language defined by grammars, 153
Lbas, 258
Leftmost and rightmost derivations, 154
Lexical analyser, 33
Lexical analysis, 121
Linear bound automata, 1
Linear bound automata and context sensitive language, 258
Linguist Noam Chomsky, 297

M
Many–one reducibility, 296
Many-to-one function, 10
Mapping reducibility, 296
Mealy machine, 60
Membership, 176
Minimum automaton, 54
Modified version of PCP, 290
Moore machine, 60
MPCP, 290
Multiple heads, 252
Multiple tapes, 251
Multiplication of two integers, 270
Multi-track tape, 243
Myhill Nerode theorem, 58

N
N-ary function, 10
Non-deterministic Finite Automata (NFA), 37
Non-deterministic turing machines, 249
NP problems, 309
NPC, 310
NP-complete, 309, 310
NPDA, 231
NP-hard, 309, 312
Null set, 1

O
One-dimensional tape, 251
One-to-one function, 10
Onto B, 10
Onto function, 10
Optimal length, 289
Optimal solution, 289

P
P problems, 309
Partial functions, 298
Partially decidable, 282
PDA as a parser, 223
Person's ancestors, 297
Post's correspondence problem, 288
Power sets, 3
Pre-ordering, 295
Prime or not, 243
Proof by contradiction, 19, 20
Proof by example, 19
Proof by induction, 19
Provable, 282
Pseudo-polynomial time, 319
PSPACE, 314
PTIME, 309
Pumping lemma, 114
Pumping lemma for CFL, 173
Pushdown automata, 1, 201

R
RAM, 253
Range, 7
Recursion, 296
Recursion theorem, 296
Recursion theory, 281
Recursive and recursively enumerable languages, 257
Recursive functions, 298
Reducibility, 294
Reflexive, 8
Registers, 256
Regular expression, 101
Regular grammar, 115
Regular grammars, 14
Regular sets, 101
Relations, 1
Removing ambiguity, 159
Rice's theorem, 298

S
Sequence, 6
Set, 1
Shifting over, 248
Solvable, 282
State register, 256
Storage in finite control, 241
String sequence detector, 67
Subroutines, 246
Subset, 3
Surjection, 10
Symbol, 11
Symmetric, 8

T
Theorem proving, 18
Top-own parser using the PDA, 224
Tractable problems, 310
Transition diagram, 34, 239
Transition table, 34
Travelling salesman problem, 321
Trees, 17
Turing computability, 237
Turing machine, 1, 237
Two's complement, 272
Two-dimensional tapes, 250
Two-stack PDA, 222
Two-way infinite tapes, 252
Type 0 grammars–unrestricted grammars, 14
Type 1 grammars, 14

Telegram Channel: @konkurcomputer [www.konkurcomputer.ir] کنکور کامپیوتر konkurcomputer.ir

Type 2 grammars, 14
Type 3 grammars, 14
Types of turing machines, 249

U
Unary function, 10
Undecidability of post's correspondence problem, 290
Undecidable, 282
Undecidable problems, 288, 309
Undirected graph, 17
Union, 101, 212
Union, intersection, 3
Universal turing machine, 254
Universe of discourse, 4
Useless symbols, 161